



Red Hat System Administration I

Red Hat Enterprise Linux 10.0 RH124

Student Workbook
Edition 3

Table of Contents

Red Hat System Administration I

Document Conventions

Admonitions

Use of Artificial Intelligence

Inclusive Language

Introduction

Red Hat System Administration I

Orientation to the Classroom Environment

Performing Lab Exercises

Chapter 1. Introduction to Red Hat Enterprise Linux

1.1. Getting Started with Red Hat Enterprise Linux

1.2. Quiz Getting Started with Red Hat Enterprise Linux

1.3. Summary

Chapter 2. Accessing the Command Line

2.1. Accessing the Command Line in a Text Interface

2.2. Quiz Accessing the Command Line in a Text Interface

2.3. Accessing the Command Line with the Desktop Environment

2.4. Guided Exercise Access the Command Line with the Desktop Environment

2.5. Executing Commands with the Bash Shell

2.6. Quiz Executing Commands with the Bash Shell

2.7. Lab Access the Command Line

2.8. Summary

Chapter 3. Getting Help from Local Documentation

3.1. Getting Help from Manual Pages

3.2. Guided Exercise Get Help from Manual Pages

3.3. Lab Get Help from Local Documentation

3.4. Summary

Chapter 4. Registering Systems for Red Hat Support

4.1. Registering Systems for Red Hat Support

4.2. Quiz Registering Systems for Red Hat Support

4.3. Summary

Chapter 5. Getting AI-assisted Help with Red Hat Enterprise Linux Lightspeed

5.1. Asking Questions and Evaluating Answers from the Command-line Assistant

5.2. Guided Exercise Ask Questions and Evaluate Answers from the Command-line Assistant

5.3. Summary

Chapter 6. Navigating the File-system Hierarchy

6.1. The Linux File-system Hierarchy

6.2. Quiz The Linux File-system Hierarchy

6.3. Specifying Files by Name

6.4. Quiz Specifying Files by Name

6.5. Summary

Chapter 7. Managing Files from the Command Line

7.1. Managing Files with Command-line Tools

7.2. Guided Exercise Manage Files with Command-line Tools

7.3. Creating Links Between Files

7.4. Guided Exercise Create Links Between Files

7.5. Matching File Names with Shell Expansions

7.6. Quiz Match File Names with Shell Expansions

7.7. Lab Manage Files from the Command Line

7.8. Summary

Chapter 8. Editing Text Files

8.1. Editing Text Files from the Command Line

8.2. Guided Exercise Edit Text Files from the Command Line

8.3. Summary

Chapter 9. Redirecting Shell Input and Output

- 9.1. Redirecting Output to Files and Programs
- 9.2. Guided Exercise Redirect Output to Files and Programs
- 9.3. Quiz Redirecting Shell Input and Output
- 9.4. Summary

Chapter 10. Managing Local Users and Groups

- 10.1. Linux Users and Groups
- 10.2. Quiz Linux Users and Groups
- 10.3. Gaining Superuser Access
- 10.4. Guided Exercise Gain Superuser Access
- 10.5. Managing Local User Accounts
- 10.6. Guided Exercise Manage Local User Accounts
- 10.7. Managing Local Group Accounts
- 10.8. Guided Exercise Manage Local Group Accounts
- 10.9. Managing User Passwords
- 10.10. Guided Exercise Manage User Passwords
- 10.11. Lab Manage Local Users and Groups
- 10.12. Summary

Chapter 11. Controlling Access to Files

- 11.1. Interpreting Linux File Permissions
- 11.2. Quiz Interpreting Linux File Permissions
- 11.3. Managing File Permissions from the Command Line
- 11.4. Guided Exercise Manage File Permissions from the Command Line
- 11.5. Managing Default Permissions and Special Permissions
- 11.6. Guided Exercise Manage Default Permissions and Special Permissions
- 11.7. Lab Control Access to Files
- 11.8. Summary

Chapter 12. Installing and Updating Software with RPM

- 12.1. Investigating RPM Software Packages

- 12.2. Guided Exercise Investigate an RPM Software Package
- 12.3. Installing and Updating Packages with DNF
- 12.4. Guided Exercise Install and Update Packages with DNF
- 12.5. Enabling DNF Software Repositories
- 12.6. Guided Exercise Enable a DNF Software Repository
- 12.7. Lab Install and Update Software with RPM
- 12.8. Summary

Chapter 13. Installing and Updating Applications by Using Flatpak

- 13.1. Configuring Flatpak for Application Installation
- 13.2. Guided Exercise Configure Flatpak for Application Installation
- 13.3. Managing Applications from Flatpak
- 13.4. Guided Exercise Manage Applications from Flatpak
- 13.5. Quiz Install and Update Applications by Using Flatpak
- 13.6. Summary

Chapter 14. Accessing Removable Media

- 14.1. Identifying File Systems and Block Devices
- 14.2. Quiz Identifying File Systems and Block Devices
- 14.3. Mounting and Unmounting File Systems
- 14.4. Guided Exercise Mount and Unmount a File System
- 14.5. Locating Files on a File System
- 14.6. Guided Exercise Locate Files on a File System
- 14.7. Lab Access Removable Media
- 14.8. Summary

Chapter 15. Monitoring and Managing Linux Processes

- 15.1. Processes and the Process Lifecycle
- 15.2. Quiz Processes and the Process Lifecycle
- 15.3. Managing Processes with Job Control
- 15.4. Guided Exercise Manage Processes with Job Control
- 15.5. Sending Signals to Processes

- 15.6. Guided Exercise Send Signals to Processes
- 15.7. Monitoring Process Activity
- 15.8. Guided Exercise Monitor Process Activity
- 15.9. Lab Monitor and Manage Linux Processes
- 15.10. Summary

Chapter 16. Controlling Services and Daemons

- 16.1. Identifying System Processes that Start Automatically
- 16.2. Guided Exercise Identify System Processes that Start Automatically
- 16.3. Controlling System Services
- 16.4. Guided Exercise Control System Services
- 16.5. Lab Control Services and Daemons
- 16.6. Summary

Chapter 17. Introduction to Networking

- 17.1. Networking Concepts
- 17.2. Quiz Networking Concepts
- 17.3. Validating Network Configuration
- 17.4. Guided Exercise Validate Network Configuration
- 17.5. Summary

Chapter 18. Managing Network Configuration

- 18.1. Configuring Networking from the Command Line
- 18.2. Guided Exercise Configure Networking from the Command Line
- 18.3. Editing Network Configuration Files
- 18.4. Guided Exercise Edit Network Configuration Files
- 18.5. Configuring Hostnames and Name Resolution
- 18.6. Guided Exercise Configure Hostnames and Name Resolution
- 18.7. Lab Manage Network Configuration
- 18.8. Summary

Chapter 19. Configuring and Securing SSH

- 19.1. Managing SSH Host Keys

- 19.2. Guided Exercise Manage SSH Host Keys
- 19.3. Configuring SSH Key-based Authentication
- 19.4. Guided Exercise Configure SSH Key-based Authentication
- 19.5. Lab Configure and Secure SSH
- 19.6. Summary

Chapter 20. Comprehensive Review

- 20.1. Summary of Learning Objectives
- 20.2. Lab Manage Files from the Command Line
- 20.3. Lab Manage Users and Groups, Permissions, and Processes
- 20.4. Lab Configure and Manage a Server
- 20.5. Lab Manage Networks
- 20.6. Lab Mount File Systems and Find Files
- 20.7. Summary

Red Hat System Administration I

Red Hat Enterprise Linux 10.0 RH124

Edition 3

Publication date 2025-08-21

Authors: Alex Callejas, Ashley D'Andrea, Hernán Del Negro, Shreya Dhange, Austin Garrigus, Patrick Gomez, Ashish Lingayat, Zoltan Molnar, Mauricio Santacruz Delgado

Course Architects: Steven Bonneville, Patrick Gomez, Ashish Lingayat

DevOps Engineers: Trey Feagle, Artur Glogowski, Nikki Lucas, Travis Michette, Jordi Sola

Editors: Julian Cable, Rachel Lee

Contributors: Sajith Eyamkuzhy Sugathan, Dusty Powell, Alejandra Ramírez Palacios, David Sacco, Jonika Shank, Shashi Singh, Roberto Velazquez

© 2025 Red Hat, Inc.

The contents of this course and all its modules and related materials, including handouts to audience members, are © 2025 Red Hat, Inc.

No part of this publication may be stored in a retrieval system, transmitted or reproduced in any way, including, but not limited to, photocopy, photograph, magnetic, electronic or other record, without the prior written permission of Red Hat, Inc.

This instructional program, including all material provided herein, is supplied without any guarantees from Red Hat, Inc. Red Hat, Inc. assumes no liability for damages or legal action arising from the use or misuse of contents or details contained herein.

If you believe Red Hat training materials are being used, copied, or otherwise improperly distributed, please send email to training@redhat.com or phone toll-free (USA) +1 (866) 626-2994 or +1 (919) 754-3700.

Red Hat, Red Hat Enterprise Linux, the Red Hat logo, JBoss, OpenShift, Fedora, Hibernate, Ansible, RHCA, RHCE, RHCSA, Ceph, and Gluster are trademarks or registered trademarks of Red Hat, Inc. or its subsidiaries in the United States and other countries.

Linux® is the registered trademark of Linus Torvalds in the United States and other countries.

Java® is a registered trademark of Oracle American, Inc. and/or its affiliates.

XFS® is a registered trademark of Hewlett Packard Enterprise Development LP or its subsidiaries in the United States and/or other countries.

MySQL® is a registered trademark of MySQL AB in the United States, the European Union and other countries.

Node.js® is a trademark of Joyent. Red Hat is not formally related to or endorsed by the official Joyent Node.js open source or commercial project.

The OpenStack word mark and the Square O Design, together or apart, are trademarks or registered trademarks of OpenStack Foundation, in the United States and other countries and are used with the OpenStack Foundation's permission. Red Hat, Inc. is not affiliated with, endorsed by, or sponsored by the OpenStack Foundation or the OpenStack community.

All other trademarks are the property of their respective owners.

Document Conventions

This section describes various conventions and practices that are used throughout all Red Hat Training courses.

Admonitions

Red Hat Training courses use the following admonitions:

References

References describe where to find external documentation that is relevant to a subject.

Note

Notes are tips, shortcuts, or alternative approaches to the task at hand. Ignoring a note should have no negative consequences, but you might miss out on something that makes your life easier.

Important

Important sections provide details of information that is easily missed: configuration changes that apply only to the current session, or services that need restarting before an update applies. Ignoring these admonitions will not cause data loss, but might cause irritation and frustration.

Warning

Do not ignore warnings. Ignoring these admonitions will most likely cause data loss.

Use of Artificial Intelligence

This course contains content or media that is generated or assisted by AI tools. All Red Hat Training deliverables have primarily human authorship, and all content was reviewed by a human domain expert.

Inclusive Language

Document Conventions

Red Hat Training is currently reviewing its use of language in various areas to help remove any potentially offensive terms. This process is ongoing and requires alignment with the products and services that are covered in Red Hat Training courses. Red Hat appreciates your patience during this process.

Introduction

Red Hat System Administration I

Red Hat System Administration I (RH124) is designed for IT professionals without previous Linux system administration experience. The course provides students with Linux administration "survival skills" by focusing on core administration tasks. *Red Hat System Administration I* also provides a foundation for students who plan to become full-time Linux system administrators, by introducing key command-line concepts and enterprise-level tools. These concepts are further developed in the follow-on course, *Red Hat System Administration II* (RH134).

Course Objectives

- Operate, manage, and maintain an existing Red Hat Enterprise Linux system from the command line, and perform a core set of foundational system administration tasks.

Audience

- System administrators, platform engineers, developers, and other IT professionals who have limited Red Hat Enterprise Linux experience but need to perform essential Linux administration tasks on a single server, including basic competency with use of the Bash command-line shell, user and file management, software administration, management of local storage devices, and other key single-server duties

Prerequisites

- Basic familiarity with computer systems as a user. Previous experience as a Linux user or with system administration of other operating systems might be beneficial.

Orientation to the Classroom Environment

In this course, the main computer system that is used for hands-on learning activities is the workstation machine.

Introduction

You also use three other machines for these activities: the servera, serverb, and serverc machines.

These machines are in the lab.example.com DNS domain.

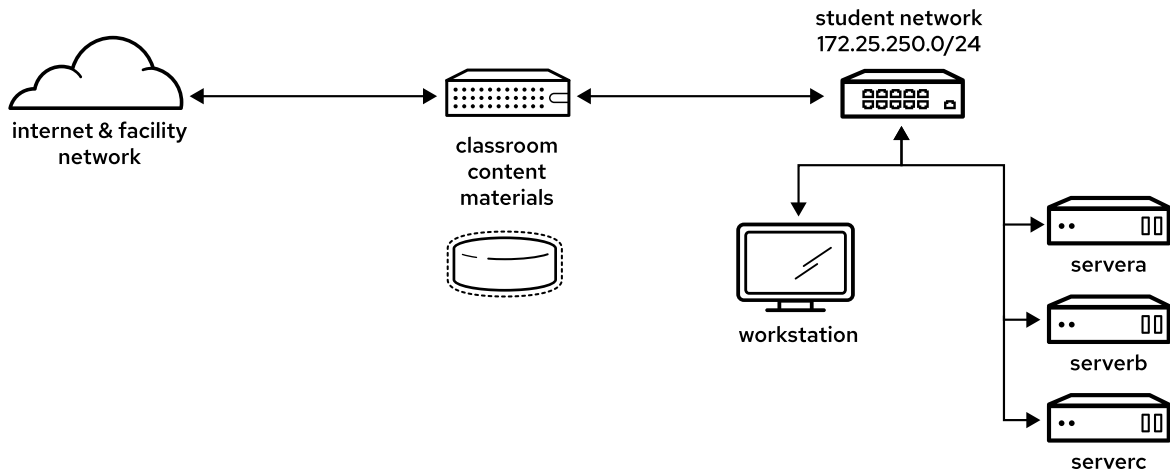


Figure 1. Classroom environment

Table 1. Classroom Machines

Machine name	IP addresses	Role
classroom.example.com content.example.com materials.example.com	172.25.252.254 172.25.254.254	Server that hosts the required classroom materials
utility.lab.example.com	172.25.250.8	Server that hosts course-specific support services
workstation.lab.example.com	172.25.250.9	Graphical workstation that students use
servera.lab.example.com	172.25.250.10	Managed server "A"
serverb.lab.example.com	172.25.250.11	Managed server "B"

Introduction

Machine name	IP addresses	Role
serverc.lab.example.com	172.25.250.12	Managed server "C"

Controlling Your Systems

In an instructor-led training classroom, you are assigned a physical computer (foundationX.ilt.example.com), which accesses the virtual machines that run on that host. You are automatically logged in to the physical machine as the kiosk user, by using redhat as the password. The classroom systems that you work with are virtual machines that run on that host.

On the foundationX machine, you use the `rht-vmctl` and `rht-vmview` commands to work with the virtual machines. Run the commands in the following table as the kiosk user on the foundationX machine. You can use these commands with any of the virtual machines.

Table 2. `rht-vmctl` Commands

Action	Command
Start the virtual machine named <code>server</code> .	<code>rht-vmctl start server</code>
Display the system console for the virtual machine named <code>server</code> , to log in and work on that system.	<code>rht-vmview view server</code>
Reset the virtual machine named <code>server</code> to its initial course state and restart it.	<code>rht-vmctl reset server</code>

At the start of a lab exercise, if the "reset your servera" instruction appears, then run the `rht-vmctl reset servera` command on the foundationX machine as the kiosk user. Likewise, if the "reset your workstation" instruction appears, then run the `rht-vmctl reset workstation` command on the foundationX machine as the kiosk user.

Introduction

You are assigned remote computers in a Red Hat Online Learning (ROLE) classroom. Self-paced courses are accessed through a web application that is hosted at rol.redhat.com. Log in to this site with your Red Hat Customer Portal user credentials.

Controlling the Virtual Machines

The virtual machines in your classroom environment are controlled through web page interface controls. The state of each classroom virtual machine is displayed on the **Lab Environment** tab.

Course

Lab Environment

SSH Private Key & Instructions

DOWNLOAD SSH KEY

Classroom Webapp

ACCESS CLASSROOM WEBAPP

Lab Controls

WATCH TUTORIAL

DELETE

STOP

i

classroom	Active	ACTION ▾	OPEN CONSOLE
servera	Active	ACTION ▾	OPEN CONSOLE
serverb	Active	ACTION ▾	OPEN CONSOLE
serverc	Active	ACTION ▾	OPEN CONSOLE
utility	Active	ACTION ▾	OPEN CONSOLE
workstation	Active	ACTION ▾	OPEN CONSOLE

Figure 2. An example course lab environment management page

Table 3. Machine States

Virtual machine state	Description
building	The virtual machine is being created.

Introduction

Virtual machine state	Description
active	The virtual machine is running and available. If the virtual machine just started, then it might still be starting services.
stopped	The virtual machine is shut down. On starting, the virtual machine boots into the same state that it was in before shutdown. The disk state is preserved.

Table 4. Classroom Actions

Button or action	Description
CREATE	Create the ROLE classroom. Creates and starts all the necessary virtual machines for this classroom.
CREATING	The ROLE classroom virtual machines are being created. Creation can take several minutes to complete.
DELETE	Delete the ROLE classroom. Deletes all virtual machines in the classroom. All saved work on those systems' disks is lost.
START	Start all virtual machines in the classroom.
STARTING	All virtual machines in the classroom are starting.
STOP	Stop all virtual machines in the classroom.

Introduction

Table 5. Machine Actions

Button or action	Description
OPEN CONSOLE	Connect to the system console of the virtual machine in a new browser tab. You can log in directly to the virtual machine and run commands, when required. Normally, log in to the workstation virtual machine only, and from there, use ssh to connect to the other virtual machines.
ACTION > Start	Start (power on) the virtual machine.
ACTION > Shutdown	Gracefully shut down the virtual machine, and preserve disk contents.
ACTION > Power Off	Forcefully shut down the virtual machine, and still preserve disk contents. This action is equivalent to removing the power from a physical machine.
ACTION > Reset	Forcefully shut down the virtual machine and reset the associated storage to its initial state. All saved work on that system's disks is lost.

At the start of an exercise, if you are instructed to reset a single virtual machine node, then click **ACTION > Reset** for that specific virtual machine only.

At the start of an exercise, if you are instructed to reset all virtual machines, then click **ACTION > Reset** on every virtual machine in the list.

To return the classroom environment to its original state at the start of the course, you can click **DELETE** to remove the entire classroom environment. After the lab is deleted, click **CREATE** to provision a new set of classroom systems.

 **Warning**

The **DELETE** operation cannot be undone. All completed work in the classroom environment is lost.

The Auto-stop and Auto-destroy Timers

The Red Hat Online Learning enrollment entitles you to a set allotment of computer time. To help to conserve your allotted time, the ROLE classroom uses timers, which shut down or delete the classroom environment when the appropriate timer expires.

To adjust the timers, locate the two  buttons at the bottom of the course management page. Click the auto-stop  button to add another hour to the auto-stop timer. Click the auto-destroy  button to add another day to the auto-destroy timer. Auto-stop has a maximum of 11 hours, and auto-destroy has a maximum of 14 days. Be careful to keep the timers set when you are working, so that your environment is not unexpectedly shut down. Be careful not to set the timers unnecessarily high, which could waste your subscription time allotment.

Performing Lab Exercises

You might see the following lab activity types in this course:

- A *guided exercise* is a hands-on practice exercise that follows a presentation section. It walks you through a procedure to perform, step by step.
- A *quiz* is typically used when checking knowledge-based learning, or when a hands-on activity is impractical for some other reason.
- An *end-of-chapter lab* is a gradable hands-on activity to help you to check your learning. You work through a set of high-level steps, based on the guided exercises in that chapter, but the steps do not walk you through every command. A solution is provided with a step-by-step walk-through.

To prepare your lab environment at the start of each hands-on activity, run the `lab start` command with a specified activity name from the activity's instructions. Likewise, at the end of each hands-on activity, run the `lab finish` command with that same activity name to clean up after the activity. Each hands-on activity has a unique name within a course.

The syntax for running an exercise script is as follows:

```
student@workstation:~$ lab action exercise
```

The *action* is a choice of `install`, `select`, `list`, `start`, `grade`, or `finish`. All exercises support `start` and `finish`. Only end-of-chapter labs and comprehensive review labs support the `grade` action.

Introduction

install

The `install` action installs the exercise files for the specified course or lesson.

select

The `select` action helps to select a course or a lesson from multiple installed packages.

list

This action lists all the currently installed packages in your lab environment.

start

The `start` action verifies the required resources to begin an exercise. It might include configuring settings, creating resources, checking prerequisite services, and verifying necessary outcomes from previous exercises. Usually, you can perform an exercise at any time, even without performing preceding exercises. Occasionally, a previous activity might be required.

grade

For gradable activities, the `grade` action directs the `lab` command to evaluate your work, and shows a list of grading criteria with the PASS or FAIL status for each one. To achieve the PASS status for all criteria, fix the failures and rerun the `grade` action.

finish

The `finish` action cleans up resources that were configured during the exercise. You can perform an exercise as many times as you want.

The `lab` command supports tab completion. For example, to list all exercises that you can start, enter `lab start` and then press the `Tab` key twice. The `lab` command also includes suggestions to avoid potential issues when running the `start` and `finish` actions.

Chapter 1.

Introduction to Red Hat Enterprise Linux

Goal

Define and explain the following concepts: open source, Linux, Linux distributions, and Red Hat Enterprise Linux.

Sections

- Getting Started with Red Hat Enterprise Linux (and Quiz)

1.1. Getting Started with Red Hat Enterprise Linux

Objectives

- Define and explain the purpose of Linux, open source, Linux distributions, and Red Hat Enterprise Linux.

Why Should You Learn about Linux?

Linux is a critical technology for IT professionals to understand.

Linux is in widespread use worldwide. Internet users interact with Linux applications and web server systems daily, by browsing the World Wide Web and using e-commerce sites to buy and sell products.

Linux is in use for much more than the internet. Linux manages point-of-sale systems and the world's stock markets, powers smart TVs and in-flight entertainment systems, and runs most of the top 500 supercomputers in the world. Linux provides the core technologies that power the cloud revolution and the tools to build the latest generations of container-based microservices applications, software-based storage technologies, and big data solutions.

In the modern data center, Linux and Microsoft Windows are the predominant operating systems. Linux use continues to expand in enterprise, cloud, and device spaces. Due to its widespread adoption, you have many reasons to learn Linux:

- A Windows user needs to interoperate with Linux systems and applications.
- In application development, Linux commonly hosts the application and its runtime.
- In cloud computing, both private and public cloud instances use Linux as the operating system.
- Mobile applications and Internet of Things (IoT) devices commonly run on Linux.
- When looking for new IT career opportunities, Linux skills are in high demand.

What Makes Linux Great?

If someone asks you "What makes Linux great?", then you have many answers to pick from:

Linux is *open source* software.

Being open source means that you can see how a program or system works. You can also experiment with changes and share them freely for others to use. The open source model means that improvements are easier to make, enabling faster innovation.

Linux provides a *command-line interface* (CLI) for easy access and powerful scripting.

Linux is built around a basic design philosophy that users can perform all administration tasks from the CLI. Using the CLI enables easier automation, deployment, and provisioning, and simplifies both local and remote system administration. Unlike many other operating systems, these capabilities were in the architecture from the start, and result in ease of use and stability.

Linux is a *modular* operating system that is designed to easily replace or remove components.

System components can be upgraded and updated when needed. A Linux system can be a general-purpose development workstation or a purposefully minimized software appliance.

What Is Open Source Software?

Open source software is software with *source code* that anyone can use, study, modify, and share.

Source code is the set of human-readable instructions that are used to make a program. Code might be in interpretive form, such as a script, or compiled into a binary executable that the computer runs directly. Source code becomes copyrighted when created, and the copyright holder controls the terms under which the software can be copied, adapted, and distributed. Users can use the software according to its software license.

Some software uses *closed source* or *proprietary* source code that only the originating person, team, or organization can see, change, or distribute. Proprietary licenses typically restrict the user to running the program, and provide limited or no access to the source.

Open source software is different. When a copyright holder provides software under an open source license, they grant the user the right to run the program and to view, modify, compile, and redistribute the source to others, royalty-free. Open source licensing promotes collaboration, sharing, transparency, and rapid innovation, because it encourages more people to modify and improve the software and to share enhancements more widely.

Open source software can still be provided for commercial use. Open source is a critical part of many organizations' commercial operations. Some open source licenses allow code to be reused in proprietary products. Anyone can sell open source code, but open source licensing generally allows the customer to redistribute the source code. Open source vendors such as Red Hat provide commercial support for deploying, managing, and building solutions that are based on open source products.

Open source has many benefits for the user:

- *Control*: See what the code does and improve it.

- *Training*: Learn from real-world code and develop more applications that are useful.
- *Security*: Inspect sensitive code, and fix it even without the original developers' help.
- *Stability*: Rely on code that can survive the loss of the original developer.

Types of Open Source Licenses

The developers of open source software can license their software in different ways. The software license terms control how the source can be reused or combined with other code. To be open source, licenses must allow users to freely use, view, change, compile, and distribute the code.

Two general classes of open source license are particularly important:

- *Copyleft* licenses are designed to encourage keeping the code open source.
- *Permissive* licenses are designed to maximize code reusability.

Copyleft, or *share-alike* licenses, require that anyone who distributes the source code, with or without changes, must pass along the freedom for others to also copy, change, and distribute the code. The advantage of copyleft licenses is that they help to keep existing code, and improvements to that code, open and increase the amount of available open source code. Common copyleft licenses include the GNU General Public License (GPL) and the Lesser GNU Public License (LGPL).

Permissive licenses maximize the reusability of source code. You can use the source for any purpose if the copyright and license statements are preserved, including reusing code under more restrictive or proprietary licenses. Although permissive licensing makes it easy to reuse code, it risks encouraging proprietary-only enhancements. Examples of permissive licenses include the MIT/X11 license, the Simplified BSD license, and the Apache Software License 2.0.

Who Develops Open Source Software?

Open source development today is overwhelmingly professional. Open source is no longer solely developed by volunteers. Today, most open source developers work for organizations that pay them to participate with open source projects to construct and contribute the enhancements that the organization and their customers need.

Volunteers and the academic community still play a significant role and make vital contributions, especially in emerging technology. The combination of formal and informal development creates a highly dynamic and productive environment.

What Is a Linux Distribution?

A *Linux distribution* is an installable operating system that is constructed from a Linux kernel and that supports user programs and libraries. A complete *Linux system* is developed by multiple independent development communities that work cooperatively on individual components. A distribution provides an easy method to install and manage a working Linux system.

In 1991, graduate student Linus Torvalds developed a UNIX-like kernel that he named Linux, and licensed it as open source software under the GPL. The kernel is the core of the operating system and manages hardware, memory, and the scheduling of running programs. The Linux kernel is supplemented with other open source software, including utilities and programs from the GNU Project, and the Wayland graphical platform. The Linux kernel also includes other open source components, such as the Sendmail mail server and the Apache HTTP Server, to become a complete open source UNIX-like operating system.

A major challenge for Linux users is to assemble all these software pieces from many sources. Early Linux developers provided a distribution of prebuilt and tested tools that users could download and install to quickly implement Linux systems.

Many Linux distributions exist, each with differing goals and support criteria. Generally, distributions have some common characteristics:

- Distributions consist of a Linux kernel and support user-space programs.
- Distributions can be small and single-purpose, or can include thousands of open source programs.
- Distributions provide a means to install and update the software and its components.
- The distribution provider supports the software, and ideally, participates in the development community.

Who Is Red Hat?

Red Hat is the world's leading provider of open source software solutions, by using a community-powered approach to reliable and high-performance cloud, Linux, middleware, storage, and virtualization technologies. The mission of Red Hat is to be the catalyst in communities of customers, contributors, and partners to create better technology the open source way.

The role of Red Hat is to help customers to connect with the open source community and their partners to effectively use open source software solutions. Red Hat actively participates in and supports the open source community. Many years of experience have convinced the company of the importance of open source to the future of the IT industry.

Red Hat is best known for its participation in the Linux community and the Red Hat Enterprise Linux distribution. Red Hat is also active in other open source communities, including middleware projects that are centered on the JBoss developer community. Red Hat also provides virtualization solutions, cloud technologies such as OpenStack and OpenShift, and the Ceph software-based storage project, plus others.

Red Hat Enterprise Linux Ecosystem

Red Hat Enterprise Linux (RHEL) is Red Hat's commercial production-grade Linux distribution. Red Hat develops and integrates open source software into RHEL through a multistage process.

- Red Hat *participates* in supporting individual open source projects. It contributes code, developer time, resources, and support, and often collaborates with developers from other Linux distributions, to improve the general quality of software for everyone.
- Red Hat *sponsors* and *integrates* open source projects into the community-driven Fedora distribution. Fedora provides a free working environment to serve as a development lab and proving ground for features to be incorporated into CentOS Stream and RHEL products.
- Red Hat *stabilizes* the CentOS Stream software to be ready for long-term support and standardization, and integrates it into RHEL, the production-ready distribution.

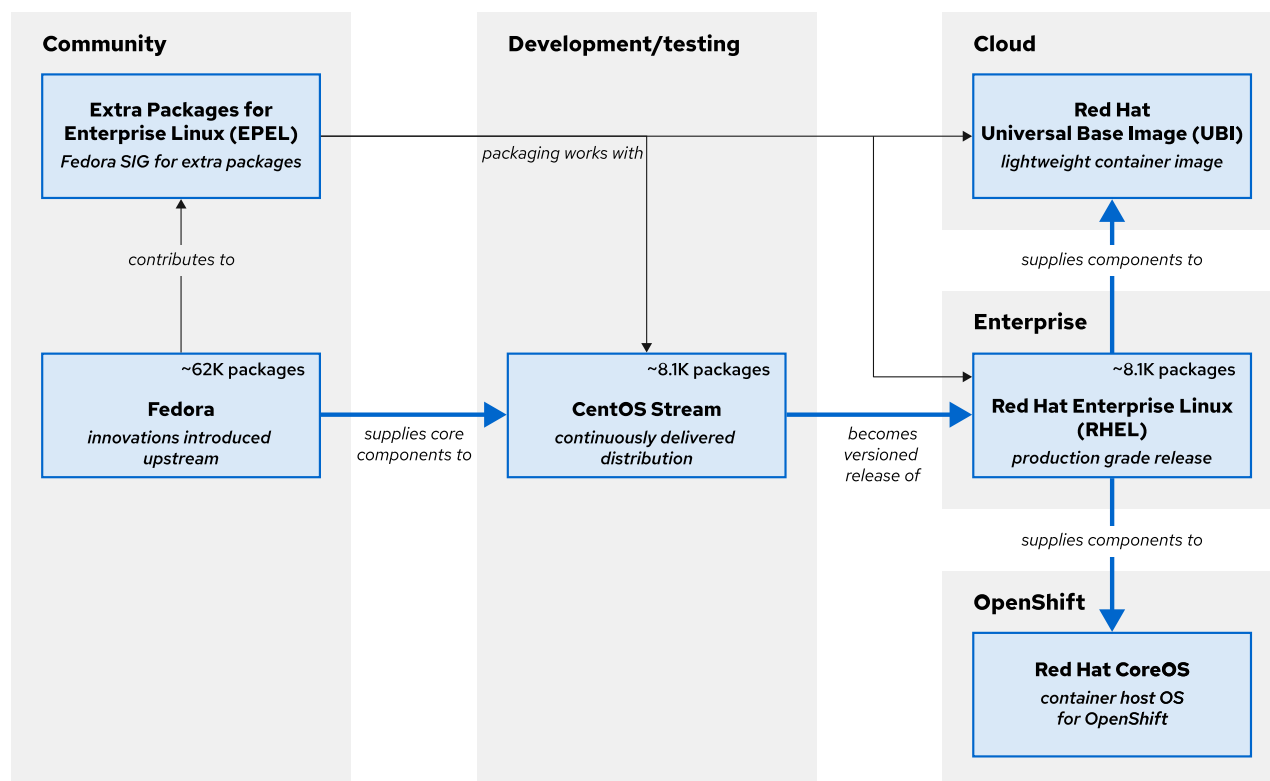


Figure 3. The Red Hat Enterprise Linux ecosystem

Fedora

Fedora is a community project that produces and releases a free and comprehensive Linux-based operating system. Red Hat sponsors and works with the Fedora community to integrate the latest upstream software into a fast-moving, secure distribution. The Fedora project contributes back to the open source world, and anyone can participate.

Fedora prioritizes innovation and excellence above long-term stability. Major updates occur every six months, and bring significant changes. Fedora supports releases for about a year, which means the latest two updates, making it less suited for supportable production use. Fedora remains the source of innovation for the entire Enterprise Linux ecosystem. In general, packages start out in Fedora and are included into CentOS Stream only when they are considered mature in terms of stability, security, performance, and customer demand.

Extra Packages for Enterprise Linux

A Fedora project Special Interest Group (SIG) builds and maintains a community-supported package repository called Extra Packages for Enterprise Linux (EPEL). EPEL versions align with major RHEL releases, and enable RHEL customers to run workloads with software dependencies that are not supported in RHEL. EPEL packages are not included in Red Hat support, but are equivalent to Fedora's level of quality.

Typically, EPEL packages are built against RHEL releases. EPEL Next is an additional repository for package maintainers to build against CentOS Stream. This repository is useful when CentOS Stream contains an upcoming RHEL library rebase, or if an EPEL package has a minimum version build requirement that is already in CentOS Stream but not yet in RHEL.

CentOS Stream

CentOS Stream is the upstream project for RHEL. Development of the next RHEL version is transparent and open for community contributions that can directly influence the next release. Patches that are submitted to CentOS Stream are integrated quickly into RHEL, which enables significant changes during the current RHEL version lifecycle. CentOS Stream is a continuous integration and delivery distribution, with tested and stable nightly builds.

The CentOS project welcomes contributors worldwide, to give RHEL derivatives the opportunity to contribute to CentOS Stream for their own benefit. The CentOS project also aims to promote sustainable open source software that responds quickly to security exploits, emerging technologies, and changing customer requirements.

Note

Before 2019, CentOS Linux was a freely available and unsupported distribution that was community-built from Red Hat's source code after each major RHEL release. Although the CentOS community enjoyed having a freely available RHEL clone, this model had disadvantages. Commonly, developer contributions to CentOS Linux were not backported to Fedora or RHEL without considerable duplicate effort. Also, significant delays occurred between a RHEL release and its corresponding CentOS distribution build, with a similar delay for critical RHEL security, driver, and tuning fixes. Red Hat switched to the CentOS Stream model to address these issues.

A benefit of CentOS Stream is that, as the source for RHEL development, it is available in the same architectures as RHEL, including Intel/AMD x86_64, ARM64, IBM Power, and IBM Z.

Many innovative technology organizations have proven that CentOS Stream is a viable replacement for the original downstream CentOS Linux. CentOS Stream can be freely downloaded and installed for many use cases, including development and light production. For community users with use cases that are not suitable for a continuously delivered distribution with asynchronous patch releases, Red Hat provides free individual RHEL developer subscriptions for small-scale use, such as demos, prototyping, quality assurance, and limited production.

Red Hat Enterprise Linux

Red Hat Enterprise Linux (RHEL) is Red Hat's production-ready, commercially supported Linux distribution. In the computing industry, RHEL is acknowledged as the leading platform for open source computing. RHEL is extensively tested and has a worldwide ecosystem of support partners for hardware and software certifications, consulting services, training, and multiyear support and maintenance guarantees.

Red Hat builds RHEL major releases directly from the CentOS Stream continuous development project, which is sourced from Fedora. The CentOS Stream development model is open and available to all, for feedback and contribution, and the code is prepared to be the next major RHEL release.

RHEL uses a subscription-based support model, and does not charge license fees for open-source software. Red Hat subscriptions provide product support, maintenance, updates, security patches, and access to the Customer Portal Knowledgebase, utilities, and downloadable releases of Red Hat products.

The following table lists some key differences between Fedora, CentOS Stream, and RHEL.

	Fedora	CentOS Stream	RHEL
Expected lifecycle	12-18 months	5 years	10 years
Software vendor certified	No	Usually not	Yes
Documentation provided by	Community	Community	Red Hat
Expert support available	No	No	Yes
Product security team	No	No	Yes
Security certifications	No	No	Yes
No-cost options	Yes	Yes	Yes

	Fedora	CentOS Stream	RHEL
Management tools	No	No	Yes

RHEL-based Image Optimized for Edge

Red Hat provides a collection of products and services that are optimized for edge use cases, including a customized RHEL-based image.

RHEL provides the ability to create purpose-built operating system images with Red Hat Enterprise Linux image builder. With this mechanism, IT teams can build, deploy, and maintain these RHEL images in less time over the life of the system. Image-based deployments are optimized for various edge architectures, and are customizable for specific edge deployments.

The edge features in RHEL include secure management and scaling capabilities, including zero-touch provisioning, system health visibility, and quick security remediations from within a single interface.

Red Hat Enterprise Linux CoreOS

Red Hat Enterprise Linux CoreOS (RHCOS) is not a stand-alone operating system. CoreOS is built from RHEL components, and is then released, upgraded, and managed as part of the Red Hat OpenShift Container Platform (RHOCP) for cloud-native applications. RHCOS is fundamentally an image-based RHEL container host, which uses the Container Runtime Interface (CRI-O)-compliant container engine that is integrated in RHOCP. To learn more about Red Hat Enterprise Linux CoreOS, begin by becoming familiar with OpenShift and containers.

Red Hat Universal Base Image

A Red Hat Universal Base Image (UBI) is essentially a freely redistributable derivative of RHEL. UBI is designed to be a foundation for cloud-native and web application use cases that are developed in containers. All UBI content is a subset of RHEL, with packages sourced from secure RHEL channels, and UBI is supported similar to RHEL when run on Red Hat-supported platforms such as OpenShift and RHEL hosts.

With UBI, developers can focus their efforts on their application in the container image. UBI is a set of base images, and a set of application images (such as Python, Ruby, Node.js, httpd, or nginx). UBI also consists of a set of RPM repositories, from which you can update any UBI base image to include the package dependencies that your application requires.

Red Hat Enterprise Linux Continuous Development

In the Fedora upstream community, Fedora Rawhide is the continuous development environment for a regular cadence of public Fedora releases. The community tests and prepares new Linux kernel versions, device drivers, utilities, and applications for the next Fedora distribution. Major RHEL release development begins with selecting the latest Fedora release as the base for the current CentOS Stream continuous development distribution.

Before a package is formally introduced to CentOS Stream, it undergoes rigorous testing to meet the standards for packages to be included in RHEL. Updates that are posted to CentOS Stream are the same as the updates that are posted to the unreleased minor version of RHEL in development.

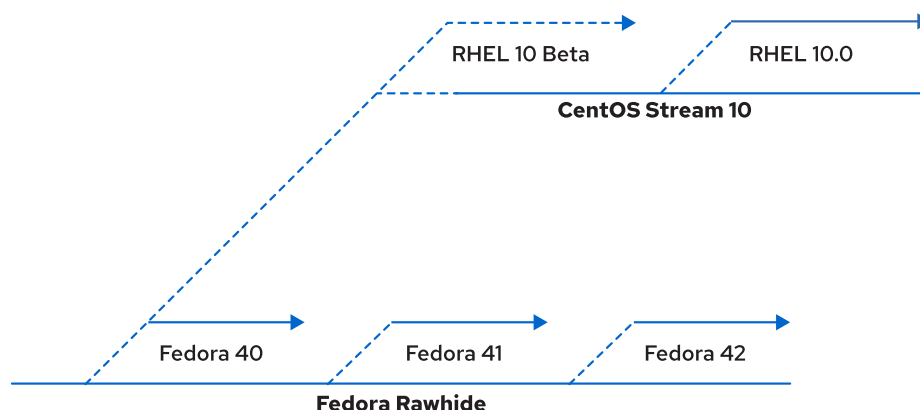


Figure 4. Red Hat Enterprise Linux continuous development

Fedora 40 was used as the original code base for RHEL 10 and for CentOS Stream 10. As packages are updated, they are then pushed into CentOS Stream and the nightly build of RHEL. The solid lines indicate distributions or builds that are available for public use.

Similar to the relationship between Fedora Rawhide and Fedora, CentOS Stream is the continuous development environment for preparing the next minor-version RHEL release. Red Hat performs extensive hardware, integration, dependency, and performance testing before releasing the next public RHEL distribution.

Obtaining Red Hat Enterprise Linux

Red Hat Enterprise Linux is typically obtained with a paid support subscription, and Red Hat provides multiple ways to obtain RHEL and other RHEL ecosystem products, many without cost.

- *Fedora Linux* and derivatives are freely available from the Fedora project at <https://fedoraproject.org/>, including an emerging version of Fedora CoreOS.
- *EPEL* and *EPEL Next* packages are freely available from the EPEL project repositories. Learn how to use EPEL at <https://docs.fedoraproject.org/en-US/epel/>.

- CentOS Stream is freely available at <https://www.centos.org/centos-stream/>.

RHEL Evaluation Download

An evaluation copy of RHEL is available at <https://www.redhat.com/en/technologies/linux-platforms/enterprise-linux/server/trial>. You must have a (free) Customer Portal account for <https://access.redhat.com> to access and download evaluation products. Product evaluations entitle you to receive updates and support for a limited period. Support ends when the evaluation period ends, but the evaluation software continues to operate. Additional information for many product evaluations is found on the Customer Portal evaluation pages.

Red Hat Developer Subscription

Red Hat provides a freely available subscription for many products through the Red Hat Developer program at <https://developer.redhat.com>. With a Red Hat Developer Subscription, developers can quickly create, prototype, test, and demonstrate their applications on the same Red Hat software as on production systems.

Create a personal account at <https://access.redhat.com>, and then register for the Red Hat Developer Subscription program. You can use an existing personal account, but do not use an account that is already associated with any organization's support subscription. The Red Hat Developer Subscription is self-supported, but provides ongoing product updates. Red Hat recommends that individuals who want to gain experience with RHEL and developer products should join the Red Hat Developer program.

Public Cloud Platforms

The major hyperscale public cloud providers, such as Amazon Web Services, Google Cloud Platform, and Microsoft Azure, offer official images for deploying Red Hat Enterprise Linux instances, with subscription management from the Red Hat Cloud Access service. Fully entitled subscriptions for RHEL and Red Hat products are available through the cloud provider, and are portable in hybrid and multivendor clouds.

Containers

You can use Red Hat Universal Base Images and associated content for development and deployment without a Red Hat subscription. For operational support and access to non-UBI tools, containers that are built on UBI must be deployed on a Red Hat-supported platform such as OpenShift or Red Hat Enterprise Linux. Access to non-UBI content requires a Red Hat subscription.

Image Mode for Red Hat Enterprise Linux

Image mode for Red Hat Enterprise Linux 10 is a new deployment and management approach that uses container-native tools to build, deploy, and manage the operating system. With this method, you can deploy the operating system from a bootable container (bootc) image.

You can add software or dependencies to build the image, and then push it to a registry. Image mode for RHEL limits infrastructure drift, enables scalability, and simplifies troubleshooting.

References

[The Open Source Way](#)

[The Fedora Project](#)

[CentOS Stream](#)

[Red Hat Enterprise Linux](#)

[No-cost Red Hat Enterprise Linux Individual Developer Subscription: FAQs](#)

[Red Hat Universal Base Images \(UBI\)](#)

[Red Hat Cloud Access](#)

1.2. Quiz

Getting Started with Red Hat Enterprise Linux

Choose the correct answers to the following questions:

1. Which two statements describe the benefits of open source software for the user? (Choose two.)
 - ☐ a. Code can survive the loss of the original developer or distributor.
 - ☐ b. Sensitive portions of code are protected and are available only to the original developer.
 - ☐ c. Users can learn from real-world code and develop more effective applications.
 - ☐ d. Code remains open provided that it is in a public repository, but the license might change when included with closed source software.
2. Which two statements describe how Red Hat develops products and interacts with the community? (Choose two.)
 - ☐ a. Sponsors and integrates open source projects into the community-driven Fedora project.
 - ☐ b. Develops specific integration tools that are available only in Red Hat distributions.
 - ☐ c. Participates in upstream projects.
 - ☐ d. Repackages and relicenses community products.
3. Which two statements describe the benefits of Linux? (Choose two.)
 - ☐ a. Linux is developed entirely by volunteers, which makes it a low-cost operating system.
 - ☐ b. Linux is modular and can be configured for a full graphical desktop or a small appliance.
 - ☐ c. Linux is locked in a known state for a minimum of one year for each release, so it is easier to develop custom software.
 - ☐ d. Linux includes a powerful and scriptable command-line interface, which enables automation and provisioning.

Solution

1. Which two statements describe the benefits of open source software for the user? (Choose two.)
 - ☒ **a. Code can survive the loss of the original developer or distributor.**
 - ☐ **b. Sensitive portions of code are protected and are available only to the original developer.**
 - ☒ **c. Users can learn from real-world code and develop more effective applications.**
 - ☐ **d. Code remains open provided that it is in a public repository, but the license might change when included with closed source software.**
2. Which two statements describe how Red Hat develops products and interacts with the community? (Choose two.)
 - ☒ **a. Sponsors and integrates open source projects into the community-driven Fedora project.**
 - ☐ **b. Develops specific integration tools that are available only in Red Hat distributions.**
 - ☒ **c. Participates in upstream projects.**
 - ☐ **d. Repackages and relicenses community products.**
3. Which two statements describe the benefits of Linux? (Choose two.)
 - ☐ **a. Linux is developed entirely by volunteers, which makes it a low-cost operating system.**
 - ☒ **b. Linux is modular and can be configured for a full graphical desktop or a small appliance.**
 - ☐ **c. Linux is locked in a known state for a minimum of one year for each release, so it is easier to develop custom software.**
 - ☒ **d. Linux includes a powerful and scriptable command-line interface, which enables automation and provisioning.**

1.3. Summary

In this lesson, you learned that open source software includes source code that anyone can freely use, study, modify, and share.

A Linux distribution is an installable operating system that is built from a Linux kernel and is designed to support user programs and libraries. Red Hat support and contributes code to open source projects. Red Hat sponsors and integrates project software into community-driven distributions and stabilizes the software to offer it as supported enterprise-ready products.

Red Hat Enterprise Linux (RHEL) is the open source, enterprise-ready, commercially supported Linux distribution that Red Hat provides. A free Red Hat Developer Subscription is a useful method for obtaining learning resources and information, including developer subscriptions to Red Hat Enterprise Linux and other Red Hat products.

Chapter 2.

Accessing the Command Line

Goal

Log in to a Linux system and run simple commands by using the shell.

Sections

- Accessing the Command Line in a Text Interface (and Quiz)
- Accessing the Command Line with the Desktop Environment (and Guided Exercise)
- Executing Commands with the Bash Shell (and Quiz)

Lab

- Access the Command Line

2.1. Accessing the Command Line in a Text Interface

Objectives

- Log in to a Linux system and run simple commands by using the shell.

Introduction to the Bash Shell

A *command line* is a text-based interface that is used to input instructions to a computer system. The Linux command line is provided by a program named the *shell*. Many shell program variants have been developed over the years. Every user can use a different shell, but Red Hat recommends using the default shell for system administration.

The default user shell in Red Hat Enterprise Linux (RHEL) is the GNU Bourne-Again Shell (Bash). The Bash shell is an improved version of the original Bourne shell (sh) on UNIX systems.

The shell displays a string when it is waiting for user input, named the *shell prompt*. When a regular user starts a shell, the prompt includes an ending dollar sign (\$):

```
user@host:~$
```

When the shell is running as the superuser (the root user), the prompt includes an ending number sign (#):

```
root@host:~#
```

The number sign indicates the superuser shell. Identifying a superuser shell helps you to avoid mistakes that can affect the whole system.

Using Bash to execute commands can be powerful. The Bash shell provides a scripting language that can support task automation. The shell has capabilities that can enable or simplify operations that are hard to accomplish at scale with graphical tools.

 Note

The Bash shell is conceptually similar to the Microsoft Windows `cmd.exe` command-line interpreter. However, Bash has a sophisticated scripting language, and is more similar to Windows PowerShell.

On macOS, the Bash shell was the default shell before macOS 10.15 Catalina. Starting from macOS 10.15 Catalina, the default shell is the `zsh` shell, an alternative shell that is also available in RHEL.

Shell Basics

Commands that are entered at the shell prompt have three basic parts:

- *Command* to run.
- *Options* to adjust the behavior of the command.
- *Arguments*, which are typically the targets of the command.

The command is the name of the program to run. It might be followed by one or more options, which adjust the behavior of the command. Options typically start with one or two dashes (`-a` or `--all`, for example) to distinguish them from arguments. Commands might also be followed by one or more arguments, which often indicate a target that the command should operate on.

For example, in the `usermod -L user01` string, `usermod` is the command, `-L` is the option, and `user01` is the argument. This command locks the password of the `user01` user account.

Logging in to a Local System

A *terminal* is a text-based interface to enter commands into and print output from a computer system. To run the shell, you must log in to the computer on a terminal.

A hardware keyboard and display for input and output might be directly connected to the computer. This is the *physical console* from the Linux machine. The physical console supports multiple *virtual consoles*, which can run on separate terminals. Each virtual console supports an independent login session. You can switch between the virtual consoles by pressing `Ctrl` + `Alt` and a function key (`F1` through `F6`) at the same time. Most of these virtual consoles run a terminal that provides a text login prompt. If you enter your username and password correctly, then you log in and get a shell prompt.

The computer might provide a graphical login prompt on one of the virtual consoles. You can use the graphical login prompt to log in to a *graphical environment*. The graphical environment also runs on a

virtual console. To get a shell prompt, you must start a terminal program in the graphical environment. The shell prompt is provided in an application window of your graphical terminal program.

Note

Many system administrators choose not to run a graphical environment on their servers, because users do not log in to servers as a desktop workspace. A server's workload can more effectively use the significant resources that a graphical environment consumes.

In Red Hat Enterprise Linux 10, if the graphical environment is available, then the login screen runs on the first virtual console, which is named `tty1`. Five additional text login prompts are available on virtual consoles two (`tty2`) through six (`tty6`).

The graphical environment starts on the first virtual console that a login session is not currently using. Typically, your graphical session replaces the login prompt on the second virtual console (`tty2`). However, if an active text login session (not *just* a login prompt) is using that console, then the next free virtual console is used instead.

The graphical login screen continues to run on the first virtual console (`tty1`). If you are already logged in to a graphical session, and switch to another user in the graphical environment without logging out, then another graphical environment is started for that user on the next available virtual console.

When you log out of a graphical environment, it exits the virtual console, and the physical console automatically switches back to the graphical login screen on the first virtual console.

A *headless server* does not have a keyboard and display that are permanently connected to it. A data center might be filled with many racks of headless servers, and not providing each with a keyboard and display saves space and expense. For administrators to log in, a login prompt for a headless server might be provided by its *serial console*, which runs on a serial port that is connected to a networked console server for remote access.

The serial console is typically used to access the server if the server network card becomes misconfigured and logging into the server over the conventional network connection becomes impossible. Most of the time, however, headless servers are accessed by other means over the network, for example by using Remote Desktop Protocol (RDP) for running a graphical interface on the target machine.

Logging in to a Remote System

Linux users and administrators often need to get shell access to a remote system by connecting to it over the network. In a modern computing environment, many headless servers are virtual machines or are running as public or private cloud instances. These systems are not physical and do not have real hardware consoles. They might not even provide access to their (*simulated*) physical console or serial console.

In Linux, the most common way to get a shell prompt on a remote system is to use Secure Shell (SSH). Most Linux systems (including Red Hat Enterprise Linux) and macOS provide the OpenSSH command-line program `ssh` for this purpose.

In the following example, a user with a shell prompt on the host machine uses the `ssh` command to log in to the remote Linux system `remotehost` as the `remoteuser` user:

```
user@host:~$ ssh remoteuser@remotehost
remoteuser@remotehost's password: password
remoteuser@remotehost:~$
```

The `ssh` command encrypts the connection to secure the communication against eavesdropping or hijacking of the passwords and content.

For increased security, some systems, such as new cloud instances, do not allow users to use a password when logging in with the SSH protocol. Another way to authenticate to a remote machine without entering a password is through *public key authentication*.

With this authentication method, users have a special identity file with a *private key*, which is equivalent to a password, and which they keep secret. Their account on the server is configured with a matching *public key*, which does not have to be secret. When logging in, users can add an argument to the `ssh` command to provide the private key. If their matching public key is installed in that account on that remote server, then it logs in the user without asking for a password.

In the next example, a user with a shell prompt on the host machine uses the public key authentication method to log in to the `remotehost` remote system as the `remoteuser` user. The `ssh` command `-i` option specifies the user's private key file, which is `mylab.pem`. The matching public key is already set up as an authorized key in the `remoteuser` account.

```
user@host:~$ ssh -i mylab.pem remoteuser@remotehost
remoteuser@remotehost:~$
```

For security reasons, only the user who owns the file is allowed to have read access to the private key file. OpenSSH blocks the private key from being used if the file's permissions are incorrect, to alert the user that their private key is insecure. In the preceding example, where the private key is in the `mylab.pem` file, you can use the `chmod 600 mylab.pem` command to ensure that only the owner can read the file.

Users might also have configured private keys that are tried automatically, but that topic is beyond the scope of this section. For more information about OpenSSH and public key authentication, refer to [Using Secure Communications between Two Systems with OpenSSH](#) in the *Red Hat Enterprise Linux 10 Securing Networks* guide.

Initial Login on a Remote System

Each time that you connect to a remote host by using the `ssh` command, the remote host sends its *host key* to authenticate itself and to help to set up encrypted communication. The `ssh` command compares the host key against a list of saved host keys to ensure that it is not changed.

When you first log in to a machine, the remote host does not *know* your keys, so you are prompted with a warning from SSH that it cannot establish the authenticity of the host:

```
user@host:~$ ssh -i mylab.pem remoteuser@remotehost
The authenticity of host 'remotehost (192.0.2.42)' can't be established.
ECDSA key fingerprint is 47:bf:82:cd:fa:68:06:ee:d8:83:03:1a:bb:29:14:a3.
Are you sure you want to continue connecting (yes/no)? yes
remoteuser@remotehost:~$
```

You get this message when your local machine does not have a saved host key for the remote host. If you enter `yes`, then the host key that the remote host sent is accepted and saved for future reference. The login process continues, and you should not see this message again when connecting to this host. If you enter `no`, then the host key is rejected and the connection is closed.

If the local machine does have a saved host key and it does not match the one that the remote host sent, then the connection is closed automatically with a warning.

 Warning

If the host key changed, then it might indicate that someone is trying to pretend to be that host to hijack the connection, which is also known as an *interceptor attack*. In SSH, host keys protect against interceptor attacks. These host keys are unique for each server, and must be changed periodically and whenever a compromise is suspected.

Logging out of a Remote System

When you are finished with the shell and want to quit, you can choose one of several ways to end the session. You can enter the `exit` command to terminate the current shell session. Alternatively, finish a session by pressing `Ctrl + D`.

The following example shows a user who logs out of an SSH session:

```
remoteuser@remotehost:~$ exit
logout
Connection to remotehost closed.
user@host:~$
```

 References

`intro(1)`, `bash(1)`, `pts(4)`, `ssh(1)`, and `ssh-keygen(1)` man pages

For more information about OpenSSH and public key authentication, refer to the *Using Secure Communications between Two Systems with OpenSSH* chapter in the *Red Hat Enterprise Linux 10 Securing Networks* guide at https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html-single/securing_networks/index#using-secure-communications-between-two-systems-with-openssh

2.2. Quiz

Accessing the Command Line in a Text Interface

Choose the correct answer to the following questions:

1. Which term describes the interpreter that executes commands that are typed as strings?

- ☐ a. Command
- ☐ b. Console
- ☐ c. Shell
- ☐ d. Terminal

2. Which term describes the visual cue that indicates that an interactive shell is waiting for the user to type a command?

- ☐ a. Argument
- ☐ b. Command
- ☐ c. Option
- ☐ d. Prompt

3. Which term describes the name of a program to run?

- ☐ a. Argument
- ☐ b. Command
- ☐ c. Option
- ☐ d. Prompt

4. Which term describes the part of the command that adjusts the behavior of a command?

- ☐ a. Argument
- ☐ b. Command
- ☐ c. Option
- ☐ d. Prompt

5. Which term describes the part of the command that specifies the target that the command should operate on?
- ☐ a. Argument
 - ☐ b. Command
 - ☐ c. Option
 - ☐ d. Prompt
6. Which term describes the hardware display and keyboard to interact with a system?
- ☐ a. Physical Console
 - ☐ b. Virtual Console
 - ☐ c. Shell
 - ☐ d. Terminal
7. Which term describes one of multiple logical consoles that can each support an independent login session?
- ☐ a. Physical Console
 - ☐ b. Virtual Console
 - ☐ c. Shell
 - ☐ d. Terminal
8. Which term describes an interface that provides a display for output and a keyboard for input to a shell session?
- ☐ a. Console
 - ☐ b. Virtual Console
 - ☐ c. Shell
 - ☐ d. Terminal

Solution

1. Which term describes the interpreter that executes commands that are typed as strings?
 - ☐ a. Command
 - ☐ b. Console
 - ☒ c. **Shell**
 - ☐ d. Terminal
2. Which term describes the visual cue that indicates that an interactive shell is waiting for the user to type a command?
 - ☐ a. Argument
 - ☐ b. Command
 - ☐ c. Option
 - ☒ d. **Prompt**
3. Which term describes the name of a program to run?
 - ☐ a. Argument
 - ☒ b. **Command**
 - ☐ c. Option
 - ☐ d. Prompt
4. Which term describes the part of the command that adjusts the behavior of a command?
 - ☐ a. Argument
 - ☐ b. Command
 - ☒ c. **Option**
 - ☐ d. Prompt
5. Which term describes the part of the command that specifies the target that the command should operate on?
 - ☒ a. **Argument**
 - ☐ b. Command
 - ☐ c. Option
 - ☐ d. Prompt

6. Which term describes the hardware display and keyboard to interact with a system?

☒ a. **Physical Console**

☐ b. Virtual Console

☐ c. Shell

☐ d. Terminal

7. Which term describes one of multiple logical consoles that can each support an independent login session?

☐ a. Physical Console

☒ b. **Virtual Console**

☐ c. Shell

☐ d. Terminal

8. Which term describes an interface that provides a display for output and a keyboard for input to a shell session?

☐ a. Console

☐ b. Virtual Console

☐ c. Shell

☒ d. **Terminal**

2.3. Accessing the Command Line with the Desktop Environment

Objectives

- Log in to a Linux system with the GNOME desktop environment and run commands from a shell prompt in a terminal program.

The GNOME Desktop Environment

The *desktop environment* is the graphical user interface on a Linux system. GNOME 47 is the default desktop environment in Red Hat Enterprise Linux 10. GNOME provides an integrated desktop for users and a unified development platform on top of a graphical framework that is powered by Wayland.

The highly customizable GNOME Shell provides the core user interface functions for the desktop environment. In RHEL 10, the default GNOME Shell appearance is the standard GNOME theme, which is used in this section. You can switch from the default to the GNOME Classic theme, which is closer to the appearance of earlier versions of the GNOME desktop. You can set the theme persistently by clicking the **Settings** icon at login. The **Settings** icon displays after selecting the user account, but before entering the password.

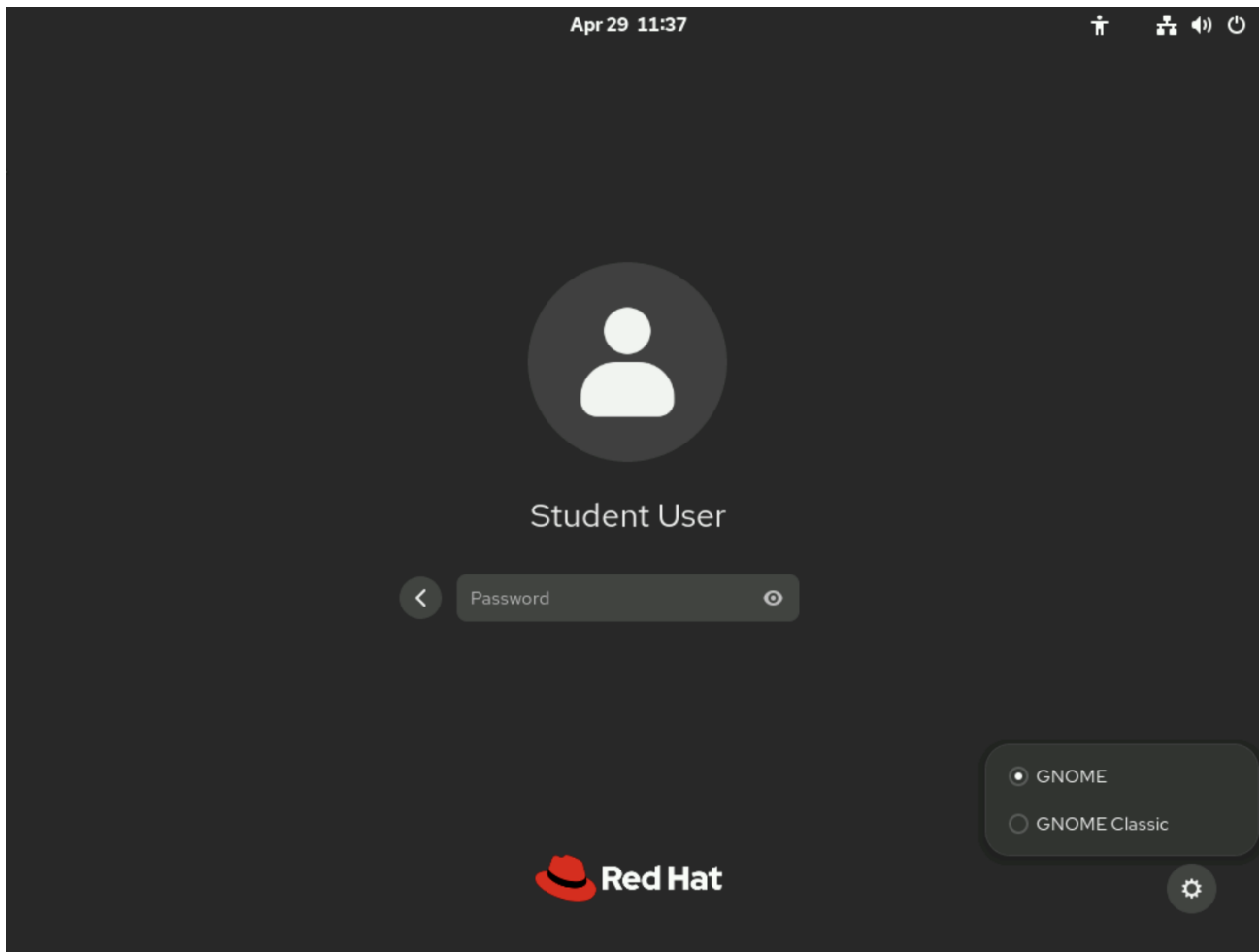


Figure 5. The RHEL 10 login screen

The first time that you log in as a new user, the Tour program starts automatically to provide a quick overview of new RHEL 10 desktop features. The tour is entirely optional and you can close the window to skip it.

Parts of the GNOME Shell

The GNOME Shell user interface includes a number of major elements, as shown in the following screenshot.

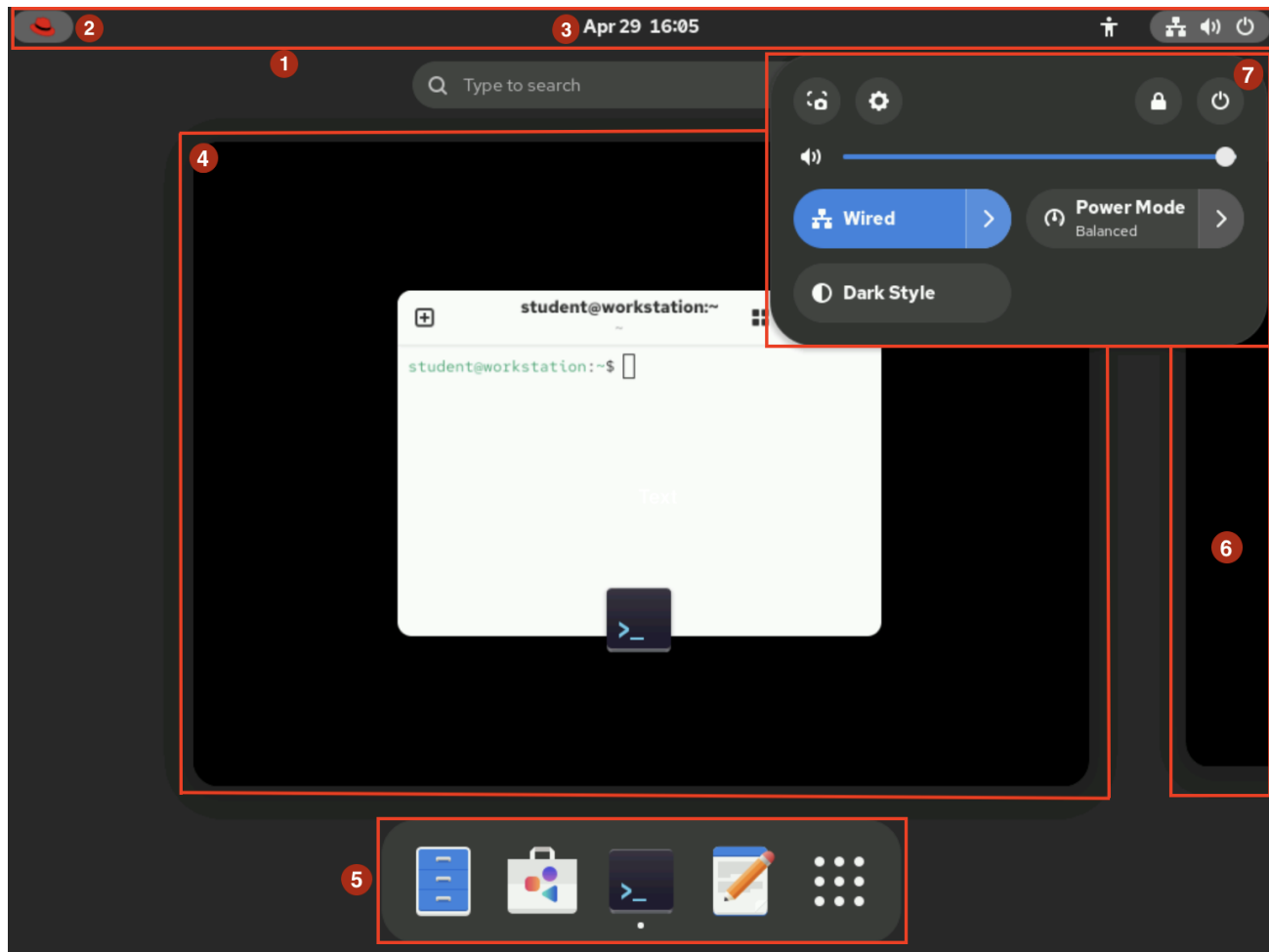


Figure 6. The GNOME desktop environment

- ① **Top bar:** The bar that runs along the top of the screen is always displayed, even when entering the Activities Overview or when you switch workspaces. The top bar provides convenient access to the Activities Overview, the message tray, and the system menu.
- ② **Activities Overview:** This mode helps to organize windows and to start applications. Enter the Activities Overview by clicking the **Red Hat** logo at the upper-left corner of the top bar, or by pressing the **Super** key. Find the **Super** key (sometimes called the **Windows** key or **Command** key) near the lower-left corner of most common keyboards. The three main areas of the Activities Overview are the *workspace selector* to the right of the windows overview, the *windows overview* in the center, and the *dash* at the bottom of the screen.

- ③ *Message tray*: The message tray displays notifications from applications or system components. Typically, the notification first appears briefly as a single line at the top of the screen, and a persistent indicator appears in the top bar next to the clock to inform you of recently received notifications. Open the message tray to review these notifications by clicking the clock on the top bar or by pressing **Super** + **M**. Close the message tray by clicking the clock on the top bar, or by pressing **Esc** or **Super** + **M** again. The message tray also shows the calendar and information about calendar events.

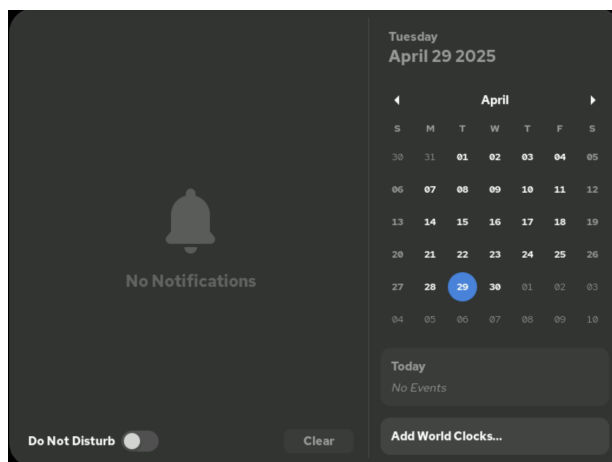


Figure 7. Message tray

- ④ *Windows overview*: The area in the center of the Activities Overview that displays thumbnails of active windows in the current workspace. You can bring windows to the foreground on a cluttered workspace, or move them to another workspace.
- ⑤ *Dash*: This configurable list of icons displays your favorite applications, running applications, and a **Show Applications** icon to select additional applications. Start applications by clicking the application's icon or by using **Show Applications** to find applications that are used less often. The dash is also called the *dock*.
- ⑥ *Workspace selector*: An area in the Activities Overview to the right of the windows overview. The workspace selector displays active workspaces and enables you to switch between workspaces.
- ⑦ *System menu*: The menu at the upper-right corner on the top bar provides access to several functions and settings. You can record the screen, access account settings, lock the screen, log out of the system, and shut down the system. You can also adjust the brightness of the screen, and switch network connections on or off. You can also turn on Dark Style, which affects all windows except legacy applications. The system menu also enables you to switch network and VPN connections on or off.

Keyboard Shortcuts

The GNOME Shell enables you to customize keyboard shortcuts for navigating the desktop environment. Open the system menu in the upper-right corner, and click the **Settings** icon. In the application window that opens, select **Keyboard** from the left pane. The right pane displays your current shortcut settings in **Keyboard Shortcuts > View and Customize Shortcuts**.

Some keyboard shortcuts, such as function keys or the **Super** key, might be difficult to send to a virtual machine. Special keystrokes that those shortcuts use might be captured by your local operating system, or by the application that you are using to access the graphical desktop of your virtual machine.

Important

In some Red Hat training environments, your web browser might not pass the **Super** key to the virtual machine in the classroom environment. To address this issue, you can use an on-screen keyboard.

At the top of your browser window that displays the interface for your virtual machine, click the keyboard icon on the right side. An on-screen keyboard opens. Click the icon again to close the on-screen keyboard.

The on-screen keyboard treats the **Super** key as a modifier key that is often held down while pressing another key. If you click it once, then the key turns yellow, to indicate that the key is being held down. So for example, to enter the **Super** + **M** key combination in the on-screen keyboard, click **Super**, and then click **M**.

To press and release **Super** in the on-screen keyboard, click it twice. The first click holds the **Super** key down, and the second click releases it.

The other keys that the on-screen keyboard treats as modifier keys are **Shift**, **Ctrl**, **Alt**, and **Caps**. The **Esc** and **Menu** keys are treated like typical keys and *not* modifier keys.

Accessing Workspaces

Workspaces are separate desktop screens that display various application windows. You can use workspaces to organize the working environment by grouping open application windows by task. For example, you can group windows for a particular system maintenance activity (such as setting up a new remote server) in one workspace, and then group email and other communication applications in another workspace.

To switch between workspaces, choose between two methods. The first method is to press **Ctrl** + **Alt** + **LeftArrow** or **Ctrl** + **Alt** + **RightArrow** to switch between workspaces sequentially.

The second method is to enter the Activities Overview by clicking the **Red Hat** logo in the upper-left corner, and then clicking the desired workspace.

Important

Similar to using the **Super** key, in some Red Hat training environments, your web browser does not usually pass **Ctrl** + **Alt** key combinations to the virtual machine in the classroom environment.

You can enter these key combinations to switch workspaces by using the on-screen keyboard. At least two workspaces must be in use. Open the on-screen keyboard and click **Ctrl**, click **Alt**, and then click either **LeftArrow** or **RightArrow**.

However, in those training environments, it is generally simpler to avoid the keyboard shortcuts and the on-screen keyboard. Instead, use the workspace selector in the Activities Overview.

An advantage of using the Activities Overview to select workspaces is that you can click and drag windows between workspaces. You can view active workspaces by clicking **Show Applications** in the dash, and the active workspaces display below the search bar.

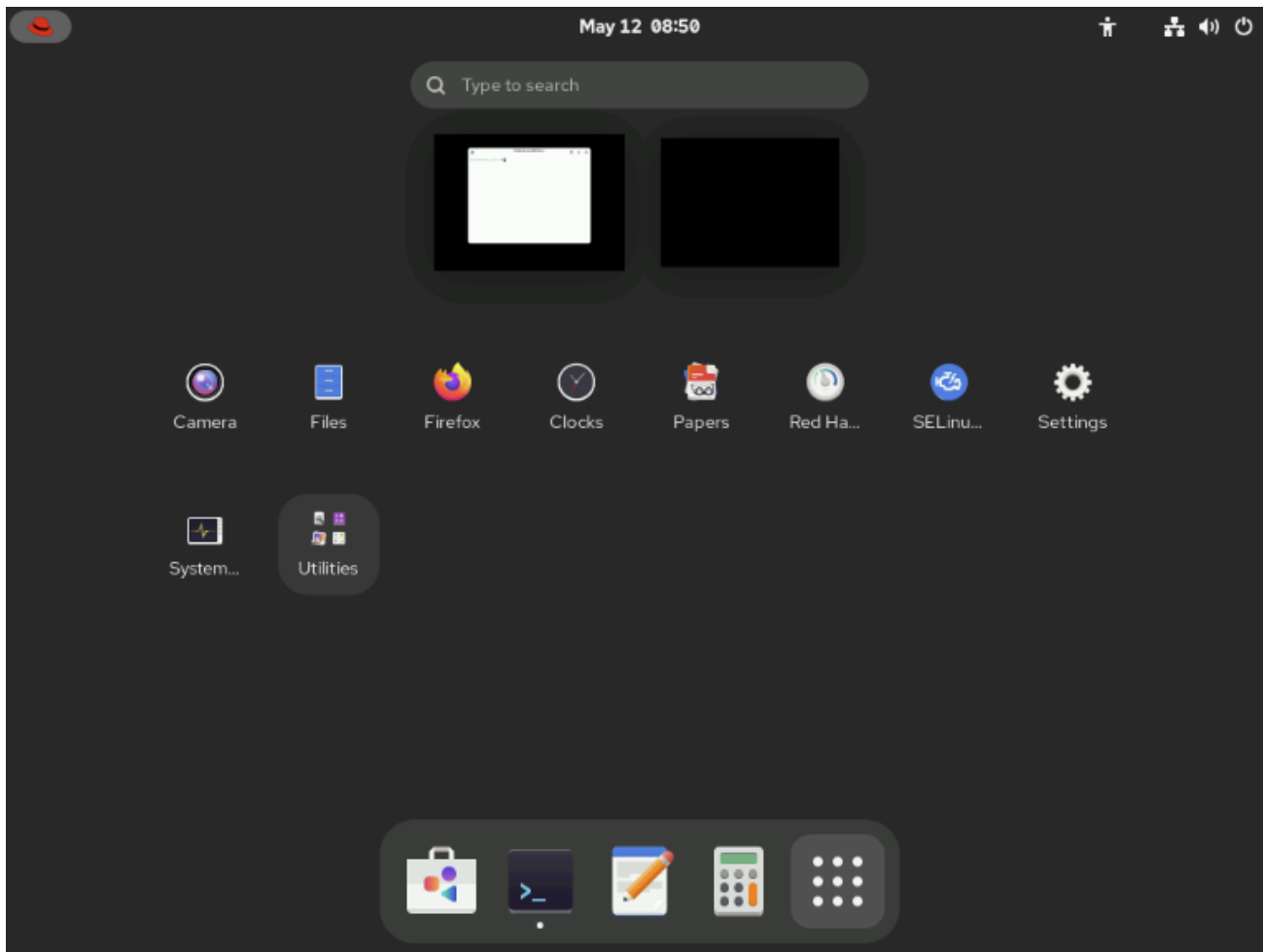


Figure 8. Display active workspaces

Workspaces can be configured as *dynamic* or *fixed*. **Dynamic Workspaces** is the default setting. With dynamic workspaces, you create workspaces by dragging windows from one workspace to another, and empty workspaces are removed. Alternatively, you can set a fixed number of workspaces that will always display, including empty workspaces.

To configure workspace settings, go to **System menu > Settings > Multitasking**. You can select **Dynamic Workspaces** or **Fixed Number of Workspaces**.

Starting a Terminal

In RHEL 10, the Ptyxis terminal emulator replaces the GNOME Terminal. Use one of the following methods to start a terminal:

- From the Activities Overview, select **Terminal** from the dash, either by clicking its icon in the dash or by using **Show Applications**.
- Search for `terminal` in the search field at the top of the windows overview.

- Press the **Alt** + **F2** key combination to open the **Run a Command** screen, and enter `ptx is` to start the terminal.

When you open a terminal window, a shell prompt displays for the user who started the graphical terminal program. The shell prompt and the terminal window's title bar indicate the current username, hostname, and working directory.

Locking the Screen and Logging Out

Lock the screen, or log out entirely, by using the system menu in the upper-right corner.

To lock the screen, go to the system menu, and click the **Lock** icon at the right side of the menu, or press **Super** + **L** (which might be easier to remember as **Windows** + **L**). The screen also locks if the graphical session is idle for a few minutes.

A lock screen curtain appears that shows the system time and the name of the logged-in user. To unlock the screen, you can press **Enter**, press **Space**, or click the left mouse button. Then, enter that user's password on the lock screen.

To log out and end the current graphical login session, go to the system menu in the upper-right corner, click the **Power** icon, and then click **Log Out**. The confirmation dialog provides the option to cancel or confirm the log out action.

Powering off or Rebooting the System

To shut down the system, go to the system menu in the upper-right corner, click the **Power** icon, and then click **Power Off**. Alternatively, press **Ctrl** + **Alt** + **Del**. The confirmation dialog provides the option to cancel or confirm the power off action. If you do not make a choice, then the system automatically shuts down after 60 seconds.

To reboot the system, go to the system menu in the upper-right corner, click the **Power** icon, and then click **Restart**. The confirmation dialog provides the option to cancel or confirm the restart action. If you do not make a choice, then the system automatically restarts after 60 seconds.

In RHEL 10, when rebooting or shutting down, GNOME might give you an option to **Install pending software updates** if updates are available. You can deselect the checkbox to avoid the update when shutting down or rebooting.

References

For more information about using GNOME desktop environment in Red Hat Enterprise Linux 10, refer to the *Using the GNOME Desktop Environment* guide at https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html-single/using_the_gnome_desktop_environment/index

2.4. Guided Exercise

Access the Command Line with the Desktop Environment

Log in to a Linux system with the GNOME desktop environment and run commands from a shell prompt in a terminal program.

Outcomes

- Log in to a Linux system by using the GNOME desktop environment.
- Run commands from a shell prompt in a terminal program.

Prerequisites

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
student@workstation:~$ lab start cli-desktop
```

Instructions

1. Log out of the GNOME desktop.
 - 1.1. On the workstation machine, click the system menu in the upper-right corner.
 - 1.2. Click the **Power** icon, and then click **Log Out**.
 - 1.3. Click **Log Out** in the confirmation dialog box.
2. Log in to the workstation machine as the student user with student as the password.
 - 2.1. Click the student user account on the GNOME login screen. Enter student when prompted for the password.
 - 2.2. Press **Enter**.
3. Use the terminal to change the password for the student user from student to redhat!23.



Important

The lab finish script at the end of this exercise resets the password for the student user to student. You must execute the script at the end of the exercise.

- 3.1. Open a terminal.

Press the **Super** key twice to enter the Activities Overview, or click the **Red Hat** logo in the upper-left corner. Type `terminal`, and then press **Enter** to open the terminal.

- 3.2.** In the terminal window, enter the `passwd` command at the shell prompt. Follow the prompts in the terminal to change the student user's password from `student` to `redhat!23`.

```
student@workstation:~$ passwd
Changing password for user student.
Current password: student
New password: redhat!23
Retype new password: redhat!23
passwd: all authentication tokens updated successfully.
```

- 4.** Log out of the GNOME desktop, and then log back in as the student user with `redhat!23` as the password, to verify the changed password.

4.1. Click the system menu in the upper-right corner.

4.2. Click the **Power** icon, and then click **Log Out**.

4.3. Click **Log Out** in the confirmation dialog box.

4.4. At the GNOME login screen, click the student user account. Enter `redhat!23` when prompted for the password.

4.5. Press **Enter**.

- 5.** Lock the screen.

5.1. From the system menu in the upper-right corner, press the **Lock** icon.

- 6.** Unlock the screen.

6.1. Press **Enter** to unlock the screen.

6.2. In the **Password** field, enter `redhat!23` as the password.

6.3. Press **Enter**.

- 7.** Determine how to shut down the workstation machine from the graphical interface, but cancel the operation without shutting down the system.

7.1. From the system menu in the upper-right corner, click the **Power** icon, and then click **Power Off**. A dialog box is displayed with the options to either **Cancel** or **Power Off** the machine.

7.2. Click **Cancel**.

8. Open a terminal and use the following lab command to finish the exercise.

8.1. Press the **Super** key twice to enter the Activities Overview, or click the **Red Hat** logo in the upper-left corner. Type `terminal`, and then press **Enter** to open the terminal.

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish cli-desktop
```

2.5. Executing Commands with the Bash Shell

Objectives

- Save time when running commands from a Bash shell prompt by using some of its productivity features.

Basic Command Syntax

The GNU Bourne-Again Shell (Bash) is a program that interprets commands that the user types. Commands are the names of programs that are installed on the system. Each command that is typed into the shell can have up to three parts:

- *Command* to run.
- *Options*, which usually begin with a dash (-) or double dashes (--).
- *Arguments*, which are typically the targets of the command.

Each command has its options and arguments. Each word that is typed into the shell is separated from other words with spaces.

Type each command on a separate line. When you are ready to execute a command, press the Enter key. After you run a command, the system returns output, and then another shell prompt appears.

```
user@host:~$ whoami
user
user@host:~$
```

To type more than one command on a single line, use the semicolon (;) as a command separator. A semicolon is a member of a class of characters called *metacharacters* that have a special interpretation for Bash. In this case, the output of both commands is displayed before the following shell prompt appears.

The following example shows how to combine two commands (command1 and command2) on the command line.

```
user@host:~$ command1 ; command2
command1 output
command2 output
user@host:~$
```

Write Simple Commands

The `date` command displays the current date and time. The superuser or a privileged user can also use the `date` command to set the system clock. Use the plus sign (+) as an argument to specify a format string for the `date` command.

```
user@host:~$ date
Tue Apr 29 17:58:22 UTC 2025
user@host:~$ date +%R
17:58
user@host:~$ date +%x
04/29/25
```

The `passwd` command with no options changes the current user's password. To change the password, first specify the original password for the account. By default, the `passwd` command is configured to require a strong password; to consist of lowercase letters, uppercase letters, numbers, and symbols; and not to be based on a dictionary word.

```
user@host:~$ passwd
Current password: old_password
New password: new_password
Retype new password: new_password
passwd: password updated successfully
```

A superuser or privileged user can use the `passwd` command to change another user's password.

Linux does not require file name extensions to classify files by type. The `file` command scans the compiled header of a file for a 2-digit magic number and displays its type. Text files are recognized because they are not compiled.


```

user@host:~$ file /etc/passwd
/etc/passwd: ASCII text
user@host:~$ file /bin/passwd
/bin/passwd: setuid ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV),
dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2,
BuildID[sha1]=631f3ed767f58db446ce1551a762a859721edf1d, for GNU/Linux 3.2.0,
stripped
user@host:~$ file /home
/home: directory

```

View the Contents of Files

The `cat` command is often used in Linux. Use this command to create single or multiple files, to view the contents of files, to concatenate the contents from various files, and to redirect contents of the file to a terminal or to files.

The following example shows how to view the contents of the `/etc/passwd` file:

```

user@host:~$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
bin:x:1:1:bin:/bin:/sbin/nologin
daemon:x:2:2:daemon:/sbin:/sbin/nologin
adm:x:3:4:adm:/var/adm:/sbin/nologin
...output omitted...

```

To display the contents of multiple files, add the file names to the `cat` command as arguments:

```

user@host:~$ cat file1 file2
Hello World!!
Introduction to Linux commands.

```

Some files are long and might need more space to be displayed than the terminal provides. The `cat` command does not display the contents of a file as pages. The `less` command displays one page of a file at a time and you can scroll at your leisure.

Use the `less` command to page forward and backward through longer files than can fit on one terminal window. Use the `UpArrow` key and the `DownArrow` key to scroll up and down. Press `Q` to exit the command.

The `head` and `tail` commands display the beginning and the end of a file, respectively. By default, these commands display 10 lines of the file, but they both have a `-n` option to specify a different

number of lines.

```
user@host:~$ head /etc/passwd
root:x:0:0:Super User:/root:/bin/bash
bin:x:1:1:bin:/bin:/usr/sbin/nologin
daemon:x:2:2:daemon:/sbin:/usr/sbin/nologin
adm:x:3:4:adm:/var/adm:/usr/sbin/nologin
lp:x:4:7:lp:/var/spool/lpd:/usr/sbin/nologin
sync:x:5:0:sync:/sbin:/bin/sync
shutdown:x:6:0:shutdown:/sbin:/sbin/shutdown
halt:x:7:0:halt:/sbin:/sbin/halt
mail:x:8:12:mail:/var/spool/mail:/usr/sbin/nologin
operator:x:11:0:operator:/root:/usr/sbin/nologin

user@host:~$ tail -n 3 /etc/passwd
libstoragemgmt:x:981:981:daemon account for libstoragemgmt:/usr/sbin/nologin
gnome-initial-setup:x:980:979:./run/gnome-initial-setup:/sbin/nologin
dnsmasq:x:979:978:Dnsmasq DHCP and DNS
server:/var/lib/dnsmasq:/usr/sbin/nologin
```

The `wc` command counts lines, words, and characters in a file. Use the `-l`, `-w`, or `-c` options to display only the given number of lines, words, or characters, respectively.

```
user@host:~$ wc /etc/passwd
 45  113 2738 /etc/passwd

user@host:~$ wc -l /etc/passwd ; wc -l /etc/group
 45 /etc/passwd
 70 /etc/group

user@host:~$ wc -c /etc/group /etc/hosts
1027 /etc/group
 348 /etc/hosts
1375 total
```

Understand Tab Completion

With tab completion, users can quickly complete commands or file names after typing enough at the prompt to make it unique. If the typed characters are not unique, then pressing the `Tab` key twice displays all commands that begin with the typed characters.

```

user@host:~$ pasTab+Tab ①
passt          passt.avx2      pasta          paste
passt-repair   passwd          pasta.avx2
user@host:~$ passTab+Tab ②
passt          passt-repair   passt.avx2    passwd
user@host:~$ passwTab ③
Current password:

```

- ① Press `Tab` twice.
- ② Press `Tab` twice.
- ③ Press `Tab` once.

Tab completion helps to complete file names when typing them as arguments to commands. Press `Tab` to complete as much of the file name as possible. Pressing `Tab` a second time causes the shell to list all files that the current pattern matches. Type additional characters until the name is unique, and then use tab completion to complete the command.

```

user@host:~$ ls /etc/pasTab ①
user@host:~$ ls /etc/passwdTab ②
passwd  passwd-

```

- ① Press `Tab` once.
- ② Press `Tab` once.

Use the `useradd` command to create users on the system. The `useradd` command has many options that might be hard to remember. By using tab completion, you can complete the option name with minimal typing.

```

root@host:~# useradd --Tab+Tab ①
--badnames          --gid              --no-log-init      --shell
--base-dir          --groups           --non-unique       --skel
--btrfs-subvolume-home --help            --no-user-group    --
system
--comment           --home-dir         --password         --uid
--create-home       --inactive         --prefix           --user-
group
--defaults          --key              --root
--expiredate        --no-create-home   --selinux-user

```

- ① Press `Tab` twice.

Write a Long Command on Multiple Lines

Commands with many options and arguments can quickly become long and are automatically wrapped by the command window when the cursor reaches the right margin. Instead, type a long command by using more than one line for easier reading.

To write one command in more than one line, use a backslash character (`\`), which is referred to as the *escape character*. The backslash character ignores the meaning of the following character.

Previously, you learned that to complete a command entry, you press the `Enter` key, the newline character. By escaping the newline character, the shell moves to a new command line without executing the command. This way, the shell acknowledges the request by displaying a continuation prompt on an empty new line, which is known as the secondary prompt, and uses the greater-than character (`>`) by default. Commands can continue over many lines.

One issue with the secondary prompt's use of the greater-than character (`>`) is that new learners might mistakenly insert it as part of the typed command. Then, the shell interprets a typed greater-than character as *output redirection*, which the user did not intend. To avoid confusion, this lesson does not show secondary prompts in system output. A user still sees the secondary prompt in their shell window, but the course material intentionally displays only the characters to be typed, as demonstrated in the following example.

```
user@host:~$ head -n 3 \
/usr/share/dict/words \
/usr/share/dict/linux.words
==> /usr/share/dict/words <==
1080
10-point
10th

==> /usr/share/dict/linux.words <==
1080
10-point
10th
```

Display the Command History

The `history` command displays a list of previously executed commands that are prefixed with a command number.

The exclamation point character (`!`) is a metacharacter to expand previous commands without retyping them. The `!number` command expands to the command that matches the specified number.

The `!string` command expands to the most recent command that begins with the specified string.

```
user@host:~$ history
...output omitted...
23 clear
24 who
25 pwd
26 ls /etc
27 uptime
28 ls -l
29 date
30 history

user@host:~$ !ls
ls -l
total 0
drwxr-xr-x. 2 student student 6 Feb  5 13:21 Desktop
...output omitted...

user@host:~$ !26
ls /etc
abrt                      hosts                      pulse
adjtime                   hosts.allow                purple
aliases                   hosts.deny                  qemu-ga
...output omitted...
```

The arrow keys help to navigate through previous commands in the shell's history. The `UpArrow` edits the previous command in the history list. The `DownArrow` edits the next command in the history list. The `LeftArrow` and `RightArrow` move the cursor left and right in the current command from the history list so that you can edit the command before running it.

Use either the `Esc` + `.` or `Alt` + `.` key combination simultaneously to insert the last word of the previous command at the cursor's current location. The repeated use of the key combination replaces that text with the last word of earlier commands in history. The `Alt` + `.` key combination is particularly convenient, because you can hold down `Alt` and press `.` repeatedly to quickly cycle earlier commands.

Edit the Command Line

When used interactively, Bash has a command-line editing feature. Use the text editor commands to move around and modify the currently typed command. Using the arrow keys to move within the

current command and to step through the command history was introduced earlier in this section. The following table shows additional powerful editing commands.

Table 6. Useful Command-line Editing Shortcuts

Shortcut	Description
Ctrl + A	Jump to the beginning of the command line.
Ctrl + E	Jump to the end of the command line.
Ctrl + U	Clear from the cursor to the beginning of the command line.
Ctrl + K	Clear from the cursor to the end of the command line.
Ctrl + LeftArrow	Jump to the beginning of the previous word on the command line.
Ctrl + RightArrow	Jump to the end of the next word on the command line.
Ctrl + R	Search the history list of commands for a pattern.

These command-line editing commands are the most helpful for new users. For other commands, refer to the `bash(1)` man page.

 References

`bash(1)`, `date(1)`, `file(1)`, `magic(5)`, `cat(1)`, `more(1)`, `less(1)`, `head(1)`, `passwd(1)`, `tail(1)`, and `wc(1)` man pages

2.6. Quiz

Executing Commands with the Bash Shell

Choose the correct answers to the following questions:

1. Which Bash command displays the last five lines of the `/var/log/messages` file?

- ☐ a. `head -n 10 /var/log/messages`
- ☐ b. `tail 10 /var/log/messages`
- ☐ c. `tail -n 5 /var/log/messages`
- ☐ d. `tail -l 10 /var/log/messages`
- ☐ e. `less /var/log/messages`

2. Which Bash shortcut or command separates commands on the same line?

- ☐ a. Pressing `Tab`
- ☐ b. `history`
- ☐ c. `;`
- ☐ d. `!string`
- ☐ e. Pressing `Esc` + `.`

3. Which Bash command changes a user's password?

- ☐ a. `password`
- ☐ b. `pass`
- ☐ c. `passwd`
- ☐ d. `usermod`
- ☐ e. `userpassword`

4. Which Bash command displays the file type?

- ☐ a. `file`
- ☐ b. `less`
- ☐ c. `cat`
- ☐ d. `history`
- ☐ e. `view`

5. Which Bash shortcut or command is used to complete commands, file names, and options?

- ☐ a. ;
- ☐ b. !*number*
- ☐ c. history
- ☐ d. Pressing Tab
- ☐ e. Pressing Esc + .

6. Which Bash shortcut or command re-executes a specific command in the history list?

- ☐ a. Pressing Tab
- ☐ b. !*number*
- ☐ c. !*string*
- ☐ d. history
- ☐ e. Pressing Esc + .

7. Which Bash shortcut or command jumps to the beginning of the command line?

- ☐ a. !*number*
- ☐ b. !*string*
- ☐ c. Pressing Ctrl + LeftArrow
- ☐ d. Pressing Ctrl + K
- ☐ e. Pressing Ctrl + A

8. Which Bash shortcut or command displays the list of previously executed commands?

- ☐ a. Pressing Tab
- ☐ b. !*string*
- ☐ c. !*number*
- ☐ d. history
- ☐ e. Pressing Esc + .

9. Which Bash shortcut or command copies the last argument of previous commands?

- ☐ a. Pressing Ctrl + K
- ☐ b. Pressing Ctrl + A
- ☐ c. !*number*
- ☐ d. Pressing Esc + .

Solution

1. Which Bash command displays the last five lines of the `/var/log/messages` file?

- ☒ a. `head -n 10 /var/log/messages`
- ☒ b. `tail 10 /var/log/messages`
- ☒ c. `tail -n 5 /var/log/messages`
- ☒ d. `tail -l 10 /var/log/messages`
- ☒ e. `less /var/log/messages`

2. Which Bash shortcut or command separates commands on the same line?

- ☒ a. Pressing `Tab`
- ☒ b. `history`
- ☒ c. `;`
- ☒ d. `!string`
- ☒ e. Pressing `Esc` + `.`

3. Which Bash command changes a user's password?

- ☒ a. `password`
- ☒ b. `pass`
- ☒ c. `passwd`
- ☒ d. `usermod`
- ☒ e. `userpassword`

4. Which Bash command displays the file type?

- ☒ a. `file`
- ☒ b. `less`
- ☒ c. `cat`
- ☒ d. `history`
- ☒ e. `view`

5. Which Bash shortcut or command is used to complete commands, file names, and options?

- ☒ a. `;`
- ☒ b. `!number`
- ☒ c. `history`
- ☒ d. Pressing `Tab`
- ☒ e. Pressing `Esc` + `.`

6. Which Bash shortcut or command re-executes a specific command in the history list?

- ☒ a. Pressing `Tab`
- ☒ b. `!number`
- ☒ c. `!string`
- ☒ d. `history`
- ☒ e. Pressing `Esc` + `.`

7. Which Bash shortcut or command jumps to the beginning of the command line?

- ☒ a. `!number`
- ☒ b. `!string`
- ☒ c. Pressing `Ctrl` + `LeftArrow`
- ☒ d. Pressing `Ctrl` + `K`
- ☒ e. Pressing `Ctrl` + `A`

8. Which Bash shortcut or command displays the list of previously executed commands?

- ☒ a. Pressing `Tab`
- ☒ b. `!string`
- ☒ c. `!number`
- ☒ d. `history`
- ☒ e. Pressing `Esc` + `.`

9. Which Bash shortcut or command copies the last argument of previous commands?

- ☒ a. Pressing `Ctrl` + `K`
- ☒ b. Pressing `Ctrl` + `A`
- ☒ c. `!number`
- ☒ d. Pressing `Esc` + `.`

2.7. Lab

Access the Command Line

Access the command line on a Linux system and use the Bash shell to run commands.

Outcomes

- Successfully run simple programs from the Bash shell command line.
- Execute commands to identify file types and to display parts of text files.
- Practice using Bash command history shortcuts to more efficiently repeat commands or parts of commands.

Prerequisites

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
student@workstation:~$ lab start cli-review
```

Instructions

1. Use a command to display the current time and date.
2. Display the current time in 24-hour clock time (for example, 13:57). Hint: The format string that displays that output is %R.
3. What kind of file is /home/student/zcat? Is it readable by humans?
4. Use the wc command and Bash shortcuts to display the size of the zcat file.
5. Display the first 10 lines of the zcat file.
6. Display the last 10 lines of the zcat file.
7. Repeat the previous command exactly with four or fewer keystrokes.
8. Use the tail command -n 20 option to display the last 20 lines in the zcat file. Use command-line editing to accomplish this task with a minimal number of keystrokes.
9. Use the shell history to run the date +%R command again.

Evaluation

As the student user on the workstation machine, use the lab command to grade your work. Correct any reported failures and rerun the command until successful.

```
student@workstation:~$ lab grade cli-review
```

Finish

On the workstation machine, change to the student user home directory and use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish cli-review
```

Solution

Access the command line on a Linux system and use the Bash shell to run commands.

Outcomes

- Successfully run simple programs from the Bash shell command line.
- Execute commands to identify file types and to display parts of text files.
- Practice using Bash command history shortcuts to more efficiently repeat commands or parts of commands.

Prerequisites

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
student@workstation:~$ lab start cli-review
```

Instructions

1. Use a command to display the current time and date.

```
student@workstation:~$ date
Thu May 1 15:49:31 UTC 2025
```

2. Display the current time in 24-hour clock time (for example, 13:57). Hint: The format string that displays that output is %R.

- 2.1. Use the +%R argument with the date command to display the current time in 24-hour clock time.

```
student@workstation:~$ date +%R
15:49
```

3. What kind of file is /home/student/zcat? Is it readable by humans?

- 3.1. Use the file command to determine its file type.

```
student@workstation:~$ file zcat
zcat: a /usr/bin/sh script, ASCII text executable
```

ASCII text files are human readable.

4. Use the `wc` command and Bash shortcuts to display the size of the `zcat` file.

- 4.1.** You can use the `wc` command to display the number of lines, words, and bytes in the `zcat` script. Instead of retyping the file name, use the Bash history shortcut `[Esc] + [.]` to reuse the argument from the previous command. Press the `[Esc]` and `[.]` keys at the same time.

```
student@workstation:~$ wc [Esc] + [.]
student@workstation:~$ wc zcat
  51  298 1977 zcat
```

5. Display the first 10 lines of the `zcat` file.

- 5.1.** The `head` command displays the beginning of the file. Try the `[Esc] + [.]` shortcut again.

```
student@workstation:~$ head [Esc] + [.]
student@workstation:~$ head zcat
#!/usr/bin/sh
# Uncompress files to standard output.

# Copyright (C) 2007, 2010-2023 Free Software Foundation, Inc.

# This program is free software; you can redistribute it and/or modify
# it under the terms of the GNU General Public License as published by
# the Free Software Foundation; either version 3 of the License, or
# (at your option) any later version.
```

6. Display the last 10 lines of the `zcat` file.

- 6.1.** Use the `tail` command to display the last 10 lines of the `zcat` file.


```

student@workstation:~$ tail Esc + .
student@workstation:~$ tail zcat
With no FILE, or when FILE is -, read standard input.

Report bugs to <bug-gzip@gnu.org>."

case $1 in
--help)    printf '%s\n' "$usage"    || exit 1; exit;;
--version) printf '%s\n' "$version" || exit 1; exit;;
esac

exec gzip -cd "$@"

```

7. Repeat the previous command exactly with four or fewer keystrokes.

7.1. One option is to press the `UpArrow` key once to scroll back one command through the command history, and then press `Enter`. This approach uses two keystrokes. An alternative method is to enter the shortcut command `!!` and then press `Enter` to run the most recent command in the command history. This approach uses four keystrokes. Try both methods.

```

student@workstation:~$ !!
tail zcat
With no FILE, or when FILE is -, read standard input.

Report bugs to "<bug-gzip@gnu.org>."

case $1 in
--help)    printf '%s\n' "$usage"    || exit 1;;
--version) printf '%s\n' "$version" || exit 1;;
esac

exec gzip -cd "$@"

```

8. Use the `tail` command `-n 20` option to display the last 20 lines in the `zcat` file. Use command-line editing to accomplish this task with a minimal number of keystrokes.

8.1. Use the `UpArrow` key to display the previous command. Next, use the `Ctrl + A` key combination to move the cursor to the beginning of the line. Then, use the `Ctrl + RightArrow` key combination to jump to the next word. Add the `-n 20` option and press `Enter` to execute the command.

```
student@workstation:~$ tail -n 20 zcat
-l, --list          list compressed file contents
-q, --quiet         suppress all warnings
-r, --recursive     operate recursively on directories
-S, --suffix=SUF    use suffix SUF on compressed files
--synchronous       synchronous output (safer if system crashes, but slower)
-t, --test          test compressed file integrity
-v, --verbose        verbose mode
--help              display this help and exit
--version            display version information and exit
```

With no FILE, or when FILE is -, read standard input.

Report bugs to "<bug-gzip@gnu.org>."

```
case $1 in
--help)  printf '%s\n' "$usage"  || exit 1; exit;;
--version) printf '%s\n' "$version" || exit 1; exit;;
esac

exec gzip -cd "$@"
```

9. Use the shell history to run the `date +%R` command again.

9.1. Use the `history` command to display the list of previous commands and to identify which `date` command to execute. Use `!number` to run the command, where *number* is the command number to use from the output of the `history` command.

Your shell history might differ from the following example. Determine the command number to use based on the output of your own `history` command.

```
student@workstation:~$ history
1  date
2  date +%R
3  file zcat
4  wc zcat
5  head zcat
6  tail zcat
7  tail -n 20 zcat
8  history
student@workstation:~$ !2
date +%R
15:59
```

Evaluation

As the student user on the workstation machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

```
student@workstation:~$ lab grade cli-review
```

Finish

On the workstation machine, change to the student user home directory and use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish cli-review
```

2.8. Summary

In this lesson, you learned that the Bash shell is a command interpreter that helps users to specify Linux commands interactively.

You learned some basic commands and shortcuts to help you to manage files, to complete file names, and to execute commands with fewer keystrokes.

You also learned that you can use workspaces to organize the windows of multiple applications. The Activities Overview helps to organize windows and to start applications.

You can use the graphical interface for many administrative tasks, and you can also disable the interface to preserve resources for running applications.

Chapter 3.

Getting Help from Local Documentation

Goal

Resolve problems by using information in local help systems and documentation.

Sections

- Getting Help from Manual Pages (and Guided Exercise)

Lab

- Get Help from Local Documentation

3.1. Getting Help from Manual Pages

Objectives

- Find information in local Linux system manual pages.

Introduction to the Linux Manual Pages

One source of documentation that is generally available on the local system is system manual pages or *man pages*. Software packages ship these pages to provide documentation, and you can access them from the command line by using the `man` command. The pages are stored in subdirectories of the `/usr/share/man` directory.

Man pages originated from the historical Linux Programmer's Manual, which because of its size is split into multiple sections. Each section contains information about a particular topic.

Table 7. Common Sections of the Linux Manual

Section	Content type	Description
1	User commands	Both executable and shell programs
2	System calls	Kernel routines that are invoked from user space
3	Library functions	Provided by program libraries
4	Special files	For example, device files
5	File formats	For many configuration files and structures
6	Games and screensavers	Historical section for amusing programs
7	Conventions, standards, and miscellaneous	Protocols and file systems

Section	Content type	Description
8	System administration and privileged commands	Maintenance tasks
9	Linux kernel API	Internal kernel calls

Popular system administration topics include user commands (section 1), file formats (section 5), and administrative commands (section 8). Administrators who use certain troubleshooting tools also use system calls (section 2). The remaining sections are generally for programmer reference or advanced administration.

To distinguish the same topics from different sections, man page references include the section number in parentheses after the topic. For example, `passwd(1)` describes the command to change passwords, whereas `passwd(5)` explains the `/etc/passwd` file format for storing local user accounts.

Accessing Man Pages

The `man` command searches manual sections in alphanumeric order. To access a specific man page, use the `man topic` command.

For example, the `man ssh` command shows the `ssh(1)` man page:

```

user@host:~$ man -k ssh
SSH(1)                                General Commands Manual
SSH(1)

NAME
    ssh - OpenSSH remote login client

SYNOPSIS
    ssh [-46AaCfGgKkMNnqsTtVvXxYy] [-B bind_interface] [-b
bind_address]
        [-c cipher_spec] [-D [bind_address:]port] [-E
log_file]
        ...output omitted...
    ssh [-Q query_option]

DESCRIPTION
    ssh (SSH client) is a program for logging into a remote machine and for
ex-    ecuting commands on a remote machine. It is intended to provide secure
en-    crypted communications between two untrusted hosts over an insecure
net-    work. X11 connections, arbitrary TCP ports and Unix-domain sockets
can    also be forwarded over the secure channel.

    ...output omitted...

    The options are as follows:

-4        Forces ssh to use IPv4 addresses only.

-6        Forces ssh to use IPv6 addresses only.

    ...output omitted...

```

Note

To display the man page topic from a specific section, you can use the `man section topic` command.

Navigating and Searching a Man Page

Searching efficiently for topics and navigating man pages are critical administration skills. Although you can use GUI tools to configure common system resources, using the command-line interface is more efficient. To navigate the command line effectively, you must be able to find the information that you need in the man pages.

Man pages are organized into sections. Many man pages use the same headings in the same order. Some man pages do not feature all headings, because not all headings apply to all topics.

The following table lists common headings.

Table 8. Headings

Heading	Description
NAME	Subject name, usually a command or file name, with a brief definition
SYNOPSIS	Summary of the command syntax
DESCRIPTION	Description to provide a basic understanding of the topic
OPTIONS	Explanation of the command execution options
EXAMPLES	Examples of how to use the command, function, or file
FILES	A list of files and directories that are related to the man page
SEE ALSO	Related information, typically other man page topics
BUGS	Known bugs in the software
AUTHOR	Information about who contributed to the development of the topic

The man page displays contents one screen at a time. The following table lists some navigation commands for viewing man pages.

Table 9. Navigate Man Pages

Command	Result
Spacebar	Scroll forward (down) one screen.
PageDown	Scroll forward one screen.
PageUp	Scroll backward (up) one screen.
DownArrow	Scroll forward one line.
UpArrow	Scroll backward one line.
D	Scroll forward one half-screen.
U	Scroll backward one half-screen.
/string	Search forward for <i>string</i> in the man page.
N	Repeat previous search forward in the man page.
Shift + N	Repeat previous search backward in the man page.
G	Go to the start of the man page.
Shift + G	Go to the end of the man page.
Q	Exit the man page and return to the command shell prompt.

 Important

You can use regular expressions to search in man pages. Although simple text search (such as `passwd`) works as expected, regular expressions use metacharacters (such as `$`, `*`, `.`, and `^`) for more sophisticated pattern matching. Therefore, searching with strings that include program expression metacharacters, such as `make $$$`, might yield unexpected results.

You can find more information about regular expressions and syntax in the `regex(7)` man topic.

Searching for Man Pages by Keyword

Use the `man` command `-k` option (equivalent to the `apropos` command) to search for a keyword in man page titles and descriptions. As a result, the keyword search displays a list of keyword-matching man page topics with section numbers.

For example, the following command searches for man pages that contain the word `passwd` in the title or description:

```
user@host:~$ man -k passwd
chpasswd (8)          - update group passwords in batch mode
chpasswd (8)          - update passwords in batch mode
fgetpwent_r (3)       - get passwd file entry reentrantly
getpwent_r (3)        - get passwd file entry reentrantly
...output omitted...
passwd (1)            - update user's authentication tokens
passwd (1openssl)     - OpenSSL application commands
passwd (5)            - password file
passwd2des (3)        - RFS password encryption
...output omitted...
```

The `man` command `-K` (uppercase) option searches for the keyword in the full-text page, not only in the titles and descriptions. A full-text search uses greater system resources and takes more time.

With the full-text page search, the `man` command displays the first page with a match. Press  to exit this first page, and the `man` command displays the next page.

In this example, the `man` command displays each match, and you can view or skip each one.

```
user@host:~# man -K passwd
...output omitted...
--Man-- next: cut(1p) [ view (return) | skip (Ctrl-D) | quit (Ctrl-C) ]
Ctrl-D
--Man-- next: logname(1p) [ view (return) | skip (Ctrl-D) | quit (Ctrl-C) ]
Ctrl-D
--Man-- next: sort(1p) [ view (return) | skip (Ctrl-D) | quit (Ctrl-C) ]
Ctrl-D
--Man-- next: xargs(1) [ view (return) | skip (Ctrl-D) | quit (Ctrl-C) ]
Ctrl-D
--Man-- next: chage(1) [ view (return) | skip (Ctrl-D) | quit (Ctrl-C) ]
Ctrl-C
```

Note

Keyword searches rely on an index that is generated by the `mandb(8)` command, which must be run as the root user.

The `man-db-cache-update` service automatically runs the `mandb` command when installing any package with man pages.

References

`man(1)`, `mandb(8)`, `man-pages(7)`, `less(1)`, `intro(1)`, `intro(2)`, `intro(5)`, `intro(7)`, and `intro(8)` man pages

3.2. Guided Exercise

Get Help from Manual Pages

Find information in local Linux system manual pages.

Outcomes

- Search and browse the Linux manual pages to find useful information.

Prerequisites

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
student@workstation:~$ lab start help-manual
```

Instructions

1. As the student user on the workstation machine, open the default manual page for the man command.

```
student@workstation:~$ man man
MAN(1)                Manual pager utils                MAN(1)

NAME
  man - an interface to the on-line reference manuals
...output omitted...

DESCRIPTION
  man is the system's manual pager. Each page argument given to man is
  normally the name of a program, utility or function. The manual page
  associated with each of these arguments is then found and displayed.
  A section, if provided, will direct man to look only in that section
  of the manual.
...output omitted...
```

Note the MAN(1) header in the output. The (1) section number indicates the user command section.

Press **Q** to quit the man page.

2. Open the manual pages for the passwd command and the passwd file format.

- 2.1.** Open the manual page for the `passwd` command. Make sure to specify the 1 option to request the user command manual page.

```
student@workstation:~$ man 1 passwd
PASSWD(1)                                User Commands                                PASSWD(1)

NAME
    passwd - change user password

SYNOPSIS
    passwd [options] [LOGIN]

DESCRIPTION
    The passwd command changes passwords for user accounts.
    A normal user may only change the password for their own account,
    while the superuser may change the password for any account.
    passwd also changes the account or associated password validity period.
    ...output omitted...
```

Press **Q** to quit the man page.

- 2.2.** Open the manual page for the `passwd` file format. Make sure to specify the 5 option to request the file format manual page.

```
student@workstation:~$ man 5 passwd
passwd(5)                                File Formats Manual                                passwd(5)

NAME
    passwd - password file

DESCRIPTION
    The /etc/passwd file is a text file that describes user login
    accounts for the system. It should have read permission allowed
    for all users (many utilities, like ls(1) use it to map user
    IDs to usernames), but write access only for the superuser.
    ...output omitted...
```

Press **Q** to quit the man page.

- 3.** Open the `su` command man page. Review the default behavior of the command, and what the single dash option (`-`) does.

```

student@workstation:~$ man 1 su
SU(1)                User Commands                SU(1)
NAME
    su - run a command with substitute user and group ID

SYNOPSIS
    su [options] [-] [user [argument...]]

DESCRIPTION
    su allows to run commands with a substitute user and group ID.

    When called with no user specified, su defaults to running an interactive
    shell as root.
    ...output omitted...

OPTIONS
    ...output omitted...
    -, -l, --login
        Start the shell as a login shell with an environment similar
        to a real login.
    ...output omitted...

```

The description section explains that if you omit the *user* argument, then the `su` command assumes that the user is `root`. The options section explains that if the `su` command is followed by a single dash (`-`), then it starts a child login shell. Without the dash, the `su` command creates a non-login child shell that matches the user's current environment.

Press `Q` to quit the man page.

4. Find the man page with detailed information about the `zip` command.

```

student@workstation:~$ man -k zip
...output omitted...
zipinfo (1) - list detailed information about a ZIP archive
zipnote (1) - write the comments in zipfile to stdout, edit comments and
               rename files in zipfile
zipsplit (1) - split a zipfile into smaller zipfiles

```

5. Find the man page with a list of parameters that can be passed to the kernel at boot time.

```
student@workstation:~$ man -k boot
binfmt.d (5)      - Configure additional binary formats for executables at
boot
bootctl (1)      - Control EFI firmware boot settings and manage boot loader
bootparam (7)    - introduction to boot time parameters of the Linux kernel
bootup (7)      - System bootup process
...output omitted...
```

6. Find the command to tune ext4 file-system parameters.

```
student@workstation:~$ man -k ext4
...output omitted...
resize2fs (8) - ext2/ext3/ext4 file system resizer
tune2fs (8) - adjust tunable file system parameters on ext2/ext3/ext4 file
systems
```

7. The `/usr/share/man` directory contains all man pages. Locate the binary, source, and manual pages for the `passwd` utility by using the `whereis` command. Verify that the `/usr/share/man` directory contains the man pages for the `passwd` utility.

```
student@workstation:~$ whereis passwd
passwd: /usr/bin/passwd /etc/passwd /usr/share/man/man1/passwd.1.gz
/usr/share/man/man1/passwd.1ossil.gz /usr/share/man/man5/passwd.5.gz
```

8. Open the man page for the `vi` command. Review the options for editing a specific file with the Vim text editor from the command line.

In the `vi` man page, search for an option to open the `/home/student/manual` file by using the `vi` command with the cursor at the second line of the file.

- 8.1. Open the `vi` man page.


```

student@workstation:~$ man vi
VIM(1)          General Commands Manual          VIM(1)
NAME
    vim - Vi IMproved, a programmer's text editor

SYNOPSIS
    vim [options] [file ...]
    vim [options] -
    vim [options] -t tag
    vim [options] -q [errorfile]
...output omitted...

```

8.2. In the `vi` man page, review the options for editing a specific file from the command line.

```

...output omitted...

OPTIONS
    The options may be given in any order, before or after filenames.
    Options without an argument can be combined after a single dash.

    +[num]   For the first file the cursor will be positioned on line "num".
             If "num" is missing, the cursor will be positioned on the last
             line.
...output omitted...

```

The `+[num]` option of the `vi` command opens a file with the cursor at the specified line. You must replace `[num]` with the number of the desired line.

Press `Q` to quit the man page.

8.3. Use the `vi +2 manual` command to open the `manual` file with the cursor at the second line.

```

student@workstation:~$ vi +2 manual
this is the first line
the quick brown fox just came over to greet the lazy poodle!

```

Confirm that the file opens with the cursor at the second line in the file. Enter the `:q` command to exit.

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish help-manual
```

3.3. Lab

Get Help from Local Documentation

Get information from local documentation to help you to run commands and to complete tasks.

Outcomes

- Locate relevant information for commands by searching man pages.
- Explore options for some common system administration commands.

Prerequisites

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
student@workstation:~$ lab start help-review
```

Instructions

In this activity, you create the `my_task.txt` file, and then add specific information to this file for each step.

To save the output of a command to a file, use two greater-than symbols (`>>`), which redirect the standard output to a file.

As an example, the following command redirects the output of the `ls` command, the home directory's regular file names, to the `/tmp/my-file-names` file.

```
student@workstation:~$ ls >> /tmp/my-file-names
```

You can read the contents of the example file by using the `cat` command.

```
student@workstation:~$ cat /tmp/my-file-names
```

1. Search for and open the manual page for the `hostname` command. Find the command option to display all the fully qualified domain names (FQDNs) of the machine. Then, run the `hostname` command with the option to print all FQDNs and send the output to the `my_task.txt` file.

2. Open the manual page of the `date` command. Find the option to help you to determine how many seconds have passed between January 1, 1970, and January 1, 2025. Run the command and append the output to the `my_task.txt` file.

Note

In Linux systems, *Epoch time* defines the number of seconds since 00:00:00 UTC on January 1, 1970. The `date` command includes an option to obtain the number of seconds since Epoch.

3. Find the manual page of the command that identifies the current SELinux mode of the workstation machine. Run the command and append the output to the `my_task.txt` file.
4. Open the manual page of the `man` command. Find information about how to print a manual page with PostScript. Append the command, not the output, to the `my_task.txt` file.

Evaluation

As the student user on the workstation machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

```
student@workstation:~$ lab grade help-review
```

Finish

On the workstation machine, change to the student user home directory and use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish help-review
```

Solution

Get information from local documentation to help you to run commands and to complete tasks.

Outcomes

- Locate relevant information for commands by searching man pages.
- Explore options for some common system administration commands.

Prerequisites

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
student@workstation:~$ lab start help-review
```

Instructions

In this activity, you create the `my_task.txt` file, and then add specific information to this file for each step.

To save the output of a command to a file, use two greater-than symbols (`>>`), which redirect the standard output to a file.

As an example, the following command redirects the output of the `ls` command, the home directory's regular file names, to the `/tmp/my-file-names` file.

```
student@workstation:~$ ls >> /tmp/my-file-names
```

You can read the contents of the example file by using the `cat` command.

```
student@workstation:~$ cat /tmp/my-file-names
```

1. Search for and open the manual page for the `hostname` command. Find the command option to display all the fully qualified domain names (FQDNs) of the machine. Then, run the `hostname` command with the option to print all FQDNs and send the output to the `my_task.txt` file.

- 1.1. Search for man pages that contain the `hostname` string to find the man page for the `hostname` command.

```
student@workstation:~$ man -k hostname
freehostent (3)      - get network hostnames and addresses
gethostname (2)      - get/set hostname
getipnodebyaddr (3) - get network hostnames and addresses
getipnodebyname (3) - get network hostnames and addresses
hostname (1)        - show or set the system's host name
...output omitted...
```

The `hostname (1)` man page is correct.

- 1.2. Open the man page of the `hostname` command and search for the option that displays all the FQDNs of the machine.

```
student@workstation:~$ man hostname
...output omitted...
OPTIONS
    -a, --alias
        Display the alias name of the host (if used). This option is
        deprecated and should not be used anymore.

    -A, --all-fqdns
        Displays all FQDNs of the machine. This option enumerates all
        configured network addresses on all configured network interfaces,
        and translates them to DNS domain names. Addresses that cannot
        be translated (i.e. because they do not have an appropriate
reverse
to
        IP entry) are skipped. Note that different addresses may resolve
        the same name, therefore the output may contain duplicate entries.
        Do not make any assumptions about the order of the output.

...output omitted...
```

The `-A` option of the `hostname` command displays the required information.

Press `Q` to quit the man page.

- 1.3. Use redirection with the `hostname` command `-A` option to add the machine's hostnames to the `my_task.txt` file.

```
student@workstation:~$ hostname -A >> my_task.txt
```

- 1.4. Verify that the `my_task.txt` file contains the required information.

```
student@workstation:~$ cat my_task.txt
workstation.lab.example.com workstation
```

2. Open the manual page of the date command. Find the option to help you to determine how many seconds have passed between January 1, 1970, and January 1, 2025. Run the command and append the output to the `my_task.txt` file.

Note

In Linux systems, *Epoch time* defines the number of seconds since 00:00:00 UTC on January 1, 1970. The date command includes an option to obtain the number of seconds since Epoch.

- 2.1. Browse the manual page for the date command to find the appropriate option.

```
student@workstation:~$ man date
...output omitted...
    -d, --date=STRING
                display time described by STRING, not 'now'
...output omitted...
    %s          seconds since the Epoch (1970-01-01 00:00 UTC)
...output omitted...

EXAMPLES
    Convert seconds since the Epoch (1970-01-01 UTC) to a date

    $ date --date='@2147483647'
...output omitted...
```

Press **Q** to quit the man page.

- 2.2. Use the date command with the `-d` and `%s` options to get the number of seconds between the requested dates. Append the output of the command to the `my_task.txt` file.

```
student@workstation:~$ date -d "Jan 1 2025" +%s >> my_task.txt
```

- 2.3. Verify that the `my_task.txt` file contains the required information.

```
student@workstation:~$ cat my_task.txt
workstation.lab.example.com workstation
1735689600
```

2.4. You can verify your result by converting the number of seconds that you retrieved into a date.

```
student@workstation:~$ date --date='@1735689600'
Wed Jan  1 00:00:00 UTC 2025
```

3. Find the manual page of the command that identifies the current SELinux mode of the workstation machine. Run the command and append the output to the `my_task.txt` file.

3.1. Find the command that displays the current SELinux mode.

```
student@workstation:~$ man -k selinux
...output omitted...
getenforce (8)          - get the current mode of SELinux
...output omitted...
```

The `getenforce` command shows the current mode of SELinux in the machine.

3.2. Use the `getenforce` command to get the current SELinux mode and add it to the `my_tasks.txt` file.

```
student@workstation:~$ getenforce >> my_task.txt
```

3.3. Verify that the `my_task.txt` file contains the required information.

```
student@workstation:~$ cat my_task.txt
workstation.lab.example.com workstation
1735689600
Enforcing
```

4. Open the manual page of the `man` command. Find information about how to print a manual page with PostScript. Append the command, not the output, to the `my_task.txt` file.

4.1. Use the `man man` command to determine how to prepare a manual page for printing.


```

student@workstation:~$ man man
...output omitted...
    man -t bash | lpr -Pps
        Format the manual page for bash into the default troff or groff
format
        and pipe it to the printer named ps. The default output for groff
        is usually PostScript. man --help should advise as to which
processor
        is bound to the -t option.
...output omitted...

```

Press **Q** to quit the man page.

- 4.2.** Use the echo command to add the man command with the appropriate options to the my_tasks.txt file.

```

student@workstation:~$ echo "man -t bash | lpr -Pps" >> my_task.txt

```

- 4.3.** Verify that the my_task.txt file contains the required information.

```

student@workstation:~$ cat my_task.txt
workstation.lab.example.com workstation
1735689600
Enforcing
man -t bash | lpr -Pps

```

Evaluation

As the student user on the workstation machine, use the lab command to grade your work. Correct any reported failures and rerun the command until successful.

```

student@workstation:~$ lab grade help-review

```

Finish

On the workstation machine, change to the student user home directory and use the lab command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```

student@workstation:~$ lab finish help-review

```

3.4. Summary

In this lesson, you learned to use the `man` command to display information about components of a Linux system, such as files, commands, and functions.

To refer to a man page, the name of the page is followed by its section number in parentheses. Popular sections include user commands (1), file formats (5), administrative commands (8), and system calls (2).

You can search for man pages by keyword, and you can navigate a man page by using various key presses and keyboard combinations.

Chapter 4.

Registering Systems for Red Hat Support

Goal

Register a system by using your Red Hat account to get support services and software that Red Hat provides.

Sections

- Registering Systems for Red Hat Support (and Quiz)

4.1. Registering Systems for Red Hat Support

Objectives

- Register a system by using your Red Hat account.

The Benefits of a Red Hat Subscription

Red Hat provides access to software repositories through the Red Hat Content Delivery Network (CDN) in a subscription-based model. Software repositories provide regularly updated packages, including security patches, bug fixes, and enhancements.

Customers with active subscriptions can access Red Hat content repositories to install new software packages and to stay up-to-date with the latest updates. In addition to software updates, an active subscription provides customers with technical resources such as Customer Portal Labs, Knowledgebase solutions, documentation, Red Hat Insights, and support expertise.

In 2024, Red Hat transitioned to a subscription model named Simple Content Access (SCA). This access-based subscription model enforces the existence of a valid subscription on a per-organization basis, eliminating the need to attach subscriptions on a per-system level. The new model reduces complexity in subscription management and tooling. As part of this migration, customer-facing subscription services moved from the Red Hat Customer Portal (<https://access.redhat.com>) to the Red Hat Hybrid Cloud Console (<https://console.redhat.com>). The Hybrid Cloud Console provides a unified experience for systems management and closer integration with Red Hat Insights.

Registering a RHEL System

You can use different options to register a system to the Red Hat subscription service. For example, you might use the graphical RHEL web console or command-line tools such as remote host configuration (`rhc`) or subscription manager (`subscription-manager`).

You can use these tools to perform the following actions with your systems:

- *Register* a system to associate it with a Red Hat account that has an active subscription. During registration, the system receives unique identity certificates for authentication to the Red Hat subscriptions service.
- *Enable additional repositories* to provide access to software packages as needed. Default repositories are automatically enabled for installed products, which provides immediate access to content.

- Review and track available or consumed entitlements on the **Subscription Inventory** page of the Hybrid Cloud Console.
- Optionally, register with Red Hat Insights for predictive analytics and proactive issue resolution.

Registering a System with the `rhc` Tool

The `rhc` command-line tool is available for registering RHEL 8.8 and later systems. By default, the `rhc` tool enables three features: content, analytics and remote-management. These features allow your system to access content repositories and enables the Red Hat Insights analytics and remediation services with a single command.

To use the `rhc` tool, you must authenticate by using a Red Hat account that has a valid subscription. Additionally, you must have root access on the system to be able to register it.

```
root@host:~# rhc connect
Connecting host.example.com to Red Hat.
This might take a few seconds.

Features preferences: [✓]content, [✓]analytics, [✓]remote-management

Username: red-hat-username
Password: red-hat-password

[✓] Connected to Red Hat Subscription Management
[✓] Content ... Red Hat repository file generated
[✓] Analytics ... Connected to Red Hat Insights
[✓] Remote Management ... Activated the yggdrasil service

Successfully connected to Red Hat!

Manage your connected systems: https://red.ht/connector
```

The `rhc` command enables all three of the main features by default. If you want to enable only access to content repositories without registering to Red Hat Insights, then you can disable the analytics and remote-management features during registration. You can disable these features by using the `rhc connect` command with the `--disable-feature` option.

```

root@host:~# rhc connect --disable-feature=analytics,remote-management
...output omitted...
Features preferences: [✓]content, [ ]analytics, [ ]remote-management

Username: red-hat-username
Password: red-hat-password

[✓] Connected to Red Hat Subscription Management
[✓] Content ... Red Hat repository file generated
[ ] Analytics ... Connecting to Red Hat Insights disabled
[ ] Management .... Starting yggdrasil service disabled (required feature
"analytics" is disabled)
...output omitted...

```

When you go to the Hybrid Cloud Console to check this particular system, expect the dashboard to display the `Your insights-client is not reporting` warning. The inventory information for this system is limited because of the lack of data collection.

Using an Activation Key with the `rhc` Tool

To register a system securely in a more automated way, use the `rhc connect` command along with an activation key and your organization identifier (numeric identifier). This method is more secure than providing personal usernames and passwords.

An activation key is a pre-shared authentication token that you can create and manage in the Hybrid Cloud Console. On the dashboard, click **Insights for RHEL**, and then select **Inventory > System Configuration > Activation Keys**.

When you create an activation key, you have the option to associate the key with the latest RHEL version or to lock the key to a specific earlier release. You can optionally also define the *system purpose*, which is a tagging mechanism to help identify hosts in the inventory later. You can also define additional repositories for the activation key, which can further simplify configuring a system during registration.

The following example shows how to use an activation key with the `rhc` command.

```
root@host:~# rhc connect --activation-key=<activation_key_name> \
--organization=<organization_ID>
...output omitted...
Successfully connected to Red Hat!

Manage your connected systems: https://red.ht/connector
```

Unregistering a System with the rhc Tool

To unregister your system, use the `rhc disconnect` command:

```
root@host:~# rhc disconnect
Disconnecting host.example.com from Red Hat.
This might take a few seconds.

[✓] Deactivated the yggdrasil service
[✓] Disconnected from Red Hat Insights
[✓] Disconnected from Red Hat Subscription Management

Manage your connected systems: https://red.ht/connector
```

Registering a System with the subscription-manager Tool

The `rhc` command is the preferred tool to register systems, but the `rhc` command is not available for RHEL 8.7 or earlier releases. As an alternative, you can use the `subscription-manager` command to register your systems. The `subscription-manager` tool is also helpful in cases where you want to enable repository access without connecting to Red Hat Insights.

In RHEL 10, several `subscription-manager` modules that were related to attaching subscriptions on a per-system basis have been removed. However, the `subscription-manager` tool is not superseded or deprecated in RHEL 10, and continues to function similarly to its earlier versions.

Because of the transition to Simple Content Access, registering a system by using the `subscription-manager` tool is accomplished in a single step:

```

root@host:~# subscription-manager register
Registering to: subscription.rhsm.redhat.com:443/subscription
Username: red-hat-username
Password: red-hat-password
The system has been registered with ID: 686c73e1-3780...
The registered system name is: host.example.com

```

If you have created an activation key in the Hybrid Cloud Console, then you can use it to register your system without using your username and password:

```

root@host:~# subscription-manager register \
--activationkey <activation_key_name> --org <organization_ID>
The system has been registered with ID: 361d5774-76e0...
The registered system name is: host.example.com

```

To verify the registration status of your system, you can use the `subscription-manager status` command:

```

root@host:~# subscription-manager status
-----
System Status Details
-----
Overall Status: Registered

```

To verify subscription usage, you must visit the Hybrid Cloud Console because the `subscription-manager` tool no longer reports entitlement usage on a per-system basis.

The `subscription-manager` command cannot connect your system to Red Hat Insights. After registration, you can use the `insights-client` command to enable Red Hat Insights:

```

root@host:~# insights-client --register
Successfully registered host host.example.com
Automatic scheduling for Insights has been enabled.
Starting to collect Insights data for host.example.com
Writing RHSM facts to /etc/rhsm/facts/insights-client.facts ...
Uploading Insights data.
Successfully uploaded report from host.example.com to account 606...
View the Red Hat Insights console at https://console.redhat.com/insights/

```

Setting System Purpose Attributes

System purpose is an optional tagging mechanism that helps to filter your inventory. You can set the following attributes and values by using the `subscription-manager` command:

Role

- Red Hat Enterprise Linux Server
- Red Hat Enterprise Linux Workstation
- Red Hat Enterprise Linux Compute Node

Service-Level Agreement

- Premium
- Standard
- Self-Support

Usage

- Production
- Development/Test
- Disaster Recovery

Use the `subscription-manager syspurpose` command to set relevant system purpose attributes:

```
root@host:~# subscription-manager syspurpose role \
--set 'Red Hat Enterprise Linux Workstation'
role set to "Red Hat Enterprise Linux Workstation".

root@host:~# subscription-manager syspurpose service-level \
--set 'Premium'
service_level_agreement set to "Premium".

root@host:~# subscription-manager syspurpose usage \
--set 'Development/Test'
usage set to "Development/Test".
```

The `syspurpose` subcommand also supports the `--list` option to help list the available values for each attribute. For example, the following command checks the list of available values for the `usage` attribute:

```
root@host:~# subscription-manager syspurpose usage --list
-----
                Available usage
-----
- Development/Test
```

Unregistering a System with the subscription-manager Tool

With the subscription-manager tool, removing a registration is a one-step process:

```
root@host:~# subscription-manager unregister
Unregistering from: subscription.rhsm.redhat.com:443/subscription
System has been unregistered.
```

Registering a System with the RHEL Web Console

If you prefer a graphical user interface to command-line tools, then the RHEL web console is a good alternative. To register a system with the web console, you must log in as a privileged user. Click

Subscriptions to see your current subscription status and the list of installed products.

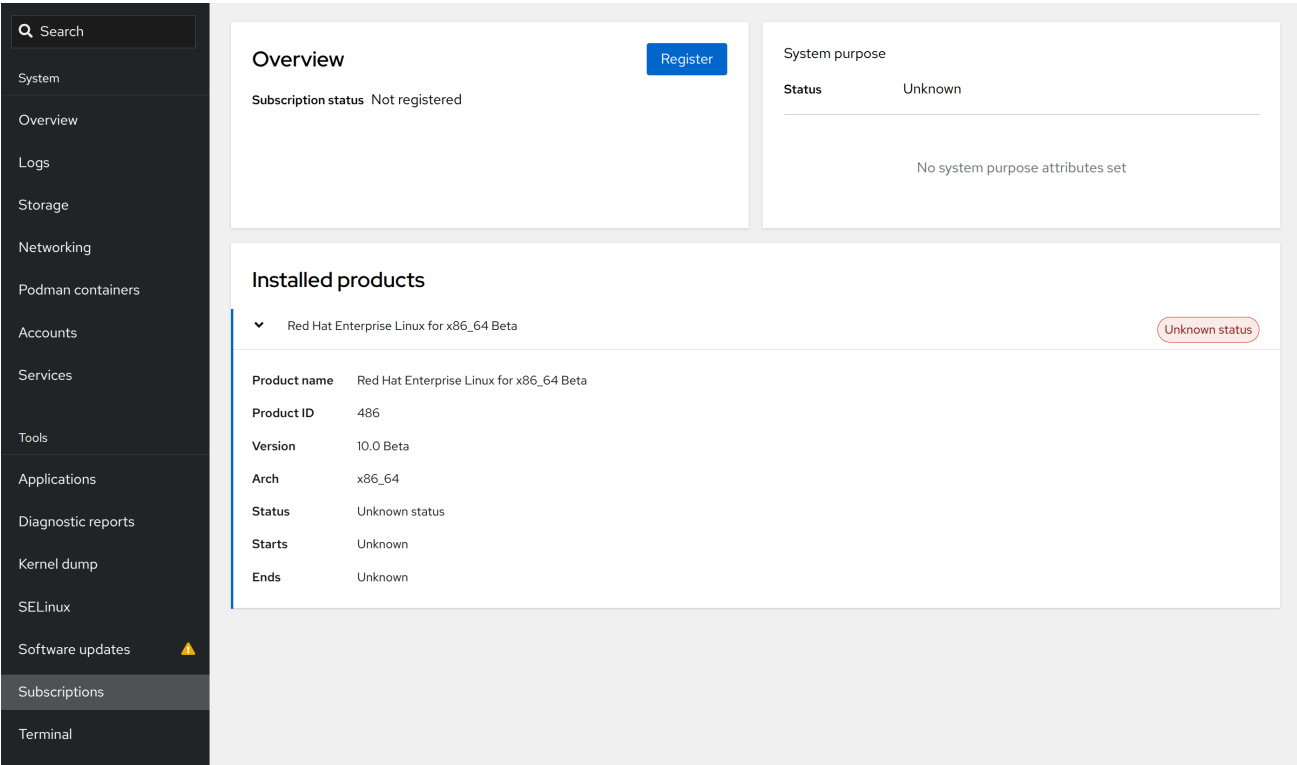


Figure 9. Web console overview

The start the registration process, click **Register** in the **Overview** section.

Register System

URL

☐ Use proxy server

Method ☒ Account ☐ Activation key

Username

Password

Organization

Insights ☒ Connect this system to [Red Hat Insights](#) 

Figure 10. Web console registration

In the **Register System** dialog box, provide the username and the password for your Red Hat account, and then click **Register** to register the system. Alternatively, instead of providing a username and password, you can provide an activation key along with an organization identifier. By default, registering to Red Hat Insights is preselected.

Registering a System During Installation

If you are deploying RHEL 10 on a bare-metal or virtual machine, then the Anaconda installer can register your target system before package installation begins. In the interactive Anaconda interface, click **Connect to Red Hat** to access the registration dialog.

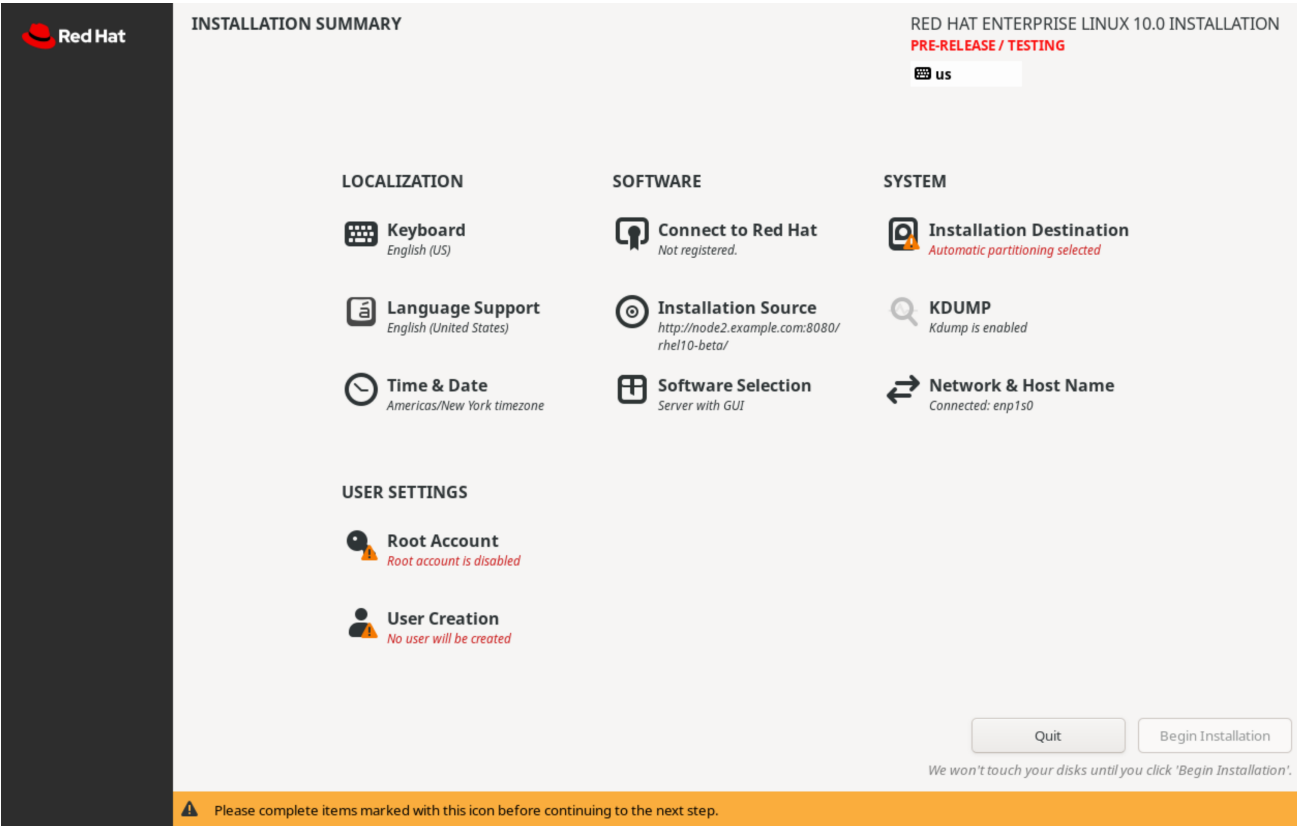


Figure 11. Anaconda installer summary

You can authenticate by using a username and password, or by using an activation key. Optionally, you can also set the system purpose and register to Red Hat Insights.

CONNECT TO RED HAT

RED HAT ENTERPRISE LINUX 10.0 INSTALLATION
PRE-RELEASE / TESTING

Done

us

Authentication ☒ Account ☐ Activation Key

User name

Password

Purpose ☐ Set System Purpose

Insights ☒ Connect to Red Hat Insights

Options

Not registered.

Register

Figure 12. Anaconda installer registration

If you wish to register your system as part of an automated Kickstart installation, then you can use the `rhsm` Kickstart command. Note that the `rhsm` Kickstart command requires an activation key and organization identifier.

Inspecting Identity Certificates

A registered system stores several certificates under the `/etc/pki` directory.

The `/etc/pki/product-default` certificates

Identify the Red Hat products that are installed on the system.

The `/etc/pki/consumer` certificate and key pair

Identifies the system.

The `/etc/pki/entitlement` certificate and key pair

Authenticates the system to repositories.

You can use the `rpm --cat-cert` command to inspect the certificates in more detail:

```

root@host:~# rct cat-cert /etc/pki/product-default/486.pem
-----
Product Certificate
-----

Certificate:
    Path: /etc/pki/product-default/486.pem
    Version: 1.0
    Serial: 12750047592154747372
    Start Date: 2023-07-18 08:19:16+00:00
    End Date: 2043-07-18 08:19:16+00:00

Subject:
    CN: Red Hat Product ID [cd0977e6-19f8-49a5-8743-288263a604d5]
...output omitted...

```

References

rhc(8), subscription-manager(8), and rct(8) man pages

For further information about registration, refer to *Getting Started with RHEL System Registration* at https://docs.redhat.com/en/documentation/subscription_central/1-latest/html-single/getting_started_with_rhel_system_registration/index

For further information about activation keys, refer to *Getting Started with Activation Keys on the Hybrid Cloud Console* at https://docs.redhat.com/en/documentation/subscription_central/1-latest/html-single/getting_started_with_activation_keys_on_the_hybrid_cloud_console/index

For further information about viewing your subscriptions, refer to *Viewing and Managing your Subscription Inventory on the Hybrid Cloud Console* at https://docs.redhat.com/en/documentation/subscription_central/1-latest/html-single/viewing_and_managing_your_subscription_inventory_on_the_hybrid_cloud_console/index

4.2. Quiz

Registering Systems for Red Hat Support

Choose the correct answers to the following questions:

1. What is the primary purpose of registering a system with a Red Hat account?
 - ☐ a. To receive free Red Hat training courses.
 - ☐ b. To enable system backups to Red Hat servers.
 - ☐ c. To access Red Hat software updates, predictive analysis, and support services.
 - ☐ d. To participate in Red Hat beta testing programs.
2. You manage multiple RHEL systems and want to streamline the registration process by not using personal credentials each time. You have access to the Red Hat Hybrid Cloud Console. How can you securely register these systems?
 - ☐ a. Use the `insights-client` command with an activation key.
 - ☐ b. Use the `rhc connect` command with a valid key pair.
 - ☐ c. Use the `rhc connect` command with an activation key.
 - ☐ d. Use the `rhn_register` command with an x509 certificate.
3. How does Simple Content Access simplify the registration process?
 - ☐ a. It removes the overhead of attaching subscriptions on a per-system basis.
 - ☐ b. It removes the requirement for a Red Hat account.
 - ☐ c. It introduces activation keys as a new concept.
 - ☐ d. It eliminates the need for system updates.
4. You registered a RHEL system by using the `subscription-manager` tool. Now you want to enable Red Hat Insights for this system. Which next step should you take?
 - ☐ a. Use the `insights-client` command to register with Red Hat Insights.
 - ☐ b. Use the `subscription-manager status` command to enable Red Hat Insights.
 - ☐ c. Use the `rhc connect` command to enable Red Hat Insights.
 - ☐ d. Use the `subscription-manager register` command again with additional options.

5. You are tasked with unregistering a RHEL system from Red Hat's subscription services. Which command should you use?
- ☐ a. `insights-client unregister`
 - ☐ b. `rhc unregister`
 - ☐ c. `rhc disconnect`
 - ☐ d. `subscription-manager register`
6. Which service-level agreement can be set by using the `subscription-manager` command?
- ☐ a. Premium
 - ☐ b. Enterprise
 - ☐ c. Basic
 - ☐ d. Advanced
7. You are responsible for inspecting the identity certificates on a registered Red Hat system. You must verify the certificates that identify installed Red Hat products. Which directory contains the certificates for installed Red Hat products?
- ☐ a. `/etc/pki/entitlement`
 - ☐ b. `/etc/pki/subscription`
 - ☐ c. `/etc/pki/product-default`
 - ☐ d. `/etc/pki/consumer`

Solution

1. What is the primary purpose of registering a system with a Red Hat account?
 - ☐ a. To receive free Red Hat training courses.
 - ☐ b. To enable system backups to Red Hat servers.
 - ☒ c. **To access Red Hat software updates, predictive analysis, and support services.**
 - ☐ d. To participate in Red Hat beta testing programs.
2. You manage multiple RHEL systems and want to streamline the registration process by not using personal credentials each time. You have access to the Red Hat Hybrid Cloud Console. How can you securely register these systems?
 - ☐ a. Use the `insights-client` command with an activation key.
 - ☐ b. Use the `rhc connect` command with a valid key pair.
 - ☒ c. **Use the `rhc connect` command with an activation key.**
 - ☐ d. Use the `rhn_register` command with an x509 certificate.
3. How does Simple Content Access simplify the registration process?
 - ☒ a. **It removes the overhead of attaching subscriptions on a per-system basis.**
 - ☐ b. It removes the requirement for a Red Hat account.
 - ☐ c. It introduces activation keys as a new concept.
 - ☐ d. It eliminates the need for system updates.
4. You registered a RHEL system by using the `subscription-manager` tool. Now you want to enable Red Hat Insights for this system. Which next step should you take?
 - ☒ a. **Use the `insights-client` command to register with Red Hat Insights.**
 - ☐ b. Use the `subscription-manager status` command to enable Red Hat Insights.
 - ☐ c. Use the `rhc connect` command to enable Red Hat Insights.
 - ☐ d. Use the `subscription-manager register` command again with additional options.

5. You are tasked with unregistering a RHEL system from Red Hat's subscription services. Which command should you use?
- ☐ a. `insights-client unregister`
 - ☐ b. `rhc unregister`
 - ☒ c. `rhc disconnect`
 - ☐ d. `subscription-manager register`
6. Which service-level agreement can be set by using the `subscription-manager` command?
- ☒ a. **Premium**
 - ☐ b. Enterprise
 - ☐ c. Basic
 - ☐ d. Advanced
7. You are responsible for inspecting the identity certificates on a registered Red Hat system. You must verify the certificates that identify installed Red Hat products. Which directory contains the certificates for installed Red Hat products?
- ☐ a. `/etc/pki/entitlement`
 - ☐ b. `/etc/pki/subscription`
 - ☒ c. `/etc/pki/product-default`
 - ☐ d. `/etc/pki/consumer`

4.3. Summary

In this lesson, you learned about the Red Hat subscription model, which provides access to content on the Red Hat Content Delivery Network (CDN) such as software updates, security patches, and technical resources. The Simple Content Access model streamlines subscription management at the organizational level and centralizes services on the Hybrid Cloud Console.

You can use the remote host configuration (`rhc`) or the subscription manager (`subscription-manager`) command-line tools to register your systems. The `rhc` and `subscription-manager` command-line tools register systems for content access; the `rhc` tool can additionally activate Red Hat Insights.

Alternatively, you can use the RHEL web console to register your system.

Registered systems receive certificates for identification and authentication.

Chapter 5.

Getting AI-assisted Help with Red Hat Enterprise Linux Lightspeed

Goal

Get AI-driven guidance to efficiently manage a RHEL server by using Red Hat Enterprise Linux Lightspeed.

Sections

- Asking Questions and Evaluating Answers from the Command-line Assistant (and Guided Exercise)

5.1. Asking Questions and Evaluating Answers from the Command-line Assistant

Objectives

- Troubleshoot problems by phrasing effective questions and evaluating the answers that RHEL Lightspeed provides.

Red Hat Enterprise Linux Lightspeed

Red Hat Enterprise Linux Lightspeed combines decades of Red Hat Enterprise Linux expertise with AI technologies to proactively inform and simplify how both newer and experienced Linux professionals build, deploy, and manage Red Hat Enterprise Linux (RHEL).

Components that are powered by RHEL Lightspeed enable customers to focus more of their time and resources on innovation and less on maintenance. RHEL Lightspeed technologies are intended to provide proactive guidance, knowledge, and recommendations, thus making RHEL more intuitive and easy to use. RHEL Lightspeed technologies are included in your Red Hat Enterprise Linux subscription.

Two components powered by RHEL Lightspeed are available simultaneously with the launch of RHEL 10:

Command-line assistant

The new Red Hat Enterprise Linux (RHEL) command-line assistant, powered by RHEL Lightspeed, lets you ask an English-language question directly in the terminal to get instant answers, recommendations, and guidance, backed by decades of RHEL knowledge and expertise. The command-line assistant simplifies complex tasks for newer Linux administrators, and boosts efficiency for seasoned admins, bringing AI-powered help straight to the command line in RHEL 10.

Red Hat Insights image builder package recommendations

New package recommendations in Red Hat Insights image builder help you to make more informed decisions early in the build process. As you select packages, the tool suggests additional recommendations to help you minimize the risk of missing key components so that you can create more complete, functional images from the start.

RHEL Lightspeed is designed for system administrators, platform engineers, and developers who manage RHEL systems, regardless of their level of Linux experience. RHEL Lightspeed technologies

are intended to improve productivity, enhance the user experience, and streamline system administration.

This lesson covers only the first of these two components, the command-line assistant powered by RHEL Lightspeed.

Command-line Assistant Powered by RHEL Lightspeed

The command-line assistant powered by RHEL Lightspeed is an optional generative AI tool that is designed to help you to be more productive. It provides contextual help by interpreting your natural language questions and offering AI-generated answers, suggestions, commands, examples, or guidance.

Warning

The command-line assistant is a generative AI tool. Therefore, while the output that it provides is generally beneficial, some of its suggestions might be incomplete or inaccurate.

Always review AI-generated content prior to use.

The command-line assistant can answer RHEL-related questions, assist with troubleshooting, assist with interpreting log entries, and help with many other tasks. By reducing the time spent searching through external documentation, the command-line assistant can help you to resolve issues and to complete tasks more efficiently.

Important

The command-line assistant cannot access specific information about the system that it runs on.

For example, the command-line assistant cannot provide answers about the available memory on the system that it runs on. Instead, the command-line assistant responds with information about a command that you can run to determine the system's available memory.

The command-line assistant is initially supported as a connected experience only, although Red Hat is investigating ways to support disconnected on-premise offerings. Therefore, the command-line assistant currently requires internet access to function correctly.

The Large Language Model

RHEL Lightspeed uses the IBM watsonx AI API large language model (LLM). The model is hosted externally and is provided as SaaS (Software-as-a-Service) for all registered RHEL systems. The LLM model is hosted on the watsonx AI platform, and uses the watsonx LLM IBM Granite model.

The LLM that powers the command-line assistant is trained on various Red Hat resources, including Knowledge Center Service (KCS) articles, RHEL documentation, and more.

Installing the Command-line Assistant

To use the command-line assistant, you must be running a subscribed system. You can use the subscription-manager tool to manage system subscriptions. RHEL 10.0 and RHEL 9.6 and later versions are supported.

The first time that you use the command-line assistant, install the command-line-assistant package:

```
root@host:~# dnf install command-line-assistant
...output omitted...
Is this ok [y/N]: y
...output omitted...
Complete!
```

This package provides the c command, which is the primary user interface for the command-line assistant. Run the c command to verify that the installation works:

```
user@host:~$ c chat "How do I list running services?"
```

```
*_* Asking RHEL Lightspeed
```

This feature uses AI technology. Do not include any personal information or other sensitive information in your input. Interactions may be used to improve Red Hat's products or services.

To list all running services on a Red Hat Enterprise Linux system, you can use the `systemctl` command with the `--type=service` option. Here's how you can do it:

```
[bash] Snippet _____
systemctl list-units --type=service --state=running
_____
```

This command will display a list of all service units that are currently running.

For more detailed information about system services, you can use the `systemctl status` command followed by the name of the service unit. For example, to check the status of the `gdm` service, you would run:

```
[bash] Snippet _____
systemctl status gdm.service
_____
```

This will provide you with detailed information about the service, including its current state, whether it's enabled to start at boot, and more.

Remember to replace `gdm.service` with the name of the service unit you're interested in. You can find the names of service units by running `systemctl list-units --type=service`.

Always review AI-generated content prior to use.

Note

Your output might differ from the examples.

Troubleshooting with the Command-line Assistant

The command-line assistant can be a powerful tool to help you to troubleshoot problems. Although the tool is useful for someone with limited Linux expertise, it is also helpful for someone who can quickly understand and act on the responses that the command-line assistant provides.

To use the command-line assistant effectively, begin by asking clear, well-structured questions. This approach is especially important for troubleshooting problems, because clear and precise phrasing

generates more relevant responses, which helps you to identify and resolve problems more quickly.

The simplest way to interact with the command-line assistant is by asking a direct, natural-language question. For example, to ask a question about how to check the status of the SSH daemon on your system, you can enter the following text:

```
user@host:~$ c chat "How do I check the status of the sshd service?"
```

Interactive Mode

For more complex scenarios, you might engage in a turn-based conversation. This approach can provide more context for the command-line assistant to use when answering your questions. In such cases, you can launch the assistant in interactive mode:

```
user@host:~$ c chat --interactive
Welcome to the interactive mode for command line assistant! To exit, press Ctrl
+ C or type '.exit'.
>>>
```

Analyzing Command Output

Context is vital in troubleshooting, and the command-line assistant becomes far more useful when it has specific data to analyze. One of the most powerful features is the ability to pipe command output directly into the command-line assistant.

For instance, if you are reviewing logs from Apache HTTP Server and want help diagnosing recent errors, then you could run the following command:

```
user@host:~$ sudo journalctl -u httpd.service --since "1 hour ago" | \
c chat "What could be causing errors in these logs?"
```

Another common and complex scenario is troubleshooting a suspected memory leak. You can pipe the output of tools like `top`, `ps`, or even `smem` into the command-line assistant and ask it to interpret the results.

For example, you can provide the assistant with the current memory usage of running processes, so that it can identify unusually high or increasing memory consumption.

```
user@host:~$ ps aux --sort=-%mem | \
head -n 20 | \
c chat "Is there a possible memory leak in one of these processes?"
```

Analyzing File Content

You can also send file content to the command-line assistant for analysis. This functionality is particularly helpful when reviewing configuration files or long logs, for example:

```
user@host:~$ cat /var/log/messages | \
c chat "Look for critical errors in this log file."
```

Important

The command-line assistant does not access your files or data unless you provide your files or data.

Analyzing Data

You can also attach data directly by using the `--attachment` or `-a` flags. For example, to analyze the system boot log, you might run the following command:

```
user@host:~$ c chat -a /var/log/boot.log "Why did the last boot take so long?"
```

References

c(1) man page

For more information, refer to the *Interacting with the Command-line Assistant Powered By RHEL Lightspeed* guide at https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html-single/interacting_with_the_command-line_assistant_powered_by_rhel_lightspeed/index

5.2. Guided Exercise

Ask Questions and Evaluate Answers from the Command-line Assistant

Troubleshoot problems by phrasing effective questions and evaluating the answers that RHEL Lightspeed provides.

Outcomes

- Install the command-line assistant
- Ask for a recommendation
- Ask for an explanation
- Troubleshoot and resolve an issue

Prerequisites

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
student@workstation:~$ lab start lightspeed-assistant
```

Instructions

1. Log in to the servera machine as the student user.

```
student@workstation:~$ ssh servera
...output omitted...
[student@servera ~]$
```

2. Install the command-line assistant on the servera machine.

- 2.1. Install the command-line-assistant package on the servera machine. Use student as the sudo password.

```
[student@servera ~]# sudo dnf install command-line-assistant
[sudo] password for student: student
...output omitted...
Is this ok [y/N]: y
...output omitted...
Complete!
```

3. Ask the command-line assistant for tools that can list processes that are running under the student user.

 Note

The AI-generated output might differ every time that you run the command-line assistant.

```
[student@servera ~]$ c chat "Provide a list of tools that can be used to view the processes running under the student user"
```

```
*_+ Asking RHEL Lightspeed
```

To view the processes running under the "student" user, you can use the following tools in Red Hat Enterprise Linux (RHEL):

1. **top**: This command provides a dynamic real-time view of a running system. It displays system summary information as well as a list of processes sorted by their CPU usage. To filter processes for a specific user, use the **-u** option followed by the username:

```
top -u student
```

2. **ps**: The **ps** command allows you to list currently running processes. To list processes for a specific user, you can use the **-u** option:

```
ps -u student
```

To see more detailed information, consider using **ps aux** or **ps -ef** (for more comprehensive details).

3. **pgrep**: This command is useful for searching for processes based on their name. To find all processes owned by the "student" user, use:

```
pgrep -u student
```

4. **pidof**: Similar to **pgrep**, **pidof** finds the process ID (PID) of a specified process by name. However, it only returns one PID per process name. Use it with caution as it might not list all instances of a process if multiple are running:

```
pidof -u student
```

5. **SystemTap** (as described in the provided context): While primarily used for

more complex monitoring and tracing, SystemTap can also be used to list processes associated with a specific user. The provided context describes how to use `strace.stp`` to monitor system calls of a process, but you can adapt it to list running processes for a user by modifying the script.

Remember, for all these commands, you need appropriate permissions to view processes of other users. Typically, this requires root or sudo privileges. Always ensure you follow best practices for system security and user privacy.

Always review AI-generated content prior to use.

4. Following the instructions that the command-line assistant provides, run the `top` command to list the processes run by the student user. Pipe the `top` command output to the command-line assistant, and ask the command-line assistant to explain the output.

```
[student@servera ~]$ top -b -n 1 -u student | \
c chat "Can you explain how to read the output of the top command for the
user student?"
*++ Asking RHEL Lightspeed
```

Sure, I'd be happy to explain how to read the output of the top command.

The top command provides a dynamic real-time view of the running system. Here's a breakdown of the output:

1. Header Information

- top - 13:07:57 up 1:08, 2 users, load average: 0.03, 0.01, 0.00: This line shows the current time, system uptime, number of logged-in users, and the load average over the last 1, 5, and 15 minutes respectively.

- Tasks: 136 total, 2 running, 134 sleeping, 0 stopped, 0 zombie: This line displays the total number of tasks (processes), the number of tasks currently running, the number of tasks in uninterruptible sleep (usually waiting for I/O), stopped tasks, and zombie tasks.

- %Cpu(s): 39.1 us, 17.4 sy, 0.0 ni, 43.5 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st: This line shows the CPU usage breakdown. us (user space), sy (system space), ni (nice), id (idle), wa (waiting for I/O), hi (hardware interrupt), si (software interrupt), and st (steal time).

2. Memory Information

- MiB Mem : 1705.2 total, 1032.8 free, 454.4 used, 378.4 buff/cache: This line shows the total, free, used, and buffer/cache memory in Megabytes (MiB).

- MiB Swap: 0.0 total, 0.0 free, 0.0 used. 1250.9 avail Mem: This line shows the total, free, used swap space, and the available memory including swap.

3. PID Information

- The rest of the output lists the Process ID (PID), username, CPU usage (%CPU), nice value (NI), virtual memory usage (VIRT), resident memory usage (RES), shared memory (SHR), status (S, R, D for sleeping, running, and zombie respectively), CPU usage (%CPU), memory usage (%MEM), time (TIME+), and the command name (COMMAND).

For example, the first line under PID information 5708 student 20 0 272848 37744 15708 R 90.9 2.2 0:00.21 c indicates:

- PID: 5708
- User: student
- Priority (nice value): 20

```

- CPU usage: 20 (nice value) + 0 (priority) = 20
- Virtual memory usage: 272848 KiB
- Resident memory usage: 37744 KiB
- Shared memory: 15708 KiB
- Status: R (running)
- %CPU: 90.9%
- %MEM: 2.2%
- Time: 0:00.21 (CPU time used by the process)
- Command: c

```

Remember, the `top` command updates this information every 3 seconds by default. You can press `q` to quit the `top` command.

For more detailed information about a specific process, you can use commands like `ps`, `top -p <PID>`, or `htop` (a more interactive version of `top`).

Always review AI-generated content prior to use.

5. Troubleshoot and resolve an issue with the web server (the `httpd` service) by using the command-line assistant.

5.1. Use the `sudo systemctl start httpd` command to try to start the web server. Use `student` as the password.

```

[student@servera ~]$ sudo systemctl start httpd
[sudo] password for student: student
Job for httpd.service failed because the control process exited with error code.
See "systemctl status httpd.service" and "journalctl -xeu httpd.service" for
details.

```

5.2. Ask the command-line assistant to help interpret the failure messages so that you can troubleshoot why the `httpd` service failed to start.


```
[student@servera ~]$ echo "student" | sudo -S systemctl status httpd.service | \
c chat "Check the logs, help me to fix the issue"
+_- Asking RHEL Lightspeed
```

The logs indicate that the Apache HTTP Server, httpd, failed to start due to a syntax error in the configuration file `/etc/httpd/conf/httpd.conf`. Specifically, there are two issues:

1. An unset environment variable `OPTIONS`.
2. An invalid command `Lislisten` (likely a typo) on line 47 of `httpd.conf`.

To resolve this issue, follow these steps:

1. Open the configuration file using a text editor. For example, you can use `nano`:

```
bash
sudo nano /etc/httpd/conf/httpd.conf
```

2. Navigate to line 47 and correct the invalid command. It appears that `Lislisten` is a typo for `Listen`. Replace it with the correct command:

```
apache
Listen 80
```

3. Check if the `OPTIONS` environment variable is set correctly. Look for a line similar to this in your `httpd.conf`:

```
apache
ExportListen httpd:80
```

Ensure that the `OPTIONS` variable is correctly defined in the parent configuration or in the environment.

4. Save the changes and exit the text editor.

5. Restart the httpd service:

```
bash
sudo systemctl restart httpd
```

6. Check the status of the httpd service to ensure it's running:

```
bash
sudo systemctl status httpd
```

If you still encounter issues, consider reviewing the entire `httpd.conf` file for any other syntax errors or misconfigurations. Additionally, ensure that any required modules are included in the server configuration.

Always review AI-generated content prior to use.

5.3. Ask the command-line assistant to provide a solution to the issue.

```
[student@servera ~]$ c chat "Use sed to fix a syntax error on line 47 of
/etc/httpd/conf/httpd.conf due to the misspelled command Lissten (likely meant
to be Listen)"
^_+ Asking RHEL Lightspeed
```

To correct the syntax error on line 47 of `/etc/httpd/conf/httpd.conf` due to the misspelled command `Lissten`, you can use the `sed` command. Here's how you can do it:

1. Open a terminal.
2. Run the following `sed` command:

```
[bash] Snippet _____
sudo sed -i '47s/Lissten/Listen/' /etc/httpd/conf/httpd.conf
```

This command will search for the word `Lissten` on line 47 and replace it with `Listen`. The `-i` option tells `sed` to edit files in place (saves the changes directly to the file).

3. After running the command, verify the change by checking the file:

```
[bash] Snippet _____
cat /etc/httpd/conf/httpd.conf | grep "Listen"
```

You should see `Listen` on line 47, confirming the correction.

Remember to always exercise caution when editing configuration files directly, and consider backing up the file before making changes.

Always review AI-generated content prior to use.

5.4. Fix the syntax error in the `/etc/httpd/conf/httpd.conf` file by using the solution that the command-line assistant provides. Use `student` as the password.

```
[student@servera ~]$ sudo sed -i '47s/Lissten/Listen/'  
/etc/httpd/conf/httpd.conf  
[sudo] password for student: student
```

5.5. Restart the httpd service. Use student as the password.

```
[student@servera ~]$ sudo systemctl restart httpd
```

5.6. Verify that the httpd service started successfully.

```
[student@servera ~]$ sudo systemctl status httpd.service  
• httpd.service - The Apache HTTP Server  
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; disabled; preset:  
disabled)  
   Active: active (running) since Thu 2025-04-24 03:00:24 UTC; 59s ago  
   ...output omitted...
```

6. Return to the workstation machine as the student user.

```
[student@servera ~]$ exit  
logout  
Connection to servera closed.  
student@workstation:~$
```

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish lightspeed-assistant
```

5.3. Summary

In this lesson, you learned how to use Red Hat Enterprise Linux (RHEL) Lightspeed to troubleshoot problems.

RHEL Lightspeed combines decades of Red Hat Enterprise Linux expertise with AI technologies to help Linux professionals to build, deploy, and manage Red Hat Enterprise Linux.

The command-line assistant powered by RHEL Lightspeed is an optional generative AI assistant that is available within the RHEL command-line interface. It provides contextual help by interpreting your natural language questions and offering AI-generated answers, suggestions, commands, examples, or guidance.

To use the command-line assistant effectively, use clear, well-structured questions using natural language.

You can send files, command output, and other content to the command-line assistant for analysis along with your question.

Chapter 6.

Navigating the File-system Hierarchy

Goal

Describe the organization of files in the Linux file-system hierarchy and specify files by name, by using relative paths or absolute paths.

Sections

- The Linux File-system Hierarchy (and Quiz)
- Specifying Files by Name (and Quiz)

For use by Krishna Jeevan Upadhyay redhat.krishna@gmail.com Copyright © 2025 Red Hat, Inc. | For use by Krishna Jeevan Upadhyay redhat.krishna@gmail.com

6.1. The Linux File-system Hierarchy

Objectives

- Describe how Linux files are organized, and the purposes of various directories in the file-system hierarchy.

The File-system Hierarchy

The Linux system stores all files on file systems, which are organized into a single inverted tree known as a file-system hierarchy. This hierarchy is an inverted tree because the tree root is at the top, and the branches of directories and subdirectories stretch below the root.

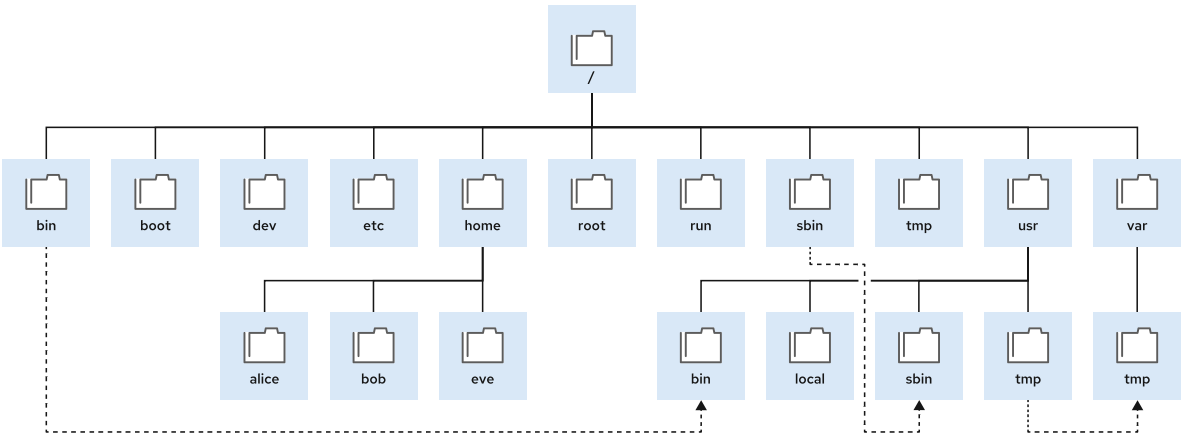


Figure 13. Significant file-system directories in Red Hat Enterprise Linux 10

The root directory, which is denoted by a forward slash (/), is at the top of the file-system hierarchy. The forward slash (/) is also used as a directory separator in file names. For example, if etc is a subdirectory of the / directory, then refer to that directory as /etc. Likewise, if the /etc directory contains a file that is named issue, then refer to that file as /etc/issue.

Subdirectories of the / directory are used for standardized purposes to organize files by type and purpose to make it easier to find files. For example, in the root directory, the /boot subdirectory is used for storing files to boot the system.

 Note

The following terms describe the various types of file-system directory contents:

- *Static* content remains unchanged until explicitly edited or reconfigured.
- *Dynamic* or *variable* content might be modified or appended by active processes.
- *Persistent* content remains after a reboot, such as configuration settings.
- *Runtime* content from a process or from the system is deleted on reboot.

The following table lists some significant directories on the system by name and purpose.

Table 10. Significant Red Hat Enterprise Linux Directories

Location	Purpose
/boot	Files to start the boot process.
/dev	Special device files that the system uses to access hardware.
/etc	System-specific configuration files.
/home	Home directory, where regular users store their data and configuration files.
/root	Home directory for the administrative superuser, root.
/run	Runtime data for processes that started since the last boot. This data includes process ID files and lock files. The contents of this directory are re-created on reboot. This directory consolidates the /var/run and /var/lock directories from earlier versions of Red Hat Enterprise Linux.
/tmp	A world-writable space for temporary files. Files that are not accessed, changed, or modified for 10 days are deleted from this directory automatically. The /var/tmp directory is also a temporary directory, in which files that are not accessed, changed, or modified in more than 30 days are deleted automatically.

Location	Purpose
/usr	Installed software, shared libraries, including files, and read-only program data. Significant subdirectories in the /usr directory include the following commands: • /usr/bin: User commands • /usr/sbin: System administration commands • /usr/local: Locally customized software
/var	System-specific variable data should persist between boots. Files that dynamically change, such as databases, cache directories, log files, printer-spoiled documents, and website content, might be found under the /var directory.

 **Important**

In Red Hat Enterprise Linux 7 and later, four directories in the root directory contain the same files as their counterparts in the /usr directory:

- /bin and /usr/bin
- /sbin and /usr/sbin
- /lib and /usr/lib
- /lib64 and /usr/lib64

Earlier versions of Red Hat Enterprise Linux had distinct directories with different sets of files. In Red Hat Enterprise Linux 7 and later, the directories in the / directory are symbolic links to the matching directories in the /usr directory.

 **References**

hier(7) man page

6.2. Quiz

The Linux File-system Hierarchy

Choose the correct answers to the following questions:

1. Which directory contains persistent, system-specific configuration data?

- ☐ a. /etc
- ☐ b. /root
- ☐ c. /run
- ☐ d. /usr

2. Which directory is the top of the system's file-system hierarchy?

- ☐ a. /etc
- ☐ b. /
- ☐ c. /home/root
- ☐ d. /root

3. Which directory contains user home directories?

- ☐ a. /
- ☐ b. /home
- ☐ c. /root
- ☐ d. /user

4. Which directory contains files to boot the system?

- ☐ a. /boot
- ☐ b. /home/root
- ☐ c. /bootable
- ☐ d. /etc

5. Which directory contains system files to access hardware?

- ☐ a. /etc
- ☐ b. /run
- ☐ c. /dev
- ☐ d. /usr

6. Which directory is the administrative superuser's home directory?

- ☐ a. /etc
- ☐ b. /
- ☐ c. /home/root
- ☐ d. /root

7. Which directory contains regular commands and utilities?

- ☐ a. /commands
- ☐ b. /run
- ☐ c. /usr/bin
- ☐ d. /usr/sbin

8. Which directory contains nonpersistent process runtime data?

- ☐ a. /tmp
- ☐ b. /etc
- ☐ c. /run
- ☐ d. /var

9. Which directory contains installed software programs and libraries?

- ☐ a. /etc
- ☐ b. /lib
- ☐ c. /usr
- ☐ d. /var

Solution

1. Which directory contains persistent, system-specific configuration data?

- ☒ a. /etc
- ☐ b. /root
- ☐ c. /run
- ☐ d. /usr

2. Which directory is the top of the system's file-system hierarchy?

- ☐ a. /etc
- ☒ b. /
- ☐ c. /home/root
- ☐ d. /root

3. Which directory contains user home directories?

- ☐ a. /
- ☒ b. /home
- ☐ c. /root
- ☐ d. /user

4. Which directory contains files to boot the system?

- ☒ a. /boot
- ☐ b. /home/root
- ☐ c. /bootable
- ☐ d. /etc

5. Which directory contains system files to access hardware?

- ☐ a. /etc
- ☐ b. /run
- ☒ c. /dev
- ☐ d. /usr

6. Which directory is the administrative superuser's home directory?

- ☐ a. /etc
- ☐ b. /
- ☐ c. /home/root
- ☒ d. /root

7. Which directory contains regular commands and utilities?

- ☐ a. /commands
- ☐ b. /run
- ☒ c. /usr/bin
- ☐ d. /usr/sbin

8. Which directory contains nonpersistent process runtime data?

- ☐ a. /tmp
- ☐ b. /etc
- ☒ c. /run
- ☐ d. /var

9. Which directory contains installed software programs and libraries?

- ☐ a. /etc
- ☐ b. /lib
- ☒ c. /usr
- ☐ d. /var

6.3. Specifying Files by Name

Objectives

- Specify the absolute location and relative location of files to the current working directory, determine and change the working directory, and list the contents of directories.

Absolute Paths and Relative Paths

The *path* of a file or directory specifies its unique file-system location. Following a file path traverses one or more named subdirectories, which are delimited by a forward slash (/), until the destination is reached. Directories, also called *folders*, can contain other files and other subdirectories. Directories are referenced in the same manner as files.

Important

A space character is acceptable in a Linux file name. The shell also uses spaces to distinguish options and arguments on the command line. If a command includes a file with a space in its name, then the shell can misinterpret the command and assume that the file name is multiple arguments. To avoid this mistake, surround such file names in quotation marks so that the shell interprets the name as a single argument. Red Hat recommends avoiding spaces at all in file names.

Absolute Paths

An *absolute path* is a *fully qualified name* that specifies the exact location of the file in the file-system hierarchy. The absolute path begins at the root (/) directory and includes each subdirectory that must be traversed to reach the specific file. Every file in a file system has a unique absolute path name, which is recognized with a simple rule: a path name with a forward slash (/) as the first character is an absolute path name.

For example, the absolute path name for the system message log file is `/var/log/messages`. Absolute path names can be long to type, so files can also be located relative to the current working directory of your shell prompt.

The Current Working Directory and Relative Paths

When a user logs in and opens a command window, the initial location is typically the user's home directory. System processes also have an initial directory. Users and processes change to other

directories as needed. The *current working directory* is the current location in the file-system hierarchy that a process such as your shell prompt is using.

Similar to an absolute path, a *relative* path identifies a unique location, and specifies only the necessary path to reach the location from the working directory. Relative path names follow this rule: a path name with *anything other than* a forward slash (/) as the first character is a relative path name. For example, relative to the /var directory, the message log file with the absolute path /var/log/messages is log/messages. If there are no slashes in a relative path name, then you are referring to a file or directory in the current working directory.

Linux file systems, including ext4 and XFS, are case-sensitive. Creating the MyExample.txt and myexample.txt files in the same directory results in two unique files.

Non-Linux file systems might work differently. For example, VFAT, Microsoft NTFS, and the default settings for Apple HFS+ and the Apple File System (APFS) have *case-preserving behavior*. Although these file systems are *not* case-sensitive, they do display file names with the file's original capitalization. If you created the files in the preceding example on a VFAT file system, then both names would point to the same file instead of to two different files.

Navigating Paths in the File System

The pwd command displays the full path name of the current working directory for that shell. This command helps you to determine the syntax to reach files by using relative path names. The ls command lists directory contents for the specified directory or, if no directory is given, for the current working directory.

```
user@host:~$ pwd
/home/user
user@host:~$ ls
Desktop Documents Downloads Music Pictures Public Templates Videos
user@host:~$
```

The prompt displays the tilde character (~) when your current working directory is your home directory. When navigating within your home directory, the prompt displays the tilde (as a shortcut to your home directory) and the path to the current working directory. When your shell's current working directory is outside of your home directory, the prompt displays the absolute path to the current location.

Use the cd command to change your shell's current working directory. If you do not specify any arguments to the command, then the shell changes to your home directory.

The following example changes the current working directory by using a mixture of absolute and relative paths with the `cd` command.

```
user@host:~$ pwd
/home/user
user@host:~$ cd Videos
user@host:~/Videos$ pwd
/home/user/Videos
user@host:~/Videos$ cd /home/user/Documents
user@host:~/Documents$ pwd
/home/user/Documents
user@host:~/Documents$ cd
user@host:~$ pwd
/home/user
user@host:~$
```

In the preceding example, the default shell prompt displays the tilde (to represent the user's home directory) and the path to the current working directory. For example, after changing to the `/home/user/Videos` directory, the prompt displays the `~/Videos` path.

The `touch` command updates the time stamp of a file to the current date and time without otherwise modifying it. This command is useful for creating empty files, and can be used for practice, because when you use the `touch` command with a file name that does not exist, the file is created. In the following example, the `touch` command creates practice files in the `Documents` and `Videos` subdirectories.

```
user@host:~$ touch Videos/blockbuster1.ogg
user@host:~$ touch Videos/blockbuster2.ogg
user@host:~$ touch Documents/thesis_chapter1.txt
user@host:~$ touch Documents/thesis_chapter2.txt
user@host:~$
```

The `ls` command has multiple options for displaying attributes on files. The most common options are `-l` (long listing format), `-a` (all files, including *hidden* files), and `-R` (recursive, to include the contents of all subdirectories).

```

user@host:~$ ls -l
total 0
drwxr-xr-x. 2 user user 6 Mar  2 02:45 Desktop
drwxr-xr-x. 2 user user 6 Mar  2 02:45 Documents
drwxr-xr-x. 2 user user 6 Mar  2 02:45 Downloads
drwxr-xr-x. 2 user user 6 Mar  2 02:45 Music
drwxr-xr-x. 2 user user 6 Mar  2 02:45 Pictures
drwxr-xr-x. 2 user user 6 Mar  2 02:45 Public
drwxr-xr-x. 2 user user 6 Mar  2 02:45 Templates
drwxr-xr-x. 2 user user 6 Mar  2 02:45 Videos
user@host:~$ ls -la
total 40
drwx-----. 17 user user 4096 Mar  2 03:07 .
drwxr-xr-x.  4 root root  35 Feb 10 10:48 ..
drwxr-xr-x.  4 user user   27 Mar  2 03:01 .ansible
-rw-----.  1 user user  444 Mar  2 04:32 .bash_history
-rw-r--r--.  1 user user   18 Aug  9 2021 .bash_logout
-rw-r--r--.  1 user user  141 Aug  9 2021 .bash_profile
-rw-r--r--.  1 user user  492 Aug  9 2021 .bashrc
drwxr-xr-x.  9 user user 4096 Mar  2 02:45 .cache
drwxr-xr-x.  9 user user 4096 Mar  2 04:32 .config
drwxr-xr-x.  2 user user    6 Mar  2 02:45 Desktop
drwxr-xr-x.  2 user user    6 Mar  2 02:45 Documents
...output omitted...

```

At the top of the listing are two special directories. One dot (.) refers to the current directory, and two dots (..) refer to the parent directory. These special directories exist in every directory on the system, and they are useful when using file management commands.

Important

File names that begin with a dot (.) indicate *hidden* files; they are not included in the regular output of `ls` and other commands. This behavior is *not* a security feature. Hidden files keep necessary user configuration files from cluttering home directories. Many commands process hidden files only with specific command-line options, and prevent one user's configuration from being accidentally copied to other directories or users.

Protecting file *contents* from improper access requires the use of *file permissions*.

You can also use the tilde special character (~) in combination with other commands for better interaction with the home directory.


```

user@host:~$ cd /var/log/
user@host:/var/log$ ls -l ~
total 0
drwxr-xr-x. 2 user user 6 Mar  2 02:45 Desktop
drwxr-xr-x. 2 user user 6 Mar  2 02:45 Documents
drwxr-xr-x. 2 user user 6 Mar  2 02:45 Downloads
drwxr-xr-x. 2 user user 6 Mar  2 02:45 Music
drwxr-xr-x. 2 user user 6 Mar  2 02:45 Pictures
drwxr-xr-x. 2 user user 6 Mar  2 02:45 Public
drwxr-xr-x. 2 user user 6 Mar  2 02:45 Templates
drwxr-xr-x. 2 user user 6 Mar  2 02:45 Videos
user@host:~$

```

The `cd` command has many options. Some options are useful to practice early and to use often.

For example, the `cd -` command returns your shell prompt to the *previous* directory, the working directory you were using immediately before the current working directory. The following example illustrates this behavior, and alternates between two directories, which is useful when processing a series of similar tasks.

```

user@host:~$ cd Videos
user@host:~/Videos$ pwd
/home/user/Videos
user@host:~/Videos$ cd /home/user/Documents
user@host:~/Documents$ pwd
/home/user/Documents
user@host:~/Documents$ cd -
user@host:~/Videos$ pwd
/home/user/Videos
user@host:~/Videos$ cd -
user@host:~/Documents$ pwd
/home/user/Documents
user@host:~/Documents$ cd -
user@host:~/Videos$ pwd
/home/user/Videos
user@host:~/Videos$ cd
user@host:~$

```

The `cd ..` command uses the hidden parent directory (`..`) to move the current working directory of your shell prompt up one level, without requiring the absolute path to the parent directory. The other

hidden directory (`.`) specifies the *current directory* on commands where the current location is either the source or destination argument, and avoids the need to type the directory's absolute path name.

```
user@host:~/Videos$ pwd
/home/user/Videos
user@host:~/Videos$ cd .
user@host:~/Videos$ pwd
/home/user/Videos
user@host:~/Videos$ cd ..
user@host:~$ pwd
/home/user
user@host:~$ cd ..
user@host:/home$ pwd
/home
user@host:/home$ cd ..
user@host:/$ pwd
/
user@host:/$ cd
user@host:~$ pwd
/home/user
user@host:~$
```

References

info libc 'file name resolution' (*GNU C Library Reference Manual*)

- Section 11.2.2 File Name Resolution

bash(1), cd(1), ls(1), pwd(1), unicode(7), and utf-8(7) man pages

[UTF-8 and Unicode](#)

6.4. Quiz

Specifying Files by Name

Choose the correct answers to the following questions:

1. Which command returns to your home directory, assuming that the current working directory is /tmp and /home/user is your home directory?

- ☐ a. cd
- ☐ b. cd ..
- ☐ c. cd .
- ☐ d. cd *
- ☐ e. cd /home

2. Which command displays the absolute path name of the current working directory?

- ☐ a. cd
- ☐ b. pwd
- ☐ c. ls ~
- ☐ d. ls -d

3. Which command returns your shell prompt to the working directory you were using immediately before the current working directory?

- ☐ a. cd -
- ☐ b. cd -p
- ☐ c. cd ~
- ☐ d. cd ..

4. Which command always changes the working directory up two levels from the current working directory?

- ☐ a. cd ~/..
- ☐ b. cd ../..
- ☐ c. cd ../../
- ☐ d. cd ~/

5. Which command lists files in the current working directory, with a long format, and including hidden files?

- ☐ a. `llong ~`
- ☐ b. `ls -a`
- ☐ c. `ls -l`
- ☐ d. `ls -al`

6. Which command creates an empty file named `helloworld.py` in the user home directory, assuming that `/home` is your current directory?

- ☐ a. `touch ./helloworld.py`
- ☐ b. `touch ~/helloworld.py`
- ☐ c. `touch helloworld.py`
- ☐ d. `touch ../helloworld.py`

7. Which command changes the working directory to the parent of the current working directory?

- ☐ a. `cd ~`
- ☐ b. `cd ..`
- ☐ c. `cd ../../`
- ☐ d. `cd -u1`

8. Which command changes the working directory to `/tmp` if `/home/student` is the current working directory?

- ☐ a. `cd tmp`
- ☐ b. `cd ..`
- ☐ c. `cd ../../tmp`
- ☐ d. `cd ~tmp`

Solution

1. Which command returns to your home directory, assuming that the current working directory is /tmp and /home/user is your home directory?
 - ☒ a. `cd`
 - ☐ b. `cd ..`
 - ☐ c. `cd .`
 - ☐ d. `cd *`
 - ☐ e. `cd /home`
2. Which command displays the absolute path name of the current working directory?
 - ☐ a. `cd`
 - ☒ b. `pwd`
 - ☐ c. `ls ~`
 - ☐ d. `ls -d`
3. Which command returns your shell prompt to the working directory you were using immediately before the current working directory?
 - ☒ a. `cd -`
 - ☐ b. `cd -p`
 - ☐ c. `cd ~`
 - ☐ d. `cd ..`
4. Which command always changes the working directory up two levels from the current working directory?
 - ☐ a. `cd ~/..`
 - ☐ b. `cd ../..`
 - ☒ c. `cd ../../`
 - ☐ d. `cd ~/`

5. Which command lists files in the current working directory, with a long format, and including hidden files?

- ☐ a. `llong ~`
- ☐ b. `ls -a`
- ☐ c. `ls -l`
- ☒ d. `ls -al`

6. Which command creates an empty file named `helloworld.py` in the user home directory, assuming that `/home` is your current directory?

- ☐ a. `touch ./helloworld.py`
- ☒ b. `touch ~/helloworld.py`
- ☐ c. `touch helloworld.py`
- ☐ d. `touch ../helloworld.py`

7. Which command changes the working directory to the parent of the current working directory?

- ☐ a. `cd ~`
- ☒ b. `cd ..`
- ☐ c. `cd ../..`
- ☐ d. `cd -u1`

8. Which command changes the working directory to `/tmp` if `/home/student` is the current working directory?

- ☐ a. `cd tmp`
- ☐ b. `cd ..`
- ☒ c. `cd ../../tmp`
- ☐ d. `cd ~tmp`

6.5. Summary

In this lesson, you learned that Linux stores all files on file systems, organized into a single inverted tree called the file-system hierarchy.

Each subdirectory under the root directory (/) is used for different purposes, such as the /boot directory, which contains files used to boot the system, and the /etc directory, which contains system configuration files.

Files in Linux are case-sensitive and you can refer to files by using its absolute path or its relative path. You can navigate the file system by using the `cd` command, which accepts paths to directories as parameters, including the special directories named with a single dot (.) for the current working directory, and with a double dot (..) for its parent directory.

You can list the contents of directories by using the `ls` command, which by default does not show hidden files. Any file that has a name that starts with a dot (.) is a hidden file.

Chapter 7.

Managing Files from the Command Line

Goal

Copy, move, create, delete, and organize files from the command line.

Sections

- Managing Files with Command-line Tools (and Guided Exercise)
- Creating Links Between Files (and Guided Exercise)
- Matching File Names with Shell Expansions (and Quiz)

Lab

- Manage Files from the Command Line

7.1. Managing Files with Command-line Tools

Objectives

- Create, copy, move, and remove files and directories.

Command-line File Management

Creating, copying, moving, and, removing files and directories are common operations for a system administrator.

When you are running a command to manage files, be aware of the command's options. Without options, some commands are used to interact with files. Or, with the appropriate set of options, some commands can manipulate directories. Additionally, the meaning of some options might differ between commands.

Creating Files

In Linux, you can use several command-line tools to create files. For example, you can use the `cat` and `echo` commands to direct output to a specified file name. Or, you can use command-line text editors, such as `Vim` and `nano`.

In some cases, you might want to create an empty file to reserve a file name for later use. In this scenario, you can use the `touch` command. The `touch` command creates a new file or, if the file name exists, then the `touch` command updates the file timestamp.

```
user@host:~$ touch mytasklist.txt
user@host:~$ ls mytasklist.txt
mytasklist.txt
```

Creating Directories

The `mkdir` command creates one or more directories or subdirectories. It takes as an argument a list of paths to the directories that you want to create.

In the following example, files and directories are organized beneath the `/home/user/Documents` directory. Use the `mkdir` command and a space-delimited list of the directory names to create multiple directories.

```
user@host:~$ cd Documents
user@host:~/Documents$ mkdir ProjectX ProjectY ProjectZ
user@host:~/Documents$ ls
ProjectX ProjectY ProjectZ
```

If the directory that you are trying to create already exists, then the `mkdir` command fails and displays an error.

The `mkdir` command also fails if a parent directory of the requested directory does not exist. The `mkdir` command and `-p` (*parent*) option creates any missing parent directories for the requested destination. In the following example, the `mkdir` command creates three `ChapterN` subdirectories with one command. The `-p` option creates the missing `Thesis` parent directory.

```
user@host:~/Documents$ mkdir -p Thesis/Chapter1 Thesis/Chapter2 Thesis/Chapter3
user@host:~/Documents$ ls -R Thesis/
Thesis/:
Chapter1 Chapter2 Chapter3

Thesis/Chapter1:

Thesis/Chapter2:

Thesis/Chapter3:
```

Use the `mkdir` command `-p` option with caution, because spelling mistakes can create unintended directories without generating error messages. In the following example, imagine that you are trying to create a `Watched` subdirectory in the `Videos` directory, but you accidentally omit the letter "s" in `Videos` in your `mkdir` command:

```
user@host:~$ mkdir Video/Watched
mkdir: cannot create directory Video/Watched: No such file or directory
```

The `mkdir` command fails because the `Video` directory does not exist. If you had used the `mkdir` command with the `-p` option, then the `Video` directory would be created unintentionally. The `Watched` subdirectory would be created in that incorrect directory.

Copying Files and Directories

The `cp` command copies a file, and creates a file either in the current directory or in a different specified directory.

```

user@host:~$ cd Videos
user@host:~/Videos$ cp blockbuster1.ogg blockbuster3.ogg
user@host:~/Videos$ ls -l
total 0
-rw-rw-r--. 1 user user 0 Feb 8 16:23 blockbuster1.ogg
-rw-rw-r--. 1 user user 0 Feb 8 16:24 blockbuster2.ogg
-rw-rw-r--. 1 user user 0 Feb 8 16:34 blockbuster3.ogg

```

You can also use the `cp` command to copy multiple files to a directory. In this scenario, the last argument must be a directory. The copied files retain their original names in the new directory. If a file with the same name exists in the target directory, then the existing file is overwritten.

Note

By default, the `cp` command does not copy directories; it ignores them.

In the following example, two directories are listed as arguments, the `Thesis` and `ProjectX` directories. The last argument, the `ProjectX` directory, is the target and is valid as a destination. The `Thesis` argument is ignored by the `cp` command, because it is intended to be copied and it is a directory.

```

user@host:~/Documents$ cp thesis_chapter1.txt thesis_chapter2.txt Thesis
ProjectX
cp: -r not specified; omitting directory 'Thesis'
user@host:~/Documents$ ls Thesis ProjectX
ProjectX:
thesis_chapter1.txt  thesis_chapter2.txt

Thesis:
Chapter1  Chapter2  Chapter3

```

The `Thesis` directory failed to copy, but the copying of the `thesis_chapter1.txt` and `thesis_chapter2.txt` files succeeded.

You can copy directories and their contents by using the `cp` command with `-r` option. Keep in mind that you can use the `.` and `..` special directories in command combinations. In the following example, the `Thesis` directory and its contents are copied to the `ProjectY` directory:

```

user@host:~/Documents$ cd ProjectY
user@host:~/ProjectY$ cp -r ../Thesis/ .
user@host:~/ProjectY$ ls -lR
.:
total 0
drwxr-xr-x. 5 user user 54 Mar  7 15:08 Thesis

./Thesis:
total 0
drwxr-xr-x. 2 user user 6 Mar  7 15:08 Chapter1
drwxr-xr-x. 2 user user 6 Mar  7 15:08 Chapter2
drwxr-xr-x. 2 user user 6 Mar  7 15:08 Chapter3

./Thesis/Chapter1:
total 0

./Thesis/Chapter2:
total 0

./Thesis/Chapter3:
total 0

```

Renaming and Moving Files and Directories

If you think of the absolute path to a file as its full name, then moving a file is effectively the same as renaming a file. The `mv` command moves files from one location to another. The contents of the files that are moved remain unchanged.

You can use the `mv` command to *rename* a file. In the following example, the `mv thesis_chapter2.txt` command renames the `thesis_chapter2.txt` file to `thesis_chapter2_reviewed.txt` in the same directory.

```

user@host:~/Documents$ ls -l
-rw-r--r--. 1 user user 7100 Mar  7 14:37 thesis_chapter1.txt
-rw-r--r--. 1 user user 11431 Mar  7 14:39 thesis_chapter2.txt
...output omitted...
user@host:~/Documents$ mv thesis_chapter2.txt thesis_chapter2_reviewed.txt
user@host:~/Documents$ ls -l
-rw-r--r--. 1 user user 7100 Mar  7 14:37 thesis_chapter1.txt
-rw-r--r--. 1 user user 11431 Mar  7 14:39 thesis_chapter2_reviewed.txt
...output omitted...

```

Use the `mv` command to *move* a file to a different directory. In the next example, the `thesis_chapter1.txt` file is moved from the `~/Documents` directory to the `~/Documents/Thesis/Chapter1` directory. You can use the `mv` command with `-v` option to display a detailed output of the command operations.

```

user@host:~/Documents$ ls Thesis/Chapter1
user@host:~/Documents$
user@host:~/Documents$ mv -v thesis_chapter1.txt Thesis/Chapter1
renamed 'thesis_chapter1.txt' -> 'Thesis/Chapter1/thesis_chapter1.txt'
user@host:~/Documents$ ls Thesis/Chapter1
thesis_chapter1.txt
user@host:~/Documents$ ls -l
-rw-r--r--. 1 user user 11431 Mar  7 14:39 thesis_chapter2_reviewed.txt
...output omitted...

```

Removing Files and Directories

The `rm` command removes files. By default, `rm` does not remove directories. You can use the `rm` command with `-r` option or the `--recursive` option to enable the `rm` command to remove directories and their contents. The `rm -r` command traverses each subdirectory first, and individually removes their files before removing each directory.

In the following example, the `rm` command removes the `thesis_chapter1.txt` file without options, but to remove the `Thesis/Chapter1` directory, you must add the `-r` option.

```

user@host:~/Documents$ ls -l Thesis/Chapter1
-rw-r--r--. 1 user user 7100 Mar  7 14:37 thesis_chapter1.txt
user@host:~/Documents$ rm Thesis/Chapter1/thesis_chapter1.txt
user@host:~/Documents$ rm Thesis/Chapter1
rm: cannot remove 'Thesis/Chapter1': Is a directory
user@host:~/Documents$ rm -r Thesis/Chapter1
user@host:~/Documents$ ls -l Thesis
drwxr-xr-x. 2 user user 6 Mar  7 12:37 Chapter2
drwxr-xr-x. 2 user user 6 Mar  7 12:37 Chapter3

```

Important

Red Hat Enterprise Linux does not provide a way to undo deletions, nor a "trash bin" from which you can restore files held for deletion. A trash bin is a component of a desktop environment such as GNOME, but is not used by commands that are run from a shell.

A good practice is to verify your current working directory before you remove a file or directory by using relative paths.

```

user@host:~/Documents$ pwd
/home/user/Documents
user@host:~/Documents$ ls -l thesis*
-rw-r--r--. 1 user user 11431 Mar  7 14:39 thesis_chapter2_reviewed.txt
user@host:~/Documents$ rm thesis_chapter2_reviewed.txt
user@host:~/Documents$ ls -l thesis*
ls: cannot access 'thesis*': No such file or directory

```

You can use the `rm` command with `-i` option to interactively prompt for confirmation before deleting. This option is the opposite of using the `rm` command `-f` option, which forces the removal without prompting the user for confirmation.

```
user@host:~/Documents$ rm -ri Thesis
rm: descend into directory 'Thesis'? y
rm: descend into directory 'Thesis/Chapter2'? y
rm: remove regular empty file 'Thesis/Chapter2/thesis_chapter2.txt'? y
rm: remove directory 'Thesis/Chapter2'? y
rm: remove directory 'Thesis/Chapter3'? y
rm: remove directory 'Thesis'? y
```

⚠ Warning

If you specify both the `-i` and `-f` options, then the `-f` option takes priority and you are not prompted for confirmation before the `rm` command removes files.

You can also use the `rmdir` command to remove empty directories. Use the `rm` command `-r` option to remove non-empty directories.

```
user@host:~/Documents$ pwd
/home/user/Documents
user@host:~/Documents$ rmdir ProjectZ
user@host:~/Documents$ rmdir ProjectX
rmdir: failed to remove 'ProjectX': Directory not empty
user@host:~/Documents$ rm -r ProjectX
user@host:~/Documents$
```

📄 References

`cp(1)`, `mkdir(1)`, `mv(1)`, `rm(1)`, and `rmdir(1)` man pages

7.2. Guided Exercise

Manage Files with Command-line Tools

Create, copy, move, and remove files and directories.

Outcomes

- Create, organize, copy, and remove files and directories.

Prerequisites

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
student@workstation:~$ lab start files-manage
```

Instructions

1. Log in to the servera machine as the student user. In the student user's home directory, create three subdirectories: Music, Pictures, and Videos.

- 1.1. Log in to the servera machine as the student user.

```
student@workstation:~$ ssh student@servera
...output omitted...
[student@servera ~]$
```

- 1.2. In the student user's home directory, use the `mkdir` command to create three subdirectories: Music, Pictures, and Videos.

```
[student@servera ~]$ mkdir Music Pictures Videos
```

2. Use the `touch` command to create sets of empty practice files to use during this activity. In each set, replace the *X* placeholder in the file name with the numbers 1 through 6.
 - Create six files with names in the following format: `songX.mp3`.
 - Create six files with names in the following format: `snapX.jpg`.
 - Create six files with names in the following format: `filmX.avi`.


```
[student@servera ~]$ touch song1.mp3 song2.mp3 song3.mp3 \
song4.mp3 song5.mp3 song6.mp3
```

```
[student@servera ~]$ touch snap1.jpg snap2.jpg snap3.jpg \
snap4.jpg snap5.jpg snap6.jpg
```

```
[student@servera ~]$ touch film1.avi film2.avi film3.avi \
film4.avi film5.avi film6.avi
```

```
[student@servera ~]$ ls -l
total 0
-rw-r--r--. 1 student student 0 Mar  7 20:58 film1.avi
-rw-r--r--. 1 student student 0 Mar  7 20:58 film2.avi
-rw-r--r--. 1 student student 0 Mar  7 20:58 film3.avi
-rw-r--r--. 1 student student 0 Mar  7 20:58 film4.avi
-rw-r--r--. 1 student student 0 Mar  7 20:58 film5.avi
-rw-r--r--. 1 student student 0 Mar  7 20:58 film6.avi
drwxr-xr-x. 2 student student 6 Mar  7 20:58 Music
drwxr-xr-x. 2 student student 6 Mar  7 20:58 Pictures
-rw-r--r--. 1 student student 0 Mar  7 20:58 snap1.jpg
-rw-r--r--. 1 student student 0 Mar  7 20:58 snap2.jpg
-rw-r--r--. 1 student student 0 Mar  7 20:58 snap3.jpg
-rw-r--r--. 1 student student 0 Mar  7 20:58 snap4.jpg
-rw-r--r--. 1 student student 0 Mar  7 20:58 snap5.jpg
-rw-r--r--. 1 student student 0 Mar  7 20:58 snap6.jpg
-rw-r--r--. 1 student student 0 Mar  7 20:58 song1.mp3
-rw-r--r--. 1 student student 0 Mar  7 20:58 song2.mp3
-rw-r--r--. 1 student student 0 Mar  7 20:58 song3.mp3
-rw-r--r--. 1 student student 0 Mar  7 20:58 song4.mp3
-rw-r--r--. 1 student student 0 Mar  7 20:58 song5.mp3
-rw-r--r--. 1 student student 0 Mar  7 20:58 song6.mp3
drwxr-xr-x. 2 student student 6 Mar  7 20:58 Videos
```

3. Move the song files (.mp3 extension) to the Music directory, the snapshot files (.jpg extension) to the Pictures directory, and the movie files (.avi extension) to the Videos directory.

```
[student@servera ~]$ mv song1.mp3 song2.mp3 song3.mp3 \
song4.mp3 song5.mp3 song6.mp3 Music
```

```
[student@servera ~]$ mv snap1.jpg snap2.jpg snap3.jpg \
snap4.jpg snap5.jpg snap6.jpg Pictures
```

```
[student@servera ~]$ mv film1.avi film2.avi film3.avi \
film4.avi film5.avi film6.avi Videos
```

```
[student@servera ~]$ ls -l Music Pictures Videos
```

Music:

total 0

```
-rw-r--r--. 1 student student 0 Mar  7 20:58 song1.mp3
-rw-r--r--. 1 student student 0 Mar  7 20:58 song2.mp3
-rw-r--r--. 1 student student 0 Mar  7 20:58 song3.mp3
-rw-r--r--. 1 student student 0 Mar  7 20:58 song4.mp3
-rw-r--r--. 1 student student 0 Mar  7 20:58 song5.mp3
-rw-r--r--. 1 student student 0 Mar  7 20:58 song6.mp3
```

Pictures:

total 0

```
-rw-r--r--. 1 student student 0 Mar  7 20:58 snap1.jpg
-rw-r--r--. 1 student student 0 Mar  7 20:58 snap2.jpg
-rw-r--r--. 1 student student 0 Mar  7 20:58 snap3.jpg
-rw-r--r--. 1 student student 0 Mar  7 20:58 snap4.jpg
-rw-r--r--. 1 student student 0 Mar  7 20:58 snap5.jpg
-rw-r--r--. 1 student student 0 Mar  7 20:58 snap6.jpg
```

Videos:

total 0

```
-rw-r--r--. 1 student student 0 Mar  7 20:58 film1.avi
-rw-r--r--. 1 student student 0 Mar  7 20:58 film2.avi
-rw-r--r--. 1 student student 0 Mar  7 20:58 film3.avi
-rw-r--r--. 1 student student 0 Mar  7 20:58 film4.avi
-rw-r--r--. 1 student student 0 Mar  7 20:58 film5.avi
-rw-r--r--. 1 student student 0 Mar  7 20:58 film6.avi
```

4. Create three subdirectories for organizing your files, and name the subdirectories friends, family, and work. Use a single command to create all three subdirectories at the same time.

```
[student@servera ~]$ mkdir friends family work
[student@servera ~]$ ls -l
total 0
drwxr-xr-x. 2 student student 6 Mar 7 21:01 family
drwxr-xr-x. 2 student student 6 Mar 7 21:01 friends
drwxr-xr-x. 2 student student 108 Mar 7 21:00 Music
drwxr-xr-x. 2 student student 108 Mar 7 21:00 Pictures
drwxr-xr-x. 2 student student 108 Mar 7 21:00 Videos
drwxr-xr-x. 2 student student 6 Mar 7 21:01 work
```

5. Copy files that contain numbers 1 and 2 to the friends directory, and files that contain numbers 3 and 4 to the family directory. Keep in mind that you are making copies; therefore, the original files must remain in their original locations after you complete the step.

Important

When you copy files from multiple locations into a single location, Red Hat recommends that you change to the destination directory before you copy the files. Use the simplest path syntax, whether absolute or relative, to reach the source for each file management task.

- 5.1. Copy files that contain numbers 1 and 2 to the friends directory.

```
[student@servera ~]$ cd friends
[student@servera friends]$ cp ~/Music/song1.mp3 ~/Music/song2.mp3 \
~/Pictures/snap1.jpg ~/Pictures/snap2.jpg ~/Videos/film1.avi \
~/Videos/film2.avi .
[student@servera friends]$ ls -l
total 0
-rw-r--r--. 1 student student 0 Mar 7 21:02 film1.avi
-rw-r--r--. 1 student student 0 Mar 7 21:02 film2.avi
-rw-r--r--. 1 student student 0 Mar 7 21:02 snap1.jpg
-rw-r--r--. 1 student student 0 Mar 7 21:02 snap2.jpg
-rw-r--r--. 1 student student 0 Mar 7 21:02 song1.mp3
-rw-r--r--. 1 student student 0 Mar 7 21:02 song2.mp3
```

- 5.2. Copy files that contain numbers 3 and 4 to the family directory.

```
[student@servera friends]$ cd ../family
[student@servera family]$ cp ~/Music/song3.mp3 ~/Music/song4.mp3 \
~/Pictures/snap3.jpg ~/Pictures/snap4.jpg ~/Videos/film3.avi \
~/Videos/film4.avi .
[student@servera family]$ ls -l
total 0
total 0
-rw-r--r--. 1 student student 0 Mar  7 21:04 film3.avi
-rw-r--r--. 1 student student 0 Mar  7 21:04 film4.avi
-rw-r--r--. 1 student student 0 Mar  7 21:04 snap3.jpg
-rw-r--r--. 1 student student 0 Mar  7 21:04 snap4.jpg
-rw-r--r--. 1 student student 0 Mar  7 21:04 song3.mp3
-rw-r--r--. 1 student student 0 Mar  7 21:04 song4.mp3
```

6. Copy the family and friends directories and their contents to the work directory.

```
[student@servera family]$ cd ../work
[student@servera work]$ cp -r ~/family ~/friends .
[student@servera work]$ ls -l
total 0
drwxr-xr-x. 2 student student 108 Mar  7 21:05 family
drwxr-xr-x. 2 student student 108 Mar  7 21:05 friends
```

7. Your project tasks are now complete, and it is time to clean up the directories.

7.1. Use the tree command to verify the contents of directories before deleting them.

```
[student@servera work]$ cd ..  
[student@servera ~]$ tree
```

```
.  
├── Music  
│   ├── song1.mp3  
│   ├── song2.mp3  
│   ├── song3.mp3  
│   ├── song4.mp3  
│   ├── song5.mp3  
│   └── song6.mp3  
├── Pictures  
│   ├── snap1.jpg  
│   ├── snap2.jpg  
│   ├── snap3.jpg  
│   ├── snap4.jpg  
│   ├── snap5.jpg  
│   └── snap6.jpg  
├── Videos  
│   ├── film1.avi  
│   ├── film2.avi  
│   ├── film3.avi  
│   ├── film4.avi  
│   ├── film5.avi  
│   └── film6.avi  
├── family  
│   ├── film3.avi  
│   ├── film4.avi  
│   ├── snap3.jpg  
│   ├── snap4.jpg  
│   ├── song3.mp3  
│   └── song4.mp3  
├── friends  
│   ├── film1.avi  
│   ├── film2.avi  
│   ├── snap1.jpg  
│   ├── snap2.jpg  
│   ├── song1.mp3  
│   └── song2.mp3  
└── work  
    ├── family  
    │   ├── film3.avi  
    │   ├── film4.avi  
    │   ├── snap3.jpg  
    │   ├── snap4.jpg  
    │   ├── song3.mp3  
    │   └── song4.mp3  
    └── friends  
        ├── film1.avi  
        └── film2.avi
```

```
├── snap1.jpg
├── snap2.jpg
├── song1.mp3
└── song2.mp3
```

9 directories, 42 files

- 7.2.** Use the `rm -rf` command to recursively delete the `family`, `friends`, and `work` directories and their contents.

```
[student@servera ~]$ rm -rf family/ friends/ work/
[student@servera ~]$ ls -l
total 0
drwxr-xr-x. 2 student student 108 Mar  7 21:00 Music
drwxr-xr-x. 2 student student 108 Mar  7 21:00 Pictures
drwxr-xr-x. 2 student student 108 Mar  7 21:00 Videos
```

- 8.** Return to the workstation system as the student user.

```
[student@servera ~]$ exit
logout
Connection to servera closed.
student@workstation:~$
```

Finish

On the workstation machine, change to the student user home directory and use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish files-manage
```

7.3. Creating Links Between Files

Objectives

- Create and compare the use of hard links and symbolic links to reference the same file.

Managing Links Between Files

You can create multiple file names that point to the same file. These file names are called *links*.

You can create two types of links: a *hard link* or a *symbolic link* (sometimes called a *soft link*). Each type of link is useful for different use cases.

Hard Links to Files

Every file starts with a single hard link, from its initial name to the data on the file system. When you create a hard link to a file, you create another name that points to that same data. The new hard link acts exactly like the original file name. After the link is created, you cannot notice the difference between the new hard link and the original name of the file.

You can determine whether a file has multiple hard links by using the `ls -l` command. One item in this command's output is each file's *link count*, which is the number of hard links that the file has. In the next example, the link count of the `newfile.txt` file is 1. It has exactly one absolute path, which is the `/home/user/newfile.txt` location.

```
user@host:~$ pwd
/home/user
user@host:~$ ls -l newfile.txt
-rw-r--r--. 1 user user 0 May 30 19:19 newfile.txt
```

You can use the `ln` command to create a hard link (another file name) that points to an existing file. The command needs at least two arguments: a path to the existing file, and the path to the hard link that you want to create.

The following example creates a hard link named `newfile-hlink2.txt` in the `/tmp` directory for the existing `newfile.txt` file.

```

user@host:~$ ln newfile.txt /tmp/newfile-hlink2.txt
user@host:~$ ls -l newfile.txt /tmp/newfile-hlink2.txt
-rw-rw-r--. 2 user user 12 May 30 19:19 /tmp/newfile-hlink2.txt
-rw-rw-r--. 2 user user 12 May 30 19:19 newfile.txt

```

To determine whether two files are hard linked, use the `ls` command `-i` option to list each file's *inode number*. If the files are on the same file system and their inode numbers are the same, then the files are hard links that point to the same data file content.

In the following example, you can verify that the `newfile.txt` and the `/tmp/newfile-hlink2.txt` files share the 8924107 inode number in the first column of the `ls -il` command.

```

user@host:~$ ls -il newfile.txt /tmp/newfile-hlink2.txt
8924107 -rw-rw-r--. 2 user user 12 May 30 19:19 /tmp/newfile-hlink2.txt
8924107 -rw-rw-r--. 2 user user 12 May 30 19:19 newfile.txt

```

Important

Hard links that reference the same file share the inode structure with the link count, access permissions, user and group ownership, time stamps, and file content.

When that information is changed for one hard link, the other hard links for the same file also show the new information. This behavior is because each hard link points to the same data on the storage device.

Even if the original file is deleted, you can still access the contents of the file provided that at least one other hard link exists. Data is deleted from storage only when the last hard link is deleted, which makes the file contents unreferenced by any hard link.

```

user@host:~$ rm -f newfile.txt
user@host:~$ ls -l /tmp/newfile-hlink2.txt
-rw-rw-r--. 1 user user 12 Mar 30 19:19 /tmp/newfile-hlink2.txt
user@host:~$ cat /tmp/newfile-hlink2.txt
Hello World

```

Limitations of Hard Links

Hard links have some limitations. First, you can use hard links only with regular files. You cannot use the `ln` command to create a hard link to a directory or a special file.

Second, you can use hard links only if both files are on the same *file system*. The file-system hierarchy can include multiple storage devices. Depending on the configuration of your system, when you change into a new directory, that directory and its contents might be stored on a different file system.

You can list the directories that are on different file systems by using the `df` command.

The following example shows an example file system distribution. The `Mounted on` column shows the directory where the file system is mounted.

```
user@host:~$ df
Filesystem      1K-blocks    Used Available Use% Mounted on
/dev/sda3       20699116 6570308  14128808  32% /
devtmpfs         4096         0       4096     0% /dev
tmpfs           2899412      84    2899328   1% /dev/shm
tmpfs           1159768    17764   1142004   2% /run
tmpfs            1024         0       1024     0% /run/credentials/systemd-
journald.service
/dev/sda2        204580     8580    196000    5% /boot/efi
/dev/sr0         536        536         0 100% /run/media/meta-data
tmpfs           579880     2548    577332   1% /run/user/1000
tmpfs            1024         0       1024     0% /run/credentials/serial-
getty@ttyS0.service
```

Files and directories under the same `Mounted on` directory are part of the same file system. In this example, you can create a hard link between the `/var/tmp/link1` and `/home/user/file` files, because they are both subdirectories of the root (`/`) directory and belong to the same file system. However, you cannot create a hard link between the `/boot/efi/test/newlink` and `/home/user/mybackup` files because the `newlink` file belongs to the file system mounted on the `/boot/efi` directory and the `mybackup` file does not.

Symbolic Links to Files

The `ln` command `-s` option creates a symbolic link, which is also called a *soft link*. A symbolic link is not a regular file. It is a special type of file that points to an existing file or directory.

Symbolic links have some advantages over hard links:

- Symbolic links can link two files on different file systems.
- Symbolic links can point to a directory or a special file, in addition to a regular file.

In the following example, the `ln -s` command creates a symbolic link for the `/home/user/newfile-link2.txt` file. The name for the symbolic link is `/tmp/newfile-symlink.txt`.

```
user@host:~$ ln -s /home/user/newfile-link2.txt /tmp/newfile-symlink.txt
user@host:~$ ls -l newfile-link2.txt /tmp/newfile-symlink.txt
-rw-rw-r--. 1 user user 12 May 30 19:19 newfile-link2.txt
lrwxrwxrwx. 1 user user 11 May 30 20:59 /tmp/newfile-symlink.txt ->
/home/user/newfile-link2.txt
user@host:~$ cat /tmp/newfile-symlink.txt
Symbolic Hello World
```

In the preceding example, the first character of the long listing for the `/tmp/newfile-symlink.txt` file is `l` (the letter "l") instead of a dash (-). This character indicates that the file is a symbolic link and not a regular file.

When the original regular file is deleted, the symbolic link still points to the file but the target is gone. A symbolic link that points to a missing file is called a *dangling symbolic link*.

```
user@host:~$ rm -f newfile-link2.txt
user@host:~$ ls -l /tmp/newfile-symlink.txt
lrwxrwxrwx. 1 user user 11 May 30 20:59 /tmp/newfile-symlink.txt ->
/home/user/newfile-link2.txt
user@host:~$ cat /tmp/newfile-symlink.txt
cat: /tmp/newfile-symlink.txt: No such file or directory
```

Important

One side effect of the dangling symbolic link in the preceding example is that if you create a file with the same name as the deleted file (`/home/user/newfile-link2.txt`), then the symbolic link is no longer "dangling" and points to the new file. Hard links do not work in this way. If you delete a hard link and then use other tools (rather than the `ln` command) to create a file with the same name, then the new file is not linked to the deleted file. Consider the following way to compare hard links and symbolic links, to understand how they work:

- A hard link points a name to data on a storage device.
- A symbolic link points a name to another name, which points to data on a storage device.

A symbolic link can point to a directory. The symbolic link then acts like the directory. If you use the `cd` command to change to the symbolic link, then the current working directory becomes the linked directory. Some tools might track that you followed a symbolic link. For example, by default, the `cd` command updates your current working directory by using the name of the symbolic link rather than the name of the actual directory. If you want to update the current working directory by using the name of the actual directory, then you can use the `-P` option.

The following example creates a symbolic link named `/home/user/configfiles` that points to the `/etc` directory.

```
user@host:~$ ln -s /etc /home/user/configfiles
user@host:~$ cd /home/user/configfiles
user@host:~/configfiles$ pwd
/home/user/configfiles
user@host:~/configfiles$ cd -P /home/user/configfiles
user@host:/etc$ pwd
/etc
```

References

`info ln` (*Make links between files*)

`ln(1)` and `df(1)` man pages

7.4. Guided Exercise

Create Links Between Files

Create and compare the use of hard links and symbolic links to reference the same file.

Outcomes

- Create hard links and symbolic links between files.

Prerequisites

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
student@workstation:~$ lab start files-make
```

Instructions

1. Log in to the servera machine as the student user.

```
student@workstation:~$ ssh student@servera
...output omitted...
[student@servera ~]$
```

2. Create the `/home/student/links/file.hardlink` hard link for the `/home/student/files/target.file` file. Verify the link count for the original file and the new linked file.

- 2.1. View the link count and the index number (inode) for the `/home/student/files/target.file` file.

The inode number of the file might vary in your environment.

```
[student@servera ~]$ ls -li files/target.file
8838187 -rw-r--r--. 1 student student 11 May 27 13:15 files/target.file
```

The link count of the `files/target.file` is 1 because the file does not have additional links to it.

- 2.2.** Create the `/home/student/links/file.hardlink` hard link. Link it to the `/home/student/files/target.file` file.

```
[student@servera ~]$ ln /home/student/files/target.file \
/home/student/links/file.hardlink
```

- 2.3.** Verify the link count and the inode number for the `/home/student/files/target.file` original file and the `/home/student/links/file.hardlink` link file. The link count should be 2 and the inode number must be the same for both files.

```
[student@servera ~]$ ls -li files/target.file links/file.hardlink
8838187 -rw-r--r--. 2 student student 11 May 27 13:15 files/target.file
8838187 -rw-r--r--. 2 student student 11 May 27 13:15 links/file.hardlink
```

The file and the link have the same inode number because they point to the same data.

- 3.** Create the `/home/student/tempdir` symbolic link that points to the `/tmp` directory on the `servera` machine. Verify the newly created symbolic link.

- 3.1.** Create the `/home/student/tempdir` symbolic link to the `/tmp` directory.

```
[student@servera ~]$ ln -s /tmp /home/student/tempdir
```

- 3.2.** Use the `ls -l` command to verify the newly created symbolic link.

```
[student@servera ~]$ ls -l /home/student/tempdir
lrwxrwxrwx. 1 student student 4 May 27 13:53 /home/student/tempdir -> /tmp
```

- 4.** Return to the workstation system as the student user.

```
[student@servera ~]$ exit
logout
Connection to servera closed.
student@workstation:~$
```

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish files-make
```


7.5. Matching File Names with Shell Expansions

Objectives

- Efficiently run commands that affect many files by using pattern matching features of the Bash shell.

Command-line Expansions

When you type a command at the Bash shell prompt, the shell processes that command line through multiple *expansions* before running it. You can use these shell expansions to perform complex tasks that would otherwise be difficult or impossible.

The Bash shell performs the following expansions:

- *Pathname expansion*, which helps to select one or more files by pattern matching.
- *Brace expansion*, which can generate multiple strings of characters.
- *Tilde expansion*, which expand to a path to a user home directory.
- *Variable expansion*, which replaces text with the value that is stored in a shell variable.
- *Command substitution*, which replaces text with the output of a command.

Pathname Expansion and Pattern Matching

Pathname expansion, historically called *globbing*, is one of the most useful features of Bash. Pathname expansion uses wildcards, which are *metacharacters* that expand to match the file and path names that you are looking for. As a result, commands can act on a focused set of files during a single execution. This feature helps system administrators to manage many files.

Pathname expansion expands a pattern of special characters that represent wildcards or classes of characters into a list of file names that match the pattern. Before the shell runs your command, it replaces the pattern with the list of file names that match. If the pattern does not match anything, then the shell tries to use the pattern as a literal argument for the command that it runs.

The following table lists common metacharacters and pattern classes that are used for pattern matching.

Table 11. Table of Metacharacters and Matches

Pattern	Matches
*	Any string of zero or more characters
?	Any single character
[abc...]	Any one character in the enclosed class (between the square brackets)
[!abc...]	Any one character <i>not</i> in the enclosed class
[^abc...]	Any one character <i>not</i> in the enclosed class
[:alpha:]	Any alphabetic character
[:lower:]	Any lowercase character
[:upper:]	Any uppercase character
[:alnum:]	Any alphabetic character or digit
[:punct:]	Any printable character that is not a space or alphanumeric
[:digit:]	Any single digit from 0 to 9
[:space:]	Any single white space character, which might include tabs, newlines, carriage returns, form feeds, or spaces

For the next example, imagine that you ran the following commands to create some sample files:


```

user@host:~$ mkdir glob; cd glob
user@host:~/glob$ touch alfa bravo charlie delta echo able baker cast dog easy
user@host:~/glob$ ls
able alfa baker bravo cast charlie delta dog easy echo
user@host:~/glob$

```

In the next example, the first two commands use simple pattern matches with the asterisk (*) to match all the file names that start with a and all the file names that contain an a, respectively. The third command uses the asterisk and square brackets to match all the file names that start with a or c.

```

user@host:~/glob$ ls a*
able alfa
user@host:~/glob$ ls *a*
able alfa baker bravo cast charlie delta easy
user@host:~/glob$ ls [ac]*
able alfa cast charlie

```

The next example uses question marks (?) to match file names. The two commands match only file names that are four and five characters long, respectively.

```

user@host:~/glob$ ls ???
able cast easy echo
user@host:~/glob$ ls ?????
baker bravo delta

```

Brace Expansion

Brace expansion generates discretionary strings of characters. Braces contain a comma-separated list of strings, or a sequence of expression. The result includes the text that precedes or follows the brace definition. Brace expansions might be nested, one inside another. You can also use double-dot syntax (. .), which expands to a sequence.

For example, the {m. .p} double-dot syntax inside braces expands to m n o p.

```

user@host:~/glob$ echo {Sunday,Monday,Tuesday,Wednesday}.log
Sunday.log Monday.log Tuesday.log Wednesday.log
user@host:~/glob$ echo file{1..3}.txt
file1.txt file2.txt file3.txt
user@host:~/glob$ echo file{a..c}.txt
filea.txt fileb.txt filec.txt
user@host:~/glob$ echo file{a,b}{1,2}.txt
filea1.txt filea2.txt fileb1.txt fileb2.txt
user@host:~/glob$ echo file{a{1,2},b,c}.txt
filea1.txt filea2.txt fileb.txt filec.txt

```

A practical use of brace expansion is to create multiple files or directories.

```

user@host:~/glob$ mkdir ../RHEL{8,9,10}
user@host:~/glob$ ls ../RHEL*
RHEL8 RHEL9 RHEL10

```

Tilde Expansion

In the Bash shell, the tilde character (~) is a shortcut for the current user's home directory. You can also use a tilde (~) followed by a username to resolve the home directory of the provided user.

In the following example, the tilde is a shortcut for the home directory of the current user and for the home directory of a different user (manager1).

```

user@host:~/scripts$ echo ~
/home/user
user@host:~/scripts$ echo ~manager1
/home/manager1

```

This shortcut is especially useful for referring to files inside your home directory from other directories. To do so, you use the tilde character (~), add a forward slash (/), and add the path of the file or directory relative to your home directory. In the following example, you refer to the /home/user/scripts/run-app.sh file from the /var/log directory by using the shortcut:

```

user@host:/var/log$ ls ~/scripts/run-app.sh
/home/user/scripts/run-app.sh

```

The Bash shell executes the home directory substitution based on a few factors. If the tilde is followed by a string of characters other than a forward slash (/), then the shell interprets the string up to that slash as a username. If the substring matches an actual username, then the shell replaces the string

with the absolute path to that user's home directory. If the substring does not match a username, then the shell uses an actual tilde followed by the string of characters.

In the following example, the root user uses the shortcut to list the `scripts/run-app.sh` file in the user home directory from the `/var/log` directory. Then, the root user tries to list the same file, but for the nonexistent `dev1` user. In the second command, the shell interprets the tilde (`~`) as a regular character because the `dev1` user does not exist.

```
root@host:/var/log# ls ~user/scripts/run-app.sh
/home/user/scripts/run-app.sh
root@host:/var/log# ls ~dev1/scripts/run-app.sh
ls: cannot access '~dev1/scripts/run-app.sh': No such file or directory
```

Variable Expansion

A variable acts like a named container that stores a value in memory. Variables simplify accessing and modifying the stored data either from the command line or within a shell script.

You can assign data as a value to a variable with the following syntax.

```
user@host:~$ VARIABLENAME=value
```

You can use variable expansion to convert the variable name to its value on the command line. If a string starts with a dollar sign (`$`), then the shell tries to use the rest of that string as a variable name and replaces it with the variable value.

```
user@host:~$ USERNAME=operator
user@host:~$ echo $USERNAME
operator
```

To prevent mistakes due to other shell expansions, you can put the name of the variable in curly braces, for example `${VARIABLENAME}`.

```
user@host:~$ USERNAME=operator
user@host:~$ echo ${USERNAME}
operator
```

Variable names can contain only letters (uppercase and lowercase), numbers, and underscores. Variable names are case-sensitive and cannot start with a number.

Command Substitution

Command substitution enables the output of a command to replace the command itself on the command line. Command substitution occurs when you enclose a command in parentheses and precede it by a dollar sign (\$). The `$(command)` form can nest multiple command expansions inside each other.

```
user@host:~/glob$ echo Today is $(date +%A).
Today is Wednesday.
user@host:~/glob$ echo The time is $(date +%M) minutes past $(date +%l%p).
The time is 26 minutes past 11AM.
```

Note

An earlier form of command substitution uses backticks: ``command``. Although the Bash shell still accepts this format, try to avoid it because you might visually confuse backticks with single quotation marks, and backticks cannot be nested.

Protecting Arguments from Expansion

Many characters have a special meaning in the Bash shell. To prevent shell expansions on parts of your command line, you can quote and escape characters and strings.

The backslash (`\`) is an escape character in the Bash shell. It protects the following character from expansion.

```
user@host:~/glob$ echo The value of $HOME is your home directory.
The value of /home/user is your home directory.
user@host:~/glob$ echo The value of \$HOME is your home directory.
The value of $HOME is your home directory.
```

In the preceding example, with the dollar sign protected from expansion, Bash treats it as a regular character, and does not expand the `$HOME` variable.

To protect longer character strings, you can use single quotation marks (`'`) or double quotation marks (`"`) to enclose strings. They have slightly different effects. Single quotation marks stop all shell expansion. Double quotation marks stop *most* shell expansion.

Double quotation marks suppress special characters other than the dollar sign (`$`), backslash (`\`), backtick (```), and exclamation point (`!`) from operating inside the quoted text. Double

quotation marks block pathname expansion, but still allow command substitution and variable expansion to occur.

```
user@host:~/glob$ myhost=$(hostname -s); echo $myhost
host
user@host:~/glob$ echo "***** hostname is ${myhost} *****"
***** hostname is host *****
```

Use single quotation marks to interpret *all* text between the quotes literally.

```
user@host:~/glob$ echo "Will variable $myhost evaluate to $(hostname -s)?"
Will variable host evaluate to host?
user@host:~/glob$ echo 'Will variable $myhost evaluate to $(hostname -s)?'
Will variable $myhost evaluate to $(hostname -s)?
```

Important

Make sure to not confuse the single quotation mark (`'`) and the command substitution backtick (```), on both the screen and the keyboard. The use of one character when you mean to use the other leads to unexpected shell behavior.

References

`bash(1)`, `cd(1)`, `glob(7)`, `isalpha(3)`, `ls(1)`, `path_resolution(7)`, and `pwd(1)` man pages

7.6. Quiz

Match File Names with Shell Expansions

Choose the correct answers to the following questions:

1. Which pattern matches only file names that end with b?

- ☐ a. b*
- ☐ b. *b
- ☐ c. *b*
- ☐ d. [!b]*

2. Which pattern matches only file names that begin with b?

- ☐ a. b*
- ☐ b. *b
- ☐ c. *b*
- ☐ d. [!b]*

3. Which pattern matches only file names where the first character is not b?

- ☐ a. b*
- ☐ b. *b
- ☐ c. *b*
- ☐ d. [!b]*

4. Which pattern matches all file names that contain a b?

- ☐ a. b*
- ☐ b. *b
- ☐ c. *b*
- ☐ d. [!b]*

5. Which pattern matches only file names that contain a number?

- ☐ a. `*#*`
- ☐ b. `*[[:digit:]]*`
- ☐ c. `*[digit]*`
- ☐ d. `[0-9]`

6. Which pattern matches only file names that begin with an uppercase letter?

- ☐ a. `^?*`
- ☐ b. `^*`
- ☐ c. `[upper]*`
- ☐ d. `[[:upper:]]*`
- ☐ e. `[[CAP]]*`

7. Which pattern matches only file names with at least three characters?

- ☐ a. `???*`
- ☐ b. `???`
- ☐ c. `\3*`
- ☐ d. `+*`
- ☐ e. `...*`

Solution

1. Which pattern matches only file names that end with b?

- ☐ a. `b*`
- ☒ b. `*b`
- ☐ c. `*b*`
- ☐ d. `[!b]*`

2. Which pattern matches only file names that begin with b?

- ☒ a. `b*`
- ☐ b. `*b`
- ☐ c. `*b*`
- ☐ d. `[!b]*`

3. Which pattern matches only file names where the first character is not b?

- ☐ a. `b*`
- ☐ b. `*b`
- ☐ c. `*b*`
- ☒ d. `[!b]*`

4. Which pattern matches all file names that contain a b?

- ☐ a. `b*`
- ☐ b. `*b`
- ☒ c. `*b*`
- ☐ d. `[!b]*`

5. Which pattern matches only file names that contain a number?

- ☐ a. `*#*`
- ☒ b. `*[[:digit:]]*`
- ☐ c. `*[digit]*`
- ☐ d. `[0-9]`

6. Which pattern matches only file names that begin with an uppercase letter?

- ☒ a. `^?*`
- ☒ b. `^*`
- ☒ c. `[upper]*`
- ☒ d. `[[ :upper: ]]*`
- ☒ e. `[[CAP]]*`

7. Which pattern matches only file names with at least three characters?

- ☒ a. `???*`
- ☒ b. `???`
- ☒ c. `\3*`
- ☒ d. `+*`
- ☒ e. `...*`

7.7. Lab

Manage Files from the Command Line

Copy, move, create, delete, and organize files from the command line.

Outcomes

- Use wildcards to locate and manipulate files.

Prerequisites

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
student@workstation:~$ lab start files-review
```

Instructions

1. As the student user on the workstation machine, log in to the serverb machine as the student user.
2. Create a directory named `project_plans` in the Documents directory. The Documents directory is placed in the student user's home directory. Create two empty files in the `project_plans` directory named `season1_project_plan.odf` and `season2_project_plan.odf`.

Hint: If the `~/Documents` directory does not exist, then use the `mkdir` command `-p` option to create it.

3. Create sets of empty practice files to use in this lab. If you do not immediately recognize the intended shell expansion shortcut, then use the solution to learn and practice. Use shell tab completion to locate file path names.

Create 12 files with `tv_seasonX_episodeY.ogg` names in the `/home/student` directory. Replace the `X` placeholder with the season number and the `Y` placeholder with that season's episode, for two seasons of six episodes each.

4. As the author of a successful series of mystery novels, you are editing your next bestseller's chapters for publishing. Create eight files with `mystery_chapterX.odf` names. Replace the `X` placeholder with the numbers 1 through 8.

5. Use a single command to create the `~/Videos/season1` and `~/Videos/season2` directories to organize the TV episodes. Move the appropriate TV episodes into the season subdirectories. Use only two commands, and specify destinations with relative syntax.
6. Create a two-level directory hierarchy with a single command to organize the mystery book chapters. Create the `my_bestseller` subdirectory under the Documents directory, and create the `chapters` subdirectory under the new `my_bestseller` directory. Create three more subdirectories directly under the `my_bestseller` directory with a single command. Name these subdirectories `editor`, `changes`, and `vacation`. You do not need to use the `mkdir -p` command to create parents because the `my_bestseller` parent directory exists.
7. Change to the `chapters` directory. Use the tilde (`~`) home directory shortcut to move all book chapters to the `chapters` directory, which is now your current directory. Use the simplest syntax to specify the destination directory.

You want to send the first two chapters to the editor for review. Move only those two chapters to the `editor` directory to avoid modifying them during the review. Starting from the `chapters` subdirectory, use brace expansion with a range to specify the chapter file names to move and a relative path for the destination directory.

While on vacation, you intend to write chapters 7 and 8. Use a single command to move the files from the `chapters` directory to the `vacation` directory. Specify the chapter file names by using brace expansion with a list of strings and without using wildcard characters.

8. Change your working directory to `~/Videos/season2`, and then copy the first episode of the season to the `vacation` directory. Use a single `cd` command to change from your working directory to the `~/Documents/my_bestseller/vacation` directory. List its files. Use the *previous working directory* argument to return to the `season2` directory. This argument succeeds if the last directory change with the `cd` command used only one command rather than several `cd` commands. From the `season2` directory, copy the episode 2 file into the `vacation` directory. Use the shortcut again to return to the `vacation` directory.
9. The authors of chapters 5 and 6 want to experiment with possible changes. Copy both files from the `~/Documents/my_bestseller/chapters` directory to the `~/Documents/my_bestseller/changes` directory to prevent these changes from modifying original files.

Go to the `~/Documents/my_bestseller` directory. Use square-bracket pattern matching to specify which chapter numbers to match in the `filename` argument of the `cp` command.

10. Change your current directory to the `changes` directory and use the `date +%F` command with command substitution to copy the `mystery_chapter5.odf` file to a new file that includes the full date. Use the `mystery_chapter5_YYYY-MM-DD.odf` name format.

By using command substitution with the `date +%s` command, make another copy of the `mystery_chapter5.odf` file, and append the current time stamp (as the number of seconds since the epoch, 1970-01-01 00:00 UTC) to ensure a unique file name.

11. After further review, you decide that you do not need the `plot changes` directory. Delete the `changes` directory.

If it is necessary, then go to the `changes` directory and delete all the files in the directory. You cannot delete a directory when it is the current working directory.

Change to the parent directory of the `changes` directory. Try to delete the empty directory by using the `rm` command without the `-r` recursive option. This attempt should fail. Finally, use the `rmdir` command to delete the empty directory, which succeeds.

When the vacation is over, you no longer need the `vacation` directory. Delete it by using the `rm` command with the *recursive* option.

When finished, return to the `student` user's home directory.

12. Create a hard link to the `~/Documents/project_plans/season2_project_plan.odf` file named `~/Documents/backups/season2_project_plan.odf.back`. A hard link protects against accidental deletion of the original file and keeps the backup file updated as you change the original file.

Hint: If the `~/Documents/backups` directory does not exist, then use the `mkdir` command to create it.

13. After completing the tasks of this activity, return to the workstation machine as the `student` user.

Evaluation

As the `student` user on the workstation machine, use the `lab` command to grade your work.

Correct any reported failures and rerun the command until successful.

```
student@workstation:~$ lab grade files-review
```

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish files-review
```

Solution

Copy, move, create, delete, and organize files from the command line.

Outcomes

- Use wildcards to locate and manipulate files.

Prerequisites

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
student@workstation:~$ lab start files-review
```

Instructions

1. As the student user on the workstation machine, log in to the serverb machine as the student user.

```
student@workstation:~$ ssh student@serverb  
...output omitted...
```

2. Create a directory named `project_plans` in the Documents directory. The Documents directory is placed in the student user's home directory. Create two empty files in the `project_plans` directory named `season1_project_plan.odf` and `season2_project_plan.odf`.

Hint: If the `~/Documents` directory does not exist, then use the `mkdir` command `-p` option to create it.

```
[student@serverb ~]$ mkdir -p ~/Documents/project_plans
[student@serverb ~]$ touch \
~/Documents/project_plans/{season1,season2}_project_plan.odf
[student@serverb ~]$ ls -lR Documents/
Documents/:
total 0
drwxr-xr-x. 2 student student 70 Jun  3 09:03 project_plans

Documents/project_plans:
total 0
-rw-r--r--. 1 student student 0 Jun  3 09:02 season1_project_plan.odf
-rw-r--r--. 1 student student 0 Jun  3 09:02 season2_project_plan.odf
```

3. Create sets of empty practice files to use in this lab. If you do not immediately recognize the intended shell expansion shortcut, then use the solution to learn and practice. Use shell tab completion to locate file path names.

Create 12 files with `tv_seasonX_episodeY.ogg` names in the `/home/student` directory. Replace the `X` placeholder with the season number and the `Y` placeholder with that season's episode, for two seasons of six episodes each.

```
[student@serverb ~]$ touch tv_season{1..2}_episode{1..6}.ogg
[student@serverb ~]$ ls tv*
tv_season1_episode1.ogg  tv_season1_episode5.ogg  tv_season2_episode3.ogg
tv_season1_episode2.ogg  tv_season1_episode6.ogg  tv_season2_episode4.ogg
tv_season1_episode3.ogg  tv_season2_episode1.ogg  tv_season2_episode5.ogg
tv_season1_episode4.ogg  tv_season2_episode2.ogg  tv_season2_episode6.ogg
```

4. As the author of a successful series of mystery novels, you are editing your next bestseller's chapters for publishing. Create eight files with `mystery_chapterX.odf` names. Replace the `X` placeholder with the numbers 1 through 8.

```
[student@serverb ~]$ touch mystery_chapter{1..8}.odf
[student@serverb ~]$ ls mys*
mystery_chapter1.odf  mystery_chapter4.odf  mystery_chapter7.odf
mystery_chapter2.odf  mystery_chapter5.odf  mystery_chapter8.odf
mystery_chapter3.odf  mystery_chapter6.odf
```

5. Use a single command to create the `~/Videos/season1` and `~/Videos/season2` directories to organize the TV episodes. Move the appropriate TV episodes into the season subdirectories. Use

only two commands, and specify destinations with relative syntax.

- 5.1.** Create two subdirectories named `season1` and `season2` under the `Videos` directory by using a single command.

```
[student@serverb ~]$ mkdir -p Videos/season{1..2}
[student@serverb ~]$ ls Videos
season1  season2
```

- 5.2.** Move the appropriate TV episodes into the season subdirectories by using only two commands.

```
[student@serverb ~]$ mv tv_season1* Videos/season1
[student@serverb ~]$ mv tv_season2* Videos/season2
[student@serverb ~]$ ls -R Videos
Videos:
season1  season2

Videos/season1:
tv_season1_episode1.ogg  tv_season1_episode3.ogg  tv_season1_episode5.ogg
tv_season1_episode2.ogg  tv_season1_episode4.ogg  tv_season1_episode6.ogg

Videos/season2:
tv_season2_episode1.ogg  tv_season2_episode3.ogg  tv_season2_episode5.ogg
tv_season2_episode2.ogg  tv_season2_episode4.ogg  tv_season2_episode6.ogg
```

- 6.** Create a two-level directory hierarchy with a single command to organize the mystery book chapters. Create the `my_bestseller` subdirectory under the `Documents` directory, and create the `chapters` subdirectory under the new `my_bestseller` directory. Create three more subdirectories directly under the `my_bestseller` directory with a single command. Name these subdirectories `editor`, `changes`, and `vacation`. You do not need to use the `mkdir -p` command to create parents because the `my_bestseller` parent directory exists.

- 6.1.** Create the `my_bestseller` directory under the `Documents` directory. Create the `chapters` directory under the `my_bestseller` directory.


```
[student@serverb ~]$ mkdir -p Documents/my_bestseller/chapters
[student@serverb ~]$ ls -R Documents
Documents:
my_bestseller  project_plans

Documents/my_bestseller:
chapters

Documents/my_bestseller/chapters:

Documents/project_plans:
season1_project_plan.odf  season2_project_plan.odf
```

6.2. Create three directories named `editor`, `changes`, and `vacation`, under the `my_bestseller` directory by using a single command.

```
[student@serverb ~]$ mkdir Documents/my_bestseller/{editor,changes,vacation}
[student@serverb ~]$ ls -R Documents
Documents:
my_bestseller  project_plans

Documents/my_bestseller:
changes  chapters  editor  vacation

Documents/my_bestseller/changes:

Documents/my_bestseller/chapters:

Documents/my_bestseller/editor:

Documents/my_bestseller/vacation:

Documents/project_plans:
season1_project_plan.odf  season2_project_plan.odf
```

7. Change to the `chapters` directory. Use the tilde (`~`) home directory shortcut to move all book chapters to the `chapters` directory, which is now your current directory. Use the simplest syntax to specify the destination directory.

You want to send the first two chapters to the `editor` for review. Move only those two chapters to the `editor` directory to avoid modifying them during the review. Starting from the `chapters` subdirectory, use brace expansion with a range to specify the chapter file names to move and a relative path for the destination directory.

While on vacation, you intend to write chapters 7 and 8. Use a single command to move the files from the `chapters` directory to the `vacation` directory. Specify the chapter file names by using brace expansion with a list of strings and without using wildcard characters.

- 7.1.** Change to the `chapters` directory and use the tilde (`~`) home directory shortcut to move all book chapters to the `chapters` directory.

```
[student@serverb ~]$ cd Documents/my_bestseller/chapters
[student@serverb chapters]$ mv ~/mystery_chapter* .
[student@serverb chapters]$ ls
mystery_chapter1.odf  mystery_chapter4.odf  mystery_chapter7.odf
mystery_chapter2.odf  mystery_chapter5.odf  mystery_chapter8.odf
mystery_chapter3.odf  mystery_chapter6.odf
```

- 7.2.** Move the first two chapters to the `editor` directory. Use brace expansion with a range to specify the chapter file names to move and a relative path for the destination directory.

```
[student@serverb chapters]$ mv mystery_chapter{1..2}.odf ../editor
[student@serverb chapters]$ ls
mystery_chapter3.odf  mystery_chapter5.odf  mystery_chapter7.odf
mystery_chapter4.odf  mystery_chapter6.odf  mystery_chapter8.odf
[student@serverb chapters]$ ls ../editor
mystery_chapter1.odf  mystery_chapter2.odf
```

- 7.3.** Use a single command to move chapters 7 and 8 from the `chapters` directory to the `vacation` directory. Specify the chapter file names by using brace expansion with a list of strings and without using wildcard characters.

```
[student@serverb chapters]$ mv mystery_chapter{7,8}.odf ../vacation
[student@serverb chapters]$ ls
mystery_chapter3.odf  mystery_chapter5.odf
mystery_chapter4.odf  mystery_chapter6.odf
[student@serverb chapters]$ ls ../vacation
mystery_chapter7.odf  mystery_chapter8.odf
```

- 8.** Change your working directory to `~/Videos/season2`, and then copy the first episode of the season to the `vacation` directory. Use a single `cd` command to change from your working directory to the `~/Documents/my_bestseller/vacation` directory. List its files. Use the *previous working directory* argument to return to the `season2` directory. This argument succeeds if the last directory change with the `cd` command used only one command rather than several `cd` commands. From the `season2` directory, copy the episode 2 file into the `vacation` directory. Use the shortcut again to return to the `vacation` directory.

- 8.1.** Change your working directory to `~/Videos/season2`, and then copy the first episode of the season to the vacation directory.

```
[student@serverb chapters]$ cd ~/Videos/season2
[student@serverb season2]$ cp *episode1.ogg ~/Documents/my_bestseller/vacation
```

- 8.2.** Use a single `cd` command to change from your working directory to the `~/Documents/my_bestseller/vacation` directory, list its files, and use the `-` argument to return to the previous directory. Copy the episode 2 file into the vacation directory. Use the `cd` command with the `-` argument to return to the vacation directory.

```
[student@serverb season2]$ cd ~/Documents/my_bestseller/vacation
[student@serverb vacation]$ ls
mystery_chapter7.odf  mystery_chapter8.odf  tv_season2_episode1.ogg
[student@serverb vacation]$ cd -
/home/student/Videos/season2
[student@serverb season2]$ cp *episode2.ogg ~/Documents/my_bestseller/vacation
[student@serverb season2]$ cd -
/home/student/Documents/my_bestseller/vacation
[student@serverb vacation]$ ls
mystery_chapter7.odf  tv_season2_episode1.ogg
mystery_chapter8.odf  tv_season2_episode2.ogg
```

- 9.** The authors of chapters 5 and 6 want to experiment with possible changes. Copy both files from the `~/Documents/my_bestseller/chapters` directory to the `~/Documents/my_bestseller/changes` directory to prevent these changes from modifying original files.

Go to the `~/Documents/my_bestseller` directory. Use square-bracket pattern matching to specify which chapter numbers to match in the `filename` argument of the `cp` command.

```
[student@serverb vacation]$ cd ~/Documents/my_bestseller
[student@serverb my_bestseller]$ cp chapters/mystery_chapter[56].odf changes
[student@serverb my_bestseller]$ ls chapters
mystery_chapter3.odf  mystery_chapter5.odf
mystery_chapter4.odf  mystery_chapter6.odf
[student@serverb my_bestseller]$ ls changes
mystery_chapter5.odf  mystery_chapter6.odf
```

- 10.** Change your current directory to the `changes` directory and use the `date +%F` command with command substitution to copy the `mystery_chapter5.odf` file to a new file that includes the full

date. Use the `mystery_chapter5_YYYY-MM-DD.odf` name format.

By using command substitution with the `date +%s` command, make another copy of the `mystery_chapter5.odf` file, and append the current time stamp (as the number of seconds since the epoch, 1970-01-01 00:00 UTC) to ensure a unique file name.

```
[student@serverb my_bestseller]$ cd changes
[student@serverb changes]$ cp mystery_chapter5.odf \
mystery_chapter5_$(date +%F).odf
[student@serverb changes]$ cp mystery_chapter5.odf \
mystery_chapter5_$(date +%s).odf
[student@serverb changes]$ ls
mystery_chapter5_1748944514.odf  mystery_chapter5.odf
mystery_chapter5_2025-06-03.odf  mystery_chapter6.odf
```

11. After further review, you decide that you do not need the plot changes. Delete the changes directory.

If it is necessary, then go to the changes directory and delete all the files in the directory. You cannot delete a directory when it is the current working directory.

Change to the parent directory of the changes directory. Try to delete the empty directory by using the `rm` command without the `-r` recursive option. This attempt should fail. Finally, use the `rmdir` command to delete the empty directory, which succeeds.

When the vacation is over, you no longer need the vacation directory. Delete it by using the `rm` command with the *recursive* option.

When finished, return to the student user's home directory.

- 11.1. Delete the changes directory. Change to the parent directory of the changes directory, and try to delete the empty directory by using the `rm` command without the `-r` recursive option, which should fail. Use the `rmdir` command to delete the empty directory.

```
[student@serverb changes]$ rm mystery*
[student@serverb changes]$ cd ..
[student@serverb my_bestseller]$ rm changes
rm: cannot remove 'changes': Is a directory
[student@serverb my_bestseller]$ rmdir changes
[student@serverb my_bestseller]$ ls
chapters  editor  vacation
```

11.2. Delete the vacation directory by using the `rm` command with the `-r` option. Return to the student user's home directory.

```
[student@serverb my_bestseller]$ rm -r vacation
[student@serverb my_bestseller]$ ls
chapters  editor
[student@serverb my_bestseller]$ cd
[student@serverb ~]$
```

12. Create a hard link to the `~/Documents/project_plans/season2_project_plan.odf` file named `~/Documents/backups/season2_project_plan.odf.back`. A hard link protects against accidental deletion of the original file and keeps the backup file updated as you change the original file.

Hint: If the `~/Documents/backups` directory does not exist, then use the `mkdir` command to create it.

12.1. Create a hard link to the `~/Documents/project_plans/season2_project_plan.odf` file named `~/Documents/backups/season2_project_plan.odf.back`.

```
[student@serverb ~]$ mkdir ~/Documents/backups
[student@serverb ~]$ ln ~/Documents/project_plans/season2_project_plan.odf \
~/Documents/backups/season2_project_plan.odf.back
[student@serverb ~]$ ls -lR ~/Documents/
/home/student/Documents/:
total 0
drwxr-xr-x. 2 student student 43 Jun  3 10:01 backups
drwxr-xr-x. 4 student student 36 Jun  3 09:58 my_bestseller
drwxr-xr-x. 2 student student 70 Jun  3 09:03 project_plans

/home/student/Documents/backups:
total 0
-rw-r--r--. 2 student student 0 Jun  3 09:02 season2_project_plan.odf.back

/home/student/Documents/my_bestseller:
total 0
drwxr-xr-x. 2 student student 118 Jun  3 09:20 chapters
drwxr-xr-x. 2 student student  62 Jun  3 09:19 editor

/home/student/Documents/my_bestseller/chapters:
total 0
-rw-r--r--. 1 student student 0 Jun  3 09:08 mystery_chapter3.odf
-rw-r--r--. 1 student student 0 Jun  3 09:08 mystery_chapter4.odf
-rw-r--r--. 1 student student 0 Jun  3 09:08 mystery_chapter5.odf
-rw-r--r--. 1 student student 0 Jun  3 09:08 mystery_chapter6.odf

/home/student/Documents/my_bestseller/editor:
total 0
-rw-r--r--. 1 student student 0 Jun  3 09:08 mystery_chapter1.odf
-rw-r--r--. 1 student student 0 Jun  3 09:08 mystery_chapter2.odf

/home/student/Documents/project_plans:
total 0
-rw-r--r--. 1 student student 0 Jun  3 09:02 season1_project_plan.odf
-rw-r--r--. 2 student student 0 Jun  3 09:02 season2_project_plan.odf
```

Notice that the link count is 2 for both `season2_project_plan.odf.back` and `season2_project_plan.odf` files.

13. After completing the tasks of this activity, return to the workstation machine as the student user.

```
[student@serverb ~]$ exit
logout
Connection to serverb closed.
student@workstation:~$
```

Evaluation

As the student user on the workstation machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

```
student@workstation:~$ lab grade files-review
```

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish files-review
```

Chapter 8.

Editing Text Files

Goal

Create, view, and edit text files from the command line.

Sections

- Editing Text Files from the Command Line (and Guided Exercise)

8.1. Editing Text Files from the Command Line

Objectives

- Create, view, and edit text files by using the Vim editor.

Editing Files with Vim

The fundamental design principle of Linux is that it supports storing information and configuration settings in text-based files. These files follow various structures such as lists of settings, INI-like formats, structured XML or YAML, and others. The advantage of storing files in a text-based structure is that they can be edited with any text editor.

Vim is an improved version of the `vi` editor, which is distributed with Linux and UNIX systems. Vim is a highly configurable and efficient editor that provides split-screen editing, color formatting, and highlighting for editing text.

Benefits of the Vim Editor

When a system uses a text-only shell prompt, you should know how to use at least one text editor for editing files. You can then edit text-based configuration files from a terminal window or remote logins through the `ssh` command or the web console. You also do not need access to a graphical desktop to edit files on a server, and that server might not need to run a graphical desktop environment.

The key reason to learn Vim is that it is almost always installed by default on a server for editing text-based files. The *Portable Operating System Interface (POSIX)* standard specifies the `vi` editor on Linux, and many other UNIX-like operating systems largely do likewise.

Vim is also used often as the `vi` implementation on other standard operating systems or distributions. For example, macOS currently includes a lightweight installation of Vim by default. So, Vim skills that are learned for Linux might also prove useful elsewhere.

Installing Vim

You can install the Vim editor in Red Hat Enterprise Linux by using one of two packages. These packages provide different features and Vim commands for editing text-based files.

With the `vim-minimal` package, you might install the `vi` editor with core features. This lightweight installation includes only the core features and the basic `vi` command. You can open a file for editing by using the `vi` command.

```
user@host:~$ vi filename
```

Alternatively, you can use the `vim-enhanced` package to install the Vim editor. This package provides a more comprehensive set of features, an online help system, and a tutorial program. Use the `vim` command to start Vim in this enhanced mode.

```
user@host:~$ vim filename
```

The core features of the Vim editor are available in both commands.

If `vim-enhanced` is installed, then a shell alias is set so that if regular users run the `vi` command, then they automatically get the `vim` command instead. This alias does not apply to the `root` user and to other users with UIDs below 200 (which system services use).

If `vim-enhanced` is installed and a regular user wants to use the `vi` command, then they might have to use the `\vi` command to override the alias temporarily. You can use `\vi --version` and `vim --version` to compare the feature sets of the two commands.

Vim Operating Modes

The Vim editor offers various modes of operation such as *command mode*, *extended command mode*, *edit mode*, and *visual mode*. As a Vim user, always verify the current mode, because the effect of keystrokes varies between modes.

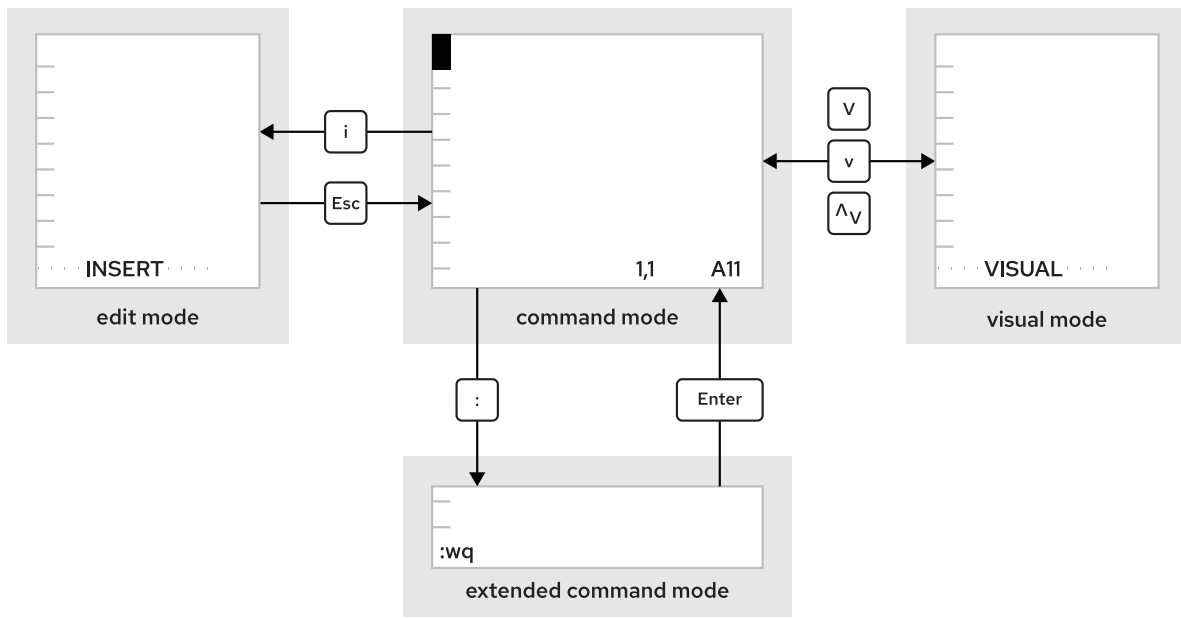


Figure 14. Moving between Vim modes

When you first open Vim, it starts in *command mode*, which is used for navigation, cut and paste, and other text modification. Pressing the required keystroke accesses specific editing functions.

- An `I` keystroke enters *insert mode*, where all typed text becomes file content. Pressing `Esc` returns to command mode.
- A `V` keystroke enters *visual mode*, where multiple characters might be selected for text manipulation. Use `Shift + V` for multiline and `Ctrl + V` for block selection. To exit the visual mode, use the `V`, `Shift + V`, or `Ctrl + V` keystrokes.
- The `:` keystroke begins *extended command mode* for tasks such as writing the file (to save it) and quitting the Vim editor.

Note

If you are unsure which mode Vim is using, then press `Esc` a few times to get back into command mode. You can safely press the `Esc` key in command mode repeatedly.

The Basic Vim Workflow

Vim has efficient, coordinated keystrokes for advanced editing tasks. Although Vim provides many benefits, Vim's capabilities can overwhelm new users.

 Note

First, take the time to familiarize yourself with the basic Vim capabilities. Then, expand your Vim vocabulary by learning more Vim keystrokes.

The exercise for this section uses the `vimtutor` command. This tutorial, from the `vim-enhanced` package, is an excellent way to learn the core Vim functions.

Red Hat recommends that you learn the following Vim keys and commands.

- The `U` key undoes the most recent edit.
- The `X` key deletes a single character.
- The `:w` command writes (saves) the file and remains in command mode for more editing.
- The `:wq` command writes (saves) the file and quits Vim.
- The `:q!` command quits Vim, and discards all file changes made after the last write.

Learning these commands helps a Vim user to accomplish any editing task.

Rearranging Existing Text

In Vim, you can *yank* and *put* (copy and paste), by using the `Y` and `P` command characters. Position the cursor on the first character to select, and then enter visual mode. Use the arrow keys to expand the visual selection. When ready, press `Y` to *yank* the selection into memory. Position the cursor at the new location, and then press `P` to *put* the selection at the cursor.

Visual Mode in Vim

Visual mode is useful to highlight and manipulate text in different lines and columns. You can enter various visual modes in Vim by using the following key combinations.

- Character mode : `V`
- Line mode : `Shift + V`
- Block mode : `Ctrl + V`

Character mode highlights sentences in a block of text. The word `VISUAL` displays at the bottom of the screen. Press `V` to enter visual character mode. Press `Shift + V` to enter line mode. `VISUAL LINE` displays at the bottom of the screen.

Visual block mode is an effective method for manipulating data files. Press the `Ctrl + V` keystroke to enter the visual block from the cursor. `VISUAL BLOCK` displays at the bottom of the screen. Use the arrow keys to highlight the section to change.

Vim Configuration Files

The `/etc/vimrc` and `~/.vimrc` configuration files alter the behavior of the Vim editor for the entire system or for a specific user, respectively. With these configuration files, you can specify behavior such as the default tab spacing, syntax highlighting, color schemes, and more. Modifying the behavior of the Vim editor is particularly useful when working with languages such as YAML, which have strict syntax requirements. Consider the following `~/.vimrc` file, which sets the default tab stop (denoted by the `ts` characters) to two spaces when editing YAML files. The file also includes the `set number` parameter to display line numbers when editing any file.

```
user@host:~$ cat ~/.vimrc
autocmd FileType yaml setlocal ts=2
set number
```

A complete list of `vimrc` configuration options is available in the references.

References

`vim(1)` man page

The `:help` command in `vim` (if the `vim-enhanced` package is installed).

[Vim Reference Manual: Vim Options](#)

8.2. Guided Exercise

Edit Text Files from the Command Line

Create, view, and edit text files by using the Vim editor.

Outcomes

- Gain competency in Vim by using the `vimtutor` command.
- Edit files with Vim.

Prerequisites

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
student@workstation:~$ lab start edit-file
```

Instructions

1. Log in to the servera machine as the student user.

```
student@workstation:~$ ssh student@servera
...output omitted...
[student@servera ~]$
```

2. Run the `vimtutor` command. Read the Welcome screen and perform *Lesson 1.1*.

Note

When running the `vimtutor` command, you might see a message indicating that a localized version of the tutorial file does not exist. This behavior is expected. Vim automatically uses the default English version, and you can continue with the tutorial.

In the presentation, keyboard arrow keys help to navigate the window. Initially, when the `vi` editor was developed, users could not rely on having arrow keys or working keyboard mappings for arrow keys to move the cursor. Therefore, the `vi` editor was initially designed to move the cursor by using commands with standard character keys, such as the conveniently grouped `H`, `J`, `K`, and `L`.

Here is a way to remember them:

hang**back**, jump**down**, kick**up**, leap**forward**.

```
[student@servera ~]$ vimtutor
```

3. In the vimtutor window, perform *Lesson 1.2*.

This lesson teaches how to quit without keeping unwanted changes. All changes are lost. In some cases, losing changes is a safer option than leaving a critical file in an incorrect state.

4. In the vimtutor window, perform *Lesson 1.3*.

Vim has fast, efficient keystrokes to delete an exact number of words, lines, sentences, or paragraphs. Any editing is possible with the **X** key for single-character deletion.

5. In the vimtutor window, perform *Lesson 1.4*.

For most editing tasks, the **I** (insert) key is pressed first.

6. In the vimtutor window, perform *Lesson 1.5*.

The previous lecture taught only the **I** key command to enter insert (or edit) mode. This lesson demonstrates other available keystrokes to change the cursor placement in insert mode. In insert mode, any typed text changes the file content.

7. In the vimtutor window, perform *Lesson 1.6*.

Type the `:wq` command to save the file and quit the editor.

8. In the vimtutor window, read the *Lesson 1 Summary*.

The vimtutor command includes six more multistep lessons. These lessons are not assigned as part of this course, but feel free to explore them to learn more.

9. Return to the workstation system as the student user.

```
[student@servera ~]$ exit
logout
Connection to servera closed.
student@workstation:~$
```

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish edit-file
```


8.3. Summary

In this lesson, you learned to create, view, and edit text files from the command line. Red Hat recommends learning to edit files with the Vim text editor because it is available in most Linux distributions.

You can use multiple keystrokes to edit a text file in a command-line text editor. You can use the `vimtutor` command to learn the core Vim functions.

You can configure the behavior of the Vim editor for the entire system or for a specific user in the `/etc/vimrc` and the `~/.vimrc` configuration files, respectively.

Chapter 9.

Redirecting Shell Input and Output

Goal

Redirect the output of commands and input to commands, by using Bash.

Sections

- Redirecting Output to Files and Programs (and Guided Exercise)
- Redirecting Shell Input and Output (Quiz)

9.1. Redirecting Output to Files and Programs

Objectives

- Save output or errors to a file with shell redirection, and process command output through multiple command-line programs with pipes.

Standard Input, Standard Output, and Standard Error

A running program, or *process*, reads input and writes output. When you run a command from the shell prompt, it typically reads its input from the keyboard and sends its output to the terminal window.

A process uses numbered channels named *file descriptors* to get input and to send output. All processes start with at least three file descriptors.

- *Standard input* (channel 0) reads input from the keyboard.
- *Standard output* (channel 1) sends regular system output to the terminal.
- *Standard error* (channel 2) sends error messages to the terminal.

If a program opens separate connections to other files, then it might use file descriptors with higher numbers.

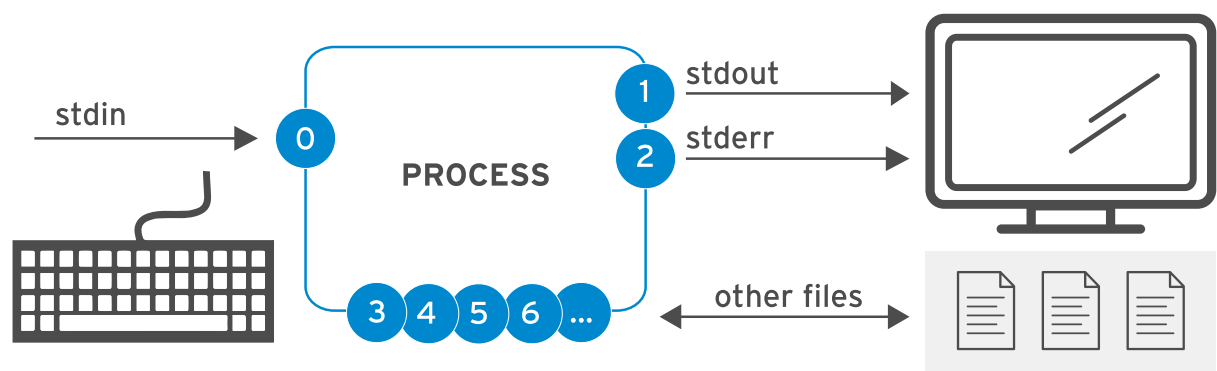


Figure 15. Process I/O channels (file descriptors)

The following table summarizes information about file descriptors:

Table 12. Channels (File Descriptors)

Number	Channel name	Description	Default connection	Usage
0	stdin	Standard input	Keyboard	Read only
1	stdout	Standard output	Terminal	Write only
2	stderr	Standard error	Terminal	Write only
3+	<i>filename</i>	Other files	None	Read, write, or both

Redirecting Output to a File

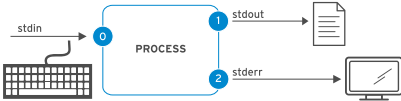
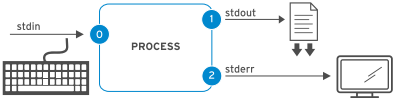
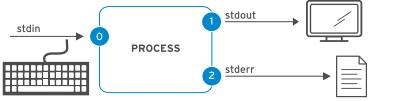
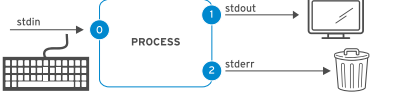
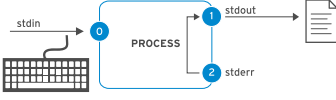
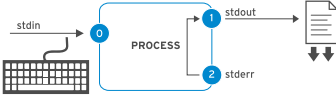
The Input/Output (I/O) redirection changes how the process gets its input or output. Instead of getting input from the keyboard, or sending output and errors to the terminal, the process can read from or write to files. With redirection, you can save messages to a file instead of displaying the output on the terminal. You can also use redirection to discard output or errors, so that the output does not appear on the terminal, or in a file.

You can redirect the standard output (stdout) of a process to suppress the process output from appearing on the terminal. If you redirect standard output to a file and the file does not exist, then the file is created. If the file does exist and the redirection does not append to the file, then the redirection overwrites the file's contents. To discard the output of a process, you can redirect to the empty `/dev/null` special file, which discards channel output that is redirected to it.

As demonstrated in the following table, redirecting *only* standard output does not suppress displaying `stderr` error messages on the terminal.

For use by Krishna Jeevan Upadhyay redhat.krishna@gmail.com redhat.krishna@gmail.com Copyright © 2025 Red Hat, Inc. | For use by Krishna Jeevan Upadhyay redhat.krishna@gmail.com redhat.krishna@gmail.com

Table 13. Output Redirection Operators

Usage	Explanation	Visual aid
> FILE	Redirect stdout to overwrite a file.	
>> FILE	Redirect stdout to append to a file.	
2> FILE	Redirect stderr to overwrite a file.	
2> /dev/null	Discard stderr error messages by redirecting them to /dev/null.	
> FILE 2>&1 &> FILE	Redirect stdout and stderr to overwrite the same file.	
>> FILE 2>&1 &>> FILE	Redirect stdout and stderr to append to the same file.	

You can combine different operators in a single command to adjust the redirection according to your needs. This feature is specially useful when you run commands that return a long mix of standard output and standard error.

For example, the `find` command returns files that comply with a specified search. But the search command might not have permissions to access certain directories that match the search, and so the command returns some errors. You can track the files found and errors encountered from the same command by redirecting each descriptor to its own file.

The following example redirects the files found to the `my-results.txt` file and redirects the errors to the `errors.txt` file.

```
user@host:~$ find /etc > my-results.txt 2> errors.txt
```

The `> my-results.txt` part of the command sends the search results to the `my-result.txt` file.

```
user@host:~$ cat my-results.txt
/etc
/etc/NetworkManager
/etc/NetworkManager/NetworkManager.conf
/etc/NetworkManager/conf.d
/etc/NetworkManager/conf.d/30-cloud-init-ip6-addr-gen-mode.conf
/etc/NetworkManager/dispatcher.d
/etc/NetworkManager/dispatcher.d/no-wait.d
...output omitted...
```

The `2> errors.txt` part of the command sends the errors to the `errors.txt` file.

```
user@host:~$ cat errors.txt
find: '/etc/audit': Permission denied
find: '/etc/credstore': Permission denied
find: '/etc/ssh/sshd_config.d': Permission denied
find: '/etc/credstore.encrypted': Permission denied
find: '/etc/polkit-1/rules.d': Permission denied
find: '/etc/polkit-1/localauthority': Permission denied
...output omitted...
```

Important

The order of redirection operations is important. The following sequence redirects standard output to the output . log file and then redirects standard error messages to the same place as standard output (output . log).

```
> output.log 2>&1
```

The next sequence redirects in the opposite order. This sequence redirects standard error messages to the default place for standard output (the terminal window, so no change) and *then* redirects only standard output to the output . log file.

```
2>&1 > output.log
```

For this reason, some people prefer to use the merging redirection operators:

- `&> output.log` instead of `> output.log 2>&1`
- `&>> output.log` instead of `>> output.log 2>&1` (in Bash 4 or RHEL 6 and later)

However, system administrators and programmers prefer to avoid the newer merging redirection operators when using alternative shells to bash (known as Bourne-compatible shells) for scripting commands. These new redirection operators are not standardized or implemented in those shells, and have other limitations.

Output Redirection Examples

Simplify many routine administration tasks by using redirection. Review the previous table, and consider the following examples.

Save a time stamp in the /tmp/saved-timestamp file for later reference.

```
user@host:~$ date > /tmp/saved-timestamp
```

Copy the last 100 lines from the `/var/log/secure` file to the `/tmp/last-100-log-secure` file.

```
user@host:~$ tail -n 100 /var/log/secure > /tmp/last-100-log-secure
```

Concatenate all four step files into one file in the `tmp` directory.

```
user@host:~$ cat step1.sh step2.log step3 step4 > /tmp/all-four-steps-in-one
```

List the home directory's hidden and regular file names, and save the output to the `my-file-names` file.

```
user@host:~$ ls -a > my-file-names
```

Append a line to the existing `/tmp/many-lines-of-information` file.

```
user@host:~$ echo "new line of information" >> /tmp/many-lines-of-information
```

The next few commands generate error messages because some system directories are inaccessible to regular users. Observe the error message redirection.

Redirect errors from the `find` command to the `/tmp/errors` file when viewing regular command output on the terminal.

```
user@host:~$ find /etc -name passwd 2> /tmp/errors
```

Save process output to the `/tmp/output` file and error messages to the `/tmp/errors` file.

```
user@host:~$ find /etc -name passwd > /tmp/output 2> /tmp/errors
```

Save process output to the `/tmp/output` file and discard error messages.

```
user@host:~$ find /etc -name passwd > /tmp/output 2> /dev/null
```

Store output and errors together to the `/tmp/all-message-output` file.


```
user@host:~$ find /etc -name passwd &> /tmp/all-message-output
```

Append output and errors to the /tmp/all-message-output file.

```
user@host:~$ find /etc -name passwd >> /tmp/all-message-output 2>&1
```

Constructing Pipelines

A *pipeline* is a sequence of one or more commands that are separated by the pipe character (`|`). A pipeline connects the standard output of the first command to the standard input of the next command.

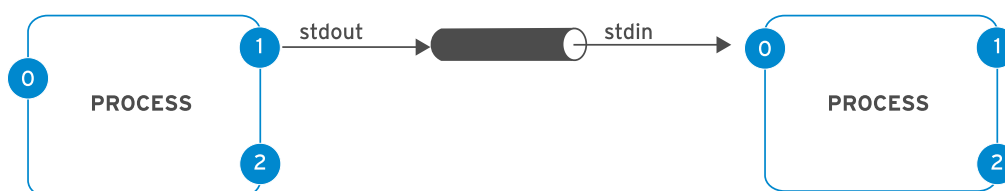


Figure 16. Process I/O piping

With pipelines, one process can manipulate and format the output of another process before the output is sent to the terminal. Imagine that data passes through the pipeline from one process to another, and is altered by each command in the pipeline that it passes through.

Note

Pipelines and I/O redirection both manipulate standard output and standard input. Pipelines send the standard output from one process to the standard input of another process. Redirection sends standard output to files, or gets standard input from files.

Pipeline Examples

The following example redirects the output of the `ls` command to the `less` command to display it on the terminal one screen at a time.

```
user@host:~$ ls -l /usr/bin | less
```

Redirect the output of the `ls` command to the `wc -l` command, which counts the number of received lines from the `ls` command and prints that value to the terminal.

```
user@host:~$ ls | wc -l
```

Redirect the output of the `ls -t` command to the `head` command to display the first 10 lines, with the final result redirected to the `/tmp/first-ten-changed-files` file.

```
user@host:~$ ls -t | head -n 10 > /tmp/first-ten-changed-files
```

Pipelines, Redirection, and Appending to a File

When you combine redirection with a pipeline, the shell sets up the entire pipeline first, and then it redirects the input/output. If you use output redirection in the *middle* of a pipeline, then the output goes to the file and not to the next command in the pipeline.

In the following example, the output of the `ls` command is redirected to the `/tmp/saved-output` file, and the `less` command displays nothing on the terminal.

```
user@host:~$ ls > /tmp/saved-output | less
```

The `tee` command overcomes this limitation. In a pipeline, the `tee` command copies its standard input to its standard output and also redirects its standard output to the files that are provided as arguments to the command. If you imagine data as water that flows through a pipeline, then you can visualize the `tee` command as a "T" joint in the pipe that directs output in two directions.

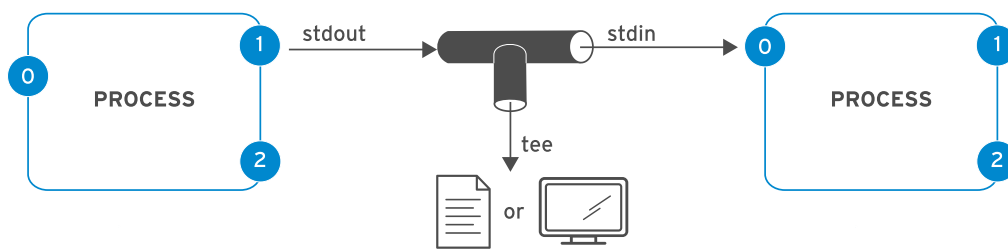


Figure 17. Process I/O piping with tee

Pipeline Examples with the tee Command

The following example redirects the output of the `ls` command to the `/tmp/saved-output` file and passes it to the `less` command, so it is displayed on the terminal one screen at a time.

```
user@host:~$ ls -l | tee /tmp/saved-output | less
```

If you use the `tee` command at the *end* of a pipeline, then the terminal shows the output of the commands in the pipeline and saves it to a file at the same time.

```
user@host:~$ ls -t | head -n 10 | tee /tmp/ten-last-changed-files
```

Use the tee command -a option to append the content to a file instead of overwriting it.

```
user@host:~$ ls -l | tee -a /tmp/append-files
```

Important

You can redirect standard error through a pipeline, but you cannot use the merging redirection operators (&> and &>>). The following example is the correct way to redirect both standard output and standard error through a pipeline:

```
user@host:~$ find / -name passwd 2>&1 | less
```

References

info bash (*The GNU Bash Reference Manual*)

- Section 3.2.3: Pipelines
- Section 3.6: Redirections

info coreutils 'tee invocation' (*The GNU coreutils Manual*)

- Section 17.1: Redirect output to multiple files or processes

bash(1), cat(1), head(1), less(1), mail(1), tee(1), tty(1), and wc(1) man pages

9.2. Guided Exercise

Redirect Output to Files and Programs

Save output or errors to a file with shell redirection, and process command output through multiple command-line programs with pipes.

Outcomes

- Use redirection to save standard output and standard error to files.
- Use pipelines to send output from one command into another.

Prerequisites

As the student user on the workstation machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
student@workstation:~$ lab start shellio-redirect
```

Instructions

In this scenario, you are a system administrator who manages a server that hosts the `my_web` application. The support team received complaints that the `my_web` application failed at some point between May 4, 2025, and May 10, 2025. Review the logs of the `my_web` application at the `/var/opt/my_web` directory and save relevant information in the specified files.

1. Examine the log files in the `/var/opt/my_web` directory on the workstation machine.

- 1.1. List the files in the `/var/opt/my_web` directory.

```
student@workstation:~$ ls /var/opt/my_web/  
20250504.log 20250505.log 20250506.log 20250507.log  
20250508.log 20250509.log 20250510.log
```

- 1.2. View the contents of the `20250504.log` file. Examine the log file for any relevant information.

```
student@workstation:~$ cat /var/opt/my_web/20250504.log
May 04 04:25:13 my_web[1]: my_web: Loading /etc/my_web/application.conf
May 04 04:17:06 my_web[4901]: my_web: Version found: 1.0.4
May 04 04:16:36 dnf[4863]: Metadata cache refreshed recently.
May 04 04:16:36 systemd[1]: dnf-makecache.service: Deactivated successfully.
May 04 04:16:36 systemd[1]: Finished dnf-makecache.service - dnf makecache.
May 04 04:24:59 systemd-logind[1051]: Session 4 logged out.
May 04 04:25:13 my_web[1]: my_web: Started successfully.
May 04 04:17:06 my_web[4901]: {
May 04 04:17:06 my_web[4901]:   "status": "OK"
May 04 04:17:06 my_web[4901]:   "msg": "All services running"
May 04 04:17:06 my_web[4901]: }
```

The first two lines of the file include the location of the configuration file and the version. The last five lines of the file contain status information.

2. Extract the first two lines of every log file to the `~/headers.txt` file. Extract the last five lines of every log file to the `~/footers.txt` file. Use redirection to save the name of any log file that contains errors to the `~/findings.txt` file.

- 2.1. Extract the first two lines of all log files to the `~/headers.txt` file.

```
student@workstation:~$ head -n 2 /var/opt/my_web/* > headers.txt
```

- 2.2. Extract the last five lines of all log files to the `~/footers.txt` file.

```
student@workstation:~$ tail -n 5 /var/opt/my_web/* > footers.txt
```

- 2.3. View the contents of the `~/headers.txt` file.

```

student@workstation:~$ cat headers.txt
==> /var/opt/my_web/20250504.log <==
May 04 04:25:13 my_web[1]: my_web: Loading /etc/my_web/application.conf
May 04 04:17:06 my_web[4901]: my_web: Version found: 1.0.4

...output omitted...

==> /var/opt/my_web/20250507.log <==
May 07 04:25:13 my_web[1]: my_web: Loading /etc/my_web/dev/application.conf
May 07 04:17:06 my_web[4901]: my_web: Version found: 2.0-beta

...output omitted...

==> /var/opt/my_web/20250510.log <==
May 10 04:25:13 my_web[1]: my_web: Loading /etc/my_web/application.conf
May 10 04:17:06 my_web[4901]: my_web: Version found: 1.0.4

```

The 20250507.log file stands out because it shows a different directory for the configuration file. The file also shows a different version of the application.

2.4. View the contents of the ~/footers.txt file.

```

student@workstation:~$ cat footers.txt
==> /var/opt/my_web/20250504.log <==
May 04 04:25:13 my_web[1]: my_web: Started successfully.
May 04 04:17:06 my_web[4901]: {
May 04 04:17:06 my_web[4901]:   "status": "OK"
May 04 04:17:06 my_web[4901]:   "msg": "All services running"
May 04 04:17:06 my_web[4901]: }
...output omitted...
==> /var/opt/my_web/20250507.log <==
May 07 04:25:13 my_web[1]: my_web: Failed.
May 07 04:17:06 my_web[4901]: {
May 07 04:17:06 my_web[4901]:   "status": "ERROR"
May 07 04:17:06 my_web[4901]:   "msg": "Could not find API version 2.0"
May 07 04:17:06 my_web[4901]: }
...output omitted...

```

The end of the 20250507.log file shows an error message and a reason for the error.

2.5. Save the name of the log file that contains errors to the ~/findings.txt file.

```

student@workstation:~$ echo "Log file with errors: 20250507.log" >
~/findings.txt

```

3. To collect more information about the error, search for all files in the `/etc` directory whose filenames end with the `.conf` string. Save the results of the search to the `~/results.txt` file and save any errors to the `~/search_errors.txt` file.

```
student@workstation:~$ find /etc -name "*.conf" > results.txt 2>
search_errors.txt
```

4. Count the number of lines in the `~/results.txt` and `~/search_errors.txt` files. Print the number of lines to the terminal and append this output to the `~/findings.txt` file in the same command. Preserve the existing content of the `~/findings.txt` file.

- 4.1. Print the number of search results and save this output to the `~/findings.txt` file.

```
student@workstation:~$ wc -l results.txt | tee -a findings.txt
386 results.txt
```

- 4.2. Print the number of search errors and append this output to the `~/findings.txt` file.

```
student@workstation:~$ wc -l search_errors.txt | tee -a findings.txt
18 search_errors.txt
```

5. Append the current date to the `~/findings.txt` file.

```
student@workstation:~$ date >> findings.txt
```

6. Verify the contents of the `~/findings.txt` file.

The contents of the file might vary in your environment.

```
student@workstation:~$ cat findings.txt
Log file with errors: 20250507.log
386 results.txt
18 search_errors.txt
Wed May 14 08:32:48 UTC 2025
```

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish shellio-redirect
```


9.3. Quiz

Redirecting Shell Input and Output

Choose the correct answers to the following questions:

1. A running program, or process, uses numbered channels named file descriptors to manage input and output. Which number is assigned to the standard output (stdout) channel?
☐ a. 0
☐ b. 1
☐ c. 2
☐ d. 3+
2. You want to save the output of a command to a file, but you need to add the new output to the end of the file without removing any existing content. Which shell redirection operator should you use?
☐ a. >
☐ b. 2>
☐ c. &>
☐ d. >>
3. You run a command that occasionally produces error messages but you do not want these error messages to be displayed on the terminal or saved in a file. Which command should you use to discard the standard error?
☐ a. &> /dev/error
☐ b. 2> /dev/error
☐ c. &> /dev/zero
☐ d. 2> /dev/null
4. You need to execute a command and save its standard output to the output.txt file and its standard error to the errors.txt file. Which command syntax correctly performs this action?
☐ a. `command > output.txt | errors.txt`
☐ b. `command & errors.txt > output.txt`
☐ c. `command > output.txt 2> errors.txt`
☐ d. `command | output.txt 2> errors.txt`

5. What is the purpose of the pipe character (|) when used between two commands?
- ☐ a. It redirects the standard error of command1 to the standard input of command2.
 - ☐ b. It redirects both the standard output and standard error of command1 to the standard input of command2.
 - ☐ c. It redirects the standard input of command1 from the standard output of command2.
 - ☐ d. It connects the standard output of command1 to the standard input of command2.
6. You are troubleshooting an application and use the `find` command to locate certain configuration files, but the command generates permission errors from directories you cannot access. You want to save both the list of found files (standard output) and the permission errors (standard error) into a single file named `find-results-and-errors.log`, overwriting the file if it exists. Which of the following commands must you append to the `find` command to perform the requested action?
- ☐ a. `&& find-results-and-errors.log`
 - ☐ b. `1&2> find-results-and-errors.log`
 - ☐ c. `2>&1 > find-results-and-errors.log`
 - ☐ d. `> find-results-and-errors.log 2>&1`
7. Consider this command sequence: `commandA > file.txt | commandB`. What happens to the standard output of `commandA`?
- ☐ a. The output is sent as standard input to `commandB`.
 - ☐ b. The output is discarded entirely because it is redirected in the middle of a pipeline.
 - ☐ c. The output is redirected to the file instead of being sent to `commandB`.
 - ☐ d. The output is sent to both the file and to `commandB`.
8. You want to process the output of a command by using a pipeline, and simultaneously save the same output to a file for later review. Which command enables you to send to both standard output and to one or more files?
- ☐ a. `cat`
 - ☐ b. `head`
 - ☐ c. `wc`
 - ☐ d. `tee`

9. You execute the command sequence: `ls -t | head -n 10 | tee -a my-list.txt`. What is the final result of this command sequence?

- ☐ a. The first 10 files sorted by modification time are displayed on the terminal, and the list overwrites the `my-list.txt` file.
- ☐ b. The first 10 files sorted by modification time are displayed on the terminal, and the list is appended to the `my-list.txt` file.
- ☐ c. The first 10 files sorted by modification time are saved to the `my-list.txt` file, and nothing is displayed on the terminal.
- ☐ d. The first 10 files sorted by modification time are discarded, and all errors are sent to the `my-list.txt` file.

Solution

1. A running program, or process, uses numbered channels named file descriptors to manage input and output. Which number is assigned to the standard output (stdout) channel?
 - ☐ a. 0
 - ☒ b. 1
 - ☐ c. 2
 - ☐ d. 3+
2. You want to save the output of a command to a file, but you need to add the new output to the end of the file without removing any existing content. Which shell redirection operator should you use?
 - ☐ a. >
 - ☐ b. 2>
 - ☐ c. &>
 - ☒ d. >>
3. You run a command that occasionally produces error messages but you do not want these error messages to be displayed on the terminal or saved in a file. Which command should you use to discard the standard error?
 - ☐ a. &> /dev/error
 - ☐ b. 2> /dev/error
 - ☐ c. &> /dev/zero
 - ☒ d. 2> /dev/null
4. You need to execute a command and save its standard output to the output.txt file and its standard error to the errors.txt file. Which command syntax correctly performs this action?
 - ☐ a. `command > output.txt | errors.txt`
 - ☐ b. `command & errors.txt > output.txt`
 - ☒ c. `command > output.txt 2> errors.txt`
 - ☐ d. `command | output.txt 2> errors.txt`

5. What is the purpose of the pipe character (|) when used between two commands?
- ☐ a. It redirects the standard error of command1 to the standard input of command2.
 - ☐ b. It redirects both the standard output and standard error of command1 to the standard input of command2.
 - ☐ c. It redirects the standard input of command1 from the standard output of command2.
 - ☒ d. **It connects the standard output of command1 to the standard input of command2.**
6. You are troubleshooting an application and use the `find` command to locate certain configuration files, but the command generates permission errors from directories you cannot access. You want to save both the list of found files (standard output) and the permission errors (standard error) into a single file named `find-results-and-errors.log`, overwriting the file if it exists. Which of the following commands must you append to the `find` command to perform the requested action?
- ☐ a. `&& find-results-and-errors.log`
 - ☐ b. `1&2> find-results-and-errors.log`
 - ☐ c. `2>&1 > find-results-and-errors.log`
 - ☒ d. `> find-results-and-errors.log 2>&1`
7. Consider this command sequence: `commandA > file.txt | commandB`. What happens to the standard output of `commandA`?
- ☐ a. The output is sent as standard input to `commandB`.
 - ☐ b. The output is discarded entirely because it is redirected in the middle of a pipeline.
 - ☒ c. **The output is redirected to the file instead of being sent to `commandB`.**
 - ☐ d. The output is sent to both the file and to `commandB`.
8. You want to process the output of a command by using a pipeline, and simultaneously save the same output to a file for later review. Which command enables you to send to both standard output and to one or more files?
- ☐ a. `cat`
 - ☐ b. `head`
 - ☐ c. `wc`
 - ☒ d. `tee`

9. You execute the command sequence: `ls -t | head -n 10 | tee -a my-list.txt`. What is the final result of this command sequence?

-  a. The first 10 files sorted by modification time are displayed on the terminal, and the list overwrites the `my-list.txt` file.
-  b. **The first 10 files sorted by modification time are displayed on the terminal, and the list is appended to the `my-list.txt` file.**
-  c. The first 10 files sorted by modification time are saved to the `my-list.txt` file, and nothing is displayed on the terminal.
-  d. The first 10 files sorted by modification time are discarded, and all errors are sent to the `my-list.txt` file.

9.4. Summary

In this lesson, you learned that Linux processes use file descriptors that you can manipulate by using redirection. With redirection, you can save standard output (`stdout`) or standard error (`stderr`) to a file, or discard it. You can use redirection operators to send `stdout` and `stderr` to the same file or device, or you can send the output to different locations.

You can connect the `stdout` of a command to the standard input (`stdin`) of another command by using pipelines. The purpose of using pipelines is to alter, display, or process output as required.

You can use redirection and pipelines to process complex command output and to write a large amount of data to files without using a graphical application.

Chapter 10.

Managing Local Users and Groups

Goal

Obtain superuser access to a system; create, manage, and delete local users and groups; and administer local password policies.

Sections

- Linux Users and Groups (and Quiz)
- Gaining Superuser Access (and Guided Exercise)
- Managing Local User Accounts (and Guided Exercise)
- Managing Local Group Accounts (and Guided Exercise)
- Managing User Passwords (and Guided Exercise)

Lab

- Manage Local Users and Groups

10.1. Linux Users and Groups

Objectives

- Describe the purpose of users and groups on a Linux system.

Linux Users

A *user* account provides security boundaries between people and programs that can run commands.

Users have *usernames* to identify them to human users and for ease of working. Internally, the system distinguishes user accounts by the unique identification number, the *user ID (UID)*, which is assigned to them. In most scenarios, if a human uses a user account, then the system assigns a secret *password* for the user to prove that they are the authorized user to log in.

User accounts are fundamental to system security. Every process (running program) on the system runs as a specific user. Every file has a specific user as its owner. With file ownership, the system enforces access control for users of the files. The user that is associated with a running process determines the files and directories that are accessible to that process.

User Types in Linux Systems

The three main types of user accounts are the *superuser*, *system users*, and *regular users*.

Superuser

The *superuser* account administers the system. The superuser name is `root` and the account's UID is 0. The superuser has full system access.

System user

The *system user* accounts are used by processes that provide supporting services. These processes, or *daemons*, usually do not need to run as the superuser and do not have a controlling terminal session. They are often assigned nonprivileged system user accounts to secure their files and other resources from each other and from regular users on the system. Users do not interactively log in with a system user account.

Regular user

Most users have *regular user* accounts for their day-to-day work. Like system users, regular users have limited access to the system.

Use the `id` command to show information about the currently logged-in user:

```
user01@host:~$ id
uid=1000(user01) gid=1000(user01) groups=1000(user01)
context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
```

To view information about another user (user02 in the following example), pass the username to the `id` command as an argument:

```
user01@host:~$ id user02
uid=1002(user02) gid=1001(user02) groups=1001(user02)
context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
```

Example Commands That Display User Information

Use the `ls -l` command to show the user that owns a file. Use the `ls -ld` command to show the user that owns a directory, rather than the contents of that directory. In the following output, the third column shows the owner's username.

```
user01@host:~$ ls -l mytextfile.txt
-rw-rw-r--. 1 user01 user01 0 Feb  5 11:10 mytextfile.txt

user01@host:~$ ls -ld Documents
drwxrwxr-x. 2 user01 user01 6 Feb  5 11:10 Documents
```

Use the `ps` command to view process information. By default, the `ps` command only displays processes that are running with the same UID as the current user and that are associated with the terminal on which the `ps` command was run.

You can add different options to change this behavior. If you use the `awwu` options together, then the `ps` command lists processes that are owned by any user that is associated with a terminal (`a`), provides more user-oriented information for each process (`u`), and does not truncate each line of output (`ww`).

In the following output, the first column shows the username of the user that owns each process.

```

user01@host:~$ ps awwu
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root      1690  0.0  0.0 220984  1052 ttyS0    Ss+   22:43   0:00 /sbin/agetty -o --
\u --noreset --noclear --keep-baud 115200,57600,38400,9600 - vt220
user01    1769  0.0  0.1 454012  5556 tty2     Ssl+  22:45   0:00 /usr/libexec/gdm-
wayland-session /usr/bin/gnome-session
user01    1800  0.0  0.3 521412 19824 tty2     Sl+   22:45   0:00 /usr/libexec/gnome-
session-binary
user01    3072  0.0  0.0 224152  5756 pts/1    Ss    22:48   0:00 /usr/bin/bash
user01    3122  0.0  0.0 225556  3652 pts/1    R+    22:49   0:00 ps awwu

```

Note

The `ps` command has two sets of similar options: UNIX-style options that start with an initial dash, and BSD-style options that do not. Each set of commands behaves slightly differently. Refer to the `ps(1)` man page for more details.

Local User Accounts

The output of most commands display users by name, but internally, the operating system uses UIDs to track users. The mappings of usernames to UIDs are defined in databases of account information. By default, systems use the `/etc/passwd` file to store information about local users.

Each line in the `/etc/passwd` file contains information about one user. The file is divided into seven colon-separated fields.

An example line from the `/etc/passwd` file provides information about the `user01` user:

```

user01@host:~$ cat /etc/passwd
...output omitted...
user01:x:1000:1000:User One:/home/user01:/bin/bash

```

Consider each part of the code block, separated by a colon:

- `user01` : The username for this user.
- `x` : The user's encrypted password was historically stored here; it is now a placeholder.
- `1000` : The UID number for this user account.
- `1000` : The GID number for this user account's primary group. Groups are discussed later in this section.
- `User One` : A brief comment, description, or the real name for this user.

- `/home/user01`: The user's home directory, and the initial working directory when the login shell starts.
- `/bin/bash`: The default shell program for this user that runs at login. Some accounts use the `/sbin/nologin` shell to disallow interactive logins with that account.

Linux Groups

A group is a collection of users that must share access to files and other system resources. By using groups, you can grant file access to a set of users rather than to a single user.

Like users, groups have *group names* for easier recognition. Internally, the system distinguishes groups by the unique identification number, the *group ID (GID)*, that is assigned to them. The mapping of group names to GIDs is defined in identity management databases of group account information. By default, systems use the `/etc/group` file to store information about local groups.

Each line in the `/etc/group` file contains information about one group. Each group entry is divided into four colon-separated fields.

An example line from the `/etc/group` file provides information about the `group01` group:

```
user01@host:~$ cat /etc/group
...output omitted...
group01:x:10000:user01,user02,user03
```

Consider each part of the code block, separated by a colon:

- `group01`: The name for this group.
- `x`: Obsolete group password field; it is now a placeholder.
- `10000`: The GID number for this group (`10000`).
- `user01,user02,user03`: A list of users that are members of this group as one of their supplementary groups.

Primary Groups and Supplementary Groups

Every user has exactly one primary group. For local users, this group is listed by GID in the `/etc/passwd` file. The primary group owns files that the user creates.

When a regular user is created, a group is created with the same name as the user, to be the primary group for the user. The user is the only member of this *User Private Group*. This group membership design simplifies the management of file permissions by separating user groups by default.

Users might also have *supplementary groups*. Membership in supplementary groups is stored in the `/etc/group` file. Users are granted access to files based on whether any of their groups have access, regardless of whether the groups are primary or supplementary. For example, if the `user01` user has a `user01` primary group and `wheel` and `webadmin` supplementary groups, then that user can read files that any of those three groups can read.

The `id` command can show group membership for a user. In the following example, the `user01` user has the `user01` group as their primary group (`gid`). The `groups` item lists all group memberships for this user, and the user also has the `wheel` and `group01` groups as supplementary groups.

```
user01@host:~$ id
uid=1001(user01) gid=1003(user01) groups=1003(user01),10(wheel),10000(group01)
context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
```

References

`id(1)`, `passwd(5)`, `group(5)`, `ls(1)`, and `ps(1)` man pages

`info libc users` (*GNU C Library Reference Manual*, "Users and Groups")

(The `glibc-devel` package must be installed for this `info` node to be available.)

10.2. Quiz

Linux Users and Groups

Choose the correct answer to the following questions:

1. Which item does Linux use to internally identify a user?

- ☐ a. Primary user
- ☐ b. UID
- ☐ c. GID
- ☐ d. Username

2. Which item or file stores local group information?

- ☐ a. Home directory
- ☐ b. /etc/passwd
- ☐ c. /etc/GID
- ☐ d. /etc/group

3. Which item or file stores the user's personal files?

- ☐ a. Home directory
- ☐ b. Login shell
- ☐ c. /etc/passwd
- ☐ d. /etc/group

4. Which item does Linux use to internally identify groups?

- ☐ a. Primary group
- ☐ b. UID
- ☐ c. GID
- ☐ d. Group key

5. Which item or file stores local user account information?

- ☐ a. Home directory
- ☐ b. /etc/passwd
- ☐ c. /etc/UID
- ☐ d. /etc/group

6. What is the fourth field in a user entry in the /etc/passwd file?

- ☐ a. Home directory
- ☐ b. UID
- ☐ c. Login shell
- ☐ d. GID of the primary group

7. Which item represents the program that provides the user's initial command-line prompt?

- ☐ a. Primary prompt
- ☐ b. Home directory
- ☐ c. Default shell
- ☐ d. Command name

Solution

1. Which item does Linux use to internally identify a user?

- ☐ a. Primary user
- ☒ b. **UID**
- ☐ c. GID
- ☐ d. Username

2. Which item or file stores local group information?

- ☐ a. Home directory
- ☐ b. /etc/passwd
- ☐ c. /etc/GID
- ☒ d. **/etc/group**

3. Which item or file stores the user's personal files?

- ☒ a. **Home directory**
- ☐ b. Login shell
- ☐ c. /etc/passwd
- ☐ d. /etc/group

4. Which item does Linux use to internally identify groups?

- ☐ a. Primary group
- ☐ b. UID
- ☒ c. **GID**
- ☐ d. Group key

5. Which item or file stores local user account information?

- ☐ a. Home directory
- ☒ b. **/etc/passwd**
- ☐ c. /etc/UID
- ☐ d. /etc/group

6. What is the fourth field in a user entry in the `/etc/passwd` file?

- ☐ a. Home directory
- ☐ b. UID
- ☐ c. Login shell
- ☒ d. **GID of the primary group**

7. Which item represents the program that provides the user's initial command-line prompt?

- ☐ a. Primary prompt
- ☐ b. Home directory
- ☒ c. **Default shell**
- ☐ d. Command name

10.3. Gaining Superuser Access

Objectives

- Switch to the superuser account to manage a Linux system, and grant other users superuser access by using Sudo.

The Superuser

Most operating systems have a *superuser* that has all power over the system. In Red Hat Enterprise Linux, the superuser is the `root` user. This user has the power to override regular privileges on the file system, and you can use it to manage and administer the system. For tasks such as installing or removing software, and to manage system files and directories, users must escalate their privileges to the `root` user.

Usually, only the `root` user can control most devices, but some exceptions apply. For example, regular users can control removable devices, such as USB devices. Thus, regular users can add and remove files and otherwise manage a removable device, but only the `root` user can manage hard drives by default.

This unlimited privilege, however, comes with responsibility. The `root` user has unlimited power to damage the system: they can remove files and directories, remove user accounts, add back doors, and so on. If the `root` user account is compromised, then the system is in danger and you might lose administrative control. Red Hat encourages system administrators to always log in as a regular user, and to escalate privileges to `root` only when needed.

Important

In RHEL 10, the `root` account has no valid password by default, so you cannot directly log in as the `root` user with a password.

The `root` account on Linux is similar to the local Administrator account on Microsoft Windows. In Linux, most system administrators log in to the system as an unprivileged user and use various tools to temporarily gain `root` privileges.

Warning

Microsoft Windows users might be familiar with the practice of logging in as the local Administrator user to perform system administrator duties. Today, this practice is not recommended; users obtain privileges to perform administration by membership in the Administrators group. Similarly in RHEL, Red Hat recommends that system administrators never log in directly as root. Instead, system administrators log in as a regular user and use mechanisms to temporarily gain superuser privileges (`su`, `sudo`, or `PolicyKit`, for example).

When logged in as root, the entire desktop environment unnecessarily runs with administrative privileges. A security vulnerability that might typically compromise only a regular user account can potentially compromise the entire system.

Switching User Accounts

With the `su` command, users can switch to a different user account. If you run the `su` command from a regular user account with another user account as a parameter, then you must provide the password of the account to switch to. When the root user runs the `su` command, you do not need to enter the user's password.

This example uses the `su` command from the `user01` account to switch to the `user02` account:

```
user01@host:~$ su - user02
Password: user02_password
user02@host:~$
```

If you omit the username, then the `su` or `su -` command attempts to switch to root by default.

```
user01@host:~$ su -
Password: root_password
root@host:~#
```

The `su` command starts an interactive *non-login shell*, but the `su -` command (with the dash option) starts an interactive *login shell*. The difference between the two commands is that the shell is initialized in slightly different ways when it starts, depending on whether it is a login shell or a non-login shell.

`su -`

Starts a new interactive login session as the user to whom you are switching, and uses the target user's environment settings.

su

Starts an interactive shell as the user to whom you are switching, but partially uses the original user's environment settings.

Generally, administrators should run the `su -` command to get a shell with the target user's environment settings. This approach ensures that the resulting shell is initialized in the same way that it would be if that user logged in as usual. For more information, refer to the `bash(1)` man page.

Running Commands with Sudo

In RHEL 10, the root account has no valid password by default. In other words, there is no password that allows a user to authenticate and to access the root account, either by attempting to log in directly as root or by attempting to use the `su` command.

Important

The system administrator could set a password for the root account, which would allow you to become the root user with the `su` command if you know that password.

For security reasons, system administrators commonly do not set a valid password for the root account. In that case, to get access to the root account, users must use a regular password to log in to a regular user account, and then use the `sudo` command to become the root user or to run commands as the root user.

Unlike the `su` command, the `sudo` command typically requires users to enter their own password for authentication, not the password of the user account that they are trying to access. That is, users who use the `sudo` command to run commands as the root user do not need to know the root password. Instead, they use their own passwords to authenticate access.

Important

By default, the `sudo` command caches your password for five minutes after a successful authentication. As a result, you do not need to enter your password multiple times if you run a quick series of `sudo` commands.

You can run the `sudo -k` command to immediately remove your password from the cache.

Additionally, you can configure the `sudo` command to allow specific users to run any command as some other user, or only some commands as that user. For example, if you configure the `sudo`

command to allow the `user01` user to run the `usermod` command as `root`, then you can run the following command to lock or unlock a user account:

```
user01@host:~$ sudo usermod -L user02
[sudo] password for user01: user01_password
user01@host:~$ su - user02
Password: user02_password
su: Authentication failure
user01@host:~$
```

If a user tries to run a command as another user, and the `sudo` configuration does not permit it, then `Bash` blocks the command, logs the attempt, and sends an email to the `root` user by default.

```
user02@host:~$ sudo tail /var/log/secure
[sudo] password for user02: user02_password
user02 is not in the sudoers file. This incident will be reported.
user02@host:~$
```

Another benefit of the `sudo` command is that it logs by default all the executed commands to the `/var/log/secure` file.

```
user01@host:~$ sudo tail /var/log/secure
...output omitted...
Mar  9 20:45:46 host sudo[2577]: user01 : TTY=pts/0 ; PWD=/home/user01 ;
USER=root ; COMMAND=/sbin/usermod -L user02
...output omitted...
```

In Red Hat Enterprise Linux 7 and later versions, all members of the `wheel` group can use `sudo` to run commands as any user, including `root`, by using their own password.

⚠ Warning

Historically, UNIX systems use membership of the `wheel` group to grant or control superuser access. RHEL 6 and earlier versions do not grant the `wheel` group any special privileges by default. System administrators who previously used this group for a nonstandard purpose must update their previous configuration, to prevent unexpected and unauthorized users from obtaining administrative access on RHEL 7 and later systems.

Getting an Interactive Root Shell with Sudo

To access the root account with the `sudo` command, use the `sudo -i` command. This command switches to the root account and runs that user's default shell (usually `bash`) and associated interactive login scripts. To run the shell without the interactive scripts, use the `sudo -s` command.

For example, an administrator can get an interactive shell as root on an AWS Elastic Cloud Computing (EC2) instance by using SSH public-key authentication to log in as the `ec2-user` regular user. Then, run the `sudo -i` command to access the root user's shell.

```
ec2-user@host:~$ sudo -i
[sudo] password for ec2-user: ec2-user_password
root@host:~#
```

 **Note**

For security reasons, the `sudo` command modifies the shell environment it sets up before it runs commands or starts interactive sessions, based on settings in its configuration files. With the default configuration, the resulting environment is similar to the one you would get if you logged in as that user to start a new session.

Configuring Sudo

The `/etc/sudoers` file is the main configuration file for the `sudo` command. To avoid problems if multiple administrators try to edit the file at the same time, you can edit it only with the special `visudo` command. The `visudo` editor also validates the file, to ensure no syntax errors.

For example, the following line from the `/etc/sudoers` file enables `sudo` access for `wheel` group members:

```
%wheel      ALL=(ALL:ALL)      ALL
```

- The `%wheel` string is the user or group that the rule applies to. The percent sign (%) before the `wheel` word specifies a group.
- The `ALL=(ALL:ALL)` command specifies that on any host with this file (the first `ALL`), users in the `wheel` group can run commands as any other user (the second `ALL`) and as any other group (the third `ALL`) on the system.
- The final `ALL` command specifies that users in the `wheel` group can run any command.

By default, the `/etc/sudoers` file also includes the contents of any files in the `/etc/sudoers.d` directory as part of the configuration file. With this hierarchy, you can add `sudo` access for a user by

putting an appropriate file in that directory.



Important

You can enable or disable sudo access by copying a file into the directory or removing it from the directory.

In this course, you create and remove files in the `/etc/sudoers.d` directory to configure sudo access for users and groups.

To enable full sudo access for the `user01` user, you can create the `/etc/sudoers.d/user01` file with the following content:

```
user01      ALL=(ALL)      ALL
```

To enable full sudo access for the `group01` group, you can create the `/etc/sudoers.d/group01` file with the following content:

```
%group01    ALL=(ALL)      ALL
```

To enable users in the `games` group to run the `id` command as the `operator` user, you can create the `/etc/sudoers.d/games` file with the following content:

```
%games ALL=(operator) /bin/id
```

You can also set up sudo to allow a user to run commands as another user without entering their password, by using the `NOPASSWD: ALL` command:

```
ansible     ALL=(ALL)      NOPASSWD: ALL
```

Although obvious security risks apply to granting this level of access to a user or group, system administrators often use this approach with cloud instances, virtual machines, and provisioning systems for configuring servers. You must protect the account with this access and require SSH public-key authentication for a user on a remote system to access it at all.

For example, the official Amazon Machine Image (AMI) for Red Hat Enterprise Linux in the Amazon Web Services Marketplace ships with the `root` and the `ec2-user` passwords locked. The `ec2-user` account is set up to allow remote interactive access through SSH public-key authentication. The `ec2-`

user user can also run any command as root without a password, because the last line of the AMI's `/etc/sudoers` file is set up as follows:

```
ec2-user          ALL=(ALL)          NOPASSWD: ALL
```

You can re-enable the requirement to enter a password for `sudo`, or introduce other changes to tighten security as part of the system configuration.

References

`info libc persona` (*GNU C Library Reference Manual*)

- Section 30.2: The Persona of a Process

(The `glibc-doc` package must be installed for this `info` node to be available.)

`su(1)`, `sudo(8)`, `visudo(8)`, `sudoers(5)`, and `bash(1)` man pages

10.4. Guided Exercise

Gain Superuser Access

Switch to the superuser account to manage a Linux system, and grant other users superuser access by using Sudo.

Outcomes

- Use Sudo to switch to the root user, and access the interactive shell as the root user without knowing the password of the superuser.
- Verify the behavior of the su command by switching users directly and by opening a login shell.
- Use Sudo privileges as a regular user to manage a restricted file.

Prerequisites

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
student@workstation:~$ lab start users-superuser
```

Instructions

1. As the student user on the workstation machine, log in to the servera machine as the student user.

```
student@workstation:~$ ssh student@servera
...output omitted...
[student@servera ~]$
```

2. Explore the shell environment of the student user. View the current user and group information and display the current working directory. Also view the environment variables that specify the user's home directory and the locations of the user's executable files.

- 2.1. View the current user and group information.

```
[student@servera ~]$ id
uid=1000(student) gid=1000(student) groups=1000(student),10(wheel)
context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
```

- 2.2. Display the current working directory.

```
[student@servera ~]$ pwd
/home/student
```

- 2.3.** Print the values of the HOME and PATH variables to determine the home directory and user executables' path, respectively.

```
[student@servera ~]$ echo $HOME
/home/student
[student@servera ~]$ echo $PATH
/home/student/.local/bin:/home/student/bin:/usr/local/bin:/usr/bin:/usr/local/sbin:/usr/sbin
```

- 3.** Switch to the root user in a nonlogin shell and explore the new shell environment.

- 3.1.** Become the root user by using the `sudo su` command. Use `student` as the password.

```
[student@servera ~]$ sudo su
[sudo] password for student: student
[root@servera student]#
```

- 3.2.** View the current user and group information.

```
[root@servera student]# id
uid=0(root) gid=0(root) groups=0(root)
context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
```

You are now running commands as the root user.

- 3.3.** Display the current working directory.

```
[root@servera student]# pwd
/home/student
```

Your current working directory stays the same after becoming the root user by using the `sudo su` command.

- 3.4.** Print the values of the HOME and PATH variables to determine the home directory and user executables' path, respectively.

```
[root@servera student]# echo $HOME
/root
[root@servera student]# echo $PATH
/root/.local/bin:/root/bin:/sbin:/bin:/usr/sbin:/usr/bin:/usr/local/sbin:/usr/local/bin
```

When you use the `su` command to become the root user, you do not keep the current path of the student user. As you can see in the next step, the path is not the root user path either.

What happened? The difference is that you do not run `su` directly. Instead, you run the `su` command as the root user by using `Sudo` because you do not have the password of the superuser. The `sudo` command overrides the `PATH` variable from the environment for security reasons. Any command that runs after the initial override can still update the `PATH` variable, as you can see in the following steps.

3.5. Exit the root user's shell.

```
[root@servera student]# exit
exit
[student@servera ~]$
```

4. Switch to the root user in a login shell and explore the new shell environment.

The `sudo` command might prompt you for the student password, depending on the `Sudo` time-out period. The default time-out period is five minutes. If you authenticated to `Sudo` within the last five minutes, then the `sudo` command does not prompt you for the password. If more than five minutes elapsed since you authenticated, then you must enter `student` as the password for authentication.

4.1. Use the `sudo su -` command to become the root user.

```
[student@servera ~]$ sudo su -
[root@servera ~]#
```

Notice the difference in the shell prompt compared to that of `sudo su` in the preceding step.

4.2. View the current user and group information.

```
[root@servera ~]# id
uid=0(root) gid=0(root) groups=0(root)
context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
```

4.3. Display the current working directory.

```
[root@servera ~]# pwd
/root
```

4.4. Print the values of the HOME and PATH variables to determine the home directory and the user executables' path, respectively.

```
[root@servera ~]# echo $HOME
/root
[root@servera ~]# echo $PATH
/root/.local/bin:/root/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin
```

As in the preceding step, after the `sudo` command resets the `PATH` variable from the settings in the student user's shell environment, the `su -` command runs the shell login scripts for the root user and sets the `PATH` variable to yet another value. The `su` command without the dash (`-`) option does not have the same behavior.

4.5. Exit the root user's shell to return to the student user's shell.

```
[root@servera ~]# exit
logout
[student@servera ~]$
```

5. Review the Sudo definition of the `operator1` user in the `/etc/sudoers.d/` directory.

```
[student@servera ~]$ sudo cat /etc/sudoers.d/operator1
operator1 ALL=(ALL) ALL
```

The `operator1` user can run any command as any user by using the `sudo` command.

6. Become the `operator1` user and view the contents of the `/var/log/messages` file.

As these operations require administrative privileges, use `Sudo` to run commands as the superuser. Do not switch to the root user by using the `sudo su` or `sudo su -` commands.

6.1. Switch to the `operator1` user. Use `redhat` as the password of the `operator1` user.

```
[student@servera ~]$ su - operator1
Password: redhat
[operator1@servera ~]$
```

6.2. List the `/var/log/messages` file. Review the user ownership and permissions.

```
[operator1@servera ~]$ ls -l /var/log/messages
-rw-----. 1 root root 1399281 May 14 13:00 /var/log/messages
```

The root user owns this file, and only the root user has read/write permissions.

6.3. Without using Sudo, try to view the last five lines of the `/var/log/messages` file. The command should fail because only the root user can view or write to the `/var/log/messages` file.

```
[operator1@servera ~]$ tail -5 /var/log/messages
tail: cannot open '/var/log/messages' for reading: Permission denied
```

6.4. Use Sudo to view the last five lines of the `/var/log/messages` file.

The output might vary in your environment.

```
[operator1@servera ~]$ sudo tail -5 /var/log/messages
[sudo] password for operator1: redhat
May 14 13:03:11 servera systemd[1]: Starting cockpit-wsinstance-https@e3b0...b855.socket - Socket for Cockpit Web Service https instance e3b0...b855...
May 14 13:03:11 servera systemd[1]: Listening on cockpit-wsinstance-https@e3b0...b855.socket - Socket for Cockpit Web Service https instance e3b0...b855.
May 14 13:03:11 servera systemd[1]: Started cockpit-wsinstance-https@e3b0...b855.service - Cockpit Web Service https instance e3b0...b855.
May 14 13:03:11 servera systemd[1]: cockpit-wsinstance-https-factory@30-1077-61789.service: Deactivated successfully.
May 14 13:03:36 servera systemd[1]: systemd-hostnamed.service: Deactivated successfully.
```

7. Return to the workstation machine as the student user.

```
[operator1@servera ~]$ exit
logout
[student@servera ~]$ exit
logout
Connection to servera closed.
student@workstation:~$
```

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish users-superuser
```

For use by Krishna Jeevan Upadhyay redhat.krishna@gmail.com redhat.krishna@gmail.com Copyright © 2025 Red Hat, Inc. | For use by Krishna Jeevan Upadhyay redhat.krishna@gmail.com

10.5. Managing Local User Accounts

Objectives

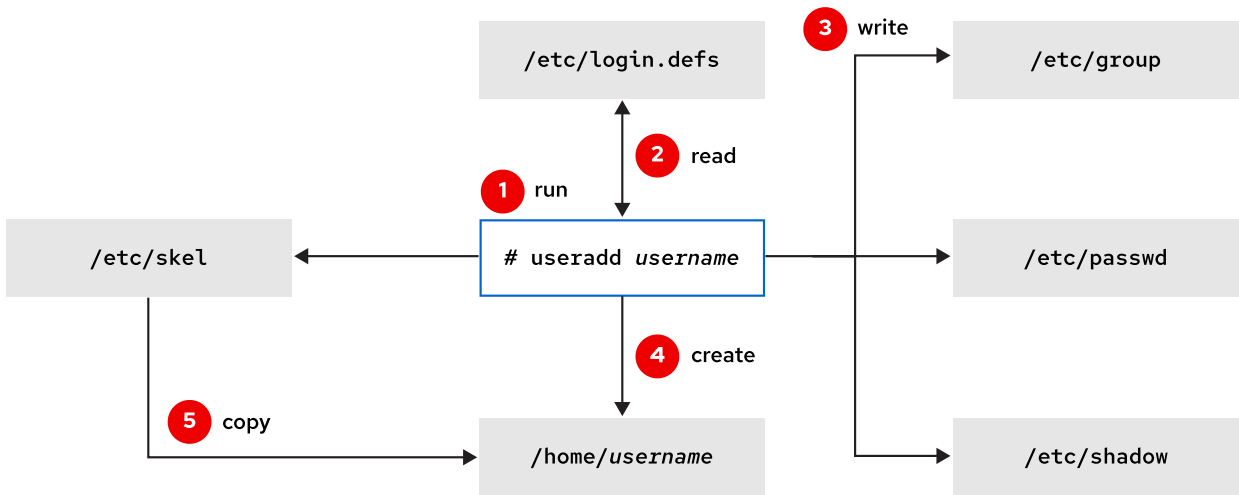
- Create, modify, and delete local user accounts.

Managing Local Users from the Command Line

You can use command-line tools to manage local user accounts. This section reviews some important tools for creating and deleting user accounts, and for setting passwords for users.

Creating Users

The `useradd username` command creates a user account named `username`. This command completes the following tasks to set up and configure the new account:



- 1 Run** the `useradd` command as the superuser.
- The `useradd` command **reads** the configuration for user creation from the `/etc/login.defs` file.
- The `useradd` command **writes** an entry in the `/etc/group`, `/etc/passwd`, and `/etc/shadow` files with the information that is assigned to the `username` user (user ID and group ID).
- The `useradd` command **creates** the home directory of the `username` user in the `/home/username` path.
- The `useradd` command **copies** the user profile files in the `/etc/skel` directory to the `username` user's home directory (under `/home/username`).

At this point, a valid password is not set for the account, and the user cannot log in until a password is set.

The `useradd --help` command displays the basic options to override the defaults. Usually, you can use the same options with the `usermod` command to modify an existing user.

By default, the `useradd` command assigns new users the first free UID that is greater than or equal to 1000, unless you explicitly specify a UID by using the `-u` option.

Modifying Existing Users

The `usermod --help` command displays the options to modify an account. The following table shows some common options for managing user accounts.

Table 14. `usermod` Command Options to Modify Users

usermod option	Usage
-a, --append	Use it with the -G option to add the supplementary groups to the user's current set of group memberships instead of replacing the set of supplementary groups with a new set.
-c, --comment COMMENT	Add the COMMENT text to the comment field.
-d, --home HOME-DIR	Specify a home directory for the user account.
-g, --gid GROUP	Specify the primary group for the user account.
-G, --groups GROUPS	Specify a comma-separated list of supplementary groups for the user account.
-L, --lock	Lock the user account.
-m, --move-home	Move the user's home directory to a new location. You must use it with the -d option.

usermod option	Usage
-s, --shell SHELL	Specify a particular login shell for the user account.
-U, --unlock	Unlock the user account.

Deleting Users

The `userdel username` command removes the `username` user from the `/etc/passwd` file, but leaves the user's home directory intact. The `userdel -r username` command removes the user from the `/etc/passwd` file and deletes the user's home directory.

 **Warning**

When you remove a user without specifying the `userdel -r` option, an unassigned UID now owns the user's files. If you create a user and that user is assigned the deleted user's UID, then the new account owns those files, which is a security risk. Typically, organization security policies disallow deleting user accounts, and instead lock them from being used, to avoid this scenario.

The following example demonstrates how this scenario can lead to information leakage:

```
root@host:~# useradd user01
root@host:~# ls -l /home
drwx-----. 4 user01      user01      93 May  9 17:40 user01
root@host:~# userdel user01
root@host:~# ls -l /home
drwx-----. 4          1000      1000    93 May  9 17:40 user01
root@host:~# useradd -u 1000 user02
root@host:~# ls -l /home
drwx-----. 4 user02      user02      93 May  9 17:40 user01
drwx-----. 4 user02      user02      93 May  9 17:41 user02
```

Notice that the `user02` user now owns all files that the `user01` user previously owned. The root user can use the `find / -nouser -o -nogroup` command to find all unowned files and directories.

Setting Passwords

The `passwd username` command sets the initial password or changes the existing password for the `username` user.

The `root` user can set a password to any value. If the password does not meet the minimum recommended criteria, then the terminal displays a message. You can retype the same password, and the `passwd` command updates it successfully.

```
root@host:~# passwd user01
New password: redhat
BAD PASSWORD: The password is shorter than 8 characters
Retype new password: redhat
passwd: password updated successfully
root@host:~#
```

A regular user must choose a password at least eight characters long. Do not use a dictionary word, the username, or the previous password.

UID Ranges

Red Hat Enterprise Linux uses specific UID numbers and ranges of numbers for specific purposes.

UID 0

The superuser (`root`) account UID.

UID 1-200

System account UIDs that are statically assigned to system processes.

UID 201-999

UIDs that are assigned to system processes that do not own files on this system. Software that requires an unprivileged UID is dynamically assigned a UID from this available pool.

UID 1000+

The UID range to assign to regular, unprivileged users.

Note

You can change the default UID ranges for the `useradd` command in the `/etc/login.defs` file.

References

useradd(8), usermod(8), userdel(8), login.defs(5), passwd(1), passwd(5), shadow(5), and group(5) man pages

10.6. Guided Exercise

Manage Local User Accounts

Create, modify, and delete local user accounts.

Outcomes

- Configure a Linux system with additional user accounts.

Prerequisites

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
student@workstation:~$ lab start users-user
```

Instructions

1. Create the operator1 user on the servera machine, and set the new user's password to redhat.

- 1.1. As the student user on the workstation machine, log in to the servera machine, and switch to the root user. Use student as the password.

```
student@workstation:~$ ssh student@servera
...output omitted...
[student@servera ~]$ sudo -i
[sudo] password for student: student
[root@servera ~]#
```

- 1.2. Create the operator1 user and confirm that it exists in the system.

```
[root@servera ~]# useradd operator1
[root@servera ~]# tail /etc/passwd
...output omitted...
operator1:x:1003:1003:~/home/operator1:/bin/bash
```

- 1.3. Set the password for the operator1 user to redhat.

```
[root@servera ~]# passwd operator1
New password: redhat
BAD PASSWORD: The password is shorter than 8 characters
Retype new password: redhat
passwd: password updated successfully
```

2. Create two additional users: operator2 and operator3. Set their passwords to redhat.

- 2.1. Add the operator2 user. Set the password for the operator2 user to redhat.

```
[root@servera ~]# useradd operator2
[root@servera ~]# passwd operator2
New password: redhat
BAD PASSWORD: The password is shorter than 8 characters
Retype new password: redhat
passwd: password updated successfully
```

- 2.2. Add the operator3 user. Set the password for the operator3 user to redhat.

```
[root@servera ~]# useradd operator3
[root@servera ~]# passwd operator3
New password: redhat
BAD PASSWORD: The password is shorter than 8 characters
Retype new password: redhat
passwd: password updated successfully
```

3. Update the operator1 and operator2 user accounts to include the Operator One and Operator Two comments, respectively. Verify that the comments exist for the user accounts.

- 3.1. Run the usermod -c command to update the comments of the operator1 user account.

```
[root@servera ~]# usermod -c "Operator One" operator1
```

- 3.2. Run the usermod -c command to update the comments of the operator2 user account.

```
[root@servera ~]# usermod -c "Operator Two" operator2
```

- 3.3. View the /etc/passwd file to confirm that the comments for the operator1 and operator2 users exist.

```
[root@servera ~]# tail /etc/passwd
...output omitted...
operator1:x:1003:1003:`Operator One`:/home/operator1:/bin/bash
operator2:x:1004:1004:`Operator Two`:/home/operator2:/bin/bash
operator3:x:1005:1005::/home/operator3:/bin/bash
```

4. Delete the operator3 user along with the user's personal data. Confirm that the operator3 does not exist.

4.1. Remove the operator3 user from the system.

```
[root@servera ~]# userdel -r operator3
```

4.2. Confirm that the operator3 user does not exist.

```
[root@servera ~]# tail /etc/passwd
...output omitted...
operator1:x:1003:1003:Operator One:/home/operator1:/bin/bash
operator2:x:1004:1004:Operator Two:/home/operator2:/bin/bash
```

Notice that the preceding output does not display the user account information for the operator3 user.

4.3. Confirm that the operator3 user home directory does not exist.

```
[root@servera ~]# ls -l /home
total 0
drwx-----. 3 cloud-user cloud-user 74 May  9 17:17 cloud-user
drwx-----. 4 devops      devops    90 Mar  6 16:24 devops
drwx-----. 2 operator1  operator1 62 May  9 22:35 operator1
drwx-----. 2 operator2  operator2 62 May  9 22:35 operator2
drwx-----. 3 student    student    74 Mar  6 16:23 student
```

4.4. Exit the root user's shell to return to the student user's shell.

```
[root@servera ~]# exit
logout
[student@servera ~]$
```

4.5. Return to the workstation machine as the student user.

```
[student@servera ~]$ exit  
logout  
Connection to servera closed.  
student@workstation:~$
```

Finish

On the workstation machine, change to the student user home directory and use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish users-user
```

10.7. Managing Local Group Accounts

Objectives

- Create, modify, and delete local group accounts.

Managing Local Groups

You can manage groups by using the **Accounts** menu in the RHEL web console (if enabled) or by using command-line tools. This section focuses on command-line tools that you can use from a shell prompt, either locally, over an SSH connection, or in the RHEL web console.

Creating Groups from the Command Line

The `groupadd` command creates groups. Without options, the `groupadd` command uses the next available group ID (GID) number from the range that the `GID_MIN` and `GID_MAX` variables specify in the `/etc/login.defs` file. By default, the command assigns a GID value that is greater than any other existing GIDs, even if a lower value becomes available.

The `groupadd` command `-g` option specifies a GID for the group to use.

```
root@host:~# groupadd -g 10000 group01
root@host:~# tail /etc/group
...output omitted...
group01:x:10000:
```

Note

Because the `useradd` command automatically creates user private groups with GID numbers assigned from 1000 and up, some administrators set aside a special range of high GID numbers to use when they create supplementary groups for other purposes. However, this extra management is unnecessary, because a user's UID and primary GID do not need to be the same number.

The `groupadd` command `-r` option creates system groups. As with regular groups, system groups use a GID from the range of listed valid system GIDs in the `/etc/login.defs` file. The `SYS_GID_MIN` and `SYS_GID_MAX` configuration items in the `/etc/login.defs` file define the range of system GIDs.


```

root@host:~# groupadd -r group02
root@host:~# tail /etc/group
...output omitted...
group01:x:10000:
group02:x:988:

```

Modifying Existing Groups from the Command Line

The `groupmod` command changes the properties of an existing group. The `groupmod` command `-n` option specifies a new name for the group.

```

root@host:~# groupmod -n group0022 group02
root@host:~# tail /etc/group
...output omitted...
group0022:x:988:

```

Notice that the group name updates from `group02` to `group0022`.

The `groupmod` command `-g` option reassigns the group name to a new GID.

```

root@host:~# groupmod -g 20000 group0022
root@host:~# tail /etc/group
...output omitted...
group0022:x:20000:

```

Notice that the GID changes from 988 to 20000.

Deleting Groups from the Command Line

The `groupdel` command removes groups.

```

root@host:~# groupdel group0022

```

Important

You cannot remove a group if it is the default primary group of an existing user. Similar to using the `userdel` command, ensure that you first locate files that the group owns to prevent potential security risks.

Primary and Supplementary Group Membership

A user's default primary group is the group that is listed on the line that defines the user's account in the `/etc/passwd` file. A user can belong to only one primary group at a time.

A user's supplementary groups are the additional groups that are configured for the user in the `/etc/group` file. A user can belong to many supplementary groups.

For configuring group-based file permissions, no difference exists between a user's primary and supplementary groups. If the user belongs to any group that is assigned access to specific files, then that user has access to those files.

The only distinction between a user's primary and supplementary memberships is when a user creates a file. New files must have a user owner and a group owner, which is assigned when the file is created. The user's current primary group is used for the new file's group ownership, unless command options override it.

Changing Group Membership from the Command Line

Use the `usermod -g` command to change a user's default primary group.

```
root@host:~# id user02
uid=1006(user02) gid=1008(user02) groups=1008(user02)
root@host:~# usermod -g group01 user02
root@host:~# id user02
uid=1006(user02) gid=10000(group01) groups=10000(group01)
```

Important

Using the `-g` option to change a user's default primary group does not automatically add the old primary group to the user's list of supplementary groups. You can see this behavior in the preceding example.

In addition, using this option does not change group ownership of any existing files in the user's home directory or elsewhere on the system.

Use the `groupmod -aU` command to add a list of one or more users to a supplementary group. The name of the group is the final argument to the command.

```

root@host:~# id user03
uid=1007(user03) gid=1009(user03) groups=1009(user03)
root@host:~# groupmod -aU user03 group01
root@host:~# id user03
uid=1007(user03) gid=1009(user03) groups=1009(user03),10000(group01)

```

⚠ Warning

The `groupmod` command `-a` option enables *append* mode. Without the `-a` option, the command replaces the existing set of users that are members of the group with those that are included in the `-U` option's list.

Temporarily Changing Your Primary Group

A user's current primary group determines the group that owns newly created files.

However, you can temporarily switch your current primary group to one of the supplementary groups to which you belong. You might do this immediately before creating a set of files that you want that group to own, instead of your usual primary group.

Use the `newgrp` command to switch your primary group for the current shell prompt. This temporary change does not affect other shell prompts on the same system.

You can switch your current primary group to any of the supplementary groups that you belong to, or back to your default primary group. Only one group at a time can be your current primary group. If you log out and log back in, then your default primary group at that shell prompt will be the current one again.

In this example, the `group01` group temporarily becomes this user's primary group.

```

user03@host:~$ id
uid=1007(user03) gid=1009(user03) groups=1009(user03),10000(group01)
user03@host:~$ newgrp group01
user03@host:~$ id
uid=1007(user03) gid=10000(group01) groups=1009(user03),10000(group01)

```

📖 References

`groupadd(8)`, `groupmod(8)`, `groupdel(8)`, `usermod(8)`, `login.defs(5)`, `group(5)`, `passwd(5)`, and `newgrp(1)` man pages

10.8. Guided Exercise

Manage Local Group Accounts

Create, modify, and delete local group accounts.

Outcomes

- Create and manage supplementary groups.
- Configure Sudo access for a group.

Prerequisites

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
student@workstation:~$ lab start users-group
```

Instructions

1. As the root user on the servera machine, create the operators and admin supplementary groups.

- 1.1. As the student user on the workstation machine, log in to the servera machine as the student user. Switch to the root user. Use student as the password.

```
student@workstation:~$ ssh student@servera
...output omitted...
[student@servera ~]$ sudo -i
[sudo] password for student: student
[root@servera ~]#
```

- 1.2. Create the operators supplementary group with a GID of 30000.

```
[root@servera ~]# groupadd -g 30000 operators
```

- 1.3. Create the admin supplementary group without specifying a GID.

```
[root@servera ~]# groupadd admin
```

- 1.4. Verify that both the operators and admin supplementary groups exist.

```
[root@servera ~]# tail /etc/group
...output omitted...
operators:x:30000:
admin:x:30001:
```

2. Add the operator1, operator2, and operator3 users to the operators group, and verify the group membership.

- 2.1. Add the operator1, operator2, and operator3 users to the operators group.

```
[root@servera ~]# usermod -aG operators operator1
[root@servera ~]# usermod -aG operators operator2
[root@servera ~]# usermod -aG operators operator3
```

- 2.2. Confirm that the users are in the group.

```
[root@servera ~]# id operator1
uid=1003(operator1) gid=1003(operator1) groups=1003(operator1),30000(operators)
[root@servera ~]# id operator2
uid=1004(operator2) gid=1004(operator2) groups=1004(operator2),30000(operators)
[root@servera ~]# id operator3
uid=1005(operator3) gid=1005(operator3) groups=1005(operator3),30000(operators)
```

3. Add the sysadmin1, sysadmin2, and sysadmin3 users to the admin group, and verify the group membership.

- 3.1. Add the sysadmin1, sysadmin2, and sysadmin3 users to the admin group.

```
[root@servera ~]# usermod -aG admin sysadmin1
[root@servera ~]# usermod -aG admin sysadmin2
[root@servera ~]# usermod -aG admin sysadmin3
```

- 3.2. Confirm that the users are in the group.

```
[root@servera ~]# id sysadmin1
uid=1006(sysadmin1) gid=1006(sysadmin1) groups=1006(sysadmin1),30001(admin)
[root@servera ~]# id sysadmin2
uid=1007(sysadmin2) gid=1007(sysadmin2) groups=1007(sysadmin2),30001(admin)
[root@servera ~]# id sysadmin3
uid=1008(sysadmin3) gid=1008(sysadmin3) groups=1008(sysadmin3),30001(admin)
```

- 3.3. Examine the /etc/group file to verify the supplementary group memberships.

```
[root@servera ~]# tail /etc/group
...output omitted...
operators:x:30000:operator1,operator2,operator3
admin:x:30001:sysadmin1,sysadmin2,sysadmin3
```

4. Enable administrative rights for all members of the `admin` group. Verify that any member of the `admin` group can run administrative commands.

- 4.1. Grant the `admin` group full administrative privileges by creating the `/etc/sudoers.d/admin` file with the appropriate permissions.

The `%admin ALL=(ALL) ALL` string means that members of the `admin` group can run any command as any user, including the root user.

```
[root@servera ~]# echo "%admin ALL=(ALL) ALL" > /etc/sudoers.d/admin
```

- 4.2. Switch to the `sysadmin1` user (a member of the `admin` group).

```
[root@servera ~]# su - sysadmin1
```

- 4.3. Without using Sudo, try to read the last entries of the `/var/log/messages` file.

The command should fail because the `sysadmin1` user does not have permission to read the file.

```
[sysadmin1@servera ~]$ tail /var/log/messages
tail: cannot open '/var/log/messages' for reading: Permission denied
```

- 4.4. Use Sudo to read the last entries of the `/var/log/messages` file. Use `redhat` as the password for the `sysadmin1` user.

The command should succeed because the `sysadmin1` user can use the Sudo privileges granted to the `admin` group.

```
[sysadmin1@servera ~]$ sudo tail /var/log/messages
[sudo] password for sysadmin1: redhat
...output omitted...
May 18 04:59:40 servera su[8080]: (to root) sysadmin1 on pts/2
May 18 04:59:40 servera systemd[1]: Starting systemd-hostnamed.service -
Hostname Service...
May 18 04:59:40 servera systemd[1]: Started systemd-hostnamed.service - Hostname
Service.
```

5. Return to the workstation machine as the student user.

```
[sysadmin1@servera ~]$ exit
logout
[root@servera ~]# exit
logout
[student@servera ~]$ exit
logout
Connection to servera closed.
student@workstation:~$
```

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish users-group
```


10.9. Managing User Passwords

Objectives

- Set a password management policy for users, and manually lock and unlock user accounts.

Shadow Passwords and Password Policy

Originally, encrypted passwords were stored in the world-readable `/etc/passwd` file. These passwords were considered adequate until dictionary attacks on encrypted passwords became common. The cryptographically hashed passwords were moved to the `/etc/shadow` file, which only the root user can read.

Like the `/etc/passwd` file, each user has an entry in the `/etc/shadow` file. An example entry from the `/etc/shadow` file has nine colon-separated fields:

```
root@host:~# cat /etc/shadow
...output omitted...
user03:$y$j9T$qybndwEWzHhr0uTGAw04Q0$0uNgGC5Mx2RrC04J0XtR2VJfTA8dLPxa7NV1tvhziH
C:20222:0:99999:7:2:20282::
```

Each field of the entry is separated by a colon:

- `user03`: Name of the user account.
- `$y$j9T$ziHC`: The cryptographically hashed password of the user.
- `20222`: The days from the epoch when the password was last changed, where the epoch is 1970-01-01 in the UTC time zone.
- `0`: The minimum days since the last password change before the user can change it again.
- `99999`: The maximum days without a password change before the password expires. An empty field means that the password never expires.
- `7`: The number of days in advance to warn the user that their password will expire.
- `2`: The number of days without activity, starting with the day that the password expired, before the account is automatically locked.
- `20282`: The day when the account expires in days since the epoch. An empty field means that the account never expires.
- The last field is typically empty and is reserved for future use.

Format of a Cryptographically Hashed Password

The cryptographically hashed password field stores three key pieces of information: the hashing algorithm in use, the *salt*, and the cryptographical hash. Salt adds random data to the password, to create a unique hash to strengthen the cryptographically hashed password. Each piece of information is delimited by the dollar sign (\$).

```
$y$j9T$qybndwEWzHhr0uTGAwO4Q0$0uNgGC5Mx2RrC04J0XtR2VJfTA8dLPxa7NV1tvhziHC
```

- **y** : The hashing algorithm in use for this password. A **y** indicates a yescrypt hash; a **6** indicates a SHA-512 hash; and a **5** indicates SHA-256. The format of the rest of the password depends on the algorithm.
- **j9T** : Parameters that tune the behavior of how yescrypt hashes. These options are standardized by the tools that set passwords to ensure secure tuning. Other algorithms might not have this field.
- **qybndwEWzHhr0uTGAwO4Q0** : The salt in use to cryptographically hash the password; it is originally chosen at random.
- **0uNgGC5Mx2RrC04J0XtR2VJfTA8dLPxa7NV1tvhziHC** : The cryptographic hash of the user's password. This segment is the result of combining the salt and the plain text password, and then using the cryptographic hashing algorithm to generate the password hash.

The primary reason to combine a salt with the password is to defend against attacks that use precomputed lists of password hashes. Adding salts expands the number of possible hashes that must be checked for each word that could be a possible password, which makes it much more difficult to use a precomputed list due to its size. If an attacker obtains a copy of an `/etc/shadow` file that uses salts, then they need to guess passwords with brute force, which requires more time and effort.

 Note

The default hashing algorithm in RHEL 10 is yescrypt, a password-based key derivation function (KDF) and password hashing scheme based on NIST-approved cryptographic primitives, developed by Solar Designer.

This default hashing algorithm is a change from RHEL 9, which used Ulrich Drepper's SHA-512 hashing algorithm by default. RHEL 10 still recognizes SHA-512 (\$6\$) password hashes that might be in the `/etc/shadow` file, but new and updated passwords are created by using yescrypt, which Red Hat considers to be more secure.

For more information about local user password hashing algorithms in RHEL, refer to the `crypt(5)` man page and the article, [About Local User Password Hashing Algorithms in RHEL](#).

Password Verification

When a user tries to log in, the system looks up the entry for the user in the `/etc/shadow` file, and combines the salt for the user with the plain text typed password. The system then cryptographically hashes the combination of the salt and plain text password with the specified hashing algorithm. If the result matches the cryptographic hash, then the user typed the right password. If the result does not match the cryptographic hash, then the user typed the wrong password and the login attempt fails. This method enables the system to determine whether the user typed the correct password without storing that password in a usable form for logging in.

Configuring Password Aging

The following diagram shows the relevant password aging parameters, which can be adjusted by using the `chage` command to implement a password aging policy. Notice that the command name is `chage`, which stands for "change age". Do not confuse the command with the word "change".

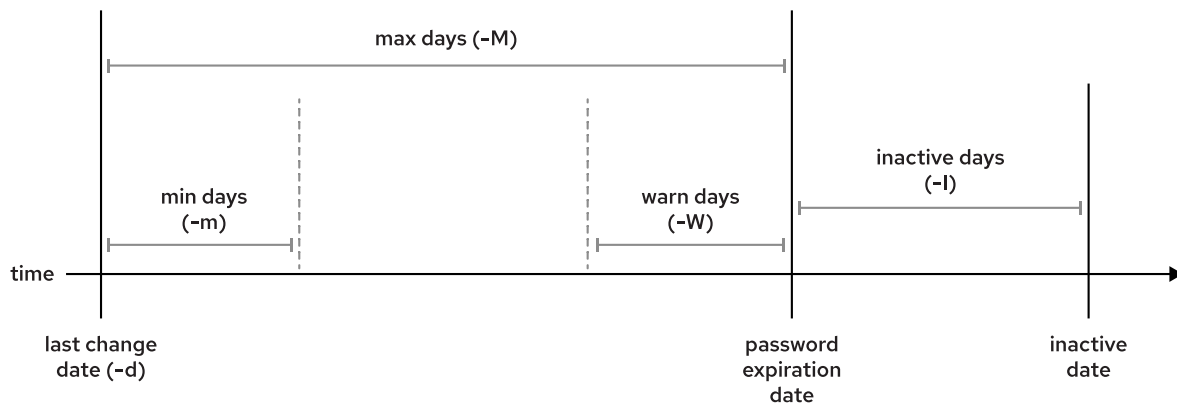


Figure 18. Password aging parameters

The following example demonstrates using the `chage` command to change the password policy of the `sysadmin05` user. The command defines a minimum age (`-m`) of 0 days, a maximum age (`-M`) of 90 days, a warning period (`-W`) of 7 days, and an inactivity period (`-I`) of 14 days.

```
root@host:~# chage -m 0 -M 90 -W 7 -I 14 sysadmin05
```

Assume that you manage the user password policies on a Red Hat server. The `cloudadmin10` user is new in the system, and you want to set a custom password aging policy. You want to set the account expiration to 30 days from today, so you use the following commands:

```
root@host:~# date +%F ①
2025-05-14
root@host:~# date -d "+30 days" +%F ②
2025-06-13
root@host:~# chage -E $(date -d "+30 days" +%F) cloudadmin10 ③
root@host:~# chage -l cloudadmin10 | grep "Account expires" ④
Account expires                               : Jun 13, 2025
```

- ① Use the `date` command to get the current date.
- ② Use the `date` command to get the date 30 days from now.
- ③ Use the `chage` command `-E` option to change the expiration date for the `cloudadmin10` user.
- ④ Use the `chage` command `-l` option to display the password aging policy for the `cloudadmin10` user.

 Note

The `date` command can calculate a future date. The `-u` option reports the time in UTC.

```
user@host:~$ date -d "+45 days" -u
Sat Jun 28 03:46:10 UTC 2025
```

After a few days, the `/var/log/secure` log file shows some unexpected behavior from the `cloudadmin10` user. The user tried to use the `sudo` command to interact with files that belong to other users. You suspect that the user might have left an `ssh` session open when working in another machine. You want the `cloudadmin10` user to change the password on the next login, so you use the following command:

```
root@host:~# chage -d 0 cloudadmin10
```

The next time that the `cloudadmin10` user logs in, the user is prompted to change the password.

You can change the default password aging configuration in the `/etc/login.defs` file. The `PASS_MAX_DAYS` and `PASS_MIN_DAYS` options set the default maximum and minimum age of the password, respectively. The `PASS_WARN_AGE` sets the default warning period of the password. Any change in the default password aging policies affects users that are created after the change. The existing users continue to use the earlier password aging settings rather than the later ones.

Restricting Access

You can use the `usermod` command to modify account expiration for a user. For example, the `usermod` command `-L` option locks a user account and the user cannot log in to the system.

```
root@host:~# usermod -L sysadmin03
user01@host:~$ su - sysadmin03
Password: redhat
su: Authentication failure
```

If a user leaves the company on a certain date, then you can lock and expire the account with a single `usermod` command. The date must be the number of days since `1970-01-01`, or else use the `YYYY-MM-DD` format. In the following example, the `usermod` command locks and expires the `cloudadmin10` user on `2025-06-01`.

```
root@host:~# usermod -L -e 2025-06-01 cloudadmin10
```

When you lock an account, you prevent the user from authenticating with a password to the system. This method is recommended to prevent access to an account by a former employee of the company. Use the `usermod` command `-U` option to enable the access to the account again.

The `nologin` Shell

The `nologin` shell acts as a replacement shell for the user accounts that are not intended to log in interactively to the system. A good security practice is to disable an account from logging in to the system when the account does not require it. For example, a mail server might require an account to store mail and a password for the user to authenticate with a mail client to retrieve mail. That user does not need to log in directly to the system.

A common solution to this situation is to set the user's login shell to `/sbin/nologin`. If the user attempts to log in to the system directly, then the `nologin` shell closes the connection.

```
root@host:~# usermod -s /sbin/nologin newapp
root@host:~# su - newapp
Last login: Wed May 14 03:49:38 UTC 2025 on pts/0
This account is currently not available.
```

Important

The `nologin` shell prevents interactive use of the system, but does not prevent all access. Users might be able to authenticate and upload or retrieve files through applications such as web applications, file transfer programs, or mail readers if they use the user's password for authentication and do not require a shell.

References

`chage(1)`, `usermod(8)`, `passwd(1)`, `passwd(5)`, `shadow(5)`, `crypt(3)`, `crypt(5)`, `login.defs(5)`, and `nologin(8)` man pages

[About Local User Password Hashing Algorithms in RHEL](#)

10.10. Guided Exercise

Manage User Passwords

Set a password management policy for users, and manually lock and unlock user accounts.

Outcomes

- Force a password change when the user logs in to the system for the first time.
- Force a password change every 90 days.
- Set a user account and passwords to expire 180 days from the current day.

Prerequisites

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
student@workstation:~$ lab start users-password
```

Instructions

1. On the servera machine, lock the operator1 user and try to switch to that user.
 - 1.1. As the student user on the workstation machine, log in to the servera machine as the student user.

```
student@workstation:~$ ssh student@servera
[student@servera ~]$
```
 - 1.2. As the student user, use administrative rights to lock the operator1 account. Use student as the password.

```
[student@servera ~]$ sudo usermod -L operator1
[sudo] password for student: student
```
 - 1.3. Try to log in as the operator1 user. Use redhat as the password.

This command should fail because the operator1 user is locked.

```
[student@servera ~]$ su - operator1
Password: redhat
su: Authentication failure
```

1.4. Unlock the operator1 account.

```
[student@servera ~]$ sudo usermod -U operator1
```

1.5. Try to log in as the operator1 user again. Use redhat as the password. This time, the command should succeed.

```
[student@servera ~]$ su - operator1
Password: redhat
...output omitted...
[operator1@servera ~]$
```

1.6. Log out of the operator1 user shell to return to the student user shell.

```
[operator1@servera ~]$ exit
logout
```

2. Change the password policy for the operator1 user to require a new password every 90 days. Confirm that the password age is successfully set.

2.1. Switch to the root user. Use student as the password.

```
[student@servera ~]$ sudo -i
[sudo] password for student: student
[root@servera ~]#
```

2.2. Set the maximum age of the operator1 user's password to 90 days.

```
[root@servera ~]# chage -M 90 operator1
```

2.3. Verify that the operator1 user's password expires 90 days after it is changed.


```
[root@servera ~]# chage -l operator1
Last password change           : May 14, 2025
Password expires                : Aug 12, 2025
Password inactive               : never
Account expires                : never
Minimum number of days between password change : 0
Maximum number of days between password change : 90
Number of days of warning before password expires : 7
```

3. Force a password change when the operator1 user logs in for the first time. After configuring the password change, return to the student user's shell.

- 3.1. Force a password change on the first login for the operator1 account.

```
[root@servera ~]# chage -d 0 operator1
```

- 3.2. Exit the root user's shell from the servera machine.

```
[root@servera ~]# exit
logout
[student@servera ~]$
```

4. Log in as the operator1 user and change the password to forsooth123. After setting the password, return to the student user's shell.

- 4.1. Log in as the operator1 user and change the password to forsooth123 when prompted.

```
[student@servera ~]$ su - operator1
Password: redhat
You are required to change your password immediately (administrator enforced).
Current password: redhat
New password: forsooth123
Retype new password: forsooth123
...output omitted...
[operator1@servera ~]$
```

- 4.2. Exit the operator1 user's shell to return to the student user.

```
[operator1@servera ~]$ exit
logout
[student@servera ~]$
```

5. Set the operator1 account to expire 180 days from the current day.

5.1. Switch to the root user. Use student as the password.

```
[student@servera ~]$ sudo -i
[sudo] password for student: student
[root@servera ~]#
```

5.2. Determine the date 180 days in the future. Use the format %F with the date command to get the exact value.

```
[root@servera ~]# date -d "+180 days" +%F
2025-11-10
```

**Important**

This returned date is an example; use the value on your system for the following steps.

5.3. Set the account to expire on the date that is displayed in the preceding step.

```
[root@servera ~]# chage -E 2025-11-10 operator1
```

5.4. Verify that the account expiration date is successfully set.

```
[root@servera ~]# chage -l operator1
Last password change      : May 14, 2025
Password expires          : Aug 12, 2025
Password inactive         : never
Account expires         : Nov 10, 2025
Minimum number of days between password change : 0
Maximum number of days between password change : 90
Number of days of warning before password expires : 7
```

6. Set passwords to expire 180 days from the current date for all new users.**6.1.** In the /etc/login.defs file, set the PASS_MAX_DAYS parameter to 180. Use administrative rights when you open the file with the text editor. You can use the vim /etc/login.defs command to perform this step.

```

...output omitted...
# Password aging controls:
#
#     PASS_MAX_DAYS    Maximum number of days a password may be
#     used.
#     PASS_MIN_DAYS    Minimum number of days allowed between
#     password changes.
#     PASS_MIN_LEN     Minimum acceptable password length.
#     PASS_WARN_AGE    Number of days warning given before a
#     password expires.
#
PASS_MAX_DAYS 180
PASS_MIN_DAYS  0
PASS_MIN_LEN   8
PASS_WARN_AGE  7
...output omitted...

```



Important

The default password and account expiration settings apply to new users but not to existing users.

7. Return to the workstation machine as the student user.

```

[root@servera ~]# exit
logout
[student@servera ~]$ exit
logout
Connection to servera closed.
student@workstation:~$

```

Finish

On the workstation machine, change to the student user home directory and use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish users-password
```

10.11. Lab

Manage Local Users and Groups

Use superuser access to manage local users and groups, and to administer local password policies.

Outcomes

- Set a default password aging policy for the local user's password.
- Create and use a supplementary group for new users.
- Create three users with the new supplementary group.
- Set an initial password for the created users.
- Configure the supplementary group members to use the sudo command to run any command as any user.
- Set a user-specific password aging policy.

Prerequisites

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
student@workstation:~$ lab start users-review
```

Instructions

1. From the workstation machine, log in to the serverb machine as the student user and switch to the root user.
2. On the serverb machine, ensure that newly created users must change their passwords every 30 days.
3. Create the consultants group with a GID of 35000.
4. Configure administrative rights to enable all consultants group members to execute any command as any user. Avoid using the visudo command-line tool to edit the /etc/sudoers file. Instead, place the configuration file in the /etc/sudoers.d directory.
5. Create the consultant1, consultant2, and consultant3 users with the consultants group as their supplementary group.
6. Set the consultant1, consultant2, and consultant3 passwords to redhat.

7. Set the `consultant1`, `consultant2`, and `consultant3` accounts to expire 90 days from the current day.
8. Change the password policy for the `consultant2` account to require a new password every 15 days.
9. Additionally, force the `consultant1`, `consultant2`, and `consultant3` users to change their passwords on the first login.
10. After completing the tasks of this activity, return to the workstation machine as the student user.

Evaluation

As the student user on the workstation machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

```
student@workstation:~$ lab grade users-review
```

Finish

On the workstation machine, change to the student user home directory and use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish users-review
```

Solution

Use superuser access to manage local users and groups, and to administer local password policies.

Outcomes

- Set a default password aging policy for the local user's password.
- Create and use a supplementary group for new users.
- Create three users with the new supplementary group.
- Set an initial password for the created users.
- Configure the supplementary group members to use the `sudo` command to run any command as any user.
- Set a user-specific password aging policy.

Prerequisites

As the student user on the workstation machine, run the following `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
student@workstation:~$ lab start users-review
```

Instructions

1. From the workstation machine, log in to the serverb machine as the student user and switch to the root user.

```
student@workstation:~$ ssh student@serverb
...output omitted...
[student@serverb ~]$ sudo -i
[sudo] password for student: student
[root@serverb ~]#
```

2. On the serverb machine, ensure that newly created users must change their passwords every 30 days.
 - 2.1. Set the `PASS_MAX_DAYS` parameter to 30 in the `/etc/login.defs` file. Use administrative rights when opening the file with the text editor. You can use the `vim /etc/login.defs` command for this step.

```
...output omitted...
# Password aging controls:
#
#     PASS_MAX_DAYS   Maximum number of days a password may be
#     used.
#     PASS_MIN_DAYS   Minimum number of days allowed between
#     password changes.
#     PASS_MIN_LEN     Minimum acceptable password length.
#     PASS_WARN_AGE   Number of days warning given before a
#     password expires.
#
PASS_MAX_DAYS 30
PASS_MIN_DAYS 0
PASS_MIN_LEN 8
PASS_WARN_AGE 7
...output omitted...
```

3. Create the consultants group with a GID of 35000.

```
[root@serverb ~]# groupadd -g 35000 consultants
```

4. Configure administrative rights to enable all consultants group members to execute any command as any user. Avoid using the visudo command-line tool to edit the /etc/sudoers file. Instead, place the configuration file in the /etc/sudoers.d directory.

- 4.1. Create the /etc/sudoers.d/consultants file and add the following content to it. You can use the vim /etc/sudoers.d/consultants command for this step.

```
%consultants  ALL=(ALL) ALL
```

5. Create the consultant1, consultant2, and consultant3 users with the consultants group as their supplementary group.

```
[root@serverb ~]# useradd -G consultants consultant1
[root@serverb ~]# useradd -G consultants consultant2
[root@serverb ~]# useradd -G consultants consultant3
```

6. Set the consultant1, consultant2, and consultant3 passwords to redhat.

```
[root@serverb ~]# passwd consultant1
Changing password for user consultant1.
New password: redhat
BAD PASSWORD: The password is shorter than 8 characters
Retype new password: redhat
passwd: password updated successfully
```

```
[root@serverb ~]# passwd consultant2
Changing password for user consultant2.
New password: redhat
BAD PASSWORD: The password is shorter than 8 characters
Retype new password: redhat
passwd: password updated successfully
```

```
[root@serverb ~]# passwd consultant3
Changing password for user consultant3.
New password: redhat
BAD PASSWORD: The password is shorter than 8 characters
Retype new password: redhat
passwd: password updated successfully
```

7. Set the consultant1, consultant2, and consultant3 accounts to expire 90 days from the current day.

- 7.1. Determine the date 90 days in the future. This returned date is an example; use the value on your system for the following steps.

```
[root@serverb ~]# date -d "+90 days" +%F
2025-08-12
```

- 7.2. Set the consultant1, consultant2, and consultant3 accounts to expire on the date that is displayed in the preceding step.

```
[root@serverb ~]# chage -E 2025-08-12 consultant1
[root@serverb ~]# chage -E 2025-08-12 consultant2
[root@serverb ~]# chage -E 2025-08-12 consultant3
```

8. Change the password policy for the consultant2 account to require a new password every 15 days.


```
[root@serverb ~]# chage -M 15 consultant2
```

9. Additionally, force the `consultant1`, `consultant2`, and `consultant3` users to change their passwords on the first login.

- 9.1. Set the last day of the password change to `0` so that users must change the password when they first log in to the system.

```
[root@serverb ~]# chage -d 0 consultant1
[root@serverb ~]# chage -d 0 consultant2
[root@serverb ~]# chage -d 0 consultant3
```

10. After completing the tasks of this activity, return to the workstation machine as the student user.

- 10.1. Return to the workstation machine as the student user.

```
[root@serverb ~]# exit
logout
[student@serverb ~]$ exit
logout
Connection to serverb closed.
student@workstation:~$
```

Evaluation

As the student user on the workstation machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

```
student@workstation:~$ lab grade users-review
```

Finish

On the workstation machine, change to the student user home directory and use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish users-review
```

Chapter 11.

Controlling Access to Files

Goal

Set standard permissions on files and interpret the security effects of different permission settings.

Sections

- Interpreting Linux File Permissions (and Quiz)
- Managing File Permissions from the Command Line (and Guided Exercise)
- Managing Default Permissions and Special Permissions (and Guided Exercise)

Lab

- Control Access to Files

11.1. Interpreting Linux File Permissions

Objectives

- List file system permissions on files and directories, and interpret how those permissions control access by users and by groups.

Linux File-system Permissions

File permissions control access to files. Linux file permissions are flexible and can handle most use cases.

A file is owned by a user, who is usually the file's creator. The file is also owned by a single group, which is usually the primary group of the user who created the file. The owning group can also be changed.

Files have three user categories that permissions apply to: the owning user (*user permissions*), the owning group (*group permissions*), and all other users on the system that are not the user or a member of the owning group (*other permissions*).

You can set different permissions for each user category, but the most specific permissions take precedence. User permissions override group permissions, which override other permissions.

Three permission types apply to each user category: read, write, and execute. The following table explains how these permissions affect access to files and directories.

Table 15. Effects of Permissions on Files and Directories

Permission	Effect on files	Effect on directories
r (read)	File contents can be read.	Contents of the directory (the file names) can be listed.
w (write)	File contents can be changed.	Any file in the directory can be created or deleted.

Permission	Effect on files	Effect on directories
x (execute)	Files can be executed as commands.	The directory can become the current working directory. You can run the <code>cd</code> command to change to it, but it also requires read permission to list files there.

Typically, users have both read and execute permissions on read-only directories so that they can list the directory and have read-only access to all its contents. If a user has only read access on a directory, then they can list the names of the files in it. However, the user cannot access other information, such as permissions or time stamps. If a user has only execute access on a directory, then they cannot list file names in the directory. If they know the name of a file that they have permission to read, then they can access the contents of that file from outside the directory by explicitly specifying the relative file name.

Anyone who owns or has both write and execute permissions on a directory can remove files from it, regardless of the ownership or permissions on the file itself. You can override this behavior by using the *sticky bit* permission.

Note

Linux file permissions work differently from the Microsoft NTFS file-system permissions. On Linux, permissions apply only to the file or directory on which they are set. The subdirectories within a directory do not automatically inherit the parent directory's permissions. However, directory permissions can block access to the directory contents, if they are set restrictively.

The read permission on a Linux directory is similar to **List folder contents** in Windows. The write permission on a Linux directory is similar to **Modify** in Windows; it implies the ability to delete files and subdirectories. In Linux, with write permissions and the *sticky bit* permission set on a directory, only the user or group owner can delete files, which is similar to the Windows **Write** behavior.

The Linux root user has the equivalent of the Windows **Full Control** permission on all files. However, SELinux policy can use process and file security contexts to restrict access even to the root user. SELinux is discussed in the *Red Hat System Administration II* (RH134) course.

Group membership enables collaboration between users. For example, imagine that your system has two users: `alice` and `bob`. The `alice` user is a member of the `alice` and `web` groups, and the `bob` user is a member of the `bob`, `wheel`, and `web` groups. When `alice` and `bob` work together, the files should be associated with the `web` group, and group permissions should allow both users to access the files.

Viewing File and Directory Permissions and Ownership

The `ls` command `-l` option shows detailed information about permissions and ownership:

```
user@host:~$ ls -l test
-rw-rw-r--. 1 student student 0 Mar  8 17:36 test
```

Use the `ls` command `-ld` option to show detailed information about a directory itself, and not its contents.

```
user@host:~$ ls -ld /home
drwxr-xr-x. 5 root root 4096 Feb 31 22:00 /home
```

The first character of the long listing is the file type:

- `-` is a regular file.
- `d` is a directory.
- `l` is a symbolic link.
- `c` is a character device file.
- `b` is a block device file.
- `p` is a named pipe file.
- `s` is a local socket file.

The next nine characters represent the file permissions. These characters are interpreted as three sets of three characters:

- The first set are permissions that apply to the file owner.
- The second set are for the file's group owner.
- The last set applies to all other (world) users.

If a set is an `rwX` string, then that set has all three permissions: read, write, and execute. If a letter is replaced by a dash (`-`), then that set does not have that permission.

After the second column (the link count), the first name specifies the owning user, and the second name is the file's group owner.

In the first example, the permissions for the `student` user are the first set of three characters: `-rw`. The `student` user has read and write permissions on the `test` file, but not execute permission. The second set of three characters are the permissions for the `student` group: `-rw`. The `student` group has read and write permissions on the `test` file, but not execute permission. The third set of three characters are the permissions for all other users: `r--`. Other users have only read permission on the `test` file.

The most specific set of permissions applies. So if the `student` user has different permissions from the `student` group, and the `student` user is also a member of that group, then only the user owner permissions apply. Therefore, you can set a more restrictive set of permissions on a user than their group membership provides, when it might not be practical to remove the user from the group.

Permission Scenarios

The following examples illustrate how file permissions interact. For these examples, your system has four users with the following group memberships:

User	Group memberships
operator1	operator1, consultant1
database1	database1, consultant1
database2	database2, operator2
contractor1	contractor1, operator2

These users work with files in the `dir` directory. A long listing of the files in that directory is as follows:

```
database@host:dir $ ls -la
total 24
drwxrwxr-x.  2 database1 consultant1  4096 Mar  4 10:23 .
drwxr-xr-x. 10 root          root      4096 Mar  1 17:34 ..
-rw-rw-r--.  1 operator1 operator1   1024 Mar  4 11:02 app1.log
-rw-r--rw-.  1 operator1 consultant1  3144 Mar  4 11:02 app2.log
-rw-rw-r--.  1 database1 consultant1 10234 Mar  4 10:14 db1.conf
-rw-r-----.  1 database1 consultant1  2048 Mar  4 10:18 db2.conf
```

The `ls` command `-a` option shows the permissions of hidden files, including the special files to represent the directory and its parent. In this example, the `.` special directory reflects the permissions of `dir` itself, and the `..` special directory reflects the permissions of its parent directory.

For the `db1.conf` file, the user that owns the file (`database1`) has read and write permissions but not execute permission. The group that owns the file (`consultant1`) has read and write permissions but not execute permission. All other users have read permission but not write or execute permissions.

The following table explores some effects of this set of permissions for these users:

Effect	Reason
The <code>operator1</code> user can change the contents of the <code>db1.conf</code> file.	The <code>operator1</code> user is a member of the <code>consultant1</code> group, and that group has both read and write permissions on the <code>db1.conf</code> file.
The <code>database1</code> user can view and modify the contents of the <code>db2.conf</code> file.	The <code>database1</code> user owns the <code>db2.conf</code> file and has both read and write access.
The <code>operator1</code> user can view but not modify the contents of the <code>db2.conf</code> file.	The <code>operator1</code> user is a member of the <code>consultant1</code> group, and that group has only read access to the <code>db2.conf</code> file.
The <code>database2</code> and <code>contractor1</code> users do not have any access to the contents of the <code>db2.conf</code> file.	The other permissions apply to the <code>database2</code> and <code>contractor1</code> users, and those permissions do not include the read or write permission.

Effect	Reason
The operator1 user is the only user who can change the contents of the app1.log file.	The operator1 user and members of the operator1 group have write permission on the file, whereas other users do not. The only member of the operator1 group is the operator1 user.
The database2 user can change the contents of the app2.log file.	The database2 user is not the owner of the app2.log file and is not in the consultant1 group, and so the other permissions apply. The other permissions grant write permission to the file.
The database1 user can view the contents of the app2.log file, but cannot modify the contents of the app2.log file.	The database1 user is a member of the consultant1 group, and that group has only read permissions on the app2.log file. Even though the other permissions include write permission, the group permissions take precedence.
The database1 user can delete the app1.log and app2.log files.	The database1 user has write permissions on the dir directory, which the . special directory shows, and therefore can delete any file in that directory. This operation is possible even if the database1 user does not have write permission on the files directly.

 References

info coreutils (*GNU Coreutils*)

- Section 13: Changing File Attributes

ls(1) man page

11.2. Quiz

Interpreting Linux File Permissions

Review the following information and use it to answer the quiz questions.

The system has four users that are assigned to the following groups:

- The consultant1 user is a member of the consultant1 and database1 groups.
- The operator1 user is a member of the operator1 and database1 groups.
- The contractor1 user is a member of the contractor1 and contractor3 groups.
- The operator2 user is a member of the operator2 and contractor3 groups.

The . special directory contains four files with the following permissions:

drwxrwxr-x.	operator1	database1	.
-rw-rw-r--.	consultant1	consultant1	app1.log
-rw-r--rw-.	consultant1	database1	app2.log
-rw-rw-r--.	operator1	database1	db1.conf
-rw-r-----.	operator1	database1	db2.conf

Choose the correct answers to the following questions:

1. Which regular file does the operator1 user own and all users can read?

- ☐ a. app1.log
- ☐ b. app2.log
- ☐ c. db1.conf
- ☐ d. db2.conf

2. Which file can the contractor1 user modify?

- ☐ a. app1.log
- ☐ b. app2.log
- ☐ c. db1.conf
- ☐ d. db2.conf

3. Which file can the operator2 user not read?

- ☐ a. app1.log
- ☐ b. app2.log
- ☐ c. db1.conf
- ☐ d. db2.conf

4. Which file does the consultant1 group own?

- ☐ a. app1.log
- ☐ b. app2.log
- ☐ c. db1.conf
- ☐ d. db2.conf

5. Which files can the operator1 user delete?

- ☐ a. Only db1.conf
- ☐ b. Only db2.conf
- ☐ c. All the files in the directory
- ☐ d. None of the files

6. Which files can the operator2 user delete?

- ☐ a. Only app1.log
- ☐ b. Only app2.log
- ☐ c. Both app1.log and app2.log
- ☐ d. None of the files

Solution

1. Which regular file does the operator1 user own and all users can read?

- ☐ a. app1.log
- ☐ b. app2.log
- ☒ c. db1.conf
- ☐ d. db2.conf

2. Which file can the contractor1 user modify?

- ☐ a. app1.log
- ☒ b. app2.log
- ☐ c. db1.conf
- ☐ d. db2.conf

3. Which file can the operator2 user not read?

- ☐ a. app1.log
- ☐ b. app2.log
- ☐ c. db1.conf
- ☒ d. db2.conf

4. Which file does the consultant1 group own?

- ☒ a. app1.log
- ☐ b. app2.log
- ☐ c. db1.conf
- ☐ d. db2.conf

5. Which files can the operator1 user delete?

- ☐ a. Only db1.conf
- ☐ b. Only db2.conf
- ☒ c. All the files in the directory
- ☐ d. None of the files

6. Which files can the operator2 user delete?

- ☐ a. Only app1.log
- ☐ b. Only app2.log
- ☐ c. Both app1.log and app2.log
- ☒ d. None of the files

11.3. Managing File Permissions from the Command Line

Objectives

- Change the permissions and ownership of files with command-line tools.

Changing File and Directory Permissions

In Linux, you can configure permissions on files and directories to define specific access for different sets of users. The main utility for configuring these permissions is the `chmod` command. The `chmod` command stands for "change mode", where mode refers to a file's permissions. The following list shows some key characteristics of the `chmod` command:

- It changes file and directory permissions from the command line.
- It takes a permission instruction followed by a list of files or directories to change.
- You can set the permission instruction either symbolically or in octal (numeric) notation.

Changing Permissions with the Symbolic Method

The *symbolic* method of changing file permissions uses letters to represent the user categories and permissions, and symbols to modify the permission settings.

Use the `chmod` command to modify file and directory permissions. The general syntax is as follows:

```
user@host:~$ chmod Who/What/Which file/directory
```

Who is the class of user, as in the following table. If you do not provide a class of user, then the `chmod` command uses the `all` group as the default.

Who	User class	Description
u	<i>user</i>	The file owner.
g	<i>group</i>	Member of the file's group.
o	<i>other</i>	Users who are not the file owner nor members of the file's group.

Who	User class	Description
-----	------------	-------------

a	all	All three previous groups.
---	-----	----------------------------

What is the operator that modifies the permissions, as in the following table.

What	Operation	Description
------	-----------	-------------

+	add	Adds the permissions to the file.
---	-----	-----------------------------------

-	remove	Removes the permissions to the file.
---	--------	--------------------------------------

=	set exactly	Sets exactly the provided permissions to the file.
---	-------------	--

Which is the mode, and specifies which permissions to set for the files or directories, as in the following table.

Which	Mode	Description
-------	------	-------------

r	read	Read access to the file. Listing access to the directory.
---	------	---

w	write	Write permissions to the file or directory.
---	-------	---

x	execute	Execute permissions to the file. Allows entering the directory, and accessing files and subdirectories inside the directory.
---	---------	--

X	special execute	Execute permissions to a directory, or execute permissions to a file if at least one of the execute bits is set.
---	-----------------	--

The symbolic method uses letters to represent the permission groups: u for user, g for group, o for other, and a for all.

With the symbolic method, you do not need to set a complete new group of permissions. Instead, you can change one or more of the existing permissions. Use the plus (+) or the minus (-) characters to add or remove permissions, respectively, or use the equal sign (=) to replace the entire set for a group of permissions.

A single letter represents the permissions themselves: r for read, w for write, and x for execute. You can use an uppercase X as the permission flag to add execute permissions only if the file is a directory or if execute is already set for user, group, or other.

The following example shows using the symbolic method to remove read and write permission for group and other on the document .pdf file:

```
user@host:~$ chmod go-rw document.pdf
```

This example shows adding execute permission for everyone on the myscript.sh file:

```
user@host:~$ chmod a+x myscript.sh
```

You can use the chmod command -R option to recursively set permissions on the files in an entire directory tree. For example, the next command recursively adds read, write, and execute permissions for the members of the group that own the myfolder directory and the files and directories inside it.

```
user@host:~$ chmod -R g+rxw /home/user/myfolder
```

You can also use the chmod command -R option with the -X option to set permissions symbolically. With the chmod command X option, you can set the execute (search) permission on directories so that their contents can be accessed, without changing permissions on most files. However, be cautious with the X option, because if any execute permission is set on a file, then the X option sets the specified execute permission on that file as well.

For example, the following command recursively sets group read and write access to the demodir directory and its contents, adding group execute only where execute is already set for user, group, or others.

```
root@host:# chmod -R g+rwX demodir
```

Changing Permissions with the Octal Method

You can use the `chmod` command to change file permissions with the octal method instead of the symbolic method. In the following example, the number sign (`#`) represents three digits:

```
user@host:~$ chmod ### file/directory
```

With the octal method, you can represent permissions as a 3-digit (or 4-digit, when setting advanced permissions) *octal/* number. A single octal digit can represent any single value from 0-7.

In the 3-digit octal representation of permissions, each digit stands for one access level, from left to right: user, group, and other. To determine each digit:

- Start with 0.
- To add read permissions for this access level, add 4.
- To add write permissions, add 2.
- To add execute permissions, add 1.

The following diagram illustrates how systems interpret the 644 octal permission value.

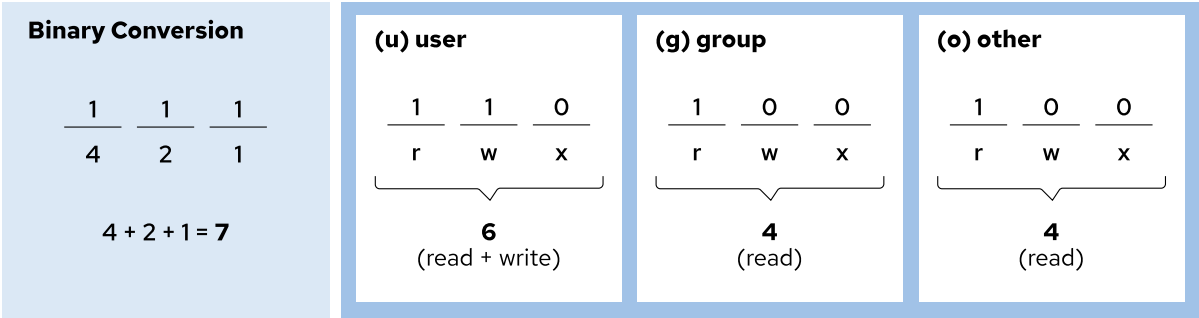


Figure 19. Visual representation of the octal method

Experienced administrators often use the octal method to implement permissions on single or matching files, and provide full permission control.

The following example shows using the octal method to set read and write permissions for user, and read permission for group and other, on the `sample.txt` file:

```
user@host:~$ chmod 644 sample.txt
```

This example shows setting read, write, and execute permissions for user, read and execute permissions for group, and no permission for other on the `sampledir` directory:


```
user@host:~$ chmod 750 sampledir
```

Changing File and Directory User or Group Ownership

The user owns a file that it creates. By default, the group owner of a new file is the primary group of the user that creates the file. In Red Hat Enterprise Linux, a user's primary group is usually a private group with only that user as a member. To grant access to a file based on group membership, you might need to change the group that owns the file.

Only the root user can change the user that owns a file. However, the file's owner and the root user can set group ownership. The root user can grant file ownership to any group, but regular users can only change the file's group ownership if they are a member of the destination group.

The following examples show how to change ownership of files and directories by using the `chown` (change owner) command.

For example, to grant ownership of the `app.conf` file to the `student` user, use the following command:

```
root@host:~# chown student app.conf
```

The `chown` command `-R` option recursively changes the ownership of an entire directory tree. The following command grants ownership of the `Pictures` directory and all files and subdirectories within it to the `student` user:

```
root@host:~# chown -R student Pictures
```

You can also use the `chown` command to change group ownership of a file by preceding the group name with a colon (`:`). For example, the following command changes the group ownership of the `Pictures` directory to `admins`:

```
root@host:~# chown :admins Pictures
```

You can use the `chown` command to change both owner and group at the same time by using the `owner:group` syntax. For example, to change the ownership of the `Pictures` directory to the `visitor` user and to the `guests` group, use the following command:

```
root@host:~# chown visitor:guests Pictures
```

Instead of using the `chown` command, some users change the group ownership by using the `chgrp` command. This command works similarly to the `chown` command, except that you can use it only to change group ownership, and the colon (:) before the group name is not required.

Important

You might encounter alternative `chown` command syntax that separates owner and group with a period instead of a colon:

```
root@host:~# chown owner.group filename
```

Red Hat recommends not using this syntax, and always using a colon. Because a period is a valid character in a username, the `chown` command might misinterpret your intent. The command might interpret the user and group as a file name. Instead, only use a colon when setting the user and group at the same time.

References

`ls(1)`, `chmod(1)`, `chown(1)`, and `chgrp(1)` man pages

11.4. Guided Exercise

Manage File Permissions from the Command Line

Change the permissions and ownership of files with command-line tools.

Outcomes

- Create a directory that all members of a group can access.

Prerequisites

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
student@workstation:~$ lab start perms-cli
```

Instructions

The `lab start` command prepares the environment by creating a group named `consultants` and creating two users, `consultant1` and `consultant2`, who are members of this group.

1. Create the `/home/consultants` directory on the `servera` machine, and change the owning group to the `consultants` group.

- 1.1. From the workstation machine, log in to the `servera` machine as the student user and switch to the root user. Use `redhat` as the password.

```
student@workstation:~$ ssh student@servera
...output omitted...
[student@servera ~]$ sudo -i
[sudo] password for student: student
[root@servera ~]#
```

- 1.2. Create the `/home/consultants` directory.

```
[root@servera ~]# mkdir /home/consultants
```

- 1.3. Change the group ownership of the `/home/consultants` directory to the `consultants` group.

```
[root@servera ~]# chown :consultants /home/consultants
```

2. Modify the permissions of the consultants group to allow its group members to create files in, and delete files from, the /home/consultants directory.

Also configure the directory to forbid others from accessing the files.

- 2.1. Verify that the permissions of the consultants group allow its group members to create files in, and delete files from, the /home/consultants directory.

Note that the consultants group currently does not have write permission.

```
[root@servera ~]# ls -ld /home/consultants
drwxr-xr-x. 2 root consultants 6 May 28 21:16 /home/consultants
```

- 2.2. Add write permission for the consultants group. Use the symbolic method for setting the appropriate permissions.

```
[root@servera ~]# chmod g+w /home/consultants
[root@servera ~]# ls -ld /home/consultants
drwxrwxr-x. 2 root consultants 6 May 28 21:16 /home/consultants
```

The new set of permissions for the consultants group (rwx) includes read, write, and execute.

- 2.3. Forbid others from accessing files in the /home/consultants directory. Use the octal method for setting the appropriate permissions.

```
[root@servera ~]# chmod 770 /home/consultants
[root@servera ~]# ls -ld /home/consultants
drwxrwx---. 2 root consultants 6 May 28 21:16 /home/consultants
```

The new set of permissions for others (---) restricts them from all access to the directory.

3. Exit the root shell and switch to the consultant1 user. Use redhat as the password.

```
[root@servera ~]# exit
logout
[student@servera ~]$ su - consultant1
Password: redhat
[consultant1@servera ~]$
```

4. Go to the /home/consultants directory and create a file named consultant1.txt. Because the consultant1 user is part of the consultants group, this user has write permission for the directory.

4.1. Change to the /home/consultants directory.

```
[consultant1@servera ~]$ cd /home/consultants
```

4.2. Create an empty file named consultant1.txt.

```
[consultant1@servera consultants]$ touch consultant1.txt
```

4.3. List the contents of the /home/consultants directory.

```
[consultant1@servera consultants]$ ls
consultant1.txt
```

5. Exit the shell and switch to the consultant2 user. Use redhat as the password.

```
[consultant1@servera consultants]$ exit
logout
[student@servera ~]$ su - consultant2
Password: redhat
[consultant2@servera ~]$
```

6. Go to the /home/consultants directory. Ensure that the consultant2 user can add files to the /home/consultants/ directory. Because the consultant2 user is part of the consultants group, this user has write permission for the directory.

6.1. Change to the /home/consultants directory. Create an empty file named consultant2.txt.

```
[consultant2@servera ~]$ cd /home/consultants/  
[consultant2@servera consultants]$ touch consultant2.txt
```

6.2. List the contents of the /home/consultants directory.

```
[consultant2@servera consultants]$ ls  
consultant1.txt  consultant2.txt
```

7. Return to the workstation system as the student user.

```
[consultant2@servera consultants]$ exit  
logout  
[student@servera ~]$ exit  
logout  
Connection to servera closed.  
student@workstation:~$
```

Finish

On the workstation machine, change to the student user home directory and use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish perms-cli
```

11.5. Managing Default Permissions and Special Permissions

Objectives

- Control the default permissions of user-created files, and explain special permissions on files.

Special Permissions

Special permissions are a fourth type of permission, in addition to user, group, and other. As the name implies, special permissions provide additional access-related features beyond what the basic permission types allow. The `setuid`, `setgid`, and `sticky` special permissions provide additional control over file and directory behavior. This section describes the impact of special permissions, which are summarized in the following table.

Table 16. Effects of Special Permissions on Files and Directories

Permission	Effect on files	Effect on directories
u+s (suid)	File executes as the user that owns the file, not as the user that ran the file.	No effect.
g+s (sgid)	File executes as the group that owns the file.	Files that are created in the directory have a group owner to match the group owner of the directory.
o+t (sticky)	No effect.	Users with write access to the directory can remove only files that they own; they cannot remove or force saves to files that other users own.

The `setuid` permission on an executable file means that commands run as the user that owns that file, rather than as the user that ran the command. One example is the `passwd` command:

```
user@host:~$ ls -l /usr/bin/passwd
-rwsr-xr-x. 1 root root 91584 Nov  4 2024 /usr/bin/passwd
```

In a long listing, you can identify the `setuid` permissions by a lowercase `s` character in the place where you would typically expect the `x` character (owner execute permissions). If the owner does not have execute permissions, then this character is replaced by an uppercase `S` character.

The `setgid` special permission on a directory means that created files in the directory inherit their group ownership from the directory, rather than inheriting group ownership from the creating user. This feature is commonly used on collaborative directories to automatically change a file from the default private group to the shared group, or in cases where a specific group should always own files in a directory. An example of this behavior is the `/run/log/journal` directory:

```
user@host:~$ ls -ld /run/log/journal
drwxr-sr-x+ 4 root systemd-journal 80 May 29 16:55 /run/log/journal
```

If `setgid` is set on an executable file, then commands run as the group that owns that file, rather than as the primary group of the user who ran the command. This behavior is similar to how `setuid` works.

One example is the `plocate` command, which is a modern replacement for `locate`. The `setgid` permission is set on the `/usr/bin/plocate` binary, as shown in the following example:

```
user@host:~$ ls -ld /usr/bin/plocate
-rwxr-sr-x. 1 root plocate 318224 Feb 14 00:00 /usr/bin/plocate
```

In a long listing, you can identify the `setgid` permissions by a lowercase `s` character in the place where you would typically expect the `x` character (group execute permissions). If the group does not have execute permissions, then this character is replaced by an uppercase `S` character.

Finally, the *sticky bit* for a directory sets a special restriction on the deletion of files. Only the owner of the file (and the root user) can delete files in the directory. An example is the `/tmp` directory:

```
user@host:~$ ls -ld /tmp
drwxrwxrwt. 11 root root 4096 May 29 19:57 /tmp
```

In a long listing, you can identify the *sticky* permissions by a lowercase `t` character in the place where you would typically expect the `x` character (other execute permissions). If other does not have execute permissions, then this character is replaced by an uppercase `T` character.

Setting Special Permissions

You can set special permissions by using symbolic notation or as a fourth digit in octal mode.

Table 17. Setting Special Permissions

Permission	Octal Notation	Symbolic Notation
setuid	4	u+s
setgid	2	g+s
sticky	1	o+t

Examples of Special Permissions

Add the setgid bit on the example directory by using the symbolic method:

```
user@host:~$ chmod g+s example
```

Remove the setuid bit on the example directory by using the symbolic method:

```
user@host:~$ chmod u-s example
```

Set the setgid bit and add read, write, and execute permissions for user and group, with no access for others, on the example directory by using the octal method:

```
user@host:~$ chmod 2770 example
```

Remove the setgid bit and add read, write, and execute permissions for user and group, with no access for others, on the example directory by using the octal method. Note that you need to add an extra 0 at the beginning of the permissions value when removing special permissions by using the octal method:

```
user@host:~$ chmod 00770 example
```

Default Permissions

In addition to setting special permissions, you can also manage the default permissions of new files and directories by using the *umask* setting. The *umask* setting alters the permission settings for newly created files or directories.

New files are assigned initial permissions based on two factors:

- Whether you are creating a regular file or a directory.
- The current *umask* (user file-creation mask).

Table 18. Initial Permissions Before *umask*

Type	Initial Octal Method	Initial Symbolic Method
File	0666	rw-rw-rw-
Directory	0777	rwxrwxrwx

You must always explicitly add execute permission to a regular file. This step makes it harder for an attacker to compromise a system, create a malicious file, and run it.

Effects of *umask* on Files and Directories

The *umask* is a permission filter that is applied to the default permissions when a file or directory is created. Each bit set in the *umask* removes a corresponding permission from the final permissions of new files or directories.

Table 19. *umask* Examples

umask	Effect on Files	Effect on Directories
0022	rw-r-r--	rwxr-xr-x
0027	rw-r-----	rwxr-x---
0002	rw-rw-r--	rwxrwxr-x

In RHEL 9 and later versions, the default umask for all regular users is `0022`. The `umask` command without arguments displays the current value of the shell's umask:

```
user@host:~$ umask
0022
```

You can temporarily set a new umask for the current shell by running the `umask` command with a single octal argument:

```
user@host:~$ umask 0027
```

You can omit any leading zeros in the umask. For example, `umask 077` is the same as `umask 0077`.

The system's default umask values for Bash shell users are defined in the `/etc/login.defs` file, and in the `/etc/bashrc` file. Users can override the system defaults in the `.bash_profile` or `.bashrc` files in their home directories.

Important

In Red Hat Enterprise Linux 8 and earlier, the default umask is `0002` for regular users who have a UID of 200 or greater and whose primary group name matches their username. The default umask for other users, including the `root` user, is `0022`.

Red Hat Enterprise Linux 9 and later versions consistently apply a default umask of `0022` for all users, in both login and non-login shells. RHEL 9 and later versions no longer make exceptions to the default umask setting based on user ID or matching group names, which improves consistency and simplifies security expectations.

Examples of umask Settings

In the following example, the system grants read and write access to the file owner, and read-only access to the group and others, reflecting a typical umask `0022` setup that restricts write access for nonowners.

```
user@host:~$ umask
0022
user@host:~$ touch default.txt
user@host:~$ ls -l default.txt
-rw-r--r--. 1 user user 0 May 29 22:11 default.txt
```

	Symbolic	Numeric octal	Numeric binary
Initial file permissions	rw-rw-rw-	0666	000 110 110 110
umask	---w--w-	0022	000 000 010 010
Resulting file permissions	rw-r--r--	0644	000 110 100 100

Figure 20. Example of umask calculation on a file

In the following example, the system grants full access to the directory owner, and read and execute access to the group and others, reflecting a typical umask 0022 setup that restricts write access for nonowners.

```
user@host:~$ umask
0022
user@host:~$ mkdir default
user@host:~$ ls -ld default
drwxr-xr-x. 2 user user 6 May 29 22:14 default
```

	Symbolic	Numeric octal	Numeric binary
Initial directory permissions	rwxrwxrwx	0777	000 111 111 111
umask	---w--w-	0022	000 000 010 010
Resulting directory permissions	rwxr-xr-x	0755	000 111 101 101

Figure 21. Example of umask calculation on a directory

In the following example, the umask value is set to 0, which changes the file permissions for others from read to read and write. The directory permissions for others change from read and execute to read, write, and execute.

```

user@host:~$ umask 0
user@host:~$ touch zero.txt
user@host:~$ ls -l zero.txt
-rw-rw-rw-. 1 user user 0 May 29 22:17 zero.txt
user@host:~$ mkdir zero
user@host:~$ ls -ld zero
drwxrwxrwx. 2 user user 6 May 29 22:17 zero

```

A umask of 027 ensures that new files have read and write permissions for user and read permission for group. New directories have read and execute permissions for group and no permissions for others.

```

user@host:~$ umask 027
user@host:~$ touch two-seven.txt
user@host:~$ ls -l two-seven.txt
-rw-r----- 1 user user 0 May 29 22:19 two-seven.txt
user@host:~$ mkdir two-seven
user@host:~$ ls -ld two-seven
drwxr-x--- 2 user user 6 May 29 22:20 two-seven

```

References

bash(1), ls(1), chmod(1), and umask(1) man pages

11.6. Guided Exercise

Manage Default Permissions and Special Permissions

Control the default permissions of user-created files, and explain special permissions on files.

Outcomes

- Create a shared directory where the operators group automatically owns new files.
- Configure the directory so that only the file owner (and the root user) can delete files.
- Temporarily change the umask setting.

Prerequisites

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
student@workstation:~$ lab start perms-default
```

Instructions

In this guided exercise, the umask setting has been set to 0007, which is different from the default 0022. This setting change enables you to inspect the permissions set for new files and directories.

1. Log in to the servera system as the operator1 user.

```
student@workstation:~$ ssh operator1@servera
...output omitted...
[operator1@servera ~]$
```

2. Create the /tmp/operators directory and change its owning group to operators. Verify the new group ownership.

- 2.1. Create the /tmp/operators directory.

```
[operator1@servera ~]$ mkdir /tmp/operators
```

- 2.2. Change the group ownership of the /tmp/operators directory to the operators group.

```
[operator1@servera ~]$ chown :operators /tmp/operators/
```

2.3. Verify that the group ownership of `/tmp/operators/` is applied correctly.

```
[operator1@servera ~]$ ls -ld /tmp/operators/
drwxrwx---. 2 operator1 operators 6 Jun  4 03:24 /tmp/operators/
```

3. Verify the `operator1` user's default umask value, and list the permissions of the `/tmp/operators` directory.

3.1. List the `operator1` user's default umask value.

```
[operator1@servera ~]$ umask
0007
```

This umask setting grants all user and group permissions, and prevents others from having any permissions on new files or directories.

3.2. Verify that the permissions of the `/tmp/operators` directory are consistent with the `0007` umask value.

```
[operator1@servera ~]$ ls -ld /tmp/operators/
drwxrwx---. 2 operator1 operators 6 Jun  4 03:24 /tmp/operators/
```

The `rwXrwx---` permissions set matches the umask rule.

4. Ensure that files created in the `/tmp/operators` directory are owned by the `operators` group. Then, verify the ownership.

4.1. Use symbolic notation to configure the `setgid` special permission on the `/tmp/operators` directory to make the `operators` group the group owner of new files.

```
[operator1@servera ~]$ chmod g+s /tmp/operators/
```

4.2. Verify that you have correctly set the `setgid` permission on the `/tmp/operators` directory.

```
[operator1@servera ~]$ ls -ld /tmp/operators/
drwxrws---. 2 operator1 operators 6 Jun  4 03:24 /tmp/operators/
```

5. Ensure that users with write access to the `/tmp/operators` directory can remove only files that they own. Then, verify the change.

- 5.1.** Use the octal method to configure the sticky bit on the `/tmp/operators` directory and preserve the existing permissions. The current permission mode is `2770` (`drwxrws---`).

```
[operator1@servera ~]$ chmod 3770 /tmp/operators/
```

- 5.2.** Verify that you have correctly applied the sticky bit to the `/tmp/operators` directory.

```
[operator1@servera ~]$ ls -ld /tmp/operators/
drwxrws--T. 2 operator1 operators 6 Jun  4 03:24 /tmp/operators/
```

- 6.** Create a file named `ops_file1.txt` in the `/tmp/operators/` directory. List the ownership and permissions of the `ops_file1.txt` file.

- 6.1.** Create a file named `ops_file1.txt` in the `/tmp/operators/` directory.

```
[operator1@servera ~]$ touch /tmp/operators/ops_file1.txt
```

- 6.2.** Change to the `/tmp/operators/` directory. List the ownership and permissions of the `ops_file1.txt` file.

```
[operator1@servera ~]$ cd /tmp/operators/
[operator1@servera operators]$ ls -l ops_file1.txt
-rw-rw----. 1 operator1 operators 0 Jun  4 03:28 ops_file1.txt
```

- 7.** Temporarily change the umask to `0022` for the `operator1` user. Create a second file named `ops_file2.txt` in the `/tmp/operators/` directory. Then, list the ownership and permissions of both the `ops_file1.txt` and `ops_file2.txt` files.

- 7.1.** Change the umask to `0022` for the `operator1` user.

```
[operator1@servera operators]$ umask 0022
```

This umask setting allows the user to read and write new files, but the group and others can only read those files.

- 7.2.** Create a file named `ops_file2.txt` in the `/tmp/operators/` directory.

```
[operator1@servera operators]$ touch ops_file2.txt
```

- 7.3.** List the ownership and permissions of both the `ops_file1.txt` and `ops_file2.txt` files.


```
[operator1@servera operators]$ ls -l ops_file{1,2}.txt
-rw-rw----. 1 operator1 operators 0 Jun  4 03:28 ops_file1.txt
-rw-r--r--. 1 operator1 operators 0 Jun  4 03:34 ops_file2.txt
```

The permissions on the `ops_file2.txt` file match the permissions from the `0022` umask setting. The `ops_file1.txt` file maintains the previous permissions because the file was created before the umask change.

8. Log in as the `operator2` user and attempt to delete a file created by the `operator1` user. You should not be able to delete the `ops_file1.txt` file as the `operator2` user because the `/tmp/operators/` directory is configured with the sticky bit permission.

- 8.1. Switch to the `operator2` user. Use `redhat` as the password.

```
[operator1@servera operators]$ su - operator2
Password: redhat
...output omitted...
[operator2@servera ~]$
```

- 8.2. Attempt to delete the `/tmp/operators/ops_file1.txt` file, which was created by the `operator1` user.

```
[operator2@servera ~]$ rm /tmp/operators/ops_file1.txt
rm: cannot remove '/tmp/operators/ops_file1.txt': Operation not permitted
```

As expected, the remove operation fails, because the sticky bit permission prevents users from deleting files owned by other users.

9. Return to the workstation system as the student user.

```
[operator2@servera ~]$ exit
logout
[operator1@servera operators]$ exit
logout
Connection to servera closed.
student@workstation:~$
```

Finish

On the workstation machine, change to the student user home directory and use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish perms-default
```

11.7. Lab

Control Access to Files

Set standard permissions on files and interpret the security effects of different permission settings.

Outcomes

- Create a directory where users can work collaboratively on files.
- Create the required file and directory structure, assigning access rights as requested.

Prerequisites

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
student@workstation:~$ lab start perms-review
```

Instructions

The `lab start` command creates the `dev1`, `dev2`, and `editor1` users, who are members of the `techdocs` group. The command also creates the `dbadmin1` user, who does not belong to this group.

In this activity, you create the `techdocs` collaborative directory. The `techdocs` directory must be configured so that only the owner of a file can delete it.

In the `techdocs` directory, you add a directory structure and files for a team (the `techdocs` group). Access to these files and directories varies based on the user, as indicated in each step. In some cases, you set the required `umask` value to create files and directories with the required permissions.

1. Log in to the `serverb` machine as the `student` user, and then switch to the `root` user.
2. Create the `techdocs` collaborative directory under the `/home` directory. Set the group ownership of the directory to the `techdocs` group, grant full permissions to user and group, and configure the directory so that only file owners can delete their files.
3. Verify that users in the `techdocs` group can create and collaborate on files in the `/home/techdocs` directory. As the `dev1` user, create the `techdoc1.txt` file in the `/home/techdocs` directory. As the `dev2` user, add the following content to the file: This is

the first tech doc. Configure the necessary group ownership and permissions to allow this collaboration.

- 4. Verify that only the owner of the techdoc1.txt file can remove it. As a nonowning user, attempt to delete the file. Then, switch to the owner, back up the file, and then delete it. Finally, restore the original file from the backup by moving the file.
- 5. In the /home/techdocs directory, create subdirectories for the dev1 and dev2 users based on the following table.

Modify permissions so that only the owner can write to the files and directories, the techdocs group can read them, and others have no access. Adjust ownerships and permissions accordingly, and verify that new files follow these access rules.

Owner	Directory	Group	Files
dev1	dev1	techdocs	dev1.txt
			dev1.log
			dev1.cfg
dev2	dev2	techdocs	dev2.txt
			dev2.log
			dev2.cfg

- 6. Verify that a user in the techdocs group who is not the file owner can read file contents in the /home/techdocs collaborative directory. As the editor1 user, read files that are owned by other users in the techdocs group.
- 7. Verify that only users in the techdocs group can access the /home/techdocs collaborative directory. As the dbadmin1 user, try to access the /home/techdocs directory.

8. After completing the tasks of this activity, return to the workstation machine as the student user.

Evaluation

As the student user on the workstation machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

```
student@workstation:~$ lab grade perms-review
```

Finish

On the workstation machine, change to the student user home directory and use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish perms-review
```

Solution

Set standard permissions on files and interpret the security effects of different permission settings.

Outcomes

- Create a directory where users can work collaboratively on files.
- Create the required file and directory structure, assigning access rights as requested.

Prerequisites

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
student@workstation:~$ lab start perms-review
```

Instructions

The `lab start` command creates the `dev1`, `dev2`, and `editor1` users, who are members of the `techdocs` group. The command also creates the `dbadmin1` user, who does not belong to this group.

In this activity, you create the `techdocs` collaborative directory. The `techdocs` directory must be configured so that only the owner of a file can delete it.

In the `techdocs` directory, you add a directory structure and files for a team (the `techdocs` group). Access to these files and directories varies based on the user, as indicated in each step. In some cases, you set the required `umask` value to create files and directories with the required permissions.

1. Log in to the `serverb` machine as the student user, and then switch to the root user.

1.1. Log in to the `serverb` machine as the student user and switch to the root user. Use `student` as the password.

```
student@workstation:~$ ssh student@serverb
...output omitted...
[student@serverb ~]$ sudo -i
[sudo] password for student: student
[root@serverb ~]#
```

2. Create the `techdocs` collaborative directory under the `/home` directory. Set the group ownership of the directory to the `techdocs` group, grant full permissions to user and group, and configure the

directory so that only file owners can delete their files.

2.1. Create a /home/techdocs directory.

```
[root@serverb ~]# mkdir /home/techdocs
```

2.2. Change the group ownership for the /home/techdocs directory to the techdocs group.

```
[root@serverb ~]# chown :techdocs /home/techdocs
```

2.3. Set read, write, and execute permissions for user and group, and no permission for others on the /home/techdocs directory.

```
[root@serverb ~]# chmod 0770 /home/techdocs
```

2.4. Assign the sticky bit to the /home/techdocs directory.

```
[root@serverb ~]# chmod o+t /home/techdocs
```

2.5. List the directory's permissions.

```
[root@serverb ~]# ls -ld /home/techdocs
drwxrwx--T. 2 root techdocs 6 Jun  6 19:21 /home/techdocs
```

The --T permission set shows that the directory is configured with the sticky bit permission.

3. Verify that users in the techdocs group can create and collaborate on files in the /home/techdocs directory. As the dev1 user, create the techdoc1.txt file in the /home/techdocs directory. As the dev2 user, add the following content to the file: This is the first tech doc. Configure the necessary group ownership and permissions to allow this collaboration.

3.1. Switch to the dev1 user. Change to the /home/techdocs directory.

```
[root@serverb ~]# su - dev1
[dev1@serverb ~]$ cd /home/techdocs/
```

3.2. Create a techdoc1.txt file in the /home/techdocs directory.

```
[dev1@serverb techdocs]$ touch techdoc1.txt
```

- 3.3.** Change the group ownership for the `techdoc1.txt` file to the `techdocs` group. Add write permissions for the group on the `techdoc1.txt` file. List file permissions.

```
[dev1@serverb techdocs]$ chown :techdocs techdoc1.txt
[dev1@serverb techdocs]$ chmod g+w techdoc1.txt
[dev1@serverb techdocs]$ ls -l techdoc1.txt
total 0
-rw-rw-r--. 1 dev1 techdocs 0 Jun  6 19:32 techdoc1.txt
```

- 3.4.** Exit from the `dev1` user shell. Switch to the `dev2` user.

```
[dev1@serverb techdocs]$ exit
logout
[root@serverb ~]# su - dev2
[dev2@serverb ~]$
```

- 3.5.** Change to the `/home/techdocs` directory. Verify that the `dev2` user can write to the `techdoc1.txt` file.

```
[dev2@serverb ~]$ cd /home/techdocs/
[dev2@serverb techdocs]$ echo "This is the first tech doc." > techdoc1.txt
[dev2@serverb techdocs]$ cat techdoc1.txt
This is the first tech doc.
```

- 4.** Verify that only the owner of the `techdoc1.txt` file can remove it. As a nonowning user, attempt to delete the file. Then, switch to the owner, back up the file, and then delete it. Finally, restore the original file from the backup by moving the file.

- 4.1.** As the `dev2` user, try to remove the `techdoc1.txt` file.

```
[dev2@serverb techdocs]$ rm techdoc1.txt
rm: cannot remove 'techdoc1.txt': Operation not permitted
```

- 4.2.** Exit from the `dev2` user shell. Switch to the `dev1` user. Change to the `/home/techdocs` directory.


```
[dev2@serverb techdocs]$ exit
logout
[root@serverb ~]# su - dev1
[dev1@serverb ~]$ cd /home/techdocs/
```

4.3. Back up the techdoc1.txt file, and then delete it.

```
[dev1@serverb techdocs]$ cp -pr techdoc1.txt techdoc1.txt.bkp
[dev1@serverb techdocs]$ rm techdoc1.txt
[dev1@serverb techdocs]$ ls -l
total 0
-rw-rw-r--. 1 dev1 techdocs 0 Jun  6 19:34 techdoc1.txt.bkp
```

4.4. Restore the techdoc1.txt file from its backup.

```
[dev1@serverb techdocs]$ mv techdoc1.txt.bkp techdoc1.txt
```

5. In the /home/techdocs directory, create subdirectories for the dev1 and dev2 users based on the following table.

Modify permissions so that only the owner can write to the files and directories, the techdocs group can read them, and others have no access. Adjust ownerships and permissions accordingly, and verify that new files follow these access rules.

Owner	Directory	Group	Files
dev1	dev1	techdocs	dev1.txt
			dev1.log
			dev1.cfg

Owner	Directory	Group	Files
dev2	dev2	techdocs	dev2.txt
			dev2.log
			dev2.cfg

5.1. As the dev1 user, create the dev1 directory. Change the group ownership for the dev1 directory to the techdocs group. Set read, write, and execute permissions for the user, read and execute permissions for group, and no permission for others on the dev1 directory.

```
[dev1@serverb techdocs]$ mkdir dev1
[dev1@serverb techdocs]$ chown :techdocs dev1
[dev1@serverb techdocs]$ chmod 0750 dev1
```

5.2. Change to the dev1 directory. List the umask value for the dev1 user. Change the default umask for the dev1 user to 0027.

```
[dev1@serverb techdocs]$ cd dev1
[dev1@serverb dev1]$ umask
0022
[dev1@serverb dev1]$ umask 0027
```

5.3. Create the files listed in the previous table for the dev1 user. Change the group ownership of these files to the techdocs group. Verify that the dev1 user can write to them.

```
[dev1@serverb dev1]$ touch dev1.txt
[dev1@serverb dev1]$ touch dev1.log
[dev1@serverb dev1]$ touch dev1.cfg
[dev1@serverb dev1]$ chown :techdocs dev1.*
[dev1@serverb dev1]$ ls -l
total 0
-rw-r-----. 1 dev1 techdocs 0 Jun  6 19:40 dev1.cfg
-rw-r-----. 1 dev1 techdocs 0 Jun  6 19:40 dev1.log
-rw-r-----. 1 dev1 techdocs 0 Jun  6 19:40 dev1.txt
[dev1@serverb dev1]$ echo 'File owned by the dev1 user' > dev1.txt
[dev1@serverb dev1]$ cat dev1.txt
File owned by the dev1 user
```

5.4. Exit from the dev1 user shell. Switch to the dev2 user. Change to the /home/techdocs directory.

```
[dev1@serverb dev1]$ exit
logout
[root@serverb ~]# su - dev2
[dev2@serverb ~]$ cd /home/techdocs/
```

5.5. As the dev2 user, create the dev2 directory. Change the group ownership for the dev2 directory to the techdocs group. Set read, write, and execute permissions for the user, read and execute permissions for the group, and no permission for others on the dev2 directory.

```
[dev2@serverb techdocs]$ mkdir dev2
[dev2@serverb techdocs]$ chown :techdocs dev2
[dev2@serverb techdocs]$ chmod 0750 dev2
```

5.6. Change to the dev2 directory. List the umask value for the dev2 user. Change the default umask for the dev2 user to 0027.

```
[dev2@serverb techdocs]$ cd dev2
[dev2@serverb dev2]$ umask
0022
[dev2@serverb dev2]$ umask 0027
```

5.7. Create the files listed in the previous table for the dev2 user. Change the group ownership for these files to the techdocs group. Verify that the dev2 user can write to them.

```
[dev2@serverb dev2]$ touch dev2.txt
[dev2@serverb dev2]$ touch dev2.log
[dev2@serverb dev2]$ touch dev2.cfg
[dev2@serverb dev2]$ chown :techdocs dev2.*
[dev2@serverb dev2]$ ls -l
total 0
-rw-r-----. 1 dev2 techdocs 0 Jun  6 19:44 dev2.cfg
-rw-r-----. 1 dev2 techdocs 0 Jun  6 19:44 dev2.log
-rw-r-----. 1 dev2 techdocs 0 Jun  6 19:44 dev2.txt
[dev2@serverb dev2]$ echo 'File owned by the dev2 user' > dev2.txt
[dev2@serverb dev2]$ cat dev2.txt
File owned by the dev2 user
```

6. Verify that a user in the techdocs group who is not the file owner can read file contents in the /home/techdocs collaborative directory. As the editor1 user, read files that are owned by other users in the techdocs group.

- 6.1. Exit from the dev2 user shell. Switch to the editor1 user. Change to the /home/techdocs directory.

```
[dev2@serverb dev2]$ exit
logout
[root@serverb ~]# su - editor1
[editor1@serverb ~]$ cd /home/techdocs/
[editor1@serverb techdocs]$
```

- 6.2. Identify directories and files that the editor1 user does not own. View the file structure and owners in the /home/techdocs collaborative directory.

```
[editor1@serverb techdocs]$ tree -ug
[root      techdocs] .
├── [dev1      techdocs] dev1
│   ├── [dev1      techdocs] dev1.cfg
│   ├── [dev1      techdocs] dev1.log
│   └── [dev1      techdocs] dev1.txt
├── [dev2      techdocs] dev2
│   ├── [dev2      techdocs] dev2.cfg
│   ├── [dev2      techdocs] dev2.log
│   └── [dev2      techdocs] dev2.txt
└── [dev1      techdocs] techdoc1.txt

3 directories, 7 files
```

- 6.3. As the editor1 user, read contents in the techdoc1.txt, dev1.txt, and dev2.txt files.

```
[editor1@serverb techdocs]$ cat techdoc1.txt
This is the first tech doc.
[editor1@serverb techdocs]$ cat dev1/dev1.txt
File owned by the dev1 user
[editor1@serverb techdocs]$ cat dev2/dev2.txt
File owned by the dev2 user
```

7. Verify that only users in the techdocs group can access the /home/techdocs collaborative directory. As the dbadmin1 user, try to access the /home/techdocs directory.

- 7.1. Exit from the editor1 user shell. Switch to the dbadmin1 user. Change to the /home/techdocs directory.

```
[editor1@serverb techdocs]$ exit
logout
[root@serverb ~]# su - dbadmin1
[dbadmin1@serverb ~]$ cd /home/techdocs/
-bash: cd: /home/techdocs/: Permission denied
[dbadmin1@serverb ~]$
```

- 7.2. Exit from the dbadmin1 user shell.

```
[dbadmin1@serverb ~]$ exit
logout
[root@serverb ~]#
```

8. After completing the tasks of this activity, return to the workstation machine as the student user.

```
[root@serverb ~]# exit
logout
[student@serverb ~]$ exit
Connection to serverb closed.
student@workstation:~$
```

Evaluation

As the student user on the workstation machine, use the lab command to grade your work. Correct any reported failures and rerun the command until successful.

```
student@workstation:~$ lab grade perms-review
```

Finish

On the workstation machine, change to the student user home directory and use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish perms-review
```

11.8. Summary

In this lesson, you learned how Red Hat Enterprise Linux manages file ownership and permissions.

All files belong to an owner and a group. Permission settings determine who can access files (user, group, and others), and what kind of access they have (read, write, and execute). The most specific permissions take precedence: user permissions take precedence over group permissions, and group permissions override others.

You can view file ownership and permissions by using the long listing format of the `ls` command.

You can change file permissions by using either symbolic or octal notation. Additionally, you can recursively change the permissions across directories by using the `chmod -R` command. The `suid`, `sgid`, and `sticky` special permissions provide additional access-related features to files.

You also learned to view and define `umask` values. The default `umask` values for Bash are defined in the `/etc/login.defs` file and might be affected by settings in the `/etc/profile` and `/etc/bashrc` files, files in `/etc/profile.d`, or your account's shell initialization files.

Red Hat Enterprise Linux 9 and later versions consistently apply a default `umask` of `0022` for all users, in both login and non-login shells.

Chapter 12.

Installing and Updating Software with RPM

Goal

Download, install, update, and manage software packages from Red Hat and DNF package repositories.

Sections

- Investigating RPM Software Packages (and Guided Exercise)
- Installing and Updating Packages with DNF (and Guided Exercise)
- Enabling DNF Software Repositories (and Guided Exercise)

Lab

- Install and Update Software with RPM

12.1. Investigating RPM Software Packages

Objectives

- Explain how Red Hat and other sources provide software as RPM packages, and investigate the installed system packages.

RPM Software Packages

The RPM Package Manager, which Red Hat originally developed, provides a standard way to package software for distribution. Managing software in the form of RPM packages is simpler than working with software that is extracted to a file system from an archive. With RPM packages, administrators can track which files the software package installs, which files the software package removes if you uninstall it, and it verifies that supporting packages are present when you install it. The local RPM database on your system stores the information about installed packages. Red Hat provides all software for Red Hat Enterprise Linux as an RPM package.

RPM package file names consist of four elements (plus the `.rpm` suffix): `name-version-release.architecture`:



Figure 22. RPM file name elements

- *Name* is one or more words to describe the contents (`coreutils`).
- *Version* is the version number of the original software (`9.5`).
- *Release* is the release number of the package based on that version, and is set by the packager, who might not be the original software developer (`6.el10`).
- *Arch* is the processor architecture that the package is compiled to run on. The `x86_64` value indicates that this package is built for the 64-bit version of the x86 instruction set (as opposed to `aarch64` for 64-bit ARM, and so on).

RPM packages are often downloaded from repositories. A repository is a central location for storing and maintaining RPM software packages.

You require only the package name to install RPM packages from repositories.

- If multiple versions exist, then the RPM Package Manager installs the package with the later version number.
- If multiple releases of a single version exist, then the RPM Package Manager installs the package with the later release number.

Each RPM package is an archive with the following components:

- The files that the package installs in your system.
- Information about the package (metadata), such as the name, version, release, and architecture; a summary and description of the package; whether it requires other packages to be installed; licensing; a package change log; and other details.
- Scripts that might run when you install, update, or remove the package. These scripts might also run when you install, update, or remove other packages.

Typically, software providers digitally sign RPM packages with *GNU Privacy Guard (GPG)* keys.

Red Hat, for example, digitally signs all packages that it releases. The RPM system verifies package integrity by confirming that the package is signed with the appropriate GPG key. The RPM system fails to install a package if the GPG signature does not match.

Updating Software with RPM Packages

Red Hat generates a complete RPM package to update software. An administrator who installs that package gets only the most recent version of the package. You do not need to install an earlier version of a package to patch it. To update software, RPM removes the earlier version of the package and installs the latest version. Updates usually retain configuration files, but the packager of the new version defines the exact behavior.

Typically, only one version of a package is installed at a time. If a package is built with nonconflicting file names, then you might install multiple versions. The `kernel` package is an example of installing multiple package versions. Because you test a new kernel only by booting to that kernel, the package is designed to support installing multiple versions. If the new kernel fails to boot, then you can revert to the previous kernel.

Inspecting RPM Packages

The `rpm` utility is a low-level tool that can retrieve information about the contents of package files and installed packages. By default, the tool gets information from a local database of installed packages.

Use the `rpm` command `-p` option to get information about a downloaded but uninstalled package file. Use this option to inspect the package contents before installing.

Retrieve general information about installed packages:

- `rpm -qa` : List all installed packages.
- `rpm -qf FILENAME` : Determine which package provides *FILENAME*.

```
user@host:~$ rpm -qf /etc/yum.repos.d
redhat-release-10.0-30.el10.x86_64
```

Get information about specific packages:

- `rpm -q` : List the currently installed package version.

```
user@host:~$ rpm -q dnf
dnf-4.20.0-12.el10_0.noarch
```

- `rpm -qi` : Get detailed package information.
- `rpm -ql` : List the files that the package installs.

```
user@host:~$ rpm -ql dnf
/usr/bin/dnf
/usr/lib/systemd/system/dnf-makecache.service
/usr/lib/systemd/system/dnf-makecache.timer
/usr/share/bash-completion/completions/dnf
...output omitted...
```

- `rpm -qc` : List only the configuration files that the package installs.

```
user@host:~$ rpm -qc openssh-clients
/etc/ssh/ssh_config
/etc/ssh/ssh_config.d/50-redhat.conf
```

- `rpm -qd` : List only the documentation files that the package installs.

```
user@host:~$ rpm -qd openssh-clients
/usr/share/man/man1/scp.1.gz
/usr/share/man/man1/sftp.1.gz
/usr/share/man/man1/ssh-add.1.gz
/usr/share/man/man1/ssh-agent.1.gz
...output omitted...
```

- `rpm -q --scripts`: List the shell scripts that run before or after you install or remove the package.

```
user@host:~$ rpm -q --scripts openssh-server
preinstall scriptlet (using /bin/sh):

# generated from openssh-server-systemd-sysusers.conf
getent group 'sshd' >/dev/null || groupadd -f -g '74' -r 'sshd' || :
if ! getent passwd 'sshd' >/dev/null; then
    if ! getent passwd '74' >/dev/null; then
        useradd -r -u '74' -g 'sshd' -d '/usr/share/empty.sshd' -s
'/usr/sbin/nologin' -c 'Privilege-separated SSH' 'sshd' || :
    else
        useradd -r -g 'sshd' -d '/usr/share/empty.sshd' -s '/usr/sbin/nologin'
-c 'Privilege-separated SSH' 'sshd' || :
    fi
fi
...output omitted...
```

- `rpm -q --changelog`: List the change log information for the package.

```
user@host:~$ rpm -q --changelog audit
* Wed Jan 08 2025 Attila Lakatos <alakatos@redhat.com> - 4.0.3-1
- Rebase to 4.0.3
- Plugininst must have .conf suffix, otherwise skipped
  Resolves: RHEL-58838
- ausearch checkpoint inode fix
  Resolves: RHEL-62333
- Audisp-filter: filter audit events and forward them to other plugins
  Resolves: RHEL-5199
- Log to console when system is halted due to audit not having enough storage
  Resolves: RHEL-990
...output omitted...
```

Query local package files:

- `rpm -qlp` : List the files that the local package installs.

```
user@host:~$ ls -l podman-5.4.0-1.el10.x86_64.rpm
-rw-r--r--. 1 user user 16494682 Jun  9 23:43 podman-5.4.0-1.el10.x86_64.rpm
```

```
user@host:~$ rpm -qlp podman-5.4.0-1.el10.x86_64.rpm
/usr/bin/podman
/usr/lib/.build-id
/usr/lib/.build-id/0b
...output omitted...
```

Installing RPM Packages

Use the `rpm` command to install an RPM package that you downloaded to your local directory.

```
root@host:~# rpm -ivh podman-5.4.0-1.el10.x86_64.rpm
Verifying...                               ##### [100%]
Preparing...                               ##### [100%]
Updating / installing...
      podman-6:5.4.0-1                       ##### [100%]
```

Warning

Be careful when installing packages from third parties. The RPM package might include unstable, outdated, or malicious software, and it might include arbitrary scripts that run as the root user as part of the installation process.

Extracting RPM Packages

Use the `rpm2cpio` command to extract files from an RPM package file without installing the package.

The `rpm2cpio` command converts an RPM package to a `cpio` archive. After the RPM package is converted to a `cpio` archive, the `cpio` command can extract a list of files.

Use the `cpio` command with the `-i` option to extract files from standard input. Use the `-d` option to create subdirectories as needed, starting in the current working directory. Use the `-v` option for verbose output.

```

user@host:~/tmp-extract$ rpm2cpio httpd-2.4.63-1.el10.x86_64.rpm | cpio -idv
./etc/httpd/conf.modules.d/00-brotli.conf
./etc/httpd/conf.modules.d/00-systemd.conf
./usr/lib/.build-id
./usr/lib/systemd/system/htcacheclean.service
./usr/lib/systemd/system/httpd.service
./usr/lib/systemd/system/httpd.socket
...output omitted...
121 blocks
user@host:~/tmp-extract$ ls -l
total 56
drwxr-xr-x. 3 user user      19 Jun  9 23:51 etc
-rw-r--r--. 1 root root    54557 Jun  9 23:50 httpd-2.4.63-1.el10.x86_64.rpm
drwxr-xr-x. 5 user user      43 Jun  9 23:51 usr

```

Extract individual files by specifying the path of the file:

```

user@host:~$ rpm2cpio httpd-2.4.63-1.el10.x86_64.rpm | \
cpio -id "/etc/httpd/conf.modules.d/00-systemd.conf"
121 blocks
user@host:~$ ls etc/httpd/conf.modules.d/
00-brotli.conf 00-systemd.conf

```

Use the `rpm2cpio` and `cpio -t` commands to list the files in an RPM package. Use the `-v` option of the `cpio` command for verbose output.

```
user@host:~$ rpm2cpio httpd-2.4.63-1.el10.x86_64.rpm | cpio -tv
-rw-r--r--  1 root  root   47 Jan 27 00:00 ./etc/httpd/conf.modules.d/00-
brotli.conf
-rw-r--r--  1 root  root   88 Jan 27 00:00 ./etc/httpd/conf.modules.d/00-
systemd.conf
-rw-r--r--  1 root  root  435 Jan 27 00:00
./usr/lib/systemd/system/htcacheclean.service
-rw-r--r--  1 root  root 1340 Jan 27 00:00
./usr/lib/systemd/system/httpd.service
-rw-r--r--  1 root  root  244 Jan 27 00:00
./usr/lib/systemd/system/httpd.socket
...output omitted...
121 blocks
```

References

rpm(8), rpm2cpio(8), cpio(1), and rpmkeys(8) man pages

12.2. Guided Exercise

Investigate an RPM Software Package

Gather information about a third-party package, extract files from the package for inspection, and then install it on a server.

Outcomes

- Install a package that is not from the software repositories.

Prerequisites

As the student user on the workstation machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
student@workstation:~$ lab start software-rpm
```

Instructions

1. From the workstation machine, log in to the servera machine as the student user.

```
student@workstation:~$ ssh student@servera
...output omitted...
[student@servera ~]$
```

2. On the servera machine, inspect the RPM package in the student user home directory. List the files in the package and review the script that the system runs when you install or uninstall the package.

- 2.1. List the contents of the student user home directory.

```
[student@servera ~]$ ls
rhcsa-script-1.0.0-1.noarch.rpm
```

- 2.2. View information for the `rhcsa-script-1.0.0-1.noarch.rpm` package.


```
[student@servera ~]$ rpm -q -p rhcsa-script-1.0.0-1.noarch.rpm -i
Name       : rhcsa-script
Version    : 1.0.0
Release    : 1
Architecture: noarch
Install Date: (not installed)
Group      : System
Size       : 593
License    : GPL
Signature  : (none)
Source RPM : rhcsa-script-1.0.0-1.src.rpm
Build Date : Wed 23 Mar 2022 08:24:21 AM EDT
Build Host : localhost
Packager   : Bernardo Gargallo
URL        : http://example.com
Summary    : RHCSA Practice Script
Description:
A RHCSA practice script.
The package changes the motd.
```

The `rhcsa-script` package modifies the *Message of the Day (MOTD)* banner of the system. The system displays the MOTD to users at login.

2.3. List files in the `rhcsa-script-1.0.0-1.noarch.rpm` package.

```
[student@servera ~]$ rpm -q -p rhcsa-script-1.0.0-1.noarch.rpm -l
/opt/rhcsa-script/mymotd
```

The `rhcsa-script` package includes the `mymotd` file, which contains the message that updates the MOTD.

2.4. View the script that runs when you install or uninstall the `rhcsa-script-1.0.0-1.noarch.rpm` package.

The script includes sections of code that execute when the package is installed or uninstalled.

```
[student@servera ~]$ rpm -q -p rhcsa-script-1.0.0-1.noarch.rpm --scripts
preinstall scriptlet (using /bin/sh):
if [ "$1" == "2" ]; then
    if [ -e /etc/motd.orig ]; then
        mv -f /etc/motd.orig /etc/motd
    fi
fi
postinstall scriptlet (using /bin/sh):
...output omitted...
postuninstall scriptlet (using /bin/sh):
...output omitted...
```

3. Extract the contents of the `rhcsa-script-1.0.0-1.noarch.rpm` package to the `/home/student` directory.

- 3.1. Use the `rpm2cpio` and `cpio -tv` commands to list the files in the `rhcsa-script-1.0.0-1.noarch.rpm` package.

```
[student@servera ~]$ rpm2cpio rhcsa-script-1.0.0-1.noarch.rpm | cpio -tv
-rw-r--r--  1 root      root           593 Mar 23 08:24 ./opt/rhcsa-script/mymotd
2 blocks
```

- 3.2. Extract all files from the `rhcsa-script-1.0.0-1.noarch.rpm` package to the `/home/student` directory. Use the `rpm2cpio` and `cpio -idv` commands to extract the files and to create the parent directories where needed in verbose mode.

```
[student@servera ~]$ rpm2cpio rhcsa-script-1.0.0-1.noarch.rpm | cpio -idv
./opt/rhcsa-script/mymotd
2 blocks
```

- 3.3. List the files in the `/home/student/opt` directory to verify that the extracted files are the same as the files inside the package.

```
[student@servera ~]$ ls -lR opt
opt:
total 0
drwxr-xr-x. 2 student student 20 Jun 18 19:29 rhcsa-script

opt/rhcsa-script:
total 4
-rw-r--r--. 1 student student 593 Jun 18 19:29 mymotd
```

4. Install the `rhcsa-script-1.0.0-1.noarch.rpm` package by using superuser privileges. Use `student` as the password to authenticate when using Sudo.

- 4.1. Use the `sudo rpm -ivh` command to install the `rhcsa-script-1.0.0-1.noarch.rpm` RPM package.

```
[student@servera ~]$ sudo rpm -ivh rhcsa-script-1.0.0-1.noarch.rpm
[sudo] password for student: student
Verifying... ##### [100%]
Preparing... ##### [100%]
Updating / installing...
   1:rhcsa-script-1.0.0-1 ##### [100%]
[student@servera ~]$
```

- 4.2. Verify that the package is correctly installed.

```
[student@servera ~]$ rpm -q rhcsa-script
rhcsa-script-1.0.0-1.noarch
```

5. Exit from the `servera` machine and log in again to view the new message of the day.

```
[student@servera ~]$
```

6. Return to the workstation system as the student user.

```
[student@servera ~]$ exit
logout
Connection to servera closed.
student@workstation:~$
```

Finish

On the workstation machine, change to the student user home directory and use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish software-rpm
```

12.3. Installing and Updating Packages with DNF

Objectives

- Find, install, and update software packages with DNF.

Managing Software Packages with DNF

Starting with Red Hat Enterprise Linux 9, DNF (Dandified YUM) replaces YUM as the package manager. DNF commands are functionally the same as YUM commands. For compatibility, YUM commands still exist as symbolic links to DNF:

```
user@host:~$ ls -l /bin/ | grep yum | awk '{print $9 " " $10 " " $11}'
yum -> dnf-3
yum-builddep -> ../libexec/dnf-utils
yum-config-manager -> ../libexec/dnf-utils
yum-groups-manager -> ../libexec/dnf-utils
yumdownloader -> ../libexec/dnf-utils
```

Red Hat recommends using the `dnf` command. Some documentation might still refer to the `yum` command, but the files are the same linked command.

The low-level `rpm` command can be used to install packages, but it is not designed to work with package repositories or to resolve dependencies from multiple sources automatically.

DNF improves RPM-based software installation and updates. With the `dnf` command, you can install, update, remove, and get information about software packages and their dependencies. You can get a history of transactions and work with multiple Red Hat and third-party software repositories.

The BaseOS and Application Stream Repositories

Red Hat Enterprise Linux 10 distributes content through two main software repositories: *BaseOS* and *Application Stream* (AppStream). Both BaseOS and AppStream are necessary parts of a Red Hat Enterprise Linux 10 system.

The BaseOS repository provides the core operating system content for Red Hat Enterprise Linux as RPM packages. BaseOS components have the same lifecycle as content in previous Red Hat Enterprise Linux releases.

Application Stream contains necessary parts of the system, as well as a wide range of applications that were previously available as part of Red Hat Software Collections and other products and programs.

Each Application Stream has a lifecycle that is either the same as Red Hat Enterprise Linux 10 or shorter.

Finding Software with DNF

The `dnf help` command displays usage information. The `dnf list` command displays installed and available packages.

The following example uses a wildcard character to list the installed and available packages that start with the `http` keyword.

```
user@host:~$ dnf list http*
Available Packages
httpcomponents-client.noarch 4.5.14-9.el10 rhel-10.0-for-x86_64-appstream-rpms
httpcomponents-core.noarch 4.4.16-9.el10 rhel-10.0-for-x86_64-appstream-rpms
httpd.x86_64 2.4.63-1.el10 rhel-10.0-for-x86_64-appstream-rpms
httpd-core.x86_64 2.4.63-1.el10 rhel-10.0-for-x86_64-appstream-rpms
httpd-devel.x86_64 2.4.63-1.el10 rhel-10.0-for-x86_64-appstream-rpms
httpd-filesystem.noarch 2.4.63-1.el10 rhel-10.0-for-x86_64-appstream-rpms
httpd-manual.noarch 2.4.63-1.el10 rhel-10.0-for-x86_64-appstream-rpms
httpd-tools.x86_64 2.4.63-1.el10 rhel-10.0-for-x86_64-appstream-rpms
```

The `dnf search KEYWORD` command lists packages by keywords that are in the name and summary fields only. To search for packages with "web server" in their name, summary, and description fields, use `search all` with the `web server` keyword:

```

user@host:~$ dnf search all 'web server'
Last metadata expiration check: 0:11:50 ago on Wed Jun 11 18:59:21 2025.
===== Summary & Description Matched: web server
=====
nginx.x86_64 : A high performance web server and reverse proxy server
pcp-pmda-weblog.x86_64 : Performance Co-Pilot (PCP) metrics from web server
logs
python3-tornado.x86_64 : Scalable, non-blocking web server and tools
===== Summary Matched: web server
=====
libcurl.x86_64 : A library for getting files from web servers
===== Description Matched: web server
=====
freeradius.x86_64 : High-performance and highly configurable free RADIUS server
git-instaweb.noarch : Repository browser in gitweb
httpd.x86_64 : Apache HTTP Server
mod_auth_openidc.x86_64 : OpenID Connect auth module for Apache HTTP Server
varnish.x86_64 : High-performance HTTP accelerator

```

The `dnf info PACKAGENAME` command returns detailed information about a package, including the needed disk space for installation. For example, the following command retrieves information about the `httpd` package:

```

user@host:~$ dnf info httpd
Available Packages
Name           : httpd
Version        : 2.4.63
Release        : 1.el10
Architecture   : x86_64
Size           : 53 k
Source          : httpd-2.4.63-1.el10.src.rpm
Repository     : rhel-10.0-for-x86_64-appstream-rpms
Summary        : Apache HTTP Server
URL            : https://httpd.apache.org/
License        : Apache-2.0 AND (BSD-3-Clause AND metemail AND HPND-sell-variant
                : AND Spencer-94)
Description    : The Apache HTTP Server is a powerful, efficient, and extensible
                : web server.

```

The `dnf provides PATHNAME` command displays packages that match the specified path name (the path names often include wildcard characters). For example, the following command finds packages that provide the `/var/www/html` directory:

```
user@host:~$ dnf provides /var/www/html
...output omitted...
httpd-filesystem-2.4.63-1.el10.noarch : The basic directory layout for the
                                         : Apache HTTP Server
Repo          : rhel-10.0-for-x86_64-appstream-rpms
Matched from:
Filename      : /var/www/html
```

Installing, Updating, and Removing Software with DNF

The `dnf install PACKAGENAME` command obtains and installs a software package, including any dependencies.


```
user@host:~$ sudo dnf install httpd
Dependencies resolved.
=====
=
Package                Arch    Version      Repository                                Size
=====
=
Installing:
httpd                  x86_64 2.4.63-1.el10 rhel-10.0-for-x86_64-appstream-rpms    53 k
Installing dependencies:
apr                    x86_64 1.7.5-2.el10  rhel-10.0-for-x86_64-appstream-rpms   132 k
apr-util              x86_64 1.6.3-21.el10 rhel-10.0-for-x86_64-appstream-rpms   101 k
apr-util-lmdb         x86_64 1.6.3-21.el10 rhel-10.0-for-x86_64-appstream-rpms    16 k
httpd-core            x86_64 2.4.63-1.el10 rhel-10.0-for-x86_64-appstream-rpms   1.5 M
httpd-filesystem      noarch 2.4.63-1.el10 rhel-10.0-for-x86_64-appstream-rpms    17 k
httpd-tools           x86_64 2.4.63-1.el10 rhel-10.0-for-x86_64-appstream-rpms    87 k
redhat-logos-httpd    noarch 100.0-2.el10  rhel-10.0-for-x86_64-appstream-rpms    18 k
Installing weak dependencies:
apr-util-openssl      x86_64 1.6.3-21.el10 rhel-10.0-for-x86_64-appstream-rpms    18 k
mod_http2             x86_64 2.0.29-2.el10 rhel-10.0-for-x86_64-appstream-rpms   169 k
mod_lua               x86_64 2.4.63-1.el10 rhel-10.0-for-x86_64-appstream-rpms    61 k

Transaction Summary
=====
=
Install 11 Packages

Total download size: 2.2 M
Installed size: 6.0 M
Is this ok [y/N]: y
Downloading Packages:
(1/11): apr-util-1.6.3-21.el10.x86_64.rpm      5.1 MB/s | 101 kB      00:00
(2/11): apr-util-lmdb-1.6.3-21.el10.x86_64.rpm 697 kB/s | 16 kB       00:00
(3/11): apr-1.7.5-2.el10.x86_64.rpm           4.2 MB/s | 132 kB      00:00
(4/11): apr-util-openssl-1.6.3-21.el10.x86_64.r 1.7 MB/s | 18 kB       00:00
(5/11): httpd-2.4.63-1.el10.x86_64.rpm        817 kB/s | 53 kB       00:00
(6/11): httpd-filesystem-2.4.63-1.el10.noarch.r 275 kB/s | 17 kB       00:00
(7/11): httpd-tools-2.4.63-1.el10.x86_64.rpm   12 MB/s | 87 kB        00:00
(8/11): httpd-core-2.4.63-1.el10.x86_64.rpm    9.0 MB/s | 1.5 MB      00:00
(9/11): mod_lua-2.4.63-1.el10.x86_64.rpm       615 kB/s | 61 kB       00:00
(10/11): mod_http2-2.0.29-2.el10.x86_64.rpm    1.5 MB/s | 169 kB      00:00
```

```

(11/11): redhat-logos-httpd-100.0-2.el10.noarch 208 kB/s | 18 kB      00:00
-----
-
Total                                          7.5 MB/s | 2.2 MB      00:00
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing      :                                1/1
  Installing     : apr-1.7.5-2.el10.x86_64        1/11
...output omitted...
  Installing     : httpd-2.4.63-1.el10.x86_64      11/11
  Running scriptlet: httpd-2.4.63-1.el10.x86_64    11/11

Installed:
  apr-1.7.5-2.el10.x86_64          apr-util-1.6.3-21.el10.x86_64
  apr-util-lmdb-1.6.3-21.el10.x86_64  apr-util-openssl-1.6.3-21.el10.x86_64
  httpd-2.4.63-1.el10.x86_64      httpd-core-2.4.63-1.el10.x86_64
...output omitted...

Complete!

```

The `dnf update PACKAGENAME` command obtains and installs a later version of the specified package, including any dependencies. Generally, the process tries to preserve configuration files in place, but in some cases, those files might be renamed if the packager considers that the previous file name will not work after the update. If no package name is specified, then the command installs all relevant updates.

```
user@host:~$ sudo dnf update
```

Because a new kernel can be tested only by booting to that kernel, the package specifically supports the installation of multiple versions at the same time. If the new kernel fails to boot, then the earlier kernel is still available. Running the `dnf update kernel` command installs the new kernel. The configuration files hold a list of packages to always install even if the administrator requests an update.

Use the `dnf list kernel` command to list all installed and available kernels. To view the currently running kernel, use the `uname` command. The `uname` command `-r` option shows only the kernel version and release, and the `uname` command `-a` option shows the kernel release and additional information.

```
user@host:~$ dnf list kernel
```

```
Installed Packages
```

```
kernel.x86_64      6.12.0-55.9.1.el10_0      @System
kernel.x86_64      6.12.0-55.12.1.el10_0     @rhel-10-for-x86_64-baseos-
rpms
```

```
user@host:~$ uname -r
```

```
6.12.0-55.12.1.el10_0.x86_64
```

```
user@host:~$ uname -a
```

```
Linux workstation 6.12.0-55.12.1.el10_0.x86_64 #1 SMP PREEMPT_DYNAMIC Tue May
13 02:51:44 EDT 2025 x86_64 GNU/Linux
```

The `dnf remove PACKAGENAME` command removes an installed software package, including any supported packages.

```
user@host:~$ sudo dnf remove httpd
```

Warning

The `dnf remove` command removes the listed packages *and any package that requires those packages* (and packages which require those packages, and so on). This command can lead to unexpected removal of packages, so carefully review the list of packages to be removed.

Installing and Removing Groups of Software with DNF

With DNF, you can also install and remove *groups*, which are collections of related software that are installed together.

In Red Hat Enterprise Linux, the `dnf` command can install two kinds of package groups. *Regular groups* are collections of packages. *Environment groups* are collections of regular groups. The packages or groups that these collections provide might be listed as mandatory (they must be installed if the group is installed), default (typically installed if the group is installed), or optional (not installed when the group is installed, unless specifically requested).

Similar to the `dnf list` command, the `dnf group list` command shows the names of installed and available groups.

```

user@host:~$ dnf group list
Available Environment Groups:
  Server with GUI
  Server
  Minimal Install
...output omitted...
Available Groups:
  Legacy UNIX Compatibility
  Smart Card Support
  Console Internet Tools
  Development Tools
...output omitted...

```

Some groups are installed through environment groups and are hidden by default. List these hidden groups with the `dnf group list hidden` command.

The `dnf group info` command displays information about a group. It includes a list of mandatory, default, and optional package names.

```

user@host:~$ dnf group info 'Security Tools'
Group: Security Tools
Description: Security tools for integrity and trust verification.
Default Packages:
  scap-security-guide
Optional Packages:
  aide
  hmaccalc
  openscap
  openscap-engine-sce
  openscap-utils
  scap-security-guide-doc
  tpm2-tools
  tss2
  udica

```

The `dnf group install` command installs a group that installs its mandatory and default packages and their dependent packages.

```
user@host:~$ sudo dnf group install 'System Tools'
...output omitted...
Installing Groups:
  System Tools

Transaction Summary
=====
Install 11 Packages

Total download size: 13 M
Installed size: 44 M
Is this ok [y/N]: y
...output omitted...

Complete!
```

Viewing Transaction History

All installation and removal transactions are logged in the /var/log/dnf.rpm.log file.

```
user@host:~$ tail -5 /var/log/dnf.rpm.log
2025-06-11T20:19:56+0000 SUBDEBUG Installed: samba-client-4.21.3-
102.el10.x86_64
2025-06-11T20:19:56+0000 SUBDEBUG Installed: nmap-4:7.92-3.el10.x86_64
2025-06-11T20:19:56+0000 SUBDEBUG Installed: zsh-5.9-15.el10.x86_64
2025-06-11T20:19:57+0000 SUBDEBUG Installed: setserial-2.17-63.el10.x86_64
2025-06-11T20:19:57+0000 SUBDEBUG Installed: openldap-clients-2.6.8.el10.x86_64
```

The dnf history command displays a summary of installation and removal transactions.

```
user@host:~$ dnf history
ID      | Command line                | Date and time      | Action(s)          |
-----|-----
11 | group install System Tool | 2025-06-11 20:19 | Install            | 12
EE
10 | install httpd             | 2025-06-11 19:40 | Install            | 11
9  | install bind               | 2025-06-10 09:42 | Install            | 2
...output omitted...
```

Summary of DNF Commands

Packages can be located, installed, updated, and removed by name or by package groups.

Command:	Task:
<code>dnf list [NAME-PATTERN]</code>	List installed and available packages by name.
<code>dnf group list</code>	List installed and available groups.
<code>dnf search KEYWORD</code>	Search for a package by keyword.
<code>dnf info PACKAGENAME</code>	Show details of a package.
<code>dnf install PACKAGENAME</code>	Install a package.
<code>dnf group install GROUPNAME</code>	Install a package group.
<code>dnf update</code>	Update all packages.
<code>dnf remove PACKAGENAME</code>	Remove a package.
<code>dnf history</code>	Display transaction history.

 References

`dnf(1)` and `dnf.conf(5)` man pages

For more information, refer to the *Managing Software with the DNF Tool* guide at https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html-single/managing_software_with_the_dnf_tool

12.4. Guided Exercise

Install and Update Packages with DNF

Find, install, and update software packages with DNF.

Outcomes

- Install and remove packages with dependencies.

Prerequisites

As the student user on the workstation machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
student@workstation:~$ lab start software-dnf
```

Instructions

1. From the workstation machine, log in to the servera machine as the student user and switch to the root user. Use student as the password.

```
student@workstation:~$ ssh student@servera
...output omitted...
[student@servera ~]$ sudo -i
[sudo] password for student: student
[root@servera ~]#
```

2. Install and then remove the `nmap` package.

- 2.1. First, attempt to use the `nmap` package by running the `nmap` command.

```
[root@servera ~]# nmap
-bash: nmap: command not found
```

The `nmap` package is not installed.

- 2.2. Search for packages with `nmap` as part of their name or summary.

```
[root@servera ~]# dnf search nmap
...output omitted...
===== Name Exactly Matched: nmap =====
nmap.x86_64 : Network exploration tool and security scanner
===== Name & Summary Matched: nmap =====
nmap-ncat.x86_64 : Nmap's Netcat replacement
```

2.3. Obtain more information about the nmap package.

```
[root@servera ~]# dnf info nmap
...output omitted...
Available Packages
Name       : nmap
Epoch     : 4
Version    : 7.92
Release    : 3.el10
Architecture : x86_64
Size       : 5.6 M
Source     : nmap-7.92-3.el10.src.rpm
Repository : rhel-10.0-for-x86_64-appstream-rpms
...output omitted...
```

2.4. Install the nmap package.

When prompted, press to install the package.


```
[root@servera ~]# dnf install nmap
```

```
...output omitted...
```

```
Dependencies resolved.
```

```
=====
Package
Arch      Version      Repository
=====
Installing:
nmap      x86_64       4:7.92-3.el10      rhel-10.0-for-x86_64-appstream-rpms  5.6 M
```

```
Transaction Summary
```

```
=====
Install 1 Package
```

```
Total download size: 5.6 M
```

```
Installed size: 24 M
```

```
Is this ok [y/N]: y
```

```
...output omitted...
```

```
Complete!
```

2.5. Verify that the nmap package is installed by obtaining the version of the tool.

```
[root@servera ~]# nmap -V
```

```
Nmap version 7.92 ( https://nmap.org )
```

```
...output omitted...
```

2.6. Remove the nmap package.

When prompted, press to remove the package.

```
[root@servera ~]# dnf remove nmap
Dependencies resolved.
```

```
=====
Package
  Arch      Version      Repository      Size
=====
Removing:
  nmap      x86_64      4:7.92-3.el10  @rhel-10.0-for-x86_64-appstream-rpms  24 M

Transaction Summary
=====
Remove 1 Package

Freed space: 24 M
Is this ok [y/N]: y
...output omitted...

Complete!
```

3. Install and then remove the Security Tools component group.

3.1. List the available component groups.

```
[root@servera ~]# dnf group list
Available Environment Groups:
  Server with GUI
  Server
...output omitted...
  Virtualization Host
Available Groups:
...output omitted...
  RPM Development Tools
  Scientific Support
  Security Tools
  System Tools
```

3.2. Obtain more information about the Security Tools component group, and review the list of included packages.

```
[root@servera ~]# dnf group info 'Security Tools'
...output omitted...
Group: Security Tools
Description: Security tools for integrity and trust verification.
Default Packages:
  scap-security-guide
Optional Packages:
  aide
  htmccalc
  openscap
  openscap-engine-sce
  openscap-utils
  scap-security-guide-doc
  tpm2-tools
  tss2
  udica
```

3.3. Install the Security Tools component group.

When prompted, press to install the package.

```
[root@servera ~]# dnf group install 'Security Tools'
...output omitted...
Installing Groups:
  Security Tools

Transaction Summary
=====
Install 7 Packages

Total download size: 3.3 M
Installed size: 100 M
Is this ok [y/N]: y
...output omitted...

Installed:
  libtool-ltdl-2.4.7-13.el10.x86_64
  libxslt-1.1.39-6.el10.x86_64
  openscap-1:1.4.1-1.el10.x86_64
  openscap-scanner-1:1.4.1-1.el10.x86_64
  scap-security-guide-0.1.76-2.el10.noarch
  xmlsec1-1:1.2.39-3.el10.x86_64
  xmlsec1-openssl-1:1.2.39-3.el10.x86_64

Complete!
```

3.4. Verify that the Security Tools group is installed.

```
[root@servera ~]# dnf group list --installed
...output omitted...
Installed Groups:
    Security Tools
```

3.5. Remove the Security Tools group.

When prompted, press to remove the package.

```
[root@servera ~]# dnf group remove 'Security Tools'
...output omitted...
Removing Groups:
    Security Tools

Transaction Summary
=====
Remove 7 Packages

Freed space: 100 M
Is this ok [y/N]: y
...output omitted...

Removed:
    libtool-ltdl-2.4.7-13.el10.x86_64
    libxslt-1.1.39-6.el10.x86_64
    openscap-1:1.4.1-1.el10.x86_64
    openscap-scanner-1:1.4.1-1.el10.x86_64
    scap-security-guide-0.1.76-2.el10.noarch
    xmlsec1-1:1.2.39-3.el10.x86_64
    xmlsec1-openssl-1:1.2.39-3.el10.x86_64

Complete!
```

4. Explore the history option of the dnf command.

4.1. Display recent dnf history. The history might vary in your environment.

```
[root@servera ~]# dnf history
```

ID	Command line	Date and time	Action(s)	Altered
9	group remove Security Too	2025-06-12 00:07	Removed	8
8	group install Security To	2025-06-11 23:57	Install	8
7	remove nmap	2025-06-11 23:41	Removed	1
6	install nmap	2025-06-11 23:35	Install	1

...output omitted...

4.2. Inspect the last ID in your output from the preceding step. The ID might vary in your environment.

```
[root@servera ~]# dnf history info 9
```

```
Transaction ID : 9
Begin time      : Thu Jun 12 00:07:29 2025
Begin rpmdb     : a62d...dada
End time        : Thu Jun 12 00:07:30 2025 (1 seconds)
End rpmdb       : 85cd...2800
User            : Student User <student>
Return-Code     : Success
Releasever      : 10
Command Line    : group remove Security Tools
Comment        :
Packages Altered:
  Removed libtool-ltdl-2.4.7-13.el10.x86_64      @@System
  Removed libxslt-1.1.39-6.el10.x86_64          @@System
  Removed openscap-1:1.4.1-1.el10.x86_64        @@System
  Removed openscap-scanner-1:1.4.1-1.el10.x86_64 @@System
  Removed scap-security-guide-0.1.76-2.el10.noarch @@System
  Removed xmlsec1-1:1.2.39-3.el10.x86_64        @@System
  Removed xmlsec1-openssl-1:1.2.39-3.el10.x86_64 @@System
  Removed @security-tools
```

5. Return to the workstation machine as the student user.

```
[root@servera ~]# exit
logout
[student@servera ~]$ exit
logout
Connection to servera closed.
student@workstation:~$
```

Finish

On the workstation machine, change to the student user home directory and use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish software-dnf
```

12.5. Enabling DNF Software Repositories

Objectives

- Enable and disable server use of Red Hat or third-party DNF repositories.

Managing Software Repositories

By default, RHEL systems include configuration to access Red Hat repositories. In many situations, you must configure additional repositories to access software from your organization or from a third party. For example, Adobe provides some of its software for Linux through DNF repositories. In the Red Hat classroom provided for this content, the `content.example.com` server hosts DNF repositories. The `dnf` command can access repositories from a website, an FTP server, or the local file system.

You might also need to enable or disable repositories to avoid conflicts when installing packages from different sources. You can manage the system repositories by manually updating the repository files, or by using DNF. The `dnf repolist all` command lists all available repositories and their statuses:

```
root@host:~# dnf repolist all
repo id                                repo name                                status
rhel-10-for-x86_64-appstream-rpms     RHEL 10 - AppStream (RPMs)             enabled
rhel-10-for-x86_64-baseos-rpms        RHEL 10 - BaseOS (RPMs)                enabled
```

 **Note**

Red Hat subscriptions grant access to specific repositories. In the past, administrators needed to attach subscriptions on a per-system basis. Simple Content Access (SCA) simplifies how systems access repositories. With SCA, systems can access any repository from any subscription that you buy, without attaching a subscription. You can enable SCA on the Red Hat Customer Portal within **My Subscriptions > Subscription Allocations**, or on your Red Hat Satellite server.

The `dnf config-manager` command can enable and disable repositories. For example, the following command enables the `rhel-10-for-x86_64-baseos-debug-rpms` repository:

```
root@host:~# dnf config-manager --enable rhel-10-for-x86_64-baseos-debug-rpms
```

Configuring DNF Repositories

You can configure Red Hat and third-party repositories by manually updating a repository file, or by using the command line.

Manual Configuration

To manually configure a DNF repository, you can either create a `.repo` file in the `/etc/yum.repos.d/` directory, or you can add a `[repository]` section to the `/etc/dnf/dnf.conf` file. Red Hat recommends using `.repo` files, and reserving the `dnf.conf` file for additional repository configurations. The `dnf` command searches both locations by default; however, the `.repo` files take precedence. A `.repo` file contains the URL of the repository, a name, and whether to use GPG keys to verify the package signatures. If GPG keys are required, then the `.repo` file also includes the URL to point to the trusted GPG key.

Command-line Configuration

The `dnf config-manager` command can also add repositories to the machine. The following command creates a `.repo` file by using an existing repository's URL.

```
root@host:~# dnf config-manager \
--add-
repo="https://https://dl.fedoraproject.org/pub/epel/10/Everything/x86_64/"
Updating Subscription Management repositories.
Adding repo from: https://dl.fedoraproject.org/pub/epel/10/Everything/x86_64/
```

The corresponding `.repo` file is visible in the `/etc/yum.repos.d/` directory:

```
root@host:~# cd /etc/yum.repos.d
root@host:/etc/yum.repos.d# cat \
dl.fedoraproject.org_pub_epel_10_Everything_x86_64_.repo
[dl.fedoraproject.org_pub_epel_10_Everything_x86_64_]
name=created by dnf config-manager from
https://dl.fedoraproject.org/pub/epel/10/Everything/x86_64/
baseurl=https://dl.fedoraproject.org/pub/epel/10/Everything/x86_64/
enabled=1
```

Verifying Signed Packages

The `rpm` command uses GPG keys to sign packages, and imports public keys to verify the integrity and authenticity of packages. The `dnf` command uses repository configuration files to provide the GPG public key locations, and imports the keys to verify the packages.

Keys are stored in various locations on the remote repository site, such as

`https://dl.fedoraproject.org/pub/epel/RPM-GPG-KEY-EPEL-10`. Administrators should

download the key to a local file rather than use the `dnf` command to retrieve the key from an external source.

For example, the following `.repo` file uses the `gpgkey` parameter to reference a local key:

```
[EPEL]
name=EPEL 10
baseurl=https://dl.fedoraproject.org/pub/epel/10/Everything/x86_64/
enabled=1
gpgcheck=1
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-EPEL-10
```

Configuring Repositories by Using RPM Packages

To simplify installation, some repositories provide a configuration file and a GPG public key as part of an RPM package. You can import the GPG public key by using the `rpm --import` command. The `dnf install` command can download and install these RPM packages.

For example, the following example imports the `RPM-GPG-KEY-EPEL-10` (EPEL) GPG public key and installs the RHEL 10 Extra Packages for Enterprise Linux (EPEL) repository RPM:

```
user@host:~$ rpm --import \
https://dl.fedoraproject.org/pub/epel/RPM-GPG-KEY-EPEL-10
user@host:~$ dnf install \
https://dl.fedoraproject.org/pub/epel/epel-release-latest-10.noarch.rpm
```

⚠ Warning

Import the RPM GPG key before installing signed packages, to ensure that packages come from a trusted source. If the RPM GPG key is not imported, then the `dnf` command fails to install signed packages.

The `dnf` command `--nogpgcheck` option ignores missing GPG keys, but might result in installing compromised or forged packages.

The `.repo` files often list multiple repository references in a single file. Each repository reference begins with a single-word name in square brackets.

```

user@host:~$ cat /etc/yum.repos.d/epel.repo
[epel]
name=Extra Packages for Enterprise Linux $releasever - $basearch
#baseurl=https://download.example/pub/epel/$releasever/Everything/$basearch/
metalink=https://mirrors.fedoraproject.org/metalink?
repo=epel-$releasever&arch=$basearch&infra=$infra&content=$contentdir
enabled=1
gpgcheck=1
countme=1
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-EPEL-$releasever
...output omitted...
[epel-source]
name=Extra Packages for Enterprise Linux $releasever - $basearch - Source
#baseurl=https://download.example/pub/epel/$releasever/Everything/source/tree/
metalink=https://mirrors.fedoraproject.org/metalink?repo=epel-
source-$releasever&arch=$basearch&infra=$infra&content=$contentdir
enabled=0
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-EPEL-$releasever
gpgcheck=1

```

To define a repository, but not to search it by default, insert the `enabled=0` parameter. Although the `dnf config-manager` command persistently enables and disables repositories, the `dnf` command `-enablerepo=PATTERN` and `--disablerepo=PATTERN` options enable and disable repositories temporarily while the command runs.

References

`dnf(8)`, `dnf.conf(5)`, and `dnf-config-manager(8)` man pages

For more information, refer to the *Managing Software with the DNF Tool* guide at https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html-single/managing_software_with_the_dnf_tool

12.6. Guided Exercise

Enable a DNF Software Repository

Configure a server to get packages from a remote DNF repository, and then update or install a package from that repository.

Outcomes

- Configure a system to obtain software updates from a classroom server, and update the system to use the latest packages.

Prerequisites

As the student user on the workstation machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
student@workstation:~$ lab start software-repo
```

Instructions

In this guided exercise, you configure two software repositories on the servera machine. First you install a package from the custom repository at `http://repo.example.com/rhel10.0/x86_64/rhcsa-practice/rht`. Software maintainers notify you about an update for the package on the errata repository. So you then install the package from the errata repository `http://repo.example.com/rhel10.0/x86_64/rhcsa-practice/errata`.

1. From the workstation machine, log in to the servera machine as the student user, and then switch to the root user. Use student as the password.

```
student@workstation:~$ ssh student@servera
...output omitted...
[student@servera ~]$ sudo -i
[sudo] password for student: student
[root@servera ~]#
```

2. Configure the servera machine to use the custom repository at `http://repo.example.com/rhel10.0/x86_64/rhcsa-practice/rht`. Disable GPG checking for this repository.

- 2.1.** Use the `dnf config-manager` command to add the custom packages repository.

```
[root@servera ~]# dnf config-manager \
--add-repo "http://repo.example.com/rhel10.0/x86_64/rhcsa-practice/rht"
Adding repo from: http://repo.example.com/rhel10.0/x86_64/rhcsa-practice/rht
```

- 2.2.** Examine the software repository file for the custom repository in the `/etc/yum.repos.d` directory.

```
[root@servera ~]# cat \
/etc/yum.repos.d/repo.example.com_rhel10.0_x86_64_rhcsa-practice_rht.repo
[repo.example.com_rhel10.0_x86_64_rhcsa-practice_rht]
name=created by dnf config-manager from
http://repo.example.com/rhel10.0/x86_64/rhcsa-practice/rht
baseurl=http://repo.example.com/rhel10.0/x86_64/rhcsa-practice/rht
enabled=1
```

- 2.3.** Edit the custom repository file, and add the `gpgcheck=0` parameter to disable the GPG key check for the repository. The repository file should match the following content:

```
[repo.example.com_rhel10.0_x86_64_rhcsa-practice_rht]
name=created by dnf config-manager from
http://repo.example.com/rhel10.0/x86_64/rhcsa-practice/rht
baseurl=http://repo.example.com/rhel10.0/x86_64/rhcsa-practice/rht
enabled=1
gpgcheck=0
```

Save and close the file when you are done.

- 2.4.** List all repositories on the system.

```
[root@servera ~]# dnf repolist all
repo id                                repo name                                status
repo.example.com_rhel10.0_x86_64_rhcsa-practice_rht created... enabled
...output omitted...
```

- 3.** Install the `rht-system` package.

- 3.1.** List the `rht-system` package to ensure that it is available.

```
[root@servera ~]# dnf list rht-system
...output omitted...
Available Packages
rht-system.noarch 1.0.0-1 repo.example.com_rhel10.0_x86_64_rhcsa-practice_rht
...output omitted...
```

3.2. Install the rht-system package.

```
[root@servera ~]# dnf install rht-system
Dependencies resolved.
=====
Package                Arch          Version      Repository    Size
=====
Installing:
rht-system              noarch        1.0.0-1      repo...       3.7 k
...output omitted...
Is this ok [y/N]: y
...output omitted...
Installed:
rht-system-1.0.0-1.noarch
Complete!
```

3.3. Verify that the rht-system package is installed, and note the version number of the package.

```
[root@servera ~]# dnf list rht-system
...output omitted...
Installed Packages
rht-system.noarch 1.0.0-1 @repo.example.com_rhel10.0_x86_64_rhcsa-practice_rht
```

4. You are notified about an update for the rht-system package, so you configure the errata repository and update the package. As an extra precaution, you disable the custom packages repository.

4.1. List the configured repositories to obtain the ID of the custom packages repository.

```
[root@servera ~]# dnf repolist all
repo id                repo name                status
repo.example.com_rhel10.0_x86_64_rhcsa-practice_rht created... enabled
...output omitted...
```

4.2. Disable the custom packages repository.

```
[root@servera ~]# dnf config-manager \
--disable repo.example.com_rhel10.0_x86_64_rhcsa-practice_rht
```

4.3. Create the `/etc/yum.repos.d/errata.repo` file to configure the errata repository. The repository file should match the following content:

```
[errata]
name=Custom Errata
baseurl=http://repo.example.com/rhel10.0/x86_64/rhcsa-practice/errata
enabled=1
gpgcheck=0
```

Save and close the file when you are done.

4.4. List all repositories on the system.

Note that the custom packages repository is disabled and the errata repository is enabled.

```
[root@servera ~]# dnf repolist all
repo id                                repo name                                status
errata                                Custom Errata                            enabled
repo.example.com_rhel10.0_x86_64_rhcsa-practice_rht created by dnf ... disabled
...output omitted...
```

4.5. Update all software packages on the servera machine.

```
[root@servera ~]# dnf update
...output omitted...
Dependencies resolved.
=====
Package                                Arch                                Version                                Repository                                Size
=====
Upgrading:
rht-system                            noarch                            1.0.0-2                                errata                                    7.5 k
...output omitted...
Is this ok [y/N]: y
...output omitted...
Complete!
```

4.6. Verify that the `rht-system` package is upgraded, and note the version number of the package.

```
[root@servera ~]# dnf list rht-system
...output omitted...
Installed Packages
rht-system.noarch                1.0.0-2                @errata
```

5. Return to the workstation machine as the student user.

```
[root@servera ~]# exit
logout
[student@servera ~]$ exit
logout
Connection to servera closed.
student@workstation:~$
```

Finish

On the workstation machine, change to the student user home directory and use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish software-repo
```

12.7. Lab

Install and Update Software with RPM

Download, install, update, and manage software packages from Red Hat and DNF package repositories.

Outcomes

- Manage software repositories.
- Install and upgrade packages from repositories.
- Install an RPM package.

Prerequisites

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
student@workstation:~$ lab start software-review
```

Instructions

1. On the serverb machine, configure a custom software repository to install specific packages. Name the repository errata and create the errata.repo repository file. Configure the errata.repo file to use the `http://repo.example.com/rhel10.0/x86_64/rhcsa-practice/errata` repository. Do not verify GPG signatures.
2. On the serverb machine, install the `rht-system` package.
3. For security reasons, you must disable the serverb machine printer features.
4. The `lab start` script downloads the `rhcsa-script-1.0.0-1.noarch.rpm` package in the `/home/student` directory on the serverb machine.

Confirm that the `rhcsa-script-1.0.0-1.noarch.rpm` package is available on the serverb machine, and install it by using root privileges. Verify that the package is installed.
5. After completing the tasks of this activity, return to the workstation machine as the student user.

Evaluation

As the student user on the workstation machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

```
student@workstation:~$ lab grade software-review
```

Finish

On the workstation machine, change to the student user home directory and use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish software-review
```

Solution

Download, install, update, and manage software packages from Red Hat and DNF package repositories.

Outcomes

- Manage software repositories.
- Install and upgrade packages from repositories.
- Install an RPM package.

Prerequisites

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
student@workstation:~$ lab start software-review
```

Instructions

1. On the serverb machine, configure a custom software repository to install specific packages. Name the repository `errata` and create the `errata.repo` repository file. Configure the `errata.repo` file to use the `http://repo.example.com/rhel10.0/x86_64/rhcsa-practice/errata` repository. Do not verify GPG signatures.

- 1.1. From the workstation machine, log in to the serverb machine as the student user and switch to the root user. Use `student` as the password.

```
student@workstation:~$ ssh student@serverb
...output omitted...
[student@serverb ~]$ sudo -i
[sudo] password for student: student
[root@serverb ~]#
```

- 1.2. Create the `/etc/yum.repos.d/errata.repo` file with the following content:

```
[errata]
name=Custom Errata Repository
baseurl=http://repo.example.com/rhel10.0/x86_64/rhcsa-practice/errata
enabled=1
gpgcheck=0
```

2. On the serverb machine, install the rht-system package.

2.1. List the rht-system package to ensure that it is available.

```
[root@serverb ~]# dnf list rht-system
...output omitted...
Last metadata expiration check: 0:00:39 ago on Fri Jun 13 00:18:19 2025.
Available Packages
rht-system.noarch                               1.0.0-2                               errata
```

2.2. Install the latest version of the rht-system package.

```
[root@serverb ~]# dnf install rht-system
...output omitted...
Dependencies resolved.
=====
Package                Architecture    Version          Repository        Size
=====
Installing:
rht-system             noarch         1.0.0-2          errata            7.5 k
Transaction Summary
=====
Install 1 Package

Total download size: 7.5 k
Installed size: 300
Is this ok [y/N]: y
...output omitted...

Complete!
```

3. For security reasons, you must disable the serverb machine printer features.

3.1. List the installed cups package.

```
[root@serverb ~]# dnf list cups
...output omitted...
Installed Packages
cups.x86_64          1:2.4.10-11.el10          @rhel-10.0-for-x86_64-appstream-rpms
```

3.2. Remove the cups package.

```
[root@serverb ~]# dnf remove cups.x86_64
Dependencies resolved.
=====
Package      Arch    Version              Repository              Size
=====
Removing:
 cups        x86_64  1:2.4.10-11.el10    @rhel-10.0-for-x86_64-appstream-rpms  8.1 M
Removing unused dependencies:
 adobe-mappings-cmap
                noarch  20230622-5.el10    @rhel-10.0-for-x86_64-appstream-rpms  14 M
...output omitted...
Transaction Summary
=====
Remove  55 Packages

Freed space: 114 M
Is this ok [y/N]: y
...output omitted...

Complete!
```

4. The `lab_start` script downloads the `rhcsa-script-1.0.0-1.noarch.rpm` package in the `/home/student` directory on the `serverb` machine.

Confirm that the `rhcsa-script-1.0.0-1.noarch.rpm` package is available on the `serverb` machine, and install it by using root privileges. Verify that the package is installed.

- 4.1. Verify that the `rhcsa-script-1.0.0-1.noarch.rpm` package is available on the `serverb` machine.

```
[root@serverb ~]# rpm -q -p -i ~/rhcsa-script-1.0.0-1.noarch.rpm
Name       : rhcsa-script
Version    : 1.0.0
Release    : 1
Architecture: noarch
Install Date: (not installed)
Group      : System
Size       : 593
License    : GPL
Signature  : (none)
Source RPM : rhcsa-script-1.0.0-1.src.rpm
Build Date : Wed Mar 23 12:24:21 2022
Build Host : localhost
Packager   : Bernardo Gargallo
URL        : http://example.com
Summary    : RHCSA Practice Script
Description:
A RHCSA practice script.
The package changes the motd.
```

4.2. Install the rhcsa-script-1.0.0-1.noarch.rpm package.

```
[root@serverb ~]# dnf install ~/rhcsa-script-1.0.0-1.noarch.rpm
...output omitted...
Dependencies resolved.
=====
Package                Architecture Version      Repository      Size
=====
Installing:
rhcsa-script          noarch      1.0.0-1      @commandline    7.5 k

Transaction Summary
=====
Install 1 Package

Total size: 7.5 k
Installed size: 593
Is this ok [y/N]: y
...output omitted...

Complete!
```

4.3. Verify that the package is installed.

```
[root@serverb ~]# rpm -q rhcsa-script
rhcsa-script-1.0.0-1.noarch
[root@serverb ~]#
```

5. After completing the tasks of this activity, return to the workstation machine as the student user.

```
[root@serverb ~]# exit
logout
[student@serverb ~]$ exit
logout
Connection to serverb closed.
student@workstation:~$
```

Evaluation

As the student user on the workstation machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

```
student@workstation:~$ lab grade software-review
```

Finish

On the workstation machine, change to the student user home directory and use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish software-review
```

12.8. Summary

In this lesson, you learned about the tools that Red Hat Enterprise Linux provides to register systems, entitle them to product subscriptions, and receive software updates. RHEL tools help to track support contracts and subscription usage across systems.

RHEL software is packaged in the RPM file format and distributed in repositories. You can install, upgrade, or remove software by using the `rpm` and `dnf` tools. The `rpm` command enables you to query the local RPM database and manage downloaded packages. The `dnf` utility is a powerful tool for managing software packages to install, update, remove, and query packages from configured repositories.

You also learned that Red Hat Enterprise Linux provides the BaseOS and Application Stream (AppStream) repositories. BaseOS provides the core operating system content and AppStream provides a range of application packages and dependencies.

Chapter 13.

Installing and Updating Applications by Using Flatpak

Goal

Install, upgrade, and use desktop software from the Red Hat Ecosystem Catalog by using Flatpak.

Sections

- Configuring Flatpak for Application Installation (and Guided Exercise)
- Managing Applications from Flatpak (and Guided Exercise)
- Install and Update Applications by Using Flatpak (Quiz)

13.1. Configuring Flatpak for Application Installation

Objectives

- Install and configure Flatpak tools and remote repositories, and explain the basic operation of Flatpak.

Introduction to Flatpak

Although the model for packaging applications in Linux has evolved significantly over the years, it still presents challenges:

- Developers must tailor their applications for a specific Linux operating system and version. As a result, developers spend more time and effort to package applications for each distribution.
- Users might run into technical difficulties in deploying an application that is not packaged specifically for their preferred distribution.
- Development and maintenance can be complex because dependencies for an application can differ between the production environment and the development platform.

Flatpak addresses these challenges by applying server-side container concepts and Linux kernel technology to the desktop environment. When a desktop application is packaged as a Flatpak image, the application can run reliably on most Linux distributions and versions. Applications do not behave incorrectly or crash due to discrepancies in dependencies.

With Flatpak, each application is built and run in an isolated environment that is called a *sandbox*. By default, the application can access only the contents of its sandbox. The application's access to files, the network, graphics sockets, the system bus, and devices must be explicitly granted. The application's access to other resources, such as other processes, is not possible by design.

Flatpak Runtimes

In a Flatpak sandbox, the application uses its own libraries, not the operating system's libraries. However, maintaining a separate copy of all libraries for each desktop application consumes too much space, so many applications can share a Flatpak runtime. A *runtime* is a file-system image with system-level libraries and other files. A system can provide multiple runtimes that different applications can use.

Another significant benefit of runtime separation is with managing security updates. When a security vulnerability is discovered in a system library, developers must update only the runtime without having

to rebuild each application.

A runtime might not provide every library that an application requires. Therefore, the application can also bundle its own libraries.

Managing Flatpak Remote Repositories

Flatpak applications and runtimes are distributed in repositories that are named *remotes*. Red Hat packages several open source applications and distributes them in a remote repository. Although unsupported, third-party remote repositories are also available; they complement the Red Hat catalog and offer a broad selection of applications. The largest and best-known third-party repository is Flathub, which is available at <https://flathub.org>. The Fedora Project also has its own Flatpak remote repository.

Important

Red Hat does not support applications from third-party remote repositories.

You can use either the `flatpak` command or the GNOME Software graphical tool to manage packages. With either tool, you can configure multiple remote repositories, and simplify Flatpak management through unified interfaces.

Initial Flatpak Setup

Flatpak is installed by default in Red Hat Enterprise Linux 10 along with the `gnome-software` RPM package.

If Flatpak is not installed, then you can install it by using the `dnf` command:

```
root@host:~# dnf install flatpak
```

To display the version of Flatpak that is installed on your system, use the `flatpak --version` command:

```
user@host:~$ flatpak --version
Flatpak 1.16.0
```

The default remote repository for RHEL 10 is the Flatpak object repository in the Red Hat Ecosystem Catalog. To access this repository, you must provide your Red Hat Customer Portal account credentials or your registry service account tokens.

Log in to Red Hat Container Catalog by using the `podman login` command:

```
user@host:~$ podman login registry.redhat.io
Username: provide-username
Password: provide-password
Login Succeeded!
```

By default, the login credentials expire after the current session. However, you can permanently save the credentials for the current user:

```
user@host:~$ cp $XDG_RUNTIME_DIR/containers/auth.json \
$HOME/.config/flatpak/oci-auth.json
```

You can also save the credentials system-wide.

```
root@host:~# cp $XDG_RUNTIME_DIR/containers/auth.json \
/etc/flatpak/oci-auth.json
```

If you want regular users to access the remote repository with the same credentials, then you must change the file permissions on the authentication token:

```
root@host:~# chmod 644 /etc/flatpak/oci-auth.json
```

To list all the remote repositories that are configured on your system, use the `flatpak remotes` command:

```
user@host:~$ flatpak remotes
Name Options
rhel system,oci,no-gpg-verify
```

To show the details for the remote repositories, add the `-d` parameter to the `flatpak remotes` command. Details include the repository name, its unique identifier, and other properties such as the repository URL and configured options.

```
user@host:~$ flatpak remotes -d
Name Title          URL                      Collection ID      ... Priority Options  ...
rhel Red Hat... oci+https://.../ com.redhat.Stable - 1          system... ...
```

To list the applications that are available in a remote repository, use the `flatpak remote-ls` command. To ignore runtimes and list only the applications, add the `--app` option:

```
user@host:~$ flatpak remote-ls --app
Name          Application ID          Version    Branch    Arch
Thunderbird   org.mozilla.Thunderbird 128.11.0   stable    x86_64
Firefox       org.mozilla.firefox     128.11.0   stable    x86_64
```

Adding a Flatpak Remote Repository

From the command line, you can configure additional remote repositories as installation sources and expand the catalog of Flatpak applications.

For example, the Fedora Project hosts a remote registry that other distributions can also use because of Flatpak's universal design. To add this remote repository, run the `flatpak remote-add` command. The first argument defines the name for the local copy of the remote repository. The second argument defines the remote repository's location, which can point either to the actual location or to the definition file with the `.flatpakrepo` extension. The `--if-not-exists` option avoids overwriting the remote repository if it exists in the system:

```
user@host:~$ flatpak remote-add --if-not-exists fedora \
oci+https://registry.fedoraproject.org
```

To confirm that the Fedora repository is added as an installation source, use the `flatpak remotes` command to list all the remote repositories that are configured on your system:

```
user@host:~$ flatpak remotes
Name    Options
fedora  system,oci
rhel    system,oci,no-gpg-verify
```

By default, the `flatpak` command applies operations to the whole system, as if the `--system` option was used explicitly. In the previous example, the newly added remote repository is available to all users on the system to search and install packages from.

However, you can also configure a repository in user installation mode by adding the `--user` option. In that mode, the Flatpak object is available only to the user that ran the command.

The following example shows configuring the Flathub repository in user mode:

```
user@host:~$ flatpak remote-add --if-not-exists --user \
flathub https://dl.flathub.org/repo/flathub.flatpakrepo
```

When multiple repositories are enabled, the `flatpak remotes` command can help to identify which repositories are available system-wide and which repositories are for the current user only.

List the remote repositories that are available to all users:

```
user@host:~$ flatpak remotes --system
Name    Options
fedora  oci
rhel    oci,no-gpg-verify
```

List the remote repositories that are available to the current user:

```
user@host:~$ flatpak remotes --user
Name    Options
flathub
```

After adding a remote repository, you can list the repository's contents by using the `flatpak remote-ls` command:

```
user@host:~$ flatpak remote-ls fedora --app
Name                               Application ID                Version  Branch
Dialect                           app.drey.Dialect              2.4.2   stable
Multiplication Puzzle              app.drey.MultiplicationPuzzle 14.0    stable
pw3270                             br.app.pw3270.terminal        5.3     stable
Dconf Editor                       ca.desrt.dconf-editor         45.0.1  stable
AbiWord                           com.abisource.AbiWord         3.0.5   stable
ConvertAll                         com.bellz.ConvertAll          0.8.0   stable
...output omitted...
```

Removing a Flatpak Remote Repository

To remove a Flatpak remote repository, use the `flatpak remote-delete` command. Before a remote repository is deleted, all software that is installed from that repository must be uninstalled. The

command prompts you to confirm the removal of all associated references (applications and runtimes) before continuing.

```
user@host:~$ flatpak remote-delete fedora
The following refs are installed from remote 'fedora':

1) runtime/org.fedoraproject.Platform.GL.default/x86_64/f41
2) runtime/org.fedoraproject.Platform.Locale/x86_64/f41
3) app/org.filezillaorg.Filezilla/x86_64/stable
4) runtime/org.fedoraproject.Platform/x86_64/f41

Remove them? [y/n]: y
Uninstalling runtime/org.fedoraproject.Platform.GL.default/x86_64/f41
Uninstalling app/org.filezillaorg.Filezilla/x86_64/stable
Uninstalling runtime/org.fedoraproject.Platform/x86_64/f41
Uninstalling runtime/org.fedoraproject.Platform.Locale/x86_64/f41
```

Instead of permanently deleting a repository and uninstalling all software that was sourced from it, you can disable the repository. Disabling a repository does not require removing previously installed Flatpaks. To disable a repository, use the `flatpak remote-modify` command.

```
user@host:~$ flatpak remote-modify --disable fedora
```

You can also re-enable the repository later if, for example, you must update one of the packages.

```
user@host:~$ flatpak remote-modify --enable fedora
```

Managing Flatpak with the GNOME Software Graphical Tool

You can also manage the configured remote repositories with the graphical desktop tool.

From the Gnome graphical desktop environment, click the **Red Hat** logo in the upper-left corner and then select **Software** from the dashboard, or use the **Show Applications** button to open the application. Click the **Main Menu** that appears as three lines in the upper-right corner of the window, and select **Software Repositories**.

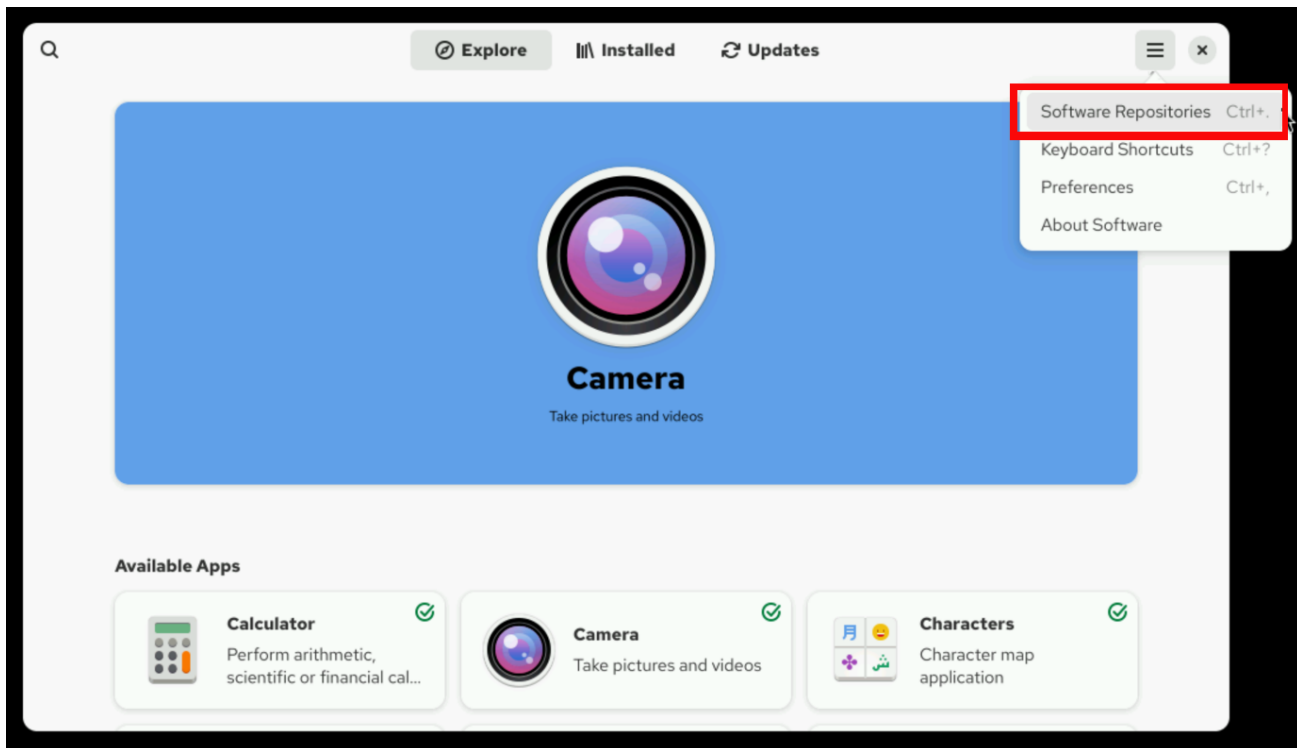


Figure 23. Software main menu

In the **Software Repositories** pop-up window, Flatpak applications are listed in the **Apps (Flatpak)** section. From this window, you can enable or disable the configured remote repositories.

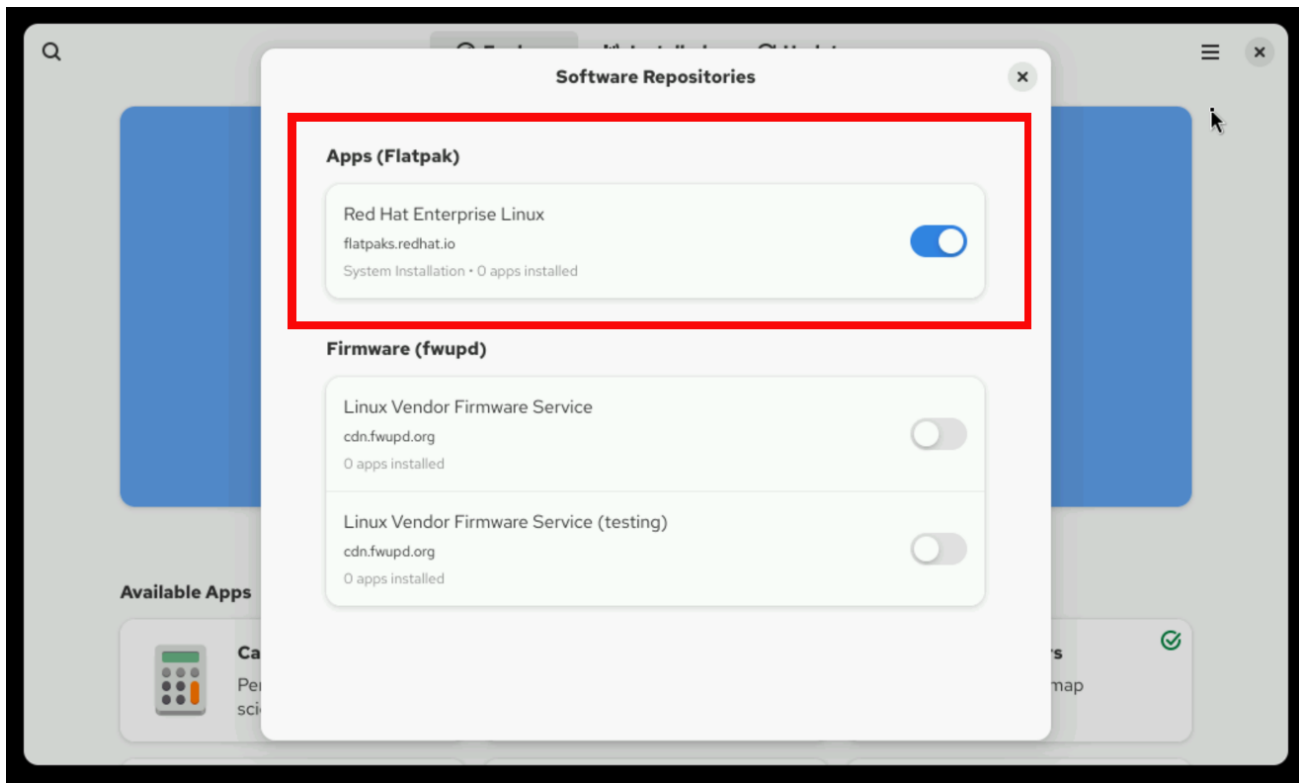


Figure 24. Software repositories

References

flatpak(1), flatpak-remotes(1), flatpak-remote-add(1), flatpak-remote-ls(1), flatpak-remote-delete(1), and flatpak-remote-modify(1) man pages

[Flatpak Documentation: Basic Concepts](#)

[Flathub](#)

13.2. Guided Exercise

Configure Flatpak for Application Installation

Install and configure Flatpak tools and access to remote repositories.

Outcomes

- Verify the configuration of the preinstalled Flatpak package manager.
- Disable and enable a remote Flatpak repository.
- Add a custom remote repository.

Prerequisites

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
student@workstation:~$ lab start flatpak-configure
```

Instructions

1. Verify that the Flatpak package manager is installed on the workstation machine and inspect its configuration.

- 1.1. Use the `rpm` command to list all installed packages that match the `flatpak` package name.

```
student@workstation:~$ rpm -qa | grep flatpak
flatpak-session-helper-1.16.0-5.el10.x86_64
flatpak-libs-1.16.0-5.el10.x86_64
flatpak-selinux-1.16.0-5.el10.noarch
flatpak-1.16.0-5.el10.x86_64
redhat-flatpak-repo-0.1-5.el10.noarch
```

- 1.2. View the system-wide configuration files for Flatpak in the `/etc/flatpak` directory.

```
student@workstation:~$ tree /etc/flatpak/
/etc/flatpak/
├── remotes.d
│   └── rhel.flatpakrepo
2 directories, 1 file
```

1.3. View the preconfigured rhel Flatpak repository file.

This remote repository is available to all users in the machine.

```
student@workstation:~$ cat /etc/flatpak/remotes.d/rhel.flatpakrepo
[Flatpak Repo]
Title=Red Hat Enterprise Linux
Url=oci+https://flatpaks.redhat.io/rhel/
AuthenticatorName=org.flatpak.Authenticator.Oci
Homepage=https://catalog.redhat.com/software/containers/explore
Comment=Red Hat Enterprise Linux Flatpak applications and runtimes
Description=This repository contains Flatpaks of applications built for Red Hat
Enterprise Linux, along with the Red Hat Flatpak Runtime and SDK.
Icon=https://www.redhat.com/misc/favicon.ico
CollectionID=com.redhat.Stable
```

1.4. List all currently enabled repositories.

Optionally, you can add the `-d` option to see verbose command output.

```
student@workstation:~$ flatpak remotes
Name Options
rhel system,oci,no-gpg-verify
```

2. Disable the rhel repository because it requires a Red Hat account for authentication and internet connectivity, which might not be available in your classroom environment.

```
student@workstation:~$ flatpak remote-modify --disable rhel
```

3. The classroom lab environment provides a local Flatpak repository. Its definition file is available at <http://flatpak.lab.example.com/myrepo.flatpakrepo>. View the contents of the `flatpakrepo` file by using the `curl` command-line browser.

```
student@workstation:~$ curl http://flatpak.lab.example.com/myrepo.flatpakrepo
[Flatpak Repo]
Title=My Flatpak Repository
Url=http://flatpak.lab.example.com/
Homepage=http://flatpak.lab.example.com
Comment=Custom Flatpak repository
Description=Repository for custom Flatpak applications
Icon=https://example.com/icon.png
```

4. Add the custom repository in user installation mode. Use myrepo as the name for the remote repository.

```
student@workstation:~$ flatpak remote-add --no-gpg-verify --user myrepo \
http://flatpak.lab.example.com/myrepo.flatpakrepo
```

5. Verify that the new remote repository is available. Install the freedesktop platform package.

5.1. List the currently enabled repositories.

Be aware that other users on the machine cannot see this repository.

```
student@workstation:~$ flatpak remotes
Name    Options
myrepo  user,no-gpg-verify
```

- 5.2. List all available applications and runtimes in the myrepo remote repository.

```
student@workstation:~$ flatpak remote-ls myrepo
```

Name	Application ID	Version
Branch		
codium	com.vscodium.codium	
stable		
Obsidian	md.obsidian.Obsidian	
stable		
freedesktop platform	org.freedesktop.Platform	24.08
freedesktop development platform	org.freedesktop.Sdk	24.08

- 5.3. Install the Freedesktop runtime by using the flatpak install command.

Runtimes contain essential libraries for the applications that are available in repositories.

```
student@workstation:~$ flatpak install org.freedesktop.Platform
Looking for matches...
      ID                               Branch Op Remote Download
  1.  org.freedesktop.Platform 24.08 i myrepo 264.5 MB
Proceed with these changes to the user installation? [Y/n]: y
Installing... ██████████ 100% 63.4 MB/s 00:00
Installation complete.
```

6. Permanently delete the myrepo remote repository by using the `flatpak remote-delete` command. Allow the tool to automatically remove all Flatpak objects that were installed from this remote repository.

```
student@workstation:~$ flatpak remote-delete myrepo
The following refs are installed from remote 'myrepo':

  1) runtime/org.freedesktop.Platform/x86_64/24.08

Remove them? [y/n]: y
Uninstalling runtime/org.freedesktop.Platform/x86_64/24.08
```

7. Re-enable the official Red Hat repository by using the GNOME Software graphical tool.

- 7.1. Click the **Red Hat** logo in the upper-left corner of the screen to open the **Activities Overview**.
- 7.2. In the search box, enter `software`, and then open the GNOME Software application from the list of results.
- 7.3. In the GNOME Software window, click the **Main Menu** that appears as three lines in the upper-right corner and select **Software Repositories**.
- 7.4. Enable the **Red Hat Enterprise Linux** remote repository by setting the switch to on.
- 7.5. Close the GNOME Software graphical tool.

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish flatpak-configure
```

13.3. Managing Applications from Flatpak

Objectives

- Use Flatpak to list, search, install, update, remove, and get metadata information of applications.

Managing Applications with the Flatpak CLI

By using Flatpak CLI commands, you can list, search, install, update, remove, and get metadata information for Flatpak applications and their runtime dependencies. You can use content from the repository from Red Hat and from third-party remote repositories. Red Hat does not support applications or runtimes from third-party remote repositories.

Flatpak Identifiers

Flatpak applications and runtimes use unique identifiers that follow the `domain.organization.App` format. This three-part identifier enables a company, developer, or organization to publish many applications with a shared prefix, such as `com.company`. The last segment (`.App`) identifies the application's name. You can use either the full or short object identifier for Flatpak operations: `com.company.App` or `App`.

In some scenarios, you must specify the CPU architecture or version, such as `stable` or `beta`. You can request these additional specifications by using an object's identifier triple. The format of an identifier triple is `ID/architecture/branch`, such as `org.organization.App/x86_64/stable`. In Flatpak, *branch* refers to different versions of the same object.

You can leave blank some parts of the identifier triple to specify part of the object. For example, `org.organization.App/x86_64//` specifies only the architecture, and `org.organization.App//stable` specifies only the branch.

Querying Remote Repositories for Available Objects

The `flatpak search` command lists matching applications and runtimes that are available in all the remote repositories that are enabled in the system. This command uses cached appstream metadata that is already downloaded from the remote repository. If the cached metadata is more than one day old, then the metadata is updated automatically when the command is executed.

The command uses a keyword to find a match in any of the metadata attributes that the application's XML file defines:

- Application or runtime name

- Description
- Application identifier
- Keyword that the developer defines

The keyword is case-insensitive, and returns information for applications and runtimes that match the query.

The following example shows a search that uses the `thunderbird` keyword to find applications in Flatpak remote repositories:

```
user@host:~$ flatpak -v search thunderbird ①
F: No installations directory in /etc/flatpak/installations.d. Skipping
F: Opening system flatpak installation at path /var/lib/flatpak ②
F: Opening user flatpak installation at path /home/user/.local/share/flatpak
F: fedora:x86_64 appstream age 491 is less than ttl 86400 ③
F: rhel:x86_64 appstream age 1136 is less than ttl 86400
F: Opening system flatpak installation at path /var/lib/flatpak ④
F: Opening user flatpak installation at path /home/user/.local/share/flatpak
Thunderbird      Thunderbird is a free and open source email, newsfeed, chat,
and calendaring client      net.thunderbird.Thunderbird      stable
fedora ⑤
Thunderbird      Thunderbird is a free and open source email, newsfeed, chat,
and calendaring client      org.mozilla.Thunderbird      stable fedora
```

- ① The Flatpak search is executed with the `-v` option for verbose output. This option is recommended for troubleshooting.
- ② Flatpak first verifies whether the cached appstream metadata is up-to-date, in both the system and user installation paths.
- ③ After verification, Flatpak detects that the cached metadata for the `rhel` remote repository is older than the upstream metadata.
- ④ Flatpak updates the outdated data by downloading only the differences between the updates and the data on disk, which saves bandwidth and disk space.
- ⑤ After all remote repositories are verified or updated, Flatpak displays the matched output in categorized columns.

Installing Flatpak Objects from Remote Repositories

The `flatpak install` command installs the requested objects, including all dependencies.

```

user@host:~$ flatpak install thunderbird
Looking for matches...
Remotes found with refs similar to 'thunderbird': ❶

    1) 'fedora' (system)
    2) 'rhel' (system)

Which do you want to use (0 to abort)? [0-2]: 2
Found ref 'app/org.mozilla.Thunderbird/x86_64/stable' in remote 'rhel'
(system).
Use this ref? [Y/n]: y
Required runtime for org.mozilla.Thunderbird/x86_64/stable
(runtime/com.redhat.Platform/x86_64/el10) found in remote rhel ❷
Do you want to install it? [Y/n]: y

org.mozilla.Thunderbird permissions: ❸
    ipc          network      pcsc                pulseaudio
    x11           dri            file access [1]     dbus access [2]

[1] home:ro, xdg-download, xdg-run/dconf,
    ~/.cache/thunderbird:create, ~/.config/dconf:ro,
    ~/.mozilla:create, ~/.thunderbird:create
[2] ca.desrt.dconf, org.a11y., org.freedesktop.Notifications


ID                               Branch Op Remote Download ❹
1.   com.redhat.Platform         el10  i  rhel   < 395.1 MB
2.   org.mozilla.Thunderbird     stable i  rhel   < 147.7 MB

Proceed with these changes to the system installation? [Y/n]: *y
Installing 1/2.. ██████████ 100% 21.5 MB/s 00:00 ❺
Installing 2/2.. ██████████ 100% 22.1 MB/s 00:00
Installation complete.
```

- 1 If you do not specify the remote repository to search and more than one repository is configured for the user who is executing the command, then Flatpak looks for all remote repositories with references that match the identifier in the operation. The identifier can be in full or short form (for example, `org.mozilla.thunderbird` or `thunderbird`).
- 2 Flatpak automatically resolves the required runtimes for the application and prompts for permission to install it. If another application requires the same runtime in the future, then it uses the previously downloaded object, which avoids duplication and saves bandwidth.

- 3 Flatpak keeps all applications in isolated environments, which require permissions to access resources in the host system. The resources that must be accessed with specific permissions are detailed at this point in the installation process.
- 4 Flatpak summarizes the operation, and enumerates the objects to be installed, with information about the full identifier, branch, remote repository, and estimated download size.
- 5 After this step, the process continues without interruption until completion.

The following table shows some useful options for the `flatpak install` command:

Option	Description
<code>--runtime</code>	Installs only runtimes
<code>--app</code>	Installs only applications
<code>--or-update</code>	Performs an update if the application or runtime is already installed
<code>--system</code>	Installs the application or runtime in the default system-wide installation
<code>-u, --user</code>	Installs the application or runtime for the current user only
<code>-y, --assumeeyes</code>	Automatically answers yes for all questions
<code>--no-deploy</code>	Downloads to local cache without installing
<code>--no-pull</code>	Installs from local cache only
<code>--noninteractive</code>	Produces minimal output and assumes yes for all questions

Querying Flatpak Objects Locally

You can list all the installed Flatpak objects by using the `flatpak list` command.

```
user@host:~$ flatpak list
```

Name	Application ID	Version	Branch	Installation
Red Hat Platform	com.redhat.Platform	10	el10	system
Thunderbird	org.mozilla.Thunderbird		stable	system

The `flatpak info` command shows information about the installed object based on the object identifier.

```
user@host:~$ flatpak info org.mozilla.Thunderbird
ID: org.mozilla.Thunderbird
Ref: app/org.mozilla.Thunderbird/x86_64/stable
Arch: x86_64
Branch: stable
Origin: rhel
Collection:
Installation: system
Installed: 324.3 MB
Runtime: com.redhat.Platform/x86_64/el10
Sdk: com.redhat.Sdk/x86_64/el10

Commit: 3dd6...1ae3
Subject: Export org.mozilla.Thunderbird
Date: 2025-05-27 09:39:04 +0000
Alt-id: 3a6f...8b6f
```

Installing Flatpak Objects from Specific Branches

Several versions of a Flatpak object might be available in a remote repository. Those versions are identified as branches. Branches can be designated by version numbers, such as 1.1 and 1.2, or by terms that represent the application lifecycle, such as *stable*, *beta*, or *nightly*.

Use the `flatpak remote-info` command to verify which branches are available for a specific object. If only one branch is available for the requested identifier, then the command shows the output immediately. If more than one branch exists for the requested identifier, then the command returns an error and prompts for more information.

```
user@host:~$ flatpak remote-info rhel com.redhat.Platform
error: Multiple branches available for com.redhat.Platform, you must specify
one of: com.redhat.Platform/x86_64/el10, com.redhat.Platform/x86_64/el9,
com.redhat.Platform/x86_64/el8
```

If you know which branch to point, then you can specify the object version by using its identifier triple.

```
user@host:~$ flatpak install com.redhat.Platform/x86_64/el8
...output omitted...
```

Updating Flatpak Objects

Use the `flatpak update` command to upgrade all your installed objects to the latest version in their respective branches. This command updates all objects. To update a single object, add the object identifier as an argument.

```
user@host:~$ flatpak update com.redhat.Platform
Looking for updates...

Nothing to do.
```

Note

The `flatpak update` command checks for updates within the same branch from which the application was initially installed. If a later version exists in a different branch, then Flatpak does not recognize the update. This behavior is similar to branches in a Git repository: checking out a branch means that you work only on that branch.

To prevent the unwanted upgrade of certain objects, use the `flatpak mask` command. To mask a single object, add the object identifier as an argument.

```
user@host:~$ flatpak mask com.redhat.Platform
```

To unmask an object, use the `flatpak mask --remove` command with the object identifier.

```
user@host:~$ flatpak mask --remove com.redhat.Platform
```

Uninstalling Flatpak Objects

The `flatpak uninstall` command removes Flatpak objects from the system.

If you intend to uninstall a runtime object, then you must ensure that no installed application uses the object as a dependency, or you must use the `--force-remove` option to ignore dependency resolution.

When you uninstall an application, the application's data directory remains in the system, in the `~/ .var/app` directory. To avoid consuming storage space with stale data, you can use the `--delete-data` option.

```
user@host:~$ flatpak uninstall --delete-data thunderbird
Found installed ref 'app/org.mozilla.Thunderbird/x86_64/stable' (system). Is
this correct? [Y/n]: y
      ID                               Branch      Op
  1. [-] org.mozilla.Thunderbird      stable      r
Uninstalling...
Delete data for org.mozilla.Thunderbird? [y/n]: y
```

Managing Flatpak Objects with the GNOME Software Graphical Tool

You can search for, install, uninstall, or update Flatpak applications by using the GNOME Software graphical tool.

From the GNOME graphical desktop environment, click **Activities**, and then select **Software** from the dashboard or use the **Show Applications** button to find the tool. You can also use the `gnome-software` command to open the GNOME Software tool.

GNOME Software integrates DNF and Flatpak packages seamlessly regardless of their different source repositories.

You can browse through the recommended applications by category, remove installed components, or check for updates. You can search for specific applications by using the search bar in the upper-left corner. In the search bar, type the keyword to search and the GNOME Software tool returns all the results that match, including both DNF packages and Flatpak objects.

After you select the intended object, the GNOME Software tool displays the details of the Flatpak component.

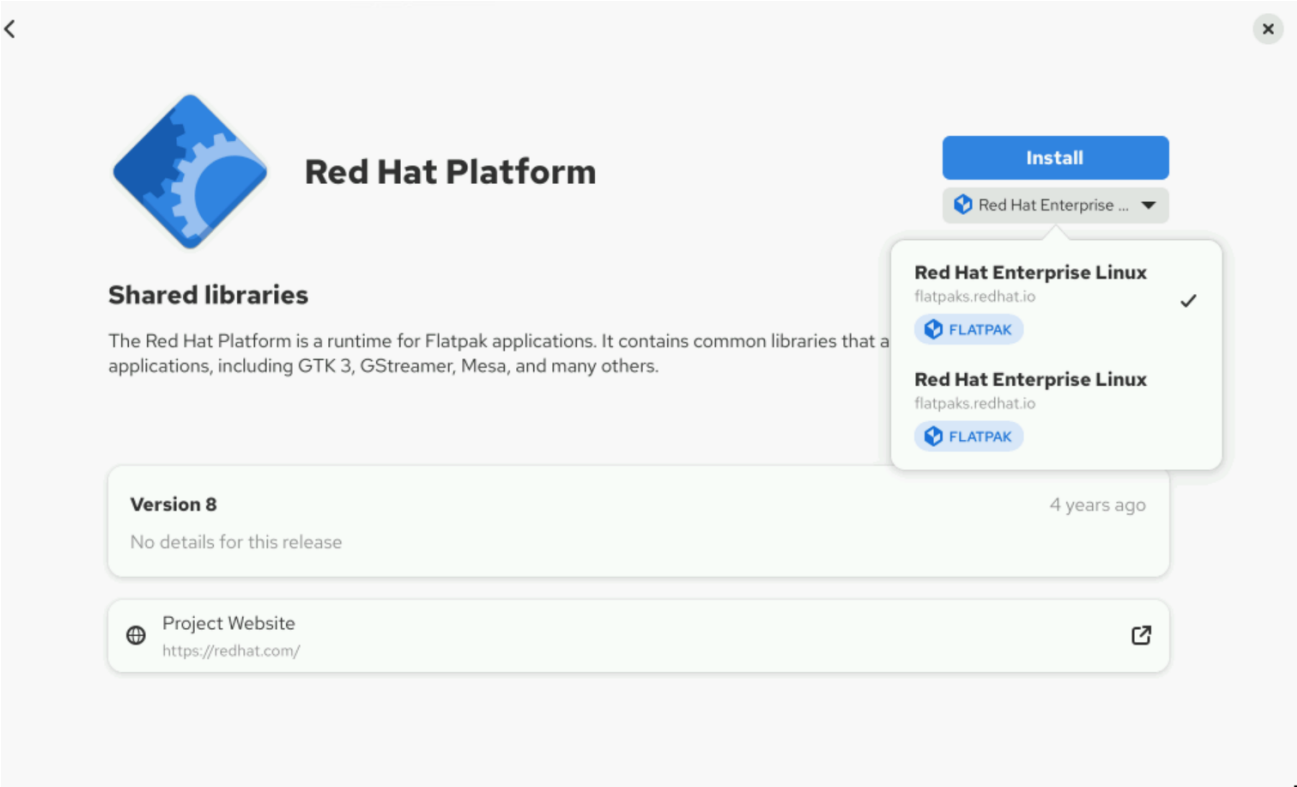


Figure 25. Flatpak object details

To install the object, select the intended branch (if more than one branch is available for the object), and click **Install**. The GNOME Software tool resolves all the dependencies automatically and does not seek further confirmation.

To uninstall a Flatpak object, click **Uninstall**.

 References

flatpak-remote-ls(1), flatpak-search(1), flatpak-install(1), flatpak-list(1), flatpak-info(1), flatpak-remote-info(1), flatpak-update(1), flatpak-mask(1), and flatpak-uninstall(1) man pages

Using Flatpak

13.4. Guided Exercise

Manage Applications from Flatpak

List, search, install, remove, and get metadata information of applications by using Flatpak.

Outcomes

- Use Flatpak commands to manage applications and runtimes.
- Install applications and review their information.
- Uninstall Flatpak applications and their dependencies.

Prerequisites

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
student@workstation:~$ lab start flatpak-application
```

Instructions

1. As the student user on the workstation machine, verify that the `rhel` and `myrepo` Flatpak remote repositories are configured.

```
student@workstation:~$ flatpak remotes
Name      Options
rhel      system,oci,no-gpg-verify
myrepo    user,no-gpg-verify
```

2. Verify that no Flatpak applications are present in the system.

```
student@workstation:~$ flatpak list --app
```

3. Identify and install the `codium` and `Obsidian` Flatpaks in user installation mode from the `myrepo` remote repository.

- 3.1. List the available Flatpak objects from the `myrepo` remote repository. Note the application identifiers and branches.

```
student@workstation:~$ flatpak remote-ls myrepo
Name                               Application ID           Version  Branch
codium                             com.vscodium.codium      stable
Obsidian                           md.obsidian.Obsidian     stable
freedesktop platform              org.freedesktop.Platform 24.08
freedesktop development platform  org.freedesktop.Sdk      24.08
```

3.2. Install the latest available versions of the codium and Obsidian packages by using the identifier triples that the stable branch uses.

```

student@workstation:~$ flatpak install com.vscodium.codium//stable \
md.obsidian.Obsidian//stable
Looking for matches...
Required runtime for md.obsidian.Obsidian/x86_64/stable
(runtime/org.freedesktop.Platform/x86_64/24.08) found in remote myrepo
Do you want to install it? [Y/n]: y
Required runtime for com.vscodium.codium/x86_64/stable
(runtime/org.freedesktop.Sdk/x86_64/24.08) found in remote myrepo
Do you want to install it? [Y/n]: y

md.obsidian.Obsidian permissions:
  ipc      network      pulseaudio      ssh-auth      x11      dri      file access [1]
tags [2]

  [1] /media, /mnt, /run/media, home, xdg-run/app/com.discordapp.Discord:create,
  xdg-run/gnupg:ro, ~/.local/share/fonts:ro
  [2] proprietary

com.vscodium.codium permissions:
  ipc      network      fallback-x11      pulseaudio
ssh-auth      wayland      x11      devices
  devel      file access [1]      dbus access [2]      system dbus access [3]

  [1] host, xdg-config/kdeglobals:ro, xdg-run/gnupg:ro
  [2] com.canonical.AppMenu.Registrar, org.freedesktop.Flatpak,
  org.freedesktop.secrets, org.kde.kwalletd5
  [3] org.freedesktop.login1

Proceed with these changes to the user installation? [Y/n]: y

  ID                                     Branch   Op   Remote   Download
1. [✓] org.freedesktop.Platform         24.08   i   myrepo   253.5 MB / 264.5 MB
2. [✓] md.obsidian.Obsidian             stable  i   myrepo   200.4 MB / 197.9 MB
3. [✓] org.freedesktop.Sdk              24.08   i   myrepo   358.6 MB / 622.0 MB
4. [✓] com.vscodium.codium              stable  i   myrepo   129.9 MB / 139.4 MB

Installing 1/4... ██████████ 100% 19.8 MB/s 00:00
Installing 2/4... ██████████ 100% 15.4 MB/s 00:00
Installing 3/4... ██████████ 100% 9.7 MB/s 00:00
Installing 4/4... ██████████ 100% 21.7 MB/s 00:00

Installation complete.

```

4. Verify that the applications were successfully installed. Note that the runtime dependency is installed as well.

```
student@workstation:~$ flatpak list
```

Name	Application ID	Version	Branch
Inst...			
VSCodium	com.vscodium.codium	1.100.23258	stable user
Obsidian	md.obsidian.Obsidian	1.8.10	stable user
Freedesktop Pl...	freedesktop.Platform	freedesktop-sdk-24.08.19	24.08 user
Freedesktop SDK	org.freedesktop.Sdk	freedesktop-sdk-24.08.19	24.08 user

5. Retrieve additional information about the installed applications.

5.1. Retrieve information for the VSCodium application.

```
student@workstation:~$ flatpak info com.vscodium.codium
```

VSCodium - Telemetry-less code editing

```

      ID: com.vscodium.codium
      Ref: app/com.vscodium.codium/x86_64/stable
      Arch: x86_64
      Branch: stable
      Version: 1.100.23258
      License: MIT
      Origin: myrepo
      Collection: org.flathub.Stable
      Installation: user
      Installed: 411.0 MB
      Runtime: org.freedesktop.Sdk/x86_64/24.08
      Sdk: org.freedesktop.Sdk/x86_64/24.08

      Commit: 8855...74b6
      Parent: 73d7...33e2
      Subject: feat: :tada: Update VSCodium v1.100.23258 (0b11b6c4d859)
      Date: 2025-05-16 23:57:37 +0000

```

5.2. Retrieve information for the Obsidian application.


```
student@workstation:~$ flatpak info md.obsidian.Obsidian

Obsidian - Markdown-based knowledge base

    ID: md.obsidian.Obsidian
    Ref: app/md.obsidian.Obsidian/x86_64/stable
    Arch: x86_64
    Branch: stable
    Version: 1.8.10
    License: LicenseRef-proprietary=https://obsidian.md/eula
    Origin: myrepo
    Collection: org.flathub.Stable
    Installation: user
    Installed: 634.3 MB
    Runtime: org.freedesktop.Platform/x86_64/24.08
    Sdk: org.freedesktop.Sdk/x86_64/24.08

    Commit: 9f43...e633
    Parent: 79b5...9a9c
    Subject: Update gh module (91b101008d04)
    Date: 2025-05-30 04:52:34 +0000
```

6. From the GNOME graphical desktop, launch the VSCode and Obsidian applications to verify that they are installed correctly.
 - 6.1. Click the **Red Hat** logo in the upper-left corner of the screen to open the **Activities Overview**.
 - 6.2. In the search box, enter `vscodium` and click the **VSCodium** icon to open the application.
 - 6.3. Verify that the application starts correctly. Explore its interface, and close the application when you are finished.
 - 6.4. In the search box, enter `obsidian` and click the **Obsidian** icon to open the application.
 - 6.5. Click **Quick Start** to explore the application, and close the application when you are finished.
7. Uninstall the VSCode Flatpak package with all its dependencies.
 - 7.1. Try to remove the `org.freedesktop.Sdk` runtime before uninstalling the application that depends on it. You receive an error message.

```

student@workstation:~$ flatpak uninstall org.freedesktop.Sdk
Info: applications using the runtime org.freedesktop.Sdk branch 24.08:
    com.vscodium.codium
Really remove? [y/n]: y

      ID                      Branch      Op
1.    org.freedesktop.Sdk     24.08      r

Proceed with these changes to the user installation? [Y/n]: y

      ID                      Branch      Op
1. [X] org.freedesktop.Sdk     24.08      r

Error: Can't remove org.freedesktop.Sdk/x86_64/24.08, it is needed for:
com.vscodium.codium
error: Failed to uninstall org.freedesktop.Sdk: Can't remove
org.freedesktop.Sdk/x86_64/24.08, it is needed for: com.vscodium.codium

```

7.2. Run the `flatpak uninstall` command again, and specify the `codium` package and the `Sdk` runtime. You can use the short-form identifiers because no other versions are installed for those objects.

Flatpak sets the uninstall priority automatically, regardless of the order of the identifiers in the command.

```
student@workstation:~$ flatpak uninstall Sdk codium
Found installed ref 'runtime/org.freedesktop.Sdk/x86_64/24.08' (user). Is this
correct? [Y/n]: y
Found installed ref 'app/com.vscodium.codium/x86_64/stable' (user). Is this
correct? [Y/n]: y

      ID                               Branch      Op
1.    com.vscodium.codium             stable      r
2.    org.freedesktop.Sdk             24.08      r

Proceed with these changes to the user installation? [Y/n]: y

      ID                               Branch      Op
1. [-] com.vscodium.codium             stable      r
2. [-] org.freedesktop.Sdk             24.08      r

Uninstalling 1/2...
Uninstalling 2/2...
```

7.3. Verify that the Flatpak objects are no longer installed.

```
student@workstation:~$ flatpak list
Name                               Application ID      Version           Branch
Inst...
Obsidian                           md.obsidian.Obsidian 1.8.9            stable user
Freedesktop Platform               org.freedesktop.Platform freedesktop-sdk-24.08 24.08 user
```

8. From the GNOME desktop, uninstall the Obsidian Flatpak with all its dependencies by using the GNOME Software graphical tool.
- 8.1. From the GNOME graphical desktop environment, click the **Red Hat** logo in the upper-left corner and then select **Software** from the dashboard, or use the **Show Applications** button to open the application.
- 8.2. Click the **Installed** tab. In the **Apps** section, find **Obsidian**, and click **Uninstall**. Click **Uninstall** to confirm.
- 8.3. In the **Add-ons** section, find **Freedesktop Platform**. Click **Uninstall**, and click **Uninstall** again when asked for confirmation.
- 8.4. Close the GNOME Software tool.
9. Verify that all Flatpak objects are removed from the workstation machine.
- 9.1. Verify that no Flatpak objects are present on disk.

```
student@workstation:~$ flatpak list
```

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish flatpak-application
```

13.5. Quiz

Install and Update Applications by Using Flatpak

Choose the correct answers to the following questions:

1. Which command lists all the Flatpak remote repositories that are configured on your system?

- ☐ a. flatpak remotes
- ☐ b. flatpak show-remotes
- ☐ c. flatpak get-remotes
- ☐ d. flatpak list-remotes

2. What is the role of a sandbox in Flatpak?

- ☐ a. To provide network access.
- ☐ b. To manage user permissions.
- ☐ c. To enhance execution performance.
- ☐ d. To isolate applications from the system.

3. Which command adds a Flatpak remote repository?

- ☐ a. flatpak remote-enable
- ☐ b. flatpak remote-add
- ☐ c. flatpak remote-install
- ☐ d. flatpak remote-set

4. You are setting up a new Linux system and want to install applications by using Flatpak. The Fedora Project hosts a remote registry that can be used across different distributions. You decide to add this repository, and want to make it available for all users on the system. Which command adds the fedora repository system-wide?

- ☐ a. `flatpak remote-add fedora https://registry.fedoraproject.org`
- ☐ b. `flatpak remote-add --if-not-exists fedora oci+https://registry.fedoraproject.org`
- ☐ c. `flatpak remote-add --user fedora oci+https://registry.fedoraproject.org`
- ☐ d. `flatpak remote-add --system fedora oci+https://registry.fedoraproject.org`

5. When adding a Flatpak remote repository, which option prevents overwriting an existing repository?

- ☐ a. `--skip-existing`
- ☐ b. `--overwrite`
- ☐ c. `--if-not-exists`
- ☐ d. `--no-replace`

6. You are troubleshooting a Linux system where a user disabled the Fedora Flatpak repository. You need to re-enable the repository to update some packages. Which command re-enables the fedora repository?

- ☐ a. `flatpak remote-modify --activate fedora`
- ☐ b. `flatpak remote-modify --add fedora`
- ☐ c. `flatpak remote-modify --enable fedora`
- ☐ d. `flatpak remote-modify --start fedora`

7. What is the naming convention for Flatpak application identifiers?

- ☐ a. `App.organization.domain`
- ☐ b. `domain.organization.App`
- ☐ c. `organization.domain.App`
- ☐ d. `App.domain.organization`

8. What does branch in a Flatpak identifier triple refer to?

- ☐ a. The application's developer
- ☐ b. Different versions of the same object
- ☐ c. The application's architecture
- ☐ d. The application's license type

9. What is the purpose of the `flatpak mask` command?

- ☐ a. To install new Flatpak objects.
- ☐ b. To update Flatpak objects.
- ☐ c. To uninstall Flatpak objects.
- ☐ d. To prevent unwanted upgrades of certain objects.

10. When using the `flatpak uninstall` command, which option ensures that the application's data is also deleted?

- ☐ a. `--purge-data`
- ☐ b. `--delete-data`
- ☐ c. `--remove-data`
- ☐ d. `--clean-data`

Solution

1. Which command lists all the Flatpak remote repositories that are configured on your system?

- ☒ **a. flatpak remotes**
- ☐ b. flatpak show-remotes
- ☐ c. flatpak get-remotes
- ☐ d. flatpak list-remotes

2. What is the role of a sandbox in Flatpak?

- ☐ a. To provide network access.
- ☐ b. To manage user permissions.
- ☐ c. To enhance execution performance.
- ☒ **d. To isolate applications from the system.**

3. Which command adds a Flatpak remote repository?

- ☐ a. flatpak remote-enable
- ☒ **b. flatpak remote-add**
- ☐ c. flatpak remote-install
- ☐ d. flatpak remote-set

4. You are setting up a new Linux system and want to install applications by using Flatpak. The Fedora Project hosts a remote registry that can be used across different distributions. You decide to add this repository, and want to make it available for all users on the system. Which command adds the fedora repository system-wide?

- ☐ a. flatpak remote-add fedora https://registry.fedoraproject.org
- ☐ b. flatpak remote-add --if-not-exists fedora oci+https://registry.fedoraproject.org
- ☐ c. flatpak remote-add --user fedora oci+https://registry.fedoraproject.org
- ☒ **d. flatpak remote-add --system fedora oci+https://registry.fedoraproject.org**

5. When adding a Flatpak remote repository, which option prevents overwriting an existing repository?
- ☐ a. `--skip-existing`
 - ☐ b. `--overwrite`
 - ☒ c. `--if-not-exists`
 - ☐ d. `--no-replace`
6. You are troubleshooting a Linux system where a user disabled the Fedora Flatpak repository. You need to re-enable the repository to update some packages. Which command re-enables the fedora repository?
- ☐ a. `flatpak remote-modify --activate fedora`
 - ☐ b. `flatpak remote-modify --add fedora`
 - ☒ c. `flatpak remote-modify --enable fedora`
 - ☐ d. `flatpak remote-modify --start fedora`
7. What is the naming convention for Flatpak application identifiers?
- ☐ a. `App.organization.domain`
 - ☒ b. `domain.organization.App`
 - ☐ c. `organization.domain.App`
 - ☐ d. `App.domain.organization`
8. What does branch in a Flatpak identifier triple refer to?
- ☐ a. The application's developer
 - ☒ b. **Different versions of the same object**
 - ☐ c. The application's architecture
 - ☐ d. The application's license type
9. What is the purpose of the `flatpak mask` command?
- ☐ a. To install new Flatpak objects.
 - ☐ b. To update Flatpak objects.
 - ☐ c. To uninstall Flatpak objects.
 - ☒ d. **To prevent unwanted upgrades of certain objects.**

10. When using the `flatpak uninstall` command, which option ensures that the application's data is also deleted?

- ☐ a. `--purge-data`
- ☒ b. `--delete-data`
- ☐ c. `--remove-data`
- ☐ d. `--clean-data`

13.6. Summary

In this lesson, you learned to configure Flatpak and used it to manage software.

Flatpak uses remote repositories that store the applications and the required runtimes to execute those applications in an isolated execution environment. The remote repositories enable the automatic resolution of dependencies.

You can use the command-line interface to install, update, list, remove, and get metadata information about Flatpak applications and runtime dependencies.

By using the GNOME Software graphical tool, you can install and uninstall Flatpak applications as you would with RPM packages.

Chapter 14.

Accessing Removable Media

Goal

Access file systems on removable media devices by mounting them on a directory in the file-system hierarchy.

Sections

- Identifying File Systems and Block Devices (and Quiz)
- Mounting and Unmounting File Systems (and Guided Exercise)
- Locating Files on a File System (and Guided Exercise)

Lab

- Access Removable Media

14.1. Identifying File Systems and Block Devices

Objectives

- Match a block storage device with the file system that a particular directory uses, and determine the amount of free space on that file system.

File Systems and Storage Devices

Red Hat Enterprise Linux (RHEL) uses the *Extents File System (XFS)* as the default local file system. RHEL supports the *Extended File System (ext4)* file system for managing local files. Starting with RHEL 10, the *Extensible File Allocation Table (exFAT)* file system is supported for removable media use.

Mounting File Systems

Access the contents of a file system by mounting it on an empty directory. This directory is called a *mount point*. When the directory is mounted, use the `ls` command to list its contents. Many file systems are automatically mounted when the system boots.

A mount point differs slightly from a Microsoft Windows drive letter, where each file system is a separate entity. Mount points enable multiple file-system devices to be available in a single tree structure. This mount point is similar to NTFS mounted folders in Microsoft Windows.

Using Block Devices

A block device is a file that provides low-level access to storage devices. A block device representing a hard disk drive or solid-state device (SSD) is usually partitioned into several smaller block devices, and a partition must be formatted with a file system before its file system can be mounted.

The `/dev` directory stores block device files, which RHEL creates automatically for all devices. In RHEL systems, the first detected Serial ATA (SATA), Serial Attached SCSI (SAS), SCSI, or USB hard drive is called the `/dev/sda` device; the second is the `/dev/sdb` device; and so on. These names represent the entire hard drive.

Table 20. Block Device Naming

Type of device	Device naming pattern
SATA/SAS/USB-attached storage (SCSI driver)	<code>/dev/sda</code> , <code>/dev/sdb</code> , <code>/dev/sdc</code> , ...

Type of device	Device naming pattern
virtio-blk paravirtualized storage (VMs)	/dev/vda, /dev/vdb, /dev/vdc,...
virtio-scsi paravirtualized storage (VMs)	/dev/sda, /dev/sdb, /dev/sdc, ...
NVMe-attached storage (SSDs)	/dev/nvme0, /dev/nvme1, ...
SD/MMC/eMMC storage (SD cards)	/dev/mmcblk0, /dev/mmcblk1, ...

Disk Partitions

Usually, a storage device is divided into multiple partitions.

With partitions, you can compartmentalize a disk: the various partitions might be formatted with different file systems or be used for other purposes. For example, one partition might contain user home directories, whereas another partition might contain system data and logs. Even when the home directory partition is loaded with data, the system partition might still have available space.

Partitions are block devices in their own right. For example, on the first SATA-attached storage, the first partition is the /dev/sda1 disk. The second partition of the same storage is the /dev/sda2 disk. The third partition on the third SATA-attached storage device is the /dev/sdc3 disk, and so on. Paravirtualized storage devices have a similar naming system. For example, the first partition on the first storage device is the /dev/vda1 disk. The second partition of the second storage device is the /dev/vdb2 disk, and so on.

An NVMe-attached SSD device names its partitions differently from a SATA-attached device. For NVMe storage devices, the nvmeX part of the name refers to the device; the nY part refers to the namespace; and the pZ part refers to the partition. For example, the first partition for the first namespace on the first disk is the /dev/nvme0n1p1 partition. The third partition for the first namespace on the second disk is the /dev/nvme1n1p3 partition, and so on.

MultiMediaCard (MMC) or Secure Digital (SD) cards can sometimes have a similar naming system to the SATA devices (/dev/sdN). In some cases, SD or MMC cards might have names such as /dev/mmcblk0p1, where the mmcblkX part of the name refers to the storage device, and the pY part of the name refers to the partition number on that device.

An extended listing of the `/dev/sda1` device file on the host machine reveals the `b` file type, which stands for a block device:

```
user@host:~$ ls -l /dev/sda1
brw-rw----. 1 root disk 8, 1 Jun 12 05:06 /dev/sda1
```

Logical Volumes

Another way to organize disks and partitions is with *Logical Volume Management (LVM)*. LVM aggregates block devices into a volume group. Disk space in the volume group is separated into logical volumes, which are the functional equivalent of a partition on a physical disk.

The LVM system assigns names to volume groups and logical volumes on their creation. LVM creates a directory in the `/dev` directory that matches the group name, and creates a symbolic link within that new directory with the same name as the logical volume. That logical volume file is then available to be mounted. For example, when a `myvg` volume group and the `mylv` logical volume are present, the full path to the logical volume is the `/dev/myvg/mylv` file.

Note

The previously mentioned logical volume device name establishes a symbolic link to the device file that accesses it, which might vary between boots. Another form of the logical volume device name, which is linked from files in the `/dev/mapper` directory, is often used for symbolic links to the device file.

Examine File Systems

Use the `df` command to display an overview of local and remote file-system devices, which includes the total disk space, used disk space, free disk space, and the percentage of the entire disk space.

The following example displays the file systems and mount points on the host machine:

```

user@host:~$ df
Filesystem      1K-blocks      Used Available Use% Mounted on
/dev/sda3        10213356 2458040    7755316 25% /
devtmpfs          4096         0         4096  0% /dev
tmpfs            873064         0    873064  0% /dev/shm
tmpfs            349228    10784    338444  4% /run
tmpfs            1024         0         1024  0% /run/credentials/systemd-
journald...
/dev/sda2        204580      8580    196000  5% /boot/efi
tmpfs            1024         0         1024  0% /run/credentials/getty@tty1...
tmpfs            1024         0         1024  0% /run/credentials/serial-getty@...
tmpfs           174612         4    174608  1% /run/user/1000

```

The partitioning shows that two physical file systems are mounted on the root (/) and /boot/efi directories that commonly exist on virtual machines. The tmpfs and devtmpfs devices are file systems in system memory. All files that are written to the tmpfs or devtmpfs file systems disappear after a system reboot.

The df command -h or -H options are human-readable options to improve the readability of the output sizes. The -h option reports in KiB (2¹⁰), MiB (2²⁰), or GiB (2³⁰), whereas the -H option reports in SI units: KB (10³), MB (10⁶), or GB (10⁹). Hard drive manufacturers usually use SI units when advertising their products.

View the file systems on the host machine with all units converted to human-readable format:

```

user@host:~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda3       20G   6.6G   14G   33% /
devtmpfs        4.0M    0   4.0M    0% /dev
tmpfs           2.8G   84K   2.8G    1% /dev/shm
tmpfs           1.2G   18M   1.1G    2% /run
tmpfs           1.0M    0   1.0M    0% /run/credentials/systemd-journald.service
/dev/sda2       200M   8.4M   192M    5% /boot/efi
/dev/sr0        536K  536K    0  100% /run/media/meta-data
tmpfs           567M  128K   567M    1% /run/user/1000

```

Use the du command for more detailed information about a specific directory tree space. The du command -h and -H options convert the output to a human-readable format. The du command shows the size of all files in the current directory tree recursively.

View the disk usage report for the /usr/share directory on the host machine:


```

root@host:~# du /usr/share
...output omitted...
4      /usr/share/mdadm
8      /usr/share/cmake/bash-completion
8      /usr/share/cmake
4      /usr/share/at
374328 /usr/share

```

View the disk usage report in human-readable format for the `/usr/share` directory:

```

root@host:~# du -h /usr/share
...output omitted...
4.0K    /usr/share/mdadm
8.0K    /usr/share/cmake/bash-completion
8.0K    /usr/share/cmake
4.0K    /usr/share/at
366M    /usr/share

```

References

`df(1)` and `du(1)` man pages

For more information, refer to the *Managing File Systems* guide at https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html-single/managing_file_systems/index

14.2. Quiz

Identifying File Systems and Block Devices

Choose the correct answers to the following questions:

1. What is the device file name of an entire SATA hard drive in the `/dev` directory?

- ☐ a. `/dev/vda`
- ☐ b. `/dev/sda1`
- ☐ c. `/dev/vg_install/lv_home`
- ☐ d. `/dev/sda`

2. Which command displays the file systems with the mount points?

- ☐ a. `du -H`
- ☐ b. `df`
- ☐ c. `du`
- ☐ d. `lsfs`

3. Which command displays the disk usage report in human-readable format for the `/home` directory?

- ☐ a. `ls /home`
- ☐ b. `df`
- ☐ c. `du -h /home`
- ☐ d. `du /home`

4. What is the correct device file name for the third partition on the second `virtio-blk` disk that is attached to a virtual machine?

- ☐ a. `/dev/vdb3`
- ☐ b. `/dev/sdb3`
- ☐ c. `/dev/sda3`
- ☐ d. `/dev/vda3`

5. Which command provides an overview of the file-system mount points and the available free space in SI units?

- ☐ a. df
- ☐ b. df -h
- ☐ c. df -H
- ☐ d. du -h

Solution

1. What is the device file name of an entire SATA hard drive in the /dev directory?

- ☐ a. /dev/vda
- ☐ b. /dev/sda1
- ☐ c. /dev/vg_install/lv_home
- ☒ d. /dev/sda

2. Which command displays the file systems with the mount points?

- ☐ a. du -H
- ☒ b. df
- ☐ c. du
- ☐ d. lsfs

3. Which command displays the disk usage report in human-readable format for the /home directory?

- ☐ a. ls /home
- ☐ b. df
- ☒ c. du -h /home
- ☐ d. du /home

4. What is the correct device file name for the third partition on the second virtio-blk disk that is attached to a virtual machine?

- ☒ a. /dev/vdb3
- ☐ b. /dev/sdb3
- ☐ c. /dev/sda3
- ☐ d. /dev/vda3

5. Which command provides an overview of the file-system mount points and the available free space in SI units?

- ☐ a. df
- ☐ b. df -h
- ☒ c. df -H
- ☐ d. du -h

14.3. Mounting and Unmounting File Systems

Objectives

- Access the contents of a file system on removable media by mounting it into the file-system hierarchy, and unmounting it when you are finished.

Mounting File Systems Manually

To access the file system on a removable storage device, you must mount the storage device. With the `mount` command, the `root` user can mount a file system manually. The first argument of the `mount` command specifies the file system to mount. The second argument specifies the directory as the mount point in the file-system hierarchy.

You can mount the file system in one of the following ways with the `mount` command:

- With the device file name in the `/dev` directory.
- With the UUID, a universally unique identifier of the device.

Then, identify the device to mount, ensure that the mount point exists, and mount the device on the mount point.

Note

If you mount a file system with the `mount` command, and then reboot your system, the file system is not automatically remounted. The *Red Hat System Administration II* (RH134) course explains how to persistently mount file systems with the `/etc/fstab` file.

Identifying a Block Device

A hot-pluggable storage device, whether a hard disk drive (HDD) or a solid-state device (SSD) in a server, or alternatively a USB storage device, might be plugged into a different port on a system each time. Use the `lsblk` command to list the details of a specified block device or of all the available devices.

```

root@host:~# lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda          8:0    0   10G  0 disk
├─sda1       8:1    0    1M  0 part
├─sda2       8:2    0  200M  0 part /boot/efi
└─sda3       8:3    0   9.8G  0 part /
sdb          8:16   0    5G  0 disk
sdc          8:32   0    5G  0 disk
sdd          8:48   0    5G  0 disk
sr0         11:0    1  514K  0 rom

```

The partition size helps to identify the device when the partition name is unknown. For example, considering the previous output, if the size of the identified partition is 9.8 GB, then mount the `/dev/sda3` partition.

Mounting a File System with the Partition Name

The following example mounts the `/dev/sda3` partition on the `/mnt/data` mount point.

```

root@host:~# mount /dev/sda3 /mnt/data

```

The mount point directory must exist before mounting the file system. The `/mnt` directory exists for use as a temporary mount point.

Important

If a directory to use as a mount point is not empty, then the existing files are hidden and not accessible when a file system is mounted there. The original files are accessible again after the mounted file system is unmounted.

Device detection order and storage device naming can change when devices are added or removed on a system. A good practice is to use an unchanging device identifier to mount file systems consistently.

Mounting a File System with Partition UUID

One stable identifier that is associated with a file system is its universally unique identifier (UUID). This UUID is stored in the file-system superblock and remains the same until the file system is re-created.

The `lsblk -fp` command lists the full path of the device, the UUIDs and mount points, and the partition's file-system type. The mount point is blank when the file system is not mounted.

```

root@host:~# lsblk -fp
NAME          FSTYPE FSVER LABEL UUID                               FSAVAIL FSUSE%
MOUNTPOINTS
/dev/sda
├─/dev/sda1
├─/dev/sda2 vfat    FAT16      7B77-95E7                191.4M   4%      /boot/efi
└─/dev/sda3 xfs      root    15507695-...-...95983f  7.4G    24%     /
/dev/sdb
/dev/sdc
/dev/sdd
/dev/sr0      iso9660 Joliet Extension config-2 2025-06-12-05-06-37-00

```

Mount the file system by the file-system UUID.

```

root@host:~# mount UUID="15507695-22bb-4c65-94e6-a438e095983f" /mnt/data

```

Automatically Mounting Removable Storage Devices

With the graphical desktop environment, the system automatically mounts removable storage media when the media presence is detected.

The removable storage device mounts at the `/run/media/USERNAME/LABEL` location. *USERNAME* is the name of the user that is logged in to the graphical environment. *LABEL* is an identifier, which is typically the label on the storage media.

To safely detach a removable device, manually unmount all file systems on the device first.

Unmounting File Systems

System shutdown and reboot procedures are used to unmount all file systems automatically. All file-system data is flushed to the storage device, to ensure file-system data integrity.

⚠ Warning

File-system data uses memory cache during regular operation. You must unmount a removable drive's file systems before unplugging the drive. The unmount procedure flushes data to disk before releasing the drive.

The `umount` command uses the mount point as an argument to unmount a file system.


```
root@host:~# umount /mnt/data
```

Unmounting is not possible when the mounted file system is in use. For the `umount` command to succeed, all processes must stop accessing data under the mount point.

In the following example, the `umount` command fails because the shell uses the `/mnt/data` directory as its current working directory, and thus generates an error message.

```
root@host:~# cd /mnt/data
root@host:~# umount /mnt/data
umount: /mnt/data: target is busy.
```

The `lsof` command lists all open files and the processes that are accessing the file system. The list helps to identify which processes are preventing the file system from successfully unmounting.

```
root@host:~# lsof /mnt/data
COMMAND  PID  USER  FD   TYPE DEVICE SIZE/OFF  NODE NAME
bash      1593 root   cwd   DIR  253,17      6  128 /mnt/data
lsof      2532 root   cwd   DIR  253,17     19  128 /mnt/data
lsof      2533 root   cwd   DIR  253,17     19  128 /mnt/data
```

Identify and wait for the processes to complete, or send the `SIGTERM` or `SIGKILL` signal to terminate them. In this case, you can change to a current working directory that is outside the mount point.

```
root@host:~# cd
root@host:~# umount /mnt/data
```

References

`lsblk(8)`, `mount(8)`, `umount(8)`, and `lsof(8)` man pages

14.4. Guided Exercise

Mount and Unmount a File System

Mount and unmount a file system that is on removable media.

Outcomes

- Identify and mount a new file system at a specified mount point, and then unmount the file system.

Prerequisites

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
student@workstation:~$ lab start fs-mount
```

Instructions

1. Log in to the servera machine as the student user and switch to the root user. Use student as the password.

```
student@workstation:~$ ssh student@servera
...output omitted...
[student@servera ~]$ sudo -i
[sudo] password for student: student
[root@servera ~]#
```

2. A new partition with a file system is added to the /dev/sdb disk on the servera machine. Mount the newly available partition by using the UUID at the /mnt/part1 mount point.

- 2.1. Create the /mnt/part1 directory.

```
[root@servera ~]# mkdir /mnt/part1
```

- 2.2. Query the UUID of the /dev/sdb1 device.

```
[root@servera ~]# lsblk -fp /dev/sdb
NAME          FSTYPE ... UUID                                FSAVAIL FSUSE% MOUNTPOINTS
/dev/sdb
└─/dev/sdb1 xfs      2dfe4a17-d529-480a-a53e-9c28ff49545e
```

The UUID of the device might be different in your environment.

The previous output omits some nonessential columns, so your output might differ.

- 2.3.** Mount the file system by using the UUID on the /mnt/part1 directory. Use the /dev/sdb1 UUID from the previous command output.

```
[root@servera ~]# mount \
UUID="2dfe4a17-d529-480a-a53e-9c28ff49545e" /mnt/part1
```

- 2.4.** Verify that the /dev/sdb1 device is mounted on the /mnt/part1 directory.

```
[root@servera ~]# lsblk -fp /dev/sdb
NAME          FSTYPE ... UUID                                FSAVAIL FSUSE% MOUNTPOINTS
/dev/sdb
└─/dev/sdb1 xfs      2dfe4a17-d529-480a-a53e-9c28ff49545e 4.8G      3%      /mnt/part1
```

The previous output omits some nonessential columns, so your output might differ.

- 3.** Change to the /mnt/part1 directory and create the testdir subdirectory. Create the /mnt/part1/testdir/newmount file to verify that you can write to the mount point.

- 3.1.** Change to the /mnt/part1 directory.

```
[root@servera ~]# cd /mnt/part1
```

- 3.2.** Create the /mnt/part1/testdir directory.

```
[root@servera part1]# mkdir testdir
```

- 3.3.** Create the /mnt/part1/testdir/newmount file.

```
[root@servera part1]# touch testdir/newmount
```

- 4.** Unmount the file system that is mounted on the /mnt/part1 directory.

- 4.1.** Try to unmount the `/mnt/part1` directory with the shell still in the `/mnt/part1` directory. The `umount` command fails to unmount the device.

```
[root@servera part1]# umount /mnt/part1
umount: /mnt/part1: target is busy.
```

- 4.2.** Change the shell's current directory to the `/root` directory.

```
[root@servera part1]# cd
[root@servera ~]#
```

- 4.3.** Unmount the `/mnt/part1` directory.

```
[root@servera ~]# umount /mnt/part1
```

- 5.** Return to the workstation machine as the student user.

```
[root@servera ~]# exit
logout
[student@servera ~]$ exit
logout
Connection to servera closed.
student@workstation:~$
```

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish fs-mount
```

14.5. Locating Files on a File System

Objectives

- Search for files on a file system that is currently mounted.

Searching for Files

A system administrator must be able to search file systems for files that match specific criteria. This section discusses two commands to search for files in the file-system hierarchy:

- The `locate` command searches a pre-generated index for file names or file paths, and returns the results instantly.
- The `find` command searches for files in real time by parsing the file-system hierarchy.

Locating Files by Name

The `locate` command searches for files based on the name or path to the file. The `locate` command is fast, because it looks up the information from the `mlocate` database. However, this database does not update in real time and requires frequent updates for accurate results. This feature also means that the `locate` command does not search for files that were created after the last database update.

The `locate` database updates automatically every day. However, the `root` user might use the `updatedb` command to force an immediate update.

```
root@host:~# updatedb
```

The `locate` command restricts results for unprivileged users. To see the resulting file name, the user must have search permission on the directory where the file resides. For example, locate the files that the `developer` user can read, and that match the `passwd` keyword in the name or path:

```
developer@host:~$ locate passwd
/etc/passwd
/etc/passwd-
/etc/pam.d/passwd
...output omitted...
```

The following example shows the file name or path for a partial match with the search query:

```
root@host:~# locate image
/etc/selinux/targeted/contexts/virtual_image_context
/usr/bin/grub2-mkimage
/usr/lib/sysimage
...output omitted...
```

The `locate` command `-i` option performs a case-insensitive search. This option returns all possible combinations of matching uppercase and lowercase letters:

```
developer@host:~$ locate -i messages
...output omitted...
/usr/share/locale/zza/LC_MESSAGES
/usr/share/makedumpfile/eppic_scripts/ap_messages_3_10_to_4_8.c
/usr/share/vim/vim82/ftplugin/msmessages.vim
...output omitted...
```

The `locate` command `-n` option limits the number of returned search results. The following example limits the search results from the `locate` command to the first five matches:

```
developer@host:~$ locate -n 5 passwd
/etc/passwd
/etc/passwd-
/etc/pam.d/passwd
...output omitted...
```

Searching for Files in Real Time

The `find` command locates files by searching in real time in the file-system hierarchy. This command is slower but more accurate than the `locate` command. The `find` command also searches for files based on criteria other than the file name, such as the file's permissions, type of file, size, or modification time.

The `find` command looks at files in the file system with the user account that executed the search. The user that runs the `find` command must have read and execute permission on a directory to examine its contents.

The first argument to the `find` command is the directory to search. If the `find` command omits the directory argument, then it starts the search in the current directory and looks for matches in any subdirectory.

To search for files by file name, use the `find` command `-name FILENAME` option to return the path of files that match `FILENAME` exactly. For example, to search for the `sshd_config` files in the root (`/`) directory, run the following command:

```
root@host:~# find / -name sshd_config
/etc/ssh/sshd_config
```

Note

For the `find` command, the complete word options use a single dash (`-`), unlike a double dash (`--`) for most other Linux commands.

Searching for Files with Wildcards

Wildcards are available to search for a file name and to return all results for a partial match. When using a wildcard to search, you must use single quotation marks around the string to search for, to prevent the terminal from misinterpreting the wildcard.

The following example starts in the root (`/`) directory and searches for file names that end with the `.txt` extension:

```
root@host:~# find / -name '*.txt'
...output omitted...
/usr/share/libgpg-error/errorref.txt
/usr/share/licenses/audit-libs/lgpl-2.1.txt
/usr/share/licenses/pam/gpl-2.0.txt
...output omitted...
```

The next command searches for file names in the `/etc/` directory that contain the `pass` string:

```
root@host:~# find /etc -name '*pass*'
/etc/passwd-
/etc/passwd
/etc/security/opasswd
...output omitted...
```

To perform a case-insensitive search for a file name, use the `find` command `-iname` option, followed by the file name to search. To search files with case-insensitive text that matches the `messages` string in their names in the root (`/`) directory, run the following command:

```
root@host:~# find / -iname '*messages*'
/sys/power/pm_debug_messages
/usr/lib/locale/C.utf8/LC_MESSAGES
/usr/lib/locale/C.utf8/LC_MESSAGES/SYS_LC_MESSAGES
...output omitted...
```

Searching for Files Based on Ownership or Permission

The `find` command can also search for files based on their ownership or permissions. The `find` command `-user` and `-group` options search by a user and group name, or by user ID and group ID.

The following example searches for files in the `/home/developer` directory that the `developer` user owns:

```
developer@host:~$ find -user developer
.
./.bash_logout
./.bash_profile
...output omitted...
```

The next example searches for files in the `/home/developer` directory that the `developer` group owns:

```
developer@host:~$ find -group developer
.
./.bash_logout
./.bash_profile
...output omitted...
```

This example searches for files in the `/home/developer` directory that the `1000` user ID owns:

```
developer@host:~$ find -uid 1000
.
./.bash_logout
./.bash_profile
...output omitted...
```

This example searches for files in the `/home/developer` directory that the `1000` group ID owns:


```
developer@host:~$ find -gid 1000
.
./.bash_logout
./.bash_profile
...output omitted...
```

The `find` command `-user` and `-group` options search for files where the file owner and group owner are different. The following example lists files that are owned by the `root` user and the `mail` group:

```
root@host:~# find / -user root -group mail
/var/spool/mail
...output omitted...
```

The `find` command `-perm` option looks for files with a particular permission set. The octal values define the permissions with 4, 2, and 1 for read, write, and execute. Permissions are preceded by a forward slash (/) or a dash (-) to control the search results.

A forward slash (/) before the octal permission matches files where at least one permission is set for user, group, or others for that permission set. A /222 permission search does not match a file with `r--r--r--` permissions, but does match a file with `rw-r--r--` permissions.

A dash (-) before the permission means that all three parts of the permissions must match. For the previous example, files with the `rw-rw-rw-` permissions match. You can also use the `find` command `-perm` option with the symbolic method for permissions.

For example, the following commands match any file in the `/home` directory for which the owning user has read, write, and execute permissions, and members of the owning group have read and write permissions, and others have read-only access. Both commands are equivalent; the first one uses the octal method for permissions, whereas the second one uses the symbolic method.

```
root@host:~# find /home -perm 764
...output omitted...
root@host:~# find /home -perm u=rwx,g=rw,o=r
...output omitted...
```

The `find` command `-ls` option is convenient when searching files by permissions, because it provides information for the files that includes their permissions.

```
root@host:~# find /home -perm 764 -ls
26207447    0 -rwxrw-r--    1 user  user    0 Jun 16 04:29 /home/user/file1
```

To search for files for which the user has at least write and execute permissions, and the group has at least write permission, and others have at least read permission, run the following command:

```
root@host:~# find /home -perm -324
...output omitted...
root@host:~# find /home -perm -u=wx,g=w,o=r
...output omitted...
```

To search for files for which the user has read permissions, or the group has at least read permissions, or others have at least write permission, run the following command:

```
root@host:~# find /home -perm /442
...output omitted...
root@host:~# find /home -perm /u=r,g=r,o=w
...output omitted...
```

When used with the / or - signs, the 0 value works as a wildcard, because it means any permission.

To search for any file in the /home/developer directory for which others have at least read access on the host machine, run the following command:

```
developer@host:~$ find -perm -004
...output omitted...
developer@host:~$ find -perm -o=r
...output omitted...
```

To search for all files in the /home/developer directory where others have write permission, run the following command:

```
developer@host:~$ find -perm -002
...output omitted...
developer@host:~$ find -perm -o=w
...output omitted...
```

Finding Files Based on Size

The find command -size option is followed by a numeric value, and the unit looks up files that match a specified size. Use the following list for the units with the find command -size option:

- For kilobytes, use the k unit with k always in lowercase.
- For megabytes, use the M unit with M always in uppercase.
- For gigabytes, use the G unit with G always in uppercase.

You can use the plus (+) and minus (-) signs to include files that are larger and smaller than the given size, respectively. The following example shows a search for files with an exact size of 10 megabytes:

```
developer@host:~$ find -size 10M
...output omitted...
```

The next example searches for files with a size of more than 10 gigabytes:

```
developer@host:~$ find -size +10G
...output omitted...
```

This example searches for files with a size of less than 10 kilobytes:

```
developer@host:~$ find -size -10k
...output omitted...
```

Important

The `find` command `-size` option rounds everything to single units. For example, the `find -size 1M` command shows files that are smaller than 1 MB, because it rounds up all files to 1 MB.

Searching for Files Based on Modification Time

The `find` command `-mmin` option, followed by the time in minutes, searches for all files with content that changed *n* minutes ago. The file's time stamp is rounded down and supports fractional values with the `+n` and `-n` range.

The following example searches for all files with content that changed 120 minutes ago:

```
root@host:~# find / -mmin 120
...output omitted...
```

The `+` modifier in front of the minutes finds all files in the root (`/`) directory that changed more than *n* minutes ago. This example searches for all files with content that changed more than 200 minutes ago:

```
root@host:~# find / -mmin +200
...output omitted...
```

The `-` modifier searches for all files in the `/` directory that changed less than `n` minutes ago. The following example lists files that changed less than 150 minutes ago:

```
root@host:~# find / -mmin -150
...output omitted...
```

Searching for Files Based on File Type

The `find` command `-type` option limits the search scope to a given file type. Use the following flags to limit the search scope:

- For regular files, use the `f` flag.
- For directories, use the `d` flag.
- For soft links, use the `l` flag.
- For block devices, use the `b` flag.

The following example searches for all directories in the `/etc` directory:

```
root@host:~# find /etc -type d
/etc
/etc/tmpfiles.d
/etc/systemd
/etc/systemd/system
/etc/systemd/system/getty.target.wants
...output omitted...
```

This example searches for all soft links in the `/` directory:

```
root@host:~# find / -type l
...output omitted...
```

This example searches for all block devices in the `/dev` directory:

```
root@host:~# find /dev -type b
/dev/vda1
/dev/vda
```

The `find` command `-links` option followed by a number looks for all files with a specific hard link count. The number preceded by a `+` modifier looks for files with a higher count than the given hard link count. If the number precedes a `-` modifier, then the search is limited to files with a lower hard link count than the given number.

Search for all regular files with more than one hard link:

```
root@host:~# find / -type f -links +1
...output omitted...
```



References

`locate(1)`, `updatedb(8)`, and `find(1)` man pages

14.6. Guided Exercise

Locate Files on a File System

Search for files on a file system that is currently mounted.

Outcomes

- Search for files with the `find` and `locate` commands.

Prerequisites

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
student@workstation:~$ lab start fs-locate
```

Instructions

1. Log in to the servera machine as the student user.

```
student@workstation:~$ ssh student@servera
...output omitted...
[student@servera ~]$
```

2. Use the `locate` command to search for the following files on the servera machine.

- 2.1. Update the `locatedb` database manually on the servera machine.

Use the `sudo updatedb` command to update the database. Use `student` as the password.

```
[student@servera ~]$ sudo updatedb
[sudo] password for student: student
[student@servera ~]$
```

- 2.2. Locate the `logrotate.conf` configuration file.

```
[student@servera ~]$ locate logrotate.conf
/etc/logrotate.conf
/usr/share/man/man5/logrotate.conf.5.gz
```

- 2.3. Locate the `networkmanager.conf` configuration file, ignoring case sensitivity.

```
[student@servera ~]$ locate -i networkmanager.conf
/etc/NetworkManager/NetworkManager.conf
/etc/dbus-1/system.d/org.freedesktop.NetworkManager.conf
/usr/share/man/man5/NetworkManager.conf.5.gz
```

3. Use the find command to search for the following files in real time on the servera machine.

3.1. Use root privilege to find all files in the /var/lib directory that the chrony user owns. If required, use student as the Sudo password.

```
[student@servera ~]$ sudo find /var/lib -user chrony
/var/lib/chrony
/var/lib/chrony/drift
```

3.2. Use root privilege to find all files in the /var directory that are owned by the root user and the mail group. Use student as the Sudo password.

```
[student@servera ~]$ sudo find /var -user root -group mail
/var/spool/mail
```

3.3. Find all files in the /usr/bin directory with a file size greater than 50 KB.

```
[student@servera ~]$ find /usr/bin -size +50k
/usr/bin/iconv
/usr/bin/locale
/usr/bin/localedef
/usr/bin/cmp
...output omitted...
```

3.4. Find all files in the /home/student directory that changed in the last 120 minutes.

Important

Your command output might differ, depending on which files have been modified in the last 120 minutes in your lab environment.

```
[student@servera ~]$ find /home/student -mmin -120
/home/student/.bash_logout
/home/student/.bash_profile
/home/student/.bashrc
...output omitted...
```

3.5. Find all block device files in the /dev directory.

```
[student@servera ~]$ find /dev -type b
/dev/sdb
/dev/sda
/dev/sdc
/dev/sdb1
/dev/sdd
/dev/sda1
/dev/sda2
/dev/sda3
/dev/sr0
```

4. Return to the workstation machine as the student user.

```
[student@servera ~]$ exit
logout
Connection to servera closed.
student@workstation:~$
```

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish fs-locate
```


14.7. Lab

Access Removable Media

Access file systems on removable media devices by mounting them on a directory in the file-system hierarchy.

Outcomes

- Mount a file system.
- Generate a disk usage report.
- Find files in the local file system.

Prerequisites

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
student@workstation:~$ lab start fs-review
```

Instructions

1. As the root user on the serverb machine, identify the UUID for the `/dev/sdb1` device, and mount it by using its UUID on the `/mnt/system-report` directory.
2. Generate a disk usage report for the `/usr/share` directory. Save the result in the `/mnt/system-report/disk-usage.txt` file.
3. Use the `locate` command to find all the files that match the `rsyslog.conf` keyword, and store the results in the `/mnt/system-report/search1.txt` file.
4. Search for all files in the `/usr/share` directory that are larger than 5 MB but smaller than 10 MB. Save the results to the `/mnt/system-report/search2.txt` file.
5. After completing the tasks of this activity, return to the workstation machine as the student user.

Evaluation

As the student user on the workstation machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

```
student@workstation:~$ lab grade fs-review
```

Finish

On the workstation machine, change to the student user home directory and use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish fs-review
```

Solution

Access file systems on removable media devices by mounting them on a directory in the file-system hierarchy.

Outcomes

- Mount a file system.
- Generate a disk usage report.
- Find files in the local file system.

Prerequisites

As the student user on the workstation machine, run the following `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
student@workstation:~$ lab start fs-review
```

Instructions

1. As the root user on the serverb machine, identify the UUID for the `/dev/sdb1` device, and mount it by using its UUID on the `/mnt/system-report` directory.

- 1.1. Log in to the serverb machine as the student user, and switch to the root user. Use `student` as the password.

```
student@workstation:~$ ssh student@serverb
...output omitted...
[student@serverb ~]$ sudo -i
[sudo] password for student: student
[root@serverb ~]#
```

- 1.2. Query the UUID of the `/dev/sdb1` device.

```
[root@serverb ~]# lsblk -fp /dev/sdb
NAME          FSTYPE FSVER LABEL UUID                               FSAVAIL FSUSE% MOUNTPOINTS
/dev/sdb
└─/dev/sdb1 xfs                                     48bd5...3337a
```

The UUID might vary in your environment.

1.3. Verify whether the `/mnt/system-report` directory exists.

```
[root@serverb ~]# ls /mnt/system-report
ls: cannot access '/mnt/system-report': No such file or directory
```

The directory does not exist, so you must create it.

1.4. Create the `/mnt/system-report` directory.

```
[root@serverb ~]# mkdir /mnt/system-report
```

1.5. Mount the `/dev/sdb1` device on the `/mnt/system-report` directory by using the UUID.

Replace the placeholder UUID in the following command with the UUID from your environment.

```
[root@serverb ~]# mount UUID="48bd5...3337a" /mnt/system-report
```

1.6. Verify that the `/dev/sdb1` device is mounted on the `/mnt/system-report` directory.

```
[root@serverb ~]# lsblk -fp /dev/sdb1
NAME      FSTYPE FSVER LABEL UUID                               FSAVAIL FSUSE% MOUNTPOINTS
/dev/sdb1 xfs                    48bd5...3337a      4.8G      3% /mnt/system-report
```

2. Generate a disk usage report for the `/usr/share` directory. Save the result in the `/mnt/system-report/disk-usage.txt` file.

```
[root@serverb ~]# du /usr/share > /mnt/system-report/disk-usage.txt
```

3. Use the `locate` command to find all the files that match the `rsyslog.conf` keyword, and store the results in the `/mnt/system-report/search1.txt` file.**3.1.** Update the `locate` database.

```
[root@serverb ~]# updatedb
```

3.2. Use the `locate` command to find all the files that match the `rsyslog.conf` keyword. Save the results in the `/mnt/system-report/search1.txt` file.

```
[root@serverb ~]# locate rsyslog.conf > /mnt/system-report/search1.txt
```

4. Search for all files in the `/usr/share` directory that are larger than 5 MB but smaller than 10 MB. Save the results to the `/mnt/system-report/search2.txt` file.

```
[root@serverb ~]# find /usr/share -size +5M -size -10M > \
/mnt/system-report/search2.txt
```

5. After completing the tasks of this activity, return to the workstation machine as the student user.

```
[root@serverb ~]# exit
logout
[student@serverb ~]$ exit
logout
Connection to serverb closed.
student@workstation:~$
```

Evaluation

As the student user on the workstation machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

```
student@workstation:~$ lab grade fs-review
```

Finish

On the workstation machine, change to the student user home directory and use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish fs-review
```

14.8. Summary

In this lesson, you learned to manage block storage device, access the contents of a file system on removable media, and search the files on the file system.

Storage devices are represented by the *block device* file type. You can use the `df` command to identify total disk space, used disk space, and free disk space on all mounted regular file systems. You can also list the details of the block devices, such as size and the UUID.

As a privileged user, you can manually mount and unmount a file system. To successfully unmount a file system, make sure that all processes that access the mount point are stopped. Removable storage devices are mounted in the `/run/media` directory when using the graphical environment.

You can search for files in the file system by using indexes or real time searches. The `locate` command uses indexes to search for files. These indexes use a database that requires frequent updates. The `find` command performs real time searches in the local file systems. With the `find` command, you can search based on various file attributes, such as size, location, ownership, and permissions.

Chapter 15.

Monitoring and Managing Linux Processes

Goal

Investigate, control, and terminate processes that run on a Red Hat Enterprise Linux system.

Sections

- Processes and the Process Lifecycle (and Quiz)
- Managing Processes with Job Control (and Guided Exercise)
- Sending Signals to Processes (and Guided Exercise)
- Monitoring Process Activity (and Guided Exercise)

Lab

- Monitor and Manage Linux Processes

15.1. Processes and the Process Lifecycle

Objectives

- Identify the processes that are currently running on a system, determine their current process states, and gather information about their use of system resources.

Definition of a Process

A *process* is a running instance of a launched, executable program. From the moment that a process is created, it consists of the following items:

- An address space of allocated memory
- Security properties, including ownership credentials and privileges
- One or more execution threads of program code
- A process state

The *environment* of a process is a list of information that includes the following items:

- Local and global variables
- A current scheduling context
- Allocated system resources, such as file descriptors and network ports

An existing *parent* process duplicates its own address space, which is known as a process *fork*, to create a *child* process structure. Every new process is assigned a unique *process ID* (PID) for tracking and security purposes. The PID and the parent's process ID (PPID) are elements of the new process environment. Any process can create a *child* process. All processes are descendants of the first system process, *systemd*, on a Red Hat system.

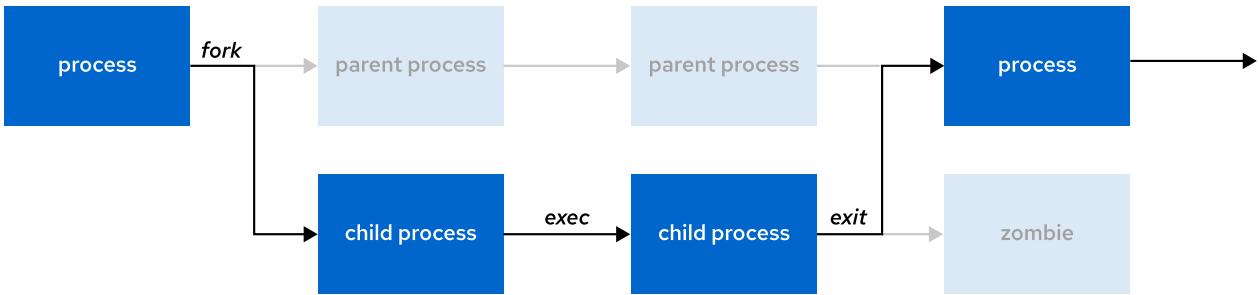


Figure 26. Process lifecycle

Through the *fork* routine, a child process inherits security identities, previous and current file descriptors, port and resource privileges, environment variables, and program code. A child process can then execute its own program code.

Typically, a parent process *sleeps* when the child process runs, and sets a *wait* request to be signaled when the child process completes. After the child process exits, it closes or discards its resources and environment, and leaves a *zombie* resource, which is an entry in the process table. The parent process, which is signaled to *wake* when the child process exits, cleans the process table of the child's entry, and it frees the last resource of the child process. The parent process then continues with its own program code execution.

Process States

In a multitasking operating system, each CPU (or CPU core) can work on one process at a time. As a process runs, its immediate requirements for CPU time and resource allocation change. Processes are assigned a *state*, which changes as circumstances dictate.

The following diagram and table describe Linux process states in detail.

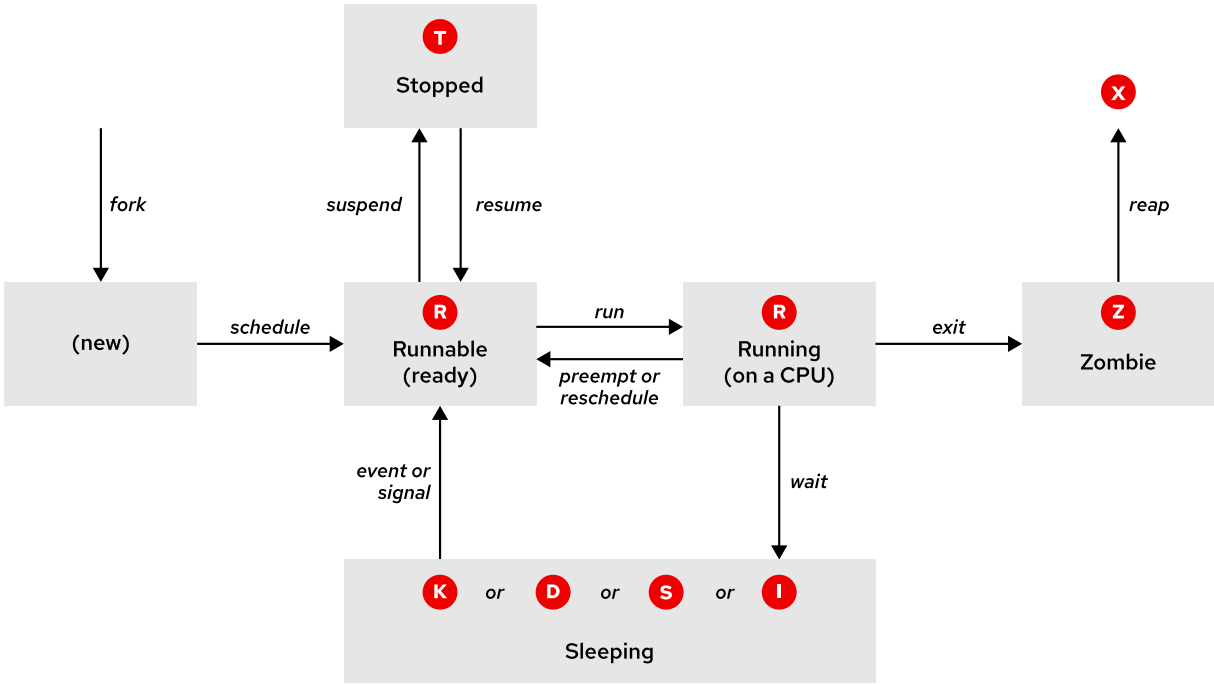


Figure 27. Linux process states

Table 21. Linux Process States

Name	Flag	Kernel-defined state name and description
Running	R	TASK_RUNNING: The process is either executing on a CPU or waiting to run. The process can be executing user routines or kernel routines (system calls), or be queued and ready when in the <i>Running</i> (or <i>Runnable</i>) state.

Name	Flag	Kernel-defined state name and description
Sleeping	S	TASK_INTERRUPTIBLE: The process is waiting for some condition: a hardware request, system resource access, or a signal. When an event or signal satisfies the condition, the process returns to <i>Running</i> .
	D	TASK_UNINTERRUPTIBLE: This process is also sleeping, but unlike the S state, it does not respond to signals. This state is used only when process interruption might cause an unpredictable device state.
	K	TASK_KILLABLE: Same as the uninterruptible D state, but modified to allow a waiting task to respond to the signal to kill it (exit completely). Utilities often display <i>Killable</i> processes as the D state.
	I	TASK_REPORT_IDLE: A subset of state D that is used for kernel threads. The kernel does not count these processes when calculating the load average. The TASK_UNINTERRUPTIBLE and TASK_NOLOAD flags are set. This state is similar to TASK_KILLABLE, and it accepts fatal signals.
Stopped	T	TASK_STOPPED: The process is stopped (suspended), usually by being signaled by a user or another process. The process can be continued (resumed) by another signal to return to running.
	T	TASK_TRACED: A process that is being debugged is also temporarily stopped and shares the T state flag.
Zombie	Z	EXIT_ZOMBIE: A child process signals to its parent as it exits. All resources except for the process identity (PID) are released.
	X	EXIT_DEAD: When the parent process cleans up (reaps) the remaining child process structure, the process is now released completely. This state cannot be observed in process-listing utilities.

Interpreting Process States

Understanding how the kernel communicates with processes, and how processes communicate with each other, can help you to troubleshoot a system.

The system assigns a state to every new process. The S column of the top command or the STAT column of the ps command displays the state of each process. On a single CPU system, only one process can run at a time. Several processes can be in an R state. However, not all processes are running consecutively; some of them are in *waiting* status.

```
user@host:~$ top
PID USER PR NI VIRT RES SHR S %CPU %MEM TIME+ COMMAND
  1 root  20  0 49516 41072 10164 S   0.0   0.7 0:02.30 systemd
  2 root  20  0      0      0      0 S   0.0   0.0 0:00.00 kthreadd
  3 root  20  0      0      0      0 S   0.0   0.0 0:00.00
pool_workqueue_release
...output omitted...
```

```
user@host:~$ ps aux
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
...output omitted...
root         2  0.0  0.0      0      0 ?        S      11:57   0:00 [kthreadd]
student  2496  0.0  0.0 231132  3872 pts/0    R+     16:45   0:00 ps aux
...output omitted...
```

Use signals to suspend, stop, resume, terminate, or interrupt processes. Processes can receive signals from the kernel, other processes, and other users on the same system.

Listing Processes

The ps command lists detailed information for current processes:

- User identification (UID), which determines process privileges
- Unique process identification (PID)
- Amount of used CPU and elapsed real time
- Amount of allocated memory
- The process stdout location, which is known as the controlling terminal
- The current process state

 Important

The Linux version of the `ps` command supports the following option formats:

- UNIX (POSIX) options, which can be grouped and must be preceded by a dash (-).
- BSD options, which can be grouped and must not be used with a dash.
- GNU long options, which are preceded by two dashes.

For example, the `ps -aux` command is not the same as the `ps aux` command.

The commonly used `ps aux` command displays all processes, including processes without a controlling terminal. In the following examples, scheduled kernel threads are displayed in brackets at the top of the list.

```
user@host:~$ ps aux
USER PID %CPU %MEM    VSZ   RSS TTY  STAT  START   TIME COMMAND
root   1   0.1   0.2 171820 16140 ?    Ss   16:47   0:01 /usr/lib/systemd/systemd
...
root   2   0.0   0.0      0      0 ?    S    16:47   0:00 [kthreadd]
root   3   0.0   0.0      0      0 ?    I<   16:47   0:00 [rcu_gp]
root   4   0.0   0.0      0      0 ?    I<   16:47   0:00 [rcu_par_gp]
root   6   0.0   0.0      0      0 ?    I<   16:47   0:00 [kworker/0:0H-events_highpri]
...output omitted...
```

The long listing (the `ps lax` command) provides more detail, and returns results more quickly by avoiding username lookups.

```
user@host:~$ ps lax
F UID PID PPID PRI  NI   VSZ   RSS WCHAN  STAT TTY  TIME  COMMAND
4  0   1    0  20   0 17820 1140 -        Ss   ?    0:01 /usr/lib/systemd/systemd
...
1  0   2    0  20   0      0      0 -        S    ?    0:00 [kthreadd]
1  0   3    2   0 -20      0      0 -        I<   ?    0:00 [rcu_gp]
1  0   4    2   0 -20      0      0 -        I<   ?    0:00 [rcu_par_gp]
1  0   6    2   0 -20      0      0 -        I<   ?    0:00 [kworker/0:0H-events_...]
...output omitted...
```

The similar UNIX syntax uses the `-ef` options to display all processes.

```

user@host:~$ ps -ef
UID    PID  PPID  C  STIME TTY   TIME      CMD
root    1     0    0  16:47 ?      00:00:01 /usr/lib/systemd/systemd ...
root    2     0    0  16:47 ?      00:00:00 [kthreadd]
root    3     2    0  16:47 ?      00:00:00 [rcu_gp]
root    4     2    0  16:47 ?      00:00:00 [rcu_par_gp]
root    6     2    0  16:47 ?      00:00:00 [kworker/0:0H-events_highpri]
...output omitted...

```

The default output of the `ps` command is sorted by process ID number (PID). Initially, the output might appear to use chronological order, but the kernel reuses process IDs, so the order is less structured than it appears. Use the `ps` command `-O` or `--sort` options to sort the output. The display order matches that of the system process table, which reuses table rows when processes die and spawn.

By default, the `ps` command with no options selects all processes with the current user's *effective user ID* (EUID), and selects processes that are associated with the terminal that is running the command. Zombie processes are listed with the `exiting` or `defunct` label.

You can use the `ps` command `--forest` option to display the processes in a tree format so that you can view the relationships between parent and child processes.

References

`ps(1)`, `top(1)`, and `signal(7)` man pages

15.2. Quiz

Processes and the Process Lifecycle

Choose the correct answers to the following questions:

1. Which command is primarily used to list currently running processes?

- ☐ a. nice
- ☐ b. lsps
- ☐ c. kill
- ☐ d. ps
- ☐ e. renice

2. Which state represents a process that is stopped or suspended?

- ☐ a. D
- ☐ b. R
- ☐ c. S
- ☐ d. T
- ☐ e. Z

3. Which state represents a process that released all its resources except its PID?

- ☐ a. D
- ☐ b. R
- ☐ c. S
- ☐ d. T
- ☐ e. Z

4. Which process duplicates the parent's address space, and creates a child process?

- ☐ a. exec
- ☐ b. fork
- ☐ c. zombie
- ☐ d. syscall
- ☐ e. reap

5. Which state represents a process that is sleeping until some condition is met?

- ☐ a. D
- ☐ b. R
- ☐ c. S
- ☐ d. T
- ☐ e. Z

Solution

1. Which command is primarily used to list currently running processes?

- ☐ a. nice
- ☐ b. lsps
- ☐ c. kill
- ☒ d. ps
- ☐ e. renice

2. Which state represents a process that is stopped or suspended?

- ☐ a. D
- ☐ b. R
- ☐ c. S
- ☒ d. T
- ☐ e. Z

3. Which state represents a process that released all its resources except its PID?

- ☐ a. D
- ☐ b. R
- ☐ c. S
- ☐ d. T
- ☒ e. Z

4. Which process duplicates the parent's address space, and creates a child process?

- ☐ a. exec
- ☒ b. fork
- ☐ c. zombie
- ☐ d. syscall
- ☐ e. reap

5. Which state represents a process that is sleeping until some condition is met?

 a. D

 b. R

 c. S

 d. T

 e. Z

15.3. Managing Processes with Job Control

Objectives

- Control foreground and background processes on the same terminal by using job control.

Jobs and Sessions

With the *job control* shell feature, you can run and manage multiple commands in a single shell instance.

A *job* is associated with each *pipeline* that is entered at a shell prompt. All processes in that pipeline are part of the job and are members of the same *process group*. You can consider a minimal pipeline to be only one command that is entered in the shell prompt to create a job with only one member.

Only one job at a time can read input and keyboard-generated signals from a particular terminal window. Processes that are part of that job are *foreground* processes of that *controlling terminal*.

A *background* process of that controlling terminal is any other job that is associated with that terminal. Background processes of a terminal cannot read input or receive keyboard-generated interrupts from the terminal, but they can write to the terminal. A background job might be stopped (suspended) or it might be running. If a running background job tries to read from the terminal, and then it is automatically suspended.

Each terminal runs in its own *session*, and can have a foreground process and any number of background processes. A job that runs in its own session belongs to its controlling terminal.

The `ps` command displays the device name of the controlling terminal in the TTY column.

```
user@host:~$ ps
  PID TTY          TIME CMD
 2328 pts/0    00:00:00 bash
 8910 pts/0    00:00:00 ps
```

Some processes, such as *system daemons*, are started by the system and not from a controlling terminal. These processes are not members of a job, and cannot be brought to the foreground. The `ps` command displays a question mark (?) in the TTY column for these processes.

Use the `ps -j` command to find process and session information, as well as details about jobs.

In the following example, the `sleep` command is currently suspended and the process state is `T` (stopped).

```
user@host:~$ ps j
PPID      PID      PGID      SID TTY      TPGID  STAT    UID      TIME  COMMAND
...output omitted...
8527      8528      8528      8528 pts/0    8633  Ss       1000     0:00  -bash
8528      8559      8559      8528 pts/0    8633  T        1000     0:00  sleep 10000
8528      8633      8633      8528 pts/0    8633  R+       1000     0:00  ps j
```

- The PID is the unique process ID of the process.
- The *parent process ID (PPID)* is the PID of the *parent process* that started (forked) this process.
- The PGID is the PID of the *process group leader*, typically the first process in the job's pipeline.
- The SID is the PID of the *session leader*, which (for a job) is typically the interactive shell that is running on its controlling terminal.

Running Jobs in the Background

You can start any command or pipeline in the background by appending an ampersand (&) to the command. The Bash shell displays a *job number* (unique to the session) and the *process ID (PID)* of the new child process. The shell does not wait for the child process to terminate, but instead displays the shell prompt.

In the following example, the `sleep` command runs in the background with the & operator, pausing for 10000 seconds. The output shows the job number that the shell assigns (`[1]`) and a process ID (PID) of 5947.

```
user@host:~$ sleep 10000 &
[1] 5947
```

When a command line with a pipe (`|`) is sent to the background, the PID of the last command in the pipeline is displayed. All pipeline processes are members of that job.

In the following example, the `ls` command is piped through the `sort` command, and the sorted output is sent via email by using the `mail -s` command. The entire pipeline runs in the background with the & operator, enabling the terminal to remain available.

```
user@host:~$ ls | sort | mail -s "Sort output" &
[1] 5998
```

Use the `jobs` command to display the list of jobs for the shell's session.

```
user@host:~$ jobs
[1]+  Running      sleep 10000 &
```

Bringing a Background Job to the Foreground

Use the `fg` command to bring a background job to the foreground. Use the (`%jobNumber`) format to specify which process to foreground.

```
user@host:~$ fg %1
sleep 10000
```

In the previous example, the `sleep` command is running in the foreground on the controlling terminal. The shell itself is asleep and waiting for this child process to exit.

Sending a Foreground Process to the Background

To send a foreground process to the background, press the keyboard-generated *suspend* request (`Ctrl + Z`) in the terminal. The job moves to the background and is suspended.

```
user@host:~$ sleep 10000
^Z
[1]+  Stopped          sleep 10000
```

Starting a Suspended Process

To start running the suspended process, use the `bg` command with the job ID:

```
user@host:~$ bg %1
[1]+ sleep 10000 &
```

The shell warns a user who attempts to exit a terminal window (session) with suspended jobs. If the user tries again to exit immediately, then the suspended jobs are stopped.

 Note

In the previous examples, the plus sign (+) indicates that this job is the current default. If you use a job-control command without the `%jobNumber` argument, then the action is taken on the default job. The minus sign (-) indicates that the previous job will become the default job when the current one completes.

 References

`info bash` (*The GNU Bash Reference Manual*)

- Section 7: Job Control

`bash(1)`, `builtins(1)`, `ps(1)`, and `sleep(1)` man pages

15.4. Guided Exercise

Manage Processes with Job Control

Control foreground and background processes on the same terminal by using job control.

Outcomes

- Use job control to suspend and to restart user processes.

Prerequisites

As the student user on the workstation machine, run the following `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
student@workstation:~$ lab start processes-control
```

Instructions

1. On the workstation machine, open two terminal windows side by side. In this activity, these two terminals are referred to as *left* and *right*. In each terminal, log in to the *servera* machine as the student user.

```
student@workstation:~$ ssh student@servera  
[student@servera ~]$
```

2. In the left terminal shell, create the `/home/student/bin` directory. Create a shell script named `control` in the `/home/student/bin` directory. Change the script permissions to make it executable.

- 2.1. Create the `/home/student/bin` directory.

```
[student@servera ~]$ mkdir /home/student/bin
```

- 2.2. Create a file named `control` in the `/home/student/bin` directory with the following contents:

```
#!/bin/bash
while true; do
    echo -n "$@ " >> ~/control_outfile
    sleep 1
done
```

The `control` script appends command-line arguments to the `~/control_outfile` file once per second. The script runs until the process is terminated.

2.3. Make the `control` file executable.

```
[student@servera ~]$ chmod +x /home/student/bin/control
```

3. Execute the `control` script. The script continuously appends the word "technical" and a space to the `~/control_outfile` file at one-second intervals.

```
[student@servera ~]$ control technical
```

4. In the right terminal shell, verify that the new process is writing to the `/home/student/control_outfile` file. Keep the `tail` command running to verify the output states in subsequent steps.

```
[student@servera ~]$ tail -f ~/control_outfile
technical technical technical technical
...output omitted...
```

5. Suspend the running process, and confirm that the process is stopped.

- 5.1. In the left terminal shell, press `Ctrl` + `Z` to suspend the running process. The shell returns the job ID in square brackets.

```
^Z
[1]+  Stopped                  control technical
[student@servera ~]$
```

- 5.2. In the right terminal shell, confirm that the process output is stopped.

```
technical technical technical technical
...no further output...
```

6. Restart the job in the background, and verify that the process is active again.

- 6.1.** In the left terminal shell, use the `jobs` command to view the list of jobs and to retrieve the job number.

```
[student@servera ~]$ jobs
[1]+  Stopped                  control technical
```

The plus sign (+) indicates the default job.

- 6.2.** Use the `bg` command to restart the `control` job in the background.

```
[student@servera ~]$ bg
[1]+ control technical &
```

- 6.3.** Use the `jobs` command to verify that the `control` job is running again.

```
[student@servera ~]$ jobs
[1]+  Running                  control technical &
```

- 6.4.** In the right terminal shell, confirm that the `tail` command is producing output.

```
...output omitted...
technical technical technical technical technical technical technical technical
```

- 7.** In the left terminal shell, start two more `control` processes to append to the `~/output` file. Use the ampersand (&) operator to start the processes in the background. Replace the `technical` argument with `documents`, and then with `database`. Replacing the arguments helps to differentiate between the three processes.

```
[student@servera ~]$ control documents &
[2] 6579
[student@servera ~]$ control database &
[3] 6654
```

- 8.** Verify that all three processes are running.

- 8.1.** In the left terminal shell, use the `jobs` command to view the three running processes.

```
[student@servera ~]$ jobs
[1]    Running                  control technical &
[2]-   Running                  control documents &
[3]+   Running                  control database &
```

8.2. In the right terminal shell, verify that all three processes are appending to the file.

```
...output omitted...
technical documents database technical documents database technical documents
database technical documents database
...output omitted...
```

9. Suspend the control technical process, and confirm that it is suspended.

9.1. In the left terminal shell, foreground the control technical process. Press **Ctrl** + **Z** to suspend the process. Verify that the process is suspended.

```
[student@servera ~]$ fg %1
control technical
^Z
[1]+  Stopped                  control technical
[student@servera ~]$ jobs
[1]+  Stopped                  control technical
[2]    Running                  control documents &
[3]-   Running                  control database &
```

9.2. In the right terminal shell, verify that the control technical process is no longer sending output.

```
database documents database documents database
...no further output...
```

10. Terminate the control documents process, and verify that it is terminated.

10.1. In the left terminal shell, run the control documents process in the foreground. Press **Ctrl** + **C** to terminate the process. Verify that the process is terminated.

```
[student@servera ~]$ fg %2
control documents
^C
[student@servera ~]$ jobs
[1]+  Stopped                  control technical
[3]-  Running                  control database &
```

10.2. In the right terminal shell, verify that the `control documents` process is no longer sending output.

```
...output omitted...
database database database database database database database database
...no further output...
```

11. In the left terminal shell, view the remaining jobs. The suspended jobs have a state of T. The other background jobs are sleeping and have a state of S.

```
[student@servera ~]$ ps jT
PPID    PID    PGID ... STAT ... COMMAND
14783   14784   14784 ... Ss   ... -bash
14784   14977   14977 ... T    ... /bin/bash /home/student/bin/control
technical
14784   15189   15189 ... S    ... /bin/bash /home/student/bin/control database
14977   15408   14977 ... T    ... sleep 1
15189   15587   15189 ... S    ... sleep 1
14784   15588   15588 ... R+   ... ps jT
```

The output might differ from your environment because it omits some nonessential columns to focus on relevant process information.

12. Terminate the `control database` process, and confirm that it is terminated.

12.1. In the left terminal shell, view the current jobs.

```
[student@servera ~]$ jobs
[1]+  Stopped                  control technical
[3]-  Running                  control database &
```

12.2. Use the `fg` command with the job ID to run the `control database` process in the foreground. Press `Ctrl` + `C` to terminate the process. Verify that the process is terminated.

```
[student@servera ~]$ fg %3
control database
^C
[student@servera ~]$ jobs
[1]+  Stopped                  control technical
```

13. In the right terminal shell, use the `Ctrl + C` command to stop the `tail` command. Delete the `~/control_outfile` file.

```
...output omitted...
^C
[student@servera ~]$ rm ~/control_outfile
```

14. Close the extra terminal. Return to the workstation system as the student user.

```
[student@servera ~]$ exit
logout
Connection to servera closed.
student@workstation:~$
```

Finish

On the workstation machine, change to the student user home directory and use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish processes-control
```

15.5. Sending Signals to Processes

Objectives

- Send signals to processes to control their activity or to terminate execution.

Controlling Processes with Signals

A *signal* is a software interrupt that is delivered to a process. Signals report events to an executing program. Events that generate a signal can be an error, an external event (an I/O request or an expired timer), or by the explicit use of a signal-sending command or keyboard sequence.

The following table lists the fundamental signals that system administrators use for routine process management. Refer to signals by their short (HUP) or proper (SIGHUP) name.

Table 22. Fundamental Process Management Signals

Signal	Name	Definition
1	HUP	Hangup : Reports termination of the controlling process of a terminal. Also requests process re-initialization (configuration reload) without termination.
2	INT	Keyboard interrupt : Causes program termination. It can be blocked or handled. Sent by pressing the INTR (Interrupt) key sequence (<code>Ctrl + C</code>).
3	QUIT	Keyboard quit : Similar to SIGINT; adds a process dump at termination. Sent by pressing the QUIT key sequence (<code>Ctrl+\</code>).
9	KILL	Kill, unblockable : Causes abrupt program termination. It cannot be blocked, ignored, or handled; consistently fatal.
15 default	TERM	Terminate : Causes program termination. Unlike SIGKILL, it can be blocked, ignored, or handled. It allows the program to complete essential operations and to self-cleanup before termination.

Signal	Name	Definition
18	CONT	Continue : Sent to a process to resume if stopped. It cannot be blocked. Even if handled, it always resumes the process.
19	STOP	Stop, unblockable : Suspends the process. It cannot be blocked or handled.
20	TSTP	Keyboard stop : Unlike SIGSTOP, it can be blocked, ignored, or handled. Sent by pressing the suspend key sequence (<code>Ctrl</code> + <code>Z</code>).

 Note

Signal numbers vary between Linux hardware platforms, but signal names and meanings are standard. The recommended practice is to use signal names rather than numbers when signaling. The numbers that are discussed in this section are for x86_64 architecture systems.

Each signal has a *default action*, which is usually one of the following actions:

- Term : Terminate a program (exit) immediately.
- Core : Save a program's memory image (core dump), and then terminate.
- Stop : Stop a running program (suspend) and wait to continue (resume).

Programs react to the expected event signals by implementing handler routines to ignore, replace, or extend a signal's default action.

 Important

Red Hat recommends sending the SIGTERM signal first, and then trying SIGINT. If both signals fail, then try again with SIGKILL. Administrators commonly use SIGKILL first because this signal forces the process termination no matter the process state and cannot be ignored. However, with the SIGKILL signal, the process is not allowed to run any self-cleanup routines.

Sending Signals by Explicit Request

You can signal the current foreground process by pressing a keyboard control sequence to suspend (`Ctrl` + `Z`), kill (`Ctrl` + `C`), or core dump (`Ctrl`+`\`) the process. However, you might use signal-

sending commands to send signals to background processes in a different session.

You can specify signals either by name (for example, with the `-HUP` or `-SIGHUP` options) or by number (with the related `-1` option). Users can kill their processes, but root privilege is required to kill processes that other users own.

The `kill` command uses a *process ID (PID)* number to send a signal to a process. Despite its name, you can use the `kill` command to send any signal, not just those signals for terminating programs. You can use the `kill` command `-l` option to list the names and numbers of all available signals.

```
user@host:~$ kill -l
1) SIGHUP      2) SIGINT      3) SIGQUIT     4) SIGILL      5) SIGTRAP
6) SIGABRT     7) SIGBUS      8) SIGFPE      9) SIGKILL     10) SIGUSR1
11) SIGSEGV    12) SIGUSR2    13) SIGPIPE    14) SIGALRM     15) SIGTERM
16) SIGSTKFLT  17) SIGCHLD    18) SIGCONT    19) SIGSTOP     20) SIGTSTP
...output omitted...
```

To find the PID of the process that you want to terminate, you can use the `ps aux` command and filter by the process name. In the following example, you search for all processes that contain `task` in their name and then terminate the `task1` process by using the default `SIGTERM` signal. The `SIGTERM` signal allows the process to complete tasks and to perform cleanup routines.

```
user@host:~$ ps aux | grep task
user 6564 0.0 0.0 228796 3264 pts/0  S  17:22 0:00 /bin/bash ~/bin/task1
user 6566 0.0 0.0 228796 3244 pts/0  S  17:22 0:00 /bin/bash ~/bin/task2
user 6568 0.0 0.0 228796 3220 pts/0  S  17:22 0:00 /bin/bash ~/bin/task3
...output omitted...
user@host:~$ kill 6564
[1]  Terminated                  task1
```

Continuing with the `task` processes example, you then terminate the `task2` and `task3` processes by using the `SIGKILL` signal. You use the numerical value of the signal to terminate the `task2` process, and you use the signal name to terminate the `task3` process. The `SIGKILL` signal terminates the process abruptly.

```

user@host:~$ kill -9 6566
[2]-  Killed                  task2
user@host:~$ ps aux | grep task2
user 6568 0.0 0.0 228796 3220 pts/0  S   17:22 0:00 /bin/bash ~/bin/task3
...output omitted...
user@host:~$ kill -SIGKILL 6568
[3]+  Killed                  task3

```

Controlling Specific Processes

Use the `pkill` command to signal one or more processes that match selection criteria. Selection criteria can be a command name, a process that a specific user owns, or all system-wide processes. Processes and sessions can be individually or collectively signaled.

Because the initial process in a login session (*session leader*) is designed to handle session termination requests and to ignore unintended keyboard signals, killing all the processes and login shells for a single user requires the `SIGKILL` signal.

When a process is in the T (stopped/traced) state, such as after receiving a `SIGSTOP` signal, the process cannot respond to signals like `SIGTERM`, which is what the `pkill` and `killall` commands send by default. As a result, the process remains in the T state and is displayed in the output of the `top` and `ps` commands, even after using the `pkill` command.

Use the `pgrep` command to identify the *PID* numbers to kill. This command operates similarly to the `pkill` command, including most of the same options, except that the `pgrep` command lists processes rather than killing them.

Use the `pgrep` command with the `-l` option to list the process names and IDs. Use either command with the `-u` option to specify the ID of the user who owns the processes. The following example lists the processes for the bob user and then sends the `SIGKILL` signal to all processes of that user.

```

root@host:~# pgrep -l -u bob
6964 bash
6998 sleep
6999 sleep
7000 sleep
root@host:~# pkill -SIGKILL -u bob
root@host:~# pgrep -l -u bob

```

When the target processes are in the same login session, killing all the processes of a user might not be needed. In this scenario, you can identify the user terminal for the session, and then kill only the

processes that reference the same terminal ID. You can determine the terminal for the session by using the `w -u username` command. Then you can use the `kill -t ttyN` command, where *N* is the number of the target terminal.

Note

Unless SIGKILL is specified, the session leader (here, the Bash login shell) successfully handles and survives the termination request, but terminates all other session processes.

In the following example, you list all processes that belong to the bob user, determine the terminal for the session, and then send the default termination signal (SIGTERM). Note that all processes that belong to the bob user are terminated, except the session leader, the bash process. The commands are run as the root user so that you can observe the difference between sending the SIGTERM and the SIGKILL signals.

```
root@host:~# pgrep -l -u bob
7391 bash
7426 sleep
7427 sleep
7428 sleep
root@host:~# w -u bob
USER      TTY      FROM          LOGIN@  IDLE   JCPU   PCPU WHAT
bob       tty3                    18:37   5:04   0.03s  0.03s -bash
root@host:~# kill -t tty3
root@host:~# pgrep -l -u bob
7391 bash
```

By sending the SIGKILL signal, you also terminate the session leader, as shown in the following example.

```
root@host:~# kill -SIGKILL -t tty3
root@host:~# pgrep -l -u bob
```

You can apply the same selective process termination with parent and child process relationships. Use the `ps tree` command to view a process tree for the system or for a single user. Use the parent process's *PID* to kill all child processes that it created. The parent Bash login shell survives this time, because the signal is directed only at its child processes.

```

root@host:~# pstree -p bob
bash(8391)─┬─sleep(8425)
            │   └─sleep(8426)
            └─sleep(8427)

root@host:~# pkill -P 8391
root@host:~# pgrep -l -u bob
bash(8391)
root@host:~# pkill -SIGKILL -P 8391
root@host:~# pgrep -l -u bob
bash(8391)

```

Sending a Signal to Multiple Processes

The `killall` command can signal multiple processes, based on their command name. In the following example, you first list all processes that contain `job` in their name and then terminate all those processes.

```

user@host:~$ ps aux | grep job
user 9259 0.0 0.0 228796 3236 pts/0  S   20:05 0:00 /bin/bash ~/bin/control job1
user 9261 0.0 0.0 228796 3232 pts/0  S   20:05 0:00 /bin/bash ~/bin/control job2
user 9270 0.0 0.0 228796 3240 pts/0  S   20:06 0:00 /bin/bash ~/bin/control job3
...output omitted...
user@host:~$ killall job
[1]  Terminated                  control job1
[2]  Terminated                  control job2
[3]- Terminated                  control job3

```

Terminating Background Jobs

To terminate background jobs, use the `kill` command and specify the job number. Use the `jobs` command to find the job number of the process to terminate.

```

user@host:~$ jobs
[1]-  Running                    sleep 500 &
[2]+  Running                    sleep 1000 &

```

Terminate a specific job by using the `kill` command. Prefix the job number with a percent sign (%).

```

user@host:~$ kill -SIGTERM %1
user@host:~$ jobs
[2]+  Running                  sleep 1000 &

```

Administratively Logging out Users

You might need to log out other users for various reasons:

- The user committed a security violation.
- The user overused resources.
- The user has an unresponsive system
- The user has improper access to materials.

In these cases, you must terminate their session by using signals administratively.

To log off a user, first identify the login session to be terminated. The `w` command lists user logins and currently running processes. Note the TTY and FROM columns to determine the closing sessions.

```

user@host:~$ w -f
 12:43:06 up 27 min,  5 users,  load average: 0.03, 0.17, 0.66
USER  TTY      FROM            LOGIN@   IDLE   JCPU   PCPU WHAT
root  tty2                    12:26   14:58   0.04s   0.04s -bash
bob   tty3                    12:28   14:42   0.02s   0.02s -bash
user  pts/1 desktop.example.com 12:41    2.00s   0.03s   0.03s w
user@host:~$

```

All user login sessions are associated with a terminal device (TTY). If the device name is `pts/N`, then it is a *pseudo-terminal* that is associated with a graphical terminal window or a remote login session. If the device name is `ttyN`, then the user is on a system console, an alternative console, or another directly connected terminal device.

Discover how long a user has been on the system by viewing the session login time. CPU resources that current jobs consume, including background tasks and child processes, are in the JCPU column for each session. Current foreground process CPU consumption is in the PCPU column.

The `w` command shows who is logged in and what each user is doing. In contrast, the `who` command provides only basic information about logged-in users.

```
user@host:~$ who
root      tty2          2025-06-09 12:26 (tty2)
bob       tty3          2025-06-09 12:28 (tty3)
user      pts/1         2025-06-09 12:41 (desktop.example.com)
```

After identifying the session to terminate, use the `kill -t` command to end it.

Only the root user or a user with elevated privileges (via Sudo) can terminate processes that belong to other users. Regular users can kill only their own processes. If you attempt to run the `kill -t` command to end another user's session without appropriate privileges, then the system returns an `Operation not permitted` error.

```
user@host:~$ sudo kill -t tty3
user@host:~$ who
root      tty2          2025-06-09 12:26 (tty2)
user      pts/1         2025-06-09 12:41 (desktop.example.com)
```

The `tty3` session for the bob user is terminated, and the user is now logged out.

References

info libc signal (GNU C Library Reference Manual)

- Section 24: Signal Handling

info libc processes (GNU C Library Reference Manual)

- Section 26: Processes

`kill(1)`, `killall(1)`, `pgrep(1)`, `pkill(1)`, `pstree(1)`, `signal(7)`, and `w(1)` man pages

15.6. Guided Exercise

Send Signals to Processes

Send signals to processes to control their activity or to terminate execution.

Outcomes

- Start and stop multiple shell processes.

Prerequisites

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
student@workstation:~$ lab start processes-kill
```

Instructions

1. On the workstation machine, open two terminal windows side by side. In this activity, these terminals are referred to as *left* and *right*. In each terminal, log in to the servera machine as the student user.

```
student@workstation:~$ ssh student@servera
[student@servera ~]$
```

2. In the left terminal shell, start three processes in the background with the instance script file, by passing the network, interface, and connection arguments.

The instance script appends command-line arguments to the `~/instance_outfile` file once per second. The script runs until the process is terminated.

```
[student@servera ~]$ instance network &
[1] 19851
[student@servera ~]$ instance interface &
[2] 19857
[student@servera ~]$ instance connection &
[3] 19864
```

3. In the right terminal shell, verify that all three processes are appending content to the `/home/student/instance_outfile` file. Keep the `tail` command running to verify the output

states in subsequent steps.

```
[student@servera ~]$ tail -f ~/instance_outfile
network interface connection network interface connection network interface
...output omitted...
```

4. In the left terminal shell, list existing jobs.

```
[student@servera ~]$ jobs
[1]    Running                  instance network &
[2]-  Running                  instance interface &
[3]+  Running                  instance connection &
```

5. Use signals to suspend the `instance network` process, and verify that the process is stopped.

- 5.1. In the left terminal shell, stop the `instance network` process. Verify that the process state is Stopped.

```
[student@servera ~]$ kill -SIGSTOP %1
[1]+  Stopped                  instance network
[student@servera ~]$ jobs
[1]+  Stopped                  instance network
[2]    Running                  instance interface &
[3]-  Running                  instance connection &
```

Note

This guided exercise uses job numbers (%1, %2, and so on) to send signals, which is appropriate in this context because the processes are started in the same shell session and the job control features of Bash are available. If desired, you can also identify the parent process ID (PPID) by using the `ps aj` command, and send signals directly to the parent shell or individual process IDs (PIDs) instead. This approach is especially useful when scripting or working across multiple sessions.

- 5.2. In the right terminal shell, view the `tail` command output. Verify that the word `network` is no longer appended to the `~/instance_outfile` file.

```
...output omitted...
interface connection interface connection interface connection interface
```

6. Terminate the `instance interface` process, and verify that this process is terminated.

6.1. In the left terminal shell, terminate the `instance interface` process. Verify that the process is terminated.

```
[student@servera ~]$ kill -SIGTERM %2
[2] Terminated instance interface
[student@servera ~]$ jobs
[1]+ Stopped instance network
[3]- Running instance connection &
```

6.2. In the right terminal shell, view the `tail` command output. Verify that the word `interface` is no longer appended to the `~/instance_outfile` file.

```
...output omitted...
connection connection connection connection connection connection connection
connection
```

7. Resume the `instance network` process, and verify that the process is running.

7.1. In the left terminal shell, resume the `instance network` process. Verify that the process is in the Running state.

```
[student@servera ~]$ kill -SIGCONT %1
[student@servera ~]$ jobs
[1]+ Running instance network &
[3]- Running instance connection &
```

7.2. In the right terminal shell, view the `tail` command output. Verify that the word `network` is appended to the `~/instance_outfile` file.

```
...output omitted...
network connection network connection network connection network connection
network connection
```

8. In the left terminal shell, terminate all remaining `instance` processes. Verify that no jobs remain.

```
[student@servera ~]$ pkill -SIGTERM instance
[1]+  Terminated                  instance network
[3]-  Terminated                  instance connection
[student@servera ~]$ jobs
```

9. List the current running processes in all open terminal shells. Terminate the `tail` process that is running in the right terminal, and verify that the process is no longer running.

- 9.1. In the left terminal shell, list all current running processes. Refine the search to view only `tail` lines.

Note the PID of the `tail` command to use at a later step.

```
[student@servera ~]$ ps -ef | grep tail
student  4581 31358  0 10:02 pts/0    00:00:00 tail -f
/home/student/instance_outfile
student  4869 2252  0 10:33 pts/1    00:00:00 grep --color=auto tail
```

The output shows two processes: one to run the `tail` command (PID 4581) and one to run the `grep` command (PID 4869). The process with PID 4869 is listed because it contains the string `tail`, which matches the filtering of the `ps -ef` command (`grep tail`). The `grep` command exits immediately after the pipeline finishes, which leaves the `tail` process as the only process that matches the criteria.

- 9.2. Still in the left terminal shell, terminate the `tail` process. Use the PID that you obtained in the previous step. The PID might vary in your environment.

Verify that the process is no longer running.

```
[student@servera ~]$ kill -SIGTERM 4581
[student@servera ~]$ ps -ef | grep tail
student  4874 2252  0 10:36 pts/1    00:00:00 grep --color=auto tail
```

Alternatively, you can run the `killall tail` command. As confirmed in a previous step, only one `tail` process is running, which means that terminating all `tail` processes does not affect other processes or unintended users.

- 9.3. In the right terminal shell, verify that the `tail` command is no longer running.


```
...output omitted...  
network connection network connection network connection Terminated  
[student@servera ~]$
```

10. Close the extra terminal. Return to the workstation system as the student user.

```
[student@servera ~]$ exit  
logout  
Connection to servera closed.  
student@workstation:~$
```

Finish

On the workstation machine, change to the student user home directory and use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish processes-kill
```

15.7. Monitoring Process Activity

Objectives

- Investigate, control, and terminate processes that run on a Red Hat Enterprise Linux system.

Load Average

Load average is a measurement that the Linux kernel provides, to represent the perceived system load for a period of time. It can be used as a rough gauge of how many system resource requests are pending, to determine whether system load increases or decreases.

The kernel collects the current load number every five seconds based on the number of processes in runnable and uninterruptible states. This number is reported as an exponential moving average over the most recent 1, 5, and 15 minutes.

Calculating Load Average

Load average is a rough measurement of how many processes are currently waiting for a request to complete before they do anything else. The request might be for CPU time to run the process. Alternatively, the request might be for a critical disk I/O operation to complete, and the process cannot be run on the CPU until the request completes, even though the CPU is idle. Either way, system load is impacted, and the system appears to run more slowly because processes are waiting to run.

The load number is a running average of the number of processes that are ready to run on a CPU (in process state R) or are waiting for disk or I/O to complete (in process state D). Some UNIX systems consider only CPU usage or run queue length to indicate system load. Linux also includes disk or network usage, because the high usage of these resources can significantly impact system performance as CPU load.

For high load averages with minimal CPU activity, examine disk and network activity.

Interpreting Load Average Values

The `uptime` command is one way to display the current load average. This command prints the current time, how long the machine has been running, how many user sessions are running, and the current load average.

```
user@host:~$ uptime
15:29:03 up 14 min,  2 users,  load average: 2.92, 4.48, 5.20
```

The three values for the load average represent the load over the last 1, 5, and 15 minutes. These values indicate whether the system load appears to be increasing or decreasing.

If the main contribution to load average is from processes that are waiting for the CPU, then you can calculate the approximate load value per CPU to determine whether the system is experiencing significant waiting.

Use the `lscpu` command to determine the number of CPUs on a system. In the following example, the system is a dual-core single-socket system with two hyper threads per core. Linux treats this CPU configuration as a four-CPU system for scheduling purposes.

```
user@host:~$ lscpu
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Byte Order:            Little Endian
CPU(s):                4
On-line CPU(s) list:   0-3
Thread(s) per core:    2
Core(s) per socket:    2
Socket(s):              1
NUMA node(s):          1
...output omitted...
```

Imagine that the only contribution to the load number is from processes that need CPU time. Therefore, you can divide the displayed load average values by the number of logical CPUs in the system. A value below 1 indicates adequate resource use and minimal wait times. A value above 1 indicates resource saturation and some processing delay.

```
# From ``lscpu``, the system has four logical CPUs, so divide by 4:
#                               load average: 2.92, 4.48, 5.20
#           divide by number of logical CPUs:    4      4      4
#                               -----
#           per-CPU load average: 0.73  1.12  1.30
#
# This system's load average appears to be decreasing.
# With a load average of 2.92 on four CPUs, all CPUs were in use ~73% of the
# time.
# During the last 5 minutes, the system was overloaded by ~12%.
# During the last 15 minutes, the system was overloaded by ~30%.
```

An idle CPU queue has a load number of 0. Each process that waits for a CPU adds 1 to the load number. If one process is running on a CPU, then the load number is 1, and the resource (the CPU) is in use, but no requests are waiting. If that process runs for an entire minute, then its contribution to the one-minute load average is 1.

However, processes that are in an uninterruptible sleep state for critical I/O due to a busy disk or network resource are also included in the count and increase the load average. Although not indicating CPU use, these processes are added to the queue count, because they wait for resources and cannot run on a CPU until they get the resources. This metric is still considered as system load due to resource limitations that cause processes not to run.

Until resource saturation, a load average remains below 1, because tasks are seldom found waiting in the queue. Load average increases only when resource saturation causes requests to remain queued, and the load calculation routine counts them. When resource use approaches 100%, each extra request starts experiencing service wait time.

Real-time Process Monitoring

The `top` command displays a dynamic view of the system's processes and a summary header followed by a process or thread list. Unlike the static `ps` command output, the `top` command continuously refreshes at a configurable interval and provides column reordering, sorting, and highlighting. You can make persistent changes to the `top` settings.

```
user@host:~$ top
...output omitted...
PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM    TIME+  COMMAND
16486 root        20   0       0       0       0 I   0.3   0.0   0:00.03 kworker/1:1-
events
16811 student    20   0  232736    5624    3448 R   0.3   0.3   0:00.02 top
   1 root        20   0   48740   40544   10352 S   0.0   2.0   0:07.37 systemd
   2 root        20   0       0       0       0 S   0.0   0.0   0:00.01 kthreadd
   3 root        20   0       0       0       0 S   0.0   0.0   0:00.00
pool_workqueue_release
...output omitted...
```

The default top output columns are as follows:

- PID : The process ID.
- USER : The process owner username.
- VIRT : Virtual memory is all the memory that the process uses, including the resident set, shared libraries, and any mapped or swapped memory pages (labeled VSZ in the ps command).
- RES : Resident memory is the physical memory that the process uses, including any resident, shared objects (labeled RSS in the ps command).
- S : Process state can be one of the following states:
 - D : Uninterruptible sleep
 - R : Running or runnable
 - S : Sleeping
 - T : Stopped or traced
 - Z : Zombie
- TIME : CPU time is the total processing time since the process started. CPU time can be toggled to include a cumulative time of all previous children processes.
- COMMAND : The process command name.

Table 23. Fundamental Keystrokes in top Command

Key	Purpose
H	Help for interactive keystrokes.
L, T, M	Toggles for load, threads, and memory header lines.

Key	Purpose
1	Toggle for individual CPUs or a summary for all CPUs in the header.
S	Change the refresh (screen) rate, in decimal seconds (such as 0.5, 1, 5).
B	Toggle reverse highlighting for Running processes; the default is bold only.
Shift + B	Enables bold use in display, in the header, and for <i>Running</i> processes.
Shift + H	Toggle threads; show process summary or individual threads.
U, Shift + U	Filter for any username (effective, real).
Shift + M	Sort process listing by memory usage, in descending order.
Shift + P	Sort process listing by processor use, in descending order.
K	Kill a process. When prompted, enter PID, and then signal.
R	Renice a process. When prompted, enter PID, and then nice_value.
Shift + W	Write (save) the current display configuration for use at the next top restart.
Q	Quit.
F	Manage the columns by enabling or disabling fields. You can also set the sort field for top.

Note

The `S`, `K`, and `R` keystrokes are not available when the `top` command is started in a secure mode.

References

`ps(1)`, `top(1)`, `uptime(1)`, and `w(1)` man pages

15.8. Guided Exercise

Monitor Process Activity

Investigate, control, and terminate processes that run on a Red Hat Enterprise Linux system.

Outcomes

- Manage processes in real time.

Prerequisites

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
student@workstation:~$ lab start processes-monitor
```

Instructions

1. On the workstation machine, open two terminal windows side by side. In this section, these terminals are referred to as *left* and *right*. In each terminal, log in to the servera machine as the student user.

```
student@workstation:~$ ssh student@servera
...output omitted...
[student@servera ~]$
```

2. In the left terminal shell, create the `/home/student/bin` directory. Create a shell script named `monitor` in the new directory to generate an artificial CPU load. Make the `monitor` script file executable.

- 2.1. Create the `/home/student/bin` directory.

```
[student@servera ~]$ mkdir /home/student/bin
```

- 2.2. Create the `monitor` script file in the `/home/student/bin` directory with the following content.


```
#!/bin/bash
while true; do
    var=1
    while [[ var -lt 60000 ]]; do
        var=$((var+1))
    done
    sleep 1
done
```

The `monitor` script runs until the process is terminated. The script generates an artificial CPU load by performing sixty thousand addition calculations. After generating the CPU load, it then sleeps for one second, resets the variable, and repeats.

2.3. Change the `monitor` file permission to make it executable.

```
[student@servera ~]$ chmod a+x /home/student/bin/monitor
```

3. In the right terminal shell, run the `top` command. Resize the window to view the contents of the command. Keep the `top` utility running to verify CPU usage and load averages in later steps.

```
[student@servera ~]$ top
top - 06:28:29 up 5:16, 3 users, load average: 0.63, 0.47, 0.25
Tasks: 137 total, 1 running, 136 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.2 us, 0.0 sy, 0.0 ni, 99.8 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 1961.2 total, 1469.7 free, 406.2 used, 241.5 buff/cache
MiB Swap: 0.0 total, 0.0 free, 0.0 used. 1555.0 avail Mem

  PID USER      PR  NI   VIRT   RES   SHR S  %CPU  %MEM    TIME+  COMMAND
 16811 student    20   0 232736   5624  3448 R   0.3   0.3   0:00.02 top
     1 root       20   0  48740  40544 10352 S   0.0   2.0   0:07.37 systemd
     2 root       20   0     0     0     0 S   0.0   0.0   0:00.01 kthreadd
...output omitted...
```

4. In the left terminal shell, verify the number of logical CPUs on this virtual machine.

```
[student@servera ~]$ lscpu
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Address sizes:          46 bits physical, 48 bits virtual
Byte Order:             Little Endian
CPU(s):                 2
...output omitted...
```

5. In the left terminal shell, run a single instance of the `monitor` script file in the background.

Note the process ID (PID) for this process. The PID in your system might vary.

```
[student@servera ~]$ monitor &
[1] 16968
```

6. In the right terminal shell, monitor the `top` command.

- 6.1. Use the `T` and `M` single keystrokes to toggle the CPU and memory header lines.

The header of the `top` command shows a graphical bar that displays the CPU and memory usage. Monitor these graphical bars during this activity.

```
top - 06:28:56 up 5:16, 3 users, load average: 0.38, 0.43, 0.24
Tasks: 139 total, 1 running, 138 sleeping, 0 stopped, 0 zombie
%Cpu(s):  8.8/1.2  10[|||||]
MiB Mem : 23.4/1705.2  [|||||]
MiB Swap:  0.0/0.0    [
```

- 6.2. View the CPU percentage for the process, which is expected to hover around 15% to 25%.

```
...output omitted...
PID    USER     PR  NI   VIRT   RES    SHR S  %CPU  %MEM    TIME+  COMMAND
16968  student  20   0  228796  3136   2880 S   19.9   0.2    0:05.33 monitor
...output omitted...
```

- 6.3. Monitor the load average of the `servera` machine. The first value of the load average list, the one-minute load, is currently less than 1. The observed value might be affected by resource contention from another virtual machine or from the virtual host.

```
top - 06:29:26 up 5:17, 3 users, load average: 0.29, 0.40, 0.24
```

7. In the left terminal shell, run a second instance of the `monitor` script file in the background.

Note the PID for this process. The PID in your system might vary.

```
[student@servera ~]$ monitor &
[2] 17157
```

8. In the right terminal shell, monitor the top utility for the second monitor process.

- 8.1. View the CPU percentage for the process, which is also expected to hover between 15% to 25%.

```
top - 06:30:05 up 5:18, 3 users, load average: 0.27, 0.38, 0.24
Tasks: 141 total, 1 running, 140 sleeping, 0 stopped, 0 zombie
%Cpu(s): 13.5/1.7 15[|||||||]
MiB Mem : 23.7/1705.2 [|||||||]
MiB Swap: 0.0/0.0 [
...output omitted...
  PID USER      PR  NI   VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
 16968 student    20   0  228796   3136   2880 S   20.0   0.2   0:21.50 monitor
 17157 student    20   0  228796   3168   2912 S   19.7   0.2   0:04.38 monitor
...output omitted...
```

- 8.2. Again, view the one-minute load average, which remains less than 1. Wait at least one minute for the calculation to adjust to the new workload.

```
top - 06:30:35 up 5:18, 3 users, load average: 0.29, 0.38, 0.24
```

9. In the left terminal shell, run a third instance of the monitor script file in the background.

Note the PID for this process. The PID in your system might vary.

```
[student@servera ~]$ monitor &
[3] 17444
```

10. In the right terminal shell, monitor the top utility for the third monitor process. View the CPU percentage for the process, which is again expected to hover between 15% to 25%.

...output omitted...

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
17444	student	20	0	228796	3112	2856	S	21.6	0.2	0:04.89	monitor
16968	student	20	0	228796	3136	2880	S	20.6	0.2	0:42.47	monitor
17157	student	20	0	228796	3168	2912	R	15.6	0.2	0:25.25	monitor

...output omitted...

- To push the load average above 1, you must start more monitor processes. The classroom setup has two CPUs, so running only three processes is not enough to stress it. In the left terminal shell, start three more monitor processes in the background.

```
[student@servera ~]$ monitor &
[4] 17726
[student@servera ~]$ monitor &
[5] 17736
[student@servera ~]$ monitor &
[6] 17745
```

- In the right terminal shell, view again the one-minute load average, which is now expected to be above 1. Wait at least one minute for the calculation to adjust to the new workload.

Note

The one-minute load average might vary depending on your system. For example, your load average might be above or below 1. This variation depends on factors like CPU allocation, virtualization overhead, and timing. As long as each monitor process shows CPU usage and the load average increases when more processes are started, then your lab environment is functioning correctly.

```
top - 06:33:11 up 5:21, 3 users, load average: 1.22, 0.62, 0.35
Tasks: 147 total, 3 running, 144 sleeping, 0 stopped, 0 zombie
%Cpu(s): 44.7/5.2 50[|||||]
MiB Mem : 23.4/1705.2 [|||||]
MiB Swap: 0.0/0.0 [
...output omitted...
```

- When you are finished observing the load average values, terminate each of the monitor processes from within the top command.

- 13.1.** In the right terminal shell, press **K** to observe the prompt below the headers and above the columns.

```
...output omitted...
PID to signal/kill [default pid = 16968]
```

- 13.2.** The monitor processes at the top of the list is selected. Press **Enter** to kill the process.

```
...output omitted...
Send pid 16968 signal [15/sigterm]
```

- 13.3.** Press **Enter** again to confirm the SIGTERM signal 15.

Verify that the selected process is no longer present in the top command. If the PID exists, then repeat these steps to terminate the processes, and substitute SIGKILL signal 9 when prompted.

```
...output omitted...
17157 student 20 0 228796 3176 2920 R 90.9 0.2 0:40.85 monitor
17444 student 20 0 228796 3140 2884 R 63.6 0.2 0:35.17 monitor
17726 student 20 0 228796 3248 2992 S 0.0 0.2 0:20.48 monitor
17736 student 20 0 228796 3192 2936 S 0.0 0.2 0:21.74 monitor
17745 student 20 0 228796 3136 2880 S 0.0 0.2 0:20.31 monitor
...output omitted...
```

- 14.** Repeat the previous step for each remaining monitor process. Verify that no monitor processes remain in the top command.
- 15.** In the right terminal shell, press **Q** to exit the top command. Close the right terminal.
- 16.** Return to the workstation machine as the student user.

```
[student@servera ~]$ exit
logout
Connection to servera closed.
student@workstation:~$
```

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish processes-monitor
```

15.9. Lab

Monitor and Manage Linux Processes

Interpret and monitor system metrics, and investigate the implications of those measurements to improve your system's performance.

Outcomes

- Manage processes with `top` as a process management tool.

Prerequisites

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
student@workstation:~$ lab start processes-review
```

Instructions

1. On the workstation machine, open two terminal windows side by side. In this section, these terminals are referred to as *left* and *right*. On each terminal window, log in to the `serverb` machine as the student user.

Create the `task101.sh` script in the `/home/student/bin` directory. The `task101.sh` script generates artificial CPU load by performing arithmetic operations endlessly.

```
#!/bin/bash
touch ~/bin/.$(basename $0)
while true; do
    var=1
    while [[ var -lt 50000 ]]; do
        var=$((var+1))
    done
    sleep 1
done
```

2. In the right terminal, monitor all processes that are running in the `serverb` machine.
3. In the left terminal shell, verify the number of logical CPUs on the virtual machine. Run the `task101.sh` script in the background.

4. In the right terminal shell, observe running processes. Find the process ID (PID) of the `task101.sh` process and the amount of CPU that the process consumes.

Examine the consumption of system load, threads, and memory. Make sure to find the system resources that the `task101.sh` process impacts.
5. Copy the `task101.sh` script to a new `task102.sh` file, and increase the artificial CPU load to `100000` in the new script. Start the `task102.sh` process in the background.
6. In the right terminal shell, verify that the `task102.sh` process is running and uses the most CPU resources. Examine the load average of the `serverb` machine.
7. Copy the `task101.sh` script to a new script named `task103.sh`. Increase the addition count to `800000`. Start `task103.sh` in the background.

Monitor the `serverb` machine load average. The load average might take a few minutes to increase.
8. In the left terminal shell, switch to the `root` user and use `redhat` as the password. Suspend the `task101.sh` process. List the remaining jobs. Verify that the `task101.sh` process is now in the `T` state.
9. Resume the `task101.sh` process.
10. Terminate the `task101.sh`, `task102.sh`, and `task103.sh` processes from the command line. After terminating these processes, examine the CPU usage and load average.
11. Stop the `top` utility and return to the workstation machine.

Evaluation

As the `student` user on the workstation machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

```
student@workstation:~$ lab grade processes-review
```

Finish

On the workstation machine, change to the `student` user home directory and use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.


```
student@workstation:~$ lab finish processes-review
```

Solution

Interpret and monitor system metrics, and investigate the implications of those measurements to improve your system's performance.

Outcomes

- Manage processes with `top` as a process management tool.

Prerequisites

As the student user on the workstation machine, run the following `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
student@workstation:~$ lab start processes-review
```

Instructions

1. On the workstation machine, open two terminal windows side by side. In this section, these terminals are referred to as *left* and *right*. On each terminal window, log in to the `serverb` machine as the student user.

Create the `task101.sh` script in the `/home/student/bin` directory. The `task101.sh` script generates artificial CPU load by performing arithmetic operations endlessly.

```
#!/bin/bash
touch ~/bin/.$(basename $0)
while true; do
    var=1
    while [[ var -lt 50000 ]]; do
        var=$((var+1))
    done
    sleep 1
done
```

- 1.1. On the workstation machine, open two terminal windows side by side. In each terminal, log in to the `serverb` machine as the student user.

```
student@workstation:~$ ssh student@serverb
...output omitted...
[student@serverb ~]$
```

1.2. In the left terminal shell, create the `/home/student/bin` directory.

```
[student@serverb ~]$ mkdir /home/student/bin
```

1.3. In the left terminal shell, create the `task101.sh` file in the `~/bin` directory.

The file must contain the following content.

```
#!/bin/bash
touch ~/bin/.$(basename $0)
while true; do
    var=1
    while [[ var -lt 50000 ]]; do
        var=$((var+1))
    done
    sleep 1
done
```

1.4. Make the `task101.sh` script executable.

```
[student@serverb ~]$ chmod +x /home/student/bin/task101.sh
```

2. In the right terminal, monitor all processes that are running in the `serverb` machine.

2.1. In the right terminal, run the `top` utility to monitor all processes. Fit the window to be as tall as possible to view more information.

Keep the `top` utility running to verify CPU usage and load averages in later steps.

```
[student@serverb ~]$ top
top - 20:35:57 up 1:17, 3 users, load average: 0.00, 0.00, 0.00
Tasks: 136 total, 1 running, 135 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 4.5 sy, 0.0 ni, 95.5 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 1705.2 total, 1288.8 free, 380.0 used, 183.4 buff/cache
MiB Swap: 0.0 total, 0.0 free, 0.0 used. 1325.2 avail Mem

  PID USER  PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
    1 root   20   0   48740   40532   10316 S   0.0   2.3   0:02.18 systemd
    2 root   20   0        0        0        0 S   0.0   0.0   0:00.01 kthreadd
    3 root   20   0        0        0        0 S   0.0   0.0   0:00.00
pool_workqueue_+
    4 root    0 -20        0        0        0 I   0.0   0.0   0:00.00 kworker/R-
rcu_gp
    5 root    0 -20        0        0        0 I   0.0   0.0   0:00.00 kworker/R-
sync_wq
    6 root    0 -20        0        0        0 I   0.0   0.0   0:00.00 kworker/R-
slub_+
    7 root    0 -20        0        0        0 I   0.0   0.0   0:00.00 kworker/R-netns
...output omitted...
```

3. In the left terminal shell, verify the number of logical CPUs on the virtual machine. Run the `task101.sh` script in the background.

3.1. Verify the number of logical CPUs.

```
[student@serverb ~]$ grep "model name" /proc/cpuinfo | wc -l
2
```

3.2. Run the `task101.sh` script in the background.

The `task101.sh` script is in the `~/bin` subdirectory. Because of this location, the `PATH` environment variable locates the script without defining its full path.

```
[student@serverb ~]$ task101.sh &
[1] 2608
```

4. In the right terminal shell, observe running processes. Find the process ID (PID) of the `task101.sh` process and the amount of CPU that the process consumes.

Examine the consumption of system load, threads, and memory. Make sure to find the system resources that the `task101.sh` process impacts.

4.1. Examine the output of the `top` utility.

The top utility orders the processes by CPU consumption by default. The task101.sh process is at the top of the list. The PID of the process is 2608. The PID in your system might vary.

Note that the CPU percentage of the task101.sh process uses hovers around 10% to 16%.

```
top - 20:44:19 up 1:25, 3 users, load average: 0.14, 0.09, 0.02
Tasks: 136 total, 1 running, 135 sleeping, 0 stopped, 0 zombie
%Cpu(s): 5.1 us, 0.5 sy, 0.0 ni, 94.0 id, 0.0 wa, 0.2 hi, 0.2 si, 0.0 st
MiB Mem : 1705.2 total, 1285.2 free, 383.5 used, 183.4 buff/cache
MiB Swap: 0.0 total, 0.0 free, 0.0 used. 1321.7 avail Mem

  PID USER      PR  NI   VIRT   RES   SHR S  %CPU  %MEM    TIME+  COMMAND
 2608 student    20   0 228796   3056   2800 S   11.0   0.2   0:35.00 task101.sh
 2568 student    20   0 232152   5164   2988 R    0.3   0.3   0:00.33 top
    1 root       20   0  48740  40532  10316 S    0.0   2.3   0:02.23 systemd
    2 root       20   0      0      0      0 S    0.0   0.0   0:00.01 kthreadd
...output omitted...
```

4.2. Sort the processes by memory usage by pressing Shift + M.

The task101.sh process does not consume many memory resources.

```
top - 21:00:39 up 1:42, 3 users, load average: 0.14, 0.13, 0.09
Tasks: 137 total, 1 running, 136 sleeping, 0 stopped, 0 zombie
%Cpu(s): 7.5 us, 1.0 sy, 0.0 ni, 91.2 id, 0.2 wa, 0.2 hi, 0.0 si, 0.0 st
MiB Mem : 1705.2 total, 1305.1 free, 363.6 used, 183.5 buff/cache
MiB Swap: 0.0 total, 0.0 free, 0.0 used. 1341.6 avail Mem

  PID USER      PR  NI   VIRT   RES   SHR S  %CPU  %MEM    TIME+  COMMAND
  916 root       20   0 350960  42620  16788 S    0.0   2.4   0:01.62 firewallld
    1 root       20   0  48740  40532  10316 S    0.0   2.3   0:02.28 systemd
...output omitted...
 2608 student    20   0 228796   3056   2800 S   16.7   0.2   2:55.59 task101.sh
...output omitted...
```

4.3. Display more memory details by pressing M.

The memory usage bar does not show significant usage in the serverb machine.

```
top - 21:05:56 up 1:47, 3 users, load average: 0.27, 0.17, 0.10
Tasks: 138 total, 4 running, 134 sleeping, 0 stopped, 0 zombie
%Cpu(s): 7.5 us, 1.0 sy, 0.0 ni, 91.2 id, 0.2 wa, 0.2 hi, 0.0 si, 0.0 st
MiB Mem : 21.6/1705.2 [|||||||||||||]
]
MiB Swap: 0.0/0.0 [
]

  PID USER      PR  NI   VIRT    RES    SHR S  %CPU  %MEM    TIME+  COMMAND
  916 root        20   0 350960 42620 16788 S   0.0   2.4   0:01.71 firewallld
    1 root        20   0 48740  40532 10316 S   0.0   2.3   0:02.30 systemd
...output omitted...
2608 student  20   0 228796  3056   2800 S  16.3   0.2   3:41.10 task101.sh
...output omitted...
```

4.4. Display the thread information by pressing **Shift** + **H**. The threads in the system show a total of 157.

Exit the top utility by pressing **Q**.

```
top - 21:23:28 up 2:05, 3 users, load average: 0.09, 0.12, 0.09
Threads: 157 total, 1 running, 156 sleeping, 0 stopped, 0 zombie
%Cpu(s): 7.7 us, 1.0 sy, 0.0 ni, 91.2 id, 0.0 wa, 0.2 hi, 0.0 si, 0.0 st
MiB Mem : 22.4/1705.2 [|||||||||||||]
]
MiB Swap: 0.0/0.0 [
]

  PID USER      PR  NI   VIRT    RES    SHR S  %CPU  %MEM    TIME+  COMMAND
  916 root        20   0 350960 42620 16788 S   0.0   2.4   0:01.89 firewallld
  996 root        20   0 350960 42620 16788 S   0.0   2.4   0:00.00 gmain
...output omitted...
2608 student  20   0 228796  3056   2800 S  11.0   0.2   4:25.37 task101.sh
...output omitted...
```

Q

4.5. View all the threads that are running for the task101.sh process. To do so, run the top utility with the **-H** and the **-p** options. Replace the 2608 process ID with the actual PID on your system.

The task101.sh process is not running multiple threads.

Exit the top utility by pressing **Q**.

```
[student@serverb ~]$ top -H -p 2608
top - 21:28:42 up 2:10, 3 users, load average: 0.19, 0.13, 0.10
Threads: 1 total, 1 running, 0 sleeping, 0 stopped, 0 zombie
%Cpu(s): 6.9 us, 1.1 sy, 0.0 ni, 91.8 id, 0.0 wa, 0.2 hi, 0.0 si, 0.0 st
MiB Mem : 1705.2 total, 1300.1 free, 368.4 used, 184.0 buff/cache
MiB Swap: 0.0 total, 0.0 free, 0.0 used. 1336.9 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
 2608 student    20   0 228796   3132   2876 S   15.7   0.2   6:56.92
task101.sh
```

Q

4.6. Run the top utility with its default options.

```
[student@serverb ~]$ top
top - 21:32:43 up 2:14, 3 users, load average: 0.17, 0.15, 0.10
Tasks: 136 total, 1 running, 135 sleeping, 0 stopped, 0 zombie
%Cpu(s): 7.5 us, 1.0 sy, 0.0 ni, 91.5 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 1705.2 total, 1295.6 free, 372.8 used, 184.0 buff/cache
MiB Swap: 0.0 total, 0.0 free, 0.0 used. 1332.4 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
 2608 student    20   0 228796   3056   2800 S   16.3   0.2   7:31.29 task101.sh
    1 root        20   0  48740   40532  10316 S    0.0   2.3   0:02.44 systemd
    2 root        20   0     0         0        0 S    0.0   0.0   0:00.01 kthreadd
...output omitted...
```

5. Copy the task101.sh script to a new task102.sh file, and increase the artificial CPU load to 100000 in the new script. Start the task102.sh process in the background.

5.1. In the left terminal shell, copy the task101.sh file to a new task102.sh file.

```
[student@serverb ~]$ cp bin/task101.sh bin/task102.sh
```

- 5.2. Edit the task102.sh script and increase the addition calculations from 50000 to 100000.

The task102.sh script much match the following content:

```
#!/bin/bash
touch ~/bin/.$(basename $0)
while true; do
    var=1
    while [[ var -lt 1000000 ]]; do
        var=$((var+1))
    done
    sleep 1
done
```

5.3. Start the `task102.sh` process in the background.

```
[student@serverb ~]$ task102.sh &
[2] 6430
```

5.4. Verify that both processes are running in the background.

```
[student@serverb ~]$ jobs
[1]-  Running                  task101.sh &
[2]+  Running                  task102.sh &
```

6. In the right terminal shell, verify that the `task102.sh` process is running and uses the most CPU resources. Examine the load average of the `serverb` machine.

6.1. In the right terminal shell, verify that the process is running. The CPU usage should hover between 25% to 35%.

Note

If you do not see the `task101.sh` and `task102.sh` processes at the top of the process list, then press `Shift + P` to ensure that the `top` utility sorts the output by CPU usage.


```
top - 21:42:10 up 2:23, 3 users, load average: 0.33, 0.20, 0.11
Tasks: 136 total, 3 running, 133 sleeping, 0 stopped, 0 zombie
%Cpu(s): 19.3 us, 2.0 sy, 0.0 ni, 78.4 id, 0.0 wa, 0.2 hi, 0.2 si, 0.0 st
MiB Mem : 1705.2 total, 1284.4 free, 383.9 used, 184.2 buff/cache
MiB Swap: 0.0 total, 0.0 free, 0.0 used. 1321.3 avail Mem

  PID USER      PR  NI   VIRT   RES   SHR S  %CPU  %MEM    TIME+  COMMAND
 6430 student    20   0 228796  3080  2824 R   30.7   0.2   0:19.73 task102.sh
 2608 student    20   0 228796  3056  2800 R   11.7   0.2   8:53.08 task101.sh
...output omitted...
```

6.2. Examine the load average of the serverb machine.

The load average of the serverb machine stays below 1, which means that the machine is not overloaded.

```
top - 21:44:53 up 2:26, 3 users, load average: 0.51, 0.30, 0.16
...output omitted...
```

7. Copy the task101.sh script to a new script named task103.sh. Increase the addition count to 800000. Start task103.sh in the background.

Monitor the serverb machine load average. The load average might take a few minutes to increase.

- 7.1. In the left terminal shell, copy the task101.sh script to a new task103.sh script.

```
[student@serverb ~]$ cp bin/task101.sh bin/task103.sh
```

- 7.2. Edit the task103.sh script. Increase the addition count to 800000.

The task103.sh script much match the following content:

```
#!/bin/bash
touch ~/bin/.$(basename $0)
while true; do
    var=1
    while [[ var -lt 8000000 ]]; do
        var=$((var+1))
    done
    sleep 1
done
```

7.3. Start the `task103.sh` process in the background.

```
[student@serverb ~]$ task103.sh &
[3] 7581
```

7.4. Verify that the three jobs are running in the background.

```
[student@serverb ~]$ jobs
[1]   Running                  task101.sh &
[2]-  Running                  task102.sh &
[3]+  Running                  task103.sh &
```

7.5. In the right terminal shell, verify the CPU usage of the `task103.sh` process and the load average of the machine. The CPU usage for the `task03.sh` process hovers between 60% to 85%. The load average might take a few minutes to increase.

Note that the sum of the CPU consumption of the three processes is more than 100. With the `task103.sh` process running, the `serverb` machine uses 100% of the CPU, but it is not enough to fulfill the resource requirements of all processes. Because of this situation, the load average slowly increases to above 1.

```
top - 21:51:54 up 2:33, 3 users, load average: 1.13, 0.61, 0.33
...output omitted...
  PID USER      PR  NI   VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
 7581 student    20   0  228796   3148   2892 R   63.1   0.2   1:59.98 task103.sh
 6430 student    20   0  228796   3120   2864 R   27.2   0.2   0:41.86 task102.sh
 2608 student    20   0  228796   3260   3004 S   17.3   0.2   0:36.07 task101.sh
...output omitted...
```

8. In the left terminal shell, switch to the root user and use `redhat` as the password. Suspend the `task101.sh` process. List the remaining jobs. Verify that the `task101.sh` process is now in the T state.

8.1. Switch to the root user. Use redhat as the password.

```
[student@serverb ~]$ su -
Password: redhat
```

8.2. Suspend the task101.sh process.

```
[root@serverb ~]# pkill -SIGSTOP task101.sh
```

8.3. In the right terminal shell, confirm that the task101.sh process is no longer running.

```
top - 21:54:04 up 2:35, 3 users, load average: 1.06, 0.76, 0.42
Tasks: 141 total, 2 running, 137 sleeping, 1 stopped, 1 zombie
%Cpu(s): 40.3 us, 4.1 sy, 0.0 ni, 55.1 id, 0.0 wa, 0.3 hi, 0.2 si, 0.0 st
MiB Mem : 1705.2 total, 1274.5 free, 393.6 used, 184.4 buff/cache
MiB Swap: 0.0 total, 0.0 free, 0.0 used. 1311.6 avail Mem

  PID USER      PR  NI   VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
 7581 student    20   0  228796   3132   2876 R   66.4   0.2   2:25.59 task103.sh
 6430 student    20   0  228796   3080   2824 S   22.3   0.2   3:15.12 task102.sh
...output omitted...
```

8.4. In the left terminal shell, view the remaining jobs.

```
[root@serverb ~]# ps jT
PPID  PID  ... TPGID STAT  UID  TIME COMMAND
2395 2396 ... 8047 Ss   1000 0:00 -bash
2396 2608 ... 8047 T    1000 10:34 /bin/bash /home/student/bin/task101.sh
2396 6430 ... 8047 S    1000 3:25 /bin/bash /home/student/bin/task102.sh
2396 7581 ... 8047 R    1000 2:57 /bin/bash /home/student/bin/task103.sh
...output omitted...
```

Note that task101.sh has a status of T. This status means that the process is currently suspended.

9. Resume the task101.sh process.**9.1.** In the left terminal shell, resume the task101.sh process.

```
[root@serverb ~]# pkill -SIGCONT task101.sh
```

9.2. In the right terminal shell, verify that the process is running again.

```
top - 21:56:13 up 2:37, 3 users, load average: 1.17, 0.88, 0.51
Tasks: 141 total, 2 running, 139 sleeping, 0 stopped, 0 zombie
%Cpu(s): 58.5 us, 5.8 sy, 0.0 ni, 35.2 id, 0.0 wa, 0.5 hi, 0.0 si, 0.0 st
MiB Mem : 1705.2 total, 1278.6 free, 389.5 used, 184.4 buff/cache
MiB Swap: 0.0 total, 0.0 free, 0.0 used. 1315.7 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
7581	student	20	0	228796	3132	2876	R	86.4	0.2	3:59.96	task103.sh
6430	student	20	0	228796	3080	2824	S	24.6	0.2	3:47.45	task102.sh
2608	student	20	0	228796	3056	2800	S	16.6	0.2	10:34.92	task101.sh

...output omitted...

10. Terminate the task101.sh, task102.sh, and task103.sh processes from the command line. After terminating these processes, examine the CPU usage and load average.

10.1. In the left terminal shell, terminate the task101.sh, task102.sh, and task103.sh processes.

```
[root@serverb ~]# pkill task101.sh
[root@serverb ~]# pkill task102.sh
[root@serverb ~]# pkill task103.sh
```

10.2. In the right terminal shell, verify that the processes no longer appear in the top utility output.

The load average slowly decreases.

```
top - 21:57:40 up 2:39, 3 users, load average: 0.79, 0.89, 0.55
Tasks: 136 total, 1 running, 135 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.2 us, 0.0 sy, 0.0 ni, 99.8 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 1705.2 total, 1283.0 free, 385.1 used, 184.4 buff/cache
MiB Swap: 0.0 total, 0.0 free, 0.0 used. 1320.1 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
6120	student	20	0	232164	5324	3148	R	0.3	0.3	0:01.40	top
1	root	20	0	48740	40532	10316	S	0.0	2.3	0:02.56	systemd
2	root	20	0	0	0	0	S	0.0	0.0	0:00.01	kthreadd

...output omitted...

11. Stop the top utility and return to the workstation machine.

11.1. In the left terminal shell, log out of the root user and close the terminal.

```
[root@serverb ~]# exit
logout
[1] Terminated task101.sh
[2] Terminated task102.sh
[3]- Terminated task103.sh
```

11.2. In the right terminal shell, press **Q** to quit the top utility. Return to the workstation system as the student user.

```
[student@serverb ~]$ exit
logout
Connection to serverb closed.
student@workstation:~$
```

Evaluation

As the student user on the workstation machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

```
student@workstation:~$ lab grade processes-review
```

Finish

On the workstation machine, change to the student user home directory and use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish processes-review
```

15.10. Summary

In this lesson, you learned to manage, monitor, and send signals to the processes in the system. A process is a running instance of an executable program. In Linux, processes are assigned to a state, which can be running, sleeping, stopped, or zombie.

By using job control features, you can manage foreground and background processes on the same terminal. You can also control processes with signals, which is a software interrupt that reports events to an executing program.

Finally, you can monitor process activity by understanding the system's load average, which is an estimate of how busy the system is.

Chapter 16.

Controlling Services and Daemons

Goal

Control and monitor the system services and daemons that systemd starts.

Sections

- Identifying System Processes that Start Automatically (and Guided Exercise)
- Controlling System Services (and Guided Exercise)

Lab

- Control Services and Daemons

16.1. Identifying System Processes that Start Automatically

Objectives

- List the system daemons and network services that start at boot time as services, or that start and stop dynamically when a network connection is established.

The systemd Daemon

The `systemd` daemon manages the startup process for Linux, which includes starting services and managing services in general. The `systemd` daemon activates system resources, server daemons, and other processes, both at boot time and on a running system.

Daemons are processes that either wait or run in the background to perform various tasks. By convention, daemon names end with `d`, such as `httpd`, `vsftpd`, `chronyd`, and so on. In the context of `systemd`, a *service* often refers to one or more daemons.

Generally, daemons start automatically at boot time and continue to run until the system shuts down or until you manually stop them. However, starting or stopping a service might instead change the state of the system once, without leaving a running daemon process afterward (a *oneshot* service type).

In Red Hat Enterprise Linux, the first process that starts (PID 1) is the `systemd` daemon, which provides these features:

- Parallelization capabilities (starting multiple services simultaneously), which increase the boot speed of a system.
- On-demand starting of daemons without requiring a separate service.
- Automatic service dependency management, which can prevent long timeouts. For example, a network-dependent service does not try to start until the network is available.
- A method of tracking related processes together by using Linux control groups.

Service Units Description

A `systemd` *unit* defines objects that the system knows how to manage.

Units are represented by configuration files named *unit files*, which encapsulate information about system services, listening sockets, and other relevant objects for the `systemd` init system.

A unit has a name and a unit type. The name provides a unique identity to the unit. The unit type enables grouping units together with other similar unit types.

The `systemd` daemon uses units to manage different types of objects:

Service units

Service units have a `.service` extension and represent system services. You can use service units to start frequently used daemons, such as a web server.

Socket units

Socket units have a `.socket` extension and represent interprocess communication (IPC) sockets that `systemd` should monitor. If a client connects to the socket, then the `systemd` manager starts a daemon and passes the connection to it. You can use socket units to delay the start of a service at boot time and to start on demand the services that are used less often.

Path units

Path units have a `.path` extension and delay the activation of a service until a specific file-system change occurs. You can use path units for services that use spool directories, such as a printing system.

To manage units, use the `systemctl` command. For example, display available unit types with the `systemctl -t help` command. The `systemctl` command can take abbreviated unit names, process tree entries, and unit descriptions.

Listing Service Units

Use the `systemctl` command to explore the system's current state. For example, the following command lists and paginates all currently loaded service units.

```
root@host:~# systemctl list-units --type=service
UNIT                                LOAD    ACTIVE SUB    DESCRIPTION
accounts-daemon.service            loaded active running Accounts Service
atd.service                         loaded active running Deferred execution ...
auditd.service                     loaded active running Security Audit ...
avahi-daemon.service                loaded active running Avahi mDNS/DNS-SD Stack
chronyd.service                     loaded active running NTP client/server
...output omitted...
```

In this example, the `--type=service` option limits the type of `systemd` units to service units. The output has the following columns:

- `UNIT` : The service unit name.

- **LOAD** : Whether the systemd daemon correctly parsed the unit's configuration and loaded the unit into memory.
- **ACTIVE** : The high-level activation state of the unit. This information indicates whether the unit started successfully.
- **SUB** : The low-level activation state of the unit. This information indicates more detailed information about the unit. The information varies based on unit type, state, and how the unit is executed.
- **DESCRIPTION** : The short description of the unit.

By default, the `systemctl list-units --type=service` command lists only the service units with active activation states. The `systemctl list-units --all` option lists all service units regardless of the activation states. Use the `--state=` option to filter by the values in the **LOAD**, **ACTIVE**, or **SUB** fields.

```
root@host:~# systemctl list-units --type=service --all
UNIT                                LOAD    ACTIVE    SUB    DESCRIPTION
accounts-daemon.service            loaded  active    running Accounts Service
alsa-restore.service               loaded  inactive  dead   Save/Restore Sound ...
• apt-daily.service                not-found inactive  dead   apt-daily.service
atd.service                         loaded  active    running Deferred execution ...
audit-rules.service               loaded  inactive  dead   Load Audit Rules
auditd.service                    loaded  active    running Security Audit ...
• auto-cpufreq.service             not-found inactive  dead   auto-cpufreq.service
...output omitted...
```

The `systemctl` command without any arguments lists units that are both loaded and active.

```

root@host:~# systemctl
UNIT                                LOAD    ACTIVE SUB    DESCRIPTION
proc-sys-fs-binfmt_misc.automount  loaded active waiting Arbitrary...
sys-devices-....device             loaded active plugged Virtio network...
sys-subsystem-net-devices-ens3.device loaded active plugged Virtio network...
...output omitted...
-.mount                             loaded active mounted Root Mount
boot.mount                          loaded active mounted /boot
...output omitted...
systemd-ask-password-plymouth.path loaded active waiting Forward
Password...
systemd-ask-password-wall.path      loaded active waiting Forward
Password...
init.scope                          loaded active running System and
Servi...
session-1.scope                     loaded active running Session 1 of...
atd.service                         loaded active running Job spooling tools
auditd.service                      loaded active running Security
Auditing...
chronyd.service                     loaded active running NTP client/server
crond.service                       loaded active running Command Scheduler
...output omitted...

```

The `systemctl` command `list-units` option displays units that the `systemd` service attempts to parse and load into memory. This option does not display services that are installed but not enabled. You can use the `systemctl` command `list-unit-files` option to see the state of all the installed unit files:

```

root@host:~# systemctl list-unit-files --type=service
UNIT FILE                                STATE      VENDOR PRESET
arp-ethers.service                      disabled   disabled
atd.service                             enabled    enabled
auditd.service                          enabled    enabled
auth-rpcgss-module.service              static     -
autovt@.service                         alias      -
blk-availability.service                 disabled   disabled
...output omitted...

```

In the output of the `systemctl list-unit-files` command, some common entries for the `STATE` field are `enabled`, `disabled`, `static`, and `masked`. All `STATE` values are listed in the `systemctl` command manual pages.

Viewing Service States

View a unit's status with the `systemctl status name.type` command. If the unit type is omitted, then the command expects a service unit with that name.

```
root@host:~# systemctl status sshd.service
• sshd.service - OpenSSH server daemon
  Loaded: loaded (/usr/lib/systemd/system/sshd.service; enabled; preset:
enabled)
  Active: active (running) since Thu 2025-05-22 14:47:33 UTC; 2h 48min ago
  Invocation: 215ee4e3b9e2426880cb727cb3933db1
    Docs: man:sshd(8)
          man:sshd_config(5)
 Main PID: 2212 (sshd)
   Tasks: 1 (limit: 35724)
  Memory: 3.2M (peak: 6.4M)
    CPU: 8.984s
  CGroup: /system.slice/sshd.service
          └─2212 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

May 22 17:33:34 workstation sshd-session[5124]: Connection closed by ...
May 22 17:33:46 workstation sshd-session[5125]: Connection closed by ...
May 22 17:33:59 workstation sshd-session[5126]: Connection closed by ...
...output omitted...
```

The following table lists some output fields from the `systemctl` command status option:

Table 24. Service Unit Information

Field	Description
Loaded	Whether the service unit is loaded into memory
Active	Whether the service unit is running, and if so, for how long
Docs	Where to find more information about the service
Main PID	The main process ID of the service, including the command name

Field	Description
Status	More information about the service
Process	More information about related processes
CGroup	More information about related control groups

Not all these fields are always present in the command output.

Keywords in the status output indicate the state of the service. The following table defines keywords for service states.

Table 25. Service States in `systemctl` Output

Keyword	Description
loaded	The unit configuration file is processed.
active (running)	The service is running with continuing processes.
active (exited)	The service successfully completed a one-time configuration.
active (waiting)	The service is running but is waiting for an event.
inactive	The service is not running.
enabled	The service starts at boot time.
disabled	The service is not set to start at boot time.
static	The service cannot be enabled, but an enabled unit might start it automatically.

Verifying Service Status

The `systemctl` command verifies the specific states of a service. For example, use the `systemctl is-active` option to verify whether a service unit is active (running):

```
root@host:~# systemctl is-active sshd.service
active
```

The command returns the service unit state, which is usually `active` or `inactive`.

Run the `systemctl is-enabled` option to verify whether a service unit is enabled to start automatically during system boot:

```
root@host:~# systemctl is-enabled sshd.service
enabled
```

The command returns whether the service unit is enabled to start at boot time, and the state is usually `enabled` or `disabled`.

To verify whether the unit failed during startup, run the `systemctl is-failed` option:

```
root@host:~# systemctl is-failed sshd.service
active
```

The command returns `active` if the service is correctly running, or `failed` if an error occurred during startup. If the unit was stopped, then it returns `unknown` or `inactive`.

To list all the failed units, run the `systemctl --failed --type=service` command.

References

`systemd(1)`, `systemd.unit(5)`, `systemd.service(5)`, `systemd.socket(5)`, and `systemctl(1)` man pages.

For more information, refer to the *Using systemd Unit Files to Customize and Optimize Your System* guide at https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html-single/using_systemd_unit_files_to_customize_and_optimize_your_system

16.2. Guided Exercise

Identify System Processes that Start Automatically

List the installed service units and identify which services are currently enabled and active on a server.

Outcomes

- List the installed service units.
- Identify active and enabled services on the system.

Prerequisites

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
student@workstation:~$ lab start services-identify
```

Instructions

1. List all the installed service units on the servera machine.

- 1.1. As the student user on the workstation machine, log in to the servera machine.

```
student@workstation:~$ ssh student@servera
[student@servera ~]$
```

- 1.2. List all the installed service units.

```
[student@servera ~]$ systemctl list-units --type=service
UNIT                                LOAD    ACTIVE SUB    DESCRIPTION
atd.service                        loaded active running Deferred execution scheduler
auditd.service                    loaded active running Security Audit Logging Service
chronyd.service                   loaded active running NTP client/server
cloud-config.service              loaded active exited Cloud-init: Config Stage
cloud-final.service               loaded active exited Cloud-init: Final Stage
...output omitted...
```

Press **Q** to exit the command.

2. List all socket units, both active and inactive, on the servera machine.

```
[student@servera ~]$ systemctl list-units --type=socket --all
UNIT                                LOAD    ACTIVE SUB    DESCRIPTION
cockpit.socket                     loaded active running Cockpit Web Service Socket
dbus.socket                        loaded active running D-Bus System Message Bus
Socket
dm-event.socket                   loaded active listening Device-mapper event daemon
FIFOs
lvm2-lvmpolld.socket              loaded active listening LVM2 poll daemon socket
...output omitted...

Legend: LOAD    → Reflects whether the unit definition was properly loaded.
        ACTIVE → The high-level unit activation state, i.e. generalization of
SUB.
        SUB    → The low-level unit activation state, values depend on unit
type.

29 loaded units listed.
To show all installed unit files use 'systemctl list-unit-files'.
```

Press **Q** to exit the command.

3. Explore the status of the chronyd service. This service performs network time protocol (NTP) synchronization.

3.1. Display the status of the chronyd service. Note the process ID of any active daemon.


```
[student@servera ~]$ systemctl status chronyd
● chronyd.service - NTP client/server
   Loaded: loaded (/usr/lib/systemd/system/chronyd.service; enabled; ...
   Active: active (running) since Thu 2025-05-22 14:47:06 UTC; 4h 0min ago
  Invocation: c9e1ab4edbd1d426bbc065db07ee1d475
     Docs: man:chronyd(8)
           man:chrony.conf(5)
   Process: 929 ExecStart=/usr/sbin/chronyd $OPTIONS (code=exited, ...
  Main PID: 946 (chronyd)
     Tasks: 1 (limit: 9897)
    Memory: 4.4M (peak: 5.2M)
       CPU: 96ms
    CGroup: /system.slice/chronyd.service
            └─946 /usr/sbin/chronyd -F 2

May 22 14:47:06 servera.lab.example.com chronyd[946]: chronyd version 4.6.1 ...
May 22 14:47:06 servera.lab.example.com chronyd[946]: commandkey directive is
...
May 22 14:47:06 servera.lab.example.com chronyd[946]: generatecommandkey ...
...output omitted...
May 22 14:47:12 servera chronyd[946]: Source 172.25.254.254 online
May 22 14:47:16 servera chronyd[946]: Selected source 172.25.254.254
```

Press **Q** to exit the command.

- 3.2.** Confirm that the chronyd daemon is running by using its process ID. In the preceding command, the output of the process ID that is associated with the chronyd service is 946. The process ID might differ on your system.

```
[student@servera ~]$ ps -p 946
PID TTY          TIME CMD
 946 ?            00:00:00 chronyd
```

- 4.** Explore the status of the sshd service. You can use this service for secure encrypted communication between systems.

- 4.1.** Determine whether the sshd service is enabled to start at system boot.

```
[student@servera ~]$ systemctl is-enabled sshd
enabled
```

- 4.2.** Determine whether the sshd service is active without displaying all the status information.

```
[student@servera ~]$ systemctl is-active sshd
active
```

4.3. Display the status of the sshd service.

```
[student@servera ~]$ systemctl status sshd
• sshd.service - OpenSSH server daemon
  Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; ...
  Active: active (running) since Thu 2025-05-22 14:48:13 UTC; 4h ...
  Invocation: 8dfdf690e8134f4b9e317b91fe84ebda
  Docs: man:ssh(8)
        man:ssh_config(5)
  Main PID: 1403 (sshd)
  Tasks: 1 (limit: 9897)
  Memory: 6.8M (peak: 9.8M)
  CPU: 12.525s
  CGroup: /system.slice/ssh.service
          └─1403 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 ..."

May 22 18:48:24 servera sshd-session[13035]: Connection closed by ...
May 22 18:48:48 servera sshd-session[13074]: Connection closed by ...
May 22 18:49:00 servera sshd-session[13081]: Connection closed by ...
...output omitted...
```

Press **Q** to exit the command.

5. List the enabled or disabled states of all service units.

```
[student@servera ~]$ systemctl list-unit-files --type=service
UNIT FILE                                STATE      PRESET
arp-ethers.service                      disabled   disabled
atd.service                             enabled    enabled
auditd.service                          enabled    enabled
autovt@.service                         alias      -
blk-availability.service                 disabled   disabled
bluetooth.service                       enabled    enabled
capsule@.service                        static     -
chronyd.service                         enabled    enabled
cloud-config.service                    enabled    enabled
...output omitted...
```

Press **Q** to exit the command.

6. Return to the workstation system as the student user.

```
[student@servera ~]$ exit  
logout  
Connection to servera closed.  
student@workstation:~$
```

Finish

On the workstation machine, change to the student user home directory and use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish services-identify
```

16.3. Controlling System Services

Objectives

- Control system daemons and network services with the `systemctl` command.

Starting and Stopping Services

You can manually start, stop, or reload services to update the service, update the configuration file, uninstall the service, or manually manage an infrequently used service.

To verify the status of a service, including whether the service is running or stopped, use the `systemctl status` command:

```
root@host:~# systemctl status sshd.service
• sshd.service - OpenSSH server daemon
  Loaded: loaded (/usr/lib/systemd/system/sshd.service; enabled; preset:
  enabled)
  Active: active (running) since Thu 2025-05-22 14:47:33 UTC; 7h ago
  ...output omitted...
```

To start a service, use the `systemctl start` command as the root user (with the `sudo` command if necessary). If you run the `systemctl start` command with the service name only (without the service type), then the `systemd` service looks for `.service` files.

```
root@host:~# systemctl start sshd
```

To stop a running service, use the `systemctl` command `stop` option. The following example stops the `sshd.service` service:

```
root@host:~# systemctl stop sshd.service
```

Restarting and Reloading Services

When you restart a running service, the service first stops and then starts again. On the service restart, the new process gets a new ID during the startup and thus the process ID changes.

To restart a running service, use the `systemctl` command `restart` option. The following example restarts the `sshd` service:

```
root@host:~# systemctl restart sshd.service
```

Some services can reload their configuration files without requiring a restart, which is called a *service reload*. Reloading a service does not change the process ID that is associated with various service processes. To reload a running service, use the `systemctl reload` option. The following example shows how to reload the `sshd.service` service after configuration changes:

```
root@host:~# systemctl reload sshd.service
```

If you are unsure whether the service has the function to reload the configuration file changes, use the `systemctl reload-or-restart` option. The command reloads the configuration changes if the reloading function is available. Otherwise, the command restarts the service to implement the new configuration changes:

```
root@host:~# systemctl reload-or-restart sshd.service
```

Listing Unit Dependencies

Some services require other services to be running first, which creates dependencies on the other services. Some services start only on demand, rather than at boot time. In both cases, the `systemd` and `systemctl` commands start services as needed, whether to resolve the dependency or to start an infrequently used service. Disabling the service disables the dependencies.

To view a hierarchy mapping of dependencies to start a service unit, use the `systemctl list-dependencies UNIT` command.

The following example lists dependencies for the `sshd.service` service:

```

root@host:~# systemctl list-dependencies sshd.service
sshd.service
○ └─ssh-host-keys-migration.service
● └─system.slice
● └─sshd-keygen.target
○   └─sshd-keygen@ecdsa.service
○   └─sshd-keygen@ed25519.service
○   └─sshd-keygen@rsa.service
● └─sysinit.target
...output omitted...

```

To list reverse dependencies (units that depend on the specified unit), use the `--reverse` option with the command.

This example lists services that depend on the `NetworkManager` service:

```

root@host:~# systemctl list-dependencies --reverse NetworkManager.service
NetworkManager.service
● └─NetworkManager-wait-online.service
● └─multi-user.target
●   └─graphical.target

```

Masking and Unmasking Services

At times, some installed services on your system might conflict with each other. For example, many ways exist to manage mail servers (the `postfix` and `sendmail` services). Masking a service prevents an administrator from accidentally starting a service that conflicts with other services. Masking creates a link in the configuration directories to the `/dev/null` file, which prevents the service from starting.

To mask a service, use the `systemctl mask` command mask option.

```

root@host:~# systemctl mask sendmail.service
Unit sendmail.service does not exist, proceeding anyway.
Created symlink '/etc/systemd/system/sendmail.service' → '/dev/null'.

```

Then, verify the state of the service by using the `systemctl list-unit-files` command:

```
root@host:~# systemctl list-unit-files --type=service
UNIT FILE                                STATE                                PRESET
...output omitted...
sendmail.service                        masked                                disabled
...output omitted...
```

Attempting to start a masked service unit fails with the following output:

```
root@host:~# systemctl start sendmail.service
Failed to start sendmail.service: Unit sendmail.service is masked.
```

Use the `systemctl unmask` command to unmask the service unit.

```
root@host:~# systemctl unmask sendmail
Removed '/etc/systemd/system/sendmail.service'.
```

Important

You, or another unit file, can manually start a disabled service, but it does not start automatically at boot. A masked service does not start manually or automatically.

Enabling Services to Start or Stop at Boot

Starting a service on a running system does not guarantee that the service automatically starts when the system reboots. Similarly, stopping a service on a running system does not prevent it from starting again when the system reboots. Creating links in the `systemd` configuration directories enables the service to start at boot. You can create or remove these links by using the `systemctl` command with the `enable` or `disable` option.

```
root@root:~# systemctl enable sshd.service
Created symlink '/etc/systemd/system/multi-user.target.wants/sshd.service' →
'/usr/lib/systemd/system/sshd.service'.
```

This command creates a symbolic link from the service unit file, usually in the `/usr/lib/systemd/system` directory, to the disk location where the `systemd` command looks for files, in the `/etc/systemd/system/TARGETNAME.target.wants` directory. Enabling a service does not start the service in the current session. To start the service and enable it to start automatically during boot, you can execute both the `systemctl start` and `systemctl enable` commands, or use the equivalent `systemctl enable --now` command.

```
root@root:~# systemctl enable --now sshd.service
Created symlink '/etc/systemd/system/multi-user.target.wants/sshd.service' →
'/usr/lib/systemd/system/sshd.service'.
```

To disable the service from starting automatically, use the `systemctl disable` command, which removes the symbolic link that was created when enabling a service. Disabling a service does not stop the service if it is currently running. To disable and stop a service, you can execute both the `systemctl stop` and `systemctl disable` commands, or use the equivalent `systemctl disable --now` command.

```
root@host:~# systemctl disable --now sshd.service
Removed '/etc/systemd/system/multi-user.target.wants/sshd.service'.
```

To verify whether the service is enabled or disabled, use the `systemctl is-enabled` command.

```
root@host:~# systemctl is-enabled sshd.service
enabled
```

Summary of systemctl Commands

You can start and stop services on a running system, and enable or disable them for an automatic start at boot time.

Table 26. Useful Service Management Commands

Command	Task
<code>systemctl status UNIT</code>	View detailed information about a unit's state.
<code>systemctl stop UNIT</code>	Stop a service on a running system.
<code>systemctl start UNIT</code>	Start a service on a running system.
<code>systemctl restart UNIT</code>	Restart a service on a running system.
<code>systemctl reload UNIT</code>	Reload the configuration file of a running service.

Command	Task
<code>systemctl mask UNIT</code>	Disable a service from being started, both manually and at boot.
<code>systemctl unmask UNIT</code>	Make available a masked service.
<code>systemctl enable UNIT</code>	Configure a service to start at boot time. Use the <code>--now</code> option to also start the service.
<code>systemctl disable UNIT</code>	Disable a service from starting at boot time. Use the <code>--now</code> option to also stop the service.

 References

`systemd(1)`, `systemd.unit(5)`, `systemd.service(5)`, `systemd.socket(5)`, and `systemctl(1)` man pages.

For more information, refer to the *Using systemd Unit Files to Customize and Optimize Your System* guide at https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html-single/using_systemd_unit_files_to_customize_and_optimize_your_system

16.4. Guided Exercise

Control System Services

Control system daemons and network services with the `systemctl` command.

Outcomes

- Use the `systemctl` command to control `systemd` services.

Prerequisites

As the student user on the workstation machine, run the following `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
student@workstation:~$ lab start services-control
```

Instructions

1. Restart and reload the `sshd` service on the `servera` machine, and observe the results.
 - 1.1. As the student user on the workstation machine, log in to the `servera` machine, and switch to the root user. Use `student` as the password.

```
student@workstation:~$ ssh student@servera
...output omitted...
[student@servera ~]$ sudo -i
[sudo] password for student: student
[root@servera ~]#
```

- 1.2. Display the status of the `sshd` service. Note the process ID of the `sshd` daemon.

```
[root@servera ~]# systemctl status sshd
• sshd.service - OpenSSH server daemon
   Loaded: loaded (/usr/lib/systemd/system/sshd.service; enabled; preset:
   enabled)
   Active: active (running) since Thu 2025-05-22 14:48:13 UTC; 7h ago
   Invocation: 8dfdf690e8134f4b9e317b91fe84ebda
   Docs: man:sshd(8)
        man:sshd_config(5)
   Main PID: 1403 (sshd)
     Tasks: 1 (limit: 9897)
    Memory: 6.5M (peak: 9.8M)
...output omitted...
```

Press **Q** to exit the command.

- 1.3.** Restart the sshd service and view the status. In this example, the process ID of the daemon changes from 1403 to 23088.

```
[root@servera ~]# systemctl restart sshd
[root@servera ~]# systemctl status sshd
• sshd.service - OpenSSH server daemon
   Loaded: loaded (/usr/lib/systemd/system/sshd.service; enabled; preset:
   enabled)
   Active: active (running) since Thu 2025-05-22 22:22:57 UTC; 1s ago
   Invocation: 43b0376f99fb4224bdef70bc7a9100f2
   Docs: man:sshd(8)
        man:sshd_config(5)
   Main PID: 23088 (sshd)
     Tasks: 1 (limit: 9897)
    Memory: 1M (peak: 1.2M)
...output omitted...
```

- 1.4.** Reload the sshd service and view the status. The process ID of the daemon does not change.

```
[root@servera ~]# systemctl reload sshd
[root@servera ~]# systemctl status sshd
• sshd.service - OpenSSH server daemon
   Loaded: loaded (/usr/lib/systemd/system/sshd.service; enabled; preset:
   enabled)
   Active: active (running) since Thu 2025-05-22 22:22:57 UTC; 3min 13s ago
   Invocation: 43b0376f99fb4224bdef70bc7a9100f2
   Docs: man:sshd(8)
        man:sshd_config(5)
   Process: 23255 ExecReload=/bin/kill -HUP $MAINPID (code=exited,
   status=0/SUCCESS)
   Main PID: 23088 (sshd)
   Tasks: 1 (limit: 9897)
   Memory: 1M (peak: 2.6M)
   ...output omitted...
```

2. Stop the chronyd service.

2.1. Verify that the chronyd service is running.

```
[root@servera ~]# systemctl status chronyd
• chronyd.service - NTP client/server
   Loaded: loaded (/usr/lib/systemd/system/chronyd.service; enabled; preset:
   enabled)
   Active: active (running) since Thu 2025-05-22 14:47:06 UTC; 7h ago
   ...output omitted...
```

2.2. Stop the chronyd service and view the status.

```
[root@servera ~]# systemctl stop chronyd
[root@servera ~]# systemctl status chronyd
○ chronyd.service - NTP client/server
   Loaded: loaded (/usr/lib/systemd/system/chronyd.service; enabled; preset:
   enabled)
   Active: inactive (dead) since Thu 2025-05-22 22:28:35 UTC; 1s ago
   ...output omitted...
May 22 22:28:35 servera systemd[1]: Stopping chronyd.service - NTP
client/server...
May 22 22:28:35 servera systemd[1]: chronyd.service: Deactivated successfully.
May 22 22:28:35 servera systemd[1]: Stopped chronyd.service - NTP client/server.
```

3. Determine whether the chronyd service is enabled to start at system boot.

```
[root@servera ~]# systemctl is-enabled chronyd
enabled
```

4. Reboot the servera machine, and then view the status of the chronyd service.

4.1. Reboot the servera machine.

```
[root@servera ~]# systemctl reboot
Connection to servera closed by remote host.
Connection to servera closed.
student@workstation:~$
```

- 4.2. Log in as the student user on the servera machine, and switch to the root user. Use student as the password.

```
student@workstation:~$ ssh student@servera
...output omitted...
[student@servera ~]$ sudo -i
[sudo] password for student: student
[root@servera ~]#
```

4.3. Review the status of the chronyd service.

```
[root@servera ~]# systemctl status chronyd
• chronyd.service - NTP client/server
   Loaded: loaded (/usr/lib/systemd/system/chronyd.service; enabled; preset:
   enabled)
   Active: active (running) since Thu 2025-05-22 22:32:16 UTC; 1min 49s ago
   ...output omitted...
```

5. Disable the chronyd service so that it does not start at boot, and then view the status of the service.

```
[root@servera ~]# systemctl disable chronyd
Removed '/etc/systemd/system/multi-user.target.wants/chronyd.service'.
[root@servera ~]# systemctl status chronyd
• chronyd.service - NTP client/server
   Loaded: loaded (/usr/lib/systemd/system/chronyd.service; disabled;
   preset: enabled)
   Active: active (running) since Thu 2025-05-22 22:32:16 UTC; 3min 11s ago
   ...output omitted...
```

6. Reboot the servera machine, and then view the status of the chronyd service.

6.1. Reboot the servera machine.

```
[root@servera ~]# systemctl reboot
Connection to servera closed by remote host.
Connection to servera closed.
student@workstation:~$
```

The machine might take a few minutes to boot.

6.2. Log in as the student user on the servera machine, and review the status of the chronyd service.

```
student@workstation:~$ ssh student@servera
...output omitted...
[student@servera ~]$ systemctl status chronyd
○ chronyd.service - NTP client/server
   Loaded: loaded (/usr/lib/systemd/system/chronyd.service; disabled; preset:
   enabled)
   Active: inactive (dead)
     Docs: man:chronyd(8)
           man:chrony.conf(5)
```

7. Return to the workstation system as the student user.

```
[student@servera ~]$ exit
logout
Connection to servera closed.
student@workstation:~$
```

Finish

On the workstation machine, change to the student user home directory and use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish services-control
```

16.5. Lab

Control Services and Daemons

Control and monitor the system services and daemons that systemd starts.

Outcomes

- Enable, disable, start, and stop services.

Prerequisites

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
student@workstation:~$ lab start services-review
```

Instructions

1. Log in to the serverb machine as the student user, and switch to the root user. On the serverb machine, start the psacct service.
2. Verify the current status of the psacct service. If the psacct service is stopped, then start it.
3. Configure the psacct service to start at system boot.
4. Check the status of the rsyslog service. If the rsyslog service is running, then stop the service and verify that it is no longer active.
5. Configure the rsyslog service so that it does not start at system boot.
6. Reboot the serverb machine so that the boot configuration for the services take effect. Wait for the machine to boot before evaluating this activity.

The machine might take a few minutes to boot.

Evaluation

As the student user on the workstation machine, use the lab command to grade your work. Correct any reported failures and rerun the command until successful.


```
student@workstation:~$ lab grade services-review
```

Finish

On the workstation machine, change to the student user home directory and use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish services-review
```

Solution

Control and monitor the system services and daemons that systemd starts.

Outcomes

- Enable, disable, start, and stop services.

Prerequisites

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
student@workstation:~$ lab start services-review
```

Instructions

1. Log in to the serverb machine as the student user, and switch to the root user. On the serverb machine, start the psacct service.

- 1.1. Log in to the serverb machine as the student user and switch to the root user. Use student as the password.

```
student@workstation:~$ ssh student@serverb
...output omitted...
[student@serverb ~]$ sudo -i
[sudo] password for student: student
[root@serverb ~]#
```

2. Verify the current status of the psacct service. If the psacct service is stopped, then start it.

- 2.1. Verify the status of the psacct service.

```
[root@serverb ~]# systemctl status psacct
○ psacct.service - Kernel process accounting
   Loaded: loaded (/usr/lib/systemd/system/psacct.service; disabled; preset:
disabled)
   Active: inactive (dead)
```

The psacct service is in an inactive and disabled state. The disabled state confirms that the service does not start at boot time.

2.2. Start the psacct service.

```
[root@serverb ~]# systemctl start psacct
```

2.3. Verify that the psacct service is running.

```
[root@serverb ~]# systemctl is-active psacct
active
```

3. Configure the psacct service to start at system boot.**3.1.** Enable the psacct service to start at system boot.

```
[root@serverb ~]# systemctl enable psacct
Created symlink '/etc/systemd/system/multi-user.target.wants/psacct.service' →
'/usr/lib/systemd/system/psacct.service'.
```

3.2. Verify that the psacct service is enabled to start at system boot.

```
[root@serverb ~]# systemctl is-enabled psacct
enabled
```

4. Check the status of the rsyslog service. If the rsyslog service is running, then stop the service and verify that it is no longer active.**4.1.** Verify the status of the rsyslog service.

Notice that the rsyslog service is running and enabled to start at boot time.

```
[root@serverb ~]# systemctl status rsyslog
• rsyslog.service - System Logging Service
   Loaded: loaded (/usr/lib/systemd/system/rsyslog.service; enabled; preset:
   enabled)
   Active: active (running) since Mon 2025-05-26 14:29:48 UTC; 2h 47min ago
   ...output omitted...
```

4.2. Stop the rsyslog service.

```
[root@serverb ~]# systemctl stop rsyslog
```

4.3. Verify that the rsyslog service is stopped.

```
[root@serverb ~]# systemctl is-active rsyslog
inactive
```

5. Configure the `rsyslog` service so that it does not start at system boot.

5.1. Disable the `rsyslog` service so that it does not start at system boot.

```
[root@serverb ~]# systemctl disable rsyslog
Removed '/etc/systemd/system/multi-user.target.wants/rsyslog.service'.
```

5.2. Verify that the `rsyslog` service does not start during the boot process.

```
[root@serverb ~]# systemctl is-enabled rsyslog
disabled
```

6. Reboot the `serverb` machine so that the boot configuration for the services take effect. Wait for the machine to boot before evaluating this activity.

```
[root@serverb ~]# systemctl reboot
Connection to serverb closed by remote host.
Connection to serverb closed.
student@workstation:~$
```

The machine might take a few minutes to boot.

Evaluation

As the student user on the workstation machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

```
student@workstation:~$ lab grade services-review
```

Finish

On the workstation machine, change to the student user home directory and use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish services-review
```

16.6. Summary

In this lesson, you learned that the `systemd` utility provides tools for activating system resources, server daemons, and other processes, both at boot time and on a running system.

For example, the `systemctl` command enables and disables services, and can list the status of all daemons that are running in the system. You can manage each daemon by setting its autostart configuration or by starting, stopping, and restarting the daemon.

The `systemd` utility also manages service units, socket units, and path units. It can mask a service unit so that it does not run, even to satisfy dependencies.

Chapter 17.

Introduction to Networking

Goal

Describe basic TCP/IP networking concepts and investigate the current network configuration and functionality on a server.

Sections

- Networking Concepts (and Quiz)
- Validating Network Configuration (and Guided Exercise)

17.1. Networking Concepts

Objectives

- Describe the fundamental concepts of network addressing and routing for a server.

The TCP/IP Network Model

The TCP/IP network model is a four-layer set of communication protocols that describes how data communications are packetized, addressed, transmitted, routed, and received between computers over a network. The protocol is specified by [RFC 1122: Requirements for Internet Hosts - Communication Layers](#).

The four layers of the TCP/IP network model are: application, transport, internet, and link.

Application

Each application has specifications for communication so that clients and servers can communicate across platforms. Common protocols include SSH (remote access), HTTPS (secure web), FTP (file sharing), and SMTP (electronic mail delivery).

Transport

Transmission Control Protocol (TCP) and User Datagram Protocol (UDP) are the fundamental protocols at the transport layer. TCP provides a reliable, connection-oriented communication method, ensuring data delivery and order. In contrast, UDP is a connectionless datagram protocol, offering faster but less reliable communication.

Application protocols, such as HTTP or DNS, use specific TCP or UDP ports to communicate. A comprehensive list of these well-known and registered ports can be found in the `/etc/services` file. When data is sent across a network, the combination of an IP address and a service port forms a socket. Every network packet includes both a source socket (identifying the sender) and a destination socket (identifying the receiver). Therefore, identifying network socket information is essential for effective traffic monitoring and filtering.

Internet

The internet layer, also known as the *network layer*, is responsible for transmitting data packets from a source host to a destination host across interconnected networks. Key protocols at this layer include IPv4 and IPv6. Each host on a network has a unique IP address and a prefix, which

For use by Krishna Jeevan Upadhyay redhat.krishna@gmail.com redhat.krishna@gmail.com Copyright © 2025 Red Hat, Inc. | For use by Krishna Jeevan Upadhyay redhat.krishna@gmail.com redhat.krishna@gmail.com

together determine its network address (for example, 192.168.3.19/24). Routers play a crucial role at this layer by connecting different networks and forwarding traffic between them.

Link

The link layer, or media access layer, provides the connection to physical media. The most common types of networks are wired Ethernet (802.3) and wireless Wi-Fi (802.11). Each physical device has a *Media Access Control (MAC)* address, which is also known as a hardware address, to identify the destination of packets on the local network segment.

Although TCP/IP is the standard for networking, the Open Systems Interconnection (OSI) model remains a popular teaching reference due to its more granular structure. This granularity is particularly useful for in-depth studies, such as differentiating protocol functionalities (for example, NFS) within TCP/IP's application layer. You might find references to "Layer 2" for Ethernet, "Layer 3" for IP, or "Layer 4" for TCP; these numbers directly correspond to layers in the OSI model.

The following diagram compares the seven-layer OSI model with the four-layer TCP/IP model.

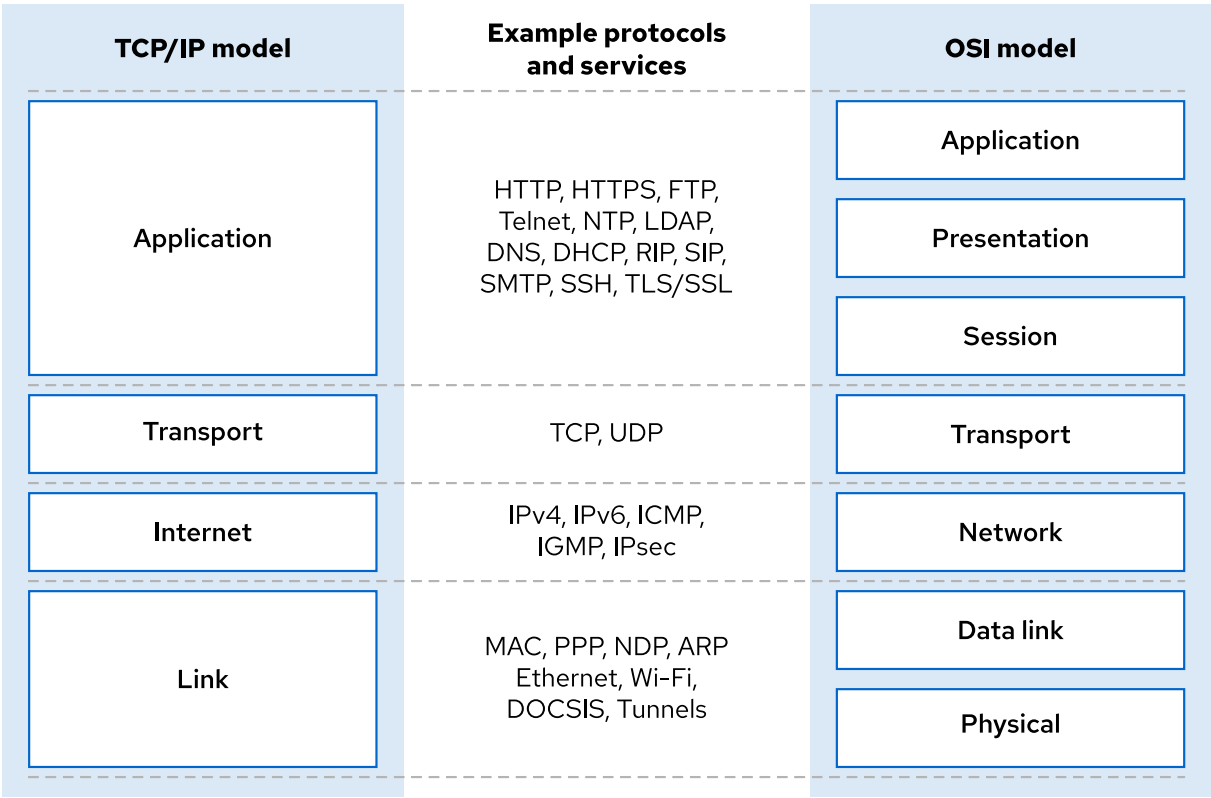


Figure 28. Comparison of the TCP/IP and OSI network models

Network Interface Naming

Each network device in a system is identified and configured by using its unique name. In Red Hat Enterprise Linux 7 and later, the system generates consistent network interface names that persist across reboots. These names are not based on detection order, but instead on information from the firmware, the Peripheral Component Interconnect (PCI) bus topology, and the type of network device.

Network interface names start with the type of interface:

- Ethernet interface names begin with `en`.
- WLAN interface names begin with `wl`.
- WWAN interface names begin with `ww`.

The rest of the interface name is based on information from the server's firmware, or is determined by the location of the device in the PCI topology.

- `oN` indicates an onboard device with unique index *N* from the server's firmware. The `eno1` name is onboard Ethernet device 1.
- `sN` indicates a device in PCI hotplug slot *N*. For example, `ens3` is an Ethernet card in PCI hotplug slot 3.
- `pMsN` indicates a PCI device on bus *M* in slot *N*. For example, `enp2s3`.

A `wlp4s0` interface is a WLAN card on PCI bus 4 in slot 0. If the card is a multifunction device (such as an Ethernet card with multiple ports, or a device with both Ethernet and another function), then you might see `fN` in the device name. An `enp0s1f0` interface is function 0 of the Ethernet card on bus 0 in slot 1. A second card interface would be named `enp0s1f1`, which is function 1 of that same device.

With persistent naming, you can be confident that your network interface names stay the same on a system, even after you add or remove hardware. The name of a device is generated based on unique hardware information, rather than reverting to the less predictable `eth0` naming convention.

Note

Earlier versions of Red Hat Enterprise Linux used names such as `eth0`, `eth1`, and `eth2` for each network interface. The `eth0` interface name was the first network port detected, `eth1` was the second interface, and so on. However, as devices were added and removed, the mechanism that named devices could change which interface was assigned to which name. Furthermore, the Peripheral Component Interconnect Express (PCIe) standard does not guarantee the order in which PCIe devices are detected on boot, which could change device naming unexpectedly due to variations during device or system startup.

IPv4 Networking

System administrators must have a basic understanding of IPv4 networking to manage networking on servers. Although IPv6 has surpassed IPv4 usage in cellular networks, IPv4 remains the most common addressing scheme in enterprise networks.

IPv4 Addresses

An *IPv4 address* is a 32-bit number, which is expressed as four 8-bit octets in a decimal format that ranges in value from 0 to 255, separated by single dots. The address is divided into two parts: the network prefix and the host identifier. The *network prefix* identifies a unique physical or virtual subnet. The *host identifier* represents a specific host on the subnet. All hosts on the same subnet have the same network prefix and can communicate to each other directly without a router. A *network gateway* connects different networks, and a network router commonly operates as the gateway for a subnet.

Note

An IP network is often divided into multiple, smaller network *segments*. Generally, *segment* refers to the physical or virtual link layer, whereas *subnet* refers to the logical IP addressing scheme applied to that segment. *Subnetting* is a technique to subdivide a large network allocation into smaller segments.

A *network mask (netmask)* is a binary mask whose length indicates how many bits belong to the network prefix that identifies the subnet. Because an IPv4 address is always 32 bits long, a subnet with a longer network mask has fewer available bits to identify hosts, which means fewer possible hosts. A subnet with a shorter network mask has more available bits to identify hosts, which means more possible hosts in the subnet.

Network masks are expressed in one of two forms, which are both used routinely. The first form, *CIDR notation*, appends a forward slash (/) and an integer up to 32 that indicates the number of bits in the binary mask. The second notation displays the number of bits in the binary mask as four 8-bit octets in decimal format.

IPv4 Subnets

The number of available host addresses in a subnet depends on the network prefix size. For example, a network prefix of /24 leaves 8 bits, or 255 possible host addresses in the subnet. A network prefix of /16 leaves 16 bits, or 65536 possible host addresses in the subnet.

- The *network address* for a subnet is the lowest possible address on a subnet, where the host identifier is all binary zeros.
- The *broadcast address* for a subnet is the highest possible address on a subnet, where the host identifier is all binary ones, and is a special address for broadcasting packets to all subnet hosts.
- The *gateway address* for a subnet can be any unique host identifier in the subnet, but is commonly set to the first available host address. This gateway numbering convention is not mandatory, and subnets that do not need external communication do not set a network gateway.

The following figures illustrate how to use a netmask to determine the network and host portions of an IP address. A bitwise AND operation is performed for each binary bit in the IP address and the corresponding netmask. If both bits in the compared position are 1, the result is 1; otherwise the result is 0.

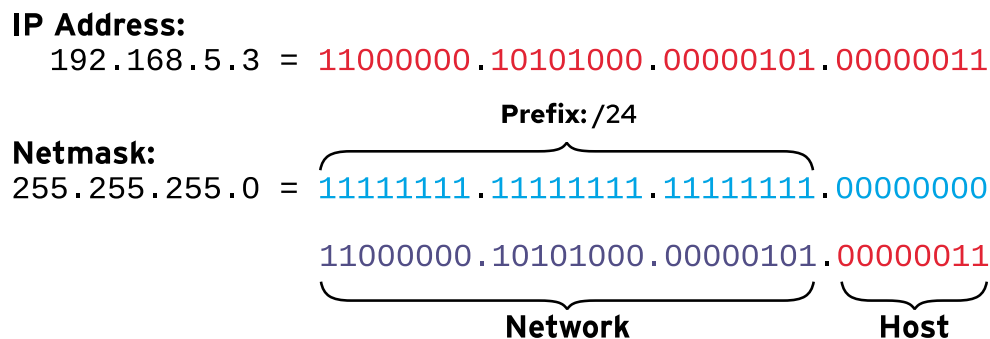


Figure 29. IPv4 netmask calculation for a small network

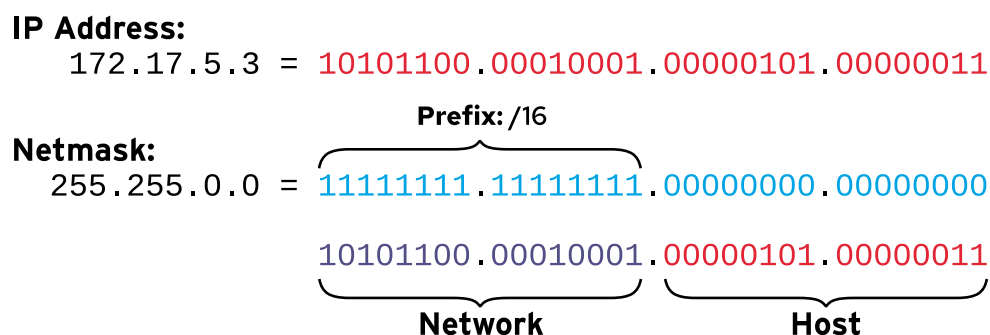


Figure 30. IPv4 netmask calculation for a larger network

Example Network Calculations

In the following example, identify the netmask first, and then perform the binary calculations. A netmask of /24 means that the leading 24 bits of the address define the network address (192.168.1.0). In this scenario, 8 bits, or 254 addresses, are available for host addressing.

Table 27. IPv4 Address of 192.168.1.107/24

Component	Decimal notation	Binary notation
Host address	192.168.1.107	11000000.10101000.00000001.01101011
Netmask	/24 or 255.255.255.0	11111111.11111111.11111111.00000000
Network address	192.168.1.0	11000000.10101000.00000001.00000000
Host range	192.168.1.1 - 192.168.1.254	11000000.10101000.00000001.00000001 to 11000000.10101000.00000001.11111110
Broadcast address	192.168.1.255	11000000.10101000.00000001.11111111

In the following example, a /19 netmask is a valid network prefix that uses only a partial octet. Variable-length netmasks enable the creation of subnets with a more precise number of hosts than is possible with full-octet netmasks. With 19 bits allocated for the network, the remaining 13 bits provide 8,190 usable addresses for individual hosts within that network.

Table 28. IPv4 Address of 172.16.181.23/19

Component	Decimal notation	Binary notation
Host address	172.16.181.23	10101100.00010000.10110101.00010111
Netmask	/19 or 255.255.224.0	11111111.11111111.11100000.00000000
Network address	172.16.160.0	10101100.00010000.10100000.00000000

Component	Decimal notation	Binary notation
Host range	172.16.160.1 - 172.16.191.254	10101100.00010000.10100000.00000001 to 10101100.00010000.10111111.11111110
Broadcast address	172.16.191.255	10101100.00010000.10111111.11111111

In the following example, the /8 netmask indicates a large network. Only the first octet is used for the network prefix (10.0.0.0). The remaining 24 bits, or 16,777,214 addresses, are available for host addressing. The 10.255.255.255 broadcast address is the last address of the network.

Table 29. IPv4 Address of 10.1.1.18/8

Component	Decimal notation	Binary notation
Host address	10.1.1.18	00001010.00000001.00000001.00010010
Netmask	/8 or 255.0.0.0	11111111.00000000.00000000.00000000
Network address	10.0.0.0	00001010.00000000.00000000.00000000
Host range	10.0.0.1 - 10.255.255.254	00001010.00000000.00000000.00000001 to 00001010.11111111.11111111.11111110
Broadcast address	10.255.255.255	00001010.11111111.11111111.11111111

Performing binary calculations manually each time that you configure a network interface is time-consuming and prone to errors. Red Hat Enterprise Linux includes the `ipcalc` command-line tool in its BaseOS repository to help with subnetting tasks.

```

user@host:~$ ipcalc 10.1.1.18/8
Address:      10.1.1.18
Network:      10.0.0.0/8
Netmask:      255.0.0.0 = 8
Broadcast:    10.255.255.255

Address space: Private Use
HostMin:      10.0.0.1
HostMax:      10.255.255.254
Hosts/Net:    16777214

```

IPv4 Routing

Network packets move from host to host on a subnet and through routers from network to network. Each host uses a routing table to determine which network interface to use for sending packets to particular networks.

A routing table entry lists the destination network, which network interface to use, and the IP address of the router that will forward the packet to the final destination. The host uses the routing table entry that matches the network prefix of the destination address to route the packet.

If multiple entries are valid for the destination address, then the host uses the entry with the longer prefix. If the destination network does not match a more specific entry, then the host routes the packet to the default gateway that is indicated by the `0.0.0.0/0` entry in its routing table. This default route must point to a router on a local subnet that the host can reach.

Networks are interconnected by routers, which are specialized devices equipped with at least two interfaces. If the router receives packets that are not addressed to itself, then the router consults its routing table to determine where to forward the packets. This forwarding decision might involve sending the packet directly to the destination host if it resides on the router's directly connected subnet, or it might forward the packet to another router. This iterative process of packet forwarding across multiple routers continues until the packet successfully reaches its intended destination network and host.

For example, consider the following network diagram and host routing table:

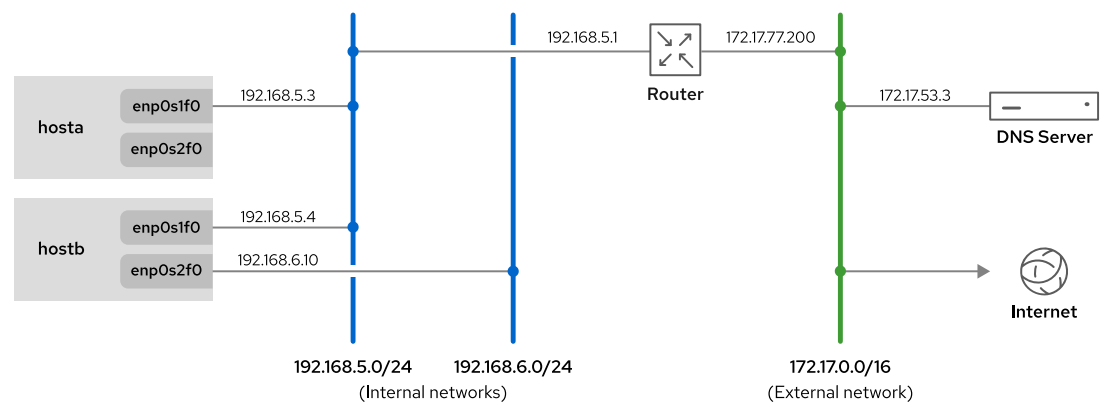


Figure 31. Example network topology

Table 30. Example Routing Table for the hostb Machine

Destination	Interface	Router (if needed)
192.168.5.0/24	enp0s1f0	
192.168.6.0/24	enp0s2f0	
0.0.0.0/0 (default)	enp0s1f0	192.168.5.1

- Network traffic from the hostb machine to any host in the 192.168.6.0/24 network is transmitted directly via the enp0s2f0 interface.
 - The reason for this forwarding decision is that the hostb machine has an interface attached to that network, and therefore a router is not required to reach the destination.
- Network traffic with a destination that does not match any entry in the routing table is sent to the default route. The default route, which is designated with 0.0.0.0/0, is shown in the third entry in the table.
 - For example, all traffic from the hostb machine to the DNS server with the 172.17.53.3 IP address is forwarded to the router with the address of 192.168.5.1, which is reachable via the enp0s1f0 interface. The router can forward the packet directly to its destination.

IPv4 Address Configuration

A host can automatically configure its IPv4 network settings by interacting with a Dynamic Host Configuration Protocol (DHCP) server. The DHCP client broadcasts a discovery request on its local

network segment to locate an available DHCP server. Upon receiving an offer from the DHCP server, the client requests and is assigned a unique IPv4 address, along with other network parameters like the subnet mask, default gateway, and DNS server addresses. This assignment is valid for a defined lease duration. The client must periodically request a lease renewal to maintain continuous use of the assigned network configuration.

Alternatively, you can configure a server to use a static network configuration. Static network settings are read from local configuration files. The settings that you use must be appropriate for your subnet. Coordinate with your network administrator to avoid conflicts with other hosts in the same subnets.

IPv6 Networking

IPv6 is widely deployed in both enterprise networks and for mobile communications. Many Internet Service Providers (ISPs) use IPv6 extensively for their core internal infrastructure and for dynamically assigning addresses to customer devices. Designed to overcome the address exhaustion of IPv4, IPv6 dramatically increases the pool of unique device addresses that are globally available.

IPv6 can also be used in parallel with IPv4 in a dual-stack mode. A network interface can have both IPv6 and IPv4 addresses. Red Hat Enterprise Linux operates in a dual-stack mode by default.

IPv6 Addresses

An IPv6 address is a 128-bit number, typically shown as eight colon-separated groups of four hexadecimal digits. Each hexadecimal digit (a nibble) represents four bits, which means that a quartet represents 16 bits of the IPv6 address.

```
2001:0db8:0000:0010:0000:0000:0000:0001
```

For convenience in writing IPv6 addresses, any leading zeros in a colon-separated quartet can be omitted. However, each quartet must retain at least one hexadecimal digit.

```
2001:db8:0:10:0:0:0:1
```

To further simplify IPv6 addresses, one or more consecutive groups of `0000` can be replaced by a double colon (`::`). This double colon can appear only one time in an address to avoid ambiguity.

```
2001:db8:0:10::1
```

Although `2001:0db8:0:0010:0:0000:0:1` is also a valid IPv6 address, its notation makes it less practical and harder to read.

Follow these tips to write consistently readable addresses:

- Suppress any leading zeros in a group. For example, `0010` becomes `10`, and `0000` becomes `0`.
- Use the double colon (`::`) to shorten the address as much as possible.
- If an address contains two consecutive groups of zeros, which are equal in length, then shorten the leftmost groups of zeros to `::` and the rightmost groups to `:0:` for each group.
- Although it is allowed, do not use `::` to shorten a single group of zeros. Use `:0:` instead, and save `::` for consecutive groups of zeros.
- Always use lowercase letters (a through f) for hexadecimal digits.

Important

When including a TCP or UDP network port after an IPv6 address, always enclose the IPv6 address in square brackets so that the port does not appear to be part of the address.

```
[2001:db8:0:10::1]:80
```

IPv6 Subnets

A standard IPv6 unicast address is divided into two parts: the *network prefix* and *interface ID*. The network prefix identifies the subnet. The interface ID identifies a particular interface on the subnet. Two network interfaces on the same subnet cannot have the same interface ID.

Unlike IPv4, IPv6 employs a standardized subnet mask: most typical unicast addresses use a /64 prefix length. This means that half of the 128 bit address represents the network prefix and the other half the interface ID. With 64 bits available for host identification, a single IPv6 subnet could theoretically accommodate an enormous number of devices (up to 2^{64} hosts).

Typically, an Internet Service Provider (ISP) allocates a shorter network prefix to an organization, commonly /48. This organizational prefix leaves the remaining bits within the network portion for the organization to define its own subnets. For instance, with a /48 allocation, 16 bits are available for subnetting ($64-48=16$), allowing for 2^{16} or 65,536 distinct subnets within that allocated block.

IPv6 address is **2001:db8:0:1::1/64**
Allocation from provider is **2001:db8::/48**

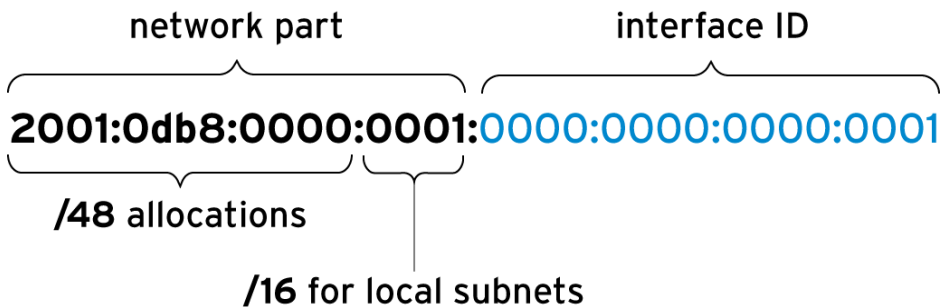


Figure 32. IPv6 address parts and subnetting

A number of addresses have special meaning in IPv6. The following table lists these special addresses and well-known IPv6 prefixes.

Table 31. Common IPv6 Addresses and Networks

IPv6 address or network	Purpose	Description
::1/128	localhost	The IPv6 equivalent to the 127.0.0.1/8 address, which is set on the loopback interface.
::	The unspecified address	The IPv6 equivalent to 0.0.0.0. For a network service, it might indicate that it is listening on all configured IP addresses.
::/0	The default route (the IPv6 internet)	The IPv6 equivalent to the 0.0.0.0/0 address. The default route in the routing table matches this network; the router for this network is where all traffic is sent in the absence of a better route.
2000::/3	Global unicast addresses	The <i>Internet Assigned Numbers Authority (IANA)</i> currently allocates public, routable IPv6 addresses from this space. The addresses include all the networks that range from 2000::/16 through 3fff::/16.

IPv6 address or network	Purpose	Description
fd00::/8	Unique local addresses (RFC 4193)	IPv6 has no direct equivalent of the RFC 1918 private address space, although this network range is close. A site can use these networks to self-allocate a private routable IP address space inside the organization. However, these networks cannot be used on the global internet. The site must randomly select a /48 from this space, but it can subnet the allocation into /64 networks normally.
fe80::/10	Link-local addresses	Every IPv6 interface automatically configures a link-local unicast address that cannot be routed to public networks.
ff00::/8	Multicast address	The IPv6 equivalent to the 224.0.0.0/4 address. Multicast sends packets to multiple hosts simultaneously. This one-to-many functionality is particularly important, because IPv6 does not use broadcast addresses.

Important

The previous table lists network address *allocations* that are reserved for specific purposes. These allocations might consist of many networks. IPv6 networks are allocated from the global unicast and link-local unicast address spaces, which have a standard /64 network mask.

A link-local address in IPv6 is an unroutable address that the system uses only to communicate to other systems on the same network link. To ensure that the IP address is unique, the system uses a specific method to compute the interface ID of the link-local address.

 Note

Originally, the interface ID for the IPv6 link-local address was constructed from the MAC address of the network device. Exposing the MAC address as part of the IPv6 address might cause some security and privacy issues, because it becomes possible to identify and track a computer on the network.

By default, NetworkManager generates a random but stable interface ID for the interface, according to the algorithm in RFC 7217. This algorithm is controlled by the `ipv6.addr-gen-mode` connection setting, which defaults to `stable-privacy`.

IPv6 Privacy Extensions (RFC 4941) are a different solution to the same concern and are controlled by different settings, which are disabled by default.

To operate correctly, IPv6 relies on the link-local address. The interface always keeps that address, regardless of whether you assign a routable IPv6 address manually or with an automated method.

With multicast, one system can send traffic to a special IP address that multiple systems receive. Multicast differs from broadcast, because broadcast packets are not routable and reach only local subnet hosts. Conversely, multicast packets are routed to specific hosts that announced a request for the uniquely addressed multicast packets. Multicast packets can be routed to other subnets, if all intermediary routers support forwarding multicast requests and routing.

Multicast plays a larger role in IPv6 than in IPv4, because IPv6 has no broadcast address. The `ff02::1` IPv6 address is a key multicast address that is used as the `all-nodes` link-local address, and behaves like a broadcast address. You can ping this address to send traffic to all nodes on the link.

IPv6 Address Configuration

IPv4 offers two methods for configuring addresses on network interfaces: manual assignment by an administrator or dynamic configuration via DHCP. In contrast, IPv6 supports manual configuration along with two dynamic methods: DHCPv6 and SLAAC (Stateless Address Autoconfiguration).

Manual Configuration

Similar to IPv4, you can select interface IDs for static IPv6 addresses, and configure them manually. Be aware that some addresses are reserved and must not be used. In IPv4, two addresses on a network cannot be used:

- The lowest address, which is the network address.
- The highest address, which is the broadcast address.

In IPv6, two interface IDs are reserved, and cannot be used as regular host interface addresses:

- The all-zeros identifier `0000:0000:0000:0000` (*subnet router anycast*) that all routers on the link use. For example, on the `2001:db8::/64` network, the anycast address is `2001:db8::`.
- The identifiers `fdff:ffff:ffff:ff80` through `fdff:ffff:ffff:ffff`.

Automatic Configuration with DHCPv6

Unlike IPv4 DHCP, DHCPv6 lease negotiations operate without the use of broadcast addresses. Instead, a host sends its DHCPv6 requests from a link-local address to the special `ff02::1:2` multicast address on UDP port 547. This `ff02::1:2` address is a reserved, link-scope multicast address that is specifically designated for all DHCPv6 servers and relay agents.

When a DHCPv6 server processes a client's request, it sends a reply with the necessary configuration information to UDP port 546 on the client's link-local address.

The `kea` package in Red Hat Enterprise Linux 10 provides support for a DHCPv6 server.

Automatic Configuration with SLAAC

In addition to DHCPv6, IPv6 also supports another dynamic configuration method: *Stateless Address Autoconfiguration (SLAAC)*. To use SLAAC, a host configures its interface with a link-local `fe80::/64` address automatically, and sends a *router solicitation* message to the `ff02::2` link-local multicast address. This address is designated for all routers on the local network segment.

An IPv6 router on the local link responds to the host's link-local address and provides the subnet's pre-configured network prefix and other relevant information. The host subsequently uses this provided network prefix along with an automatically generated interface ID to create its complete IPv6 address. To ensure network settings remain current, the router regularly sends multicast *router advertisements* to update or confirm the provided information.

The `radvd` package in RHEL provides the Router Advertisement Daemon, which implements SLAAC.

 Important

A Red Hat Enterprise Linux 10 system that is configured for dynamic IPv4 addresses with DHCP is typically configured for dynamic IPv6 by using SLAAC. Hosts with a dynamic IPv6 configuration might unexpectedly obtain additional IPv6 addresses when a new IPv6 router is added to the network.

Some IPv6 deployments combine SLAAC and DHCPv6. These deployments use SLAAC to provide the network address information, and use DHCPv6 to provide more network options, such as DNS servers and search domains.

Hostnames and Name Resolution

IP addresses are not human-friendly for daily use; people generally prefer hostnames over numeric strings. Linux provides *name resolution* mechanisms to map these hostnames to their corresponding IP addresses.

A straightforward method for name resolution is to create static entries for hostnames in each system's `/etc/hosts` file. This method comes with a significant drawback: you must manually update the `hosts` file on every system.

A more robust solution for handling name resolution is the *Domain Name System (DNS)*. Unlike static host files, DNS operates as a distributed network of servers that dynamically maps hostnames and IP addresses. To function, a host must be configured to contact a nameserver, which can reside in a different subnet as long as it is network-accessible. Nameserver settings are typically provisioned through DHCP or by specifying static addresses in the `/etc/resolv.conf` file.

References

`services(5)`, `ping(8)`, `biosdevname(1)`, `ipcalc(1)`, and `systemd.link(5)` man pages

For more information, refer to the *Configuring and Managing Networking* guide at https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html-single/configuring_and_managing_networking/index

[RFC 1122: Requirements for Internet Hosts - Communication Layers](#)

[RFC 1918: Address Allocation for Private Internets](#)

[RFC 4193: Unique Local IPv6 Unicast Addresses](#)

[RFC 4941 Privacy Extensions for Stateless Address Autoconfiguration in IPv6](#)

[RFC 7217: A Method for Generating Semantically Opaque Interface Identifiers with IPv6 Stateless Address Autoconfiguration \(SLAAC\)](#)

17.2. Quiz

Networking Concepts

Choose the correct answer to the following questions:

1. What is the size, in bits, of an IPv4 address?

- ☐ a. 4
- ☐ b. 8
- ☐ c. 16
- ☐ d. 32
- ☐ e. 64
- ☐ f. 128

2. Which term determines how many leading bits in the IP address contribute to its network address?

- ☐ a. netscope
- ☐ b. netmask
- ☐ c. subnet
- ☐ d. multicast
- ☐ e. netaddr
- ☐ f. network

3. Which address represents a valid IPv4 *host* address on the 192.168.1.0/24 network?

- ☐ a. 192.168.1.188
- ☐ b. 192.168.1.0
- ☐ c. 192.168.1.255
- ☐ d. 192.168.1.256

4. Which number is the size, in bits, of an IPv6 address?

- ☐ a. 4
- ☐ b. 8
- ☐ c. 16
- ☐ d. 32
- ☐ e. 64
- ☐ f. 128

5. Which address does not represent a valid IPv6 address?

- ☐ a. 2000:0000:0000:0000:0000:0000:0000:0001
- ☐ b. 2::1
- ☐ c. ::
- ☐ d. ff02::1:0:0
- ☐ e. 2001:3::7:0:2
- ☐ f. 2001:db8::7::2
- ☐ g. 2000::1

6. Which term refers to the ability of one system to send traffic to a special IP address that multiple systems receive?

- ☐ a. netscope
- ☐ b. netmask
- ☐ c. subnet
- ☐ d. multicast
- ☐ e. netaddr
- ☐ f. network

Solution

1. What is the size, in bits, of an IPv4 address?

- ☐ a. 4
- ☐ b. 8
- ☐ c. 16
- ☒ d. 32
- ☐ e. 64
- ☐ f. 128

2. Which term determines how many leading bits in the IP address contribute to its network address?

- ☐ a. netscope
- ☒ b. netmask
- ☐ c. subnet
- ☐ d. multicast
- ☐ e. netaddr
- ☐ f. network

3. Which address represents a valid IPv4 *host* address on the 192.168.1.0/24 network?

- ☒ a. 192.168.1.188
- ☐ b. 192.168.1.0
- ☐ c. 192.168.1.255
- ☐ d. 192.168.1.256

4. Which number is the size, in bits, of an IPv6 address?

- ☐ a. 4
- ☐ b. 8
- ☐ c. 16
- ☐ d. 32
- ☐ e. 64
- ☒ f. 128

5. Which address does not represent a valid IPv6 address?

- ☐ a. 2000:0000:0000:0000:0000:0000:0000:0001
- ☐ b. 2::1
- ☐ c. ::
- ☐ d. ff02::1:0:0
- ☐ e. 2001:3::7:0:2
- ☒ f. 2001:db8::7::2
- ☐ g. 2000::1

6. Which term refers to the ability of one system to send traffic to a special IP address that multiple systems receive?

- ☐ a. netscope
- ☐ b. netmask
- ☐ c. subnet
- ☒ d. multicast
- ☐ e. netaddr
- ☐ f. network

17.3. Validating Network Configuration

Objectives

- Test and inspect the current network configuration by using command-line utilities.

Gathering Network Information

Linux offers robust tools for gathering network information and managing components. Red Hat recommends the versatile `ip` command, which is widely available and provides comprehensive details about network interfaces, including statistics, addresses, and routes.

Identifying Network Devices

The `ip link` command lists all available network interfaces on your system. In the following example, the server has three network interfaces: `lo`, which is the loopback virtual interface that allows a device to communicate with itself, and two Ethernet interfaces, `ens3` and `ens4`.

```
user@host:~$ ip link show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN ...
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: ens3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP ...
   link/ether 52:54:00:00:fa:0a brd ff:ff:ff:ff:ff:ff
   altname enp0s3
   altname enx52540000fa0a
3: ens4: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 8942 qdisc fq_codel state UP ...
   link/ether 52:54:00:01:fa:1e brd ff:ff:ff:ff:ff:ff
   altname enp0s4
   altname enx52540001fa1e
```

When configuring network interfaces, it is essential to identify which interface connects to which network. The MAC address is a vital identifier for this purpose. In the previous command output, the MAC address is shown in the line starting with the `link/ether` flag.

Displaying IP Addresses

The `ip` command displays both device and IP address information. A single network interface can have multiple IPv4 or IPv6 addresses.

```

user@host:~$ ip addr show ens3
2: ens3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP ...
(1) link/ether 52:54:00:00:fa:0a brd ff:ff:ff:ff:ff:ff (2)
    altname enp0s3 (3)
    altname enx52540000fa0a
    inet 172.25.250.10/24 brd 172.25.250.255 scope global noprefixroute ens3 (4)
        valid_lft forever preferred_lft forever
    inet6 2001:db8:0:1:5054:ff:fe00:b/64 scope global (5)
        valid_lft forever preferred_lft forever
    inet6 fe80::5054:ff:fe00:fa0a/64 scope link noprefixroute (6)
        valid_lft forever preferred_lft forever

```

- (1) An active interface is indicated by the state `UP` flag.
- (2) The `link/ether` string specifies the hardware (MAC) address of the device.
- (3) The `altname` fields display alternative names that refer to the same device. In this example, the device is defined with two aliases: `enp0s3` and `enx52540000fa0a`.
- (4) The `inet` string shows an IPv4 address, its network prefix length, and scope.
- (5) The `inet6` string shows an IPv6 address, its network prefix length, and scope. A *global* scope indicates unlimited scope, which means that an address with a global scope is routable on the internet.
- (6) This `inet6` string shows that the interface has an IPv6 address of *link* scope. An address with a link-local scope can only be used to reach neighboring hosts that are attached to the same link. This address is automatically generated and assigned to the interface.

Displaying Performance Statistics

In addition to configuration, the `ip` command also provides valuable network performance statistics. Its network interface counters can help to detect network issues, as they record statistics including received (RX) and transmitted (TX) packet counts, packet errors, and dropped packets.

```

user@host:~$ ip -s link show ens3
2: ens3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP ...
    link/ether 52:54:00:00:fa:0a brd ff:ff:ff:ff:ff:ff
    RX:  bytes packets errors dropped missed  mcast
         10398126   75186      0      3      0      0
    TX:  bytes packets errors dropped carrier collsns
         22214542   68283      0      0      0      0
    altname enp0s3
    altname enx52540000fa0a

```

Verifying Connectivity Between Hosts

The `ping` command is a fundamental tool for verifying IPv4 connectivity. The command runs continuously until the `Ctrl + C` key combination is pressed, unless options are specified to limit the number of packets sent.

```
user@host:~$ ping -c 3 192.0.2.254
PING 192.0.2.1 (192.0.2.254) 56(84) bytes of data.
64 bytes from 192.0.2.254: icmp_seq=1 ttl=64 time=4.33 ms
64 bytes from 192.0.2.254: icmp_seq=2 ttl=64 time=3.48 ms
64 bytes from 192.0.2.254: icmp_seq=3 ttl=64 time=6.83 ms

--- 192.0.2.254 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 3.485/4.885/6.837/1.424 ms
```

The `ping6` command is the IPv6 equivalent of the `ping` command. It communicates over IPv6 and requires an IPv6 address as its argument.

```
user@host:~$ ping6 2001:db8:0:1::1
PING 2001:db8:0:1::1(2001:db8:0:1::1) 56 data bytes
64 bytes from 2001:db8:0:1::1: icmp_seq=1 ttl=64 time=18.4 ms
64 bytes from 2001:db8:0:1::1: icmp_seq=2 ttl=64 time=0.178 ms
64 bytes from 2001:db8:0:1::1: icmp_seq=3 ttl=64 time=0.180 ms
^C
--- 2001:db8:0:1::1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2001ms
rtt min/avg/max/mdev = 0.178/6.272/18.458/8.616 ms
```

When pinging IPv6 link-local addresses or the `ff02::1` all-nodes multicast group, you must explicitly specify the network interface using a scope identifier (for example, `ff02::1%ens3`). Failing to do so results in packet loss and the command displays a warning about the missing scope identifier.

```
user@host:~$ ping6 -c 1 fe80::f482:dbff:fe25:6a9f%ens4
PING fe80::f482:dbff:fe25:6a9f%ens4(fe80::f482:dbff:fe25:6a9f) 56 data bytes
64 bytes from fe80::f482:dbff:fe25:6a9f: icmp_seq=1 ttl=64 time=22.9 ms

--- fe80::f482:dbff:fe25:6a9f%ens4 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 22.903/22.903/22.903/0.000 ms
```

You can also access services such as SSH by using link-local addresses on the local network.

```

user@host:~$ ssh fe80::f482:dbff:fe25:6a9f%ens4
user@fe80::f482:dbff:fe25:6a9f%ens4's password:
Last login: Thu Jun  5 15:20:10 2014 from host.example.com
[user@server ~]$

```

The `ping6 ff02::1` command is useful for discovering other IPv6 hosts on the local network. This is a notable link-scope multicast address that corresponds to all nodes on the local network segment.

```

user@host:~$ ping6 ff02::1%ens4
PING ff02::1%ens4(ff02::1) 56 data bytes
64 bytes from fe80::78cf:7fff:fed2:f97b: icmp_seq=1 ttl=64 time=22.7 ms
64 bytes from fe80::f482:dbff:fe25:6a9f: icmp_seq=1 ttl=64 time=30.1 ms (DUP!)
64 bytes from fe80::78cf:7fff:fed2:f97b: icmp_seq=2 ttl=64 time=0.183 ms
64 bytes from fe80::f482:dbff:fe25:6a9f: icmp_seq=2 ttl=64 time=0.231 ms (DUP!)
^C
--- ff02::1%ens4 ping statistics ---
2 packets transmitted, 2 received, +2 duplicates, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 0.183/13.320/30.158/13.374 ms

```

Troubleshooting Routing Issues

Network routing is complex, and sometimes traffic does not behave as you might expect. You can use a range of diagnostic tools to effectively troubleshoot router-related problems.

Displaying the Routing Table

To view your host's routing table, use the `ip route` command.

```

user@host:~$ ip route
default via 192.0.2.254 dev ens3 proto static metric 1024 ①
192.0.2.0/24 dev ens3 proto kernel scope link src 192.0.2.2 ②
10.0.0.0/8 dev ens4 proto kernel scope link src 10.0.0.11 ③

```

- ① All packets that are not destined for the `192.0.2.0/24` or `10.0.0.0/8` networks are sent to the default router at `192.0.2.254`, by using the `ens3` interface.
- ② All packets that are destined for the `192.0.2.0/24` network are sent directly to the destination through the `ens3` device.
- ③ All packets that are destined for the `10.0.0.0/8` network are sent directly to the destination through the `ens4` device.

To display the IPv6 routing table, use the `ip -6 route` command.

```
user@host:~$ ip -6 route
2001:db8:0:1::/64 dev ens3 proto kernel metric 101 pref medium ①
fe80::/64 dev ens3 proto kernel metric 1024 pref medium ②
fe80::/64 dev ens4 proto kernel metric 1024 pref medium
default via 2001:db8:0:1::ffff dev ens3 proto static metric 101 pref medium ③
```

- ① The ens3 interface is connected to the 2001:db8:0:1::/64 network.
- ② On systems with multiple network interfaces, a route to the fe80::/64 network is present for each interface that has a link-local address.
- ③ The default IPv6 route (::/0) uses the gateway at 2001:db8:0:1::ffff and is reachable via the ens3 device.

Tracing Traffic Routes

To trace the network path to a remote host through multiple routers, use either the `tracert` or `tracert` command. These tools are valuable for identifying issues with your own routers or intermediate routers.

Both commands use UDP packets for tracing by default; however, many networks block UDP and ICMP traffic. The `tracert` command has options to trace the path with UDP (default), ICMP (-I), or TCP (-T) packets. Keep in mind that `tracert` is typically not installed by default.

```
user@host:~$ tracert access.redhat.com
...output omitted...
 4:  71-32-28-145.rcmt.qwest.net                48.853ms  asymm  5
 5:  dcp-brdr-04.inet.qwest.net                  100.732ms  asymm  7
 6:  206.111.0.153.ptr.us.xo.net                  96.245ms  asymm  7
 7:  207.88.14.162.ptr.us.xo.net                   85.270ms  asymm  8
 8:  ae1d0.cir1.atlanta6-ga.us.xo.net              64.160ms  asymm  7
 9:  216.156.108.98.ptr.us.xo.net                 108.652ms
10:  bu-ether13.atlmgamq46w-bcr00.tbone.rr.com    107.286ms  asymm 12
...output omitted...
```

Each line in the output of the `tracert` command represents a router or hop that the packet passes through between the source and the final destination. The command outputs information for each hop as it becomes available, including the *round trip timing (RTT)* and any changes in the *maximum transmission unit (MTU)* size. The `asymm` flag means that the traffic that reached the router returned from that router by different (*asymmetric*) routes. These routers are for outbound traffic, not for return traffic.

The `tracepath6` and `traceroute -6` commands are the IPv6 equivalents of the `tracepath` and `traceroute` commands, respectively.

```
user@host:~$ tracepath6 2001:db8:0:2::451
1?: [LOCALHOST] 0.091ms pmtu 1500
1: 2001:db8:0:1::ba 0.214ms
2: 2001:db8:0:1::1 0.512ms
3: 2001:db8:0:2::451 0.559ms reached
Resume: pmtu 1500 hops 3 back 3
```

The interactive `mtr` command provides another way to trace the network path to a destination host. It can run continuously if no options are specified, or operate in report mode when the `-r` option is used. When in report mode, you must define the number of packets to send with the `-c` option.

```
user@host:~$ mtr -r -c 5 access.redhat.com
Start: 2025-05-29T22:41:52+0000
HOST: servera
  Loss%  Snt  Last  Avg  Best  Wrst  StDev
1. |-- classroom.lab.example.com 0.0%    5    0.4   0.4   0.3   0.5   0.1
2. |-- 72.32.49.3 0.0%    5    1.0   1.3   0.9   2.3   0.6
3. |-- 72.32.28.27 0.0%    5    1.3   1.5   1.3   1.7   0.2
4. |-- aggr172b-54-cored.dfw3.ra 0.0%    5    1.0   1.0   0.8   1.0   0.1
5. |-- dcpe3-cored.dfw3.rackspac 0.0%    5    1.9   2.1   1.9   2.5   0.2
6. |-- 10.25.1.101 0.0%    5    2.0   2.0   2.0   2.1   0.1
7. |-- equinix-dfw2.netarch.akam 0.0%    5   15.8  15.9   6.3  27.9   9.0
...output omitted...
```

Troubleshooting Port and Service Issues

Applications use sockets as communication endpoints, which are defined by an IP address, protocol, and port number. For example, the Postfix mail service might listen on TCP port 25 and a specific IP address, which together form a socket. Services generally listen on designated standard ports, whereas client applications dynamically select a random, ephemeral port for their outgoing connections. The `/etc/services` file provides a directory of well-known service names mapped to their standard port assignments.

The `ss` command, which displays socket statistics, has superseded the earlier `netstat` tool (from the `net-tools` package), although the `netstat` command might still be familiar to experienced system administrators.

```
user@host:~$ ss -tan
```

State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port
LISTEN	0	100	127.0.0.1:25	0.0.0.0:* 1
LISTEN	0	128	0.0.0.0:22	0.0.0.0:* 2
LISTEN	0	4096	0.0.0.0:111	0.0.0.0:*
ESTAB	0	52	172.25.250.10:22	172.25.250.9:57560
3				
LISTEN	0	100	:::1:25	:::]:* 4
LISTEN	0	4096	*:9090	*:* 5
LISTEN	0	128	:::]:22	:::]:* 6
LISTEN	0	4096	:::]:111	:::]:*

- 1 The SMTP mail service listens on port 25 by using only the IPv4 loopback interface (127.0.0.1). As a result, this service is not accessible on the network.
- 2 The SSH service is listening on port 22 by using all IPv4 addresses (0.0.0.0).
- 3 This established SSH connection originates from a client with an address of 172.25.250.9.
- 4 The mail service also listens on the IPv6 loopback device (:::1).
- 5 The RHEL Web Console is listening on port 9090. The asterisk (*) character represents all when referencing IPv4 addresses or ports.
- 6 The SSH daemon also listens on IPv6. The double colon (:::) syntax represents all IPv6 interfaces.

Table 32. Options for ss and netstat

Option	Description
-n	Show numbers instead of names for interfaces and ports.
-t	Show TCP sockets.
-u	Show UDP sockets.
-l	Show only listening sockets.
-a	Show all (listening and established) sockets.
-p	Show the process that uses the sockets.

Option	Description
-A inet	Display active connections (but not listening sockets) for the inet address family. That is, ignore local UNIX domain sockets. For the ss command, both IPv4 and IPv6 connections are displayed. For the netstat command, only IPv4 connections are displayed. (The netstat -A inet6 command displays IPv6 connections, and the netstat -46 command displays IPv4 and IPv6 at the same time.)

 References

ip-link(8), ip-address(8), ip-route(8), ip(8), ping(8), tracepath(8), traceroute(8), mtr(8), ss(8), and netstat(8) man pages

For more information, refer to the *Configuring and Managing Networking* guide at https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html-single/configuring_and_managing_networking/index

For use by Krishna Jeevan Upadhyay redhat.krishna@gmail.com redhat.krishna@gmail.com Copyright © 2025 Red Hat, Inc. | For use by Krishna Jeevan Upadhyay redhat.krishna@gmail.com redhat.krishna@gmail.com

17.4. Guided Exercise

Validate Network Configuration

Test and inspect the current network configuration by using command-line utilities.

Outcomes

- Identify the current network interfaces and network addresses.

Prerequisites

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
student@workstation:~$ lab start netbasics-validate
```

Instructions

Network interface names are determined by their bus type and the detection order of devices during boot. Your network interface names might vary according to the course platform and hardware in use.

On your system, locate the interface name (such as `eth1`, `ens06`, or `enp0p2`) that is associated with the Ethernet address `52:54:00:01:fa:0a`. Record or remember this name, and use it to replace the `ens3` name in subsequent commands if different.

1. On the servera machine, display the list of network interfaces in your system by using the `ip` command.

- 1.1. As the student user on the workstation machine, log in to the servera machine as the student user.

```
student@workstation:~$ ssh student@servera
...output omitted...
[student@servera ~]$
```

- 1.2. Locate the network interface name that is associated with the `52:54:00:00:fa:0a` Ethernet address. Record or remember this name, and use it to replace the `ens3` placeholder in subsequent commands.

```
[student@servera ~]# ip link
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN ...
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: ens3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP ...
   link/ether 52:54:00:00:fa:0a brd ff:ff:ff:ff:ff:ff
   altname enp0s3
   altname enx52540000fa0a
3: ens4: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 8942 qdisc fq_codel state UP ...
   link/ether 52:54:00:01:fa:1e brd ff:ff:ff:ff:ff:ff
   altname enp0s4
   altname enx52540001fa1e
```

2. Display the current IP address and netmask for all interfaces.

```
[student@servera ~]$ ip -br addr
lo                UNKNOWN          127.0.0.1/8 ::1/128
ens3              UP              172.25.250.10/24 fe80::5054:ff:fe00:fa0a/64
ens4              UP
```

3. Display packet statistics for the ens3 interface.

```
[student@servera ~]$ ip -s link show ens3
2: ens3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP
...
   link/ether 52:54:00:00:fa:0a brd ff:ff:ff:ff:ff:ff
RX:  bytes packets errors dropped missed  mcast
     1214412    7146      0      3      0      0
TX:  bytes packets errors dropped carrier collsns
     2099808    6288      0      0      0      0
   altname enp0s3
   altname enx52540000fa0a
```

4. View the host's routing table.

```
[student@servera ~]$ ip route
default via 172.25.250.254 dev ens3 proto static metric 100
172.25.250.0/24 dev ens3 proto kernel scope link src 172.25.250.10 metric 100
```

5. Verify that the router is accessible.

```
[student@servera ~]$ ping -c3 172.25.250.254
PING 172.25.250.254 (172.25.250.254) 56(84) bytes of data.
64 bytes from 172.25.250.254: icmp_seq=1 ttl=64 time=0.196 ms
64 bytes from 172.25.250.254: icmp_seq=2 ttl=64 time=0.436 ms
64 bytes from 172.25.250.254: icmp_seq=3 ttl=64 time=0.361 ms

--- 172.25.250.254 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 49ms
rtt min/avg/max/mdev = 0.196/0.331/0.436/0.100 ms
```

6. Show all the hops between the local system and the `classroom.example.com` system.

```
[student@servera ~]$ tracepath classroom.example.com
1?: [LOCALHOST] pmtu 1500
1: classroom.example.com 0.296ms reached
1: classroom.example.com 0.125ms reached
Resume: pmtu 1500 hops 1 back 1
```

7. Display listening TCP sockets on the local system.

```
[student@servera ~]$ ss -lt
```

State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port
LISTEN	0	128	0.0.0.0:ssh	0.0.0.0:*
LISTEN	0	4096	0.0.0.0:sunrpc	0.0.0.0:*
LISTEN	0	128	:::ssh	:::*
LISTEN	0	4096	:::sunrpc	:::*
LISTEN	0	4096	*:websm	*:*

8. Return to the workstation system as the student user.

```
[student@servera ~]$ exit
logout
Connection to servera closed.
student@workstation:~$
```

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish netbasics-validate
```

17.5. Summary

In this lesson, you learned the fundamentals of networking in Red Hat Enterprise Linux, including the TCP/IP model, IP addressing, interface naming, routing, configuration methods, and troubleshooting commands.

The key protocols for the internet layer are IPv4 and IPv6. An IPv4 address is a 32-bit number shown as four dot-separated octets. An IPv4 address is divided into a network prefix and host identifier by a netmask, and is configurable via DHCP or manually. IPv6 addresses are 128-bit hexadecimal numbers, commonly shown as eight colon-separated hexadecimal groups. IPv6 addresses are often shortened and used with a /64 prefix.

Red Hat Enterprise Linux uses consistent network interface names that persist across reboots, based on firmware and PCI bus topology, rather than detection order.

Hosts use a routing table to determine which network interface to use and which router to forward packets to.

You can use several command-line utilities to gather network information and to troubleshoot issues. For example, you can display address and routing information (the `ip` command), verify connectivity (the `ping` and `ping6` commands), trace traffic paths (the `traceroute`, `tracpath`, and `mtr` commands), and display socket statistics (the `ss` and `netstat` commands).

Chapter 18.

Managing Network Configuration

Goal

Configure network interfaces and settings on Red Hat Enterprise Linux servers.

Sections

- Configuring Networking from the Command Line (and Guided Exercise)
- Editing Network Configuration Files (and Guided Exercise)
- Configuring Hostnames and Name Resolution (and Guided Exercise)

Lab

- Manage Network Configuration

18.1. Configuring Networking from the Command Line

Objectives

- Manage network settings and devices from the command line by using the `nmccli` command.

Managing Network Devices and Connections

In RHEL systems, you use the NetworkManager service to monitor and manage network settings. The NetworkManager service, for example, provides the network and VPN connection status shown in the system menu of the GNOME graphical environment.

The NetworkManager service manages network *devices* and *connections*. A *device* is any physical or virtual network interface that handles network traffic. A *connection* (also known as a *network profile*) has related configuration settings for a single network device. Each connection must have a unique name or ID, which can match the device name that it configures. Service configuration files are stored in the `/etc/NetworkManager/system-connections/` directory.

A single device can have multiple connection profiles, and the user can switch between them, but only one connection at a time can be active for the same device. For example, you might configure a laptop wireless device to use a fixed IP address at a work site, and to use a second connection with an automated address for home.



Important

In Red Hat Enterprise Linux 10, support for `ifcfg` configuration files and the `/etc/sysconfig/network-scripts/` directory is fully removed.

NetworkManager now uses an INI-style keyfile format, which is a key-value pair structure to organize properties. NetworkManager stores network profiles in the `/etc/NetworkManager/system-connections/` directory.

Viewing Network Information

Use the `nmccli` utility to create and edit connection files from the command line. The `nmccli device status` command displays the status of all network devices:

```
root@host:~# nmcli dev status
```

DEVICE	TYPE	STATE	CONNECTION
ens3	ethernet	connected	ethernet-ens3
lo	loopback	connected (externally)	lo

In the previous command output, the loopback interface is marked as external, which means that NetworkManager does not manage that device.

Note

The `nmcli` command accepts abbreviations for its options and arguments. For example, you can abbreviate `nmcli device disconnect` as `nmcli dev dis`, and abbreviate `nmcli connection modify` as `nmcli con mod`. The abbreviation can be as short as a single letter, as long as it uses enough characters to uniquely identify the object to manage.

The `nmcli connection show` command displays a list of all connections. Use the `--active` option to list only active connections.

```
root@host:~# nmcli con show
```

NAME	UUID	TYPE	DEVICE
ethernet-ens3	fab1e257-61a6-3526-abf9-8c50c83a28c7	ethernet	ens3
enp8s0	55e68e96-905d-4613-a29a-f5f88d3a12e8	ethernet	--
lo	e6274c48-3de6-4a73-b088-f3abde27842f	loopback	lo

```
root@host:~# nmcli con show --active
```

NAME	UUID	TYPE	DEVICE
ethernet-ens3	fab1e257-61a6-3526-abf9-8c50c83a28c7	ethernet	ens3
lo	e6274c48-3de6-4a73-b088-f3abde27842f	loopback	lo

Adding a Network Connection

Use the `nmcli connection add` command to add network connections. The data for the added network connection is stored in the `/etc/NetworkManager/system-connections/` directory as a file with a `.nmconnection` suffix.

The following example adds a connection of the `ethernet` type for the `enp8s0` network interface. Notice that NetworkManager automatically names the connection as `ethernet-enp8s0`.

```
root@host:~# nmcli con add ifname enp8s0 type ethernet
Connection 'ethernet-enp8s0' (db408049-af1d-44df-b451-6c7f4dbf352b)
successfully added.
```

The next example creates an `enp9s0` connection of the `ethernet` type for the `enp9s0` network interface with a static IPv4 network setting. This command configures the `192.168.0.5` IP address with a network prefix of `/24` and a network gateway of `192.168.0.254`. The `nmcli connection add` command fails if the connection name that you try to add already exists.

```
root@host:~# nmcli con add con-name enp9s0 type ethernet ifname enp9s0 \
ipv4.method manual ipv4.addresses 192.168.0.5/24 ipv4.gateway 192.168.0.254
Connection 'enp9s0' (1a2ddcef-9daa-4610-9214-1c11d23bef3b) successfully added.
```

The next example creates an `enp10s0` connection for the `enp10s0` device with static IPv6 and IPv4 addresses. This command configures the `2001:db8:0:1::c000:207` IPv6 address with the `/64` network prefix and the `2001:db8:0:1::1` address as the default gateway. The command also configures the `192.0.2.7` IPv4 address with the `/24` network prefix and the `192.0.2.1` address as the default gateway.

```
root@host:~# nmcli con add con-name enp10s0 type ethernet ifname enp10s0 \
ipv6.method manual ipv6.addresses '2001:db8:0:1::c000:207/64' ipv6.gateway \
'2001:db8:0:1::1' ipv4.method manual ipv4.addresses '192.0.2.7/24' \
ipv4.gateway '192.0.2.1'
Connection 'enp10s0' (2e779f16-5fc5-4ecc-a92e-c16b9a0cd0a4) successfully added.
```

Note

IP addresses do not require quotation marks with the `nmcli` command, but using them can help tab completion in the shell.

Managing Network Connections

The `nmcli connection up` command activates a network connection on the device that it is bound to. Activating a network connection requires the connection name, not the device name.

```

root@host:~# nmcli con show
NAME      UUID                                  TYPE      DEVICE
enp1s0    55e68e96-905d-4613-a29a-f5f88d3a12e8  ethernet  enp1s0
lo        8ddeb3e7-1399-4619-a1bc-c82c9aa11034  loopback  lo
enp8s0    2e779f16-5fc5-4ecc-a92e-c16b9a0cd0a4  ethernet  --

root@host:~# nmcli con up enp8s0
Connection successfully activated (D-Bus active path:
/org/freedesktop/NetworkManager/ActiveConnection/9)

```

The `nmcli device disconnect` command disconnects the network device and closes the connection:

```

root@host:~# nmcli dev dis enp8s0
Device 'enp8s0' successfully disconnected.

```

Important

Use `nmcli device disconnect` to stop traffic on a network interface and to deactivate the connection.

Updating Network Connection Settings

NetworkManager service connections have two setting types. *Static connection* properties are configured by the administrator and are stored in the `/etc/NetworkManager/system-connections/*.nmconnection` configuration files. *Dynamic connection* properties are requested from a DHCP server and are not stored persistently.

To list the current settings for a connection, use the `nmcli connection show` command. Settings in lowercase are static properties that the administrator can change. Settings in uppercase are settings that are active only during runtime. The `-p` option is available to pretty-print the output to make it more readable.

```

root@host:~# nmcli -p con show enp8s0
=====
                        Connection profile details (enp8s0)
=====
connection.id:                enp8s0
connection.uuid:              2e779f16-5fc5-...-16b9a0cd0a4
connection.stable-id:        --
connection.type:              802-3-ethernet
connection.interface-name:    enp8s0
connection.autoconnect:      yes
connection.autoconnect-priority: 0
...output omitted...

```

You can also select specific fields when printing connection details:

```

root@host:~# nmcli -f ipv4.method,ipv4.addresses,ipv4.gateway con show enp8s0
ipv4.method:                manual
ipv4.addresses:              192.0.2.7/24
ipv4.gateway:                192.0.2.1

```

Updating Connection Settings from the Command Line

Use the `nmcli connection modify` command to update connection settings. These changes are saved in the `/etc/NetworkManager/system-connections/name.nmconnection` file. Consult the `nm-settings(5)` man page for the available settings.

Use the following command to update the `enp8s0` connection to set the `192.0.2.2/24` IPv4 address and the `192.0.2.254` default gateway. Use the `nmcli connection autoconnect` parameter to automatically enable or disable the connection at system boot.

```

root@host:~# nmcli con mod enp8s0 ipv4.addresses 192.0.2.2/24 \
ipv4.gateway 192.0.2.254 connection.autoconnect yes

```

Use the following command to update the `enp8s0` connection to set the `2001:db8:0:1::a00:1/64` IPv6 address and the `2001:db8:0:1::1` default gateway:

```
root@host:~# nmcli con mod enp8s0 ipv6.addresses '2001:db8:0:1::a00:1/64' \
ipv6.gateway '2001:db8:0:1::1'
```

Important

To change a DHCP connection configuration to be static, update the `ipv4.method` setting from `auto` or `dhcp` to `manual`. For an IPv6 connection, update the `ipv6.method` setting. If the method is not set correctly, then the connection might hang or be incomplete when activated, or it might obtain an address from DHCP or SLAAC in addition to the configured static address.

Some settings can have multiple values. A specific value can be added to the list or deleted from the connection settings by adding a plus (+) or minus (-) symbol to the start of the setting name. If a plus or minus is not included, then the specified value replaces the setting's current list.

The following example adds the 2.2.2.2 DNS server to the `enp8s0` connection:

```
root@host:~# nmcli con mod enp8s0 +ipv4.dns 2.2.2.2
```

Updating Connection Settings by Using Configuration Files

You can also modify network profiles by editing the connection's configuration file in the `/etc/NetworkManager/system-connections/` directory. Whereas `nmcli` commands communicate directly with NetworkManager to implement modifications immediately, connection file edits are not implemented until NetworkManager reloads the configuration file. With manual editing, you can create complex configurations in steps, and then load the final configuration when ready.

After manually updating the connection's configuration file, use the `nmcli` command to reload all connection profiles:

```
root@host:~# nmcli con reload
```

The next example rereads only the `enp8s0` connection profile at `/etc/NetworkManager/system-connections/enp8s0.nmconnection`:

```
root@host:~# nmcli con reload enp8s0
```

Deleting a Network Connection

The `nmcli connection delete` command deletes a connection from the system. This command disconnects the device and removes the connection configuration file.

```
root@host:~# nmcli con del enp8s0
```

Permissions to Modify NetworkManager Settings

Use the `nmcli general permissions` command to view your current permissions to change network configuration. As shown in the following example, the `root` user is granted all NetworkManager permissions:

```
root@host:~# nmcli gen permissions
PERMISSION                                VALUE
org.freedesktop.NetworkManager.checkpoint-rollback    yes
org.freedesktop.NetworkManager.enable-disable-connectivity-check    yes
org.freedesktop.NetworkManager.enable-disable-network    yes
org.freedesktop.NetworkManager.enable-disable-statistics    yes
org.freedesktop.NetworkManager.enable-disable-wifi    yes
org.freedesktop.NetworkManager.enable-disable-wimax    yes
org.freedesktop.NetworkManager.enable-disable-wwan    yes
org.freedesktop.NetworkManager.network-control    yes
org.freedesktop.NetworkManager.reload    yes
org.freedesktop.NetworkManager.settings.modify.global-dns    yes
org.freedesktop.NetworkManager.settings.modify.hostname    yes
org.freedesktop.NetworkManager.settings.modify.own    yes
org.freedesktop.NetworkManager.settings.modify.system    yes
org.freedesktop.NetworkManager.sleep-wake    yes
org.freedesktop.NetworkManager.wifi.scan    yes
org.freedesktop.NetworkManager.wifi.share.open    yes
org.freedesktop.NetworkManager.wifi.share.protected    yes
```

Nonprivileged users that are logged in on the physical or virtual console can also make most network configuration changes. If a user is on the system's console, then the system is likely being used as a workstation or laptop where the user needs to configure, activate, and deactivate connections.

Nonprivileged users who log in by using SSH are not granted many permissions and must switch to the `root` user to change network settings. The following example lists the user's NetworkManager permissions when connected by using SSH.

```
user@host:~$ nmcli gen permissions
PERMISSION                                     VALUE
org.freedesktop.NetworkManager.checkpoint-rollback    auth
org.freedesktop.NetworkManager.enable-disable-connectivity-check    no
org.freedesktop.NetworkManager.enable-disable-network    no
org.freedesktop.NetworkManager.enable-disable-statistics    no
org.freedesktop.NetworkManager.enable-disable-wifi    no
org.freedesktop.NetworkManager.enable-disable-wimax    no
org.freedesktop.NetworkManager.enable-disable-wwan    no
org.freedesktop.NetworkManager.network-control    auth
org.freedesktop.NetworkManager.reload    auth
org.freedesktop.NetworkManager.settings.modify.global-dns    auth
org.freedesktop.NetworkManager.settings.modify.hostname    auth
org.freedesktop.NetworkManager.settings.modify.own    auth
org.freedesktop.NetworkManager.settings.modify.system    auth
org.freedesktop.NetworkManager.sleep-wake    no
org.freedesktop.NetworkManager.wifi.scan    auth
org.freedesktop.NetworkManager.wifi.share.open    no
org.freedesktop.NetworkManager.wifi.share.protected    no
```

Useful NetworkManager Commands

The following table lists the key `nmcli` commands that are discussed in this section:

Command	Purpose
<code>nmcli dev status</code>	Show the NetworkManager status of all network interfaces
<code>nmcli con show</code>	List all connections
<code>nmcli con show <i>name</i></code>	List the current settings for the connection name
<code>nmcli con add con-name <i>name</i></code>	Add and name a new connection profile
<code>nmcli con mod <i>name</i></code>	Modify the connection name

Command	Purpose
<code>nmcli con reload</code>	Reload the configuration files, after manual file editing
<code>nmcli con up <i>name</i></code>	Activate the connection name
<code>nmcli dev dis <i>dev</i></code>	Disconnect the interface, which also deactivates the current connection
<code>nmcli con del <i>name</i></code>	Delete the specified connection and its configuration file



References

NetworkManager(8), nmcli(1), nmcli-examples(7), nm-settings(5), hostnamectl(1), resolv.conf(5), hostname(5), ip(8), and ip-address(8) man pages

For more information, refer to the *Configuring an Ethernet Connection* chapter in the *Configuring and Managing Networking* guide at https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html-single/configuring_and_managing_networking/index#configuring-an-ethernet-connection

For use by Krishna Jeevan Upadhyay redhat.krishna@gmail.com redhat.krishna@gmail.com Copyright © 2025 Red Hat, Inc. | For use by Krishna Jeevan Upadhyay redhat.krishna@gmail.com

18.2. Guided Exercise

Configure Networking from the Command Line

Manage network settings and devices from the command line by using the `nmc li` command.

Outcomes

- Update a network connection setting from DHCP to static.

Prerequisites

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
student@workstation:~$ lab start net-configure
```

Instructions

Network interface names are determined by their bus type and the detection order of devices during boot. Your network interface names might vary according to the course platform and hardware in use.

On your system, locate the interface name (such as `eth1`, `ens06`, or `enp0p2`) that is associated with the Ethernet address `52:54:00:01:fa:1e`. Use this interface name to replace the `ens4` placeholder throughout this exercise if different.

1. As the student user on the workstation machine, log in to the servera machine as the student user. Switch to the root user. Use student as the password.

```
student@workstation:~$ ssh student@servera
...output omitted...
[student@servera ~]$ sudo -i
[sudo] password for student: student
[root@servera ~]#
```

2. Display the list of network interfaces in your system by using the `ip` command.

Locate the network interface name that is associated with the `52:54:00:01:fa:1e` Ethernet address. Record or remember this name, and use it to replace the `ens4` name in subsequent commands if needed.

```
[root@servera ~]# ip link
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode
DEFAULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: ens3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP
mode DEFAULT group default qlen 1000
    link/ether 52:54:00:00:fa:0a brd ff:ff:ff:ff:ff:ff
    altname enp0s3
    altname enx52540000fa0a
3: ens4: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 8942 qdisc fq_codel state UP
mode DEFAULT group default qlen 1000
    link/ether 52:54:00:01:fa:1e brd ff:ff:ff:ff:ff:ff
    altname enp0s4
    altname enx52540001fa1e
```

3. Use the `nmcli` command to view current network connections. Active connections have device names associated with them. The `nmcli` command shows two dashes (--) in the device column for inactive connections.

```
[root@servera ~]# nmcli con show
```

NAME	UUID	TYPE	DEVICE
cloud-init ens3	fab1e257-61a6-3526-abf9-8c50c83a28c7	ethernet	ens3
lo	1fdb0926-cde3-40ab-b68d-14068716c650	loopback	lo
Wired connection 1	e5a48013-c870-3a20-9ad2-b5d13d9603f2	ethernet	--

Although NetworkManager created the `Wired connection 1` temporary connection, it does not get activated successfully. You might observe the connection briefly highlighted in yellow after a reboot; however it consistently fails after a few minutes, resulting in two dashes (--) appearing in the device column.

4. Display all settings for the `Wired connection 1` connection to investigate why it is not currently active.

```
[root@servera ~]# nmcli con show 'Wired connection 1'
connection.id:                Wired connection 1
connection.uuid:              508dcf19-e8dc-345c-b04d-217bfbcc4ed2
connection.stable-id:         --
connection.type:              802-3-ethernet
connection.interface-name:    ens4
...output omitted...
ipv4.method:                  auto
ipv4.dns:                     --
ipv4.dns-search:              --
ipv4.dns-options:             --
ipv4.dns-priority:            0
ipv4.addresses:               --
ipv4.gateway:                 --
...output omitted...
```

The `Wired connection 1` connection is associated with the secondary network interface; the `ens4` device in this environment.

Notice the `auto` address configuration method shown in the previous output. This connection is trying to use the DHCP protocol to configure the interface. This connection remained inactive during boot because a DHCP server is not present on the classroom network.

5. Show the status of the network devices by using the `nmcli dev status` command. Notice that the secondary interface is disconnected.

```
[root@servera ~]# nmcli dev status
```

DEVICE	TYPE	STATE	CONNECTION
ens3	ethernet	connected	cloud-init ens3
lo	loopback	connected (externally)	lo
ens4	ethernet	disconnected	--

6. Create the `static-addr` static connection with the `172.24.250.30` address and `/24` network prefix.

```
[root@servera ~]# nmcli con add con-name static-addr type ethernet ifname
ens4 \
ipv4.method manual ipv4.addresses 172.24.250.30/24
Connection 'static-addr' (227e5a9e-b96b-45e7-ac90-ca8f9c3a71bc) successfully
added.
```

7. Show the current status of the secondary interface that you configured.

```
[root@servera ~]# nmcli device show ens4
GENERAL.DEVICE:                ens4
GENERAL.TYPE:                  ethernet
GENERAL.HWADDR:                52:54:00:01:FA:1E
GENERAL.MTU:                   8942
GENERAL.STATE:                 100 (connected)
GENERAL.CONNECTION:            static-addr
GENERAL.CON-PATH:              /org/freedesktop/NetworkManager/ActiveConnection/20
WIRED-PROPERTIES.CARRIER:    on
IP4.ADDRESS[1]:                172.24.250.30/24
IP4.GATEWAY:                   --
IP4.ROUTE[1]:                  dst = 172.24.250.0/24, nh = 0.0.0.0, mt = 101
IP6.ADDRESS[1]:                fe80::5054:ff:fe01:fa1e/64
IP6.GATEWAY:                   --
IP6.ROUTE[1]:                  dst = fe80::/64, nh = ::, mt = 1024
```

8. Reboot the servera machine.

```
[root@servera ~]# systemctl reboot
Connection to servera closed by remote host.
Connection to servera closed.
student@workstation:~$
```

9. Verify that the static-addr connection is active following the reboot.

- 9.1. Log back in to the servera machine.

```
student@workstation:~$ ssh student@servera
...output omitted...
```

- 9.2. Display the list of all network connections on the system.

```
[student@servera ~]$ nmcli con show
```

NAME	UUID	TYPE	DEVICE
cloud-init ens3	fab1e257-61a6-3526-abf9-8c50c83a28c7	ethernet	ens3
static-addr	227e5a9e-b96b-45e7-ac90-ca8f9c3a71bc	ethernet	ens4
lo	4614a696-227a-4b8e-adab-4105feabb95e	loopback	lo

Notice that the secondary interface, ens4, has been activated by the static connection that you created. In addition, the temporary Wired connection 1 DHCP-based connection has disappeared following the reboot.

- 9.3.** Verify the IP address that is configured on the ens4 interface by using the `nmcli dev` command.

```
[student@servera ~]$ nmcli -f IP4 dev show ens4
```

IP4.ADDRESS[1]:	172.24.250.30/24
IP4.GATEWAY:	--
IP4.ROUTE[1]:	dst = 172.24.250.0/24, nh = 0.0.0.0, mt = 101

- 9.4.** Return to the workstation system as the student user.

```
[student@servera ~]$ exit
logout
Connection to servera closed.
student@workstation:~$
```

- 10.** Create a static network connection on the serverb machine to communicate with the servera system on the secondary network. Similarly, this machine has a secondary network interface which is failing due to the default DHCP connection.

- 10.1.** Log in to the serverb machine as the student user. Switch to the root user. Use student as the password.

```
student@workstation:~$ ssh student@serverb
...output omitted...
[student@serverb ~]$ sudo -i
[sudo] password for student: student
[root@serverb ~]#
```

- 10.2.** Identify the name of the secondary network interface that has the MAC address of 52:54:00:01:fa:28.

```
[root@serverb ~]# ip link
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT
    group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: ens3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode
    DEFAULT group default qlen 1000
    link/ether 52:54:00:00:fa:0b brd ff:ff:ff:ff:ff:ff
    altname enp0s3
    altname enx52540000fa0b
3: ens4: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 8942 qdisc fq_codel state UP mode
    DEFAULT group default qlen 1000
    link/ether 52:54:00:01:fa:28 brd ff:ff:ff:ff:ff:ff
    altname enp0s4
    altname enx52540001fa28
```

Notice that in the previous output, the correct interface is named ens4. Depending on your hardware, the correct interface name might be different on your system.

10.3.Delete the temporary Wired connection 1 connection that NetworkManager created for the ens4 interface.

```
[root@serverb ~]# nmcli con del 'Wired connection 1'
Connection 'Wired connection 1' (508dcf19-e8dc-345c-b04d-217bfbcc4ed2)
successfully deleted.
```

10.4.Create the static-addr static connection for the ens4 interface with the 172.24.250.40 address and /24 network prefix.

```
[root@serverb ~]# nmcli con add con-name static-addr type ethernet ifname ens4 \
    ipv4.method manual ipv4.addresses 172.24.250.40/24
Connection 'static-addr' (c2131041-1fec-4563-9856-3bc582801758) successfully
added.
```

10.5.Display the list of network connections to verify that the new connection has activated automatically.

```
[root@serverb ~]# nmcli con show
```

NAME	UUID	TYPE	DEVICE
cloud-init ens3	fab1e257-61a6-3526-abf9-8c50c83a28c7	ethernet	ens3
static-addr	c2131041-1fec-4563-9856-3bc582801758	ethernet	ens4
lo	9c62f5c6-d9fb-4088-8308-50fcdace41b5	loopback	lo

10.6.Verify the device status of the ens4 interface by using the nmcli dev command.

```
[root@serverb ~]# nmcli dev show ens4
GENERAL.DEVICE:           ens4
GENERAL.TYPE:             ethernet
GENERAL.HWADDR:           52:54:00:01:FA:28
GENERAL.MTU:              8942
GENERAL.STATE:            100 (connected)
GENERAL.CONNECTION:       static-addr
GENERAL.CON-PATH:         /org/freedesktop/NetworkManager/ActiveConnection/10
WIRED-PROPERTIES.CARRIER: on
IP4.ADDRESS[1]:           172.24.250.40/24
IP4.GATEWAY:              --
IP4.ROUTE[1]:             dst = 172.24.250.0/24, nh = 0.0.0.0, mt = 101
IP6.ADDRESS[1]:           fe80::5054:ff:fe01:fa28/64
IP6.GATEWAY:              --
IP6.ROUTE[1]:             dst = fe80::/64, nh = ::, mt = 1024
```

11. To validate the network configuration, test connectivity to the servera machine by using the ping command.

```
[root@serverb ~]# ping -c 3 172.24.250.30
PING 172.24.250.30 (172.24.250.30) 56(84) bytes of data.
64 bytes from 172.24.250.30: icmp_seq=1 ttl=64 time=3.83 ms
64 bytes from 172.24.250.30: icmp_seq=2 ttl=64 time=0.339 ms
64 bytes from 172.24.250.30: icmp_seq=3 ttl=64 time=0.347 ms

--- 172.24.250.30 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2008ms
rtt min/avg/max/mdev = 0.339/1.505/3.830/1.643 ms
```

12. Return to the workstation system as the student user.

```
[root@serverb ~]# exit
logout
[student@serverb ~]$ exit
logout
Connection to serverb closed.
student@workstation:~$
```

Finish

On the workstation machine, use the lab command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.


```
student@workstation:~$ lab finish net-configure
```

18.3. Editing Network Configuration Files

Objectives

- Modify network configuration by editing configuration files.

Connection Configuration Files

As an alternative to using the `nmcli` command, you can configure networks by directly editing the connection configuration files. Although the `nmcli` command is the preferred method for profile management, you can manually create or modify these configuration files. This alternative is especially useful when you plan to use the same or similar configuration files for several servers.

NetworkManager connection profiles are stored in the `/etc/NetworkManager/system-connections/` directory. These files use the keyfile syntax, which replaces the deprecated `ifcfg` format. When you add or modify connections by using the `nmcli` command, those changes are also persistently stored in this directory as files ending with the `.nmconnection` extension.

After any manual edits, you must use the `nmcli con reload` command to reload the configuration files and to inform NetworkManager of the changes. To apply the changes, you must manually activate the connection by using the `nmcli con up` command.

The Keyfile Syntax

NetworkManager employs an INI-style keyfile syntax for its network connection profiles. Configuration items are structured into sections (groups), with each setting represented as a key-value pair within these groups. This configuration syntax offers readability benefits, such as specifying IP addresses as `192.168.0.1/24` instead of less intuitive integer arrays.

The following table shows some `nmcli` commands along with their equivalent settings in the `*.nmconnection` file.

Table 33. Comparison of NetworkManager Settings and the Keyfile Syntax

<code>nmcli con mod</code>	<code>*.nmconnection file</code>	Effect
<code>ipv4.method manual</code>	<code>[ipv4]</code> <code>method=manual</code>	Configure IPv4 addresses statically.

nmcli con mod	*.nmconnection file	Effect
ipv4.method auto	[ipv4] method=auto	Obtain configuration settings from a DHCPv4 server.
ipv4.addresses 192.0.2.1/24	[ipv4] address1=192.0.2.1/24	Set a static IPv4 address and network prefix. Increment the index as needed when you have more IP addresses to configure. For example, use address2 to define the secondary IP address.
ipv4.gateway 192.0.2.254	[ipv4] gateway=192.0.2.254	Set the default gateway.
ipv4.dns 8.8.8.8	[ipv4] dns=8.8.8.8	Modify /etc/resolv.conf to use this name server.
ipv4.dns-search example.com	[ipv4] dns-search=example.com	Modify /etc/resolv.conf to use this domain in the search directive.
ipv4.ignore-auto-dns true	[ipv4] ignore-auto-dns=true	Ignore DNS server information from the DHCP server.
ipv6.method manual	[ipv6] method=manual	Configure IPv6 addresses statically.

nmcli con mod	*.nmconnection file	Effect
ipv6.method auto	[ipv6] method=auto	Configure network settings with SLAAC from router advertisements.
ipv6.method dhcp	[ipv6] method=dhcp	Configure network settings by using DHCPv6, but not by using SLAAC.
ipv6.addresses 2001:db8::a/64	[ipv6] address1=2001:db8::a/64	Set a static IPv6 address and network prefix. When using more than one address for a connection, the address2 key defines the second address, and the address3 key defines the third address.
ipv6.gateway 2001:db8::1	[ipv6] gateway=2001:db8::1	Set the default gateway.
ipv6.dns fde2:6494:1e09:2::d	[ipv6] dns=fde2:6494:1e09:2::d	Modify /etc/resolv.conf to use this name server.
ipv6.dns-search example.com	[ipv6] dns-search=example.com	Modify /etc/resolv.conf to use this domain in the search directive.
ipv6.ignore-auto-dns true	[ipv6] ignore-auto-dns=true	Ignore DNS server information from the DHCP server.

nmcli con mod	*.nmconnection file	Effect
connection.autoconnect yes	[connection] autoconnect=true	Automatically activate this connection at boot.
connection.id ens3	[connection] id=Main eth0	Set the name of the connection.
connection.interface-name ens3	[connection] interface-name=ens3	Assign a connection to a specific interface.
802-3-ethernet.mac-address 00:1A:2B:3C:4D:5E	[802-3-ethernet] mac-address= 00:1A:2B:3C:4D:5E	Associate the connection with a specific MAC address.

Configuration File Locations

Connection configuration files, which are in the `/etc/NetworkManager/system-connections/` directory, are typically named `name.nmconnection`, where `name` corresponds to the device or connection that it controls.

NetworkManager organizes its configuration files in the following directories:

`/etc/NetworkManager/system-connections/`

This directory holds persistent connection profiles that users have created or modified.

`/run/NetworkManager/system-connections/`

This directory stores temporary profiles. These profiles are automatically deleted upon system reboot.

`/usr/lib/NetworkManager/system-connections/`

This directory contains preconfigured, immutable connection profiles.

Examples of Editing Configuration Files

The following sample file configures a static IPv4 connection:

```
[connection]
id=Wired connection 1
uuid=27afa607-ee36-43f0-b8c3-9d245cdc4bb3
type=ethernet
autoconnect=true
interface-name=ens4

[ethernet]
mac-address=00:1A:2B:3C:4D:5E

[ipv4]
method=manual
address1=172.25.0.10/24
gateway=172.25.0.254
dns=172.25.254.254
dns-search=example.com
```

For added specificity, the connection can include the MAC address, but it is not mandatory.

In the following scenario, imagine that you want to permanently change the IP address of the `static-ens4` connection from `172.24.250.30` to `172.24.250.40`. To accomplish this task by directly editing the connection file, you must make the change by using a text editor, reload the connection, and then re-activate the connection.

First, verify the current interface settings. Initially, both the `ip` and `nmcli` commands show the active IP address of `172.25.250.30`. Note that the `ip` command displays real-time network information, whereas the `nmcli con show` command displays the persistent configuration that is used during system startup.

```
root@host:~# ip -4 -br addr show dev ens4
ens4                UP                172.24.250.30/24

root@host:~# nmcli -f ipv4.addresses con show static-ens4
ipv4.addresses:      172.24.250.30/24
```

Next, edit the connection file to permanently change the IP address information. For brevity, this example uses the `sed` stream editor, although you can use any text editor.

```
root@host:~# sed -i 's/172.24.250.30/172.24.250.40/' \
/etc/NetworkManager/system-connections/static-ens4.nmconnection
```

To apply the changes from the connection file, use the `nmcli con reload` command to instruct NetworkManager to reload the connection:

```
root@host:~# nmcli con reload
```

After reloading, the `nmcli con show` command displays the new IP address, but the `ip` command still shows the original address in the current runtime configuration.

```
root@host:~# nmcli -f ipv4.addresses con show static-ens4
ipv4.addresses:                172.24.250.40/24

root@host:~# ip -4 -br addr show dev ens4
ens4                UP                172.24.250.30/24
```

To see the new IP address in the runtime configuration, the connection must be re-activated by using the `nmcli con up` command:

```
root@host:~# nmcli con up static-ens4
Connection successfully activated (D-Bus active path:
/org/freedesktop/NetworkManager/ActiveConnection/11)

root@host:~# ip -4 -br addr show dev ens4
ens4                UP                172.24.250.40/24
```

References

`nmcli(1)`, `nm-settings(5)`, and `nm-settings-keyfile(5)` man page

For more information, refer to the *Manually Creating a NetworkManager Profile in Keyfile Format* chapter in the *Configuring and Managing Networking* guide at https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html-single/configuring_and_managing_networking/index#manually-creating-a-networkmanager-profile-in-keyfile-format

18.4. Guided Exercise

Edit Network Configuration Files

Modify network configuration by editing configuration files.

Outcomes

- Configure additional network addresses on each system.

Prerequisites

As the student user on the workstation machine, run the following lab command to prepare your system for this exercise, and to ensure that all required resources are available:

```
student@workstation:~$ lab start net-edit
```

Instructions

Network interface names are determined by their bus type and the detection order of devices during boot. Your network interface names might vary according to the course platform and hardware in use.

On your system, locate the network interface name (such as `eth1`, `ens06`, or `enp0p2`) that is associated with the `52:54:00:01:fa:1e` Ethernet address. Use this interface name to replace the `ens4` placeholder throughout this exercise if different.

1. On the servera machine, display the list of network interfaces in your system by using the `ip` command.

- 1.1. Log in to the servera machine as the student user.

```
student@workstation:~$ ssh student@servera
...output omitted...
[student@servera ~]$
```

- 1.2. Display the list of network interfaces in your system by using the `ip` command.

Locate the network interface name that is associated with the `52:54:00:01:fa:1e` Ethernet address. Record or remember this name, and use it to replace the `ens4` placeholder in subsequent commands.


```
[student@servera ~]$ ip link
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT
    group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: ens3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode
    DEFAULT group default qlen 1000
    link/ether 52:54:00:00:fa:0a brd ff:ff:ff:ff:ff:ff
    altname enp0s3
    altname enx52540000fa0a
3: ens4: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 8942 qdisc fq_codel state UP mode
    DEFAULT group default qlen 1000
    link/ether 52:54:00:01:fa:1e brd ff:ff:ff:ff:ff:ff
    altname enp0s4
    altname enx52540001fa1e
```

2. View the current network connections by using the `nmcli` command.

2.1. Use the `nmcli` command to view the current network connections. Although NetworkManager created the `Wired connection 1` temporary connection for the secondary network interface, it does not get activated successfully. You might briefly see the connection highlighted in yellow after a reboot, but it consistently fails within minutes, showing two dashes (--) in the device column.

```
[student@servera ~]$ nmcli con show
```

NAME	UUID	TYPE	DEVICE
cloud-init ens3	fab1e257-61a6-3526-abf9-8c50c83a28c7	ethernet	ens3
Wired connection 1	e5a48013-c870-3a20-9ad2-b5d13d9603f2	ethernet	--
lo	e4d720f0-e59d-451c-b4ef-5830ac3bc3f9	loopback	lo

2.2. Identify the interface name and the current configuration method that the `Wired connection 1` connection uses.

```
[student@servera ~]$ nmcli -f connection.interface-name,ipv4.method \
con show 'Wired connection 1'
```

connection.interface-name:	ens4
ipv4.method:	auto

3. As the root user, modify the `Wired connection 1` profile to use the static IP address of `172.24.250.30/24`, replacing the default DHCP configuration.

3.1. Switch to the root user. Use `student` as the password.

```
[student@servera ~]$ sudo -i
[sudo] password for student: student
[root@servera ~]#
```

3.2. Modify the `Wired connection 1` profile to use the static IP address of `172.24.250.30/24`.

```
[root@servera ~]# nmcli con mod 'Wired connection 1' ipv4.method manual \
ipv4.addresses 172.24.250.30/24
```

- 4.** Because of the previous step, NetworkManager created a persistent profile at `/etc/NetworkManager/system-connections/Wired connection 1.nmconnection`. Note that NetworkManager creates this default connection with spaces in the file name. To copy this file name into your terminal, either use the surrounding quotation marks or use the backslashes before the spaces, but not both.

Using your preferred text editor, open this file and add `10.0.1.1/24` as a secondary IP address.

```
...output omitted...
[ipv4]
address1=172.24.250.30/24
method=manual
address2=10.0.1.1/24
...output omitted...
```

Save and close the file when you are finished.

- 5.** Activate the new network address with the `nmcli` command.

5.1. Inform NetworkManager of the changes by reloading connections.

```
[root@servera ~]# nmcli con reload
```

5.2. Re-activate the `Wired connection 1` connection.

```
[root@servera ~]# nmcli con up "Wired connection 1"
Connection successfully activated (D-Bus active path:
/org/freedesktop/NetworkManager/ActiveConnection/4)
```

5.3. Verify that the new IP address is active in the runtime configuration.

```
[root@servera ~]# ip -br addr show ens4
ens4      UP      172.24.250.30/24 10.0.1.1/24 fe80::5054:ff:fe01:fa1e/64
```

6. Return to the workstation machine as the student user.

```
[root@servera ~]# exit
logout
[student@servera ~]$ exit
logout
Connection to servera closed.
student@workstation:~$
```

7. Perform the same procedure on the serverb machine. Configure 172.24.250.40/24 as the primary static address for the ens4 interface. Additionally, ensure that 10.0.1.2/24 is persistently added to the connection profile as a secondary IP address.

- 7.1. Log in to the serverb machine as the student user and switch to the root user. Use student as the password.

```
student@workstation:~$ ssh student@serverb
...output omitted...
[student@serverb ~]$ sudo -i
[sudo] password for student: student
[root@serverb ~]#
```

- 7.2. Identify the interface name and the configuration method that the Wired connection 1 profile uses.

```
[root@serverb ~]# nmcli -f connection.interface-name,ipv4.method \
con show 'Wired connection 1'
connection.interface-name:      ens4
ipv4.method:                    auto
```

- 7.3. Modify the profile to use a static IP address of 172.24.250.40/24, replacing the default DHCP configuration.

```
[root@serverb ~]# nmcli con mod 'Wired connection 1' ipv4.method manual \
ipv4.addresses 172.24.250.40/24
```

- 7.4. Edit the Wired connection 1.nmconnection configuration file at the /etc/NetworkManager/system-connections directory. Add 10.0.1.2/24 as a

secondary IP address.

```
...output omitted...
[ipv4]
address1=172.24.250.40/24
method=manual
address2=10.0.1.2/24
...output omitted...
```

Save and close the file when you are finished.

7.5. Use the `nmcli` command to prompt NetworkManager to reread its configuration files.

```
[root@serverb ~]# nmcli con reload
```

7.6. Activate the modified connection.

```
[root@serverb ~]# nmcli con up 'Wired connection 1'
Connection successfully activated (D-Bus active path:
/org/freedesktop/NetworkManager/ActiveConnection/5)
```

7.7. Verify that the new IP address is active.

```
[root@serverb ~]# ip -br addr show ens4
ens4      UP      172.24.250.40/24 10.0.1.2/24 fe80::5054:ff:fe01:fa28/64
```

8. Test connectivity between the servera and serverb machines by using the new network addresses. Ping the new address of the servera machine.

```
[root@serverb ~]# ping -c3 10.0.1.1
PING 10.0.1.1 (10.0.1.1) 56(84) bytes of data.
64 bytes from 10.0.1.1: icmp_seq=1 ttl=64 time=1.25 ms
64 bytes from 10.0.1.1: icmp_seq=2 ttl=64 time=0.483 ms
64 bytes from 10.0.1.1: icmp_seq=3 ttl=64 time=0.413 ms

--- 10.0.1.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2056ms
rtt min/avg/max/mdev = 0.413/0.715/1.250/0.379 ms
```

9. Return to the workstation machine as the student user.

```
[root@serverb ~]# exit
logout
[student@serverb ~]$ exit
logout
Connection to serverb closed.
student@workstation:~$
```

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish net-edit
```

18.5. Configuring Hostnames and Name Resolution

Objectives

- Configure a server's static hostname and its name resolution, and test the results.

Updating the System Hostname

The *fully qualified domain hostname (FQDN)* uniquely identifies each server in the network. The `hostname` command displays or temporarily modifies the system's FQDN.

In the following example, you temporarily change the `host.example.com` hostname to `web1.example.com`:

```
root@host:~# hostname
host.example.com
root@host:~# hostname web1.example.com
root@host:~# hostname
web1.example.com
```

Using the `hostname` command results in a temporary hostname change. If you reboot the system, then the FQDN returns to its previous definition.

To set a persistent hostname, it must be specified in the `/etc/hostname` file, which is read when the system starts. You can manually add the required hostname to the file, or you can use the `hostnamectl` command. The `hostnamectl` command offers a convenient way to both modify this file and to display the system's fully qualified hostname.

You can use the `hostnamectl hostname` command to view the current hostname, or use the same command with an argument to update the hostname to the provided name. In the next example, you persistently change the `host.example.com` hostname to `db1.example.com`:

```
root@host:~# hostnamectl hostname
host.example.com
root@host:~# hostnamectl hostname db1.example.com
root@host:~# hostnamectl hostname
db1.example.com
```

You can verify that the `/etc/hostname` file is updated with the new hostname:

```
root@host:~# cat /etc/hostname
db1.example.com
```

Note

In these examples, the prompt shows the previous hostname even after an update, because the shell uses the environment variables that were set when it first started. To see the new hostname reflected in your prompt, you can either open a new terminal session or manually update your shell's environment variables.

In situations where the `/etc/hostname` file does not exist or it is empty, the system automatically sets a transient hostname based on a reverse DNS lookup on the IP address that is assigned to its network interface.

```
root@host:~# hostnamectl
  Static hostname: (unset)
Transient hostname: host.example.com
...output omitted...
```

Important

In Red Hat Enterprise Linux 7 and later versions, the static hostname is stored in the `/etc/hostname` file. Red Hat Enterprise Linux 6 and earlier versions store the hostname as a variable in the `/etc/sysconfig/network` file.

Configuring Name Resolution

A stub resolver is a lightweight DNS client that runs on your local machine. Its primary job is to receive DNS queries from applications and to forward those queries to a recursive DNS name server if the stub resolver cannot resolve the query. The order of name resolution is configured in the `/etc/nsswitch.conf` file. By default, the system attempts to resolve the query by first using the `/etc/hosts` file.

```

root@host:~# cat /etc/hosts
127.0.0.1          localhost localhost.localdomain localhost4
localhost4.localdomain4
::1               localhost localhost.localdomain localhost6
localhost6.localdomain6
172.25.254.254    classroom.example.com classroom
172.25.254.254    content.example.com content

```

If the `/etc/hosts` file does not contain a matching entry, then the stub resolver queries a DNS name server to look up the hostname. The `/etc/resolv.conf` file controls how this DNS query is performed:

- `search` : A list of domain names to append when short hostnames are looked up.
- `nameserver` : The IP address of a name server to query. Up to three name server directives can be given to provide backups if one name server is down.

```

root@host:~# cat /etc/resolv.conf
# Generated by NetworkManager
search example.com
nameserver 172.25.250.100

```

NetworkManager uses DNS settings stored in connection profiles to update the `/etc/resolv.conf` file. Use the `nmcli` command to modify connections:

```

root@host:~# nmcli con mod 'Wired connection 1' ipv4.dns 172.25.250.220
root@host:~# cat /etc/NetworkManager/system-connections/Wired\ connection\
1.nmconnection
...output omitted...
[ipv4]
...output omitted...
dns=172.25.250.220;
...output omitted...
root@host:~# nmcli con up 'Wired connection 1'

```

By default, the `nmcli con mod ID ipv4.dns IP` command overwrites existing name server entries with the provided IP address. To add or remove a specific DNS server, prepend the `ipv4.dns` option with a plus (+) or minus (-) character, respectively.


```

root@host:~# nmcli con mod 'Wired connection 1' +ipv4.dns 172.25.254.254
root@host:~# nmcli con up 'Wired connection 1'
root@host:~# cat /etc/resolv.conf
# Generated by NetworkManager
search example.com
nameserver 172.25.250.220
nameserver 172.25.254.254

```

In the following example, the DNS server with an IPv6 address of `2001:4860:4860::8888` is added to the list of name servers stored in the `static-ens4` profile:

```

root@host:~# nmcli con mod static-ens4 +ipv6.dns 2001:4860:4860::8888

```

On a dual-stack system, ensure that you list at least one reachable IPv4 name server and one IPv6 name server to maintain connectivity during issues with either protocol stack.

Important

NetworkManager automatically rewrites the `/etc/resolv.conf` file on DHCP-enabled connections unless you configure the `ignore-auto-dns yes` setting in the connection profile by using the `nmcli` command.

```

root@host:~# nmcli con mod "static-ens3" ipv4.ignore-auto-dns yes

```

Testing DNS Name Resolution

The host `HOSTNAME` command helps to diagnose DNS server connectivity by attempting to resolve the specified hostname.

```

root@host:~# host servera.lab.example.com
servera.lab.example.com has address 172.25.250.10
root@host:~# host 172.25.250.10
10.250.25.172.in-addr.arpa domain name pointer servera.lab.example.com.

```

The `dig` command is a more powerful alternative to the basic `host` command for querying DNS servers. The `dig` command supports a wider range of options and provides more detailed output.

```

root@host:~# dig servera.lab.example.com

; <<>> DiG 9.18.33 <<>> servera.lab.example.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 15579
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 2f004a94e9e7cc4c010000006827613b8bbbd0e05e77cf14 (good)
;; QUESTION SECTION:
;servera.lab.example.com.      IN      A

;; ANSWER SECTION:
servera.lab.example.com. 86400 IN      A      172.25.250.10

;; Query time: 2 msec
;; SERVER: 172.25.250.220#53(172.25.250.220) (UDP)
;; WHEN: Fri May 16 16:00:59 UTC 2025
;; MSG SIZE rcvd: 96

```

Neither the `host` nor the `dig` commands consider hostname entries in the `/etc/hosts` file. These are lookup tools that connect to DNS servers directly.

You can use the `getent hosts` command to test hostname resolution by using standard C library calls, which follows the same procedure as other applications on your system. Keep in mind that the default configuration prioritizes checking the `/etc/hosts` file before resorting to a DNS server lookup.

```

root@host:~# getent hosts servera.lab.example.com
172.25.250.10 servera.lab.example.com

```

References

`nmcli(1)`, `hostnamectl(1)`, `hosts(5)`, `getent(1)`, `host(1)`, `dig(1)`, `nsswitch.conf(5)`, `nss-myhostname(8)`, and `resolv.conf(5)` man pages

For more information, refer to the *Configuring and Managing Networking* guide at https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html-single/configuring_and_managing_networking/index

18.6. Guided Exercise

Configure Hostnames and Name Resolution

Configure a server's static hostname and its name resolution, and test the results.

Outcomes

- Set a customized hostname.
- Configure name resolution settings.

Prerequisites

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
student@workstation:~$ lab start net-hostnames
```

Instructions

1. Verify the current hostname settings on the servera machine.

1.1. From the workstation machine, log in to the servera machine as the student user and switch to the root user. Use student as the password.

```
student@workstation:~$ ssh student@servera
...output omitted...
[student@testa ~]$ sudo -i
[sudo] password for student: student
[root@testa ~]#
```

Note that the prompt shows the hostname as testa.

1.2. Display the current hostname.

```
[root@testa ~]# hostname
testa
```

1.3. Display the hostname status by using the `hostnamectl` command.

```
[root@testa ~]# hostnamectl status
  Static hostname: servera
Transient hostname: testa
...output omitted...
```

The output shows `testa` as a transient hostname and `servera` as a static hostname. In this case, the `testa` hostname was defined by a DNS or DHCP service, or was temporarily assigned with the `hostname` command.

The transient hostname overrides the static hostname, which is why `testa` displays in the command prompt.

2. Set the hostname to `servera` to remove the transient hostname.

2.1. Use the `hostnamectl` command to configure the hostname. This change is saved persistently.

```
[root@testa ~]# hostnamectl hostname servera
```

2.2. View the contents of the `/etc/hostname` file, which sets the hostname when the system starts.

```
[root@testa ~]# cat /etc/hostname
servera
```

2.3. Log out and log back in to `servera` as the `student` user to see the updated hostname in the shell prompt. Switch to the root user. Use `student` as the password.

```
[root@testa ~]# exit
logout
[student@testa ~]$ exit
logout
Connection to servera closed.
student@workstation:~$ ssh student@servera
...output omitted...
[student@servera ~]$ sudo -i
[sudo] password for student: student
[root@servera ~]#
```

2.4. Display the hostname status by using the `hostnamectl` command.

```
[root@servera ~]# hostnamectl status
Static hostname: servera
...output omitted...
```

Because a static hostname is now configured, the transient hostname no longer displays.

3. Temporarily change the hostname to testname.

3.1. Change the hostname by using the hostname command.

```
[root@servera ~]# hostname testname
```

3.2. Display the current hostname.

```
[root@servera ~]# hostname
testname
```

3.3. View the content of the /etc/hostname file.

```
[root@servera ~]# cat /etc/hostname
servera
```

The /etc/hostname file contains the static hostname definition. Because the hostname was changed temporarily, the /etc/hostname file is not updated.

3.4. To test the temporary hostname change, reboot the system to see whether the testname hostname persists.

```
[root@servera ~]# systemctl reboot
Connection to servera closed by remote host.
Connection to servera closed.
student@workstation:~$
```

3.5. Log in to the servera machine as the student user and switch to the root user. Use student as the password.

```
student@workstation:~$ ssh student@servera
...output omitted...
[student@servera ~]$ sudo -i
[sudo] password for student: student
[root@servera ~]#
```

3.6. Display the current hostname.

```
[root@servera ~]# hostname
servera
```

The testname hostname was set temporarily, so it does not persist after the system reboot. The hostname command now shows the servera static hostname.

4. Add `class` as a local hostname alias for the `classroom` server, and ensure that you can ping the server by using that alias.

4.1. Look up the IP address of the `classroom.example.com` server.

```
[root@servera ~]# host classroom.example.com
classroom.example.com has address 172.25.254.254
```

- 4.2. Update the `/etc/hosts` file by appending the `class` hostname alias to the `classroom` line with the following contents:

```
127.0.0.1    localhost localhost.localdomain localhost4 localhost4.localdomain4
::1         localhost localhost.localdomain localhost6 localhost6.localdomain6
172.25.254.254 classroom.example.com classroom class
```

Save and close the file when you are finished.

- 4.3. Look up the IP address of the `class` server by using the `host` command.

```
[root@servera ~]# host class
Host class not found: 3(NXDOMAIN)
```

The `host` command queries the DNS server directly, and it was not able to resolve the short, unqualified hostname.

- 4.4. Look up the IP address of the `class` server by using the `getent` command.

```
[root@servera ~]# getent hosts class
172.25.254.254 classroom.example.com classroom class
```

The `getent hosts` command uses the standard order of name resolution, which takes into account the entries in the `/etc/hosts` file.

4.5. Use the `ping` command to test connectivity to the `class` server.

```
[root@servera ~]# ping -c3 class
PING classroom.example.com (172.25.254.254) 56(84) bytes of data.
64 bytes from classroom.example.com (172.25.254.254): icmp_seq=1 ttl=64
time=0.373 ms
64 bytes from classroom.example.com (172.25.254.254): icmp_seq=2 ttl=64
time=0.314 ms
64 bytes from classroom.example.com (172.25.254.254): icmp_seq=3 ttl=64
time=0.272 ms

--- classroom.example.com ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2028ms
rtt min/avg/max/mdev = 0.272/0.319/0.373/0.041 ms
```

5. Return to the workstation machine as the student user.

```
[root@servera ~]# exit
logout
[student@servera ~]$ exit
logout
Connection to servera closed.
student@workstation:~$
```

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish net-hostnames
```


18.7. Lab

Manage Network Configuration

Configure network interfaces and settings on Red Hat Enterprise Linux servers.

Outcomes

- Configure two static IPv4 addresses for the primary network interface.

Prerequisites

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
student@workstation:~$ lab start net-review
```

Instructions

1. Log in to the serverb machine as the student user and switch to the root user. Use student as the password.
2. Create a connection that uses a static network configuration by using the information from the following table.

Parameter	Setting
Connection name	custom-profile
Interface name	Use the interface with the 52:54:00:00:fa:0b MAC address.
IP address	172.25.250.11/24
Gateway address	172.25.250.254
DNS address	172.25.250.254

 Important

The name of the network interface that is associated with the provided MAC address might vary according to the course platform and hardware in use. This activity uses the `ens3` interface name.

Locate the default connection name for the classroom (such as `System-ens3`) that is associated with the MAC address provided in the table. Use this connection name to replace the `System-ens3` placeholder throughout this exercise if different.

3. Configure the `custom-profile` connection to start automatically. For all other Ethernet connections, turn off the `autoconnection` setting.
4. Add the `172.25.250.150/24` IP address as a secondary address to the `custom-profile` connection. Verify that the new configuration is applied to the connection.
5. Reboot the `serverb` machine.
6. On the workstation machine, verify that the `serverb` machine is initialized and that it receives ICMP packages. The `serverb` machine should start after a minute or so.
7. On the workstation machine as the `root` user, add an entry to the `/etc/hosts` file that maps the `172.25.250.150` IP address to the `serverb-secondary` name. The secondary IP address of the `serverb` machine must be resolved by that name from the workstation machine.

Evaluation

As the student user on the workstation machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

```
student@workstation:~$ lab grade net-review
```

Finish

On the workstation machine, change to the student user home directory and use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish net-review
```

Solution

Configure network interfaces and settings on Red Hat Enterprise Linux servers.

Outcomes

- Configure two static IPv4 addresses for the primary network interface.

Prerequisites

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
student@workstation:~$ lab start net-review
```

Instructions

1. Log in to the serverb machine as the student user and switch to the root user. Use student as the password.

```
student@workstation:~$ ssh student@serverb
...output omitted...
[student@serverb ~]$ sudo -i
[sudo] password for student: student
[root@serverb ~]#
```

2. Create a connection that uses a static network configuration by using the information from the following table.

Parameter	Setting
Connection name	custom-profile
Interface name	Use the interface with the 52:54:00:00:fa:0b MAC address.
IP address	172.25.250.11/24

Parameter	Setting
Gateway address	172.25.250.254
DNS address	172.25.250.254

 Important

The name of the network interface that is associated with the provided MAC address might vary according to the course platform and hardware in use. This activity uses the `ens3` interface name.

Locate the default connection name for the classroom (such as `System-ens3`) that is associated with the MAC address provided in the table. Use this connection name to replace the `System-ens3` placeholder throughout this exercise if different.

2.1. List the network interfaces and determine which interface is associated with the `52:54:00:00:fa:0b` MAC address.

```
[root@serverb ~]# ip link
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT
   group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: ens3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode
   DEFAULT group default qlen 1000
    link/ether 52:54:00:00:fa:0b brd ff:ff:ff:ff:ff:ff
    altname enp0s3
    altname enx52540000fa0b
3: ens4: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 8942 qdisc fq_codel state UP mode
   DEFAULT group default qlen 1000
    link/ether 52:54:00:01:fa:28 brd ff:ff:ff:ff:ff:ff
    altname enp0s4
    altname enx52540001fa28
```

The `ens3` interface uses the `52:54:00:00:fa:0b` MAC address.

2.2. Create the custom-profile connection profile based on the table information in the instructions. Associate the profile with the network interface that uses the `52:54:00:00:fa:0b` MAC address.

```
[root@serverb ~]# nmcli con add con-name custom-profile \
ifname ens3 \
type ethernet \
ipv4.method manual \
ipv4.dns '172.25.250.254' \
ipv4.addresses '172.25.250.11/24' \
ipv4.gateway '172.25.250.254'
Connection 'custom-profile' (7aff9d3d-aeb9-4ac9-ba51-cac24f660687) successfully
added.
```

3. Configure the custom-profile connection to start automatically. For all other Ethernet connections, turn off the autoconnection setting.

3.1. Find other Ethernet connections.

```
[root@serverb ~]# nmcli con
```

NAME	UUID	TYPE	DEVICE
System-ens3	fab1e257-61a6-3526-abf9-8c50c83a28c7	ethernet	ens3
lo	0ef452de-b94d-431f-86f8-f08e85fad57f	loopback	lo
Wired connection 1	07284cd0-41c1-360e-bacd-477319a8e90e	ethernet	--
custom-profile	7aff9d3d-aeb9-4ac9-ba51-cac24f660687	ethernet	--

In addition to the custom-profile connection, the nmcli command returns the System-ens3 and Wired connection 1 connections.

- 3.2. Review the System-ens3 connection and turn off the autoconnect setting if needed.

```
[root@serverb ~]# nmcli con show System-ens3
connection.id: System-ens3
...output omitted...
connection.autoconnect: yes
...output omitted...
[root@serverb ~]# nmcli con mod System-ens3 connection.autoconnect no
```

- 3.3. Review the Wired connection 1 connection and turn off the autoconnect setting if needed.

```
[root@serverb ~]# nmcli con show "Wired connection 1"
connection.id:                Wired connection 1
...output omitted...
connection.autoconnect:       yes
...output omitted...
[root@serverb ~]# nmcli con mod "Wired connection 1" connection.autoconnect no
```

4. Add the 172.25.250.150/24 IP address as a secondary address to the custom-profile connection. Verify that the new configuration is applied to the connection.

- 4.1. Add the 172.25.250.150/24 IP address as a secondary address to the custom-profile connection.

```
[root@serverb ~]# nmcli con mod "custom-profile" \
+ipv4.addresses '172.25.250.150/24'
```

- 4.2. Review the custom-profile connection. The secondary IP address should now be listed.

```
[root@serverb ~]# nmcli con show custom-profile
connection.id:                custom-profile
...output omitted...
ipv4.addresses:               172.25.250.11/24, 172.25.250.150/24
ipv4.gateway:                 172.25.250.254
...output omitted...
```

5. Reboot the serverb machine.

```
[root@serverb ~]# systemctl reboot
...output omitted...
student@workstation:~$
```

6. On the workstation machine, verify that the serverb machine is initialized and that it receives ICMP packages. The serverb machine should start after a minute or so.

```
student@workstation:~$ ping -c3 serverb
PING serverb.lab.example.com (172.25.250.11) 56(84) bytes of data.
64 bytes from serverb.lab.example.com (172.25.250.11): icmp_seq=1 ttl=64
time=0.666 ms
64 bytes from serverb.lab.example.com (172.25.250.11): icmp_seq=2 ttl=64
time=0.417 ms
64 bytes from serverb.lab.example.com (172.25.250.11): icmp_seq=3 ttl=64
time=0.513 ms

--- serverb.lab.example.com ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2052ms
rtt min/avg/max/mdev = 0.417/0.532/0.666/0.102 ms
```

7. On the workstation machine as the root user, add an entry to the `/etc/hosts` file that maps the `172.25.250.150` IP address to the `serverb-secondary` name. The secondary IP address of the `serverb` machine must be resolved by that name from the workstation machine.

7.1. Switch to the root user. Use `student` as the password.

```
student@workstation:~$ sudo -i
[sudo] password for student: student
root@workstation:~#
```

- 7.2. Edit the `/etc/hosts` file and include the `172.25.250.150 serverb-secondary` line at the end of the file. The following example shows the expected contents of the `/etc/hosts` file:

```
127.0.0.1    localhost localhost.localdomain localhost4 localhost4.localdomain4
::1         localhost localhost.localdomain localhost6 localhost6.localdomain6
172.25.254.254 classroom.example.com classroom
172.25.254.254 content.example.com content
172.25.254.254 materials.example.com materials
172.25.250.9 workstation.lab.example.com workstation
172.25.250.150 serverb-secondary
```

Save and close the file when you are finished.

- 7.3. Exit from the root user.

```
root@workstation:~# exit
logout
```

7.4. Verify that you can reach the serverb secondary IP address with the serverb-secondary name.

```
student@workstation:~$ ping -c3 serverb-secondary
PING serverb-secondary (172.25.250.150) 56(84) bytes of data.
64 bytes from serverb-secondary (172.25.250.150): icmp_seq=1 ttl=64 time=0.385
ms
64 bytes from serverb-secondary (172.25.250.150): icmp_seq=2 ttl=64 time=0.381
ms
64 bytes from serverb-secondary (172.25.250.150): icmp_seq=3 ttl=64 time=0.345
ms

--- serverb-secondary ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2084ms
rtt min/avg/max/mdev = 0.345/0.370/0.385/0.018 ms
```

Evaluation

As the student user on the workstation machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

```
student@workstation:~$ lab grade net-review
```

Finish

On the workstation machine, change to the student user home directory and use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish net-review
```


18.8. Summary

In this lesson, you learned to manage network settings by using the NetworkManager daemon, and to manage network devices and connections by using the NetworkManager command-line interface (`nmcli`). RHEL 10 uses an INI-style keyfile format to store network profiles in the `/etc/NetworkManager/system-connections` directory. Support for the earlier `ifcfg` file format for network connections is fully removed.

You can uniquely identify servers with an FQDN, which you can manage by using the `hostnamectl` command. You also learned to configure and to test local name resolution.

Chapter 19.

Configuring and Securing SSH

Goal

Protect SSH communication by managing host keys and by implementing key-based authentication for users.

Sections

- Managing SSH Host Keys (and Guided Exercise)
- Configuring SSH Key-based Authentication (and Guided Exercise)

Lab

- Configure and Secure SSH

19.1. Managing SSH Host Keys

Objectives

- Prevent impersonating attacks by configuring the SSH service of client systems to use trusted host keys.

The Role of SSH Host Keys

SSH (Secure Shell) creates a secure communication channel primarily through public-key encryption. When an SSH client attempts to connect to an SSH server, the server first sends a copy of its public key to the client. The server's public key is important because the client uses this host key to authenticate the server's identity.

When you run the `ssh` command, your client checks for a record of the server's public key in its local *known hosts file*. This file can be centrally managed in `/etc/ssh/ssh_known_hosts` or, more commonly, stored in your user's home directory at `~/.ssh/known_hosts`.

If your client already has a copy of the server's public key, then the `ssh` command compares the key that it received from the server with the locally stored one. If these keys do not match, then the client assumes a potential security risk, such as an *interceptor* or *man-in-the-middle* attack, and refuses to connect to the host. This strict verification process is a critical layer of security because it prevents your system from unknowingly connecting to a compromised or malicious server.

After the server's identity is successfully verified (or accepted by the user), the connection is deemed trustworthy enough to proceed to the next critical stage: the key exchange. During the key exchange (often by using the Diffie-Hellman method), a temporary, shared secret session key is generated for faster, symmetric encryption of all subsequent communications.

Strict Host Key Checking on Clients

For security, the SSH client's default setting is to require manual confirmation before a new host key is accepted and stored.

```

user@host:~$ ssh hostb
The authenticity of host 'hostb (192.168.250.11)' can't be established.
ED25519 key fingerprint is SHA256:Qit7NeebQCgP2otXSaVXbw1rGmUVMlybXyV68qaND/M.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? no
Host key verification failed.
user@host:~$

```

However, you can adjust this behavior by setting the `StrictHostKeyChecking` option, either in a configuration file or from the command line.

Configure Host Key Checking with Configuration Files

You can customize how your SSH client handles host keys by editing one of two configuration files:

- The `/etc/ssh/ssh_config` file configures system-wide SSH client settings.
- The `~/.ssh/config` file configures user-specific SSH client settings.

Keep in mind that any settings in your personal `~/.ssh/config` file override those settings in the system-wide configuration.

To control how your SSH client handles host keys, modify the `StrictHostKeyChecking` parameter in your preferred configuration file. Set the parameter to one of these options:

yes

This setting is the most secure and offers the strongest protection against interceptor attacks. If the server's public key does not match the one that is stored in your `known_hosts` file, then your SSH client always ends the connection. With the `yes` option, you must also manually add new host keys to your `known_hosts` file, because SSH does not do it automatically. Although this option is highly secure, it can be inconvenient due to the manual key management.

accept-new

With this option, if you connect to a new host, then the `ssh` command automatically adds its host key to the `known_hosts` file without prompting. However, if a host's key changed (if it is different from the existing one in your `known_hosts` file), then the `ssh` command fails to connect, and prevents potential security risks.

no

This setting offers the least security. With this option, SSH automatically adds new host keys to your `known_hosts` file without prompting. Crucially, this option also allows connections to

proceed even if a host's key changed, which significantly increases your vulnerability to interceptor attacks.

ask

This setting is the default and is often recommended because it gives a good balance between security and convenience. When connecting to a new host, the `ssh` command prompts you for confirmation before adding its key to your `known_hosts` file. If a host's key changed, then SSH refuses to connect, and alerts you to a potential issue, to prevent a compromised connection.

Configure Host Key Checking from the Command Line

You can also set the `StrictHostKeyChecking` parameter from the command line by specifying the `-o StrictHostKeyChecking` option when you run the `ssh` command. The option uses the same settings as the configuration file parameter.

Command-line options always take precedence over settings in both your personal `~/.ssh/config` file and the system-wide `/etc/ssh/ssh_config` file. This precedence means that you can temporarily adjust host key checking behavior for a single connection without altering your permanent configuration.

For example, to connect to a new host and to automatically accept its host key (which is generally discouraged for security reasons unless you are certain of the host's authenticity), you could use the following command:

```
user@host:~$ ssh -o StrictHostKeyChecking=accept-new hostb
Warning: Permanently added 'hostb' (ED25519) to the list of known hosts.
user@hostb's password: password
...output omitted...
user@hostb:~$
```

Verifying SSH Host Key Fingerprints

To ensure the authenticity of a target SSH server and to protect against interceptor attacks, the client must verify the host's public key. However, SSH public keys are long, complex strings of characters that are typically Base64-encoded. Trying to manually compare or verify a key against a trusted source is error-prone and impractical.

An SSH host key fingerprint is a short, cryptographic hash of the server's public key that enables the SSH client (and user) to confirm that they are connecting to the intended, legitimate server and not to an imposter. Fingerprints are easier for a human to visually inspect and compare.

Because you cannot connect via SSH yet, this verification must be performed by logging in to the target host locally and using an alternative communication method to compare the fingerprints (for example, in a phone call or an email).

On the SSH server, use the `ssh-keygen` command to display its public key fingerprint:

```
user@host:~$ ssh-keygen -l -f /etc/ssh/ssh_host_ed25519_key.pub
256 SHA256:Qit7NeebQCgP2otXSaVXbw1rGmUVMlybXyV68qaND/M root@hostb (ED25519)
```

You might have to adjust the path to the public key file if your server uses a different algorithm such as RSA or ECDSA.

After you verify the server's public key fingerprint, you must securely communicate this fingerprint to the client-side administrator by using a trusted alternative communication method, such as a phone call or video conference.

The client-side administrator must compare this received fingerprint with the one that the `ssh` command presents when they first attempt to connect. If the fingerprints match, then the server's identity is confirmed and the connection can proceed. If the fingerprints do not match, then the connection should be stopped because of a potential security risk.

In the following example, the user must carefully compare the fingerprint of the SSH server with a trusted source, before typing `yes` to proceed with the connection.

```
user@host:~$ ssh hostb
The authenticity of host 'hostb (192.168.250.11)' can't be established.
ED25519 key fingerprint is SHA256:Qit7NeebQCgP2otXSaVXbw1rGmUVMlybXyV68qaND/M.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'hostb' (ED25519) to the list of known hosts.
user@hostb's password: password
...output omitted...
user@hostb:~$
```

Regenerating SSH Host Keys on Servers

You might want to regenerate SSH host keys for security reasons, such as if you suspect that a key is compromised or if you are creating a new server image and need unique identifiers for each instance.

Starting in Red Hat Enterprise Linux 10, the SSH server by default uses three pairs of keys and the ED25519, ECDSA, and RSA host key algorithms. These host key files are in the `/etc/ssh/` directory

on the server:

```
user@host:~$ ls -l /etc/ssh/ssh_host_*
-rw-----. 1 root root 480 Jun 13 16:56 /etc/ssh/ssh_host_ecdsa_key
-rw-r--r--. 1 root root 162 Jun 13 16:56 /etc/ssh/ssh_host_ecdsa_key.pub
-rw-----. 1 root root 387 Jun 13 16:56 /etc/ssh/ssh_host_ed25519_key
-rw-r--r--. 1 root root 82 Jun 13 16:56 /etc/ssh/ssh_host_ed25519_key.pub
-rw-----. 1 root root 2578 Jun 13 16:56 /etc/ssh/ssh_host_rsa_key
-rw-r--r--. 1 root root 554 Jun 13 16:56 /etc/ssh/ssh_host_rsa_key.pub
```

If you need to generate new SSH host key pairs, then delete the existing key pairs and restart the SSH daemon. The `sshd.service` unit automatically detects the absence of these keys during service start and regenerates all three key pairs.

```
root@host:~# rm -f /etc/ssh/ssh_host_*
root@host:~# systemctl restart sshd.service
```

Note

On systems that use the Cloud-init package, Cloud-init is typically responsible for managing SSH host key generation as part of its instance provisioning process. As a result, SSH host key pairs might not regenerate automatically, or Cloud-init might overwrite any manually generated keys during subsequent provisioning steps.

Managing Known Host Files on Clients

After regenerating host keys, for any SSH client that previously connected to your server, the earlier host key is still stored in its `known_hosts` file. When these clients attempt to connect again, they receive a warning about a potential interceptor attack because the server's identity no longer matches what the client expects.

```

user@host:~$ ssh user@hostb
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
@    WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!    @
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
IT IS POSSIBLE THAT SOMEONE IS DOING SOMETHING NASTY!
Someone could be eavesdropping on you right now (man-in-the-middle attack)!
It is also possible that a host key has just been changed.
The fingerprint for the ED25519 key sent by the remote host is
SHA256:0sQuJcXztSoxwi+2hNrj6iRYQpjq2kXSCzcZA8m5Izs.
Please contact your system administrator.
Add correct host key in /home/user/.ssh/known_hosts to get rid of this message.
Offending ECDSA key in /home/user/.ssh/known_hosts:3
Host key for hostb has changed and you have requested strict checking.
Host key verification failed.
user@host:~$

```

SSH clients keep a list of known remote systems and their corresponding host keys in one of two locations:

The `/etc/ssh/ssh_known_hosts` file

This system-wide file is not present by default; a system administrator must create it and add trusted host keys to it. It stores public keys for hosts that are known to the entire system, which means that it benefits all users on the system. By prepopulating this file, administrators help users to avoid the dilemma of trusting a new, unknown host key, which is especially beneficial in large environments.

The `~/.ssh/known_hosts` file

This user-specific file is in the home directory of each user. If the SSH client does not find a remote system's public key in the system-wide known hosts file, then it then looks for the key in the user's file. By default, SSH automatically adds a server's host key to this file the first time that a user connects and accepts it. Users can, however, configure their SSH client to prevent this automatic addition.

If a remote server's host key changes, then you can remove the outdated entry from your `~/.ssh/known_hosts` file. You can update this file by using a text editor to delete the line that corresponds to the changed host, or by using the `ssh_keygen` utility, which is the safer option.

For example, to remove a key entry for `hostb`, you can run the command with the `-R` option:


```

user@host:~$ ssh-keygen -R hostb
# Host hostb found: line 1
# Host hostb found: line 2
# Host hostb found: line 3
/home/user/.ssh/known_hosts updated.
Original contents retained as /home/user/.ssh/known_hosts.old

```

If the offending key is present in the system-wide known hosts file, then the administrator must delete the matching entry:

```

root@host:~# ssh-keygen -R hostb -f /etc/ssh/ssh_known_hosts
# Host hostb found: line 1
# Host hostb found: line 2
# Host hostb found: line 3
/etc/ssh/ssh_known_hosts updated.
Original contents retained as /etc/ssh/ssh_known_hosts.old

```

The previous command identified three matching key entries for the host and deleted the lines.

Populating Known Host Files

The `ssh-keyscan` command is a valuable tool for collecting the public host keys of remote servers. You can use this command to manually populate `known_hosts` files, which is particularly useful for scripting, automation, or setting up a system-wide `ssh_known_hosts` file.

To retrieve and save the public key of a remote host, append the output of the `ssh-keyscan` command to your personal known hosts file:

```

user@host:~$ ssh-keyscan hostb >> ~/.ssh/known_hosts

```

By default, the command saves all key pairs that the server might have presented. You can verify the fingerprint of your stored host keys by using the `ssh-keygen` command:

```

user@host:~$ ssh-keygen -l -f ~/.ssh/known_hosts
3072 SHA256:1PpHYtWyiyoQh548NRXxRn6GCMvHSzljrtbAaHKqPc hostb (RSA)
256 SHA256:rMXtr4eS4bCSE6Si81a3v7hNb9RXdMCv8ngjFs7Yj1s hostb (ECDSA)
256 SHA256:89iCusliyLGcq5m0J0nvoTUaa9xkjaKa0G0DgbXiTmk hostb (ED25519)

```

You can also open the known hosts file in a text editor to view complete public keys instead of just their fingerprints.

A system administrator might use the `ssh-keyscan` command to add a new host to the system-wide known hosts file. The following command demonstrates that the tool can also operate in quiet mode (`-q`) and collect specific types (`-t`) of host keys:

```
root@host:~# ssh-keyscan -q -t ed25519 hostb >> /etc/ssh/ssh_known_hosts
```

Another valid scenario is for the system administrator to use the `ssh-keyscan` command to scan an entire network segment for host keys. This approach can be useful for quickly gathering keys from a range of servers in a controlled environment.

For example, you can scan a range of IP addresses to collect RSA host keys:

```
root@host:~# ssh-keyscan -q -t rsa 172.25.250.0/24 >> /etc/ssh/ssh_known_hosts
```

Warning

Although the `ssh-keyscan` command is convenient, you must exercise caution, especially when scanning an entire network. When using the `ssh-keyscan` command, you are trusting the key that is presented by the remote server at that moment. On an untrusted network, where a server's identity might be spoofed, you should verify the retrieved key's fingerprint through an alternative method before adding it to your `known_hosts` file. Directly appending the `ssh-keyscan` command output to a `known_hosts` file without verification can expose you to interceptor attacks.

References

`ssh(1)`, `ssh-keygen(1)`, `ssh-keyscan(1)`, `ssh_config(5)`, `sshd(8)`, and `sshd_config(5)` man pages

For more information, refer to *Using Secure Communications Between Two Systems with OpenSSH* at https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html-single/securing_networks/index#using-secure-communications-between-two-systems-with-openssh

19.2. Guided Exercise

Manage SSH Host Keys

Preconfigure client systems with the SSH host keys of the servers that they use.

Outcomes

- Verify the fingerprints of SSH host keys before logging in to remote hosts.
- Regenerate host key pairs on SSH servers.
- Prepopulate a system-wide known hosts file to help users log in to trusted hosts.

Prerequisites

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
student@workstation:~$ lab start ssh-hostkeys
```

Instructions

1. On the workstation machine, open two terminal windows side by side. In this activity, these terminals are referred to as *left* and *right*.
 - 1.1. In the left terminal, log in to the servera machine as the student user and then switch to the operator1 user. Use redhat as the password.

```
student@workstation:~$ ssh student@servera
...output omitted...
[student@servera ~]$ su - operator1
Password: redhat
[operator1@servera ~]$
```
 - 1.2. In the right terminal, log in to the serverb machine as the student user and switch to the root user. Use student as the password.

```
student@workstation:~$ ssh student@serverb
...output omitted...
[student@serverb ~]$ sudo -i
[sudo] password for student: student
[root@serverb ~]#
```

2. In the left terminal, verify the SSH settings for the `operator1` user, and attempt to log in to the `serverb` machine as the `student` user.

2.1. List the contents of the `~/ .ssh` directory.

```
[operator1@servera ~]$ ls -l ~/.ssh
ls: cannot access '/home/operator1/.ssh': No such file or directory
[operator1@servera ~]$
```

If this directory is absent, then no SSH settings, including trusted hosts, are configured for this user.

- 2.2. Attempt to log in to the `serverb` machine as the `student` user. Because the `operator1` user did not previously log in to the `serverb` machine, the public SSH key of the remote host is not trusted. Cancel the login attempt by typing `no` when prompted.

```
[operator1@servera ~]$ ssh student@serverb
The authenticity of host 'serverb (172.25.250.11)' can't be established.
ED25519 key fingerprint is SHA256:LxmngmgYc8FYihzYWwSorAhgSq0YanKF5oPYi82MH2Q0.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? no
Host key verification failed.
[operator1@servera ~]$
```

Notice that the ED25519 host key fingerprint is displayed to the user by default. The fingerprint might be different on your system.

3. In the right terminal, verify the fingerprint of the ED25519 host key of the `serverb` machine by using the `ssh-keygen` command.

Note the host key fingerprint for use in a future step.

```
[root@serverb ~]# ssh-keygen -l -f /etc/ssh/ssh_host_ed25519_key.pub
256 SHA256:LxmngmgYc8FYihzYWySorAhgSq0YanKF5oPYi82MH2Q0 root@serverb (ED25519)
[root@serverb ~]#
```

The fingerprint might be different on your system.

4. In the left terminal, log in to the serverb machine, and confirm that the serverb machine is added to the operator1 user's known hosts file as a trusted host.
 - 4.1. Log in to the serverb machine as the student user. When prompted, verify and accept the serverb host key fingerprint if it matches the fingerprint that you retrieved in the right terminal. Because the fingerprints match, you can safely proceed with the connection by typing yes. Use student as the password.

Subsequent connection attempts to the serverb machine no longer require this confirmation.

```
[operator1@servera ~]$ ssh student@serverb
The authenticity of host 'serverb (172.25.250.11)' can't be established.
ED25519 key fingerprint is SHA256:LxmngmgYc8FYihzYWySorAhgSq0YanKF5oPYi82MH2Q0.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'serverb' (ED25519) to the list of known hosts.
student@serverb's password: student
...output omitted...
[student@serverb ~]$
```

- 4.2. Return to the servera machine as the operator1 user.

```
[student@serverb ~]$ exit
logout
Connection to serverb closed.
[operator1@servera ~]$
```

- 4.3. Using the ssh-keygen command, verify that the fingerprints of the serverb host keys were added to the operator1 user's known hosts file during the previous login.

```
[operator1@servera ~]$ ssh-keygen -l -f ~/.ssh/known_hosts
256 SHA256:LxmngmgYc8FYihzYWYSorAhgSq0YanKF5oPYi82MH2Q0 serverb (ED25519)
3072 SHA256:UwhwMIROfJj9e/iiB5kczTRLU+FMPfFmY1sZQD8iLCY serverb (RSA)
256 SHA256:FkkFht+Sht7vyU44qE99NLSZeT1wBH6bsXEflzMOD/w serverb (ECDSA)
[operator1@servera ~]$
```

The ssh command added all three public keys of the serverb machine. Each public key represents a different host key algorithm. In this output, the first key matches the fingerprint from the previous step.

- 4.4.** Remotely obtain the hostname of the serverb machine by using the ssh command. Use student as the password.

```
[operator1@servera ~]$ ssh student@serverb 'hostname'
student@serverb's password: student
serverb
[operator1@servera ~]$
```

The ssh client on the servera machine no longer prompts to confirm the serverb machine's host key because it is now a trusted host.

- 5.** Occasionally, administrators must regenerate SSH host keys. In the right terminal, regenerate all SSH host keys that the serverb machine uses.

- 5.1.** Delete all three existing host key pairs by using a wildcard in the shell. Then, restart the sshd service for the server to generate and use new keys.

```
[root@serverb ~]# rm -f /etc/ssh/ssh_host_*
[root@serverb ~]# systemctl restart sshd
[root@serverb ~]#
```

- 5.2.** Verify that the key pairs are regenerated.

```
[root@serverb ~]# ls -l /etc/ssh/ssh_host_*
-rw-----. 1 root root 492 Jun 18 20:30 /etc/ssh/ssh_host_ecdsa_key
-rw-r--r--. 1 root root 162 Jun 18 20:30 /etc/ssh/ssh_host_ecdsa_key.pub
-rw-----. 1 root root 387 Jun 18 20:30 /etc/ssh/ssh_host_ed25519_key
-rw-r--r--. 1 root root 82 Jun 18 20:30 /etc/ssh/ssh_host_ed25519_key.pub
-rw-----. 1 root root 2578 Jun 18 20:30 /etc/ssh/ssh_host_rsa_key
-rw-r--r--. 1 root root 554 Jun 18 20:30 /etc/ssh/ssh_host_rsa_key.pub
[root@serverb ~]#
```

- 5.3.** Verify the fingerprint of the new ED25519 host key.

```
[root@serverb ~]# ssh-keygen -l -f /etc/ssh/ssh_host_ed25519_key.pub
256 SHA256:19V8xuwZzVwtNs5x2YXABmUqz+8pguBikxXZ+X1QCLo no comment (ED25519)
[root@serverb ~]#
```

The fingerprint might be different in your environment. Note its value for your next SSH login.

5.4. Exit from the right terminal SSH session and close the terminal window.

```
[root@serverb ~]# exit
logout
[student@serverb ~]$ exit
logout
Connection to serverb closed.
student@workstation:~$
```

6. When SSH host keys change, you must adjust the SSH client settings accordingly. Following the SSH host key change on the serverb machine, attempt to log in from the servera client and take the necessary steps to re-establish trust.

6.1. In the left terminal, use SSH to remotely run the hostname command on the serverb system as the student user.

```
[operator1@servera ~]$ ssh student@serverb 'hostname'
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
@    WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!    @
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
IT IS POSSIBLE THAT SOMEONE IS DOING SOMETHING NASTY!
Someone could be eavesdropping on you right now (man-in-the-middle attack)!
It is also possible that a host key has just been changed.
The fingerprint for the ED25519 key sent by the remote host is
SHA256:19V8xuwZzVwtNs5x2YXABmUqz+8pguBikxXZ+X1QCLo.
Please contact your system administrator.
Add correct host key in /home/operator1/.ssh/known_hosts to get rid of this
message.
Offending ECDSA key in /home/operator1/.ssh/known_hosts:3
Host key for serverb has changed and you have requested strict checking.
Host key verification failed.
[operator1@servera ~]$
```

The SSH client fails to connect to the serverb machine because its host keys no longer match the ones that are stored in the known hosts file for this user.

6.2. Delete the offending key from the user's known hosts file by using the ssh-keygen command.

```
[operator1@servera ~]$ ssh-keygen -R serverb
# Host serverb found: line 1
# Host serverb found: line 2
# Host serverb found: line 3
/home/operator1/.ssh/known_hosts updated.
Original contents retained as /home/operator1/.ssh/known_hosts.old
[operator1@servera ~]$
```

- 6.3.** Re-attempt the login process by executing the `hostname` command on the `serverb` machine remotely. The host key fingerprint should match the new value that you retrieved in a previous step. Type `yes` to proceed, and use `student` as the password.

```
[operator1@servera ~]$ ssh student@serverb 'hostname'
The authenticity of host 'serverb (172.25.250.11)' can't be established.
ED25519 key fingerprint is SHA256:19V8xuwZzVwtNs5x2YXABmUqz+8pguBikxXZ+X1QCLo.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'serverb' (ED25519) to the list of known hosts.
student@serverb's password: student
serverb
[operator1@servera ~]$
```

- 6.4.** Return to the `student` user session.

```
[operator1@servera ~]$ exit
logout
[student@servera ~]$
```

- 7.** Although the `operator1` user account is configured to re-establish trust, a system-wide known hosts file, which an administrator creates with the `serverb` machine's correct key, would benefit other users on the system. Create the `/etc/ssh/ssh_known_hosts` file on the `servera` client, and add the `serverb` machine's public key to this file.

- 7.1.** Use the `ssh-keyscan` command to retrieve the ED25519 host key of the `serverb` machine.

```
[student@servera ~]$ ssh-keyscan -t ed25519 serverb
# serverb:22 SSH-2.0-OpenSSH_9.9
serverb ssh-ed25519
AAAAC3NzaC1lZDI1NTE5AAAAIEg3+nm5bXhFDbmYzQx2ov6/XVF2Saf4Xv0VcP04cVBH
[student@servera ~]$
```

The key might differ in your environment.

- 7.2.** Save the key in the system-wide known hosts file to help other users on the servera system to identify trusted hosts. Use student for the Sudo password.

```
[student@servera ~]$ sudo bash -c 'ssh-keyscan -t ed25519 \
serverb >> /etc/ssh/ssh_known_hosts'
[sudo] password for student: student
[student@servera ~]$
```

- 7.3.** Retrieve the fingerprint of the host key in the new system-wide known hosts file.

```
[student@servera ~]$ ssh-keygen -l -f /etc/ssh/ssh_known_hosts
256 SHA256:19V8xuwZzVwtNs5x2YXABmUqz+8pguBikxXZ+X1QCLo serverb (ED25519)
[student@servera ~]$
```

- 8.** Verify that the system-wide host key configuration is effective for other users. As the operator2 user, attempt a connection from the servera client to the serverb machine; no host key confirmation prompt should appear.

- 8.1.** Switch to the operator2 user. Use redhat as the password.

```
[student@servera ~]$ su - operator2
Password: redhat
[operator2@servera ~]$
```

- 8.2.** Execute the hostname command remotely on the serverb system as the student user by using the ssh command. Use student as the password.

```
[operator2@servera ~]$ ssh student@serverb 'hostname'
student@serverb's password: student
serverb
[operator2@servera ~]$
```

The operator2 user is not prompted to accept the host key of the serverb machine because it is a trusted host.

- 8.3.** Try to view the user's personal known hosts file.

```
[operator2@servera ~]$ cat ~/.ssh/known_hosts
cat: /home/operator2/.ssh/known_hosts: No such file or directory
[operator2@servera ~]$
```

The file is not populated because the SSH client relied on the system-wide known hosts file for validation.

9. Return to the workstation system as the student user.

```
[operator2@servera ~]$ exit
logout
[student@servera ~]$ exit
logout
Connection to servera closed.
student@workstation:~$
```

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish ssh-hostkeys
```

19.3. Configuring SSH Key-based Authentication

Objectives

- Configure user accounts to use key-based authentication so that an authorized user can securely log in to a specific account on a remote system without a password.

SSH Key-based Authentication

Passwords can be learned or stolen, but securely stored private keys are harder to compromise. You can configure user accounts for passwordless access to SSH servers by enabling key-based authentication, which is based on public key encryption (PKI).

Prepare your account by generating a cryptographically related pair of key files. One key is private and is held only by you, and the other key is public, and is not secret. The private key acts as your authentication credential, and it must be stored securely. The public key is copied to your account on remote servers and verifies your private key during authentication.

When you log in to a remote server, the remote server encrypts a challenge message by using your public key and sends it to your SSH client. Your SSH client proves ownership of the corresponding private key by decrypting the message. If the verification succeeds, then the server trusts your request and grants access without requiring a password.

Generating SSH Keys

Red Hat Enterprise Linux 10 uses the Ed25519 algorithm by default to create SSH key files. Ed25519 uses a more sophisticated algorithm than the RSA algorithm, which makes the Ed25519 public key shorter, but it still provides strong security and high performance. You can create earlier key types such as RSA; these keys remain functional if the remote SSH server accepts them.

Note

RSA is the default algorithm when the system runs in Federal Information Processing Standard (FIPS) mode.

Use the `ssh-keygen` command to create an SSH key pair. The `ssh-keygen` command stores the keys in the `~/.ssh/id_ed25519` and `~/.ssh/id_ed25519.pub` files by default. The following example creates a key pair with the default key type, saves the keys to the default location, and does not define a passphrase.

```

user@host:~$ ssh-keygen
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/user/.ssh/id_ed25519): 
Created directory '/home/user/.ssh'.
Enter passphrase for "/home/user/.ssh/id_ed25519" (empty for no passphrase):

Enter same passphrase again: 
Your identification has been saved in /home/user/.ssh/id_ed25519
Your public key has been saved in /home/user/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:+w3lTa7BdiZwV01AuAP9GjLwRdzwqk+JKcrhosmftuM user@host.lab.example.com
The key's randomart image is:
+--[ED25519 256]--+
|      .  **+  |
|      o +.o.o |
|      +.+ o.  |
|      .* ...  |
|      S . o=o  |
|      . *.B .  |
|      o o X B  |
|      . .++ + = O  |
|      +=E++ . o .  |
+-----[SHA256]-----+

```

You can define a passphrase to encrypt your private key. When a passphrase is set, you must enter the passphrase each time that you use the private key. The passphrase decrypts the private key locally and is never transmitted over the network, unlike a password. A passphrase is recommended to prevent unauthorized use of your private key.

The `ssh-keygen -f` command specifies the file path to store the key pair. The following example creates the private key at `/home/user/.ssh/key-with-pass`, secures the key with the `my-secret` passphrase, and creates the corresponding public key at `/home/user/.ssh/key-with-pass.pub`.

```

user@host:~$ ssh-keygen -f .ssh/key-with-pass
Generating public/private ed25519 key pair.
Enter passphrase for ".ssh/key-with-pass" (empty for no passphrase): my-secret
Enter same passphrase again: my-secret
Your identification has been saved in .ssh/key-with-pass
Your public key has been saved in .ssh/key-with-pass.pub
The key fingerprint is:
SHA256:sb6vKBnwC4Wcip2BwBPWG0lnpNvZQGlyNw186xjN3YQ user@host.lab.example.com
The key's randomart image is:
+--[ED25519 256]--+
|.+* o.o .      |
|o* B + o E .   |
|oo0o. = + o    |
|.o*+.. + + .   |
|.o=*+ + S      |
|o.+ooo o       |
| . + .         |
| + . .         |
| .. oo.        |
+----[SHA256]-----+

```

⚠ Warning

If you use the `ssh-keygen` command and specify the name of an existing key pair, then you overwrite that key pair. This behavior includes the default `id_ed25519` files. You can restore the original key pair only if you have a backup. Overwriting a key pair deletes the original private key, which is required to access accounts that use the matching public key.

If you cannot restore your private key, then you lose access to those remote servers. You must then distribute a new public key to each server to regain access. Always create backups of your keys to prevent permanent loss.

By default, SSH stores generated keys in the `.ssh` subdirectory of your home directory. The private key must be readable and writable only by its owner, so the `600` octal permission is set. The public key is not confidential, and others on the system can read it, so it has `644` octal permissions.

Sharing the Public Key

Configure your remote account access by copying your public key to the remote system. The `ssh-copy-id` command transfers the public SSH key to the remote system. The `ssh-copy-id` command uses the default `~/.ssh/id_ed25519.pub` file if you do not specify one. If the remote server does

not have a public key already configured, then the system prompts you to enter the remote user's password.

The following example uses the `-i` option to send the `.ssh/key-with-pass.pub` public key to the user account on the `remotehost` machine. The system prompts you to enter the remote user's password, which is *user-password* in this example.

```
user@host:~$ ssh-copy-id -i .ssh/key-with-pass.pub user@remotehost
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: ".ssh/key-with-pass.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new keys
user@remotehost's password: user-password

Number of key(s) added: 1

Now try logging into the machine, with: "ssh -i .ssh/key-with-pass 'user@remotehost'"
and check to make sure that only the key(s) you wanted were added.
```

After you add the public key, test the remote access with the corresponding private key. If the configuration is correct, then you can access your account on the remote system without being prompted for your account password. If you do not specify a private key file, then the `ssh` command uses the default `~/.ssh/id_ed25519` file if it exists.

If you configured a passphrase to protect your private key, then SSH requests the passphrase on first use. However, if the key authentication succeeds, then you are not asked for your account password.

The following example uses the `.ssh/key-with-pass` private key to authenticate on the `remotehost` machine with the user account. The system prompts you to enter the passphrase for the key, which is *my-secret* in this example.

```
user@host:~$ ssh -i .ssh/key-with-pass user@remotehost
Enter passphrase for key '.ssh/key-with-pass': my-secret
...output omitted...
user@remotehost:~$
```

Noninteractive Authentication with the `ssh-agent` Key Manager

If you encrypt your private key with a passphrase, then you must enter the passphrase each time that you use the private key for authentication. However, you can configure the `ssh-agent` key manager to cache passphrases. Then, each time that you use SSH, the `ssh-agent` key manager provides the passphrase for you. This approach improves convenience and security by reducing passphrase exposure.

The `ssh-agent` key manager can be configured to start automatically when you log in to a graphical desktop session or a terminal shell. The GNOME graphical desktop environment starts and configures the `ssh-agent` key manager automatically. If you log in to a text environment, then you must start the `ssh-agent` program manually for each session. Start the `ssh-agent` program with the following command:

```
user@host:~$ eval $(ssh-agent)
Agent pid 1850
```

The `ssh-agent` command runs additional shell commands to set environment variables when it is started manually. These environment variables are required when using the `ssh-add` command. You can load your private key passphrase into the key manager by using the `ssh-add` command.

The following examples use the `ssh-add` command to add the private keys from the default `~/.ssh/id_ed25519` file, and then add a private key from the `~/.ssh/key-with-pass` file:

```
user@host:~$ ssh-add
Identity added: /home/user/.ssh/id_ed25519 (user@host.lab.example.com)
user@host:~$ ssh-add .ssh/key-with-pass
Enter passphrase for .ssh/key-with-pass: my-secret
Identity added: .ssh/key-with-pass (user@host.lab.example.com)
```

The `-l` option of the `ssh-add` command lists the fingerprints of all SSH keys that are currently loaded in the `ssh-agent` key manager.

```
user@host:~$ ssh-add -l
256 SHA256:+w3lT...mftuM user@host (ED25519)
256 SHA256:sb6vK...jN3YQ user@host (ED25519)
```

The following `ssh` command uses the `~/.ssh/key-with-pass` private key to access your account on the remote server. The private key in this example was previously decrypted and added to the `ssh-agent` key manager; therefore the `ssh` command does not prompt you for the passphrase to decrypt the private key.

```
user@host:~$ ssh -i .ssh/key-with-pass user@remotehost
...output omitted...
Last login: Thu Jun 19 21:50:35 2025 host.example.com
user@remotehost:~$
```

When you log out of a session that used the `ssh-agent` key manager, all cached passphrases are cleared from memory.

SSH Connection Troubleshooting

SSH can appear complex when remote access with key pair authentication is not succeeding. The `ssh` command provides three verbosity levels with the `-v`, `-vv`, and `-vvv` options, which respectively provide increasing debugging information.

This example demonstrates the information that is provided when using the lowest verbosity option:

```
user@host:~$ ssh -v user@remotehost
OpenSSH_9.9p1, OpenSSL 3.2.2 4 Jun 2024 ①
debug1: Reading configuration data /etc/ssh/ssh_config ②
debug1: Reading configuration data /etc/ssh/ssh_config.d/01-training.conf
debug1: /etc/ssh/ssh_config.d/01-training.conf line 1: Applying options for *
debug1: Reading configuration data /etc/ssh/ssh_config.d/50-redhat.conf
...output omitted...
debug1: Connecting to remotehost [192.168.1.10] port 22. ③
debug1: Connection established.
...output omitted...
debug1: Authenticating to remotehost:22 as 'user' ④
...output omitted...
debug1: Authentications that can continue: publickey,gssapi-keyex,gssapi-with-
mic,password ⑤
...output omitted...
debug1: Next authentication method: publickey ⑥
debug1: Offering public key: /home/user/.ssh/id_ed25519 ED25519
SHA256:+w3lTa7BdiZwV01AuAP9GjLwRdzwqk+JKcrhosmftuM ⑦
debug1: Server accepts key: /home/user/.ssh/id_ed25519 ED25519
SHA256:+w3lTa7BdiZwV01AuAP9GjLwRdzwqk+JKcrhosmftuM ⑧
Authenticated to remotehost ([192.168.1.10]:22) using "publickey".
...output omitted...
user@remotehost:~$
```

- ① OpenSSH and OpenSSL versions
- ② OpenSSH configuration files
- ③ Connection to the remote host

- ④ Trying to authenticate the user on the remote host
- ⑤ Authentication methods that the remote host allows
- ⑥ Trying to authenticate the user by using the SSH key
- ⑦ Using the `/home/user/.ssh/id_rsa` key file to authenticate
- ⑧ The remote hosts accepts the SSH key.

If an attempted authentication method fails, then a remote SSH server falls back to other allowed authentication methods, until all the available methods are tried. The next example demonstrates a remote access with an SSH key that fails, but then the SSH server offers password authentication that succeeds.

```
user@host:~$ ssh -v user@remotehost
...output omitted...
debug1: Next authentication method: publickey
debug1: Offering public key: /home/user/.ssh/id_ed25519 ED25519
SHA256:+w3lTa7BdiZwV01AuAP9GjLwRdzWqk+JKcrhosmftuM
debug1: Authentications that can continue: publickey,gssapi-keyex,gssapi-with-
mic,password
...output omitted...
debug1: Next authentication method: password
user@remotehost's password: user-password
Authenticated to remotehost ([172.25.250.10]:22) using "password".
...output omitted...
user@remotehost:~$
```

Configuring an SSH Client

You can create the `~/.ssh/config` file to preconfigure SSH connections. Within the configuration file, you can specify connection parameters such as users, keys, and ports for specific hosts. This file eliminates the need to manually specify command parameters each time that you connect to a host. Consider the following `~/.ssh/config` file, which preconfigures two host connections with different users and keys:

```

user@host:~$ cat ~/.ssh/config
host remotehosta
    HostName          remotehosta.example.com
    User              usera
    IdentityFile       ~/.ssh/id_ed25519_remotehosta

host remotehostb
    HostName          remotehostb.example.com
    User              userb
    IdentityFile       ~/.ssh/id_ed25519_remotehostb

```

Configuring an SSH Server

The `sshd` daemon provides the OpenSSH service. You can configure the service by editing the `/etc/ssh/sshd_config` file.

The default configuration of the OpenSSH server works well for many use cases. However, you might make changes to strengthen the security of your system. For example, you might want to prohibit direct remote logins to the `root` account, and to prohibit password-based authentication (in favor of SSH private key authentication).

Prohibiting the Superuser from Logging In

A good practice is to prohibit direct logins to the `root` user account from remote systems. Allowing direct logins as the `root` user incurs the following risks:

- The `root` username exists on every Linux system by default, so a potential attacker needs only to guess the password, instead of a valid username and password combination. Using a valid username and password reduces the likelihood of a successful attack.
- The `root` user has unrestricted privileges, so its compromise can lead to maximum damage to the system.
- From an auditing perspective, tracking which authorized user logged in as the `root` user and made changes can be difficult. If users must log in as a regular user and switch to the `root` account, then you can view a log event for accountability.

 Important

Starting in Red Hat Enterprise Linux 9, the `PermitRootLogin` parameter is set to the `prohibit-password` value by default. This value enforces the use of key-based authentication instead of passwords for logging in as the root user, and reduces the risk of brute-force attacks.

The OpenSSH server uses the `PermitRootLogin` configuration setting in the `/etc/ssh/sshd_config` file to allow or prohibit users to log in to the system as the root user, as in the following example:

```
PermitRootLogin yes
```

If the `PermitRootLogin` parameter is set to `yes`, then anyone can log in as the root user remotely. To prevent this situation, set the value to `no`.

Alternatively, to prevent password-based authentication but to allow private key-based authentication for the root user, set the `PermitRootLogin` parameter to `without-password`.

You must reload the `sshd` service to apply the changes.

```
root@host:~# systemctl reload sshd
```

Prohibiting Password-based Authentication

Allowing only private key-based authentication has the following advantages:

- Attackers cannot use password-guessing attacks to remotely break into known accounts on the system.
- With passphrase-protected private keys, an attacker needs both the passphrase and a copy of the private key. With passwords, an attacker needs only the password.
- By using passphrase-protected private keys with the `ssh-agent` key manager, the passphrase is entered and exposed less often, and logging in is more convenient for the user.

SSH uses the `PasswordAuthentication` parameter in the `/etc/ssh/sshd_config` file to control whether users can use password-based authentication to log in to the system.

```
PasswordAuthentication yes
```

With the default value of `yes` for the `PasswordAuthentication` parameter in the `/etc/ssh/sshd_config` file, the SSH server allows users to use password-based authentication when logging in. The value of `no` for `PasswordAuthentication` prevents users from using password-based authentication.

Whenever you change the `/etc/ssh/sshd_config` file, you must reload the `sshd` service to apply the changes.

Important

If you turn off password-based authentication for SSH, then you must ensure that the user's `~/.ssh/authorized_keys` file on the remote server is populated with their public key, so that they can log in.

References

`ssh(1)`, `ssh-keygen(1)`, `ssh-copy-id(1)`, `ssh-agent(1)`, `ssh-add(1)`, and `sshd_config(5)` man pages

19.4. Guided Exercise

Configure SSH Key-based Authentication

Enable secure remote access for authorized users by configuring key-based authentication.

Outcomes

- Generate an SSH key pair with and without passphrase protection.
- Configure the `ssh-agent` key manager to use a passphrase-protected key for authentication.

Prerequisites

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
student@workstation:~$ lab start ssh-keyauth
```

Instructions

1. As the student user on the workstation machine, log in to the servera machine as the student user. Switch to the operator1 user. Use redhat as the password.

```
student@workstation:~$ ssh student@servera
...output omitted...
[student@servera ~]$ su - operator1
Password: redhat
[operator1@servera ~]$
```

2. Generate an unprotected SSH key pair and store it in the default location. Send the public key to the operator1 user on the serverb machine. Verify that you can log in to the serverb machine as the operator1 user without a password.

- 2.1. Use the `ssh-keygen` command to generate a set of SSH keys. When prompted, press to use the default location, and press again to leave the passphrase field empty.

```
[operator1@servera ~]$ ssh-keygen
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/operator1/.ssh/id_ed25519): 
Created directory '/home/operator1/.ssh'.
Enter passphrase for "/home/operator1/.ssh/id_ed25519" (empty for no
passphrase): 
Enter same passphrase again: 
Your identification has been saved in /home/operator1/.ssh/id_ed25519
Your public key has been saved in /home/operator1/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:mBKTu90SoYdczEOmEELroDiUjUqgmxtiistLJdeoagg operator1@servera
The key's randomart image is:
+--[ED25519 256]--+
|+.o.  o          |
|o.=. B          |
|o*  .= *          |
|Oo  .oB =         |
|X+  +*.= S        |
|E+=  = o          |
|*o   . o .        |
|=o    .           |
|+o.              |
+-----[SHA256]-----+
```

- 2.2.** Send the public key of the SSH key pair to the operator1 user on the serverb machine. When prompted, type yes to accept the connection, and use redhat as the password.

```
[operator1@servera ~]$ ssh-copy-id operator1@serverb
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed:
"/home/operator1/.ssh/id_ed25519.pub"
The authenticity of host 'serverb (172.25.250.11)' can't be established.
ED25519 key fingerprint is SHA256:i5IURCZ+/4moD3P1TIYzZdFzqBmgm3Kl80YNbh8z7yU.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter
out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are
prompted now it is to install the new keys
operator1@serverb's password: redhat

Number of key(s) added: 1

Now try logging into the machine, with: "ssh 'operator1@serverb'"
and check to make sure that only the key(s) you wanted were added.
```

2.3. Verify that you can log in to the serverb machine as the operator1 user. Then, return to the servera machine as the operator1 user.

```
[operator1@servera ~]$ ssh operator1@serverb
...output omitted...
[operator1@serverb ~]$ exit
logout
Connection to serverb closed.
[operator1@servera ~]$
```

Note

The preceding ssh command does not prompt you for a password, because it uses the private key without a passphrase against the exported public key to authenticate as the operator1 user on the serverb machine.

This approach is not secure, because anyone who has access to the private key file can log in to the serverb machine as the operator1 user.

In a following step in this exercise, you make your private key more secure by encrypting it and protecting access to it by adding a passphrase.

3. Generate another set of SSH keys that are protected by a passphrase. Send the public key to the operator1 user on the serverb machine. Verify that you can log in to the serverb machine as the operator1 user without a password.

- 3.1.** Generate another set of SSH keys, but this time, set a passphrase. Save the key as `/home/operator1/.ssh/protected-key`. Use `redhatpass` as the passphrase of the private key.

```
[operator1@servera ~]$ ssh-keygen -f ~/.ssh/protected-key
Generating public/private ed25519 key pair.
Enter passphrase for ".ssh/protected-key" (empty for no passphrase): redhatpass
Enter same passphrase again: redhatpass
Your identification has been saved in .ssh/protected-key
Your public key has been saved in .ssh/protected-key.pub
The key fingerprint is:
SHA256:UZCLLSJl8JVXszAR7FnBoR9r6c3XJHa2lsoejx4R2ag operator1@servera
The key's randomart image is:
+--[ED25519 256]--+
|      .oo o**==o   |
|      .o..=.o+oo +  |
|      ..+.+.oo + .  |
|      . ..o. = .    |
|      S   E + +     |
|      o + *+        |
|      . =o+         |
|      ..B           |
|      o* .          |
+-----[SHA256]-----+
```

- 3.2.** Send the public key of the passphrase-protected key pair to the `operator1` user on the `serverb` machine. The command does not prompt you for a password, because it uses the public key of the passphrase-less private key that you exported to the `serverb` machine in the preceding step.


```
[operator1@servera ~]$ ssh-copy-id -i ~/.ssh/protected-key.pub operator1@serverb
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: ".ssh/protected-
key.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter
out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are
prompted now it is to install the new keys

Number of key(s) added: 1

Now try logging into the machine, with: "ssh -i .ssh/protected-key
'operator1@serverb'"
and check to make sure that only the key(s) you wanted were added.
```

- 3.3.** Verify that you can log in to the serverb machine as the operator1 user by using SSH to run the hostname command remotely. Use the /home/operator1/.ssh/protected-key key as the identity file. Use redhatpass as the passphrase, which you set for the private key in the preceding step.

The command prompts you for the passphrase that you used to protect the private key of the SSH key pair. If an attacker gains access to the private key, then the attacker cannot use it to access other systems, because a passphrase protects the private key.

```
[operator1@servera ~]$ ssh -i ~/.ssh/protected-key operator1@serverb hostname
Enter passphrase for key '.ssh/protected-key': redhatpass
serverb
```

Note

To avoid interactively typing the passphrase when logging in with SSH, use the ssh-agent program, as in the following step. Using the ssh-agent program is more convenient and more secure for administrators who regularly log in to remote systems.

- 4.** Run the ssh-agent program in your Bash shell, and add the protected-key passphrase-protected private key of the SSH key pair to the shell session. Verify that you can log in to the serverb machine as the operator1 user without a password.

- 4.1.** Run the ssh-agent program in your Bash shell.

```
[operator1@servera ~]$ eval $(ssh-agent)
Agent pid 2051
```

The command starts the `ssh-agent` program and configures the shell session to use it.

- 4.2.** Add the `.ssh/protected-key` passphrase-protected private key to the shell session. Use the `ssh-add` command to provide the unlocked private key to the `ssh-agent` program. Use `redhatpass` as the passphrase.

```
[operator1@servera ~]$ ssh-add ~/.ssh/protected-key
Enter passphrase for ~/.ssh/protected-key: redhatpass
Identity added: .ssh/protected-key (operator1@servera)
```

- 4.3.** Verify that you can log in to the `serverb` machine as the `operator1` user by using SSH to run the `hostname` command remotely. Use the `/home/operator1/.ssh/protected-key` key as the identity file.

The command does not prompt you to enter the passphrase interactively.

```
[operator1@servera ~]$ ssh -i ~/.ssh/protected-key operator1@serverb hostname
serverb
```

Note

Logging out of a session that used the `ssh-agent` key manager clears all cached passphrases from memory. Therefore, you must re-enter the passphrase the next time that you use the passphrase-protected private key.

- 5.** Return to the workstation machine as the `student` user.

```
[operator1@servera ~]$ exit
logout
[student@servera ~]$ exit
logout
Connection to servera closed.
student@workstation:~$
```

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish ssh-keyauth
```

19.5. Lab

Configure and Secure SSH

Protect SSH communication by managing host keys and by implementing key-based authentication for users.

Outcomes

- Authenticate with SSH keys.
- Prevent users from directly logging in as the root user with SSH.
- Configure key-based authentication to prevent users from logging in by using passwords.

Prerequisites

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
student@workstation:~$ lab start ssh-review
```

Instructions

1. From the workstation machine, log in to the servera machine as the student user.
2. Switch to the production1 user on the servera machine. Enter redhat as the password.
3. Generate SSH keys for the production1 user on the servera machine. Do not set a passphrase.
4. Send the SSH public key for the production1 user on the servera machine to the production1 user on the serverb machine. Use redhat as the password.
5. From the servera machine, verify that the production1 user can successfully log in to the serverb machine by using SSH keys.
6. Configure the sshd service on the serverb machine to prevent users from logging in as the root user. Use redhat as the root password. Open a second terminal to verify that the production1 user cannot log in to the serverb machine as the root user.
7. Use the first terminal to configure the sshd service on the serverb machine so that users must authenticate with SSH keys and cannot authenticate with their passwords. Use the second terminal to verify that users must use SSH keys to log in to the serverb machine. First, attempt to log in to the serverb machine as the production2 user, where SSH keys are not configured. Then, attempt to log in as the production1 user by using SSH keys.

8. After completing the tasks of this activity, return to the workstation machine as the student user.

Evaluation

As the student user on the workstation machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

```
student@workstation:~$ lab grade ssh-review
```

Finish

On the workstation machine, change to the student user home directory and use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish ssh-review
```

Solution

Protect SSH communication by managing host keys and by implementing key-based authentication for users.

Outcomes

- Authenticate with SSH keys.
- Prevent users from directly logging in as the root user with SSH.
- Configure key-based authentication to prevent users from logging in by using passwords.

Prerequisites

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
student@workstation:~$ lab start ssh-review
```

Instructions

1. From the workstation machine, log in to the servera machine as the student user.

```
student@workstation:~$ ssh student@servera  
[student@servera ~]$
```

2. Switch to the production1 user on the servera machine. Enter redhat as the password.

```
[student@servera ~]$ su - production1  
Password: redhat  
[production1@servera ~]$
```

3. Generate SSH keys for the production1 user on the servera machine. Do not set a passphrase.

```
[production1@servera ~]$ ssh-keygen
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/production1/.ssh/id_ed25519):
Enter
Created directory '/home/production1/.ssh'.
Enter passphrase for "/home/production1/.ssh/id_ed25519" (empty for no
passphrase): Enter
Enter same passphrase again: Enter
Your identification has been saved in /home/production1/.ssh/id_ed25519
Your public key has been saved in /home/production1/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:asl68gkyR7bSe5uiV5Y+63PdZpJBq2hZmwDRaxhkRQE production1@servera
The key's randomart image is:
+--[ED25519 256]--+
|   .E=+.          |
|   .o .           |
|    + .           |
|   o o .          |
|   oo .S. .       |
|  + o=o. o        |
| + +=== = +       |
|  +=Xo= + +       |
| .ooOB*   +       |
+-----[SHA256]-----+
```

4. Send the SSH public key for the production1 user on the servera machine to the production1 user on the serverb machine. Use redhat as the password.

```
[production1@servera ~]$ ssh-copy-id production1@serverb
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed:
"/home/production1/.ssh/id_ed25519.pub"
The authenticity of host 'serverb (172.25.250.11)' can't be established.
ED25519 key fingerprint is
SHA256:aBluK0igd9ugAsUBd5A5QtVTr7gg3IBzT/MJiTtMPkU.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to
filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are
prompted now it is to install the new keys
production1@serverb's password: redhat

Number of key(s) added: 1

Now try logging into the machine, with: "ssh 'production1@serverb'"
and check to make sure that only the key(s) you wanted were added.
```

5. From the servera machine, verify that the production1 user can successfully log in to the serverb machine by using SSH keys.

```
[production1@servera ~]$ ssh production1@serverb
...output omitted...
[production1@serverb ~]$
```

6. Configure the sshd service on the serverb machine to prevent users from logging in as the root user. Use redhat as the root password. Open a second terminal to verify that the production1 user cannot log in to the serverb machine as the root user.

- 6.1. Switch to the root user on the serverb machine.

```
[production1@serverb ~]$ su -
Password: redhat
[root@serverb ~]#
```

- 6.2. Set the PermitRootLogin parameter to prohibit-password in the /etc/ssh/sshd_config file and restart the sshd service. Edit the active uncommented parameter and not a commented example.


```
...output omitted...
PermitRootLogin prohibit-password
...output omitted...
[root@serverb ~]# systemctl restart sshd.service
```

6.3. Verify that you cannot log in to the serverb machine as the root user. Open a second terminal and log in to the servera machine as the production1 user. From the servera machine, try to log in to the serverb machine as the root user, with redhat as the password. This command should fail after three login attempts.

By default, SSH first tries to use SSH keys to authenticate. If SSH keys for the user are not configured, then SSH requests the user's password for authentication.

```
student@workstation:~$ ssh production1@servera
...output omitted...
[production1@servera ~]$ ssh root@serverb
root@serverb's password: redhat
Permission denied, please try again.
root@serverb's password: redhat
Permission denied, please try again.
root@serverb's password: redhat
root@serverb: Permission denied (publickey,gssapi-keyex,gssapi-with-
mic,password).
[production1@servera ~]$
```

Note

The passwordless root login is configured in the classroom environment for your convenience. However, this configuration is highly insecure, and is not recommended for any production environment.

7. Use the first terminal to configure the sshd service on the serverb machine so that users must authenticate with SSH keys and cannot authenticate with their passwords. Use the second terminal to verify that users must use SSH keys to log in to the serverb machine. First, attempt to log in to the serverb machine as the production2 user, where SSH keys are not configured. Then, attempt to log in as the production1 user by using SSH keys.

7.1. Return to the first terminal with the root active shell on the serverb machine. Set the PasswordAuthentication parameter to no in the /etc/ssh/sshd_config file and restart the sshd service. Edit the active uncommented parameter and not a commented example.

```
...output omitted...
PasswordAuthentication no
...output omitted...
[root@serverb ~]# systemctl restart sshd
```

- 7.2.** Go to the second terminal with the `production1` active shell on the `servera` machine, and try to log in to the `serverb` machine as the `production2` user. This command should fail, because SSH keys are not configured for the `production2` user, and the `sshd` service on the `serverb` machine does not allow the use of passwords for authentication.

```
[production1@servera ~]$ ssh production2@serverb
production2@serverb: Permission denied (publickey,gssapi-keyex,gssapi-with-mic).
```

Note

For more granularity, you can use the explicit `ssh` command with the `-o` `PubkeyAuthentication=no` and `-o PasswordAuthentication=yes` options. With these options, you can override the `ssh` command defaults, and confidently establish whether the preceding command failed based on the settings that you adjusted in the `/etc/ssh/sshd_config` file in the preceding step.

- 7.3.** Return to the first terminal with the `root` active shell on the `serverb` machine. Verify that the `PubkeyAuthentication` parameter is enabled in the `/etc/ssh/sshd_config` file.

```
[root@serverb ~]# grep PubkeyAuthentication /etc/ssh/sshd_config
#PubkeyAuthentication yes
```

The `PubkeyAuthentication` line is commented. Commented lines indicate the default values of a parameter. Public key authentication is active by default, which the commented line indicates.

- 7.4.** Return to the second terminal with the `production1` active shell on the `servera` machine, and try to log in to the `serverb` machine as the `production1` user. This command should succeed, because SSH keys are configured for the `production1` user to log in to the `serverb` machine from the `servera` machine.

```
[production1@servera ~]$ ssh production1@serverb
...output omitted...
[production1@serverb ~]$
```

7.5. Exit and close the second terminal.

```
[production1@serverb ~]$ exit
logout
Connection to serverb closed.
[production1@servera ~]$ exit
logout
student@workstation:~$ exit
```

8. After completing the tasks of this activity, return to the workstation machine as the student user.

```
[root@serverb ~]# exit
logout
[production1@serverb ~]$ exit
logout
Connection to serverb closed.
[production1@servera ~]$ exit
logout
[student@servera ~]$ exit
logout
Connection to servera closed.
student@workstation:~$
```

Evaluation

As the student user on the workstation machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

```
student@workstation:~$ lab grade ssh-review
```

Finish

On the workstation machine, change to the student user home directory and use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish ssh-review
```

19.6. Summary

In this lesson, you learned how SSH uses host keys to verify server identities to clients. By default, SSH clients maintain records of all hosts whose authenticity a user explicitly confirmed. These known host files can be configured both system-wide and on a per-user basis. You also explored how the `ssh-keygen` and `ssh-keyscan` commands are valuable tools for managing these host keys.

SSH supports various methods for user authentication, including password-based and key-based authentication. For security and ease of use, users can generate a personal keypair by using the `ssh-keygen` command. To enable key-based access to a remote system, you can copy your public key to that system, to enable your corresponding private key to secure subsequent logins.

For enhanced security, SSH servers can be hardened by disabling direct login for the `root` user and by turning off password-based authentication system-wide.

Chapter 20.

Comprehensive Review

Goal

This lesson tests the knowledge that you acquired by completing the *Red Hat System Administration I (RH124)* course.

Objectives

- Validate the knowledge that you acquired after completing the *Red Hat System Administration I (RH124)* course.

Sections

- Summary of Learning Objectives
- Manage Files from the Command Line (Lab)
- Manage Users and Groups, Permissions, and Processes (Lab)
- Configure and Manage a Server (Lab)
- Manage Networks (Lab)
- Mount File Systems and Find Files (Lab)

20.1. Summary of Learning Objectives

Objectives

- After completing this section, you should have reviewed and refreshed the knowledge and skills that you learned in *Red Hat System Administration I*.

Reviewing Red Hat System Administration I

Before beginning the comprehensive review for the *Red Hat System Administration I* course, you should be comfortable with the topics that are covered in each chapter. Do not hesitate to ask the instructor for extra guidance or clarification on these topics.

Chapter 1, Introduction to Red Hat Enterprise Linux

Define and explain the following concepts: open source, Linux, Linux distributions, and Red Hat Enterprise Linux.

- Define and explain the following concepts: open source, Linux, Linux distributions, and Red Hat Enterprise Linux.

Chapter 2, Accessing the Command Line

Log in to a Linux system and run simple commands by using the shell.

- Log in to a Linux system and run simple commands by using the shell.
- Log in to a Linux system with the GNOME desktop environment and run commands from a shell prompt in a terminal program.
- Save time when running commands from a Bash shell prompt by using some of its productivity features.
- Access the command line on a Linux system and use the Bash shell to run commands.

Chapter 3, Getting Help from Local Documentation

Resolve problems by using information in local help systems and documentation.

- Find information in local Linux system manual pages.
- Get information from local documentation to help you to run commands and to complete tasks.

Chapter 4, Registering Systems for Red Hat Support

Register a system by using your Red Hat account to get support services and software that Red Hat provides.

- Register a system by using your Red Hat account.

Chapter 5. Getting AI-assisted Help with RHEL Lightspeed

Get AI-driven guidance and suggestions to efficiently manage a RHEL server by using Red Hat Enterprise Linux Lightspeed.

- Troubleshoot problems by phrasing effective questions and evaluating the answers that RHEL Lightspeed provides.

Chapter 6. Navigating the File-system Hierarchy

Describe the organization of files in the Linux file-system hierarchy and specify files by name, by using relative paths or absolute paths.

- Describe how Linux files are organized, and the purposes of various directories in the file-system hierarchy.
- Specify the absolute location and relative location of files to the current working directory, determine and change the working directory, and list the contents of directories.

Chapter 7. Managing Files from the Command Line

Copy, move, create, delete, and organize files from the command line.

- Create, copy, move, and remove files and directories.
- Create and compare the use of hard links and symbolic links to reference the same file.
- Efficiently run commands that affect many files by using pattern matching features of the Bash shell.
- Copy, move, create, delete, and organize files from the command line.

Chapter 8. Editing Text Files

Create, view, and edit text files from the command line.

- Create, view, and edit text files from the command line.

Chapter 9. Redirecting Shell Input and Output

Redirect the output of commands and input to commands, by using Bash.

- Save output or errors to a file with shell redirection, and process command output through multiple command-line programs with pipes.

Chapter 10. Managing Local Users and Groups

Obtain superuser access to a system; create, manage, and delete local users and groups; and administer local password policies.

- Describe the purpose of users and groups on a Linux system.
- Switch to the superuser account to manage a Linux system, and grant other users superuser access by using Sudo.
- Create, modify, and delete local user accounts.
- Create, modify, and delete local group accounts.
- Set a password management policy for users, and manually lock and unlock user accounts.
- Obtain superuser access to a system; create, manage, and delete local users and groups; and administer local password policies.

Chapter 11. Controlling Access to Files

Set standard permissions on files and interpret the security effects of different permission settings.

- List file system permissions on files and directories, and interpret how those permissions control access by users and by groups.
- Change the permissions and ownership of files with command-line tools.
- Control the default permissions of user-created files, and explain special permissions on files.
- Set standard permissions on files and interpret the security effects of different permission settings.

Chapter 12. Installing and Updating Software with RPM

Download, install, update, and manage software packages from Red Hat and DNF package repositories.

- Explain how Red Hat and other sources provide software as RPM packages, and investigate the installed system packages.
- Find, install, and update software packages with DNF.
- Enable and disable server use of Red Hat or third-party DNF repositories.

Chapter 13. Installing and Updating Applications by Using Flatpak

Install, upgrade, and use desktop software from the Red Hat Ecosystem Catalog by using Flatpak.

- Install and configure Flatpak tools and access to remote repositories.
- Search for, install, run, update, and remove applications by using Flatpak.

Chapter 14. Accessing Removable Media

Access file systems on removable media devices by mounting them on a directory in the file-system hierarchy.

- Match a block storage device with the file system that a particular directory uses, and determine the amount of free space on that file system.
- Access the contents of a file system on removable media by mounting it into the file-system hierarchy, and unmounting it when you are finished.
- Search for files on a file system that is currently mounted.

Chapter 15. Monitoring and Managing Linux Processes

Investigate, control, and terminate processes that run on a Red Hat Enterprise Linux system.

- Identify the processes that are currently running on a system, determine their current process states, and gather information about their use of system resources.
- Control foreground and background processes on the same terminal by using job control.
- Send signals to processes to control their activity or to terminate execution.
- Interpret and monitor system metrics, and investigate the implications of those measurements to improve your system's performance.

Chapter 16. Controlling Services and Daemons

Control and monitor the system services and daemons that systemd starts.

- List the system daemons and network services that start at boot time as services, or that start and stop dynamically when a network connection is established.
- Control system daemons and network services with the `systemctl` command.

Chapter 17. Introduction to Networking

Describe basic TCP/IP networking concepts and investigate the current network configuration and functionality on a server.

- Describe the fundamental concepts of network addressing and routing for a server.

- Test and inspect the current network configuration by using command-line utilities.

Chapter 18. Managing Network Configuration

Configure network interfaces and settings on Red Hat Enterprise Linux servers.

- Manage network settings and devices from the command line by using the `nmc li` command.
- Modify network configuration by editing configuration files.
- Configure a server's static hostname and its name resolution, and test the results.
- Configure network interfaces and settings on Red Hat Enterprise Linux servers.

Chapter 19. Configuring and Securing SSH

Protect SSH communication by managing host keys and by implementing key-based authentication for users.

- Prevent impersonating attacks by configuring the SSH service of client systems to use trusted host keys.
- Configure user accounts to use key-based authentication so that an authorized user can securely log in to a specific account on a remote system without a password.

20.2. Lab

Manage Files from the Command Line

Manage files, redirect a specific set of lines from a text file to another file, and edit the text files.

Outcomes

- Manage files from the command line.
- Capture specific content from a text file or a command, and redirect the output to another file.
- Edit text files.

Note

The allocated time for this activity is 15 minutes. If you need additional time to complete the task, then you must revisit the course content, or practice more.

Prerequisites

If you did not reset your workstation and serverX machines at the end of the last chapter, then save any work that you want to keep from earlier exercises on those machines, and reset them now.

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
student@workstation:~$ lab start rhcsa-rh124-review1
```

Specifications

You have joined an organization as an intern system administrator. You are tasked to complete the organization's level 1 training to make sure that you can manage the text editor and its shortcuts, and creating files and directories.

Perform the following tasks on the serverb machine:

- Create the /home/student/grading directory.
- Create three empty files named grade1, grade2, and grade3, in the /home/student/grading directory.

- Capture the first five lines of the `/home/student/bin/manage` file in the `/home/student/grading/review.txt` file.
- Without overwriting any existing text, append the last three lines of the `/home/student/bin/manage` file to the `/home/student/grading/review.txt` file.
- Copy the `/home/student/grading/review.txt` file to the `/home/student/grading/review-copy.txt` file.
- Duplicate the `Test JJ` line in the `/home/student/grading/review-copy.txt` file. Make sure that the duplicated line appears immediately after the original line.
- Remove the `Test HH` line from the `/home/student/grading/review-copy.txt` file.
- Add the `Level 1 Training` line between the `Test BB` line and the `Test CC` line in the `/home/student/grading/review-copy.txt` file.
- Create a link named `data-backup` that points to the same data as the `/home/student/grading/grade1` file.
- Create a link named `filename-backup` that points to the `/home/student/grading/grade2` file name.
- List the contents of the `/boot` directory and save the output to the `/home/student/grading/longlisting.txt` file. The output should be a long listing that includes the file permissions, owner and group owner, size, and modification date of each file. The output must omit hidden files.

Evaluation

As the `student` user on the workstation machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful. The grading scripts provide remediation hints to help you identify what content you need to revisit to complete the tasks.

```
student@workstation:~$ lab grade rhcsa-rh124-review1
```

Evaluate the allocated time for this activity against the time that you took to complete it. If you used more time than is allocated, then you must revisit the course content or practice more.

Finish

On the workstation machine, change to the `student` user home directory and use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish rhcsa-rh124-review1
```

Solution

Manage files, redirect a specific set of lines from a text file to another file, and edit the text files.

Outcomes

- Manage files from the command line.
- Capture specific content from a text file or a command, and redirect the output to another file.
- Edit text files.

Note

The allocated time for this activity is 15 minutes. If you need additional time to complete the task, then you must revisit the course content, or practice more.

Prerequisites

If you did not reset your workstation and serverX machines at the end of the last chapter, then save any work that you want to keep from earlier exercises on those machines, and reset them now.

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
student@workstation:~$ lab start rhcsa-rh124-review1
```

Specifications

You have joined an organization as an intern system administrator. You are tasked to complete the organization's level 1 training to make sure that you can manage the text editor and its shortcuts, and creating files and directories.

Perform the following tasks on the serverb machine:

- Create the /home/student/grading directory.
- Create three empty files named grade1, grade2, and grade3, in the /home/student/grading directory.
- Capture the first five lines of the /home/student/bin/manage file in the /home/student/grading/review.txt file.

- Without overwriting any existing text, append the last three lines of the `/home/student/bin/manage` file to the `/home/student/grading/review.txt` file.
- Copy the `/home/student/grading/review.txt` file to the `/home/student/grading/review-copy.txt` file.
- Duplicate the `Test JJ` line in the `/home/student/grading/review-copy.txt` file. Make sure that the duplicated line appears immediately after the original line.
- Remove the `Test HH` line from the `/home/student/grading/review-copy.txt` file.
- Add the `Level 1 Training` line between the `Test BB` line and the `Test CC` line in the `/home/student/grading/review-copy.txt` file.
- Create a link named `data-backup` that points to the same data as the `/home/student/grading/grade1` file.
- Create a link named `filename-backup` that points to the `/home/student/grading/grade2` file name.
- List the contents of the `/boot` directory and save the output to the `/home/student/grading/longlisting.txt` file. The output should be a long listing that includes the file permissions, owner and group owner, size, and modification date of each file. The output must omit hidden files.

Solution

1. Log in to the `serverb` machine as the `student` user.
2. Create the `/home/student/grading` directory, and inside the directory, create three empty files named `grade1`, `grade2`, and `grade3`.

```
[student@serverb ~]$ mkdir /home/student/grading \
&& touch grading/grade{1,2,3}
```

3. Create the `review.txt` file with content from another file, and create the `review-copy.txt` file as a copy of the `review.txt` file.

```
[student@serverb ~]$ head -5 bin/manage > grading/review.txt \
&& tail -3 bin/manage >> grading/review.txt \
&& cp grading/review.txt grading/review-copy.txt
```

4. Edit the `/home/student/grading/review-copy.txt` file and update the content to match the following text:

```
Test AA
Test BB
Level 1 Training
Test CC
Test DD
Test EE
Test II
Test JJ
Test JJ
```

5. Create the hard and soft links, and create the `longlisting.txt` file with the output of the `/boot` directory contents.

```
[student@serverb ~]$ ln grading/grade1 data-backup && \
ln -s grading/grade2 filename-backup && \
ls -l /boot > grading/longlisting.txt
```

6. After completing the tasks of this activity, return to the workstation machine as the student user.

Evaluation

As the student user on the workstation machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful. The grading scripts provide remediation hints to help you identify what content you need to revisit to complete the tasks.

```
student@workstation:~$ lab grade rhcsa-rh124-review1
```

Evaluate the allocated time for this activity against the time that you took to complete it. If you used more time than is allocated, then you must revisit the course content or practice more.

Finish

On the workstation machine, change to the student user home directory and use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish rhcsa-rh124-review1
```


20.3. Lab

Manage Users and Groups, Permissions, and Processes

Manage user and group accounts, set permissions on files and directories, and manage processes.

Outcomes

- Manage user accounts and groups.
- Set permissions on files and directories.
- Identify and manage high CPU-consuming processes.

Note

The allocated time for this activity is 15 minutes. If you need additional time to complete the task, then you must revisit the course content, or practice more.

Prerequisites

If you did not reset your workstation and serverX machines at the end of the last chapter, then save any work that you want to keep from earlier exercises on those machines, and reset them now.

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
student@workstation:~$ lab start rhcsa-rh124-review2
```

Specifications

You have moved into the next stage of the systemd administrator internship where you are asked to do some administrative tasks. In your new project, database users have access to a specific location to create, edit, and store database files. To allow these database users to manage files collaboratively, you must modify permissions so that all users have access to all files that are created in this location. The other restriction to consider is that only owners of the files can delete them.

Perform the following tasks on the serverb machine:

- Identify and terminate the process that currently uses the most CPU time.
- Create the database group with a group ID (GID) of 50000.

- Create the `dbadmin1` user and configure it with the following requirements:
 - Add the database group as a supplementary group.
 - Set the password to `redhat` and force a password change on first login.
 - Allow the password to be changed 10 days after the last password change.
 - Configure the password to expire 30 days after the last password change.
 - Allow the user to use Sudo to run any command as the superuser by creating its own `sudoers` file.
 - Configure the default `umask` as `027` for the `dbadmin1` user.
- Create the `/home/dbadmin1/grading/review2` directory and set all its contents to be owned by the `dbadmin1` user and the database group.
- Configure the `/home/dbadmin1/grading/review2` directory so that users are allowed to delete only files that they own. Configure the permissions on the directory to allow members of the database group to access the directory and to create contents in it. All other users must have read and execute permissions on the directory.

Evaluation

As the `student` user on the workstation machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful. The grading scripts provide remediation hints to help you identify what content you need to revisit to complete the tasks.

```
student@workstation:~$ lab grade rhcsa-rh124-review2
```

Evaluate the allocated time for this activity against the time that you took to complete it. If you used more time than is allocated, then you must revisit the course content or practice more.

Finish

On the workstation machine, change to the `student` user home directory and use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish rhcsa-rh124-review2
```

Solution

Manage user and group accounts, set permissions on files and directories, and manage processes.

Outcomes

- Manage user accounts and groups.
- Set permissions on files and directories.
- Identify and manage high CPU-consuming processes.

Note

The allocated time for this activity is 15 minutes. If you need additional time to complete the task, then you must revisit the course content, or practice more.

Prerequisites

If you did not reset your workstation and serverX machines at the end of the last chapter, then save any work that you want to keep from earlier exercises on those machines, and reset them now.

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
student@workstation:~$ lab start rhcsa-rh124-review2
```

Specifications

You have moved into the next stage of the systemd administrator internship where you are asked to do some administrative tasks. In your new project, database users have access to a specific location to create, edit, and store database files. To allow these database users to manage files collaboratively, you must modify permissions so that all users have access to all files that are created in this location. The other restriction to consider is that only owners of the files can delete them.

Perform the following tasks on the serverb machine:

- Identify and terminate the process that currently uses the most CPU time.
- Create the database group with a group ID (GID) of 50000.
- Create the dbadmin1 user and configure it with the following requirements:

- Add the database group as a supplementary group.
 - Set the password to redhat and force a password change on first login.
 - Allow the password to be changed 10 days after the last password change.
 - Configure the password to expire 30 days after the last password change.
 - Allow the user to use Sudo to run any command as the superuser by creating its own sudoers file.
 - Configure the default umask as 027 for the dbadmin1 user.
- Create the /home/dbadmin1/grading/review2 directory and set all its contents to be owned by the dbadmin1 user and the database group.
 - Configure the /home/dbadmin1/grading/review2 directory so that users are allowed to delete only files that they own. Configure the permissions on the directory to allow members of the database group to access the directory and to create contents in it. All other users must have read and execute permissions on the directory.

Solution

1. Log in to the serverb machine as the student user.
2. Identify and terminate the dd process that currently uses the most CPU resources.
3. Switch to the root user, and create the database group with a group ID (GID) of 50000 and create the dbadmin1 user. Set the password expiration policies.

```
[student@serverb ~]$ sudo -i
[sudo] password for student: student
[root@serverb ~]# groupadd -g 50000 database \
&& useradd -G database dbadmin1 \
&& chage -d 0 -m 10 -M 30 dbadmin1
```

4. Update the dbadmin1 user password to redhat.
5. Create the /etc/sudoers.d/dbadmin1 file with the following content:

```
dbadmin1 ALL=(ALL) ALL
```

6. Switch to the dbadmin1 user. Append the umask 027 line to the /home/dbadmin1/.bashrc file.

```
[root@serverb ~]# su - dbadmin1
[dbadmin1@serverb ~]$ echo "umask 027" >> .bashrc
[dbadmin1@serverb ~]$ source ~/.bashrc
```

7. Create the /home/dbadmin1/grading/review2 directory and set the appropriate permissions.

```
[dbadmin1@serverb ~]$ mkdir -p /home/dbadmin1/grading/review2 \
&& chown -R dbadmin1:database /home/dbadmin1/ \
&& chmod o+t /home/dbadmin1/grading/review2 \
&& chmod g+w /home/dbadmin1/grading/review2 \
&& chmod o+r /home/dbadmin1/grading/review2
```

8. After completing the tasks of this activity, return to the workstation machine as the student user.

Evaluation

As the student user on the workstation machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful. The grading scripts provide remediation hints to help you identify what content you need to revisit to complete the tasks.

```
student@workstation:~$ lab grade rhcsa-rh124-review2
```

Evaluate the allocated time for this activity against the time that you took to complete it. If you used more time than is allocated, then you must revisit the course content or practice more.

Finish

On the workstation machine, change to the student user home directory and use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish rhcsa-rh124-review2
```

20.4. Lab

Configure and Manage a Server

Configure, secure, and use the SSH service to access a remote machine.

Configure dnf and manage packages with the dnf utility.

Outcomes

- Create an SSH key pair.
- Disable SSH logins as the root user.
- Disable password-based SSH logins.
- Configure the software repositories on the servera machine to obtain updates.
- Install packages and package modules by using the dnf command.

Note

The allocated time for this activity is 10 minutes. If you need additional time to complete the task, then you must revisit the course content, or practice more.

Prerequisites

If you did not reset your workstation and serverX machines at the end of the last chapter, then save any work that you want to keep from earlier exercises on those machines, and reset them now.

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
student@workstation:~$ lab start rhcsa-rh124-review3
```

Specifications

As a system administrator, you are tasked to commission, manage, and support developers who use RHEL 10 servers. The system administration team recently provisioned a machine and shared a list of users and directories to log in and manage servers. To avoid sharing passwords, you must configure secure key-based authentication. You must also configure a custom repository and ensure that common system administration tools are installed.

Perform the following tasks on the serverb machine:

- Generate an SSH key pair for the student user. Do not protect the private key with a passphrase. Save the private key as the `/home/student/.ssh/review3_key` file, and save the public key as the `/home/student/.ssh/review3_key.pub` file.
- Configure SSH to prevent the root user from logging in.
- Configure SSH so that users log in by using key-based authentication and not by password-based authentication.
- Configure DNF to get packages from `http://repo.example.com/rhel10.0/x86_64/rhcsa-practice/errata`. Disable GPG checking for this repository.
- Install the `zsh` and `rht-system` packages.
- The student user on the serverb machine must be able to log in to the servera machine by using the `review3_key.pub` SSH key.

Evaluation

As the student user on the workstation machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful. The grading scripts provide remediation hints to help you identify what content you need to revisit to complete the tasks.

```
student@workstation:~$ lab grade rhcsa-rh124-review3
```

Evaluate the allocated time for this activity against the time that you took to complete it. If you used more time than is allocated, then you must revisit the course content or practice more.

Finish

On the workstation machine, change to the student user home directory and use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish rhcsa-rh124-review3
```

Solution

Configure, secure, and use the SSH service to access a remote machine.

Configure dnf and manage packages with the dnf utility.

Outcomes

- Create an SSH key pair.
- Disable SSH logins as the root user.
- Disable password-based SSH logins.
- Configure the software repositories on the servera machine to obtain updates.
- Install packages and package modules by using the dnf command.

Note

The allocated time for this activity is 10 minutes. If you need additional time to complete the task, then you must revisit the course content, or practice more.

Prerequisites

If you did not reset your workstation and serverX machines at the end of the last chapter, then save any work that you want to keep from earlier exercises on those machines, and reset them now.

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
student@workstation:~$ lab start rhcsa-rh124-review3
```

Specifications

As a system administrator, you are tasked to commission, manage, and support developers who use RHEL 10 servers. The system administration team recently provisioned a machine and shared a list of users and directories to log in and manage servers. To avoid sharing passwords, you must configure secure key-based authentication. You must also configure a custom repository and ensure that common system administration tools are installed.

Perform the following tasks on the serverb machine:

- Generate an SSH key pair for the student user. Do not protect the private key with a passphrase. Save the private key as the `/home/student/.ssh/review3_key` file, and save the public key as the `/home/student/.ssh/review3_key.pub` file.
- Configure SSH to prevent the root user from logging in.
- Configure SSH so that users log in by using key-based authentication and not by password-based authentication.
- Configure DNF to get packages from `http://repo.example.com/rhel10.0/x86_64/rhcsa-practice/errata`. Disable GPG checking for this repository.
- Install the `zsh` and `rht-system` packages.
- The student user on the serverb machine must be able to log in to the servera machine by using the `review3_key.pub` SSH key.

Solution

1. Log in to serverb as the student user.
2. Generate the `/home/student/.ssh/review3_key` SSH key pair for the student user. Do not protect the private key with a passphrase.

```
[student@serverb ~]$ ssh-keygen -f /home/student/.ssh/review3_key
...output omitted...
```

3. Export the `review3_key` public key from the serverb machine to the servera machine.

```
[student@serverb ~]$ ssh-copy-id -i .ssh/review3_key.pub student@servera
```

4. Edit the `/etc/ssh/sshd_config` file, and set the `PermitRootLogin` and `PasswordAuthentication` parameters to `no`. Reload the `sshd` service.
5. Configure the custom repository at `http://repo.example.com/rhel10.0/x86_64/rhcsa-practice/errata` and install the `zsh` and `rht-system` packages.

```
[student@serverb ~]$ sudo dnf config-manager --add-repo \  
"http://repo.example.com/rhel10.0/x86_64/rhcsa-practice/errata" \  
&& sudo dnf config-manager --save \  
--setopt=repo.example.com_rhel10.0_x86_64_rhcsa-practice_errata.gpgcheck=0 \  
&& sudo dnf install rht-system zsh -y
```

6. After completing the tasks of this activity, return to the workstation machine as the student user.

Evaluation

As the student user on the workstation machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful. The grading scripts provide remediation hints to help you identify what content you need to revisit to complete the tasks.

```
student@workstation:~$ lab grade rhcsa-rh124-review3
```

Evaluate the allocated time for this activity against the time that you took to complete it. If you used more time than is allocated, then you must revisit the course content or practice more.

Finish

On the workstation machine, change to the student user home directory and use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish rhcsa-rh124-review3
```

20.5. Lab

Manage Networks

Configure and test network connectivity.

Outcomes

- Configure network settings.
- Test network connectivity.
- Set a static hostname.
- Use locally resolvable canonical hostnames to connect to systems.

Note

The allocated time for this activity is 10 minutes. If you need additional time to complete the task, then you must revisit the course content, or practice more.

Prerequisites

If you did not reset your workstation and serverX machines at the end of the last chapter, then save any work that you want to keep from earlier exercises on those machines, and reset them now.

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
student@workstation:~$ lab start rhcsa-rh124-review4
```

Important

When you adjust network settings, running an incorrect command might lock out your session or cause it to hang, to make your system inaccessible. To avoid locking yourself out of the system, consider adjusting the network configuration through the remote console.

Specifications

As a system administrator, you are tasked to configure network settings and the hostname on a newly provisioned machine. Completing the network configuration of a server is a prerequisite to delegate the machine to developers and application users to set up and test their applications.

Perform the following tasks on the serverb machine:

- Find the unused network device and use it to create and activate a static connection profile. Configure the static profile to statically set network settings and not to use DHCP. Use the information from the following table for other connection settings:

Parameter	Setting
IPv4 address	172.25.250.111
Netmask	255.255.255.0
Gateway	172.25.250.254
DNS Server	172.25.250.254

- Set the hostname to `server-review4.lab4.example.com`.
- Set `client-review4` as the canonical hostname for the servera `172.25.250.10` IPv4 address.
- Configure the static connection profile with an additional IPv4 address of `172.25.250.211` with a netmask of `255.255.255.0`. Do not remove the existing IPv4 address. Ensure that the serverb machine responds to all addresses when the static connection is active.

Evaluation

As the student user on the workstation machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful. The grading scripts provide remediation hints to help you identify what content you need to revisit to complete the tasks.

```
student@workstation:~$ lab grade rhcsa-rh124-review4
```

Evaluate the allocated time for this activity against the time that you took to complete it. If you used more time than is allocated, then you must revisit the course content or practice more.

Finish

On the workstation machine, change to the student user home directory and use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish rhcsa-rh124-review4
```

Solution

Configure and test network connectivity.

Outcomes

- Configure network settings.
- Test network connectivity.
- Set a static hostname.
- Use locally resolvable canonical hostnames to connect to systems.

Note

The allocated time for this activity is 10 minutes. If you need additional time to complete the task, then you must revisit the course content, or practice more.

Prerequisites

If you did not reset your workstation and serverX machines at the end of the last chapter, then save any work that you want to keep from earlier exercises on those machines, and reset them now.

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
student@workstation:~$ lab start rhcsa-rh124-review4
```

Important

When you adjust network settings, running an incorrect command might lock out your session or cause it to hang, to make your system inaccessible. To avoid locking yourself out of the system, consider adjusting the network configuration through the remote console.

Specifications

As a system administrator, you are tasked to configure network settings and the hostname on a newly provisioned machine. Completing the network configuration of a server is a prerequisite to delegate the machine to developers and application users to set up and test their applications.

Perform the following tasks on the serverb machine:

- Find the unused network device and use it to create and activate a static connection profile. Configure the static profile to statically set network settings and not to use DHCP. Use the information from the following table for other connection settings:

Parameter	Setting
IPv4 address	172.25.250.111
Netmask	255.255.255.0
Gateway	172.25.250.254
DNS Server	172.25.250.254

- Set the hostname to `server-review4.lab4.example.com`.
- Set `client-review4` as the canonical hostname for the servera `172.25.250.10` IPv4 address.
- Configure the static connection profile with an additional IPv4 address of `172.25.250.211` with a netmask of `255.255.255.0`. Do not remove the existing IPv4 address. Ensure that the serverb machine responds to all addresses when the static connection is active.

Solution

- Use the `serverb` console to log in as the `student` user on the `serverb` machine, and switch to the `root` user.
- Use the `nmcli device status` command to determine the unused Ethernet device.



Important

The name of the network interface that is associated with the provided MAC address might vary according to the course platform and hardware in use. This activity assumes that the interface name is `ens4`.

3. Create a static connection profile, activate the new connection profile, and configure a new hostname.

```
[root@serverb ~]# nmcli connection add con-name static type ethernet \
  ifname ens4 ipv4.addresses '172.25.250.111/24' ipv4.gateway '172.25.250.254' \
  \
  ipv4.dns '172.25.250.254' ipv4.method manual \
  && nmcli connection up static \
  && hostnamectl hostname server-review4.lab4.example.com
```

4. Update the /etc/hosts file with the client-review4 server name for the 172.25.250.10 IPv4 address.
5. Modify the static connection profile, configure the additional 172.25.250.211/24 IPv4 address, and activate the new IP address.

```
[root@serverb ~]# nmcli connection modify static \
  +ipv4.addresses '172.25.250.211/24' \
  && nmcli connection up static
```

6. After completing the tasks of this activity, return to the workstation machine as the student user.

Evaluation

As the student user on the workstation machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful. The grading scripts provide remediation hints to help you identify what content you need to revisit to complete the tasks.

```
student@workstation:~$ lab grade rhcsa-rh124-review4
```

Evaluate the allocated time for this activity against the time that you took to complete it. If you used more time than is allocated, then you must revisit the course content or practice more.

Finish

On the workstation machine, change to the student user home directory and use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.


```
student@workstation:~$ lab finish rhcsa-rh124-review4
```

20.6. Lab

Mount File Systems and Find Files

Mount a file system and locate files based on different criteria.

Outcomes

- Mount an existing file system.
- Find files based on their file name, permissions, and size.

Note

The allocated time for this activity is 10 minutes. If you need additional time to complete the task, then you must revisit the course content, or practice more.

Prerequisites

If you did not reset your workstation and serverX machines at the end of the last chapter, then save any work that you want to keep from earlier exercises on those machines, and reset them now.

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
student@workstation:~$ lab start rhcsa-rh124-review5
```

Specifications

As a system administrator, you are tasked to configure the necessary storage that the applications team requested. Before allowing the applications team to access the server, you must mount the data partition. Additionally, you must use command-line tools to gather file information to confirm that the machine is configured and ready for delegation.

Perform the following tasks on the serverb machine:

- Identify the unused, unmounted block device that contains an XFS file system on the serverb machine. Mount the block device on the /review5-disk directory.
- Find the review5-path file. Create the /review5-disk/review5-path.txt file that contains a single line with the absolute path to the review5-path file.

- Find all the files with the 640 octal permissions, and that are owned by the contractor1 user and the contractor group. Save the list of these files in the /review5-disk/review5-perms.txt file.
- Find all files with a size of 100 bytes. Save the absolute paths of these files in the /review5-disk/review5-size.txt file.

Evaluation

As the student user on the workstation machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful. The grading scripts provide remediation hints to help you identify what content you need to revisit to complete the tasks.

```
student@workstation:~$ lab grade rhcsa-rh124-review5
```

Evaluate the allocated time for this activity against the time that you took to complete it. If you used more time than is allocated, then you must revisit the course content or practice more.

Finish

On the workstation machine, change to the student user home directory and use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish rhcsa-rh124-review5
```

Solution

Mount a file system and locate files based on different criteria.

Outcomes

- Mount an existing file system.
- Find files based on their file name, permissions, and size.

Note

The allocated time for this activity is 10 minutes. If you need additional time to complete the task, then you must revisit the course content, or practice more.

Prerequisites

If you did not reset your workstation and serverX machines at the end of the last chapter, then save any work that you want to keep from earlier exercises on those machines, and reset them now.

As the student user on the workstation machine, run the following lab command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
student@workstation:~$ lab start rhcsa-rh124-review5
```

Specifications

As a system administrator, you are tasked to configure the necessary storage that the applications team requested. Before allowing the applications team to access the server, you must mount the data partition. Additionally, you must use command-line tools to gather file information to confirm that the machine is configured and ready for delegation.

Perform the following tasks on the serverb machine:

- Identify the unused, unmounted block device that contains an XFS file system on the serverb machine. Mount the block device on the /review5-disk directory.
- Find the review5-path file. Create the /review5-disk/review5-path.txt file that contains a single line with the absolute path to the review5-path file.
- Find all the files with the 640 octal permissions, and that are owned by the contractor1 user and the contractor group. Save the list of these files in the /review5-disk/review5-

perms.txt file.

- Find all files with a size of 100 bytes. Save the absolute paths of these files in the /review5-disk/review5-size.txt file.

Solution

- Log in to the serverb machine as the student user, and switch to the root user.
- Use the `lsblk -fs` command to identify the unmounted block device that contains an XFS file system.
- Create the /review5-disk directory and mount the sdb1 block device on it.

```
[root@serverb ~]# mkdir /review5-disk && \
mount /dev/sdb1 /review5-disk
```

- Create the /review5-disk/review5-path.txt, /review5-disk/review5-perms.txt, and /review5-disk/review5-size.txt files with the specified content.

```
[root@serverb ~]# find / -iname review5-path 2>/dev/null \
> /review5-disk/review5-path.txt ; \
find / -user contractor1 \
-group contractor -perm 640 2>/dev/null \
> /review5-disk/review5-perms.txt ; \
find / -type f -size 100c 2>/dev/null \
> /review5-disk/review5-size.txt
```

- After completing the tasks of this activity, return to the workstation machine as the student user.

Evaluation

As the student user on the workstation machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful. The grading scripts provide remediation hints to help you identify what content you need to revisit to complete the tasks.

```
student@workstation:~$ lab grade rhcsa-rh124-review5
```

Evaluate the allocated time for this activity against the time that you took to complete it. If you used more time than is allocated, then you must revisit the course content or practice more.

Finish

On the workstation machine, change to the student user home directory and use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
student@workstation:~$ lab finish rhcsa-rh124-review5
```

20.7. Summary

In this comprehensive review, you practiced what you learned in the *Red Hat System Administration I (RH124)* course.

Next Steps

After the successful completion of all the exercise tasks in the allocated time, you might want to consider taking the *Red Hat System Administration II (RH134)* course.

References

- [Red Hat System Administration II \(RH134\)](#)